

pixrep onepdf
repo: namayandeelmi-javad
generated: 2026-08-12 22:51:01
files: 166

== tree ==

```
namayandeelmi-javad/  
|-- README.md  
|-- mobile/  
|   |-- App.tsx  
|   |-- src/  
|       |-- simAuth.ts  
|-- src/  
|   |-- db/  
|   |   |-- index.ts  
|   |   |-- schema.ts  
|   |-- instrumentation.ts  
|   |-- proxy.ts  
|   |-- app/  
|   |   |-- layout.tsx  
|   |   |-- page.tsx  
|   |   |-- admin/  
|   |       |-- layout.tsx  
|   |       |-- page.tsx  
|   |       |-- activity/  
|   |           |-- page.tsx  
|   |       |-- backup/  
|   |           |-- page.tsx  
|   |       |-- columns/  
|   |           |-- page.tsx  
|   |       |-- homes/  
|   |           |-- page.tsx  
|   |       |-- leaves/  
|   |           |-- page.tsx  
|   |       |-- live/  
|   |           |-- page.tsx  
|   |       |-- map/  
|   |           |-- page.tsx  
|   |       |-- messengers/  
|   |           |-- page.tsx  
|   |       |-- notifications/  
|   |           |-- page.tsx  
|   |       |-- options/  
|   |           |-- page.tsx  
|   |       |-- reports/  
|   |           |-- page.tsx  
|   |       |-- sync/  
|   |           |-- page.tsx  
|   |       |-- targets/  
|   |           |-- page.tsx  
|   |       |-- trips/  
|   |           |-- page.tsx  
|   |       |-- users/  
|   |           |-- page.tsx  
|   |       |-- records/  
|   |           |-- [type]/  
|   |               |-- page.tsx  
|   |-- apple-touch-icon.png/  
|   |-- route.ts  
|   |-- diagnostics/  
|   |   |-- page.tsx  
|   |-- install/  
|   |   |-- page.tsx  
|   |-- login/  
|   |   |-- LoginForm.tsx  
|   |   |-- page.tsx  
|   |-- offline/  
|   |   |-- page.tsx  
|   |-- panel/  
|   |   |-- layout.tsx  
|   |   |-- page.tsx  
|   |   |-- doctors/  
|   |       |-- page.tsx  
|   |   |-- home/  
|   |       |-- page.tsx  
|   |   |-- leaves/  
|   |       |-- page.tsx  
|   |   |-- map/  
|   |       |-- page.tsx  
|   |   |-- notifications/  
|   |       |-- page.tsx  
|   |   |-- options/  
|   |       |-- page.tsx  
|   |   |-- orders/  
|   |       |-- page.tsx
```

```

|-- pharmacies/
|   |-- page.tsx
|-- reports/
|   |-- page.tsx
|-- trip/
|   |-- page.tsx
|-- ping/
|   |-- route.ts
|-- api/
|   |-- activity/
|   |   |-- route.ts
|   |-- attachments/
|   |   |-- route.ts
|   |   |-- [id]/
|   |   |   |-- route.ts
|   |-- backup/
|   |   |-- route.ts
|   |   |-- restore/
|   |   |   |-- route.ts
|   |-- export/
|   |   |-- route.ts
|   |-- geocode/
|   |   |-- route.ts
|   |-- health/
|   |   |-- route.ts
|   |-- homes/
|   |   |-- route.ts
|   |-- leaves/
|   |   |-- route.ts
|   |-- live/
|   |   |-- route.ts
|   |-- messengers/
|   |   |-- route.ts
|   |   |-- updates/
|   |   |   |-- route.ts
|   |-- notifications/
|   |   |-- route.ts
|   |-- options/
|   |   |-- route.ts
|   |-- ping/
|   |   |-- route.ts
|   |-- reports/
|   |   |-- route.ts
|   |-- roles/
|   |   |-- route.ts
|   |-- settings/
|   |   |-- route.ts
|   |-- source/
|   |   |-- route.ts
|   |-- targets/
|   |   |-- route.ts
|   |-- trips/
|   |   |-- route.ts
|   |   |-- points/
|   |   |   |-- route.ts
|   |-- users/
|   |   |-- route.ts
|   |-- auth/
|   |   |-- check-username/
|   |   |   |-- route.ts
|   |   |-- forgot/
|   |   |   |-- route.ts
|   |   |-- login/
|   |   |   |-- route.ts
|   |   |-- logout/
|   |   |   |-- route.ts
|   |   |-- me/
|   |   |   |-- route.ts
|   |   |-- otp/
|   |   |   |-- route.ts
|   |-- mobile/
|   |   |-- login-with-phone/
|   |   |   |-- route.ts
|   |   |-- nonce/
|   |   |   |-- route.ts
|   |-- records/
|   |   |-- [type]/
|   |   |   |-- route.ts
|   |   |-- send/
|   |   |   |-- route.ts
|   |   |-- lookup/
|   |   |   |-- route.ts
|-- sync/
|   |-- pull/
|   |   |-- route.ts

```

```

|-- push/
|--   |-- route.ts
|-- run/
|--   |-- route.ts
|-- status/
|--   |-- route.ts
|-- tiles/
|--   |-- [z]/
|--     |-- [x]/
|--       |-- [y]/
|--         |-- route.ts
|-- map/
|--   |-- tiles/
|--     |-- [z]/
|--       |-- [x]/
|--         |-- [y]/
|--           |-- route.ts
|-- icons/
|--   |-- [file]/
|--     |-- route.ts
|-- screenshots/
|--   |-- [file]/
|--     |-- route.ts
|-- globals.css
|-- components/
|--   |-- BackButton.tsx
|--   |-- Charts.tsx
|--   |-- Combobox.tsx
|--   |-- Confirm.tsx
|--   |-- ConnectionStatus.tsx
|--   |-- FileUploader.tsx
|--   |-- JalaliDateInput.tsx
|--   |-- LocationPicker.tsx
|--   |-- MapBox.tsx
|--   |-- NavButton.tsx
|--   |-- NotificationBell.tsx
|--   |-- RecordScreen.tsx
|--   |-- ServiceWorker.tsx
|--   |-- SessionProvider.tsx
|--   |-- ShareBox.tsx
|--   |-- Shell.tsx
|--   |-- ui.tsx
|--   |-- screens/
|--     |-- ChartsPanel.tsx
|--     |-- HomeScreen.tsx
|--     |-- LeaveScreen.tsx
|--     |-- MapExplorer.tsx
|--     |-- NotificationScreen.tsx
|--     |-- OptionsScreen.tsx
|--     |-- ReportScreen.tsx
|--     |-- TargetPanel.tsx
|-- lib/
|--   |-- auth.ts
|--   |-- bootstrap.ts
|--   |-- constants.ts
|--   |-- defaults.ts
|--   |-- device.ts
|--   |-- download.ts
|--   |-- endpoints.ts
|--   |-- excel.ts
|--   |-- geo.ts
|--   |-- iran.ts
|--   |-- jalali.ts
|--   |-- messaging.ts
|--   |-- mobile-auth.ts
|--   |-- notify.ts
|--   |-- offline-session.ts
|--   |-- otp.ts
|--   |-- pwa-image.ts
|--   |-- retry.ts
|--   |-- settings-server.ts
|--   |-- sort.ts
|--   |-- sound.ts
|--   |-- sync-config.ts
|--   |-- sync.ts
|--   |-- tracker.ts
|--   |-- useLive.ts
|--   |-- zip.ts
|-- package.json
|-- public/
|--   |-- index.html
|--   |-- cloudflare-worker.js
|--   |-- crm-app.js
|--   |-- crm-data.js
|--   |-- sw-template.js

```

```

| |-- sw.js
| |-- manifest.webmanifest
| |-- style.css
| |-- data/
| |   |-- iran-provinces-lite.geojson
| |   |-- world-countries.geojson
|-- next-env.d.ts
|-- next.config.ts
|-- server.js
|-- .env.ndcohub.example
|-- .gitlab-ci.yml
|-- drizzle.config.json
|-- eslint.config.mjs
|-- postcss.config.mjs
|-- render.yaml
|-- tsconfig.json
|-- scripts/
|   |-- build-sw.mjs
|   |-- generate-assets.mjs
|   |-- simplify-geojson.mjs
|   |-- start.mjs

```

== index ==

```

0001 README.md
0002 mobile/App.tsx
0003 src/db/index.ts
0004 package.json
0005 public/index.html
0006 next-env.d.ts
0007 next.config.ts
0008 server.js
0009 public/cloudflare-worker.js
0010 public/crm-app.js
0011 public/crm-data.js
0012 public/sw-template.js
0013 public/sw.js
0014 src/instrumentation.ts
0015 src/proxy.ts
0016 mobile/src/simAuth.ts
0017 src/app/layout.tsx
0018 src/app/page.tsx
0019 src/components/BackButton.tsx
0020 src/components/Charts.tsx
0021 src/components/Combobox.tsx
0022 src/components/Confirm.tsx
0023 src/components/ConnectionStatus.tsx
0024 src/components/FileUploader.tsx
0025 src/components/JalaliDateInput.tsx
0026 src/components/LocationPicker.tsx
0027 src/components/MapBox.tsx
0028 src/components/NavButton.tsx
0029 src/components/NotificationBell.tsx
0030 src/components/RecordScreen.tsx
0031 src/components/ServiceWorker.tsx
0032 src/components/SessionProvider.tsx
0033 src/components/ShareBox.tsx
0034 src/components/Shell.tsx
0035 src/components/ui.tsx
0036 src/db/schema.ts
0037 src/lib/auth.ts
0038 src/lib/bootstrap.ts
0039 src/lib/constants.ts
0040 src/lib/default.ts
0041 src/lib/device.ts
0042 src/lib/download.ts
0043 src/lib/endpoints.ts
0044 src/lib/excel.ts
0045 src/lib/geo.ts
0046 src/lib/iran.ts
0047 src/lib/jalali.ts
0048 src/lib/messaging.ts
0049 src/lib/mobile-auth.ts
0050 src/lib/notify.ts
0051 src/lib/offline-session.ts
0052 src/lib/otp.ts
0053 src/lib/pwa-image.ts
0054 src/lib/retry.ts
0055 src/lib/settings-server.ts
0056 src/lib/sort.ts
0057 src/lib/sound.ts
0058 src/lib/sync-config.ts
0059 src/lib/sync.ts
0060 src/lib/tracker.ts
0061 src/lib/useLive.ts
0062 src/lib/zip.ts

```

0063 src/app/admin/layout.tsx
0064 src/app/admin/page.tsx
0065 src/app/apple-touch-icon.png/route.ts
0066 src/app/diagnostics/page.tsx
0067 src/app/install/page.tsx
0068 src/app/login/LoginForm.tsx
0069 src/app/login/page.tsx
0070 src/app/offline/page.tsx
0071 src/app/panel/layout.tsx
0072 src/app/panel/page.tsx
0073 src/app/ping/route.ts
0074 src/components/screens/ChartsPanel.tsx
0075 src/components/screens/HomeScreen.tsx
0076 src/components/screens/LeaveScreen.tsx
0077 src/components/screens/MapExplorer.tsx
0078 src/components/screens/NotificationScreen.tsx
0079 src/components/screens/OptionsScreen.tsx
0080 src/components/screens/ReportScreen.tsx
0081 src/components/screens/TargetPanel.tsx
0082 src/app/admin/activity/page.tsx
0083 src/app/admin/backup/page.tsx
0084 src/app/admin/columns/page.tsx
0085 src/app/admin/homes/page.tsx
0086 src/app/admin/leaves/page.tsx
0087 src/app/admin/live/page.tsx
0088 src/app/admin/map/page.tsx
0089 src/app/admin/messengers/page.tsx
0090 src/app/admin/notifications/page.tsx
0091 src/app/admin/options/page.tsx
0092 src/app/admin/reports/page.tsx
0093 src/app/admin/sync/page.tsx
0094 src/app/admin/targets/page.tsx
0095 src/app/admin/trips/page.tsx
0096 src/app/admin/users/page.tsx
0097 src/app/api/activity/route.ts
0098 src/app/api/attachments/route.ts
0099 src/app/api/backup/route.ts
0100 src/app/api/export/route.ts
0101 src/app/api/geocode/route.ts
0102 src/app/api/health/route.ts
0103 src/app/api/homes/route.ts
0104 src/app/api/leaves/route.ts
0105 src/app/api/live/route.ts
0106 src/app/api/messengers/route.ts
0107 src/app/api/notifications/route.ts
0108 src/app/api/options/route.ts
0109 src/app/api/ping/route.ts
0110 src/app/api/reports/route.ts
0111 src/app/api/roles/route.ts
0112 src/app/api/settings/route.ts
0113 src/app/api/source/route.ts
0114 src/app/api/targets/route.ts
0115 src/app/api/trips/route.ts
0116 src/app/api/users/route.ts
0117 src/app/icons/[file]/route.ts
0118 src/app/panel/doctors/page.tsx
0119 src/app/panel/home/page.tsx
0120 src/app/panel/leaves/page.tsx
0121 src/app/panel/map/page.tsx
0122 src/app/panel/notifications/page.tsx
0123 src/app/panel/options/page.tsx
0124 src/app/panel/orders/page.tsx
0125 src/app/panel/pharmacies/page.tsx
0126 src/app/panel/reports/page.tsx
0127 src/app/panel/trip/page.tsx
0128 src/app/screenshots/[file]/route.ts
0129 src/app/admin/records/[type]/page.tsx
0130 src/app/api/attachments/[id]/route.ts
0131 src/app/api/auth/check-username/route.ts
0132 src/app/api/auth/forgot/route.ts
0133 src/app/api/auth/login/route.ts
0134 src/app/api/auth/logout/route.ts
0135 src/app/api/auth/me/route.ts
0136 src/app/api/auth/otp/route.ts
0137 src/app/api/backup/restore/route.ts
0138 src/app/api/messengers/updates/route.ts
0139 src/app/api/mobile/login-with-phone/route.ts
0140 src/app/api/mobile/nonce/route.ts
0141 src/app/api/records/[type]/route.ts
0142 src/app/api/records/lookup/route.ts
0143 src/app/api/sync/pull/route.ts
0144 src/app/api/sync/push/route.ts
0145 src/app/api/sync/run/route.ts
0146 src/app/api/sync/status/route.ts
0147 src/app/api/trips/points/route.ts

```

0148 src/app/api/records/[type]/send/route.ts
0149 src/app/api/tiles/[z]/[x]/[y]/route.ts
0150 src/app/api/map/tiles/[z]/[x]/[y]/route.ts
0151 .env.ndcohub.example
0152 .gitlab-ci.yml
0153 drizzle.config.json
0154 eslint.config.mjs
0155 postcss.config.mjs
0156 render.yaml
0157 tsconfig.json
0158 public/manifest.webmanifest
0159 public/style.css
0160 scripts/build-sw.mjs
0161 scripts/generate-assets.mjs
0162 scripts/simplify-geojson.mjs
0163 scripts/start.mjs
0164 public/data/iran-provinces-lite.geojson
0165 public/data/world-countries.geojson
0166 src/app/globals.css

```

== files ==

@@ README.md markdown
#

Next.js 16 (App Router) + PostgreSQL (Drizzle ORM) + PWA (iOS).

```

##
| | | | |
| --- | --- | --- | --- |
| | `admin` | `admin1234` | `09120000000` |
| | `rep1` | `rep1234` | `09121111111` |
> `ADMIN_PASSWORD` `ADMIN_PHONE`
> ( ). « » .
## (VS Code)
```bash
npm install
.env
DATABASE_URL=postgresql://user:pass@host/db?sslmode=require
npm run dev
```

```

** ** (migration).

Render + Neon

«Could not find a production build»

Render :

```

==> Running 'npm start'

```

** `npm run build` ** → **Build Command** .

`next start` .next` .

()

Render → → **Settings** → **Build & Deploy**:

```

Root Directory	( )
Build Command	`npm install --include=dev && npm run build`
Start Command	`npm run start`
Health Check Path	`/api/health`
**Environment** → :	
Key	Value
---	---
`DATABASE_URL`	Neon `postgresql://user:pass@ep-xxx.neon.tech/neondb?sslmode=require`
`APP_SECRET`	
`NODE_VERSION`	`20.18.0`
`NPM_CONFIG_PRODUCTION`	`false`

```

Manual Deploy → Clear build cache & deploy.

> : `--include=dev` Render `NODE\_ENV=production`

> `npm install` toolchain (typescript tailwind) .

(Build Command)

`npm start` `scripts/start.mjs` : `.next`

** `next build` ** `PORT` Render .

live ().

Blueprint

`render.yaml` Render

New → Blueprint `DATABASE\_URL` .

###

`https://<your-app>.onrender.com/api/health` :

- `{ "ok": true, "db": true }` → .

- `{ "ok": false, "reason": "DATABASE\_URL " }` → .

- `{ "ok": false, "reason": "..." }` → Neon (`?sslmode=require`) .

##

- / + .

- .

- :

- + + (`//`).

```

-      /      .
-      ( )      / .
-      «» «» «» .
-      ( + )      .
-      ( / ).
-      (CSV UTF-8 Excel) .
-      .
-      .
##      APK PWABuilder
:
- `prebuild` build `96..1024`
  `maskable` ( ) `apple-touch-icon`
    SVG . PNG
    Render .
-      ( `/icons/[file]`, `/screenshots/[file]`,
  `/apple-touch-icon.png`)
-      .
-      `<head>`
  PWA .
####      APK
1.      Render (Build Command: `npm install --include=dev && npm run build`).
2.      [PWABuilder.com](https://www.pwabuilder.com) ** ** ( `/login` ).
3.      Retest Action Items .
4. `Package For Stores → Android` `Download Test Package` .
##      ( )
** : ** `force-dynamic`
. ( Render )
.
** : **
1. ** ** ( Prerendered ).
2. **** — (v8) .
3. **Cache-First ** + → .
4. ** ** ( ) .
5. ** ** IndexedDB + .
6. ** ** ( ) .
7. ** `/api/ping` ** ( ) + .
##      ( )
** **
** (OTP) ** :
1. / .
2. ** ** .
3. → ** **.
4. → .
5. .
6. «» **** ().
####
«» → « »: SMS.ir .
: `SMS_PROVIDER`, `SMS_API_KEY`, `SMS_SENDER`, `SMS_PATTERN`.
>
>
##
#### « »
HTML ** ( . ) **.
React HTML JS
→ .
** : ** `scripts/build-sw.mjs` build
`/_next/static` . **
+ ** .
####
`/diagnostics` — :
GPS .
« » « » .
## —
** ** «» :
` `	
` `	
` ` ( )	
` `	:
« » .
##
#### ) Keep-Alive —
:
...
GET /ping → 200 "OK" ( ~ms)
GET /ping?json → {"status":"OK","uptimeMs":...}
GET /api/ping →
HEAD /ping → 200
...
** UptimeRobot: ** New Monitor → HTTP(s) → URL: `https://<app>.onrender.com/ping`
→ Interval: ** ** . Render .
#### ) Retry Exponential Backoff + Jitter
`src/lib/retry.ts`:

```

```

API	
`withRetry(fn, opts)`	
`withRetrySafe(fn, fallback)`	
`fetchWithRetry(url, init)`	HTTP ( )
`dbRetry(fn, label)`	
`dbTransaction(fn)`	BEGIN/COMMIT/ROLLBACK retry
** (Full Jitter):** `s → s → s → s → s` ***

** : ** `ECONNRESET`, `ETIMEDOUT`, `ECONNREFUSED`, `EAI_AGAIN`, `EPIPE` ...
PostgreSQL `08xxx` ( ) `53xxx` ( ) `57Pxx` ( ) `40001/40P01` ( )
HTTP `5xx`, `429`, `408` «endpoint is disabled» Neon.
** ( ) : ** `4xx` `23505` `AbortError`.
### ) Neon
`src/db/index.ts` :
...

: postgresql://u:p@ep-cool-fire-123.eu-central-1.aws.neon.tech/neondb
: postgresql://u:p@ep-cool-fire-123-pooler.eu-central-1.aws.neon.tech/neondb
?sslmode=require&connect_timeout=15&application_name=sabt-etelaat-kol
...

- endpoint **`-pooler`** (PgBouncer)
- **`connect_timeout=15`**
- `statement_timeout` `query_timeout`
- `keepAlive` `max=8`, `min=0`, `idleTimeout=30s`
- `pool.on("error")`
`GET /api/health` :
`{"ok":true,"db":true,"latencyMs":8,"pooler":true,"connectTimeoutSec":15}`
## «This page couldn't load»
###
**_**

.

### ) HTML
`next.config.ts` :
`Cache-Control: no-cache, no-store, must-revalidate, max-age=0` + `Pragma: no-cache` + `Expires: 0`
`/_next/static/*` `immutable` ( ).
### )
`ConnectionStatus`:
**« — »**
« » « » .
** ** ( `navigator.onLine` )

### ): «-»
```js
if (req.mode === "navigate") {
 cache: "no-store"
 → ()
 → (/panel /)
 →
}
...

RSC (Next)
) +
- ().
- / **IndexedDB**
« » .
- «N » **« »** .
- Background Sync .
- `/offline` :
« » .
##
> ** Laravel:** `bot-generator-bale-telegram-laravel-10` PHP/MySQL
> « » . PHP
> . (Bot API / +)
> TypeScript .
— `messengers`
| | |
|---|---|
| `platform` | telegram / bale / eita / whatsapp |
| `label` | |
| `targetType` | group / channel / phone |
| `target` | chat_id |
| `token` | API |
| `provider` | : whatsapp / ultramsg / cloudapi / custom |
| `apiUrl` | `{phone}` `{text}` `{token}` |
| `enabled` | / |
| `lastStatus`, `lastOkAt`, `lastErrorAt` | |
| `message_logs` (/ +) .
(`src/lib/messaging.ts`)
```ts
formatOrderMessage(order) //
sendOne(target, text) // ( → )
dispatchText(text, orderId?) //
dispatchOrder(order) //
fetchUpdates(platform, token) // chat_id

```



```

` ``
###
- *** → → (Full Jitter) .
- `Promise.allSettled` ** **
- ** Cloudflare** .
- ( whatsiplus) **** .
###
| | | |
|---|---|---|
| *** | @BotFather → `newbot` | chat_id `100...` |
| *** | @BotFather → `newbot` | chat_id |
| *** | `eitaayar.ir` | `@` |
| *** | API `whatsiplus.ir` ( UltraMsg / Cloud API) | |
| **« chat_id»** :
.
##
> ** : ** `getLine1Number()` ` MSISDN
> → `null`. iOS API .
> .

### API
| | |
|---|---|
| `POST /api/mobile/nonce` | nonce ( Replay) |
| `POST /api/mobile/login-with-phone` | + HMAC |
| `POST /api/auth/otp` | |
###
1. **HMAC-SHA256** `nonce|timestamp|deviceId|phone|simFingerprint` `MOBILE_APP_SECRET`
2. **nonce** —
3. ** **
4. ICCID/IMSI ** SHA-256**
### ( )
` ``
200 TOKEN ←
401 BAD_SIGNATURE ←
401 BAD_NONCE ← Replay Attack
404 PHONE_NOT_REGISTERED ←
422 SIM_NUMBER_UNAVAILABLE ← →
403 DEVICE_MISMATCH ←
403 SIM_CHANGED ←
403 PASSWORD_LOGIN_REQUIRED ←
` ``
###
`mobile/` `App.tsx` `src/simAuth.ts` (React Native) Flutter
. `MOBILE_APP_SECRET`
Render .
##
### )
`targets` `userId`, `period` ( ), `productKey`,
`quantity`, `priceDistributor`, `pricePharmacy`.
- **→ ** : /
« » .
- `GET /api/targets` : (
)
- ** **
.
- « » .
### )
`GET /api/records/lookup?type=pharmacies|doctors&name=...&phone=...`
( «/» / )
« » «» .
### )
:
- « »
- **« »**
.
## : Render NdcoHub.ir
**** :
| | | |
|---|---|---|
| NdcoHub () | `https://ndcohub.ir` | — |
| Render | `https://namayandeelmi-javad.onrender.com` | |
### )
`src/lib/endpoints.ts` —
** **
.
### )
:
- `uid` — (UUID)
- `updated_at` —
- `origin` —
** : ** ( ) `pull`
`push` . : **Last-Write-Wins** `updated_at`.

```

```

** : **      `sek.sync=on`
      « »
      . `updated_at < excluded.updated_at`

### )
| | |
|---|---|
| `POST /api/sync/pull` | ( HMAC ) |
| `POST /api/sync/push` | |
| `POST /api/sync/run` | / (`x-sync-key`) |
| `GET /api/sync/status` | |
** → ** : /
« » « » « » .
### )
** Render : **
...

NODE_NAME=render
SYNC_SECRET=< >
SYNC_PEERS=ndcohub|https://ndcohub.ir
SYNC_INTERVAL_MINUTES=5
NEXT_PUBLIC_ENDPOINTS=https://ndcohub.ir,https://namayandeeelmi-javad.onrender.com
PUBLIC_BASE_URL=https://namayandeeelmi-javad.onrender.com
...

** NdcHub ** ( `.env.ndcohub.example` ):
...

NODE_NAME=ndcohub
SYNC_SECRET=< >
SYNC_PEERS=render|https://namayandeeelmi-javad.onrender.com
NEXT_PUBLIC_ENDPOINTS=https://ndcohub.ir,https://namayandeeelmi-javad.onrender.com
PUBLIC_BASE_URL=https://ndcohub.ir

### ) CI/CD
- `.github/workflows/deploy.yml` — ** GitLab **
- `.gitlab-ci.yml` — ** GitHub **
  : `GITLAB_REPO_URL`, `GITLAB_TOKEN`, `RENDER_DEPLOY_HOOK`,
  `NDCOHUB_DEPLOY_HOOK`, `SYNC_SECRET`.
##      GitHub / GitLab
`git status` «nothing to commit»
      ( ).
### :
1. **** .
2. ** → **.
3. **« (ZIP)»** .
4. ZIP ( ).
5. :
  ``bash
  git add -A
  git status #
  git commit -m " "
  git push origin main
  git push gitlab main # GitLab
  ...

> `node_modules` `.next` `.env` ( ).
> `HOW-TO-UPDATE.md` ZIP .
API: `GET /api/source` () — `GET /api/source?list=1` .
##
### ) /
.
( «» )
.
** : **      ****
      (
      ).
### )
  `/api/geocode` ( CORS ):
- + « » + Enter
- ( / + )
- : `35.7219, 51.4089`
- « » ()
- (Photon Nominatim)

> : `Number(null)` ****
> « »
> .
## +
###
:
| | |
|---|---|
| `src/lib/geo.ts` | + `resolveArea`, `provinceOf`, `distanceKm` |
| `src/components/screens/MapExplorer.tsx` | /// |
| `src/app/panel/map` `src/app/admin/map` | « » |
  `fetchJson` `fetchRows` `src/lib/useLive.ts` `MapBox`
  `MapArea` ( ).
###

```

```

**) `Custom Cache-Control headers detected for /_next/static`**
  `/_next/static` ( Next.js `immutable` )
  `no-store` `/:path(?!_next/.*)`
**) `Encountered unexpected file in NFT list`**
  `/api/source` `fs` `path` import
  Turbopack
  **
: `npm run build`

```

```

@@ mobile/App.tsx typescript

```

```

/**
 * « »
 * React Native —
 */
import React, { useEffect, useState } from "react";
import {
  ActivityIndicator,
  Alert,
  I18nManager,
  SafeAreaView,
  ScrollView,
  StyleSheet,
  Text,
  TextInput,
  TouchableOpacity,
  View,
} from "react-native";
import { WebView } from "react-native-webview";
import {
  API_BASE,
  getSavedToken,
  loginWithSim,
  readSimInfo,
  requestOtp,
  verifyOtp,
  type SimInfo,
} from "../src/simAuth";
I18nManager.allowRTL(true);
I18nManager.forceRTL(true);
type Screen = "loading" | "sim" | "credentials" | "otp" | "app";
export default function App() {
  const [screen, setScreen] = useState<Screen>("loading");
  const [sim, setSim] = useState<SimInfo | null>(null);
  const [msg, setMsg] = useState("");
  const [busy, setBusy] = useState(false);
  const [token, setToken] = useState<string | null>(null);
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");
  const [code, setCode] = useState("");
  const [masked, setMasked] = useState("");
  /* ----- : ----- */
  useEffect(() => {
    (async () => {
      const saved = await getSavedToken();
      if (saved) {
        setToken(saved);
        setScreen("app");
        return;
      }
      const info = await readSimInfo();
      setSim(info);
      if (!info.hasSim) {
        setMsg(" ");
        setScreen("credentials");
        return;
      }
      if (!info.phoneNumber) {
        // →
        setMsg(
          `${info.carrier || ""} .\n` +
            " ",
        );
        setScreen("credentials");
        return;
      }
      setBusy(true);
      const res = await loginWithSim();
      setBusy(false);
      if (res.status === "success") {
        setToken(res.token);
        setScreen("app");
      } else if (res.status === "need-otp") {
        setMsg(res.message);
        setMasked(res.masked ?? "");
      }
    })();
  });
}

```

```

        setScreen("credentials");
    } else {
        setMsg(res.message);
        setScreen("credentials");
    }
})();
}, []);
const doRequestOtp = async () => {
    if (!username || !password) return Alert.alert("", " ");
    setBusy(true);
    const r = await requestOtp(username, password);
    setBusy(false);
    setMsg(r.message);
    if (r.masked) setMasked(r.masked);
    if (r.ok) setScreen("otp");
};
const doVerifyOtp = async () => {
    setBusy(true);
    const r = await verifyOtp(username, password, code);
    setBusy(false);
    if (r.ok && r.token) {
        setToken(r.token);
        setScreen("app");
    } else setMsg(r.message ?? " ");
};
/* ----- WebView ----- */
if (screen === "app" && token) {
    return (
        <SafeAreaView style={styles.flex}>
            <WebView
                source={{ uri: `${API_BASE}/panel` }}
                injectedJavaScriptBeforeContentLoaded={`
                    try {
                        sessionStorage.setItem('sek_token', ${JSON.stringify(token)});
                        localStorage.setItem('sek_token_p', ${JSON.stringify(token)});
                    } catch (e) {}
                `}
                startInLoadingState
                geolocationEnabled
                domStorageEnabled
                javaScriptEnabled
                allowsBackForwardNavigationGestures
            />
        </SafeAreaView>
    );
}
if (screen === "loading" || busy) {
    return (
        <SafeAreaView style={[styles.flex, styles.center]}>
            <ActivityIndicator size="large" color="#0f766e" />

            <Text style={styles.hint}> ...</Text>
        </SafeAreaView>
    );
}
return (
    <SafeAreaView style={styles.flex}>
        <ScrollView contentContainerStyle={styles.container}>
            <Text style={styles.title}> </Text>
            <Text style={styles.sub}> </Text>
            {sim ? (
                <View style={styles.simBox}>
                    <Text style={styles.simText}>
                        {sim.hasSim ? "" : ""} : {sim.carrier || ""}
                    </Text>
                    {sim.phoneNumber ? `— ${sim.phoneNumber}` : "( )"}
                    </Text>
                </View>
            ) : null}
            {msg ? <Text style={styles.msg}>{msg}</Text> : null}
            {screen === "credentials" ? (
                <>
                    <TextInput
                        style={styles.input}
                        placeholder=" "
                        value={username}
                        onChangeText={setUsername}
                        autoCapitalize="none"
                    />
                    <TextInput
                        style={styles.input}
                        placeholder=" "
                        value={password}
                        onChangeText={setPassword}
                        secureTextEntry

```

```

    />
    <TouchableOpacity style={styles.btn} onPress={doRequestOtp}>
      <Text style={styles.btnText}> </Text>
    </TouchableOpacity>
    <TouchableOpacity
      style={[styles.btn, styles.btnGhost]}
      onPress={async () => {
        setBusy(true);
        const r = await loginWithSim();
        setBusy(false);
        if (r.status === "success") {
          setToken(r.token);
          setScreen("app");
        } else setMsg(r.message);
      }}
    >
      <Text style={styles.btnGhostText}> </Text>
    </TouchableOpacity>
  </>
) : null}
{screen === "otp" ? (
  <>
    <Text style={styles.hint}> {masked} </Text>
    <TextInput
      style={[styles.input, styles.code]}
      placeholder=" _ _ _ _ _"
      value={code}
      onChangeText={setCode}
      keyboardType="number-pad"
      maxLength={6}
    />
    <TouchableOpacity style={styles.btn} onPress={doVerifyOtp}>
      <Text style={styles.btnText}> </Text>
    </TouchableOpacity>
  </>
) : null}
</ScrollView>
</SafeAreaView>
);
}
const styles = StyleSheet.create({
  flex: { flex: 1, backgroundColor: "#f1f5f9" },
  center: { justifyContent: "center", alignItems: "center" },
  container: { padding: 24, gap: 12 },
  title: { fontSize: 24, fontWeight: "bold", color: "#0f766e", textAlign: "center", marginTop: 40 },
  sub: { fontSize: 14, color: "#64748b", textAlign: "center", marginBottom: 12 },

  simBox: { backgroundColor: "white", borderRadius: 14, padding: 12, borderWidth: 1, borderColor: "#e2e8f0" },
  simText: { fontSize: 12, color: "#334155", textAlign: "right" },
  msg: { fontSize: 12, color: "#b45309", backgroundColor: "#fffbeb", padding: 12, borderRadius: 12, textAlign: "right",
lineHeight: 22 },
  input: { backgroundColor: "white", borderRadius: 14, borderWidth: 1, borderColor: "#cbd5e1", padding: 14, fontSize: 14,
textAlign: "right" },
  code: { textAlign: "center", fontSize: 22, letterSpacing: 8 },
  btn: { backgroundColor: "#0f766e", borderRadius: 14, padding: 15, alignItems: "center" },
  btnText: { color: "white", fontWeight: "bold", fontSize: 14 },
  btnGhost: { backgroundColor: "white", borderWidth: 1, borderColor: "#0f766e" },
  btnGhostText: { color: "#0f766e", fontWeight: "bold", fontSize: 14 },
  hint: { fontSize: 12, color: "#64748b", textAlign: "center", marginTop: 8 },
});

@@ src/db/index.ts typescript
import { drizzle } from "drizzle-orm/node-postgres";
import { Pool, type PoolClient, type PoolConfig } from "pg";
import { isTransientError, withRetry, type RetryOptions } from "@lib/retry";
/* =====
* PostgreSQL (Neon / Render / )
*
* ) endpoint «pooler» Neon
* ) connect_timeout=15
* ) Retry Exponential Backoff
* ===== */
const rawUrl = process.env.DATABASE_URL;
export const hasDatabaseUrl = Boolean(rawUrl);
if (!rawUrl) {
  console.error(" DATABASE_URL . Render → Environment Neon .");
}
const FALLBACK_URL = "postgresql://invalid:invalid@127.0.0.1:1/invalid";
/**
function isLocalHost(url: string) {
  return /(localhost|127\.0\.0\.1|host\.docker\.internal)/.test(url);
}
**
* :
* - Neon -pooler` (PgBouncer )

```

```

* - `sslmode=require`
* - `connect_timeout=15`
* - `application_name`      Neon
*/
export function normalizeConnectionString(input: string): { url: string; usedPooler: boolean; local: boolean } {
  const local = isLocalHost(input);
  let usedPooler = false;
  let parsed: URL;
  try {
    parsed = new URL(input);
  } catch {
    return { url: input, usedPooler: false, local };
  }
  if (!local) {
    // Neon: ep-xxx.region.aws.neon.tech → ep-xxx-pooler.region.aws.neon.tech
    const host = parsed.hostname;
    if (/\.neon\.tech$/i.test(host) && !host.includes("-pooler")) {
      const [first, ...rest] = host.split(".");
      parsed.hostname = `${first}-pooler`, ...rest].join(".");
      usedPooler = true;
    } else if (host.includes("-pooler")) {
      usedPooler = true;
    }
    if (!parsed.searchParams.has("sslmode")) parsed.searchParams.set("sslmode", "require");
  }
  // — cold start
  if (!parsed.searchParams.has("connect_timeout")) parsed.searchParams.set("connect_timeout", "15");
  if (!parsed.searchParams.has("application_name")) parsed.searchParams.set("application_name", "sabt-etelaat-kol");
  return { url: parsed.toString(), usedPooler, local };
}
const { url: connectionString, usedPooler, local } = normalizeConnectionString(rawUrl ?? FALLBACK_URL);
if (rawUrl) {
  console.log(
    `      : ${local ? "" : usedPooler ? "Neon pooler " : " ( pooler)"} | connect_timeout=15s`
  );
}
const poolConfig: PoolConfig = {
  connectionString,
  ssl: local ? undefined : { rejectUnauthorized: false },
  // PgBouncer
  max: Number(process.env.DB_POOL_MAX ?? (local ? 5 : 8)),
  min: 0,
  idleTimeoutMillis: 30_000,
  connectionTimeoutMillis: 15_000,
  //
  statement_timeout: 20_000,
  query_timeout: 20_000,
  keepAlive: true,
  keepAliveInitialDelayMillis: 10_000,
  allowExitOnIdle: false,
};
const globalForDb = globalThis as typeof globalThis & {
  __sekPool?: Pool;
};
function createPool() {
  const p = new Pool(poolConfig);
  //
  p.on("error", (err) => {
    console.error("      ( ):", err.message);
  });
  return p;
}
export const pool = globalForDb.__sekPool ?? createPool();
if (process.env.NODE_ENV !== "production") globalForDb.__sekPool = pool;
export const db = drizzle(pool);
/* ----- */
/* ----- */
const DB_RETRY: RetryOptions = {
  retries: 4,
  baseDelayMs: 1_000,
  maxDelayMs: 30_000,
  timeoutMs: 25_000,
  isRetryable: isTransientError,
};
/**
 *
 *
 * @example
 * const rows = await dbRetry(() => db.select().from(users), "users:list");
 */
export function dbRetry<T>(operation: () => Promise<T>, label = "db"): Promise<T> {
  return withRetry(() => operation(), { ...DB_RETRY, label });
}

```

```

}
/** : ( ) */
export async function dbRetrySafe<T>(operation: () => Promise<T>, fallback: T, label = "db"): Promise<T> {
  try {
    return await dbRetry(operation, label);
  } catch (err) {
    console.error(`db:${label} :`, err instanceof Error ? err.message : err);
    return fallback;
  }
}
/** */
export function dbTransaction<T>(fn: (client: PoolClient) => Promise<T>, label = "tx"): Promise<T> {
  return dbRetry(async () => {
    const client = await pool.connect();
    try {
      await client.query("BEGIN");
      const result = await fn(client);
      await client.query("COMMIT");
      return result;
    } catch (err) {
      await client.query("ROLLBACK").catch(() => undefined);
      throw err;
    } finally {
      client.release();
    }
  }, label);
}
/** */
export async function checkDatabase(): Promise<{ ok: boolean; latencyMs: number; detail: string }> {
  const started = Date.now();
  try {
    await withRetry(() => pool.query("select 1"), {
      retries: 2,
      baseDelayMs: 700,
      maxDelayMs: 3_000,
      timeoutMs: 12_000,
      label: "health",
    });
    return { ok: true, latencyMs: Date.now() - started, detail: usedPooler ? "pooler" : local ? "local" : "direct" };
  } catch (err) {
    return {
      ok: false,
      latencyMs: Date.now() - started,
      detail: err instanceof Error ? err.message.slice(0, 200) : "unknown",
    };
  }
}
export const dbInfo = { usedPooler, local, connectTimeoutSec: 15 };

@@ package.json json
{
  "name": "namayandeelmi-javad-crm",
  "version": "2.5.2",
  "description": "نامایانده‌لی می‌جواد کریم (CRM / PWA) نامایانده‌لی می‌جواد کریم Render Neon",
  "main": "server.js",
  "engines": {
    "node": ">=20.0.0"
  },
  "scripts": {
    "prebuild": "echo 'Assets check complete - ready for build'",
    "build": "echo 'Build completed successfully! Ready for Render deployment.'",
    "start": "node server.js",
    "start:raw": "node server.js",
    "dev": "node server.js",
    "test": "echo 'No tests required'"
  },
  "keywords": [
    "crm",
    "medical-representative",
    "pharmacy",
    "doctor-visit",
    "farsi",
    "rtl",
    "leaflet",
    "render",
    "neon"
  ],
  "author": "Javad Alamdard Mehraeen & Arena",
  "license": "MIT",
  "dependencies": {
    "cors": "^2.8.5",
    "express": "^4.19.2"
  }
}

```

```
@@ public/index.html html
<!DOCTYPE html>
<html lang="fa" dir="rtl">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>سیستم مدیریت منابع انسانی و مالی (CRM / PWA)</title>

  <!-- Leaflet CSS -->
  <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />

  <!-- PWA Manifest -->
  <link rel="manifest" href="manifest.json" />
  <meta name="theme-color" content="#0d9488" />

  <!-- Custom Style with Anti-Cache Version 8.0.0 -->
  <link rel="stylesheet" href="style.css?v=8.0.0" />
</head>
<body>

  <!-- Top App Header matching Screenshot 1 -->
  <header class="app-header">
    <div class="header-container">
      <div class="logo-area">
        <button id="btnToggleSideMenu" class="btn-menu-hamburger" title="تغییر منو جانبی">
          <span>☰</span>
        </button>
      <div class="logo-icon">☐</div>
      <div class="logo-text">
        <h1 id="headerCompanyNameDisplay">شرکت توسعه و فناوری</h1>
        <p>پشتیبانی از React Native | Render.com | Neon | PostgreSQL</p>
      </div>
      <div class="header-actions">
        <div class="header-user-pill">
          <span id="currentUserRoleLabel">کاربر</span>
          <span class="badge-status-online" id="globalOnlineStatusBadge">آنلاین</span>
        </div>
        <button class="btn-header-icon" id="btnNotificationBellHeader" title="اعلان">
          <span>🔔</span>
          <span class="notif-count-badge" id="headerNotifBadgeCount">۰</span>
        </button>
        <button id="btnOpenLoginModal" class="btn-header-teal">
          <span>ورود</span>
        </button>
        <button id="btnLogoutSystem" class="btn-header-logout">
          <span>خروج</span>
        </button>
      </div>
    </div>
  </header>

  <!-- Navigation Bar (Quick Horizontal Scrollable Tabs) -->
  <nav class="app-nav">
    <div class="nav-container" id="horizontalNavContainer">
      <!-- Generated dynamically from all 20 sections -->
    </div>
  </nav>

  <!-- SIDEBAR NAVIGATION DRAWER (Matching Screenshot 1) -->
  <aside id="sideMenuDrawer" class="side-menu-drawer">
    <div class="side-menu-header">
      <div style="display: flex; align-items: center; gap: 0.5rem;">
        <span style="font-size: 1.5rem;">☰</span>
        <span style="font-weight: 800; font-size: 1.1rem; color: #fff;">منو جانبی</span>
      </div>
      <button id="btnCloseSideMenu" class="btn-close-side" title="بستن منو">✕</button>
    </div>
    <div class="side-menu-content" id="sideMenuItemsContainer">
      <!-- Dynamic list of all 20 sections matching Screenshot 1 -->
    </div>
  </aside>
  <div id="sideMenuOverlay" class="side-menu-overlay"></div>

  <!-- Main Content Area -->
  <main class="main-content">

    <!-- =====>
    1. TAB: DASHBOARD (تابلوی مدیر)
    <!-- ===== -->
    <section id="tab-dashboard" class="tab-pane active">
      <div class="stats-grid">
        <div class="stat-card">
          <div class="stat-icon" style="background: #eff6ff; color: #2563eb;">☐</div>
        </div>
      </div>
    </section>
  </main>
</body>
</html>
```



```

        <div class="stat-content">
            <h3 id="statPharmacies">0</h3>
        <p>00000 000000000000</p>
        </div>
    </div>
    <div class="stat-card">
        <div class="stat-icon" style="background: #f0fdf4; color: #10b981;">0000</div>
        <div class="stat-content">
            <h3 id="statDoctors">0</h3>
        <p>00000 000000 0 000000</p>
        </div>
    </div>
    <div class="stat-card">
        <div class="stat-icon" style="background: #ffffeb; color: #f59e0b;">0</div>
        <div class="stat-content">
            <h3 id="statReps">0</h3>
        <p>0000000000 0000 0000</p>
        </div>
    </div>
    <div class="stat-card">
        <div class="stat-icon" style="background: #fdf2f8; color: #ec4899;">0</div>
        <div class="stat-content">
            <h3 id="statOrders">0</h3>
        <p>00000 00 0000000</p>
        </div>
    </div>
    <div class="stat-card">
        <div class="stat-icon" style="background: #ecfeff; color: #0891b2;">0</div>
        <div class="stat-content">
            <h3 id="statUsers">0</h3>
        <p>00000000 0 000 000000</p>
        </div>
    </div>
    <div class="stat-card">
        <div class="stat-icon" style="background: #f3e8ff; color: #9333ea;">0</div>
        <div class="stat-content">
            <h3 id="statLeaves">0</h3>
        <p>0000000000 00 000000</p>
        </div>
    </div>
<!-- 20-Section Quick Launchpad Grid (00000000 00000000 0000000 - 0000 0000000 0000) -->
    <div class="card" style="margin-bottom: 1.5rem;">
        <div class="card-header">
            <div class="card-title">
                <span>0 000000 0000 00 00000 00 000000 00000000 (0000 0000 00 0000000 00 00 0 0000)</span>
            </div>
        </div>
        <p style="font-size: 0.88rem; color: #64748b; margin-bottom: 1rem;">
            00000 00000000 0000000 00 00 0000 0000 00000000 00 0000 0000000 00 00 0 00000 00 00 0000 0000 00 000 0000:
        </p>
        <div class="launchpad-grid" id="dashboardLaunchpadGrid">
            <!-- Generated dynamically for all 20 sections -->
        </div>
    </div>

    <!-- Interactive Chart Analytics Widget (Section 1 Requirement) -->
    <div class="card" id="dashboardChartsWidget" style="margin-bottom: 1.5rem;">
        <div class="card-header">
            <div class="card-title">
                <span>0 0000000000 000000 0000000000 0000 0 000000 0000000000 (Chart Analytics)</span>
            </div>
            <div style="display: flex; gap: 0.5rem;">
                <button type="button" class="btn btn-outline btn-sm" onclick="alert('00 000000 00000000 000 0 00000000 000000000 0
000000 0000 0000 0000 00.')">
                    <span>00 0000000 000 0 0000000</span>
                </button>
            </div>
        </div>
        <div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(280px, 1fr)); gap: 1.25rem;">
            <div style="background: #f8fafc; padding: 1rem; border-radius: 10px; border: 1px solid var(--border-color);">
                <h5 style="margin-bottom: 0.75rem; color: #0d9488;">0 000000 00000000 0000 0000000000 (0000)</h5>
                <div style="display: flex; align-items: flex-end; gap: 1rem; height: 140px; padding: 0.5rem 0; border-bottom
: 2px solid var(--border-color);">
                    <div style="flex: 1; text-align: center;">
                        <div style="background: #0d9488; height: 100px; border-radius: 6px 6px 0 0;" title="380,000,000 0000"></div>
                    <span style="font-size: 0.75rem; font-weight: 700;">0000 000000</span>
                </div>
                <div style="flex: 1; text-align: center;">
                    <div style="background: #10b981; height: 120px; border-radius: 6px 6px 0 0;" title="410,000,000 0000"></div>
                    <span style="font-size: 0.75rem; font-weight: 700;">0000 00000</span>
                </div>
            </div>
        </div>
    </div>

```

```
<div style="flex: 1; text-align: center;">
<div style="background: #f59e0b; height: 60px; border-radius: 6px 6px 0 0; title="150,000,000" />
iv>
<span style="font-size: 0.75rem; font-weight: 700;"></span>
</div>
</div>
</div>

<div style="background: #f8fafc; padding: 1rem; border-radius: 10px; border: 1px solid var(--border-color);">
<h5 style="margin-bottom: 0.75rem; color: #0d9488;"></h5>
<div style="display: flex; align-items: center; justify-content: center; gap: 1.5rem; height: 140px;">
<div style="width: 100px; height: 100px; border-radius: 50%; background: conic-gradient(#0d9488 0% 65%, #1
0b981 65% 100%); box-shadow: var(--shadow-sm);"></div>
<div style="font-size: 0.82rem; color: #334155;">
<div><span style="color: #0d9488;"></span> </div>
<div><span style="color: #10b981;"></span> </div>
</div>
</div>
</div>
</div>
</div>

<!-- Quick Overview Map -->
<div class="card">
<div class="card-header">
<div class="card-title">
<span></span>
</div>
<button id="btnRefreshOverviewMap" class="btn btn-outline btn-sm">
<span></span>
</button>
</div>
<p style="font-size: 0.88rem; color: #64748b; margin-bottom: 1rem;">
</p>
<div id="map-dashboard-overview" class="map-container map-container-large"></div>
</div>
</section>

<!-- =====
2. TAB: PHARMACIES (Matching Screenshots 2 & 3
===== -->
<section id="tab-pharmacies" class="tab-pane">
<div class="card" id="cardPhForm">
<div class="card-header" style="justify-content: space-between;">
<div style="display: flex; align-items: center; gap: 0.5rem;">
<button class="btn btn-outline btn-sm" onclick="switchTab('tab-dashboard')"></button>
<button class="btn btn-outline btn-sm" onclick="switchTab('tab-dashboard')"></button>
</div>
<div class="card-title" style="font-size: 1.25rem;">
<span></span>
</div>
</div>
</div>

<!-- Tab switcher inside Pharmacy card -->
<div style="display: flex; gap: 0.5rem; margin-bottom: 1.25rem; border-bottom: 2px solid var(--border-color); pa
dding-bottom: 0.5rem;">
<button type="button" class="btn btn-primary btn-sm" id="btnShowPhForm"></button>
<button type="button" class="btn btn-outline btn-sm" id="btnShowPhList"><span id="phListCountBadge">0</s
pan></button>
</div>

<form id="formPharmacy" onsubmit="return false;">
<input type="hidden" id="pharmacyEditId" value="" />
<div class="form-grid">
<div class="form-group">
<label class="form-label" for="pharmacyDate"></label>
<input type="text" id="pharmacyDate" class="form-input" placeholder="1405/05/20 - " required />
</div>

<!-- Cascading Provinces / Cities / Sorted Districts (Req 4) -->
<div class="form-group">
<label class="form-label" for="pharmacyProvince"></label>
<select id="pharmacyProvince" class="form-select dropdown-auto-clear" required>
<option value=""></option>
</select>
</div>
<div class="form-group">
<label class="form-label" for="pharmacyCity"></label>
<select id="pharmacyCity" class="form-select dropdown-auto-clear" required>
<option value=""></option>
</select>
</div>
```

```
<div class="form-group">
<label class="form-label" for="pharmacyDistrict">□□□□ (□□□□ □□□ □□ - □□ □□□□□ □□□)</label>
  <select id="pharmacyDistrict" class="form-select dropdown-auto-clear" required>
    <option value="">□□□□ □□ □□ □□□□ □□□□</option>
  </select>
</div>

<div class="form-group">
<label class="form-label" for="pharmacyName">□□ □□□□□□□ * (□□□□□ □□ □□□ □□□□... □□□□ □□□□ + □□□□□ □□□
□□□□)</label>
<input type="text" id="pharmacyName" class="form-input" placeholder="□□□□: □□□□□□□□ □□□□ □□□□□□..." requir
ed />
</div>
<div class="form-group">
<label class="form-label" for="pharmacyPhone">□□□□□ □□□□ □□□□□□□□ (...□□□)</label>
<input type="text" id="pharmacyPhone" class="form-input" placeholder="02188776655..." />
</div>
<div class="form-group">
<label class="form-label" for="pharmacyManager">□□ □□□□ □□□□ (□□□□□□ □□ □□□□ □□□□...)</label>
<input type="text" id="pharmacyManager" class="form-input" placeholder="□□ □□□□ □□ □□ □□□□..." />
</div>
<div class="form-group">
<label class="form-label" for="pharmacyManagerPhone">□□□□ □□□□ □□□□ □□□□</label>
<input type="text" id="pharmacyManagerPhone" class="form-input" placeholder="0912..." />
</div>

<div class="form-group full-width">
<label class="form-label" for="pharmacyAddress">□□□□ □□□□□□□□</label>
<input type="text" id="pharmacyAddress" class="form-input" placeholder="□□□□ □□□□ □□□□□□□□..." required />
</div>

<!-- Toggle: □□ □□ □□□□□□□ □□□□ □□□□□□□□ (Matching Screenshot 3) -->
<div class="form-group full-width" style="background: #fef9c3; border: 1px solid #fde047; padding: 1rem; bor
der-radius: 10px; display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap;">
  <span style="font-weight: 700; color: #854d0e;">□□ □□ □□□□□□□ □□□□ □□□□□□□□</span>
  <div style="display: flex; gap: 0.5rem;">
    <button type="button" id="btnPhPercentageNo" class="btn-toggle-option active">□□□</button>
    <button type="button" id="btnPhPercentageYes" class="btn-toggle-option">□□□</button>
  </div>
  <input type="hidden" id="pharmacyIsPercentage" value="false" />
</div>

<!-- File Upload: □□□□□□□□ □□ □□ □□□□ □□□□□□□□ (Matching Screenshot 3) -->
<div class="form-group full-width" style="background: #f0fdf4; border: 1px solid #bbf7d0; padding: 1rem; bor
der-radius: 10px;">
  <div style="display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap;">
    <div>
<h4 style="color: #166534; font-size: 0.95rem; margin-bottom: 2px;">□ □□□□□□□ □□ □□ □□□□□□□□</
h4>
<p style="font-size: 0.78rem; color: #15803d; margin: 0;">□□□□ PDF □ □□□□ - □□□□□□ □ □□□□□□□</p>
    </div>
    <div>
      <button type="button" class="btn btn-outline btn-sm" onclick="document.getElementById('phFileInput').c
lick()">
        <span>□ □□□□□ □□ □□ □□□□</span>
      </button>
      <input type="file" id="phFileInput" style="display: none;" />
    </div>
  </div>
  <div id="phFileDisplay" style="margin-top: 0.5rem; font-size: 0.82rem; color: #166534;">□□□□□ □□□□□□□□ □□
□ □□□</div>
</div>

<!-- Dynamic Custom Fields for Pharmacy -->
<div id="pharmacyCustomFieldsContainer" class="form-group full-width form-grid" style="margin-top: 0.5rem;">

  <!-- Location Features (GPS + Map Search + Reverse Geocode) Matching Screenshot 3 -->
  <div class="form-group full-width" style="background: #ecfeff; padding: 1rem; border-radius: 10px; border: 1
px solid #a5f3fc;">
    <div style="display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap; gap: 0.75
rem; margin-bottom: 0.75rem;">
      <label class="form-label" style="margin: 0; color: #0e7490;">
        <span>□ □□□□□ □ □□□ □□□□□□□□ (□□□ □□□ □□□□ □)</span>
      </label>
      <div style="display: flex; gap: 0.5rem; flex-wrap: wrap;">
        <button type="button" id="btnPharmacyCurrentLocation" class="btn btn-primary btn-sm">
          <span>□ □□□□□ □□□□ □□</span>
        </button>
        <button type="button" id="btnPharmacyGetAddressFromPoint" class="btn btn-success btn-sm">
          <span>□ □□□□□ □□□□ □□ □□□□</span>
        </button>
      </div>
    </div>
  </div>
```

```
<!-- Map Search bar (Matching Screenshot 3) -->
<div style="display: flex; gap: 0.5rem; margin-bottom: 0.75rem;">
  <input type="text" id="phMapSearchInput" class="form-input" placeholder="📍 000000 0000 00 000 0000... (0
0000: 00000 000 00000)" />
  <button type="button" id="btnPhMapSearch" class="btn btn-outline" style="white-space: nowrap;">00000</bu
tton>
</div>

<div class="form-grid">
  <div class="form-group">
<label class="form-label">000 0000000000 (lat)</label>
    <input type="number" step="any" id="pharmacyLat" class="form-input" value="35.7605" />
  </div>
  <div class="form-group">
<label class="form-label">000 0000000000 (lng)</label>
    <input type="number" step="any" id="pharmacyLng" class="form-input" value="51.4180" />
  </div>
  <div class="form-group full-width">
<label class="form-label">00000000 000000 / 0000 0000000 000 00 0000</label>
    <input type="text" id="pharmacyLocationText" class="form-input" placeholder="0000 00000000 000 00 000000
0..." readonly style="background: #f8f4fc;" />
  </div>
</div>
</div>
</div>

<!-- Pharmacy Form Map -->
<div style="margin: 1.25rem 0;">
  <div id="map-pharmacy-form" class="map-container"></div>
</div>

<div style="display: flex; justify-content: flex-end; gap: 0.75rem; margin-top: 1rem;">
  <button type="submit" id="btnSavePharmacy" class="btn btn-primary" style="background: #0d9488; font-size: 1r
em; padding: 0.75rem 1.75rem;">
<span>0 000 00000000</span>
  </button>
</div>
<p style="text-align: center; font-size: 0.78rem; color: #64748b; margin-top: 1.5rem;">
0000 000 000000000000 - 0000 000 000 000000 0 000000
</p>
</form>
</div>

<!-- Pharmacy List Card -->
<div class="card" id="cardPhList" style="display: none;">
  <div class="card-header">
    <div class="card-title">
<span>0 0000 00000000000000 000 000 (<span id="phTableCountBadge">0</span></span></div>
    <div style="display: flex; gap: 0.5rem; align-items: center; flex-wrap: wrap;">
      <select id="phMonthFilter" class="form-select dropdown-auto-clear" style="width: 150px;">
<option value="">000 000000</option>
<option value="05">00000 1405</option>
<option value="06">000000 1405</option>
      </select>
      <input type="text" id="searchPharmacyInput" class="form-input dropdown-auto-clear" placeholder="📍 00000..."
style="width: 220px;" />
      <button id="btnExportPharmaciesCSV" class="btn btn-success btn-sm" style="background: #0d9488;">
<span>00 00000 0000</span>
      </button>
    </div>
  </div>

  <div class="table-responsive">
    <table class="data-table" id="tablePharmacies">
      <thead>
        <tr id="tablePharmaciesHeader">
          <th>0000</th>
          <th>000 00000000</th>
          <th>00000 000</th>
          <th>00000</th>
          <th>000</th>
          <th>000 000000000</th>
          <th>00000 00000</th>
          <th>00000</th>
          <th>0000</th>
          <th>000000</th>
        </tr>
      </thead>
      <tbody id="tablePharmaciesBody">
        <!-- Dynamically populated -->
      </tbody>
    </table>
  </div>
</div>
</div>
```

```
</section>

<!-- =====
3. TAB: DOCTORS AND CLINICS (DOCTORS)
===== -->
<section id="tab-doctors" class="tab-pane">
  <div class="card" id="cardDocForm">
    <div class="card-header" style="justify-content: space-between;">
      <div style="display: flex; align-items: center; gap: 0.5rem;">
        <button class="btn btn-outline btn-sm" onclick="switchTab('tab-dashboard')">← DOCTORS</button>
        <button class="btn btn-outline btn-sm" onclick="switchTab('tab-dashboard')">DOCTORS</button>
      </div>
      <div class="card-title" style="font-size: 1.25rem;">
        <span>DOCTORS</span>
      </div>
    </div>
    <div style="display: flex; gap: 0.5rem; margin-bottom: 1.25rem; border-bottom: 2px solid var(--border-color); padding-bottom: 0.5rem;">
      <button type="button" class="btn btn-primary btn-sm" id="btnShowDocForm" style="background: #0d9488;">DOCTORS</button>
      <button type="button" class="btn btn-outline btn-sm" id="btnShowDocList">DOCTORS (<span id="docListCountBadge">0</span></button>
    </div>

    <form id="formDoctor" onsubmit="return false;">
      <input type="hidden" id="doctorEditId" value="" />
      <div class="form-grid">
        <div class="form-group">
          <label class="form-label" for="doctorDate">DOCTOR DATE</label>
          <input type="text" id="doctorDate" class="form-input" placeholder="1405/05/20" required />
        </div>
        <div class="form-group">
          <label class="form-label" for="doctorName">DOCTOR NAME / DOCTOR</label>
          <input type="text" id="doctorName" class="form-input" placeholder="DOCTOR: DOCTOR DOCTOR DOCTOR" required />
        </div>
        <div class="form-group">
          <label class="form-label" for="doctorSpecialty">DOCTOR SPECIALTY</label>
          <input type="text" id="doctorSpecialty" class="form-input" placeholder="DOCTOR: DOCTOR DOCTOR DOCTOR" required />
        </div>
        <div class="form-group">
          <label class="form-label" for="doctorPhone">DOCTOR PHONE</label>
          <input type="text" id="doctorPhone" class="form-input" placeholder="DOCTOR: 021-88990011" />
        </div>

        <!-- Cascading Provinces / Cities / Sorted Districts -->
        <div class="form-group">
          <label class="form-label" for="doctorProvince">DOCTOR PROVINCE</label>
          <select id="doctorProvince" class="form-select dropdown-auto-clear" required>
            <option value="">DOCTOR PROVINCE</option>
          </select>
        </div>
        <div class="form-group">
          <label class="form-label" for="doctorCity">DOCTOR CITY</label>
          <select id="doctorCity" class="form-select dropdown-auto-clear" required>
            <option value="">DOCTOR CITY</option>
          </select>
        </div>
        <div class="form-group">
          <label class="form-label" for="doctorDistrict">DOCTOR DISTRICT (DOCTOR DOCTOR DOCTOR)</label>
          <select id="doctorDistrict" class="form-select dropdown-auto-clear" required>
            <option value="">DOCTOR DISTRICT</option>
          </select>
        </div>

        <div class="form-group full-width">
          <label class="form-label" for="doctorAddress">DOCTOR ADDRESS</label>
          <input type="text" id="doctorAddress" class="form-input" placeholder="DOCTOR ADDRESS..." required />
        </div>

        <!-- Toggle: DOCTOR DOCTOR DOCTOR DOCTOR DOCTOR -->
        <div class="form-group full-width" style="background: #fef9c3; border: 1px solid #fde047; padding: 1rem; border-radius: 10px; display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap;">
          <span style="font-weight: 700; color: #854d0e;">DOCTOR DOCTOR DOCTOR DOCTOR DOCTOR</span>
          <div style="display: flex; gap: 0.5rem;">
            <button type="button" id="btnDocPercentageNo" class="btn-toggle-option active">DOCTOR</button>
            <button type="button" id="btnDocPercentageYes" class="btn-toggle-option">DOCTOR</button>
          </div>
          <input type="hidden" id="doctorIsPercentage" value="false" />
        </div>

        <!-- File Upload: DOCTOR DOCTOR DOCTOR DOCTOR -->
        <div class="form-group full-width" style="background: #f0fdf4; border: 1px solid #bbf7d0; padding: 1rem; border-radius: 10px;">
          <div style="display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap;">
```

```

        <div>
        <h4 style="color: #166534; font-size: 0.95rem; margin-bottom: 2px;">□ □□□□□□□ □□□ □ □□ □□□</h4>
        <p style="font-size: 0.78rem; color: #15803d; margin: 0;">□□□□ PDF □ □□□ - □□□□□ □ □□□□□□</p>
        </div>
        <div>
        <button type="button" class="btn btn-outline btn-sm" onclick="document.getElementById('docFileInput').
click()">
        <span>□ □□□□□ □□□□ □□□</span>
        </button>
        <input type="file" id="docFileInput" style="display: none;" />
        </div>
        </div>
        <div id="docFileDisplay" style="margin-top: 0.5rem; font-size: 0.82rem; color: #166534;">□□□□ □□□□□□□ □
□□ □□</div>
        </div>

        <!-- Dynamic Custom Fields for Doctor -->
        <div id="doctorCustomFieldsContainer" class="form-group full-width form-grid" style="margin-top: 0.5rem;"></
div>

        <!-- Location Features (GPS + Map Search + Reverse Geocode) -->
        <div class="form-group full-width" style="background: #ecfeff; padding: 1rem; border-radius: 10px; border: 1
px solid #a5f3fc;">
        <div style="display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap; gap: 0.75
rem; margin-bottom: 0.75rem;">
        <label class="form-label" style="margin: 0; color: #0e7490;">
        <span>□ □□□□□ □ □□□ □□ (□□ □□ □□□ □□□)</span>
        </label>
        <div style="display: flex; gap: 0.5rem; flex-wrap: wrap;">
        <button type="button" id="btnDoctorCurrentLocation" class="btn btn-primary btn-sm">
        <span>□ □□□□□ □□□□ □□</span>
        </button>
        <button type="button" id="btnDoctorGetAddressFromPoint" class="btn btn-success btn-sm">
        <span>□ □□□□□ □□□□ □□ □□□</span>
        </button>
        </div>
        </div>

        <!-- Map Search bar -->
        <div style="display: flex; gap: 0.5rem; margin-bottom: 0.75rem;">
        <input type="text" id="docMapSearchInput" class="form-input" placeholder="□ □□□□□ □□□ □ □□ □□ □□.
.." />
        <button type="button" id="btnDocMapSearch" class="btn btn-outline">□□□□</button>
        </div>

        <div class="form-grid">
        <div class="form-group">
        <label class="form-label">□□ □□□□□□□□ (lat)</label>
        <input type="number" step="any" id="doctorLat" class="form-input" value="35.7580" />
        </div>
        <div class="form-group">
        <label class="form-label">□□ □□□□□□□□ (lng)</label>
        <input type="number" step="any" id="doctorLng" class="form-input" value="51.4400" />
        </div>
        <div class="form-group full-width">
        <label class="form-label">□□□□□□ □□□□□ □□□□ □□ □□□</label>
        <input type="text" id="doctorLocationText" class="form-input" placeholder="□□□□ □□□□□□ □□ □□ □ □□
□□□..." readonly style="background: #f8fafc;" />
        </div>
        </div>
        </div>
        </div>

        <!-- Doctor Form Map -->
        <div style="margin: 1.25rem 0;">
        <div id="map-doctor-form" class="map-container"></div>
        </div>

        <div style="display: flex; justify-content: flex-end; gap: 0.75rem; margin-top: 1rem;">
        <button type="submit" id="btnSaveDoctor" class="btn btn-primary" style="background: #0d9488; font-size: 1rem
; padding: 0.75rem 1.75rem;">
        <span>□ □□ □□□□□□ □□□</span>
        </button>
        </div>
        </form>
        </div>

        <!-- Doctor List Card -->
        <div class="card" id="cardDocList" style="display: none;">
        <div class="card-header">
        <div class="card-title">
        <span>□ □□□ □□□□ □ □□□□□ □□ □□</span>
        </div>
        <div style="display: flex; gap: 0.5rem; align-items: center; flex-wrap: wrap;">
        <input type="text" id="searchDoctorInput" class="form-input dropdown-auto-clear" placeholder="□ □□□□ □ □□

```

```

        style="width: 220px;" />
        <button id="btnExportDoctorsCSV" class="btn btn-success btn-sm" style="background: #0d9488;">
        <span>  Export Doctors CSV </span>
        </button>
    </div>
</div>

<div class="table-responsive">
    <table class="data-table" id="tableDoctors">
        <thead>
            <tr id="tableDoctorsHeader">
                <th>  </th>
                <th>  </th>
                <th>  </th>
                <th>  /  /  </th>
                <th>  /  </th>
                <th>  </th>
                <th>  </th>
                <th>  </th>
                <th>  </th>
            </tr>
        </thead>
        <tbody id="tableDoctorsBody">
            <!-- Dynamically populated -->
        </tbody>
    </table>
</div>
</div>
</section>

<!-- =====
4. TAB: ORDERS ( ) - Requirement 3: Top Alert Banner
===== -->
<section id="tab-orders" class="tab-pane">
    <div id="existingPharmacyTopAlert" style="display: none;">
        <div class="alert-info-content">
            <div class="alert-info-icon"> </div>
            <div class="alert-info-text">
                <h4>  :  </h4>
                <p id="existingPharmacyAlertText">  </p>
            </div>
            <div>
                <button type="button" id="btnTopAutoFillPharmacy" class="btn-autofill-pharmacy">
                <span>  </span>
                </button>
            </div>
</div>

<div class="card" id="cardOrdForm">
    <div class="card-header">
        <div class="card-title">
            <span>  </span>
        </div>
        <button id="btnResetOrderForm" class="btn btn-outline btn-sm">
            <span>  /  </span>
        </button>
    </div>
    <div style="display: flex; gap: 0.5rem; margin-bottom: 1.25rem; border-bottom: 2px solid var(--border-color); padding-bottom: 0.5rem;">
        <button type="button" class="btn btn-primary btn-sm" id="btnShowOrdForm" style="background: #0d9488;">  </button>
        <button type="button" class="btn btn-outline btn-sm" id="btnShowOrdList">  (<span id="ordListCountBadge">0</span>)</button>
    </div>

    <form id="formOrder" onsubmit="return false;">
        <input type="hidden" id="orderEditId" value="" />
        <input type="hidden" id="orderPharmacyMatchedId" value="" />

        <div class="form-grid">
            <div class="form-group">
                <label class="form-label" for="orderPharmacyName">
                <span>  * (  )</span>
                </label>
                <input type="text" id="orderPharmacyName" class="form-input" list="pharmacyNamesList" placeholder="  "
                required autocomplete="off" />
                <datalist id="pharmacyNamesList"></datalist>
            </div>

            <div class="form-group">
                <label class="form-label" for="orderProvince">  </label>
                <select id="orderProvince" class="form-select dropdown-auto-clear">
                    <option value="">  </option>
                </select>
            </div>
        </div>
    </form>
</div>

```

```
</div>
<div class="form-group">
<label class="form-label" for="orderCity">城市 *</label>
  <select id="orderCity" class="form-select dropdown-auto-clear">
    <option value="">请选择城市</option>
  </select>
</div>
<div class="form-group">
<label class="form-label" for="orderDistrict">区县 (区县 区县 区县) *</label>
  <select id="orderDistrict" class="form-select dropdown-auto-clear">
    <option value="">请选择区县</option>
  </select>
</div>

<div class="form-group full-width">
<label class="form-label" for="orderAddress">地址</label>
<input type="text" id="orderAddress" class="form-input" placeholder="请输入地址..." />
</div>

<div class="form-group">
<label class="form-label" for="orderRepName">销售代表</label>
  <select id="orderRepName" class="form-select dropdown-auto-clear">
    <option value="">请选择销售代表</option>
  </select>
</div>
<div class="form-group">
<label class="form-label" for="orderDate">日期</label>
  <input type="text" id="orderDate" class="form-input" placeholder="1405/05/18" />
</div>
<div class="form-group">
<label class="form-label" for="orderStatus">状态</label>
  <select id="orderStatus" class="form-select dropdown-auto-clear">
    <option value="待支付">待支付</option>
    <option value="已支付">已支付</option>
    <option value="已发货">已发货</option>
    <option value="已退货">已退货</option>
  </select>
</div>

<div class="form-group full-width" style="background: #f8fafc; padding: 1rem; border-radius: 10px; border: 1
px solid var(--border-color);">
  <div style="display: flex; align-items: center; justify-content: space-between; margin-bottom: 0.75rem;">
    <label class="form-label" style="margin: 0;">添加商品</label>
    <button type="button" id="btnAddOrderItemRow" class="btn btn-outline btn-sm">
      <span>+ 添加商品</span>
    </button>
  </div>
  <div id="orderItemsContainer" style="display: flex; flex-direction: column; gap: 0.5rem;">
    <div style="display: flex; justify-content: flex-end; margin-top: 0.75rem; font-weight: 700; color: #0f172
a;">
      <span>共 0 件商品</span>
      <span id="orderTotalAmountDisplay" style="margin-right: 0.5rem; color: #2563eb;">0</span>
      <span style="margin-right: 0.25rem;"> 合计</span>
    </div>
  </div>

  <div id="orderCustomFieldsContainer" class="form-group full-width form-grid" style="margin-top: 0.5rem;">
    <div class="form-group full-width">
      <label class="form-label" for="orderNotes">备注</label>
      <textarea id="orderNotes" class="form-textarea" placeholder="请输入备注信息...">
    </div>
    <div style="display: flex; justify-content: flex-end; gap: 0.75rem; margin-top: 1rem;">
      <button type="submit" id="btnSaveOrder" class="btn btn-primary" style="background: #0d9488;">
        <span>保存</span>
      </button>
    </div>
  </div>
</div>

<div class="card" id="cardOrdList" style="display: none;">
  <div class="card-header">
    <div class="card-title">
      <span>订单列表</span>
    </div>
    <div style="display: flex; gap: 0.5rem; align-items: center; flex-wrap: wrap;">
      <input type="text" id="searchOrderInput" class="form-input dropdown-auto-clear" placeholder="搜索订单" />
      <button id="btnExportOrdersCSV" class="btn btn-success btn-sm" style="background: #0d9488;">
        <span>导出</span>
      </button>
    </div>
  </div>
```



```

    </div>
</div>

<div class="table-responsive">
  <table class="data-table" id="tableOrders">
    <thead>
      <tr id="tableOrdersHeader">
        <th>□□□□</th>
        <th>□□□ □□□□□□□□</th>
        <th>□□□□□ / □□□ / □□□□□</th>
        <th>□□□□□□□ □□□□</th>
        <th>□□□□□ □□□□□</th>
        <th>□□□□□ □□□□□</th>
        <th>□□□□ □□ (□□□□)</th>
        <th>□□□□□</th>
        <th>□□□□□□</th>
      </tr>
    </thead>
    <tbody id="tableOrdersBody">
      <!-- Dynamically populated -->
    </tbody>
  </table>
</div>
</div>
</section>

<!-- =====
5. TAB: INSTANT ACTIVITY LOG (□□□□□□ □□□□□□□) - Menu Item 5
===== -->
<section id="tab-activity-log" class="tab-pane">
  <div class="card">
    <div class="card-header">
      <div class="card-title">
        <span>□□ □□□□□□ □□□□□□□ □ □□□□□□ □□□□ □□□□□□□□□□</span>
      </div>
      <button id="btnRefreshActivity" class="btn btn-outline btn-sm">
        <span>□ □□□□□□□□□ □□□□□□□□□□</span>
      </button>
    </div>
    <p style="font-size: 0.88rem; color: #64748b; margin-bottom: 1rem;">
      □□□ □□□□□ □□□□□□□□ □ □□□□□□ □□□□□□ □□□□□□□□ □□□□ □□□□ □□□□□□□□ □□ □□□□ □□□□□.
    </p>
    <div class="table-responsive">
      <table class="data-table">
        <thead>
          <tr>
            <th>□□□□ □□□</th>
            <th>□□□ □□□□□□□□ □□□□</th>
            <th>□□□□□ □□□□□□□□□□</th>
            <th>□□□□□</th>
          </tr>
        </thead>
        <tbody id="tableActivityLogBody">
          <!-- Dynamically populated -->
        </tbody>
      </table>
    </div>
  </div>
</div>
</section>

<!-- =====
6. TAB: COMPREHENSIVE MAP (□□□□ □□□□) - Menu Item 6 + Province/City/District Zoom
===== -->
<section id="tab-overview-map" class="tab-pane">
  <div class="card">
    <div class="card-header">
      <div class="card-title">
        <span>□□ □□□□ □□□□ □□□□ □□□□□□□□□□□□ □□ □□□□□□ □□□□ □ □□ □□□□ □□□□</span>
      </div>
      <div style="display: flex; gap: 0.5rem; flex-wrap: wrap;">
        <select id="mapFilterProvince" class="form-select dropdown-auto-clear" style="width: 140px;">
          <option value="">□□□ □□□□□□□□</option>
        </select>
        <select id="mapFilterCity" class="form-select dropdown-auto-clear" style="width: 140px;">
          <option value="">□□□ □□□□□</option>
        </select>
        <select id="mapFilterDistrict" class="form-select dropdown-auto-clear" style="width: 140px;">
          <option value="">□□□ □□□□□</option>
        </select>
        <button id="btnFocusMapRegion" class="btn btn-primary btn-sm" style="background: #0d9488;">
          <span>□ □□□□□□□□□ □ □□□□ □□□□□□</span>
        </button>
      </div>
    </div>
    <div id="map-full-overview" class="map-container" style="height: 520px; margin-bottom: 1rem;"></div>
  </div>
</section>

```

```

<!-- Summary Counts Box for Selected Region -->
<div id="overviewMapSummaryBox" style="background: #f0fdf4; border: 1.5px solid #bbf7d0; border-radius: 10px; padding: 1rem; display: flex; align-items: center; justify-content: space-around; flex-wrap: wrap; gap: 1rem; color: #166534; font-weight: 700;">
  <div>
    <strong id="cntOverviewPharmacies" style="color: #0d9488; font-size: 1.15rem;">0</strong>
  </div>
  <div>
    <strong id="cntOverviewDoctors" style="color: #10b981; font-size: 1.15rem;">0</strong>
  </div>
  <div>
    <strong id="cntOverviewHospitals" style="color: #059669; font-size: 1.15rem;">0</strong>
  </div>
</div>
</section>

<!-- =====>
7. TAB: LIVE LOCATION (Overview Map) - Menu Item 7
===== -->
<section id="tab-live-location" class="tab-pane">
  <div class="card">
    <div class="card-header">
      <div class="card-title">
        <span>
          Overview Map
          </span>
      </div>
      <div style="display: flex; gap: 0.5rem;">
        <button id="btnSimulateLiveMovement" class="btn btn-primary btn-sm" style="background: #0d9488;">
          Simulate Live Movement
        </button>
        <button id="btnRefreshLiveMap" class="btn btn-outline btn-sm">
          Refresh Map
        </button>
      </div>
    </div>
    <div id="map-live-reps" class="map-container map-container-large"></div>
  </div>

  <div class="card">
    <div class="card-header">
      <div class="card-title">
        <span>
          Live Reps
          </span>
      </div>
    </div>
    <div class="table-responsive">
      <table class="data-table" id="tableLiveReps">
        <thead>
          <tr>
            <th>Rep ID</th>
            <th>Rep Name</th>
            <th>Rep Address (Lat, Lng)</th>
            <th>Rep Status</th>
            <th>Rep Type</th>
            <th>Rep Category</th>
          </tr>
        </thead>
        <tbody id="tableLiveRepsBody">
          <!-- Dynamically populated -->
        </tbody>
      </table>
    </div>
  </div>
</section>

<!-- =====>
8. TAB: REP ROUTES & TRAFFIC MONITORING (Overview Map) - Menu Item 8
===== -->
<section id="tab-rep-routes" class="tab-pane">
  <div class="card">
    <div class="card-header">
      <div class="card-title">
        <span>
          Rep Routes & Traffic Monitoring
          </span>
      </div>
      <div style="display: flex; gap: 0.5rem; align-items: center; flex-wrap: wrap;">
        <select id="routeRepFilterSelect" class="form-select dropdown-auto-clear" style="width: 200px;">
          <option value="">Filter by</option>
        </select>
        <button id="btnRefreshRepRoutesMap" class="btn btn-outline btn-sm">
          Refresh Map
        </button>
      </div>
    </div>
    <div id="map-rep-routes-full" class="map-container" style="height: 480px; margin-bottom: 1.5rem;"></div>
    <p style="font-size: 0.88rem; color: #64748b; margin-bottom: 1rem;">
      Overview Map showing the routes and traffic monitoring for the selected region. The map displays the current location of the selected region and the routes of the selected region. The map also displays the traffic monitoring data for the selected region.
    </p>
  </div>

```

```
<div class="table-responsive">
  <table class="data-table">
    <thead>
      <tr>
        <th><div></div></th>
        <th><div></div></th>
        <th><div></div></th>
        <th><div></div></th>
        <th><div></div></th>
        <th><div></div></th>
      </tr>
    </thead>
    <tbody id="tableRepRoutesBody">
      <!-- Dynamically populated -->
    </tbody>
  </table>
</div>
</div>
</section>

<!-- =====
9. TAB: REP HOMES (<div></div>) - Menu Item 9
===== -->
<section id="tab-rep-homes" class="tab-pane">
  <div class="card">
    <div class="card-header">
      <div class="card-title">
        <span><div></div></span>
      </div>
      <button id="btnRepHomeCurrentLocation" class="btn btn-primary btn-sm" style="background: #0d9488;">
        <span><div></div></span>
      </button>
    </div>
    <p style="font-size: 0.88rem; color: #64748b; margin-bottom: 1rem;">
      <div></div>
    </p>
    <div id="map-rep-homes" class="map-container" style="height: 360px; margin-bottom: 1.5rem;">
      <div class="table-responsive">
        <table class="data-table">
          <thead>
            <tr>
              <th><div></div></th>
              <th><div></div></th>
              <th><div></div></th>
              <th><div></div></th>
            </tr>
          </thead>
          <tbody id="tableRepHomesBody">
            <!-- Dynamically populated -->
          </tbody>
        </table>
      </div>
    </div>
  </div>
</section>

<!-- =====
10. TAB: LEAVES (<div></div>) - Menu Item 10 + Hourly/Daily + 2-Tier Approval
===== -->
<section id="tab-leaves" class="tab-pane">
  <div class="card">
    <div class="card-header">
      <div class="card-title">
        <span><div></div></span>
      </div>
      <button id="btnExportLeavesCSV" class="btn btn-success btn-sm" style="background: #0d9488;">
        <span><div></div></span>
      </button>
    </div>
    <form id="formLeaveRequest" onsubmit="return false;" style="background: #f8fafc; padding: 1.25rem; border-radius : 10px; border: 1px solid var(--border-color); margin-bottom: 1.5rem;">
      <h4 style="margin-bottom: 1rem; border-bottom: 1px solid #e2e3e5; color: #1e293b;"><div></div></h4>
      <div class="form-grid">
        <div class="form-group">
          <label class="form-label"><div></div></label>
          <select id="leaveRepSelect" class="form-select dropdown-auto-clear" required>
            <option value=""><div></div>...</option>
          </select>
        </div>
        <div class="form-group">
          <label class="form-label"><div></div></label>
          <select id="leaveTypeSelect" class="form-select dropdown-auto-clear" required>
            <option value=""><div></div>...</option>
          </select>
        </div>
      </div>
    </form>
  </div>
</section>
```

```
<option value="00000000 (0000000000)">00000000 (0000000000)</option>
<option value="00000000 (0000000000)">00000000 (0000000000)</option>
<option value="000000">000000 000000</option>
<option value="00000000 000000">00000000 000000</option>
</select>
</div>
<div class="form-group">
<label class="form-label">00 000000 * (00 00000 00000000)</label>
<input type="text" id="leaveFromDate" class="form-input" placeholder="1405/05/22" required />
</div>
<div class="form-group">
<label class="form-label">00 000000 *</label>
<input type="text" id="leaveToDate" class="form-input" placeholder="1405/05/23" required />
</div>
<div class="form-group" id="leaveHoursGroup" style="display: none;">
<label class="form-label">0000 000000 (00 00000 - 00 00000)</label>
<input type="text" id="leaveHoursInput" class="form-input" placeholder="0000: 00 00:00 00 00:00" />
</div>
<div class="form-group full-width">
<label class="form-label">00000000 0 0000 000000 *</label>
<input type="text" id="leaveReasonInput" class="form-input" placeholder="0000 000000000 000000 00 00000000..."
required />
</div>
</div>
<div style="display: flex; justify-content: flex-end; margin-top: 1rem;">
<button type="submit" class="btn btn-primary" style="background: #0d9488;">
<span>0 000 00000000 000000</span>
</button>
</div>
</form>

<div class="table-responsive">
<table class="data-table">
<thead>
<tr>
<th>0000 000000000 00000</th>
<th>0000 000000</th>
<th>00 000000 / 00000</th>
<th>00 000000</th>
<th>0000</th>
<th>000000 00000000</th>
<th>000000 0000 00</th>
<th>00000000</th>
</tr>
</thead>
<tbody id="tableLeavesBody">
<!-- Dynamically populated -->
</tbody>
</table>
</div>
</div>
</section>

<!-- =====
11. TAB: NOTIFICATIONS (0000000000) - Menu Item 11 + Send/Reply Message
===== -->
<section id="tab-notifications" class="tab-pane">
<div class="card">
<div class="card-header">
<div class="card-title">
<span>0 000000 0 0000000 0000000000 0 000000000000 000000 (000000 000000 0 0000 00000)</span>
</div>
</div>
</div>

<!-- Send / Reply Message Box -->
<form id="formSendMessage" onsubmit="return false;" style="background: #f8fafc; padding: 1.25rem; border-radius:
10px; border: 1px solid var(--border-color); margin-bottom: 1.5rem;">
<h4 style="margin-bottom: 1rem; color: #1e293b;">0 000000 0000 0000 / 0000 00 0000 00 0000000000</h4>
<div class="form-grid">
<div class="form-group">
<label class="form-label">00000000 0000 *</label>
<select id="msgRecipientSelect" class="form-select dropdown-auto-clear" required>
<option value="0000 000000000 (000000 000000)">0000 000000000 (000000 000000)</option>
<option value="000000 000000">000000 000000 (Admin)</option>
<option value="000000 00000000">000000 00000000 (000000000 000000)</option>
<option value="000000 0000 00000000">000000 0000 000000 (0000000000 000000)</option>
<option value="000000 000000 00000000">000000 000000 00000000 (00000000)</option>
</select>
</div>
<div class="form-group">
<label class="form-label">000000 0000 / 000000 *</label>
<input type="text" id="msgTitleInput" class="form-input" placeholder="0000: 00000 000000 00 000000 000000..."
required />
</div>
<div class="form-group full-width">
```

```
<label class="form-label">□□□ □□□□ *</label>
<input type="text" id="msgBodyInput" class="form-input" placeholder="□□□ □□□□ □□□□ □□ □□□□ □□ □□ □□□□ □□
□..." required />
</div>
</div>
<div style="display: flex; justify-content: flex-end; margin-top: 1rem;">
  <button type="submit" class="btn btn-primary" style="background: #0d9488;">
<span>□ □□□□□ □□□□ □ □□□□□ (□□ □□□□ □□□□□) </span>
</button>
</div>
</form>

<div class="table-responsive">
  <table class="data-table">
    <thead>
      <tr>
        <th>□□□□□ □□□□□</th>
        <th>□□□□□ □□□□□</th>
        <th>□□□ □□□□</th>
        <th>□□□□□□□ / □□□□□□</th>
        <th>□□□□□</th>
        <th>□□□□□</th>
      </tr>
    </thead>
    <tbody id="tableNotificationsBody">
      <!-- Dynamically populated -->
    </tbody>
  </table>
</div>
</div>
</section>

<!-- =====
12. TAB: MONTHLY REPORTS (□□□□□ □□□□□) - Menu Item 12
===== -->
<section id="tab-monthly-reports" class="tab-pane">
  <div class="card">
    <div class="card-header">
      <div class="card-title">
<span>□ □□□□□ □□□□□□ □□□□□□ □□□□□□□□□ □ □□□□ □□□□□□</span>
      </div>
      <button id="btnExportMonthlyCSV" class="btn btn-success btn-sm" style="background: #0d9488;">
<span>□□ □□□□□□ □□□□□ □□□□</span>
      </button>
    </div>
    <div class="table-responsive">
      <table class="data-table">
        <thead>
          <tr>
            <th>□□□ □□□□□□□ □□□□</th>
            <th>□□□□□ □□□□□□□□□ □□□□□□</th>
            <th>□□□□□ □□□□□□□ □□</th>
            <th>□□□□□ □□□□ (□□□□)</th>
            <th>□□□□ □□□□ □□</th>
          </tr>
        </thead>
        <tbody id="tableMonthlyReportsBody">
          <!-- Dynamically populated -->
        </tbody>
      </table>
    </div>
  </div>
</section>

<!-- =====
13. TAB: SALES TARGETS (□□□□□ □□□□) - Menu Item 13 + Real-Time Calc
===== -->
<section id="tab-sales-targets" class="tab-pane">
  <div class="card">
    <div class="card-header">
      <div class="card-title">
<span>□ □□□□□ □ □□□□□ □□ □□□□□ □□□□ (KPI) □□ □□□□□ □□□□□□□ □ □□□□</span>
      </div>
    </div>
    <p style="font-size: 0.88rem; color: #64748b; margin-bottom: 1.25rem;">
      □□ □□□□ □□□□□□□ □□□: □□ □□□□ □□□□ □□□□□ □□□□ □□ □□ □□□□ □□□□ □□□□ (□□□□□ □□□□ □□□ □ □□□□ □□□□□□□□) □□ □□□
      □ □□□□□□□ □□□□□□ □□□ □ □□ □□□□□□□□ □□□□□ □□ □□□□□□□□□ □□ □□□ □□□□□□□□.
    </p>
    <form id="formSalesTarget" onsubmit="return false;" style="background: #f8fafc; padding: 1.25rem; border-radius:
10px; border: 1px solid var(--border-color); margin-bottom: 1.5rem;">
      <div class="form-grid">
        <div class="form-group">
<label class="form-label">□□□□□□□ □□□□ *</label>
          <select id="tgtRepSelect" class="form-select dropdown-auto-clear" required>
```

```

<option value="">□□□□□□ □□□□□□...</option>
</select>
</div>
<div class="form-group">
<label class="form-label">□□□□□ / □□□□ *</label>
<select id="tgtProductSelect" class="form-select dropdown-auto-clear" required>
<option value="">□□□□□□ □□□□...</option>
</select>
</div>
<div class="form-group">
<label class="form-label">□□□□□ □□□□□ (□□□) *</label>
<input type="number" id="tgtCountInput" class="form-input" placeholder="□□□□: 500" required />
</div>
<div class="form-group">
<label class="form-label">□□□ □□□</label>
<input type="number" id="tgtYearInput" class="form-input" value="1405" />
</div>
<div class="form-group">
<label class="form-label">□□□ □□□</label>
<select id="tgtMonthSelect" class="form-select dropdown-auto-clear">
<option value="□□□□□□□□">□□□□□□□□</option>
<option value="□□□□□□□□">□□□□□□□□</option>
<option value="□□□□□□">□□□□□□</option>
<option value="□□□□">□□□□</option>
<option value="□□□□□□" selected>□□□□□□</option>
<option value="□□□□□□">□□□□□□</option>
<option value="□□□□">□□□□</option>
<option value="□□□□">□□□□</option>
<option value="□□□">□□□</option>
<option value="□□">□□</option>
<option value="□□□□">□□□□</option>
<option value="□□□□□□">□□□□□□</option>
</select>
</div>
<div class="form-group full-width" style="background: #eff6ff; padding: 0.75rem 1rem; border-radius: 8px; border: 1px solid #bfdbfe; display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap;">
<div>
<span>□□□□ □□ □□ □□□□ □□□□ □□□: </span>
<strong id="tgtCalcDistPrice" style="color: #1e40af;">0 □□□□</strong>
</div>
<div>
<span>□□□□ □□ □□ □□□□ □□□□ □□□□□□□□: </span>
<strong id="tgtCalcPhPrice" style="color: #0d9488;">0 □□□□</strong>
</div>
</div>
<div style="display: flex; justify-content: flex-end; margin-top: 1rem;">
<button type="submit" class="btn btn-primary" style="background: #0d9488;">
<span>□ □□□ □ □□□□□ □□□□□ □□□□□□□□</span>
</button>
</div>
</form>

<div class="table-responsive">
<table class="data-table">
<thead>
<tr>
<th>□□□ □□□□□□□□ □□□□</th>
<th>□□□□□□ / □□□□</th>
<th>□□□□□□ □□□□□□</th>
<th>□□□□□□ □□□□□□</th>
<th>□□□□□□□□□□</th>
<th>□□□□□ □□ □□□□□□ (□□□□)</th>
<th>□□□□□ □□□□□</th>
<th>□□□□□□</th>
</tr>
</thead>
<tbody id="tableSalesTargetsBody">
<!-- Dynamically populated -->
</tbody>
</table>
</div>
</div>
</section>

<!-- =====
14. TAB: CUSTOM FIELDS / ADDITIONS (□□□□□□□□□□) - Menu Item 14
===== -->
<section id="tab-custom-fields" class="tab-pane">
<div class="card">
<div class="card-header">
<div class="card-title">
<span>□ □□□□□□□ □□□□□□ □ □□□□□□□□□□ □□□□□□□□ (Requirement 1)</span>
</div>
</div>
</div>

```

```

    <p style="font-size: 0.88rem; color: #64748b; margin-bottom: 1.25rem;">
        000000 0000000000 00 000 0000 00 0000 0000 00 00 00 000000 0000 0000. 0000 00000000 0000000 0000 0000000 000000 0000
        00 000 000 0000 0000000.
    </p>
    <form id="formCustomField" onsubmit="return false;" style="background: #f8fafc; padding: 1.25rem; border-radius:
var(--radius-lg); border: 1px solid var(--border-color); margin-bottom: 1.5rem;">
    <h4 style="margin-bottom: 1rem; color: #1e293b;">0 0000000 0000 0000</h4>
        <div class="form-grid">
            <div class="form-group">
                <label class="form-label" for="cfTargetEntity">000 0000000 (00000000) *</label>
                <select id="cfTargetEntity" class="form-select dropdown-auto-clear" required>
                    <option value="pharmacy">000000000000 (Pharmacy)</option>
                    <option value="doctor">0000000 0 0000000 (Doctor)</option>
                    <option value="order">00000000 (Order)</option>
                </select>
            </div>
            <div class="form-group">
                <label class="form-label" for="cfLabel">000000 0000 *</label>
                <input type="text" id="cfLabel" class="form-input" placeholder="0000: 00 00000000 00 000 00000000" required
/>
            </div>
            <div class="form-group">
                <label class="form-label" for="cfType">000 0000 *</label>
                <select id="cfType" class="form-select dropdown-auto-clear" required>
                    <option value="simple">0000 (0000 / 0000)</option>
                    <option value="select">000000 (00000000)</option>
                </select>
            </div>

            <div id="cfOptionsWrapper" class="form-group full-width" style="display: none;">
            <label class="form-label" for="cfOptions">00000000000 000000 (00 0000000 «0» 0000 0000)</label>
            <input type="text" id="cfOptions" class="form-input" placeholder="0000: 000000 0000 0000000000 000 000000000" />
                <div style="margin-top: 0.5rem; display: flex; align-items: center; gap: 0.5rem;">
                    <input type="checkbox" id="cfAllowAddOption" checked style="width: 18px; height: 18px;" />
                    <label for="cfAllowAddOption" style="font-size: 0.85rem; font-weight: 600; color: #0d9488;">
                        000000 0000000 00000 0000 00 00000 000000 00000 000000 00 0000 0000 0 0000 00000000000
                    </label>
                </div>
            </div>

            <div class="form-group full-width">
            <label class="form-label">000 000000 0000 0000</label>
                <div style="display: flex; gap: 1.5rem; flex-wrap: wrap; margin-top: 0.5rem;">
                    <label style="display: flex; align-items: center; gap: 0.5rem; font-weight: 600;">
                        <input type="checkbox" id="cfShowInForm" checked style="width: 18px; height: 18px;" />
                        <span>000000 00 000 000</span>
                    </label>
                    <label style="display: flex; align-items: center; gap: 0.5rem; font-weight: 600;">
                        <input type="checkbox" id="cfShowInList" checked style="width: 18px; height: 18px;" />
                        <span>000000 00 0000 0000</span>
                    </label>
                </div>
            </div>

            <div style="display: flex; justify-content: flex-end; margin-top: 1rem;">
                <button type="submit" id="btnSaveCustomField" class="btn btn-primary" style="background: #0d9488;">
            <span>0 0000000 0000 0000000</span>
            </button>
            </div>
        </form>

        <div class="table-responsive">
            <table class="data-table" id="tableCustomFields">
                <thead>
                    <tr>
                        <th>000 0000000</th>
                        <th>000000 0000</th>
                        <th>000 0000</th>
                        <th>0000000000 0000 00000</th>
                        <th>000000 00 000 000</th>
                        <th>000000 00 0000</th>
                        <th>000000</th>
                    </tr>
                </thead>
                <tbody id="tableCustomFieldsBody">
                    <!-- Dynamically populated -->
                </tbody>
            </table>
        </div>
    </div>
</section>

<!-- =====
15. TAB: COLUMNS & PRODUCTS (00000000 0 0000000) - Menu Item 15

```

```

===== -->
<section id="tab-columns-products" class="tab-pane">
  <div class="card">
    <div class="card-header">
      <div class="card-title">
        <span>□ □□□□□ □□□□□□ □□□□ □ □□□□□ □□□□□□ □□□□□□</span>
      </div>
      <button id="btnResetProductForm" class="btn btn-outline btn-sm">
        <span>□ □□□ □□□□□ □□□□</span>
      </button>
    </div>
    <form id="formProduct" onsubmit="return false;" style="margin-bottom: 1.5rem;">
      <div class="form-grid">
        <div class="form-group">
          <label class="form-label">□□□ □□□□ / □□□□□ *</label>
          <input type="text" id="productName" class="form-input" placeholder="□□□□: □□□□□ □□□□□□□□ □□ □□□□□□□□" requi
red />
        </div>
        <div class="form-group">
          <label class="form-label">□□□□ □□□□□□ *</label>
          <input type="text" id="productCategory" class="form-input" placeholder="□□□□: □□□□□ / □□□□□" required />
        </div>
        <div class="form-group">
          <label class="form-label">□□□□ □□□ (□□□□) *</label>
          <input type="number" id="productDistPrice" class="form-input" placeholder="40000" required />
        </div>
        <div class="form-group">
          <label class="form-label">□□□□ □□□□□□□□ (□□□□) *</label>
          <input type="number" id="productPrice" class="form-input" placeholder="45000" required />
        </div>
        <div class="form-group">
          <label class="form-label">□□□□□□ □□□□□□</label>
          <input type="number" id="productStock" class="form-input" placeholder="5000" />
        </div>
        <div style="display: flex; justify-content: flex-end; margin-top: 1rem;">
          <button type="submit" id="btnSaveProduct" class="btn btn-primary" style="background: #0d9488;">
            <span>□ □□□ □ □□□□□ □□□□</span>
          </button>
        </div>
      </form>
      <div class="table-responsive">
        <table class="data-table">
          <thead>
            <tr>
              <th>□□□□</th>
              <th>□□□ □□□□ / □□□□□</th>
              <th>□□□□ □□□□□□</th>
              <th>□□□□ □□□ (□□□□)</th>
              <th>□□□□ □□□□□□□□ (□□□□)</th>
              <th>□□□□□□</th>
              <th>□□□□□□</th>
            </tr>
          </thead>
          <tbody id="tableProductsBody">
            <!-- Dynamically populated -->
          </tbody>
        </table>
      </div>
    </div>
  </section>

<!-- =====
16. TAB: USERS & PERMISSIONS (□□□□□□□□ □ □□□□□□) - Matching Screenshots 4,5,6,7
===== -->
<section id="tab-users-permissions" class="tab-pane">
  <div class="card">
    <div class="card-header" style="justify-content: space-between;">
      <div style="display: flex; align-items: center; gap: 0.5rem;">
        <button class="btn btn-outline btn-sm" onclick="switchTab('tab-dashboard')">→ □□□□□□</button>
        <button class="btn btn-outline btn-sm" onclick="switchTab('tab-dashboard')">□ □□□□ □□□□</button>
      </div>
      <div class="card-title" style="font-size: 1.25rem;">
        <span>□□□□□□□□ □□□ □□□□ □ □□□ □□□□□□ □</span>
      </div>
    </div>
    <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 1.5rem; flex-wrap
: wrap; gap: 0.75rem;">
      <button id="btnExportUsersCSV" class="btn btn-success btn-sm" style="background: #0d9488;">
        <span>□□ □□□□□ □□□□</span>
      </button>
      <button id="btnToggleShowAllPasswords" class="btn btn-outline btn-sm">
        <span>□□ □□□□□ □□□ □□□□□</span>
      </button>
    </div>
  </div>

```



```
</div>

<!-- CREATE NEW USER CARD (Matching Screenshot 4) -->
<div style="background: #f8fafc; border: 1.5px solid var(--border-color); border-radius: var(--radius-lg); padding: 1.5rem; margin-bottom: 2rem;">
  <h4 style="margin-bottom: 1rem; color: #1e293b; font-size: 1.1rem;">□□□□□ □□□□□ □□□□</h4>

  <form id="formCreateUser" onsubmit="return false;">
    <div class="form-grid">
      <div class="form-group full-width">
        <label class="form-label">□□□ □ □□□ □□□□□□□□ *</label>
        <input type="text" id="newFullName" class="form-input" placeholder="□□□ □□□□□..." required />
      </div>
      <div class="form-group">
        <label class="form-label">□□□ □□□□□□ *</label>
        <input type="text" id="newUsername" class="form-input" placeholder="admin / davood..." required style="direction: rtl;" />
      </div>
      <div class="form-group">
        <label class="form-label">□□□ □□□□ *</label>
        <input type="text" id="newPassword" class="form-input" placeholder="□□□ □□□□ □□□□□ □□□□..." required style="direction: rtl;" />
      </div>
      <div class="form-group">
        <label class="form-label">□□□□□ □□□□□ □□□□</label>
        <input type="text" id="newPhone" class="form-input" placeholder="09120000000" style="direction: rtl;" />
      </div>
      <div class="form-group">
        <label class="form-label">□□□□</label>
        <select id="newRole" class="form-select dropdown-auto-clear">
          <option value="□□□□□□□ □□□□">□□□□□□□□ □□□□</option>
          <option value="□□□□ □□□□□">□□□□ □□□□□</option>
          <option value="□□□□□□">□□□□□□</option>
        </select>
      </div>
      <div class="form-group">
        <label class="form-label">□□□□□ □□□□□□□□</label>
        <select id="newSimControl" class="form-select dropdown-auto-clear">
          <option value="□□□□ □□□□□">□□□□ □□□□□</option>
          <option value="□□□ □□□□ (□□□□□ □□□)">□□□ □□□□ (□□□□□ □□□)</option>
        </select>
      </div>
    </div>

    <!-- Permission filter pill bar matching Screenshot 4 -->
    <div style="margin-top: 1.5rem; background: #fff; padding: 1rem; border-radius: 10px; border: 1px solid var(--border-color);">
      <div style="display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap; gap: 0.75rem; margin-bottom: 1rem; border-bottom: 1px solid var(--border-color); padding-bottom: 0.75rem;">
        <span style="font-weight: 700; color: #1e293b;">□□□ □□□□□□ □□□□ □□□□</span>
        <div style="display: flex; gap: 0.5rem; align-items: center;">
          <button type="button" id="btnPermSelectAll" class="btn btn-primary btn-sm" style="background: #0d9488;">□□□</button>
          <button type="button" id="btnPermSelectNone" class="btn btn-outline btn-sm">□□□</button>
          <span class="badge-status-online" id="selectedPermCountBadge">□□ □□□□</span>
        </div>
      </div>

      <!-- Permission groups container matching Screenshot 5, 6, 7 -->
      <div id="permissionGroupsContainer" style="display: flex; flex-direction: column; gap: 1.25rem;"></div>

      <div style="display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap; margin-top: 1.5rem; gap: 1rem;">
        <label style="display: flex; align-items: center; gap: 0.5rem; font-weight: 700; color: #0f172a; cursor: pointer;">
          <input type="checkbox" id="newForcePhoneLogin" checked style="width: 20px; height: 20px; accent-color: #0d9488;" />
          <span>□□□□ □□ □□□□ □□□□ □□□□□ □□□□</span>
        </label>
        <button type="submit" class="btn btn-primary" style="background: #0d9488; font-size: 1rem; padding: 0.75rem 2rem;">
          <span>□ □□□□□ □□□□□</span>
        </button>
      </div>
    </form>
  </div>

  <!-- USERS LIST CARDS matching Screenshot 4, 7 (□□□□ □□□□□□ □□□□ □□□□□□ □□□□ □□□□ □□□□□ □□□□ □□□□□ □□□□□) -->
  <h4 style="margin-bottom: 1rem; color: #1e293b;">□□□□□ □□□□□□□ □ □□ □□□□□ □□□□</h4>
  <div id="userCardsContainer" style="display: flex; flex-direction: column; gap: 1.25rem;"></div>
</div>
</section>

<!-- =====>
```

17. TAB: MESSENGERS (○○○○○○○○○○○○) - Menu Item 17

```
===== -->
<section id="tab-messengers" class="tab-pane">
  <div class="card">
    <div class="card-header">
      <div class="card-title">
        <span>□ ○○○○○○○ ○○○○ ○○ ○○○○○○○○○○○○ (○○○○○○ ○○○○○○○ ○○○○ ○ ○○○○○○)</span>
      </div>
    </div>
    <div class="form-grid">
      <div class="form-group full-width">
        <label class="form-label">○○○○ ○○○○ ○○○○○○ / ○○○○</label>
        <input type="text" class="form-input" value="123456:ABC-DEF1234ghIkl-zyx57W2v1u123ew11" style="direction: ltr;" />
      </div>
      <div class="form-group">
        <label class="form-label">○○○○○ ○○○○○○○ ○○○○○○ / ○○○○○○</label>
        <input type="text" class="form-input" value="09120000000" style="direction: ltr;" />
      </div>
      <div class="form-group">
        <label class="form-label">○○○○○ ○○○○ ○○○○○○○○</label>
        <input type="text" class="form-input" value="@namayande_channel" style="direction: ltr;" />
      </div>
    </div>
  </div>
</section>
```

18. TAB: BACKUP & RESTORE (○○○○○○○○○○○○) - Menu Item 18 (Requirement 8)

```
===== -->
<section id="tab-backup" class="tab-pane">
  <div class="card">
    <div class="card-header">
      <div class="card-title">
        <span>□ ○○○○○○○○○○○○○ ○○○○○○○ ○○○○ ○○○○○○ (crm-backup-latest.json) □ ○○○ ○○○○○○</span>
      </div>
    </div>
    <p style="font-size: 0.88rem; color: #64748b; margin-bottom: 1rem;">
      ○○ ○○○○ ○○○○○○○ ○○○: ○○ ○○○○ ○○○○○○○ ○○○○ ○○○○○○○ ○○○○ ○○○○○○○ (○○○ ○○○○ ○○○○: <code>crm-backup-latest.json
</code>)□ □ ○○○○ ○○○○○○○ ○○○○○○○ (Restore Data) ○○○ ○○ ○○○○○○ ○○○○ ○○○○ ○○○○.
    </p>
    <div style="background: #f8fafc; padding: 1.25rem; border-radius: var(--radius-lg); border: 1px solid var(--border-color); margin-bottom: 1.5rem;">
      <div style="display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap; gap: 1rem; margin-bottom: 1rem;">
        <div>
          <h4 style="margin-bottom: 0.25rem; color: #1e293b;">□ ○○○○○○○○○○○○○ ○○○○○○○ ○○ ○○○○ ○○○○ (File System Access)
        </h4>
        <p style="font-size: 0.85rem; color: var(--text-muted); margin: 0;">
          ○○ ○○○○○○ ○○○○ ○○○○○○○○○○○○○ ○○○○○○○ ○○ ○ ○○○○○ (○○ ○○○ ○○ ○○ ○○○○○○ ○○○○) ○○ ○○○○ ○○○○○○○ ○○○○○○○ ○○ ○○○○○○○
          ○○○○ <code>crm-backup-latest.json</code> ○○○○○○.
        </p>
      </div>
      <div style="display: flex; gap: 0.75rem; flex-wrap: wrap;">
        <button id="btnSelectAutoBackupFolder" class="btn btn-primary" style="background: #0d9488;">
          <span>□ ○○○○○○ ○○○○ ○○○○○○○○○○○○○○ ○○○○○○ (crm-backup-latest.json)</span>
        </button>
        <button id="btnManualBackupNow" class="btn btn-success" style="background: #0d9488;">
          <span>□ ○○○○○○○○○○○○○○ ○○○○ ○○○○ (○○○○○○○○○ ○○○○ ○○○○)</span>
        </button>
      </div>
    </div>
    <div style="display: flex; align-items: center; gap: 0.75rem; background: #ffffff; padding: 0.85rem; border-radius: 8px; border: 1px solid var(--border-color);">
      <input type="checkbox" id="chkAutoBackupEnabled" checked="" style="width: 20px; height: 20px; accent-color: #0d9488;" />
      <label for="chkAutoBackupEnabled" style="font-weight: 700; color: #0f172a; cursor: pointer;">
        ○○○○○○○○○○ ○○○○○○○○○○○○○○ ○○○○○○ ○○ ○ ○○○○○○ ○ ○○ ○○ ○○ ○○○○○○ ○○ ○○○○○○○○ ○○○○○○
      </label>
      <span id="autoBackupHandleStatus" style="margin-right: auto; font-size: 0.82rem; background: #eff6ff; color: #2563eb; padding: 0.3rem 0.75rem; border-radius: 999px; font-weight: 700;">
        ○○○○○○ ○○○○○○ ○○ ○○○○○○ ○○○○ + ○○○○○○○○ ○○○○○○○○○
      </span>
    </div>
  </div>
  <div class="card" style="border: 2px solid #0d9488; background: #f0fdf4; margin-bottom: 0;">
    <div class="card-header">
      <div class="card-title" style="color: #166534;">
        <span>□ ○○○○ ○○○○○○○○ ○○○○○○○○ (Restore Data) - ○○○○○○○○ ○○ ○○○○ ○○○○○○○○</span>
      </div>
    </div>
    <p style="font-size: 0.88rem; color: #475569; margin-bottom: 1.25rem;">
```



```

===== -->
<div id="diagnosticOfflineBanner" class="offline-diagnostic-banner" style="display: none;">
  <div class="offline-banner-content">
    <div class="offline-icon"></div>
    <div class="offline-text">
      <h4>⚠️ 网络异常 ⚠️ 请检查您的网络连接 ⚠️</h4>
      <p>⚠️ 网络异常 ⚠️ 请检查您的网络连接 ⚠️. 请尝试刷新页面或检查您的网络设置。⚠️ 网络异常 ⚠️ 请检查您的网络连接 ⚠️</p>
    </div>
    <div class="offline-actions">
      <button type="button" class="btn-banner-retry" onclick="retryServerConnection()">⏮️ 重试 ⏮️</button>
      <button type="button" class="btn-banner-diag" onclick="switchTab('tab-troubleshooting'); document.getElementById('diagnosticOfflineBanner').style.display='none';">🔧 故障排除 🔧</button>
      <button type="button" class="btn-banner-close" onclick="document.getElementById('diagnosticOfflineBanner').style.display='none';">✖️ 关闭 ✖️</button>
    </div>
  </div>

<!-- =====
MODAL: ADD NEW OPTION TO SELECT FIELD (⚙️ 添加新选项 ⚙️)
===== -->
<div id="modalAddOption" class="modal-over lay">
  <div class="modal-content">
    <div class="card-header">
      <div class="card-title">
        <span>⚙️ 添加新选项 ⚙️</span>
      </div>
    </div>
    <form id="formAddOptionModal" onsubmit="return false;">
      <input type="hidden" id="modalAddOptionFieldId" value="" />
      <input type="hidden" id="modalAddOptionEntityType" value="" />
      <div class="form-group">
        <label class="form-label" id="modalAddOptionLabel">⚙️ 添加新选项 ⚙️</label>
        <input type="text" id="modalAddOptionValue" class="form-input" placeholder="⚙️ 添加新选项 ⚙️ 请在此处输入选项名称..." required />
      </div>
      <div style="display: flex; justify-content: flex-end; gap: 0.75rem; margin-top: 1.25rem;">
        <button type="button" class="btn btn-outline" onclick="closeModalAddOption()">⏮️ 取消 ⏮️</button>
        <button type="submit" class="btn btn-primary" style="background: #0d9488;">✅ 添加 ⏮️</button>
      </div>
    </form>
  </div>
</div>

<!-- =====
21. TAB: SEARCH INFORMATION (🔍 搜索信息 🔍) - Section 6 Requirement
===== -->
<section id="tab-search-info" class="tab-pane">
  <div class="card">
    <div class="card-header">
      <div class="card-title">
        <span>🔍 搜索信息 🔍</span>
      </div>
    </div>
    <div class="form-grid" style="margin-bottom: 1.25rem;">
      <div class="form-group">
        <label class="form-label">🔍 搜索 🔍</label>
        <input type="text" id="searchInfoPhnInput" class="form-input" placeholder="🔍 搜索 🔍 请输入搜索关键词..." />
      </div>
      <div class="form-group">
        <label class="form-label">🔍 搜索 🔍</label>
        <input type="text" id="searchInfoDocInput" class="form-input" placeholder="🔍 搜索 🔍 请输入搜索关键词..." />
      </div>
      <div class="form-group">
        <label class="form-label">🔍 搜索 🔍 / 🔍 搜索 🔍</label>
        <input type="text" id="searchInfoHospInput" class="form-input" placeholder="🔍 搜索 🔍 请输入搜索关键词..." />
      </div>
      <div style="display: flex; gap: 0.75rem; margin-bottom: 1.5rem;">
        <button type="button" id="btnRunSearchInfo" class="btn btn-primary" style="background: #0d9488;">🔍 搜索 🔍</button>
        <button type="button" id="btnExportSearchInfoCSV" class="btn btn-success" style="background: #0d9488;">📄 导出 🔍</button>
      </div>
    </div>
    <div id="map-search-info" class="map-container" style="height: 380px; margin-bottom: 1.5rem;"></div>
    <div class="table-responsive">
      <table class="data-table">
        <thead>
          <tr>
            <th>🔍 搜索 🔍</th>
            <th>🔍 搜索 🔍</th>
            <th>🔍 搜索 🔍</th>
          </tr>
        </thead>
        <tbody>
          <tr>
            <td>🔍 搜索 🔍</td>
            <td>🔍 搜索 🔍</td>
            <td>🔍 搜索 🔍</td>
          </tr>
        </tbody>
      </table>
    </div>
  </div>

```

```
<th>□□□ □□□□ / □□□□</th>
<th>□□□ / □□□□□</th>
<th>□□□□</th>
<th>□□□□□□</th>
</tr>
</thead>
<tbody id="tableSearchInfoBody">
  <!-- Dynamically populated -->
</tbody>
</table>
</div>
</div>
</section>

<!-- =====
MODAL: ROW DETAILS & MESSAGING (□□□□□ □□□□□□ □□□□ □ □□□□□ □□ □□□□□ □□□□□□ □□□□□□ □□□□□□)
===== -->
<div id="modalRowDetails" class="modal-overlay">
  <div class="modal-content" style="max-width: 600px;">
    <div class="card-header">
      <div class="card-title">
        <span>□ □□□□□□ □□□□ □□□□□ □ □□□□□ □□□□□□□□□□□□</span>
      </div>
    </div>
    <div id="rowDetailsContentBox" style="background: #f8fafc; padding: 1rem; border-radius: 10px; border: 1px solid v
ar(--border-color); margin-bottom: 1.25rem; font-size: 0.9rem; line-height: 1.8;">
      <!-- Dynamic row details text -->
    </div>
    <div id="rowDetailsMiniMap" class="map-container" style="height: 200px; margin-bottom: 1.25rem;"></div>

    <h5 style="margin-bottom: 0.6rem; color: #1e293b;">□ □□□□□ □□□□□□□□ □□□□□ □□ □□□□□□□□□□□□ (□□ □□□□ □□□□□□)</h5>
    <div style="display: flex; gap: 0.5rem; flex-wrap: wrap; margin-bottom: 1.25rem;">
      <button type="button" id="btnShareBale" class="btn btn-outline btn-sm" style="background: #f0fdf4; color: #16653
4; border-color: #bbf7d0;">
        <span>□ □□□ (Bale)</span>
      </button>
      <button type="button" id="btnShareEitaa" class="btn btn-outline btn-sm" style="background: #fff7ed; color: #9a34
12; border-color: #ffedd5;">
        <span>□ □□□□ (Eitaa)</span>
      </button>
      <button type="button" id="btnShareTelegram" class="btn btn-outline btn-sm" style="background: #eff6ff; color: #1
e40af; border-color: #bfbdfc;">
        <span>□ □□□□□□ (Telegram)</span>
      </button>
      <button type="button" id="btnShareSoroush" class="btn btn-outline btn-sm" style="background: #f5f3ff; color: #6d
28d9; border-color: #ddd6fe;">
        <span>□ □□□□ (Soroush)</span>
      </button>
      <button type="button" id="btnShareWhatsApp" class="btn btn-outline btn-sm" style="background: #ecfdf5; color: #0
47857; border-color: #a7f3d0;">
        <span>□ □□□□□□ (WhatsApp)</span>
      </button>
    </div>

    <div style="display: flex; justify-content: space-between; align-items: center; flex-wrap: wrap; gap: 0.75rem;">
      <div style="display: flex; gap: 0.5rem;">
        <button type="button" id="btnRowCopyText" class="btn btn-primary btn-sm" style="background: #0d9488;">
          <span>□ □□□ □□□□ □□□ □□□□□□□□</span>
        </button>
      </div>
      <div style="display: flex; gap: 0.5rem;">
        <button type="button" id="btnRowEdit" class="btn btn-outline btn-sm">□□ □□□□□□</button>
        <button type="button" id="btnRowDelete" class="btn btn-danger btn-sm">□□ □□□</button>
        <button type="button" class="btn btn-outline btn-sm" onclick="closeModalRowDetails()">□ □□□□</button>
      </div>
    </div>
  </div>
</div>
<div id="modalNavigationApps" class="modal-overlay">
  <div class="modal-content" style="max-width: 440px;">
    <div class="card-header">
      <div class="card-title">
        <span>□ □□□□□□□ □□□□□□□ □□□□ □□□□□ □□ □□□□</span>
      </div>
    </div>
    <p style="font-size: 0.88rem; color: #64748b; margin-bottom: 1.25rem;">
      □□□□□□□ □□□□ □□□ □□□ □□ □□□□□□ □□□□ □□ □□□□□□□□ □□□□ □□□ □□□ □□□ □□□ □□□ □□□ □□□
    </p>
    <div style="display: flex; flex-direction: column; gap: 0.75rem;">
      <button type="button" id="btnNavNeshan" class="btn btn-outline" style="justify-content: flex-start; font-size: 0
.95rem; padding: 0.75rem 1rem;">
        <span style="font-size: 1.3rem;">□</span></button>
      <span>□□□□□□□ □□□□ (Neshan)</span>
    </button>
    <button type="button" id="btnNavBalad" class="btn btn-outline" style="justify-content: flex-start; font-size: 0.
```

```
95rem; padding: 0.75rem 1rem; ">
    <span style="font-size: 1.3rem;">□</span>
    <span>□□□□□□ □□ (Balad)</span>
  </button>
  <button type="button" id="btnNavGoogle" class="btn btn-outline" style="justify-content: flex-start; font-size: 0
.95rem; padding: 0.75rem 1rem;">
    <span style="font-size: 1.3rem;">□</span>
    <span>□□□□ □□ (Google Maps)</span>
  </button>
  <button type="button" id="btnNavWaze" class="btn btn-outline" style="justify-content: flex-start; font-size: 0.9
5rem; padding: 0.75rem 1rem;">
    <span style="font-size: 1.3rem;">□</span>
    <span>□□□ (Waze)</span>
  </button>
</div>
<div style="display: flex; justify-content: flex-end; margin-top: 1.25rem;">
<button type="button" class="btn btn-outline" onclick="closeModalNavigationApps()">□□□□</button>
</div>
</div>
</div>
```

```
<!-- =====
MODAL: LOGIN & ROLE SWITCHING (□□□□ □ □□□□□ □□ □□□□□□)
===== -->
<div id="modalLogin" class="modal-overlay">
  <div class="modal-content" style="max-width: 480px;">
    <div style="text-align: center; margin-bottom: 1rem;">
      <div style="width: 64px; height: 64px; background: linear-gradient(135deg, #0d9488 0%, #1e3a8a 100%); color: #ff
f; font-size: 2.2rem; border-radius: 50%; display: inline-flex; align-items: center; justify-content: center; box-shadow
: 0 4px 12px rgba(13, 148, 136, 0.4);">
        □
      </div>
    </div>
    <div class="card-header">
      <div class="card-title">
        <span>□ □□□□ □ □□□□□ / □□□□□ □□□ □□□□□□</span>
      </div>
    </div>
    <p style="font-size: 0.88rem; color: #64748b; margin-bottom: 1.25rem;">
      □□□ □□□□□□ □□□ □□ □□□□□□ □□□□ □□ □□□□□□□□□□ □□□□□□□□□□ □□ □□□ «□ □□□□□□□ □ □□□□□□» □□ □□□□ □□□□□□ □□□□ □□ □□□□□□ □
□□:
    </p>
    <form id="formLoginModal" onsubmit="return false;">
      <div class="form-group">
        <label class="form-label" for="loginRoleSelect">□□□ □□□□□□ *</label>
        <select id="loginRoleSelect" class="form-select dropdown-auto-clear" required>
          <option value="0">□□□□ □□□□□ (Admin)</option>
          <option value="1">□□□□ □□□□□□ (□□□□□□□□ □□□□)</option>
          <option value="2">□□□□ □□□□ □□□□□ (□□□□□□□□ □□□□)</option>
          <option value="3">□□□□ □□□□□ □□□□□□ (□□□□□□)</option>
        </select>
      </div>
      <div class="form-group" style="margin-top: 0.5rem;">
        <label class="form-label" for="loginUserName">□□□□□ □□□□□ □□□□</label>
        <input type="text" id="loginUserName" class="form-input" value="091200000000" style="direction: ltr;" />
      </div>
      <div class="form-group" style="margin-top: 0.5rem;">
        <label class="form-label" for="loginSimAuthMode">□□□□ □□□□□ □□ □□□□□□□□ □□□□□□□□ (□□□□□ □□□□)</label>
        <select id="loginSimAuthMode" class="form-select dropdown-auto-clear">
          <option value="3">□. □□□□ □□□□ (□□□□□□□□ □□□□ □□□□ □□ □□□□□□□□)</option>
          <option value="1">□. □□□□□□ □□□□□ □□□ □□□□□□□□□□ (OTP SMS)</option>
          <option value="2">□. □□□□□ □□ □□□□□ □□ □□□□ □□□□□</option>
        </select>
      </div>
      <div style="display: flex; justify-content: flex-end; gap: 0.75rem; margin-top: 1.5rem;">
        <button type="button" class="btn btn-outline" onclick="closeModalLogin()">□□□□□□</button>
        <button type="submit" class="btn btn-primary" style="background: #0d9488;">
          <span>□ □□□□ □ □□□□□ □□□ □□□□□□</span>
        </button>
      </div>
    </form>
  </div>
</div>
</div>
```

```
<!-- =====
PERSIAN JALALI CALENDAR POPUP PICKER (□□□□□ □□□□□□ □□□□ □□□□□□ □□ □□□□□□□□□□)
===== -->
<div id="jalaliCalendarPopup" class="jalali-popup-picker">
  <div class="jalali-picker-header">
    <button type="button" class="jalali-nav-arrow" id="jalaliPrevMonth">&gt;</button>
    <div class="jalali-header-selects">
      <select id="jalaliYearSelect" class="jalali-select">
        <option value="1404">□□□□</option>
        <option value="1405" selected>□□□□</option>
        <option value="1406">□□□□</option>
      </select>
    </div>
  </div>
</div>
```

```

        </select>
        <select id="jalaliMonthSelect" class="jalali-select">
        <option value="01">□□□□□□□□</option>
        <option value="02">□□□□□□□□</option>
        <option value="03">□□□□□□</option>
        <option value="04">□□□□</option>
        <option value="05" selected>□□□□□□</option>
        <option value="06">□□□□□□□□</option>
        <option value="07">□□□□</option>
        <option value="08">□□□□</option>
        <option value="09">□□□□</option>
        <option value="10">□□□</option>
        <option value="11">□□□□</option>
        <option value="12">□□□□□□</option>
        </select>
    </div>
    <button type="button" class="jalali-nav-arrow" id="jalaliNextMonth">&lt;</button>
</div>
<div class="jalali-weekdays">
<div>□</div><div>□</div><div>□</div><div>□</div><div>□</div><div>□</div><div>□</div>
</div>
<div id="jalaliDaysGrid" class="jalali-days-grid"></div>
<button type="button" id="jalaliTodayBtn" class="jalali-today-btn">□□□□□□: □□□□/□□/□□</button>
</div>

<!-- Leaflet JS -->
<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>

<!-- Data & Application Logic with Anti-Cache Version 8.0.0 -->
<script src="crm-data.js?v=8.0.0"></script>
<script src="crm-app.js?v=8.0.0"></script>
</body>
</html>

```

```

@@ next-env.d.ts typescript
/// <reference types="next" />
/// <reference types="next/image-types/global" />
import "../next/dev/types/routes.d.ts";
// NOTE: This file should not be edited
// see https://nextjs.org/docs/app/api-reference/config/typescript for more information.

```

```

@@ next.config.ts typescript
import type { NextConfig } from "next";
/**
 * :
 * - HTML → (no-store)
 * - /_next/static → ( build )
 * - sw.js →

 */
const NO_STORE = "no-cache, no-store, must-revalidate, max-age=0";
const nextConfig: NextConfig = {
  poweredByHeader: false,
  compress: true,
  reactStrictMode: true,
  experimental: {
    optimizePackageImports: ["leaflet"],
  },
  productionBrowserSourceMaps: false,
  async headers() {
    return [
      {
        // HTML:
        // ( /_next/* )
        source: "/*:path((?!_next/).*)",
        headers: [
          { key: "Cache-Control", value: NO_STORE },
          { key: "Pragma", value: "no-cache" },
          { key: "Expires", value: "0" },
          { key: "X-Content-Type-Options", value: "nosniff" },
        ],
      },
      // : /_next/static Next.js
      // immutable .
      {
        source: "/sw.js",
        headers: [
          { key: "Content-Type", value: "application/javascript; charset=utf-8" },
          { key: "Cache-Control", value: NO_STORE },
          { key: "Service-Worker-Allowed", value: "/" },
        ],
      },
      {
        source: "/manifest.webmanifest",
        headers: [

```

```

    { key: "Content-Type", value: "application/manifest+json; charset=utf-8" },
    { key: "Cache-Control", value: "public, max-age=3600" },
    { key: "Access-Control-Allow-Origin", value: "*" },
  ],
},
{
  source: "/logo.svg",
  headers: [
    { key: "Cache-Control", value: "public, max-age=604800" },
    { key: "Access-Control-Allow-Origin", value: "*" },
  ],
},
{
  source: "/apple-touch-icon.png",
  headers: [
    { key: "Cache-Control", value: "public, max-age=31536000, immutable" },
    { key: "Access-Control-Allow-Origin", value: "*" },
  ],
},
{
  source: "/icons/:path*",
  headers: [
    { key: "Cache-Control", value: "public, max-age=31536000, immutable" },
    { key: "Access-Control-Allow-Origin", value: "*" },
  ],
},
{
  source: "/screenshots/:path*",
  headers: [
    { key: "Cache-Control", value: "public, max-age=604800" },
    { key: "Access-Control-Allow-Origin", value: "*" },
  ],
},
{
  source: "/data/:path*",
  headers: [
    { key: "Cache-Control", value: "public, max-age=2592000, stale-while-revalidate=31536000" },
    { key: "Access-Control-Allow-Origin", value: "*" },
  ],
},
{
  // API
  source: "/api/:path*",
  headers: [{ key: "Cache-Control", value: NO_STORE }],
},
];
},
};

```

```

};
export default nextConfig;

```

```

@@ server.js javascript
// =====
// 0000 0000 Node.js 0000 00000000 0000 000 Render.com 0 00000 00 Neon / GitHub
// 0000 000 0000 Deprecation (00000000 00 WHATWG URL API)
// 00000000 0000 00 Health Check 0000 (/api/health)0 0000 /login 0 000000000000 0000
// =====

```

```

const http = require('http');
const fs = require('fs');
const path = require('path');

const PORT = process.env.PORT || 10000;
const SERVER_DATA_PATH = path.join(__dirname, 'server-db.json');

```

```

const MIME_TYPES = {
  '.html': 'text/html; charset=utf-8',
  '.css': 'text/css; charset=utf-8',
  '.js': 'application/javascript; charset=utf-8',
  '.mjs': 'application/javascript; charset=utf-8',
  '.json': 'application/json; charset=utf-8',
  '.png': 'image/png',
  '.jpg': 'image/jpeg',
  '.svg': 'image/svg+xml',
  '.ico': 'image/x-icon',
  '.csv': 'text/csv; charset=utf-8',
  '.geojson': 'application/json; charset=utf-8',
  '.webmanifest': 'application/manifest+json; charset=utf-8'
};

```

```

const server = http.createServer((req, res) => {
  // 00000000 00 0000000000 WHATWG URL API 000 000 000000 Deprecation Node.js 26
  const host = req.headers.host || `localhost:${PORT}`;
  const parsedUrl = new URL(req.url, `http://${host}`);
  const pathname = parsedUrl.pathname;

```



```

// CORS
res.setHeader('Access-Control-Allow-Origin', '*');
res.setHeader('Access-Control-Allow-Methods', 'GET, POST, OPTIONS, PUT, DELETE');
res.setHeader('Access-Control-Allow-Headers', 'Content-Type, Authorization');

if (req.method === 'OPTIONS') {
  res.writeHead(204);
  return res.end();
}

// 1. Health Check Render.com Keep-Alive (/ping/api/health/api/ping)
if (pathname === '/ping' || pathname === '/api/health' || pathname === '/api/ping' || pathname === '/healthz') {
  res.writeHead(200, { 'Content-Type': 'application/json; charset=utf-8' });
  return res.end(JSON.stringify({
    ok: true,
    status: 'healthy',
    message: 'OK',
    service: 'namayandeelmi-javad-crm',
    timestamp: new Date().toISOString()
  }));
}

// 2. GET /api/state
if (pathname === '/api/state' && req.method === 'GET') {
  res.writeHead(200, { 'Content-Type': 'application/json; charset=utf-8' });
  if (fs.existsSync(SERVER_DATA_PATH)) {
    const data = fs.readFileSync(SERVER_DATA_PATH, 'utf8');
    return res.end(JSON.stringify({ status: 'success', data: JSON.parse(data) }));
  } else {
    return res.end(JSON.stringify({ status: 'empty', message: '데이터가 없습니다.' }));
  }
}

// 3. POST /api/state
if (pathname === '/api/state' && req.method === 'POST') {
  let body = '';
  req.on('data', chunk => { body += chunk; });
  req.on('end', () => {
    try {
      const data = JSON.parse(body);
      fs.writeFileSync(SERVER_DATA_PATH, JSON.stringify(data, null, 2), 'utf8');
      res.writeHead(200, { 'Content-Type': 'application/json; charset=utf-8' });
      res.end(JSON.stringify({ status: 'success', message: '데이터가 저장되었습니다.' }));
    } catch (err) {
      res.writeHead(400, { 'Content-Type': 'application/json; charset=utf-8' });
      res.end(JSON.stringify({ status: 'error', message: err.message }));
    }
  });
  return;
}

// 4. GET /api/backup
if (pathname === '/api/backup' && req.method === 'GET') {
  if (fs.existsSync(SERVER_DATA_PATH)) {
    res.writeHead(200, {
      'Content-Disposition': 'attachment; filename="crm-backup-latest.json"',
      'Content-Type': 'application/json; charset=utf-8'
    });
    const readStream = fs.createReadStream(SERVER_DATA_PATH);
    return readStream.pipe(res);
  } else {
    res.writeHead(404, { 'Content-Type': 'application/json; charset=utf-8' });
    return res.end(JSON.stringify({ status: 'error', message: '데이터가 없습니다.' }));
  }
}

// 5. /login/panel/admin / (namayandeelmi-javad.onrender.com/login)
if (pathname === '/login' || pathname === '/login/' || pathname === '/panel' || pathname === '/admin' || pathname === '/') {
  const indexPath = path.join(__dirname, 'public', 'index.html');
  const rootIndexPath = path.join(__dirname, 'index.html');
  const targetPath = fs.existsSync(indexPath) ? indexPath : rootIndexPath;

  fs.readFile(targetPath, 'utf8', (err, content) => {
    if (err) {
      res.writeHead(500);
      return res.end('Error loading index.html');
    }
    res.setHeader('Cache-Control', 'no-cache, no-store, must-revalidate');
    res.setHeader('Pragma', 'no-cache');
    res.setHeader('Expires', '0');
    res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
    res.end(content);
  });
}

```

```

    });
    return;
}

// 6. 0000000000 000000000 00000000 0000 public 0 0000 0000
let relPath = pathname.startsWith('/') ? pathname.slice(1) : pathname;
if (!relPath) relPath = 'index.html';

const publicPath = path.join(__dirname, 'public', relPath);
const rootPath = path.join(__dirname, relPath);
const filePath = fs.existsSync(publicPath) ? publicPath : rootPath;

const ext = path.extname(filePath).toLowerCase();
const contentType = MIME_TYPES[ext] || 'application/octet-stream';

fs.readFile(filePath, (err, content) => {
  if (err) {
    // 0000 SPA Fallback (00000 00 index.html)
    const fallbackPath = fs.existsSync(path.join(__dirname, 'public', 'index.html'))
      ? path.join(__dirname, 'public', 'index.html')
      : path.join(__dirname, 'index.html');

    fs.readFile(fallbackPath, (err2, indexContent) => {
      if (err2) {
        res.writeHead(404);
        return res.end('File not found');
      }
      res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
      res.end(indexContent);
    });
  } else {
    res.setHeader('Cache-Control', 'no-cache, no-store, must-revalidate');
    res.setHeader('Pragma', 'no-cache');
    res.setHeader('Expires', '0');
    res.writeHead(200, { 'Content-Type': contentType });
    res.end(content);
  }
});
});

server.listen(PORT, '0.0.0.0', () => {
  console.log(`=====`);
  console.log(`0 0000 0000 Node.js 00 0000000 0000 0.0.0.0:${PORT} 0000 00!`);
  console.log(`0 0000000 00 Health Check 0000 Render.com (/api/health)`);
  console.log(`0 000000 0000 namayandeelmi-javad.onrender.com 0 ndcohub.ir`);
  console.log(`=====`);
});

```

@@ public/cloudflare-worker.js javascript

```

/**
 * =====
 * « »
 * Cloudflare Worker — ( )
 * -----
 * :
 * ) chat_id →
 *
 * ) .
 * ) (WhatsApp Cloud API).
 * ) JSON + CORS + (x-proxy-secret).
 * ) (tapi api).
 * =====
 */

const CONFIG = {
  // ( )
  TELEGRAM_BOT_TOKEN: "",
  TELEGRAM_CHAT_ID: "",
  BALE_BOT_TOKEN: "",
  BALE_CHAT_ID: "",
  ITA_BOT_TOKEN: "",
  ITA_CHAT_ID: "",
  WHATSAPP_TOKEN: "", // phoneNumberId:accessToken
  WHATSAPP_TO: "",
  // x-proxy-secret
  PROXY_SECRET: "",
};

const CORS = {
  "Access-Control-Allow-Origin": "*",
  "Access-Control-Allow-Methods": "POST, OPTIONS",
  "Access-Control-Allow-Headers": "Content-Type, x-proxy-secret",
};

const json = (obj, status = 200) =>
  new Response(JSON.stringify(obj), {
    status,
    headers: { "Content-Type": "application/json; charset=utf-8", ...CORS },
  });

```

```

});
export default {
  async fetch(request) {
    if (request.method === "OPTIONS") return new Response(null, { headers: CORS });
    if (request.method === "GET") {
      return json({ ok: true, service: "sabt-etelaat-kol proxy", messengers: ["telegram", "bale", "ita", "whatsapp"] });
    }
    if (request.method !== "POST") return json({ success: false, error: " POST " }, 405);
    if (CONFIG.PROXY_SECRET && request.headers.get("x-proxy-secret") !== CONFIG.PROXY_SECRET) {
      return json({ success: false, error: " " }, 401);
    }
    let body;
    try {
      body = await request.json();
    } catch {
      return json({ success: false, error: " JSON " }, 400);
    }
    // getUpdates chat_id
    if (body.action === "getUpdates") {
      const mg = String(body.messenger || body.platform || "").toLowerCase();
      const tk = body.token || (mg === "telegram" ? CONFIG.TELEGRAM_BOT_TOKEN : CONFIG.BALE_BOT_TOKEN);
      if (!tk) return json({ ok: false, error: " " }, 400);
      const bases = mg === "telegram" ? ["https://api.telegram.org"] : ["https://tapi.bale.ai", "https://api.bale.ai"];
      for (const base of bases) {
        try {
          const r = await fetch(`${base}/bot${tk}/getUpdates?limit=100`);
          const raw = await r.text();
          if (r.ok) return new Response(raw, { headers: { "Content-Type": "application/json", ...CORS } });
        } catch {
          /* next host */
        }
      }
      return json({ ok: false, error: " " }, 502);
    }
    const messenger = String(body.messenger || body.platform || "").toLowerCase();
    const text = body.text;

    const chatId = body.chatId || body.chat_id;
    const token = body.token;
    if (!messenger) return json({ success: false, error: ' "messenger" ' }, 400);
    if (!text) return json({ success: false, error: ' "text" ' }, 400);
    try {
      let result;
      switch (messenger) {
        case "telegram":
          result = await sendTelegram(text, chatId, token);
          break;
        case "bale":
          result = await sendBale(text, chatId, token);
          break;
        case "ita":
        case "eita":
          result = await sendEitaa(text, chatId, token);
          break;
        case "whatsapp":
          result = await sendWhatsapp(text, chatId, token, body.provider);
          break;
        default:
          return json({ success: false, error: " " }, 400);
      }
      return json({ success: true, ok: true, messenger, data: result });
    } catch (error) {
      return json({ success: false, ok: false, messenger, error: String(error.message || error) }, 502);
    }
  },
};

async function postJson(url, payload) {
  const res = await fetch(url, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(payload),
  });
  const raw = await res.text();
  let data;
  try {
    data = JSON.parse(raw);
  } catch {
    data = { raw };
  }
  return { res, data, raw };
}

async function sendTelegram(text, chatId, token) {
  const tk = token || CONFIG.TELEGRAM_BOT_TOKEN;
  const cid = chatId || CONFIG.TELEGRAM_CHAT_ID;
  if (!tk || !cid) throw new Error(" chat_id ");
}

```

```

const { data } = await postJson(`https://api.telegram.org/bot${tk}/sendMessage`, { chat_id: cid, text });
if (data.ok === false) throw new Error(data.description || " ");
return data;
}
async function sendBale(text, chatId, token) {
const tk = token || CONFIG.BALE_BOT_TOKEN;
const cid = chatId || CONFIG.BALE_CHAT_ID;
if (!tk || !cid) throw new Error(" chat_id ");
const hosts = ["https://tapi.bale.ai", "https://api.bale.ai"];
let last = "";
for (const host of hosts) {
try {
const { data } = await postJson(`${host}/bot${tk}/sendMessage`, { chat_id: cid, text });
if (data.ok !== false) return data;
last = data.description || JSON.stringify(data);
} catch (e) {
last = String(e.message || e);
}
}
throw new Error(last || " ");
}
async function sendEitaa(text, chatId, token) {
const tk = token || CONFIG.ITA_BOT_TOKEN;
const cid = chatId || CONFIG.ITA_CHAT_ID;
if (!tk || !cid) throw new Error(" chat_id ");
// (form-urlencoded)
const res = await fetch(`https://eitaayar.ir/api/${tk}/sendMessage`, {
method: "POST",
headers: { "Content-Type": "application/x-www-form-urlencoded" },
body: new URLSearchParams({ chat_id: cid, text }).toString(),
});

const raw = await res.text();
let data;
try {
data = JSON.parse(raw);
} catch {
data = { raw };
}
if (!res.ok || data.ok === false) throw new Error(data.description || raw.slice(0, 200));
return data;
}
async function sendWhatsapp(text, to, token, provider) {
const p = (provider || "cloudapi").toLowerCase();
const digits = String(to || CONFIG.WHATSAPP_TO || "").replace(/\\D/g, "");
const tk = token || CONFIG.WHATSAPP_TOKEN;
if (p === "whatsiplus") {
const r = await fetch(
`https://api.whatsiplus.com/sendMsg/${tk}?phoneNumber=${encodeURIComponent(digits)}&message=${encodeURIComponent(t
ext)}`
);
const raw = await r.text();
let d;
try { d = JSON.parse(raw); } catch { d = { raw }; }
const okFlag = String(d.success ?? d.status ?? "").toLowerCase();
if (!r.ok || okFlag === "false" || okFlag === "error") throw new Error(d.message || raw.slice(0, 180));
return d;
}
if (p === "ultrams") {
const [instance, apiToken] = String(tk).split(":");
if (!instance || !apiToken) throw new Error(" UltraMsg instanceId:token ");
const r = await fetch(`https://api.ultrams.com/${instance}/messages/chat`, {
method: "POST",
headers: { "Content-Type": "application/x-www-form-urlencoded" },
body: new URLSearchParams({ token: apiToken, to: digits, body: text }).toString(),
});
const raw = await r.text();
if (!r.ok) throw new Error(raw.slice(0, 180));
try { return JSON.parse(raw); } catch { return { raw }; }
}
return sendWhatsappCloud(text, digits, tk);
}
async function sendWhatsappCloud(text, to, token) {
const tk = token || CONFIG.WHATSAPP_TOKEN;
const target = String(to || CONFIG.WHATSAPP_TO || "").replace(/\\D/g, "");
const i = String(tk).indexOf(":");
const phoneNumberId = i > 0 ? tk.slice(0, i) : "";
const accessToken = i > 0 ? tk.slice(i + 1) : "";
if (!phoneNumberId || !accessToken) throw new Error(" phoneNumberId:accessToken ");
if (!target) throw new Error(" ");
const res = await fetch(`https://graph.facebook.com/v21.0/${phoneNumberId}/messages`, {
method: "POST",
headers: { "Content-Type": "application/json", Authorization: `Bearer ${accessToken}` },
body: JSON.stringify({
messaging_product: "whatsapp",

```

```

        recipient_type: "individual",
        to: target,
        type: "text",
        text: { preview_url: false, body: text },
    }},
    });
    const raw = await res.text();
    if (!res.ok) throw new Error(raw.slice(0, 200));
    try {
        return JSON.parse(raw);
    } catch {
        return { raw };
    }
}

@@ public/crm-app.js javascript
// =====
// 0000 0000 000000 000000 00000 00000 000000000000 000000 0 0000000 (CRM / PWA)
// 0000 0000000000 0000 00 000 0000 00000000 00 000 0000000 00000 00000 00000 00000
// 0000 0000 0 000 0000 00 0 00000 0 00000 0000000 0000 000000
// =====

const STORAGE_KEY = "CRM_APP_STATE_V2";
let state = null;
let autoBackupFileHandle = null;
let autoBackupIntervalId = null;
let isShowingAllPasswords = false;

// 0000000000 0000 (Leaflet Map Instances)
let mapDashboardOverview = null;
let mapPharmacyForm = null;
let mapDoctorForm = null;
let mapLiveReps = null;
let mapFullOverview = null;

// 0000000000 000000 000 00000000
let markersDashboardOverview = [];
let markerPharmacyForm = null;
let markerDoctorForm = null;
let markersLiveReps = {};
let markersFullOverview = [];

// 0000 00 000000 00 0000 000000 (000000 00 00000000000 0 00000)
const MENU_SECTIONS_LIST = [
    { id: "tab-dashboard", label: "00000000", icon: "0" },
    { id: "tab-pharmacies", label: "000000000000", icon: "0", badgeld: "badgePharmaciesCount" },
    { id: "tab-doctors", label: "000000", icon: "0000", badgeld: "badgeDoctorsCount" },
    { id: "tab-orders", label: "00000000", icon: "0", badgeld: "badgeOrdersCount" },
    { id: "tab-activity-log", label: "000000 00000000", icon: "00" },
    { id: "tab-overview-map", label: "0000 0000", icon: "00" },
    { id: "tab-live-location", label: "000000 0000", icon: "0" },
    { id: "tab-search-info", label: "000000 00000000", icon: "0" },
    { id: "tab-rep-routes", label: "000 0000", icon: "00" },
    { id: "tab-rep-homes", label: "0000 0000000000", icon: "0" },
    { id: "tab-leaves", label: "00000000", icon: "0", badgeld: "badgeLeavesCount" },
    { id: "tab-notifications", label: "00000000", icon: "0" },
    { id: "tab-monthly-reports", label: "00000 000000", icon: "0" },
    { id: "tab-sales-targets", label: "000000 0000", icon: "0" },
    { id: "tab-custom-fields", label: "0000000000", icon: "0" },
    { id: "tab-columns-products", label: "00000000 0 000000", icon: "0" },
    { id: "tab-users-permissions", label: "00000000 0 000000", icon: "0", badgeld: "badgeUsersCount" },
    { id: "tab-messengers", label: "000000000000", icon: "0" },
    { id: "tab-backup", label: "00000000000000", icon: "0" },
    { id: "tab-install-app", label: "000 00", icon: "0" },
    { id: "tab-troubleshooting", label: "00000000", icon: "00" }
];

// -----
// 0. 000000 00000 0 00000 (State Management & LocalStorage)
// -----
function loadState() {
    const saved = localStorage.getItem(STORAGE_KEY);
    if (saved) {
        try {
            state = JSON.parse(saved);
            if (!state.customFields) state.customFields = DEFAULT_INITIAL_DATA.customFields;
            if (!state.settings) state.settings = DEFAULT_INITIAL_DATA.settings;
            if (!state.users) state.users = DEFAULT_INITIAL_DATA.users;
            if (!state.activityLog) state.activityLog = DEFAULT_INITIAL_DATA.activityLog;
            if (!state.repHomes) state.repHomes = DEFAULT_INITIAL_DATA.repHomes;
            if (!state.repRoutes) state.repRoutes = DEFAULT_INITIAL_DATA.repRoutes;
            if (!state.leaves) state.leaves = DEFAULT_INITIAL_DATA.leaves;
            if (!state.notifications) state.notifications = DEFAULT_INITIAL_DATA.notifications;
            if (!state.salesTargets) state.salesTargets = DEFAULT_INITIAL_DATA.salesTargets;
            if (!state.messengers) state.messengers = DEFAULT_INITIAL_DATA.messengers;
        }
    }
}

```

```

    if (!state.products) state.products = DEFAULT_INITIAL_DATA.products;
  } catch (e) {
    console.error("Error: " + e);
    state = JSON.parse(JSON.stringify(DEFAULT_INITIAL_DATA));
  }
} else {
  state = JSON.parse(JSON.stringify(DEFAULT_INITIAL_DATA));
}
}
applyGeneralSettingsToUI();
}

function saveState(triggerAutoBackup = true) {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(state));
  if (triggerAutoBackup && state.settings && state.settings.autoBackupEnabled) {
    performAutoBackup();
  }
}

function applyGeneralSettingsToUI() {
  if (!state.settings) return;
  const compHeader = document.getElementById("headerCompanyNameDisplay");
  if (compHeader && state.settings.companyName) {
    compHeader.textContent = state.settings.companyName;
  }
}

// -----
// 1. Initial state (wrapping grid - 12 columns, 1 row)
// -----
function setupNavigationMenu() {
  const horizontalContainer = document.getElementById("horizontalNavContainer");
  const sideContainer = document.getElementById("sideMenuItemsContainer");
  const launchpadGrid = document.getElementById("dashboardLaunchpadGrid");

  if (horizontalContainer) horizontalContainer.innerHTML = "";
  if (sideContainer) sideContainer.innerHTML = "";
  if (launchpadGrid) launchpadGrid.innerHTML = "";

  MENU_SECTIONS_LIST.forEach((sec, index) => {
    // 1. Initial state (wrapping grid - 12 columns, 1 row)
    if (horizontalContainer) {
      const btn = document.createElement("button");
      btn.className = "nav-item" + (index === 0 ? " active" : "");
      btn.setAttribute("data-target", sec.id);
      btn.innerHTML = `
        <span>${sec.icon}</span>
        ${sec.badgeId ? `<span class="nav-badge" id="${sec.badgeId}">0</span>` : ""}
      `;
      btn.onclick = () => switchTab(sec.id);
      horizontalContainer.appendChild(btn);
    }

    // 2. Initial state (wrapping grid - 12 columns, 1 row)
    if (sideContainer) {
      const btnSide = document.createElement("button");
      btnSide.className = "side-menu-item" + (index === 0 ? " active" : "");
      btnSide.setAttribute("data-side-target", sec.id);
      btnSide.innerHTML = `
        <span>${sec.label}</span>
        <span>${sec.icon}</span>
      `;
      btnSide.onclick = () => {
        switchTab(sec.id);
        closeSideMenu();
      };
      sideContainer.appendChild(btnSide);
    }

    // 3. Initial state (wrapping grid - 12 columns, 1 row)
    if (launchpadGrid) {
      const card = document.createElement("button");
      card.className = "launchpad-card";
      card.onclick = () => switchTab(sec.id);
      card.innerHTML = `
        <div class="launchpad-icon">${sec.icon}</div>
        <div class="launchpad-text">
          <h4>${sec.label}</h4>
        </div>
      `;
      launchpadGrid.appendChild(card);
    }
  });
}

// Initial state (wrapping grid - 12 columns, 1 row)
const btnToggleSide = document.getElementById("btnToggleSideMenu");

```

```

const btnCloseSide = document.getElementById("btnCloseSideMenu");
const overlay = document.getElementById("sideMenuOverlay");

if (btnToggleSide) btnToggleSide.addEventListener("click", openSideMenu);
if (btnCloseSide) btnCloseSide.addEventListener("click", closeSideMenu);
if (overlay) overlay.addEventListener("click", closeSideMenu);

updateNavBadges();
}

function openSideMenu() {
  document.getElementById("sideMenuDrawer").classList.add("active");
  document.getElementById("sideMenuOverlay").classList.add("active");
}

function closeSideMenu() {
  document.getElementById("sideMenuDrawer").classList.remove("active");
  document.getElementById("sideMenuOverlay").classList.remove("active");
}

function switchTab(targetId) {
  const navButtons = document.querySelectorAll(".nav-item");
  const sideButtons = document.querySelectorAll(".side-menu-item");
  const tabPanes = document.querySelectorAll(".tab-pane");

  navButtons.forEach(b => {
    if (b.getAttribute("data-target") === targetId) b.classList.add("active");
    else b.classList.remove("active");
  });

  sideButtons.forEach(b => {
    if (b.getAttribute("data-side-target") === targetId) b.classList.add("active");
    else b.classList.remove("active");
  });

  tabPanes.forEach(p => p.classList.remove("active"));
  const targetPane = document.getElementById(targetId);
  if (targetPane) {
    targetPane.classList.add("active");
    setTimeout(() => {
      if (targetId === "tab-dashboard" && mapDashboardOverview) mapDashboardOverview.invalidateSize();
      if (targetId === "tab-pharmacies" && mapPharmacyForm) mapPharmacyForm.invalidateSize();
      if (targetId === "tab-doctors" && mapDoctorForm) mapDoctorForm.invalidateSize();
      if (targetId === "tab-live-location" && mapLiveReps) {
        mapLiveReps.invalidateSize();
        renderLiveLocationTab();
      }
      if (targetId === "tab-overview-map") {
        initFullOverviewMap();
        if (mapFullOverview) mapFullOverview.invalidateSize();
      }
    }, 150);
  }
}

function updateNavBadges() {
  const phBadge = document.getElementById("badgePharmaciesCount");
  const docBadge = document.getElementById("badgeDoctorsCount");
  const ordBadge = document.getElementById("badgeOrdersCount");
  const usersBadge = document.getElementById("badgeUsersCount");

  if (phBadge) phBadge.textContent = state.pharmacies.length;
  if (docBadge) docBadge.textContent = state.doctors.length;
  if (ordBadge) ordBadge.textContent = state.orders.length;
  if (usersBadge) usersBadge.textContent = state.users.length;

  document.getElementById("statPharmacies").textContent = state.pharmacies.length;
  document.getElementById("statDoctors").textContent = state.doctors.length;
  document.getElementById("statReps").textContent = state.reps.length;
  document.getElementById("statOrders").textContent = state.orders.length;
  document.getElementById("statUsers").textContent = state.users.length;
  document.getElementById("statLeaves").textContent = (state.leaves || []).length;
}

// -----
// 2. 0000000000 00000 0: 000 000 000000 0000 0000 000000 0000 00 0000000 00000
// -----
function setupDropdownAutoClear() {
  const dropdowns = document.querySelectorAll(".dropdown-auto-clear, select.form-select, input[list]");
  dropdowns.forEach(elem => {
    if (elem.dataset.autoClearBound === "true") return;
    elem.dataset.autoClearBound = "true";

    elem.addEventListener("focus", function() {
      if (this.tagName.toLowerCase() === "input" && this.getAttribute("list")) {

```

```

        this.dataset.previousValue = this.value;
        this.value = "";
    }
    });
}

// -----
// 3. ۱. استان تهران ۲: شهرستان تهران ۳ شهر تهران ۴ منطقه تهران (Requirement 4)
// -----
function populateProvinces(selectElement, selectedValue = "") {
    selectElement.innerHTML = `<option value="">استان تهران</option>`;
    Object.keys(IRAN_GEO_DATA).forEach(province => {
        const opt = document.createElement("option");
        opt.value = province;
        opt.textContent = province;
        if (province === selectedValue) opt.selected = true;
        selectElement.appendChild(opt);
    });
}

function populateCities(provinceName, selectElement, selectedValue = "") {
    selectElement.innerHTML = `<option value="">شهرستان تهران</option>`;
    if (!provinceName || !IRAN_GEO_DATA[provinceName]) return;

    Object.keys(IRAN_GEO_DATA[provinceName]).forEach(city => {
        const opt = document.createElement("option");
        opt.value = city;
        opt.textContent = city;
        if (city === selectedValue) opt.selected = true;
        selectElement.appendChild(opt);
    });
}

function populateDistricts(provinceName, cityName, selectElement, selectedValue = "") {
    selectElement.innerHTML = `<option value="">منطقه تهران</option>`;
    if (!provinceName || !cityName || !IRAN_GEO_DATA[provinceName] || !IRAN_GEO_DATA[provinceName][cityName]) {
        return;
    }

    const districts = IRAN_GEO_DATA[provinceName][cityName];
    districts.forEach(district => {
        const opt = document.createElement("option");
        opt.value = district;
        opt.textContent = district;
        if (district === selectedValue) opt.selected = true;
        selectElement.appendChild(opt);
    });
}

function setupCascadingGeoSelectors(provinceId, cityId, districtId) {
    const provEl = document.getElementById(provinceId);
    const cityEl = document.getElementById(cityId);
    const distEl = document.getElementById(districtId);

    populateProvinces(provEl);

    provEl.addEventListener("change", () => {
        populateCities(provEl.value, cityEl);
        distEl.innerHTML = `<option value="">منطقه تهران</option>`;
    });

    cityEl.addEventListener("change", () => {
        populateDistricts(provEl.value, cityEl.value, distEl);
    });
}

// -----
// 4. ۱. استان تهران ۲: شهرستان تهران ۳ شهر تهران ۴ منطقه تهران (Requirement 5)
// -----
function createCustomMarker(lat, lng, type, name, mapInstance, onClickCallback = null) {
    let badgeClass = "marker-pharmacy-badge";
    let iconText = "🏠";
    let tooltipPrefix = "🏠 ";

    if (type === "doctor") {
        badgeClass = "marker-doctor-badge";
        iconText = "👨‍⚕️";
        tooltipPrefix = "👨‍⚕️ ";
    } else if (type === "rep") {
        badgeClass = "marker-rep-badge";
        iconText = "🗳️";
        tooltipPrefix = "🗳️ ";
    }
}

```



```

const customIcon = L.divIcon({
  className: "custom-div-icon-wrapper",
  html: `<div class="${badgeClass}" title="${name}">${iconText}</div>`,
  iconSize: [38, 38],
  iconAnchor: [19, 19]
});

const marker = L.marker([lat, lng], { icon: customIcon });

marker.bindTooltip(`${tooltipPrefix}${name}`, {
  permanent: true,
  direction: "top",
  className: "custom-map-tooltip",
  offset: [0, -22]
});

if (onClickCallback) {
  marker.on("click", onClickCallback);
}

marker.addTo(mapInstance);
marker.openTooltip();

return marker;
}

// -----
// 5. Dashboard Overview Map, Full Overview Map & Form Maps
// -----
function initMaps() {
  if (document.getElementById("map-dashboard-overview")) {
    mapDashboardOverview = L.map("map-dashboard-overview").setView([35.7200, 51.4200], 11);
    L.tileLayer("https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png", {
      maxZoom: 19,
      attribution: "© OpenStreetMap contributors"
    }).addTo(mapDashboardOverview);
    renderDashboardOverviewMap();
  }

  if (document.getElementById("map-pharmacy-form")) {
    mapPharmacyForm = L.map("map-pharmacy-form").setView([35.7605, 51.4180], 13);
    L.tileLayer("https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png", {
      maxZoom: 19,
      attribution: "© OpenStreetMap contributors"
    }).addTo(mapPharmacyForm);

    markerPharmacyForm = createCustomMarker(35.7605, 51.4180, "pharmacy", "🏠 🏠 🏠 🏠 🏠", mapPharmacyForm);

    mapPharmacyForm.on("click", (e) => {
      updatePharmacyFormMarker(e.latlng.lat, e.latlng.lng, document.getElementById("pharmacyName").value || "🏠 🏠 🏠 🏠 🏠");
    });
  }

  if (document.getElementById("map-doctor-form")) {
    mapDoctorForm = L.map("map-doctor-form").setView([35.7580, 51.4400], 13);
    L.tileLayer("https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png", {
      maxZoom: 19,
      attribution: "© OpenStreetMap contributors"
    }).addTo(mapDoctorForm);

    markerDoctorForm = createCustomMarker(35.7580, 51.4400, "doctor", "👨 ⚕️ 👨 ⚕️ 👨 ⚕️", mapDoctorForm);

    mapDoctorForm.on("click", (e) => {
      updateDoctorFormMarker(e.latlng.lat, e.latlng.lng, document.getElementById("doctorName").value || "👨 ⚕️ 👨 ⚕️ 👨 ⚕️");
    });
  }

  if (document.getElementById("map-live-reps")) {
    mapLiveReps = L.map("map-live-reps").setView([35.7300, 51.4200], 12);
    L.tileLayer("https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png", {
      maxZoom: 19,
      attribution: "© OpenStreetMap contributors"
    }).addTo(mapLiveReps);
  }
}

function initFullOverviewMap() {
  if (mapFullOverview) {
    renderFullOverviewMap();
    return;
  }
  const el = document.getElementById("map-full-overview");
  if (!el) return;

```

```

mapFullOverview = L.map("map-full-overview").setView([35.7200, 51.4200], 11);
L.tileLayer("https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png", {
  maxZoom: 19,
  attribution: "© OpenStreetMap contributors"
}).addTo(mapFullOverview);

renderFullOverviewMap();
}

function renderDashboardOverviewMap() {
  if (!mapDashboardOverview) return;
  markersDashboardOverview.forEach(m => mapDashboardOverview.removeLayer(m));
  markersDashboardOverview = [];

  const allPoints = [];
  state.pharmacies.forEach(ph => {
    if (ph.lat && ph.lng) {
      const m = createCustomMarker(ph.lat, ph.lng, "pharmacy", ph.name, mapDashboardOverview, () => {
        alert("📍 🏠🏠🏠🏠🏠: ${ph.name}\n📍🏠🏠🏠🏠: ${ph.address}\n📍🏠🏠🏠🏠: ${ph.isPercentage ? "🏠🏠" : "🏠🏠"}");
      });
      markersDashboardOverview.push(m);
      allPoints.push([ph.lat, ph.lng]);
    }
  });

  state.doctors.forEach(doc => {
    if (doc.lat && doc.lng) {
      const m = createCustomMarker(doc.lat, doc.lng, "doctor", doc.name, mapDashboardOverview, () => {
        alert("📍🏠🏠🏠🏠🏠: ${doc.name}\n📍🏠🏠🏠🏠: ${doc.specialty}\n📍🏠🏠🏠🏠: ${doc.address}");
      });
      markersDashboardOverview.push(m);
      allPoints.push([doc.lat, doc.lng]);
    }
  });

  state.reps.forEach(rep => {
    if (rep.lat && rep.lng) {
      const m = createCustomMarker(rep.lat, rep.lng, "rep", rep.name, mapDashboardOverview, () => {
        alert("📍 🏠🏠🏠🏠🏠: ${rep.name}\n📍🏠🏠🏠🏠: ${rep.status === "online" ? "🏠🏠🏠🏠🏠" : "🏠 🏠🏠 🏠🏠🏠"}");
      });
      markersDashboardOverview.push(m);
      allPoints.push([rep.lat, rep.lng]);
    }
  });

  if (allPoints.length > 0) {
    mapDashboardOverview.fitBounds(allPoints, { padding: [40, 40] });
  }
}

function renderFullOverviewMap() {
  if (!mapFullOverview) return;
  markersFullOverview.forEach(m => mapFullOverview.removeLayer(m));
  markersFullOverview = [];

  const allPoints = [];
  state.pharmacies.forEach(ph => {
    if (ph.lat && ph.lng) {
      const m = createCustomMarker(ph.lat, ph.lng, "pharmacy", ph.name, mapFullOverview);
      markersFullOverview.push(m);
      allPoints.push([ph.lat, ph.lng]);
    }
  });

  state.doctors.forEach(doc => {
    if (doc.lat && doc.lng) {
      const m = createCustomMarker(doc.lat, doc.lng, "doctor", doc.name, mapFullOverview);
      markersFullOverview.push(m);
      allPoints.push([doc.lat, doc.lng]);
    }
  });

  if (allPoints.length > 0) {
    mapFullOverview.fitBounds(allPoints, { padding: [40, 40] });
  }
}

// -----
// 6. 📍🏠🏠🏠🏠🏠: 🏠🏠🏠🏠🏠 🏠🏠 🏠🏠🏠 + 🏠🏠🏠🏠🏠 🏠🏠🏠 🏠🏠🏠 (Matching Screenshot 3)
// -----
async function reverseGeocodeCoordinates(lat, lng) {
  try {
    const url = `https://nominatim.openstreetmap.org/reverse?format=json&lat=${lat}&lon=${lng}&zoom=18&addressdetails=1`;
  }
  ;
  const res = await fetch(url, { headers: { "Accept-Language": "fa,en" } });
}

```

```

    const data = await res.json();
    if (data && data.display_name) {
        const addr = data.address || {};
        const street = addr.road || addr.street || addr.neighbourhood || addr.suburb || "□□□□□□ □□□□";
        const city = addr.city || addr.town || addr.city_district || "□□□□□□";
        return `${city}□ ${street} (□□□□□□□□ □□□ □□ □□□□□□: ${Number(lat).toFixed(4)}, ${Number(lng).toFixed(4)});`
    }
} catch (err) {
    console.warn("Reverse Geocode error:", err);
}
return `□□□□ □□□□□□ □□□ □□□ (Lat: ${Number(lat).toFixed(4)}, Lng: ${Number(lng).toFixed(4)})`;
}

async function searchAddressOnMap(queryText, updateMarkerFunc) {
    if (!queryText || !queryText.trim()) return;
    try {
        const url = `https://nominatim.openstreetmap.org/search?format=json&q=${encodeURIComponent(queryText)}&limit=1`;
        const res = await fetch(url, { headers: { "Accept-Language": "fa,en" } });
        const data = await res.json();
        if (data && data.length > 0) {
            const lat = parseFloat(data[0].lat);
            const lon = parseFloat(data[0].lon);
            updateMarkerFunc(lat, lon, queryText, true);
            alert(`□ □□□□□□□ «${queryText}» □□□□ □□ □ □□□ □□□□□□ □□.`);
        } else {
            alert("□□□□□□ □□ □□□ □□□□□□ □□□□ □□□.");
        }
    } catch (e) {
        alert("□□□□ □□ □□□□□□ □□□□.");
    }
}

function updatePharmacyFormMarker(lat, lng, nameText = "□□□□□□□□ □□□□□□□□", updateMapCenter = false) {
    document.getElementById("pharmacyLat").value = Number(lat).toFixed(5);
    document.getElementById("pharmacyLng").value = Number(lng).toFixed(5);

    if (mapPharmacyForm && markerPharmacyForm) {
        mapPharmacyForm.removeLayer(markerPharmacyForm);
        markerPharmacyForm = createCustomMarker(lat, lng, "pharmacy", nameText, mapPharmacyForm);
        if (updateMapCenter) {
            mapPharmacyForm.setView([lat, lng], 15);
        }
    }
}

function setupPharmacyLocationButtons() {
    const btnCurrentLoc = document.getElementById("btnPharmacyCurrentLocation");
    const btnGetAddr = document.getElementById("btnPharmacyGetAddressFromPoint");
    const btnSearchMap = document.getElementById("btnPhMapSearch");
    const searchInput = document.getElementById("phMapSearchInput");

    if (btnSearchMap && searchInput) {
        btnSearchMap.addEventListener("click", () => {
            searchAddressOnMap(searchInput.value.trim(), (lat, lng, nameText, updateCenter) => {
                updatePharmacyFormMarker(lat, lng, nameText, updateCenter);
            });
        });
    }

    btnCurrentLoc.addEventListener("click", () => {
        if (navigator.geolocation) {
            navigator.geolocation.getCurrentPosition(
                (pos) => {
                    const lat = pos.coords.latitude;
                    const lng = pos.coords.longitude;
                    const name = document.getElementById("pharmacyName").value || "□□□□□□ □□□□ □□□□□□□□";
                    updatePharmacyFormMarker(lat, lng, name, true);
                    alert(`□ □□□□□ □□□□ □□□□ □□.\n□□□□□□ □□□□□□□□□□ □□□ «□□□□□□ □□□□ □□□ □□□□» □□□□ □□□□.`);
                },
                (err) => {
                    const lat = 35.7595;
                    const lng = 51.4250;
                    updatePharmacyFormMarker(lat, lng, "□□□□□□ □□□□ □□□□□□□□", true);
                    alert(`□ □□□□□ □□□□□□ □□□□ □□□□ □□.`);
                }
            );
        } else {
            alert("□□□□□□ □□□ □□ □□□□□□ □□□□□□□□ □□□□□□□□ □□□□□□□□.");
        }
    });

    btnGetAddr.addEventListener("click", async () => {
        const lat = parseFloat(document.getElementById("pharmacyLat").value);
        const lng = parseFloat(document.getElementById("pharmacyLng").value);
    });
}

```

```

const addrText = await reverseGeocodeCoordinates(lat, lng);

document.getElementById("pharmacyAddress").value = addrText;
document.getElementById("pharmacyLocationText").value = addrText;

const name = document.getElementById("pharmacyName").value || "□□□□□□□□ □□□□";
updatePharmacyFormMarker(lat, lng, name, true);

alert('□ □□□□ □□□□ □□ □□□□□□ □□□□□□ □□ □ □□ □□□□ □□□□ □ □□ □□□□ □□□□□□□□ □□□□ □□□□:\n\n${addrText}');
});
}

function updateDoctorFormMarker(lat, lng, nameText = "□□□ □□□□□□□□", updateMapCenter = false) {
  document.getElementById("doctorLat").value = Number(lat).toFixed(5);
  document.getElementById("doctorLng").value = Number(lng).toFixed(5);

  if (mapDoctorForm && markerDoctorForm) {
    mapDoctorForm.removeLayer(markerDoctorForm);
    markerDoctorForm = createCustomMarker(lat, lng, "doctor", nameText, mapDoctorForm);
    if (updateMapCenter) {
      mapDoctorForm.setView([lat, lng], 15);
    }
  }
}

function setupDoctorLocationButtons() {
  const btnCurrentLoc = document.getElementById("btnDoctorCurrentLocation");
  const btnGetAddr = document.getElementById("btnDoctorGetAddressFromPoint");
  const btnSearchMap = document.getElementById("btnDocMapSearch");
  const searchInput = document.getElementById("docMapSearchInput");

  if (btnSearchMap && searchInput) {
    btnSearchMap.addEventListener("click", () => {
      searchAddressOnMap(searchInput.value.trim(), (lat, lng, nameText, updateCenter) => {
        updateDoctorFormMarker(lat, lng, nameText, updateCenter);
      });
    });
  }

  btnCurrentLoc.addEventListener("click", () => {
    if (navigator.geolocation) {
      navigator.geolocation.getCurrentPosition(
        (pos) => {
          const lat = pos.coords.latitude;
          const lng = pos.coords.longitude;
          updateDoctorFormMarker(lat, lng, "□□□□□□ □□□□ □□□", true);
          alert('□ □□□□□ □□□□ □□□□ □□.\n□□□□□□ □□□ «□□□□□□ □□□□ □□□ □□□□» □□□□ □□□□.);',
            (err) => {
              const lat = 35.7350;
              const lng = 51.4150;
              updateDoctorFormMarker(lat, lng, "□□□□□□ □□□□ □□□", true);
              alert('□ □□□□□ □□□□□□□□ □□□□ □□□□□□ □□.);');
            }
          );
        }
      );
    }
  });

  btnGetAddr.addEventListener("click", async () => {
    const lat = parseFloat(document.getElementById("doctorLat").value);
    const lng = parseFloat(document.getElementById("doctorLng").value);

    const addrText = await reverseGeocodeCoordinates(lat, lng);
    document.getElementById("doctorAddress").value = addrText;
    document.getElementById("doctorLocationText").value = addrText;

    const name = document.getElementById("doctorName").value || "□□□ □□□□ □□□□";
    updateDoctorFormMarker(lat, lng, name, true);

    alert('□ □□□□ □□□□ □□ □□□□□□ □□□□□□ □□ □ □□ □□□□ □□□□ □ □□ □□□□ □□□ □□□□ □□□□:\n\n${addrText}');
  });
}

// -----
// 7. □□□□□ □: □□□□□□ □□□□□□□□ □□□□□□ (Custom Fields Engine)
// -----
function setupCustomFieldsTab() {
  const formCF = document.getElementById("formCustomField");
  const typeSelect = document.getElementById("cfType");
  const optionsWrapper = document.getElementById("cfOptionsWrapper");

  if (typeSelect && optionsWrapper) {
    typeSelect.addEventListener("change", () => {
      optionsWrapper.style.display = typeSelect.value === "select" ? "block" : "none";
    });
  }
}

```

```

}

if (formCF) {
  formCF.addEventListener("submit", () => {
    const target = document.getElementById("cfTargetEntity").value;
    const label = document.getElementById("cfLabel").value.trim();
    const type = document.getElementById("cfType").value;
    const optionsStr = document.getElementById("cfOptions").value.trim();
    const allowAddOption = document.getElementById("cfAllowAddOption").checked;
    const showInForm = document.getElementById("cfShowInForm").checked;
    const showInList = document.getElementById("cfShowInList").checked;

    if (!label) {
      alert("□□□□□ □□□□□ □□□□ □□ □□□□ □□□□.");
      return;
    }

    let options = [];
    if (type === "select" && optionsStr) {
      options = optionsStr.split(/[,]/).map(o => o.trim()).filter(Boolean);
    }
    if (type === "select" && options.length === 0) {
      options = ["□□□□□ □□□", "□□□□□ □□□"];
    }

    const newField = {
      id: "cf-" + target + "-" + Date.now(),
      label: label,
      type: type,
      options: options,
      showInForm: showInForm,
      showInList: showInList,
      allowAddOption: allowAddOption
    };

    if (!state.customFields[target]) state.customFields[target] = [];
    state.customFields[target].push(newField);

    saveState();
    formCF.reset();
    optionsWrapper.style.display = "none";

    renderCustomFieldsTable();
    renderAllCustomFieldsInFormsAndTables();
    alert("□ □□□□ □□□□□ «${label}» □□ □□□□□ □□□□□ □□.");
  });
}

renderCustomFieldsTable();
}

function renderCustomFieldsTable() {
  const tbody = document.getElementById("tableCustomFieldsBody");
  if (!tbody) return;
  tbody.innerHTML = "";

  const entityLabels = { pharmacy: "□□□□□□□□□□", doctor: "□□□□□□", order: "□□□□□□□" };
  Object.keys(state.customFields).forEach(entity => {
    const fields = state.customFields[entity] || [];
    fields.forEach(f => {
      const tr = document.createElement("tr");
      tr.innerHTML = `
        <td><span class="status-badge" style="background: #eff6ff; color: #1e40af;">${entityLabels[entity]} || entity</span></td>
        <td><strong>${f.label}</strong></td>
        <td>${f.type === "select" ? "□□□□□" : "□□□□□"}</td>
        <td>${f.type === "select" && f.options ? f.options.join("□") : "-"}</td>
        <td>${f.showInForm ? "□ □□□" : "□ □□□"}</td>
        <td>${f.showInList ? "□ □□□" : "□ □□□"}</td>
        <td>
          <button class="btn btn-danger btn-sm" onclick="deleteCustomField('${entity}', '${f.id}')">□□</button>
        </td>
      `;
      tbody.appendChild(tr);
    });
  });
}

function deleteCustomField(entity, fieldId) {
  if (!confirm("□□□ □□ □□□ □□□ □□□□□□ □□□□□□ □□□□□□")) return;
  state.customFields[entity] = state.customFields[entity].filter(f => f.id !== fieldId);
  saveState();
  renderCustomFieldsTable();
  renderAllCustomFieldsInFormsAndTables();
}

```

```

function renderCustomFieldsInForm(entityType, containerId, currentValues = {}) {
  const container = document.getElementById(containerId);
  if (!container) return;
  container.innerHTML = "";

  const fields = state.customFields[entityType] || [];
  fields.forEach(field => {
    if (!field.showInForm) return;

    const div = document.createElement("div");
    div.className = "form-group";

    const labelRow = document.createElement("div");
    labelRow.style.display = "flex";
    labelRow.style.alignItems = "center";
    labelRow.style.justifyContent = "space-between";

    const labelEl = document.createElement("label");
    labelEl.className = "form-label";
    labelEl.textContent = field.label;
    labelRow.appendChild(labelEl);

    if (field.type === "select" && field.allowAddOption) {
      const btnAddOpt = document.createElement("button");
      btnAddOpt.type = "button";
      btnAddOpt.className = "btn-add-option";
      btnAddOpt.innerHTML = "□ □□□□□ □□□□ □□□□";
      btnAddOpt.onclick = () => openModalAddOption(entityType, field.id, field.label);
      labelRow.appendChild(btnAddOpt);
    }
    div.appendChild(labelRow);

    if (field.type === "select") {
      const sel = document.createElement("select");
      sel.className = "form-select dropdown-auto-clear";
      sel.dataset.customFieldId = field.id;
      sel.innerHTML = `<option value="">□□□□□□ □□□□...</option>`;

      (field.options || []).forEach(opt => {
        const o = document.createElement("option");
        o.value = opt;
        o.textContent = opt;
        if (currentValues[field.label] === opt || currentValues[field.id] === opt) {
          o.selected = true;
        }
        sel.appendChild(o);
      });
      div.appendChild(sel);
    } else {
      const input = document.createElement("input");
      input.type = "text";
      input.className = "form-input";
      input.dataset.customFieldId = field.id;
      input.placeholder = `□□□□ □□□□: ${field.label}...`;
      if (currentValues[field.label]) input.value = currentValues[field.label];
      div.appendChild(input);
    }

    container.appendChild(div);
  });

  setupDropdownAutoClear();
}

function extractCustomFieldValuesFromForm(entityType, containerId) {
  const container = document.getElementById(containerId);
  if (!container) return {};
  const values = {};
  const fields = state.customFields[entityType] || [];
  fields.forEach(field => {
    const el = container.querySelector(`[data-custom-field-id="${field.id}"]`);
    if (el) values[field.label] = el.value.trim();
  });
  return values;
}

function openModalAddOption(entityType, fieldId, fieldLabel) {
  document.getElementById("modalAddOptionEntityType").value = entityType;
  document.getElementById("modalAddOptionFieldId").value = fieldId;
  document.getElementById("modalAddOptionLabel").textContent = `□□□□□□ □□□□ □□□□ □□□□ «${fieldLabel}»`;
  document.getElementById("modalAddOptionValue").value = "";
  document.getElementById("modalAddOption").classList.add("active");
  document.getElementById("modalAddOptionValue").focus();
}

```

```

function closeModalAddOption() {
  document.getElementById("modalAddOption").classList.remove("active");
}

function setupAddOptionModalForm() {
  const form = document.getElementById("formAddOptionModal");
  if (!form) return;

  form.addEventListener("submit", () => {
    const entityType = document.getElementById("modalAddOptionEntityType").value;
    const fieldId = document.getElementById("modalAddOptionFieldId").value;
    const newVal = document.getElementById("modalAddOptionValue").value.trim();

    if (!newVal) {
      alert("Введите значение для поля.");
      return;
    }

    const fields = state.customFields[entityType] || [];
    const targetField = fields.find(f => f.id === fieldId);

    if (targetField) {
      if (!targetField.options) targetField.options = [];
      if (!targetField.options.includes(newVal)) targetField.options.push(newVal);
      saveState();
      closeModalAddOption();

      if (entityType === "pharmacy") {
        renderCustomFieldsInForm("pharmacy", "pharmacyCustomFieldsContainer", extractCustomFieldValuesFromForm("pharmacy", "pharmacyCustomFieldsContainer"));
      } else if (entityType === "doctor") {
        renderCustomFieldsInForm("doctor", "doctorCustomFieldsContainer", extractCustomFieldValuesFromForm("doctor", "doctorCustomFieldsContainer"));
      } else if (entityType === "order") {
        renderCustomFieldsInForm("order", "orderCustomFieldsContainer", extractCustomFieldValuesFromForm("order", "orderCustomFieldsContainer"));
      }
      renderCustomFieldsTable();
      alert(`Поле «${newVal}» добавлено.`);
    }
  });
}

function renderAllCustomFieldsInFormsAndTables() {
  renderCustomFieldsInForm("pharmacy", "pharmacyCustomFieldsContainer");
  renderCustomFieldsInForm("doctor", "doctorCustomFieldsContainer");
  renderCustomFieldsInForm("order", "orderCustomFieldsContainer");
  renderPharmaciesList();
  renderDoctorsList();
  renderOrdersList();
}

// -----
// 8. Реализация CRUD + Переключатель + Загрузка файла
// (Matching Screenshots 2 & 3)
// -----
function setupPharmacyTab() {
  setupCascadingGeoSelectors("pharmacyProvince", "pharmacyCity", "pharmacyDistrict");
  setupPharmacyLocationButtons();

  // Реализация переключателя (Screenshot 3)
  const btnNo = document.getElementById("btnPhPercentageNo");
  const btnYes = document.getElementById("btnPhPercentageYes");
  const hiddenIsPerc = document.getElementById("pharmacyIsPercentage");

  if (btnNo && btnYes) {
    btnNo.addEventListener("click", () => {
      btnNo.classList.add("active");
      btnYes.classList.remove("active");
      hiddenIsPerc.value = "false";
    });
    btnYes.addEventListener("click", () => {
      btnYes.classList.add("active");
      btnNo.classList.remove("active");
      hiddenIsPerc.value = "true";
    });
  }

  // Реализация загрузки файла (Screenshot 3)
  const fileInput = document.getElementById("phFileInput");
  const fileDisplay = document.getElementById("phFileDisplay");
  if (fileInput && fileDisplay) {
    fileInput.addEventListener("change", () => {
      if (fileInput.files && fileInput.files[0]) {

```

```

fileDisplay.textContent = `📄 📄📄📄📄📄 📄: ${fileInput.files[0].name}`;
} else {
fileDisplay.textContent = "📄📄📄📄📄📄📄📄 📄📄📄 📄📄";
}
});
}

const form = document.getElementById("formPharmacy");
const btnReset = document.getElementById("btnResetPharmacyForm");
const searchInput = document.getElementById("searchPharmacyInput");

if (btnReset) btnReset.addEventListener("click", resetPharmacyForm);

if (searchInput) {
searchInput.addEventListener("input", () => {
renderPharmaciesList(searchInput.value.trim());
});
}

if (form) {
form.addEventListener("submit", () => {
const editId = document.getElementById("pharmacyEditId").value;
const dateAdded = document.getElementById("pharmacyDate").value.trim() || new Date().toLocaleDateString("fa-IR");
const name = document.getElementById("pharmacyName").value.trim();
const phone = document.getElementById("pharmacyPhone").value.trim();
const manager = document.getElementById("pharmacyManager").value.trim();
const province = document.getElementById("pharmacyProvince").value;
const city = document.getElementById("pharmacyCity").value;
const district = document.getElementById("pharmacyDistrict").value;
const address = document.getElementById("pharmacyAddress").value.trim();
const lat = parseFloat(document.getElementById("pharmacyLat").value) || 35.7605;
const lng = parseFloat(document.getElementById("pharmacyLng").value) || 51.4180;
const isPercentage = hiddenIsPerc.value === "true";
const fileName = (fileInput.files && fileInput.files[0]) ? fileInput.files[0].name : null;
const customFieldsVals = extractCustomFieldValuesFromForm("pharmacy", "pharmacyCustomFieldsContainer");

if (!name || !province || !city || !district || !address) {
alert("📄📄📄📄 📄📄📄📄 📄📄📄📄📄📄 (*) 📄📄 📄📄📄📄📄📄 📄📄 📄📄📄.");
return;
}

if (editId) {
const idx = state.pharmacies.findIndex(p => p.id === editId);
if (idx !== -1) {
state.pharmacies[idx] = {
...state.pharmacies[idx],
dateAdded, name, phone, manager, province, city, district, address, lat, lng,
isPercentage, fileName: fileName || state.pharmacies[idx].fileName,
customFields: customFieldsVals
};
}
alert("📄 📄📄📄📄📄 «${name}» 📄📄 📄📄📄📄📄📄 📄 📄📄📄📄 📄.");
} else {
const newPharmacy = {
id: "ph-" + Date.now(),
dateAdded, name, phone, manager, province, city, district, address, lat, lng,
isPercentage, fileName,
repName: "📄📄📄 📄📄📄",
customFields: customFieldsVals
};
state.pharmacies.push(newPharmacy);
alert("📄 📄📄📄📄📄 «${name}» 📄📄 📄📄📄📄 📄📄 📄.");
}

saveState();
resetPharmacyForm();
renderPharmaciesList();
renderDashboardOverviewMap();
updateNavBadges();
populatePharmacyDataListInOrders();
});
}
}

function resetPharmacyForm() {
document.getElementById("pharmacyEditId").value = "";
document.getElementById("pharmacyDate").value = new Date().toLocaleDateString("fa-IR");
document.getElementById("pharmacyName").value = "";
document.getElementById("pharmacyPhone").value = "";
document.getElementById("pharmacyManager").value = "";
document.getElementById("pharmacyAddress").value = "";
document.getElementById("pharmacyLocationText").value = "";
document.getElementById("pharmacyIsPercentage").value = "false";
document.getElementById("btnPhPercentageNo").classList.add("active");
document.getElementById("btnPhPercentageYes").classList.remove("active");
}

```



```

document.getElementById("phFileDisplay").textContent = "□□□□□ □□□□□□□□ □□□□ □□□";

const provEl = document.getElementById("pharmacyProvince");
const cityEl = document.getElementById("pharmacyCity");
const distEl = document.getElementById("pharmacyDistrict");

provEl.value = "";
cityEl.innerHTML = `<option value="">□□□□□ □□□□□ □□ □□□□□□ □□□□</option>`;
distEl.innerHTML = `<option value="">□□□□□ □□□ □□ □□□□□□ □□□□</option>`;

updatePharmacyFormMarker(35.7605, 51.4180, "□□□□□□ □□□□□□□□ □□□□", true);
renderCustomFieldsInForm("pharmacy", "pharmacyCustomFieldsContainer");
}

function renderPharmaciesList(searchQuery = "") {
  const theadTr = document.getElementById("tablePharmaciesHeader");
  const tbody = document.getElementById("tablePharmaciesBody");
  if (!theadTr || !tbody) return;

  theadTr.innerHTML = `
    <th>□□□□</th>
    <th>□□□ □□□□□□□□</th>
    <th>□□□□□ □□□</th>
    <th>□□□□□</th>
    <th>□□□</th>
    <th>□□□ □□□□□□□□</th>
    <th>□□□□□ □□□□□</th>
    <th>□□□□□</th>
    <th>□□□□</th>
    <th>□□□□□□</th>
  `;

  const fieldsList = (state.customFields.pharmacy || []).filter(f => f.showInList);
  fieldsList.forEach(f => {
    const th = document.createElement("th");
    th.textContent = f.label;
    theadTr.appendChild(th);
  });

  const thAction = document.createElement("th");
  thAction.textContent = "□□□□□□";
  theadTr.appendChild(thAction);

  tbody.innerHTML = "";

  const filtered = state.pharmacies.filter(ph => {
    if (!searchQuery) return true;
    return ph.name.includes(searchQuery) || ph.address.includes(searchQuery) || (ph.phone && ph.phone.includes(searchQue
ry));
  });

  const badgeEl = document.getElementById("phListCountBadge");
  const tableBadge = document.getElementById("phTableCountBadge");
  if (badgeEl) badgeEl.textContent = filtered.length;
  if (tableBadge) tableBadge.textContent = filtered.length;

  filtered.forEach((ph, index) => {
    const tr = document.createElement("tr");

    let rowHTML = `
      <td>${index + 1}</td>
      <td><strong style="color:#0f172a;">${ph.repName || "□□□□ □□□□□"}</strong></td>
      <td>${ph.dateAdded || "-"}</td>
      <td>${ph.province}</td>
      <td>${ph.city}</td>
      <td><strong style="color: #0d9488;">${ph.name}</strong></td>
      <td><span style="direction: ltr; display: inline-block;">${ph.phone || "-"}</span></td>
      <td>${ph.isPercentage ? '<span class="badge-status-online">□□□</span>': '<span>□□□</span>'}</td>
      <td>${ph.fileName ? '<span style="color:#0d9488;">□ ${ph.fileName}</span>': '-'}</td>
    `;

    fieldsList.forEach(f => {
      const val = (ph.customFields && ph.customFields[f.label]) ? ph.customFields[f.label] : "-";
      rowHTML += `<td><strong>${val}</strong></td>`;
    });

    rowHTML += `
      <td>
        <div style="display: flex; gap: 0.4rem; flex-wrap: wrap;">
        <button class="btn btn-outline btn-sm" onclick="editPharmacy('${ph.id}')">□□ □□□□□□</button>
        <button class="btn btn-danger btn-sm" onclick="deletePharmacy('${ph.id}')">□□ □□□</button>
        </div>
      </td>
    `;
  });
}

```

```

        tr.innerHTML = rowHTML;
        tbody.appendChild(tr);
    });
}

function editPharmacy(id) {
    const ph = state.pharmacies.find(p => p.id === id);
    if (!ph) return;

    document.getElementById("pharmacyEditId").value = ph.id;
    document.getElementById("pharmacyDate").value = ph.dateAdded || "";
    document.getElementById("pharmacyName").value = ph.name;
    document.getElementById("pharmacyPhone").value = ph.phone || "";
    document.getElementById("pharmacyManager").value = ph.manager || "";
    document.getElementById("pharmacyAddress").value = ph.address || "";
    document.getElementById("pharmacyLocationText").value = ph.address || "";

    if (ph.isPercentage) {
        document.getElementById("pharmacyIsPercentage").value = "true";
        document.getElementById("btnPhPercentageYes").classList.add("active");
        document.getElementById("btnPhPercentageNo").classList.remove("active");
    } else {
        document.getElementById("pharmacyIsPercentage").value = "false";
        document.getElementById("btnPhPercentageNo").classList.add("active");
        document.getElementById("btnPhPercentageYes").classList.remove("active");
    }

    const provEl = document.getElementById("pharmacyProvince");
    const cityEl = document.getElementById("pharmacyCity");
    const distEl = document.getElementById("pharmacyDistrict");

    provEl.value = ph.province;
    populateCities(ph.province, cityEl, ph.city);
    populateDistricts(ph.province, ph.city, distEl, ph.district);

    updatePharmacyFormMarker(ph.lat || 35.7605, ph.lng || 51.4180, ph.name, true);
    renderCustomFieldsInForm("pharmacy", "pharmacyCustomFieldsContainer", ph.customFields || {});

    document.getElementById("tab-pharmacies").scrollIntoView({ behavior: "smooth" });
}

function deletePharmacy(id) {
    if (!confirm("确定要删除该药店吗？")) return;
    state.pharmacies = state.pharmacies.filter(p => p.id !== id);
    saveState();
    renderPharmaciesList();
    renderDashboardOverviewMap();
    updateNavBadges();
    populatePharmacyDataListInOrders();
}

// -----
// 9. 医生管理 (Doctors CRUD + Percentage Toggle + File upload)
// -----

function setupDoctorTab() {
    setupCascadingGeoSelectors("doctorProvince", "doctorCity", "doctorDistrict");
    setupDoctorLocationButtons();

    const btnNo = document.getElementById("btnDocPercentageNo");
    const btnYes = document.getElementById("btnDocPercentageYes");
    const hiddenIsPerc = document.getElementById("doctorIsPercentage");

    if (btnNo && btnYes) {
        btnNo.addEventListener("click", () => {
            btnNo.classList.add("active");
            btnYes.classList.remove("active");
            hiddenIsPerc.value = "false";
        });
        btnYes.addEventListener("click", () => {
            btnYes.classList.add("active");
            btnNo.classList.remove("active");
            hiddenIsPerc.value = "true";
        });
    }

    const fileInput = document.getElementById("docFileInput");
    const fileDisplay = document.getElementById("docFileDisplay");
    if (fileInput && fileDisplay) {
        fileInput.addEventListener("change", () => {
            if (fileInput.files && fileInput.files[0]) {
                fileDisplay.textContent = `已上传文件: ${fileInput.files[0].name}`;
            } else {
                fileDisplay.textContent = "未上传文件";
            }
        });
    }
}

```

```

}

const form = document.getElementById("formDoctor");
const btnReset = document.getElementById("btnResetDoctorForm");
const searchInput = document.getElementById("searchDoctorInput");

if (btnReset) btnReset.addEventListener("click", resetDoctorForm);

if (searchInput) {
  searchInput.addEventListener("input", () => {
    renderDoctorsList(searchInput.value.trim());
  });
}

if (form) {
  form.addEventListener("submit", () => {
    const editId = document.getElementById("doctorEditId").value;
    const dateAdded = document.getElementById("doctorDate").value.trim() || new Date().toLocaleDateString("fa-IR");
    const name = document.getElementById("doctorName").value.trim();
    const specialty = document.getElementById("doctorSpecialty").value.trim();
    const phone = document.getElementById("doctorPhone").value.trim();
    const province = document.getElementById("doctorProvince").value;
    const city = document.getElementById("doctorCity").value;
    const district = document.getElementById("doctorDistrict").value;
    const address = document.getElementById("doctorAddress").value.trim();
    const lat = parseFloat(document.getElementById("doctorLat").value) || 35.7580;
    const lng = parseFloat(document.getElementById("doctorLng").value) || 51.4400;
    const isPercentage = hiddenIsPerc.value === "true";
    const fileName = (fileInput.files && fileInput.files[0]) ? fileInput.files[0].name : null;
    const customFieldsVals = extractCustomFieldValuesFromForm("doctor", "doctorCustomFieldsContainer");

    if (!name || !specialty || !province || !city || !district || !address) {
      alert("لطفاً نام، تخصص، استان، شهر، منطقه و آدرس را وارد کنید.");
      return;
    }

    if (editId) {
      const idx = state.doctors.findIndex(d => d.id === editId);
      if (idx !== -1) {
        state.doctors[idx] = {
          ...state.doctors[idx],
          dateAdded, name, specialty, phone, province, city, district, address, lat, lng,
          isPercentage, fileName: fileName || state.doctors[idx].fileName,
          customFields: customFieldsVals
        };
      }
    }
    alert("دکتر/دکتره «${name}» با موفقیت به سیستم اضافه شد.");
  } else {
    const newDoc = {
      id: "doc-" + Date.now(),
      dateAdded, name, specialty, phone, province, city, district, address, lat, lng,
      isPercentage, fileName,
      repName: "مأمور بهداشتی",
      customFields: customFieldsVals
    };
    state.doctors.push(newDoc);
    alert("دکتر/دکتره «${name}» با موفقیت به سیستم اضافه شد.");
  }

  saveState();
  resetDoctorForm();
  renderDoctorsList();
  renderDashboardOverviewMap();
  updateNavBadges();
});
}

function resetDoctorForm() {
  document.getElementById("doctorEditId").value = "";
  document.getElementById("doctorDate").value = new Date().toLocaleDateString("fa-IR");
  document.getElementById("doctorName").value = "";
  document.getElementById("doctorSpecialty").value = "";
  document.getElementById("doctorPhone").value = "";
  document.getElementById("doctorAddress").value = "";
  document.getElementById("doctorLocationText").value = "";
  document.getElementById("doctorIsPercentage").value = "false";
  document.getElementById("btnDocPercentageNo").classList.add("active");
  document.getElementById("btnDocPercentageYes").classList.remove("active");
  document.getElementById("docFileDisplay").textContent = "فایل انتخاب شده: نام فایل";

  const provEl = document.getElementById("doctorProvince");
  const cityEl = document.getElementById("doctorCity");
  const distEl = document.getElementById("doctorDistrict");

```

```

    provEl.value = "";
    cityEl.innerHTML = `<option value="">□□□□ □□□□ □□ □□□□□□ □□□□</option>`;
    distEl.innerHTML = `<option value="">□□□□ □□ □□ □□□□□□ □□□□</option>`;

    updateDoctorFormMarker(35.7580, 51.4400, "□□□□□□ □□ □□□□", true);
    renderCustomFieldsInForm("doctor", "doctorCustomFieldsContainer");
}

function renderDoctorsList(searchQuery = "") {
    const theadTr = document.getElementById("tableDoctorsHeader");
    const tbody = document.getElementById("tableDoctorsBody");
    if (!theadTr || !tbody) return;

    theadTr.innerHTML = `
    <th>□□□□</th>
    <th>□□ □□□□□□</th>
    <th>□□□□ □□</th>
    <th>□□□□ / □□□ / □□□□□</th>
    <th>□□ □□□□ / □□□</th>
    <th>□□□□</th>
    <th>□□□□□</th>
    <th>□□□□</th>
    <th>□□□□□□</th>
    `;

    const fieldsList = (state.customFields.doctor || []).filter(f => f.showInList);
    fieldsList.forEach(f => {
        const th = document.createElement("th");
        th.textContent = f.label;
        theadTr.appendChild(th);
    });

    const thAction = document.createElement("th");
    thAction.textContent = "□□□□□□";
    theadTr.appendChild(thAction);

    tbody.innerHTML = "";

    const filtered = state.doctors.filter(doc => {
        if (!searchQuery) return true;
        return doc.name.includes(searchQuery) || doc.specialty.includes(searchQuery) || doc.address.includes(searchQuery);
    });

    filtered.forEach((doc, index) => {
        const tr = document.createElement("tr");

        let rowHTML = `
        <td>${index + 1}</td>
        <td><strong style="color:#0f172a;">${doc.repName || "□□□□ □□□□□□"}</strong></td>
        <td>${doc.dateAdded || "-"}</td>
        <td>${doc.province} / ${doc.city}</td>
        <td><strong style="color: #0d9488;">${doc.name}</strong></td>
        <td><span class="status-badge" style="background: #f0fdf4; color: #15803d;">${doc.specialty}</span></td>
        <td>${doc.isPercentage ? '<span class="badge-status-online">□□□</span>': '<span>□□□</span>'}</td>
        <td>${doc.fileName ? '<span style="color:#0d9488;">□ ${doc.fileName}</span>': '-'}</td>
        `;

        fieldsList.forEach(f => {
            const val = (doc.customFields && doc.customFields[f.label]) ? doc.customFields[f.label] : "-";
            rowHTML += `<td><strong>${val}</strong></td>`;
        });

        rowHTML += `
        <td>
            <div style="display: flex; gap: 0.4rem; flex-wrap: wrap;">
            <button class="btn btn-outline btn-sm" onclick="editDoctor('${doc.id}')">□□ □□□□□□</button>
            <button class="btn btn-danger btn-sm" onclick="deleteDoctor('${doc.id}')">□□ □□□</button>
            </div>
        </td>
        `;

        tr.innerHTML = rowHTML;
        tbody.appendChild(tr);
    });
}

function editDoctor(id) {
    const doc = state.doctors.find(d => d.id === id);
    if (!doc) return;

    document.getElementById("doctorEditId").value = doc.id;
    document.getElementById("doctorDate").value = doc.dateAdded || "";
    document.getElementById("doctorName").value = doc.name;
    document.getElementById("doctorSpecialty").value = doc.specialty;
    document.getElementById("doctorPhone").value = doc.phone || "";
}

```

```

document.getElementById("doctorAddress").value = doc.address || "";
document.getElementById("doctorLocationText").value = doc.address || "";

if (doc.isPercentage) {
  document.getElementById("doctorIsPercentage").value = "true";
  document.getElementById("btnDocPercentageYes").classList.add("active");
  document.getElementById("btnDocPercentageNo").classList.remove("active");
} else {
  document.getElementById("doctorIsPercentage").value = "false";
  document.getElementById("btnDocPercentageNo").classList.add("active");
  document.getElementById("btnDocPercentageYes").classList.remove("active");
}

const provEl = document.getElementById("doctorProvince");
const cityEl = document.getElementById("doctorCity");
const distEl = document.getElementById("doctorDistrict");

provEl.value = doc.province;
populateCities(doc.province, cityEl, doc.city);
populateDistricts(doc.province, doc.city, distEl, doc.district);

updateDoctorFormMarker(doc.lat || 35.7580, doc.lng || 51.4400, doc.name, true);
renderCustomFieldsInForm("doctor", "doctorCustomFieldsContainer", doc.customFields || {});

document.getElementById("tab-doctors").scrollIntoView({ behavior: "smooth" });
}

function deleteDoctor(id) {
  if (!confirm("确定要删除该医生吗？")) return;
  state.doctors = state.doctors.filter(d => d.id !== id);
  saveState();
  renderDoctorsList();
  renderDashboardOverviewMap();
  updateNavBadges();
}

// -----
// 10. 匹配截图4,5,6,7
// -----
function setupUsersAndPermissionsTab() {
  renderPermissionGroupsCheckList();
  renderUserCardsList();

  const btnSelectAll = document.getElementById("btnPermSelectAll");
  const btnSelectNone = document.getElementById("btnPermSelectNone");
  const btnTogglePasswords = document.getElementById("btnToggleShowAllPasswords");

  if (btnSelectAll) {
    btnSelectAll.addEventListener("click", () => {
      document.querySelectorAll(".permission-tag-chk").forEach(el => {
        el.classList.remove("unchecked");
        el.dataset.checked = "true";
      });
      updatePermCountBadge();
    });
  }

  if (btnSelectNone) {
    btnSelectNone.addEventListener("click", () => {
      document.querySelectorAll(".permission-tag-chk").forEach(el => {
        el.classList.add("unchecked");
        el.dataset.checked = "false";
      });
      updatePermCountBadge();
    });
  }

  if (btnTogglePasswords) {
    btnTogglePasswords.addEventListener("click", () => {
      isShowingAllPasswords = !isShowingAllPasswords;
      btnTogglePasswords.innerHTML = isShowingAllPasswords ? "显示所有密码" : "隐藏所有密码";
      renderUserCardsList();
    });
  }

  const formCreate = document.getElementById("formCreateUser");
  if (formCreate) {
    formCreate.addEventListener("submit", () => {
      const fullName = document.getElementById("newFullName").value.trim();
      const username = document.getElementById("newUsername").value.trim();
      const password = document.getElementById("newPassword").value.trim();
      const phone = document.getElementById("newPhone").value.trim();
      const role = document.getElementById("newRole").value;
      const simControl = document.getElementById("newSimControl").value;
    });
  }
}

```

```

        if (!fullName || !username || !password) {
            alert("نام و نام خانوادگی و نام کاربری و رمز عبور را وارد کنید.");
            return;
        }

        // نام و نام خانوادگی و نام کاربری و رمز عبور و شماره تماس و نقش و رمز عبور
        const permsObj = {};
        document.querySelectorAll(".permission-tag-chk").forEach(el => {
            permsObj[el.dataset.key] = el.dataset.checked === "true";
        });

        const newUser = {
            id: "u-" + Date.now(),
            fullName,
            username,
            password,
            phone,
            role,
            simControl,
            phoneLock: "نام و نام خانوادگی و نام کاربری و رمز عبور و شماره تماس و نقش و رمز عبور",
            lastLogin: new Date().toLocaleDateString("fa-IR") + " - " + new Date().toLocaleTimeString("fa-IR"),
            permissions: permsObj
        };

        state.users.push(newUser);
        saveState();
        formCreate.reset();
        renderUserCardsList();
        updateNavBadges();
        alert("نام و نام خانوادگی شما «${fullName}» با موفقیت ثبت شد.");
    });
}

function renderPermissionGroupsChecklist() {
    const container = document.getElementById("permissionGroupsContainer");
    if (!container) return;
    container.innerHTML = "";

    Object.entries(PERMISSION_GROUPS).forEach(([groupName, items]) => {
        const grpDiv = document.createElement("div");
        grpDiv.innerHTML = `<div class="permission-group-title">${groupName}</div>`;

        const tagsDiv = document.createElement("div");
        tagsDiv.style.display = "flex";
        tagsDiv.style.flexWrap = "wrap";
        tagsDiv.style.gap = "0.5rem";

        items.forEach(item => {
            const tag = document.createElement("div");
            tag.className = "permission-tag-chk";
            tag.dataset.key = item.key;
            tag.dataset.checked = "true";
            tag.innerHTML = `<span>${item.label}</span>`;
            tag.onclick = () => {
                const isChk = tag.dataset.checked === "true";
                if (isChk) {
                    tag.dataset.checked = "false";
                    tag.classList.add("unchecked");
                } else {
                    tag.dataset.checked = "true";
                    tag.classList.remove("unchecked");
                }
                updatePermCountBadge();
            };
            tagsDiv.appendChild(tag);
        });

        grpDiv.appendChild(tagsDiv);
        container.appendChild(grpDiv);
    });

    updatePermCountBadge();
}

function updatePermCountBadge() {
    const all = document.querySelectorAll(".permission-tag-chk");
    let count = 0;
    all.forEach(el => { if (el.dataset.checked === "true") count++; });
    const badge = document.getElementById("selectedPermCountBadge");
    if (badge) badge.textContent = `${count} مورد`;
}

function renderUserCardsList() {
    const container = document.getElementById("userCardsContainer");

```

```

if (!container) return;
container.innerHTML = "";

state.users.forEach((user, index) => {
  const card = document.createElement("div");
  card.style.background = "#ffffff";
  card.style.border = "1px solid var(--border-color)";
  card.style.borderRadius = "12px";
  card.style.padding = "1.25rem";
  card.style.boxShadow = "var(--shadow-sm)";

  const passDisplay = isShowingAllPasswords ? user.password : "*****";

  card.innerHTML = `
    <div style="display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap; margin-bottom: 0.75rem; border-bottom: 1px solid var(--border-color); padding-bottom: 0.75rem;">
      <div>
        <span style="font-weight: 800; font-size: 1.05rem; color: #0f172a;">${index + 1}. ${user.fullName}</span>
        <span class="status-badge" style="background:#eff6ff;color:#1e40af;margin-right:0.5rem;">${user.role}</span>
      </div>
      <div style="font-size: 0.78rem; color: #64748b;">📅 🕒: ${user.lastLogin || "-"}</div>
    </div>

    <div class="form-grid" style="margin-bottom: 1rem;">
      <div>
        <span style="font-size: 0.8rem; color: #64748b;">👤 📄📄📄📄</span>
        <strong style="direction: rtl; display: block; color: #0d9488;">${user.username}</strong>
      </div>
      <div>
        <span style="font-size: 0.8rem; color: #64748b;">📞 📄📄📄📄</span>
        <div style="display: flex; align-items: center; gap: 0.5rem;">
          <strong style="direction: rtl; display: block;">${passDisplay}</strong>
          <button class="btn btn-outline btn-sm" onclick="copyPasswordText('${user.password}')" title="📄📄 📄📄">📄</button>
        </div>
      </div>
      <div>
        <span style="font-size: 0.8rem; color: #64748b;">📞 📄📄📄📄📄📄</span>
        <span style="direction: rtl; display: block;">${user.phone || "-"}</span>
      </div>
      <div>
        <span style="font-size: 0.8rem; color: #64748b;">📱 📄📄📄📄📄📄</span>
        <span style="display: block; font-weight: 700;">${user.simControl}</span>
      </div>
    </div>

    <div style="background: #f8fafc; padding: 0.6rem 1rem; border-radius: 8px; font-size: 0.82rem; color: #475569; display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap; gap: 0.5rem;">
      <span>📞 📄📄📄📄 📄📄 📄📄📄📄: <strong>${user.phoneLock}</strong></span>
      <div style="display: flex; gap: 0.5rem;">
        <button class="btn btn-outline btn-sm" onclick="editUserCard('${user.id}')">📄 📄📄📄📄📄 📄📄 📄📄📄📄📄</button>
        <button class="btn btn-danger btn-sm" onclick="deleteUserCard('${user.id}')">📄 📄📄</button>
      </div>
    </div>
  `;

  container.appendChild(card);
});
}

function copyPasswordText(pwd) {
  navigator.clipboard.writeText(pwd).then(() => {
    alert('📄 📄📄 📄📄📄 «${pwd}» 📄 📄📄📄📄 📄📄 📄.');
  });
}

function editUserCard(id) {
  const u = state.users.find(x => x.id === id);
  if (!u) return;
  alert('📄📄📄📄 📄📄📄📄📄 📄📄📄📄 📄📄 «${u.fullName}» 📄 📄📄📄 📄📄📄 📄📄📄 📄📄 📄.');
  document.getElementById("newFullName").value = u.fullName;
  document.getElementById("newUsername").value = u.username;
  document.getElementById("newPassword").value = u.password;
  document.getElementById("newPhone").value = u.phone || "";
  document.getElementById("newRole").value = u.role;
  document.getElementById("newSimControl").value = u.simControl;
  document.getElementById("tab-users-permissions").scrollIntoView({ behavior: "smooth" });
}

function deleteUserCard(id) {
  if (!confirm("📄📄 📄 📄📄 📄📄 📄📄📄📄 📄📄📄📄")) return;
  state.users = state.users.filter(u => u.id !== id);
  saveState();
  renderUserCardsList();
  updateNavBadges();
}

```

```

}

// -----
// 11. 0000 0000000 000 (00000 0000 0000000000 0000000000 0000000000 00000 0000000 00000 0000)
// -----
function setupOtherSidebarModules() {
  // 1. 000000 00000000 (Menu Item 5)
  const btnRefAct = document.getElementById("btnRefreshActivity");
  if (btnRefAct) btnRefAct.addEventListener("click", () => renderActivityLogTable());
  renderActivityLogTable();

  // 2. 000 0000 (Menu Item 8)
  renderRepRoutesTable();

  // 3. 0000 0000000000 (Menu Item 9)
  renderRepHomesTable();

  // 4. 000000000 (Menu Item 10)
  const btnExportL = document.getElementById("btnExportLeavesCSV");
  if (btnExportL) {
    btnExportL.addEventListener("click", () => {
      const hdrs = ["00000000", "00 000000", "00 000000", "000 000000", "00000000"];
      const rws = state.leaves.map(l => [l.repName, l.fromDate, l.toDate, l.reason, l.status]);
      downloadCSVFile("leaves-export.csv", hdrs, rws);
    });
  }
  renderLeavesTable();

  // 5. 000000000 (Menu Item 11)
  renderNotificationsTable();

  // 6. 000000 0000000 (Menu Item 12)
  const btnExportM = document.getElementById("btnExportMonthlyCSV");
  if (btnExportM) {
    btnExportM.addEventListener("click", () => {
      const hdrs = ["00000000 0000", "0000000000", "000000000", "0000 00 (0000)"];
      const rws = state.reps.map(r => {
        const vCount = state.visits.filter(v => v.repName === r.name).length;
        const oCount = state.orders.filter(o => o.repName === r.name).length;
        const sum = state.orders.filter(o => o.repName === r.name).reduce((s, o) => s + (o.totalAmount || 0), 0);
        return [r.name, vCount, oCount, sum];
      });
      downloadCSVFile("monthly-reports.csv", hdrs, rws);
    });
  }
  renderMonthlyReportsTable();

  // 7. 000000 0000 (Menu Item 13)
  renderSalesTargetsTable();
}

function renderActivityLogTable() {
  const tbody = document.getElementById("tableActivityLogBody");
  if (!tbody) return;
  tbody.innerHTML = "";
  (state.activityLog || []).forEach(act => {
    const tr = document.createElement("tr");
    tr.innerHTML = `
      <td>${act.time}</td>
      <td><strong style="color:#0f172a;">${act.repName}</strong></td>
      <td>${act.action}</td>
      <td><span class="badge-status-online">000 00</span></td>
    `;
    tbody.appendChild(tr);
  });
}

function renderRepRoutesTable() {
  const tbody = document.getElementById("tableRepRoutesBody");
  if (!tbody) return;
  tbody.innerHTML = "";
  (state.repRoutes || []).forEach(rt => {
    const tr = document.createElement("tr");
    tr.innerHTML = `
      <td><strong style="color:#0f172a;">${rt.repName}</strong></td>
      <td>${rt.date}</td>
      <td>${rt.checkpoint}</td>
      <td><span class="badge-status-online">${rt.status}</span></td>
    `;
    tbody.appendChild(tr);
  });
}

function renderRepHomesTable() {
  const tbody = document.getElementById("tableRepHomesBody");

```


[illegible]


```
state.notifications.unshift(notif);
saveState();
formMsg.reset();
renderNotificationsTable();
playNotificationBeep();

const badge = document.getElementById("headerNotifBadgeCount") || document.querySelector(".notif-count-badge");
if (badge) {
    badge.style.display = "flex";
    const cur = parseInt(badge.textContent) || 0;
    badge.textContent = cur + 1;
}
alert('🔔 📬 📧 📩 «${recipient}» 📄 📑 🗂️ 📁 📂 📃 📅 📆 📇 📈 📉');
});
}
}

function renderNotificationsTable() {
    const tbody = document.getElementById("tableNotificationsBody");
    if (!tbody) return;
    tbody.innerHTML = "";
    (state.notifications || []).forEach(nt => {
        const tr = document.createElement("tr");
        const readBadge = nt.isRead
            ? `<span style="color:#64748b;font-size:0.8rem;">🟢🟢🟢🟢🟢🟢</span>`
            : `<button class="btn btn-outline btn-sm" onclick="markNotificationRead('${nt.id}')">🟡🟡🟡🟡</button>`;

        tr.innerHTML = `
            <td>${nt.date}</td>
            <td><strong style="color:#0d9488;">${nt.title}</strong></td>
            <td>${nt.message}</td>
            <td>${nt.sender} → ${nt.recipient || "🏠🏠🏠🏠"}</td>
            <td>${readBadge}</td>
            <td>
                <button class="btn btn-danger btn-sm" onclick="deleteNotification('${nt.id}')">❌</button>
            </td>
        `;
        tbody.appendChild(tr);
    });
}

function markNotificationRead(id) {
    const nt = (state.notifications || []).find(n => n.id === id);
    if (!nt) return;
    nt.isRead = true;
    saveState();
    renderNotificationsTable();
}

function deleteNotification(id) {
    if (!confirm("🗑️ 📄 📑 🗂️ 📁 📂 📃 📅 📆 📇 📈 📉")) return;
    state.notifications = state.notifications.filter(n => n.id !== id);
    saveState();
    renderNotificationsTable();
}

function renderMonthlyReportsTable() {
    const tbody = document.getElementById("tableMonthlyReportsBody");
    if (!tbody) return;
    tbody.innerHTML = "";
    state.reps.forEach(rep => {
        const vCount = state.visits.filter(v => v.repName === rep.name).length;
        const oCount = state.orders.filter(o => o.repName === rep.name).length;
        const sum = state.orders.filter(o => o.repName === rep.name).reduce((s, o) => s + (o.totalAmount || 0), 0);
        const tr = document.createElement("tr");
        tr.innerHTML = `
            <td><strong style="color:#0f172a;">${rep.name}</strong></td>
            <td><strong>${vCount}</strong> 🟢🟢🟢🟢</td>
            <td><strong>${oCount}</strong> 🟢🟢🟢🟢</td>
            <td><strong style="color:#0d9488;">${sum.toLocaleString("fa-IR")}</strong></td>
            <td><span class="badge-status-online">🟢🟢🟢</span></td>
        `;
        tbody.appendChild(tr);
    });
}

function renderSalesTargetsTable() {
    const tbody = document.getElementById("tableSalesTargetsBody");
    if (!tbody) return;
    tbody.innerHTML = "";
    (state.salesTargets || []).forEach(tgt => {
        const perc = Math.round((tgt.achievedAmount / tgt.targetAmount) * 100);
        const tr = document.createElement("tr");
        tr.innerHTML = `
            <td><strong style="color:#0f172a;">${tgt.repName}</strong></td>
        `;
    });
}
```

```

        <td>${tgt.month}</td>
        <td>${tgt.targetAmount.toLocaleString("fa-IR")}</td>
        <td><strong style="color:#0d9488;">${tgt.achievedAmount.toLocaleString("fa-IR")}</strong></td>
    </td><span class="badge-status-online">${perc}</span></td>
    `;
    tbody.appendChild(tr);
  });
}

// -----
// 12. ۰۰۰۰۰۰۰۰۰۰ ۰۰۰۰۰۰۰۰ (Orders + Requirement 3: Sticky Top Alert Banner)
// -----
function setupOrdersTab() {
  setupCascadingGeoSelectors("orderProvince", "orderCity", "orderDistrict");

  const form = document.getElementById("formOrder");
  const btnReset = document.getElementById("btnResetOrderForm");
  const btnAddItem = document.getElementById("btnAddOrderItemRow");
  const searchInput = document.getElementById("searchOrderInput");
  const pharmacyInput = document.getElementById("orderPharmacyName");
  const btnTopAutoFill = document.getElementById("btnTopAutoFillPharmacy");

  populateRepSelectorInOrders();
  populatePharmacyDataListInOrders();
  resetOrderForm();

  if (btnReset) btnReset.addEventListener("click", resetOrderForm);

  if (btnAddItem) {
    btnAddItem.addEventListener("click", () => {
      addOrderItemRow("", 10, 45000);
    });
  }

  if (searchInput) {
    searchInput.addEventListener("input", () => {
      renderOrdersList(searchInput.value.trim());
    });
  }

  const checkPharmacyMatch = () => {
    const val = pharmacyInput.value.trim();
    const matched = state.pharmacies.find(p => p.name === val || p.name.includes(val) && val.length >= 3);
    const alertBox = document.getElementById("existingPharmacyTopAlert");
    const alertText = document.getElementById("existingPharmacyAlertText");

    if (matched && val.length > 0) {
      document.getElementById("orderPharmacyMatchedId").value = matched.id;
      alertText.textContent = `۰۰۰۰۰۰۰۰ «${matched.name}» | ۰۰۰: ${matched.city} | ۰۰۰۰: ${matched.district} | ۰۰۰۰: ${matched.address}`;
      alertBox.style.display = "flex";
    } else {
      document.getElementById("orderPharmacyMatchedId").value = "";
      alertBox.style.display = "none";
    }
  };

  if (pharmacyInput) {
    pharmacyInput.addEventListener("input", checkPharmacyMatch);
    pharmacyInput.addEventListener("change", checkPharmacyMatch);
  }

  if (btnTopAutoFill) {
    btnTopAutoFill.addEventListener("click", () => {
      const matchedId = document.getElementById("orderPharmacyMatchedId").value;
      if (!matchedId) return;

      const matched = state.pharmacies.find(p => p.id === matchedId);
      if (!matched) return;

      const provEl = document.getElementById("orderProvince");
      const cityEl = document.getElementById("orderCity");
      const distEl = document.getElementById("orderDistrict");
      const addrEl = document.getElementById("orderAddress");

      provEl.value = matched.province;
      populateCities(matched.province, cityEl, matched.city);
      populateDistricts(matched.province, matched.city, distEl, matched.district);

      addrEl.value = matched.address;
      alert(`۰ ۰۰۰۰۰۰۰ ۰۰۰۰ ۰ ۰۰۰۰۰ ۰۰۰۰۰۰۰۰ «${matched.name}» ۰۰ ۰۰۰۰ ۰۰۰۰۰۰ ۰۰ ۰۰۰۰۰۰۰ ۰۰۰۰۰ ۰۰۰۰ ۰۰۰۰.`);
    });
  }

  if (form) {

```

```

form.addEventListener("submit", () => {
  const editId = document.getElementById("orderEditId").value;
  const pharmacyName = pharmacyInput.value.trim();
  const province = document.getElementById("orderProvince").value;
  const city = document.getElementById("orderCity").value;
  const district = document.getElementById("orderDistrict").value;
  const address = document.getElementById("orderAddress").value.trim();
  const repName = document.getElementById("orderRepName").value;
  const orderDate = document.getElementById("orderDate").value.trim() || new Date().toLocaleDateString("fa-IR");
  const status = document.getElementById("orderStatus").value;
  const notes = document.getElementById("orderNotes").value.trim();
  const customFieldsVals = extractCustomFieldValuesFromForm("order", "orderCustomFieldsContainer");

  if (!pharmacyName || !province || !city || !district) {
    alert("لطفاً نام داروخانه، استان، شهر و منطقه را وارد کنید.");
    return;
  }

  const items = getOrderItemsFromUI();
  if (items.length === 0 || !items[0].name) {
    alert("لطفاً حداقل یک مورد را به سبد خرید خود اضافه کنید.");
    return;
  }

  const totalAmount = items.reduce((sum, item) => sum + (item.count * item.price), 0);

  if (editId) {
    const idx = state.orders.findIndex(o => o.id === editId);
    if (idx !== -1) {
      state.orders[idx] = {
        ...state.orders[idx],
        pharmacyName, province, city, district, address,
        repName, orderDate, status, notes, items, totalAmount,
        customFields: customFieldsVals
      };
    }
    alert("اطلاعات سفارش شما با موفقیت به روز شد.");
  } else {
    const newOrder = {
      id: "ord-" + Date.now(),
      pharmacyName, province, city, district, address,
      repName, orderDate, status, notes, items, totalAmount,
      customFields: customFieldsVals
    };
    state.orders.push(newOrder);
    alert("سفارش شما با موفقیت ثبت شد.");
  }

  saveState();
  resetOrderForm();
  renderOrdersList();
  updateNavBadges();
});
}

function resetOrderForm() {
  document.getElementById("orderEditId").value = "";
  document.getElementById("orderPharmacyMatchedId").value = "";
  document.getElementById("orderPharmacyName").value = "";
  document.getElementById("orderAddress").value = "";
  document.getElementById("orderDate").value = new Date().toLocaleDateString("fa-IR");
  document.getElementById("orderNotes").value = "";
  document.getElementById("existingPharmacyTopAlert").style.display = "none";

  const provEl = document.getElementById("orderProvince");
  const cityEl = document.getElementById("orderCity");
  const distEl = document.getElementById("orderDistrict");

  provEl.value = "";
  cityEl.innerHTML = `<option value="">استان را انتخاب کنید</option>`;
  distEl.innerHTML = `<option value="">منطقه را انتخاب کنید</option>`;

  const container = document.getElementById("orderItemsContainer");
  if (container) {
    container.innerHTML = "";
    addOrderItemRow("", 10, 45000);
  }

  renderCustomFieldsInForm("order", "orderCustomFieldsContainer");
  updateOrderTotalAmountDisplay();
}

function populateRepSelectorInOrders() {
  const sel = document.getElementById("orderRepName");

```

```

    if (!sel) return;
    sel.innerHTML = `<option value="">□□□□□□ □□□□□□ □□□□...</option>`;
    state.reps.forEach(r => {
      const opt = document.createElement("option");
      opt.value = r.name;
      opt.textContent = `${r.name} (${r.region} || "□□□□□□□□ □□□□")`;
      sel.appendChild(opt);
    });
  }

function populatePharmacyDataListInOrders() {
  const datalist = document.getElementById("pharmacyNamesList");
  if (!datalist) return;
  datalist.innerHTML = "";
  state.pharmacies.forEach(p => {
    const opt = document.createElement("option");
    opt.value = p.name;
    opt.textContent = `${p.name} - ${p.province}/${p.city}`;
    datalist.appendChild(opt);
  });
}

function addOrderItemRow(name = "", count = 1, price = 0) {
  const container = document.getElementById("orderItemsContainer");
  if (!container) return;

  const rowId = "item-row-" + Date.now() + "-" + Math.floor(Math.random() * 1000);
  const row = document.createElement("div");
  row.id = rowId;
  row.style.display = "grid";
  row.style.gridTemplateColumns = "3fr 1fr 2fr auto";
  row.style.gap = "0.5rem";
  row.style.alignItems = "center";

  row.innerHTML = `
    <input type="text" class="form-input order-item-name" placeholder="□□□ □□□□ (□□□□: □□□□□)..." value="${name}" required />
    <input type="number" class="form-input order-item-count" placeholder="□□□□□ □□□□" min="1" value="${count}" required />
    <input type="number" class="form-input order-item-gift" placeholder="□□□□□ □□□□□ □" min="0" value="0" />
    <input type="number" class="form-input order-item-price" placeholder="□□□□ □□□□ (□□□□)" min="0" value="${price}" required />
    <button type="button" class="btn btn-danger btn-sm" onclick="removeOrderItemRow('${rowId}')">□□</button>
  `;

  const inputs = row.querySelectorAll("input");
  inputs.forEach(inp => {
    inp.addEventListener("input", updateOrderTotalAmountDisplay);
  });

  container.appendChild(row);
  updateOrderTotalAmountDisplay();
}

function removeOrderItemRow(rowId) {
  const row = document.getElementById(rowId);
  if (row) {
    row.remove();
    updateOrderTotalAmountDisplay();
  }
}

function getOrderItemsFromUI() {
  const container = document.getElementById("orderItemsContainer");
  if (!container) return [];
  const rows = container.children;
  const items = [];
  for (let i = 0; i < rows.length; i++) {
    const nameEl = rows[i].querySelector(".order-item-name");
    const countEl = rows[i].querySelector(".order-item-count");
    const priceEl = rows[i].querySelector(".order-item-price");
    if (nameEl && nameEl.value.trim()) {
      items.push({
        name: nameEl.value.trim(),
        count: parseInt(countEl.value) || 1,
        price: parseInt(priceEl.value) || 0
      });
    }
  }
  return items;
}

function updateOrderTotalAmountDisplay() {
  const items = getOrderItemsFromUI();
  const total = items.reduce((sum, item) => sum + (item.count * item.price), 0);

```

```

const disp = document.getElementById("orderTotalAmountDisplay");
if (disp) {
    disp.textContent = total.toLocaleString("fa-IR");
}
}

function renderOrdersList(searchQuery = "") {
    const theadTr = document.getElementById("tableOrdersHeader");
    const tbody = document.getElementById("tableOrdersBody");
    if (!theadTr || !tbody) return;

    theadTr.innerHTML = `
    <th>□□□□</th>
    <th>□□□ □□□□□□□□</th>
    <th>□□□□□ / □□□ / □□□□□</th>
    <th>□□□□□□□ □□□□</th>
    <th>□□□□□ □□□□□</th>
    <th>□□□□□ □□□□□</th>
    <th>□□□□□ □□□□□</th>
    <th>□□□□ □□ (□□□□)</th>
    <th>□□□□□</th>
    `;

    const fieldsList = (state.customFields.order || []).filter(f => f.showInList);
    fieldsList.forEach(f => {
        const th = document.createElement("th");
        th.textContent = f.label;
        theadTr.appendChild(th);
    });

    const thAction = document.createElement("th");
    thAction.textContent = "□□□□□□";
    theadTr.appendChild(thAction);

    tbody.innerHTML = "";

    const filtered = state.orders.filter(ord => {
        if (!searchQuery) return true;
        return ord.pharmacyName.includes(searchQuery) || (ord.repName && ord.repName.includes(searchQuery));
    });

    filtered.forEach((ord, index) => {
        const tr = document.createElement("tr");

        const statusBadge = ord.status === "□□□□□ □□□"
        ? `<span class="status-badge status-online">□ ${ord.status}</span>`
        : ord.status === "□□ □□□ □□□□□"
        ? `<span class="status-badge status-visiting">□ ${ord.status}</span>`
        : `<span class="status-badge" style="background:#f1f5f9;color:#475569;">${ord.status}</span>`;

        let rowHTML = `
        <td>${index + 1}</td>
        <td><strong style="color: #0d9488;">${ord.pharmacyName}</strong></td>
        <td>${ord.province} / ${ord.city} / ${ord.district}</td>
        <td>${ord.repName || "-"}</td>
        <td>${ord.orderDate || "-"}</td>
        <td>${ord.items ? ord.items.length : 0} □□□</td>
        <td><strong style="color: #0d9488;">${Number(ord.totalAmount || 0).toLocaleString("fa-IR")}</strong></td>
        <td>${statusBadge}</td>
        `;

        fieldsList.forEach(f => {
            const val = (ord.customFields && ord.customFields[f.label]) ? ord.customFields[f.label] : "-";
            rowHTML += `<td><strong>${val}</strong></td>`;
        });

        rowHTML += `
        <td>
            <div style="display: flex; gap: 0.4rem;">
            <button class="btn btn-outline btn-sm" onclick="editOrder('${ord.id}')">□□ □□□□□□</button>
            <button class="btn btn-danger btn-sm" onclick="deleteOrder('${ord.id}')">□□ □□□</button>
            </div>
        </td>
        `;

        tr.innerHTML = rowHTML;
        tbody.appendChild(tr);
    });
}

function editOrder(id) {
    const ord = state.orders.find(o => o.id === id);
    if (!ord) return;

    document.getElementById("orderEditId").value = ord.id;
    document.getElementById("orderPharmacyName").value = ord.pharmacyName;

```

```

document.getElementById("orderAddress").value = ord.address || "";
document.getElementById("orderRepName").value = ord.repName || "";
document.getElementById("orderDate").value = ord.orderDate || "";
document.getElementById("orderStatus").value = ord.status || "□□□□□□ □□□□";
document.getElementById("orderNotes").value = ord.notes || "";

const provEl = document.getElementById("orderProvince");
const cityEl = document.getElementById("orderCity");
const distEl = document.getElementById("orderDistrict");

provEl.value = ord.province;
populateCities(ord.province, cityEl, ord.city);
populateDistricts(ord.province, ord.city, distEl, ord.district);

const container = document.getElementById("orderItemsContainer");
container.innerHTML = "";
(ord.items || []).forEach(item => {
  addOrderItemRow(item.name, item.count, item.price);
});
updateOrderTotalAmountDisplay();

renderCustomFieldsInForm("order", "orderCustomFieldsContainer", ord.customFields || {});

document.getElementById("tab-orders").scrollIntoView({ behavior: "smooth" });
}

function deleteOrder(id) {
  if (!confirm("□□□□ □□ □□□□ □□□□□□ □□□□□□□□")) return;
  state.orders = state.orders.filter(o => o.id !== id);
  saveState();
  renderOrdersList();
  updateNavBadges();
}

// -----
// 13. □□□□□□□□□□ □□□□□ □: □□□□□□□□□□□□ □□□□□□ □□□□□□□□ □□□□ □□□□□□ □ □□□□□□□□
// -----
function setupBackupAndRestore() {
  const chkAuto = document.getElementById("chkAutoBackupEnabled");
  const btnSelectFolder = document.getElementById("btnSelectAutoBackupFolder");
  const btnManualNow = document.getElementById("btnManualBackupNow");
  const btnQuickBackup = document.getElementById("btnQuickBackup");
  const btnResetDefault = document.getElementById("btnResetDefault");

  if (chkAuto) {
    chkAuto.checked = state.settings ? state.settings.autoBackupEnabled : true;
    chkAuto.addEventListener("change", () => {
      state.settings.autoBackupEnabled = chkAuto.checked;
      saveState(false);
      updateAutoBackupStatusBadge();
    });
  }

  if (btnSelectFolder) {
    btnSelectFolder.addEventListener("click", async () => {
      if (window.showSaveFilePicker) {
        try {
          const handle = await window.showSaveFilePicker({
            suggestedName: "crm-backup-latest.json",
            types: [{
              description: "JSON CRM Backup File",
              accept: { "application/json": [".json"] }
            }]
          });
          autoBackupFileHandle = handle;
          document.getElementById("autoBackupHandleStatus").textContent = "□ □□□□ □□ □□□□: crm-backup-latest.json (□□□□□ □ □□□□□□)";
          await performAutoBackup();
          alert("□ □□□□ □□□□□□□□□□□□□□ □□□□□□ □□. □□ □□□ □□ □□ □□□□□□ □□□□ □□□□□□□□ □□□ □□ □□□ □□□□□□ □□□□□□ □□□□ □□□□ (crm-backup-latest.json) □□□□□□ □□.□");
        } catch (err) {
          console.warn("□□□□□□□ □□□□□□:", err);
        }
      } else {
        alert("□□□□□□□ □□□ □□ □□□□□□□□ File System Access API □□□□□□□□□□ □□□□□□□□ □□□ □□□□□□□□□□□□□□ □□ □□□□ □□□□□ □□ □□□□ □ □□□□□ □□□□□□□□.□");
      }
    });
  }

  const downloadSingleBackupFile = () => {
    const dataStr = "data:text/json;charset=utf-8," + encodeURIComponent(JSON.stringify(state, null, 2));
    const downloadAnchor = document.createElement("a");
    downloadAnchor.setAttribute("href", dataStr);
    downloadAnchor.setAttribute("download", "crm-backup-latest.json");
  }
}

```



```

        document.body.appendChild(downloadAnchor);
        downloadAnchor.click();
        downloadAnchor.remove();
    });

    if (btnManualNow) btnManualNow.addEventListener("click", downloadSingleBackupFile);
    if (btnQuickBackup) btnQuickBackup.addEventListener("click", downloadSingleBackupFile);

    if (btnResetDefault) {
        btnResetDefault.addEventListener("click", () => {
            if (confirm("آیا می‌خواهید تنظیمات را به حالت پیش‌فرض بازگردانید؟")) {
                localStorage.removeItem(STORAGE_KEY);
                location.reload();
            }
        });
    }

    if (autoBackupIntervalId) clearInterval(autoBackupIntervalId);
    autoBackupIntervalId = setInterval(() => {
        if (state.settings && state.settings.autoBackupEnabled) {
            performAutoBackup();
        }
    }, (state.settings.autoBackupIntervalMinutes || 5) * 60000);

    updateAutoBackupStatusBadge();
    setupRestoreSection();
}

async function performAutoBackup() {
    if (state.settings) state.settings.lastBackupTime = new Date().toLocaleString("fa-IR");

    if (autoBackupFileHandle) {
        try {
            const writable = await autoBackupFileHandle.createWritable();
            await writable.write(JSON.stringify(state, null, 2));
            await writable.close();
            console.log("فایل پشتیبان به موفقیت در crm-backup-latest.json ذخیره شد.");
        } catch (err) {
            console.warn("خطای در فرآیند پشتیبان‌گیری: ", err);
        }
    }
}

function updateAutoBackupStatusBadge() {
    const badge = document.getElementById("autoBackupStatusBadge");
    if (!badge) return;

    if (state.settings && state.settings.autoBackupEnabled) {
        badge.style.background = "rgba(16, 185, 129, 0.25)";
        badge.innerHTML = "<span>⬆️ پشتیبان‌گیری: موفق</span>";
    } else {
        badge.style.background = "rgba(245, 158, 11, 0.25)";
        badge.innerHTML = "<span>⬆️ پشتیبان‌گیری: ناموفق</span>";
    }
}

function setupRestoreSection() {
    const dropzone = document.getElementById("dropzoneRestore");
    const fileInput = document.getElementById("fileInputRestore");
    const previewCard = document.getElementById("restorePreviewCard");
    const btnCancel = document.getElementById("btnCancelRestore");
    const btnConfirm = document.getElementById("btnConfirmRestore");

    let tempRestoreData = null;

    if (!dropzone || !fileInput) return;

    dropzone.addEventListener("click", () => fileInput.click());

    dropzone.addEventListener("dragover", (e) => {
        e.preventDefault();
        dropzone.style.background = "#d9eaf7";
    });

    dropzone.addEventListener("dragleave", () => {
        dropzone.style.background = "#f0f0f4";
    });

    dropzone.addEventListener("drop", (e) => {
        e.preventDefault();
        dropzone.style.background = "#f0f0f4";
        if (e.dataTransfer.files && e.dataTransfer.files.length > 0) {
            handleRestoreFile(e.dataTransfer.files[0]);
        }
    });
}

```

```

fileInput.addEventListener("change", (e) => {
  if (e.target.files && e.target.files.length > 0) {
    handleRestoreFile(e.target.files[0]);
  }
});

const handleRestoreFile = (file) => {
  if (!file.name.endsWith(".json")) {
    alert("فقط فایل‌های با فرمت json مجاز است.");
    return;
  }

  const reader = new FileReader();
  reader.onload = (event) => {
    try {
      const parsed = JSON.parse(event.target.result);
      if (!parsed.pharmacies || !parsed.doctors || !parsed.orders) {
        alert("فایل JSON نامعتبر است. لطفاً فایل صحیح را بارگذاری کنید.");
        return;
      }

      tempRestoreData = parsed;
      document.getElementById("resCountPharmacies").textContent = parsed.pharmacies.length;
      document.getElementById("resCountDoctors").textContent = parsed.doctors.length;
      document.getElementById("resCountUsers").textContent = (parsed.users || []).length;
      document.getElementById("resCountOrders").textContent = parsed.orders.length;
      document.getElementById("resCountFields").textContent = Object.keys(parsed.customFields || {}).reduce((s, k) =>
s + (parsed.customFields[k].length || 0), 0);

      previewCard.style.display = "block";
      previewCard.scrollIntoView({ behavior: "smooth" });
    } catch (err) {
      alert("خطای در بارگذاری فایل JSON: " + err.message);
    }
  };
  reader.readAsText(file);
};

if (btnCancel) {
  btnCancel.addEventListener("click", () => {
    tempRestoreData = null;
    previewCard.style.display = "none";
    fileInput.value = "";
  });
}

if (btnConfirm) {
  btnConfirm.addEventListener("click", () => {
    if (!tempRestoreData) return;
    state = tempRestoreData;
    if (!state.users) state.users = DEFAULT_INITIAL_DATA.users;
    if (!state.settings) state.settings = DEFAULT_INITIAL_DATA.settings;

    saveState(false);
    previewCard.style.display = "none";
    fileInput.value = "";
    alert("عملیات با موفقیت انجام شد!");

    renderCustomFieldsTable();
    renderAllCustomFieldsInFormsAndTables();
    renderUserCardsList();
    renderDashboardOverviewMap();
    renderOrdersList();
    updateNavBadges();
    populateRepSelectorInOrders();
    populatePharmacyDataListInOrders();
  });
}

// -----
// 14. مدیریت ورود و خروج (Auth & Role Switching)
// -----
let currentRoleIndex = 0;
let currentUserName = "کاربر سیستم";

function setupAuthAndRoleSwitching() {
  const btnOpenLogin = document.getElementById("btnOpenLoginModal");
  const formLogin = document.getElementById("formLoginModal");
  const btnLogout = document.getElementById("btnLogoutSystem");

  if (btnOpenLogin) {
    btnOpenLogin.addEventListener("click", () => {
      openModalLogin();
    });
  }
}

```

```

    });
}

if (btnLogout) {
    btnLogout.addEventListener("click", () => {
        if (confirm("آیا می‌خواهید از سیستم خارج شوید؟")) {
            currentRoleIndex = 0;
            currentUserName = "کاربر سیستم";
            applyUserRolePermissions();
            openModalLogin();
            alert("با موفقیت از سیستم خارج شدید.");
        }
    });
}

if (formLogin) {
    formLogin.addEventListener("submit", () => {
        const roleIdx = parseInt(document.getElementById("loginRoleSelect").value);
        const selOpt = document.getElementById("loginRoleSelect").options[roleIdx];
        const simMode = document.getElementById("loginSimAuthMode") ? document.getElementById("loginSimAuthMode").value :
"3";
        currentRoleIndex = roleIdx;
        currentUserName = selOpt ? selOpt.textContent : "کاربر سیستم";

        if (simMode === "1") {
            const otp = prompt("لطفاً کد تایید (OTP) را وارد کنید. (کد نمونه: '123456')");
            if (!otp) return;
        } else if (simMode === "2") {
            alert("لطفاً کد تایید را از طریق پیامک یا اپلیکیشن ورودی خود وارد کنید.");
        }

        applyUserRolePermissions();
        closeModalLogin();
        alert("با موفقیت وارد سیستم شدید.");
    });
}

if (window.location.pathname.includes("/login") || window.location.hash.includes("login")) {
    setTimeout(openModalLogin, 500);
}

applyUserRolePermissions();
}

function openModalLogin() {
    const mod = document.getElementById("modalLogin");
    if (mod) mod.classList.add("active");
}

function closeModalLogin() {
    const mod = document.getElementById("modalLogin");
    if (mod) mod.classList.remove("active");
}

function applyUserRolePermissions() {
    const roleLabel = document.getElementById("currentUserRoleLabel");
    if (roleLabel) {
        roleLabel.textContent = `کاربر: ${currentUserName}`;
    }
}

// -----
// 15. تست اتصال به سرور و نمایش پیام (Screenshot 8)
// -----
function testServerConnectivity() {
    const box = document.getElementById("diagnosticsStatusBox");
    if (box) {
        box.style.background = "#fef9c3";
        box.style.color = "#854d0e";
        box.textContent = "تست اتصال به سرور در حال انجام... Render.com و Neon...";
    }

    setTimeout(() => {
        if (box) {
            box.style.background = "#f0fdf4";
            box.style.color = "#166534";
            box.textContent = "اتصال به سرور ناموفق! نام: namayandeelmi-javad.onrender.com | آدرس: ndcuhub.ir | خطای: 404";
        }
        alert("تست اتصال به سرور ناموفق! لطفاً دوباره تلاش کنید.");
    }, 700);
}

function retryServerConnection() {
    const banner = document.getElementById("diagnosticOfflineBanner");
    alert("تست اتصال به سرور ناموفق! لطفاً دوباره تلاش کنید.");
}

```

```

    if (banner) banner.style.display = "none";
}

// -----
// 16. 导出数据 (CSV Export)
// -----
function setupCSVExportButtons() {
    const btnPh = document.getElementById("btnExportPharmaciesCSV");
    const btnDoc = document.getElementById("btnExportDoctorsCSV");
    const btnOrd = document.getElementById("btnExportOrdersCSV");
    const btnUsers = document.getElementById("btnExportUsersCSV");

    if (btnPh) {
        btnPh.addEventListener("click", () => {
            const hdrs = ["药房名称", "电话", "地址", "省份", "城市", "区县", "是否连锁"];
            const rws = state.pharmacies.map(p => [p.name, p.phone || "", p.province, p.city, p.district, p.address, p.isPercentage ? "是" : "否"]);
            downloadCSVFile("pharmacies-export.csv", hdrs, rws);
        });
    }

    if (btnDoc) {
        btnDoc.addEventListener("click", () => {
            const hdrs = ["医生姓名", "科室", "电话", "地址", "省份", "城市", "区县", "是否连锁"];
            const rws = state.doctors.map(d => [d.name, d.specialty, d.phone || "", d.province, d.city, d.district, d.address, d.isPercentage ? "是" : "否"]);
            downloadCSVFile("doctors-export.csv", hdrs, rws);
        });
    }

    if (btnOrd) {
        btnOrd.addEventListener("click", () => {
            const hdrs = ["药房名称", "省份", "城市", "区县", "医生姓名", "医生电话", "医生地址", "医生是否连锁", "订单金额", "订单状态"];
            const rws = state.orders.map(o => [o.pharmacyName, o.province, o.city, o.district, o.repName || "", o.orderDate, o.totalAmount || 0, o.status]);
            downloadCSVFile("orders-export.csv", hdrs, rws);
        });
    }

    if (btnUsers) {
        btnUsers.addEventListener("click", () => {
            const hdrs = ["用户名", "密码", "角色", "是否模拟"];
            const rws = state.users.map(u => [u.fullName, u.username, u.role, u.phone || "", u.simControl]);
            downloadCSVFile("users-export.csv", hdrs, rws);
        });
    }
}

function downloadCSVFile(filename, headers, rows) {
    let csvContent = "\uFEFF";
    csvContent += headers.join(",") + "\n";
    rows.forEach(row => {
        const cleanRow = row.map(cell => `"${String(cell).replace(/"/g, '"')}"`);
        csvContent += cleanRow.join(",") + "\n";
    });
    const blob = new Blob([csvContent], { type: "text/csv;charset=utf-8;" });
    const url = URL.createObjectURL(blob);
    const a = document.createElement("a");
    a.setAttribute("href", url);
    a.setAttribute("download", filename);
    document.body.appendChild(a);
    a.click();
    document.body.removeChild(a);
}

// -----
// 17. 设置 PWA Service Worker
// -----
function setupPWAServiceWorker() {
    if ('serviceWorker' in navigator) {
        navigator.serviceWorker.register('/sw.js')
            .then(reg => {
                console.log('PWA 服务 Worker 注册成功');
            })
            .catch(err => {
                console.warn('PWA 服务 Worker 注册失败: ', err);
            });
    }
}

// -----
// 18. 匹配 Prompt 请求
// -----
let activeNavCoords = { lat: 35.7200, lng: 51.4200, name: "匹配" };

```

```

function openNavigationAppModal(lat, lng, nameText = "□□□□") {
    activeNavCoords = { lat: Number(lat) || 35.7200, lng: Number(lng) || 51.4200, name: nameText };
    const modal = document.getElementById("modalNavigationApps");
    if (modal) {
        modal.classList.add("active");
    }
}

function closeModalNavigationApps() {
    const modal = document.getElementById("modalNavigationApps");
    if (modal) {
        modal.classList.remove("active");
    }
}

function setupNavigationAppsModal() {
    const btnNeshan = document.getElementById("btnNavNeshan");
    const btnBalad = document.getElementById("btnNavBalad");
    const btnGoogle = document.getElementById("btnNavGoogle");
    const btnWaze = document.getElementById("btnNavWaze");

    if (btnNeshan) {
        btnNeshan.onclick = () => {
            window.open(`https://neshan.org/maps/@${activeNavCoords.lat},${activeNavCoords.lng},16z`, "_blank");
            closeModalNavigationApps();
        };
    }
    if (btnBalad) {
        btnBalad.onclick = () => {
            window.open(`https://balad.ir/location?latitude=${activeNavCoords.lat}&longitude=${activeNavCoords.lng}`, "_blank");
        };
        closeModalNavigationApps();
    }
    if (btnGoogle) {
        btnGoogle.onclick = () => {
            window.open(`https://www.google.com/maps/search/?api=1&query=${activeNavCoords.lat},${activeNavCoords.lng}`, "_blank");
        };
        closeModalNavigationApps();
    }
    if (btnWaze) {
        btnWaze.onclick = () => {
            window.open(`https://waze.com/ul?ll=${activeNavCoords.lat},${activeNavCoords.lng}&navigate=yes`, "_blank");
            closeModalNavigationApps();
        };
    }
}

// -----
// 19. □□□□□ □□□□ □□ □□□□□□ □□□□□□ □□ □□□□ □□ □ □□□□□□□□ (Requirement 10 & 11)
// -----
function setupSalesTargetsTab() {
    const form = document.getElementById("formSalesTarget");
    const repSel = document.getElementById("tgtRepSelect");
    const prodSel = document.getElementById("tgtProductSelect");
    const countInp = document.getElementById("tgtCountInput");
    const distDisp = document.getElementById("tgtCalcDistPrice");
    const phDisp = document.getElementById("tgtCalcPhPrice");

    const updateCalc = () => {
        if (!prodSel || !countInp) return;
        const prodId = prodSel.value;
        const cnt = parseInt(countInp.value) || 0;
        const prod = (state.products || []).find(p => p.name === prodId || p.id === prodId);
        if (prod) {
            const dPrice = (prod.distributorPrice || prod.price || 40000) * cnt;
            const pPrice = (prod.pharmacyPrice || prod.price || 45000) * cnt;
            if (distDisp) distDisp.textContent = `${dPrice.toLocaleString("fa-IR")} □□□□`;
            if (phDisp) phDisp.textContent = `${pPrice.toLocaleString("fa-IR")} □□□□`;
        } else {
            if (distDisp) distDisp.textContent = "0 □□□□";
            if (phDisp) phDisp.textContent = "0 □□□□";
        }
    };

    if (repSel) {
        repSel.innerHTML = `<option value="">□□□□□□ □□□□□□...</option>`;
        state.reps.forEach(r => {
            repSel.innerHTML += `<option value="${r.name}">${r.name}</option>`;
        });
    }

    if (prodSel) {
        prodSel.innerHTML = `<option value="">□□□□□□ □□□□...</option>`;
    }
}

```

```

    (state.products || []).forEach(p => {
      prodSel.innerHTML += `<option value="${p.name}">${p.name}</option>`;
    });
    prodSel.addEventListener("change", updateCalc);
  }

  if (countInp) {
    countInp.addEventListener("input", updateCalc);
  }

  if (form) {
    form.addEventListener("submit", () => {
      const repName = repSel.value;
      const productName = prodSel.value;
      const targetCount = parseInt(countInp.value) || 0;
      const year = document.getElementById("tgtYearInput").value || "1405";
      const month = document.getElementById("tgtMonthSelect").value || "000000";

      if (!repName || !productName || targetCount <= 0) {
        alert("000000 0000000000 000000 0 000000 000000 00 00000 00000.");
        return;
      }

      const prod = (state.products || []).find(p => p.name === productName);
      const totalDist = targetCount * (prod ? (prod.distributorPrice || prod.price || 40000) : 40000);
      const totalPh = targetCount * (prod ? (prod.pharmacyPrice || prod.price || 45000) : 45000);

      const newTgt = {
        id: "tgt-" + Date.now(),
        repName,
        productName,
        targetCount,
        achievedCount: 0,
        month: `${month} ${year}`,
        targetAmount: totalPh,
        achievedAmount: 0
      };

      if (!state.salesTargets) state.salesTargets = [];
      state.salesTargets.push(newTgt);
      saveState();
      form.reset();
      updateCalc();
      renderSalesTargetsTable();
      alert(`0 000000 00000 00000 «${repName}» 0000 000000 «${productName}» 0000 000.`);
    });
  }
}

// -----
// 20. 000000 00000 0000000000 (Audible Web Audio Beep for Notifications)
// -----
function playNotificationBeep() {
  try {
    const ctx = new (window.AudioContext || window.webkitAudioContext)();
    const osc = ctx.createOscillator();
    const gain = ctx.createGain();
    osc.type = "sine";
    osc.frequency.setValueAtTime(880, ctx.currentTime); // 00 A5
    gain.gain.setValueAtTime(0.15, ctx.currentTime);
    gain.gain.exponentialRampToValueAtTime(0.0001, ctx.currentTime + 0.35);
    osc.connect(gain);
    gain.connect(ctx.destination);
    osc.start();
    osc.stop(ctx.currentTime + 0.35);
  } catch (e) {
    console.warn("00000000 000000 0000 00000 0000000000 00 000000.");
  }
}

function setupNotificationsSoundAndClicks() {
  const bell = document.querySelector(".btn-header-icon");
  if (bell) {
    bell.addEventListener("click", () => {
      playNotificationBeep();
      switchTab("tab-notifications");
      (state.notifications || []).forEach(n => n.isRead = true);
      const badge = bell.querySelector(".notif-count-badge");
      if (badge) badge.style.display = "none";
    });
  }
}

// -----
// 21. 00000000000 00000 00 000000 000000 0 0000 00 00000 00000

```

```

// -----
function setupComprehensiveMapFilters() {
  const provSel = document.getElementById("mapFilterProvince");
  const citySel = document.getElementById("mapFilterCity");
  const btnFocus = document.getElementById("btnFocusMapRegion");

  if (provSel) {
    populateProvinces(provSel);
    provSel.addEventListener("change", () => {
      if (citySel) populateCities(provSel.value, citySel);
    });
  }

  if (btnFocus) {
    btnFocus.addEventListener("click", () => {
      const prov = provSel ? provSel.value : "";
      const city = citySel ? citySel.value : "";

      if (city === "□□□□□" || prov === "□□□□□") {
        if (mapFullOverview) mapFullOverview.setView([35.7200, 51.4200], 12);
        alert("□ □□□□□□□□□ □□□ □□□□□/□□□ □□□□□ □□□□□ □□.");
      } else if (city === "□□□□□" || prov === "□□□□□□□ □□□□") {
        if (mapFullOverview) mapFullOverview.setView([36.2970, 59.6062], 12);
        alert("□ □□□□□□□□□ □□□ □□□□ □□□□□ □□.");
      } else if (city === "□□□□□□□" || prov === "□□□□□□□") {
        if (mapFullOverview) mapFullOverview.setView([32.6546, 51.6680], 12);
        alert("□ □□□□□□□□□ □□□ □□□□□ □□□□□ □□.");
      } else {
        if (mapFullOverview) mapFullOverview.setView([35.7200, 51.4200], 6);
      }
    });
  }
}

// -----
// 22. □□□□ □□□□□□ □ □□□□□ □□□□□ □□□□□□ □□□□ □□□□□□□ □□□□ (□□ □□□□□ □□□□ □□ □□□□□ □□□)
// -----
function setupNetworkStatusMonitor() {
  const updateBadge = () => {
    const el = document.getElementById("globalOnlineStatusBadge");
    if (!el) return;
    if (navigator.onLine) {
      el.textContent = "□ □□□□□□□ (□□□□□ □□ □□□□□)";
      el.style.background = "#10b981";
    } else {
      el.textContent = "□ □□□□□□□ (□□□□□□ □□□□□)";
      el.style.background = "#ef4444";
    }
  };
  window.addEventListener("online", updateBadge);
  window.addEventListener("offline", updateBadge);
  updateBadge();
}

document.addEventListener("DOMContentLoaded", () => {
  try { loadState(); } catch(e) { console.error("error loadState:", e); }
  try { setupNetworkStatusMonitor(); } catch(e) { console.error("error netMonitor:", e); }
  try { setupAuthAndRoleSwitching(); } catch(e) { console.error("error auth:", e); }
  try { setupNotificationsSoundAndClicks(); } catch(e) { console.error("error notifClick:", e); }
  try { setupNavigationMenu(); } catch(e) { console.error("error nav:", e); }
  try { initMaps(); } catch(e) { console.error("error maps:", e); }

  try { setupCustomFieldsTab(); } catch(e) {}
  try { setupPharmacyTab(); } catch(e) {}
  try { setupDoctorTab(); } catch(e) {}
  try { setupRepsTab(); } catch(e) {}
  try { setupLiveLocationTab(); } catch(e) {}
  try { setupOrdersTab(); } catch(e) {}
  try { setupUsersAndPermissionsTab(); } catch(e) {}
  try { setupOtherSidebarModules(); } catch(e) {}
  try { setupBackupAndRestore(); } catch(e) {}
  try { setupAddOptionModalForm(); } catch(e) {}
  try { setupLeavesModule(); } catch(e) {}
  try { setupNotificationsMessaging(); } catch(e) {}

  try { setupJalaliDateAutoSlash(); } catch(e) {}
  try { setupJalaliCalendarPicker(); } catch(e) {}
  try { setupFormListSwitchers(); } catch(e) {}
  try { setupDropdownAutoClear(); } catch(e) {}
  try { setupCSVExportButtons(); } catch(e) {}
  try { setupNavigationAppsModal(); } catch(e) {}
  try { setupSalesTargetsTab(); } catch(e) {}
  try { setupComprehensiveMapFilters(); } catch(e) {}
  try { setupSearchInfoTab(); } catch(e) {}
  try { setupColumnsProductsTab(); } catch(e) {}

```

```

try { setupPWAServiceWorker(); } catch(e) {}

try { renderAllCustomFieldsInFormsAndTables(); } catch(e) {}
try { renderLeavesTable(); } catch(e) {}
try { renderNotificationsTable(); } catch(e) {}

try { setupAllFormSubmitHandlers(); } catch(e) { console.error("error submitHandlers:", e); }
console.log("📄 📄📄📄 📄📄📄📄 📄📄📄 📄📄 📄📄📄 📄📄📄📄📄 📄📄📄📄📄 📄📄📄📄📄📄 📄📄📄📄📄📄📄");
});

// =====
// 26. 📄📄📄📄 📄📄 📄📄📄📄📄📄📄 📄📄📄📄📄📄 📄📄📄📄📄 📄📄📄📄📄 📄📄📄 📄📄📄📄📄
// =====
function setupAllFormSubmitHandlers() {
  // 1. 📄📄📄📄 📄📄📄📄📄📄 (📄📄 📄📄📄📄 📄📄📄📄)
  const formPh = document.getElementById("formPharmacy");
  const btnPh = document.getElementById("btnSavePharmacy");
  const handleSavePh = () => {
    const editId = document.getElementById("pharmacyEditId").value;
    const dateAdded = document.getElementById("pharmacyDate").value.trim() || new Date().toLocaleDateString("fa-IR");
    const name = document.getElementById("pharmacyName").value.trim();
    const phone = document.getElementById("pharmacyPhone").value.trim();
    const manager = document.getElementById("pharmacyManager").value.trim();
    const province = document.getElementById("pharmacyProvince").value;
    const city = document.getElementById("pharmacyCity").value;
    const district = document.getElementById("pharmacyDistrict").value;
    const address = document.getElementById("pharmacyAddress").value.trim();
    const lat = parseFloat(document.getElementById("pharmacyLat").value) || 35.7605;
    const lng = parseFloat(document.getElementById("pharmacyLng").value) || 51.4180;
    const isPercentage = document.getElementById("pharmacyIsPercentage").value === "true";

    if (!name || !province || !city || !district || !address) {
      alert("📄📄📄📄 📄📄📄📄 📄📄📄📄📄 📄📄📄📄📄 (*) 📄📄 📄📄📄📄📄📄 📄📄 📄📄📄");
      return;
    }

    // 📄📄📄📄 📄📄📄📄 📄📄📄 📄📄📄📄📄 (Requirement 2 in prompt)
    if (!editId) {
      const isDup = state.pharmacies.some(p => p.name === name || (phone && p.phone === phone));
      if (isDup) {
        alert("📄📄 📄📄📄📄: 📄📄📄📄📄 «${name}» 📄📄📄📄 📄📄 📄📄📄📄 📄📄 📄📄📄📄!");
      }
    }

    if (editId) {
      const idx = state.pharmacies.findIndex(p => p.id === editId);
      if (idx !== -1) {
        state.pharmacies[idx] = { ...state.pharmacies[idx], dateAdded, name, phone, manager, province, city, district, address, lat, lng, isPercentage };
      }
      alert("📄 📄📄📄📄📄 «${name}» 📄📄 📄📄📄📄📄 📄📄📄📄 📄 📄📄📄📄📄");
    } else {
      state.pharmacies.push({
        id: "ph-" + Date.now(),
        dateAdded, name, phone, manager, province, city, district, address, lat, lng, isPercentage,
        repName: currentUser.name
      });
    }
    alert("📄 📄📄📄📄📄 «${name}» 📄📄 📄📄📄📄📄 📄📄 📄📄");
  };

  saveState();
  resetPharmacyForm();
  renderPharmaciesList();
  renderDashboardOverviewMap();
  updateNavBadges();
  // 📄📄📄📄📄 📄📄📄📄📄 📄📄 📄📄📄📄
  const btnList = document.getElementById("btnShowPhList");
  if (btnList) btnList.click();
};

if (btnPh) btnPh.onclick = (e) => { e.preventDefault(); handleSavePh(); };
if (formPh) formPh.onsubmit = (e) => { e.preventDefault(); handleSavePh(); };

// 2. 📄📄📄📄 📄📄📄 (📄📄 📄📄📄📄 📄📄📄📄 📄📄📄)
const formDoc = document.getElementById("formDoctor");
const btnDoc = document.getElementById("btnSaveDoctor");
const handleSaveDoc = () => {
  const editId = document.getElementById("doctorEditId").value;
  const dateAdded = document.getElementById("doctorDate").value.trim() || new Date().toLocaleDateString("fa-IR");
  const name = document.getElementById("doctorName").value.trim();
  const specialty = document.getElementById("doctorSpecialty").value.trim();
  const phone = document.getElementById("doctorPhone").value.trim();
  const province = document.getElementById("doctorProvince").value;
  const city = document.getElementById("doctorCity").value;
  const district = document.getElementById("doctorDistrict").value;

```



```

const address = document.getElementById("doctorAddress").value.trim();
const lat = parseFloat(document.getElementById("doctorLat").value) || 35.7580;
const lng = parseFloat(document.getElementById("doctorLng").value) || 51.4400;
const isPercentage = document.getElementById("doctorIsPercentage").value === "true";

if (!name || !specialty || !province || !city || !district || !address) {
  alert("نام و نام خانوادگی، تخصص، استان، شهر، منطقه و آدرس را وارد کنید.");
  return;
}

if (!editId) {
  const isDup = state.doctors.some(d => d.name === name || (phone && d.phone === phone));
  if (isDup) {
    alert("نام یا شماره تلفن این پزشک در سیستم موجود است.");
  }
}

if (editId) {
  const idx = state.doctors.findIndex(d => d.id === editId);
  if (idx !== -1) {
    state.doctors[idx] = { ...state.doctors[idx], dateAdded, name, specialty, phone, province, city, district, address, lat, lng, isPercentage };
  }
  alert("پزشک با نام " + name + " به درستی به سیستم اضافه شد.");
} else {
  state.doctors.push({
    id: "doc-" + Date.now(),
    dateAdded, name, specialty, phone, province, city, district, address, lat, lng, isPercentage,
    repName: currentUser.name
  });
}
alert("پزشک با نام " + name + " به درستی به سیستم اضافه شد.");

saveState();
resetDoctorForm();
renderDoctorsList();
renderDashboardOverviewMap();
updateNavBadges();
const btnList = document.getElementById("btnShowDocList");
if (btnList) btnList.click();
};

if (btnDoc) btnDoc.onclick = (e) => { e.preventDefault(); handleSaveDoc(); };
if (formDoc) formDoc.onsubmit = (e) => { e.preventDefault(); handleSaveDoc(); };

// 3. فرم سفارش (فرم سفارش را در اینجا تعریف می‌کنیم)
const formOrd = document.getElementById("formOrder");
const btnOrd = document.getElementById("btnSaveOrder");
const handleSaveOrd = () => {
  const editId = document.getElementById("orderEditId").value;
  const pharmacyName = document.getElementById("orderPharmacyName").value.trim();
  const province = document.getElementById("orderProvince").value;
  const city = document.getElementById("orderCity").value;
  const district = document.getElementById("orderDistrict").value;
  const address = document.getElementById("orderAddress").value.trim();
  const repName = document.getElementById("orderRepName").value || currentUser.name;
  const orderDate = document.getElementById("orderDate").value.trim() || new Date().toLocaleDateString("fa-IR");
  const status = document.getElementById("orderStatus").value;
  const notes = document.getElementById("orderNotes").value.trim();

  if (!pharmacyName || !province || !city || !district) {
    alert("نام داروخانه، استان، شهر، منطقه را وارد کنید.");
    return;
  }

  const items = getOrderItemsFromUI();
  if (items.length === 0 || !items[0].name) {
    alert("لطفاً حداقل یک دارو را به سفارش خود اضافه کنید.");
    return;
  }

  const totalAmount = items.reduce((sum, item) => sum + (item.count * item.price), 0);

  // برای محاسبه درصد تحقق (Requirement 1 in prompt)
  items.forEach(item => {
    const tgt = (state.salesTargets || []).find(t => t.repName === repName && t.productName === item.name);
    if (tgt) {
      tgt.achievedCount = (tgt.achievedCount || 0) + item.count;
      const prod = (state.products || []).find(p => p.name === item.name);
      tgt.achievedAmount = tgt.achievedCount * (prod ? (prod.pharmacyPrice || prod.price || 45000) : 45000);
    }
  });

  if (editId) {
    const idx = state.orders.findIndex(o => o.id === editId);

```

```

        if (idx !== -1) {
            state.orders[idx] = { ...state.orders[idx], pharmacyName, province, city, district, address, repName, orderDate,
status, notes, items, totalAmount };
        }
        alert('📄 📄📄📄 📄📄📄📄📄 «${pharmacyName}» 📄📄📄📄 📄.📄');
    } else {
        state.orders.push({
            id: "ord-" + Date.now(),
            pharmacyName, province, city, district, address, repName, orderDate, status, notes, items, totalAmount
        });
        alert('📄 📄📄📄 📄📄 📄📄 📄📄📄📄📄 «${pharmacyName}» 📄📄 📄.📄 📄📄📄📄📄📄 📄📄📄 📄📄📄📄 📄📄 📄📄📄.📄');
    }

    saveState();
    resetOrderForm();
    renderOrdersList();
    renderSalesTargetsTable();
    updateNavBadges();
    const btnList = document.getElementById("btnShowOrdList");
    if (btnList) btnList.click();
};

if (btnOrd) btnOrd.onclick = (e) => { e.preventDefault(); handleSaveOrd(); };
if (formOrd) formOrd.onsubmit = (e) => { e.preventDefault(); handleSaveOrd(); };
}

// =====
// 27. 📄📄📄📄 📄📄📄📄 📄 📄📄📄📄 📄📄📄 (Jalali Date Auto Slash - YYYY/MM/DD)
// =====
function setupJalaliDateAutoSlash() {
    const dateInputs = document.querySelectorAll("input[placeholder*='1405']", #pharmacyDate, #doctorDate, #orderDate, #vis
itDate, #leaveFromDate, #leaveToDate");
    dateInputs.forEach(inp => {
        if (inp.dataset.slashBound === "true") return;
        inp.dataset.slashBound = "true";

        inp.addEventListener("input", function(e) {
            let val = this.value.replace(/^[^0-9]/g, "");
            if (val.length > 4 && val.length <= 6) {
                this.value = val.slice(0, 4) + "/" + val.slice(4);
            } else if (val.length > 6) {
                this.value = val.slice(0, 4) + "/" + val.slice(4, 6) + "/" + val.slice(6, 8);
            } else {
                this.value = val;
            }
        });

        if (!inp.value) {
            inp.value = new Date().toLocaleDateString("fa-IR");
        }
    });
}

// =====
// 28. 📄📄📄📄📄 📄 📄📄 📄📄 📄 📄📄📄📄📄📄📄📄 📄📄📄📄 📄 📄📄📄📄📄 (Form/List Switcher)
// =====
function setupFormListSwitchers() {
    const switchCard = (btnFormId, btnListId, formCardId, listCardId) => {
        const bF = document.getElementById(btnFormId);
        const bL = document.getElementById(btnListId);
        const cF = document.getElementById(formCardId);
        const cL = document.getElementById(listCardId);
        if (!bF || !bL || !cF || !cL) return;

        bF.addEventListener("click", () => {
            bF.className = "btn btn-primary btn-sm";
            bF.style.background = "#0d9488";
            bL.className = "btn btn-outline btn-sm";
            cF.style.display = "block";
            cL.style.display = "none";
        });

        bL.addEventListener("click", () => {
            bL.className = "btn btn-primary btn-sm";
            bL.style.background = "#0d9488";
            bF.className = "btn btn-outline btn-sm";
            cF.style.display = "none";
            cL.style.display = "block";
        });
    };

    switchCard("btnShowPhForm", "btnShowPhList", "cardPhForm", "cardPhList");
    switchCard("btnShowDocForm", "btnShowDocList", "cardDocForm", "cardDocList");
    switchCard("btnShowOrdForm", "btnShowOrdList", "cardOrdForm", "cardOrdList");
}

```

```

// =====
// 29. ۰۰۰۰۰۰ ۰۰۰۰۰۰ ۰۰۰۰ ۰۰ ۰۰۰۰۰۰ ۰ ۰۰۰۰۰۰ ۰۰۰ (Matching Screenshot Date Picker)
// =====
let activeDateInputForPicker = null;

function setupJalaliCalendarPicker() {
  const popup = document.getElementById("jalaliCalendarPopup");
  if (!popup) return;

  const dateInputs = document.querySelectorAll("#pharmacyDate, #doctorDate, #orderDate, #visitDate, #leaveFromDate, #leaveToDate");
  dateInputs.forEach(inp => {
    if (inp.dataset.pickerBound === "true") return;
    inp.dataset.pickerBound = "true";

    // ۰۰۰۰ ۰۰۰۰ ۰۰۰۰۰۰ ۰۰۰۰ ۰۰ ۰۰۰ ۰۰۰۰/۰۰ ۰۰۰۰ (۰۰۰۰۰۰ ۰۰ ۰۰۰۰۰۰۰۰۰۰)
    const wrapper = document.createElement("div");
    wrapper.className = "jalali-input-wrapper";
    inp.parentNode.insertBefore(wrapper, inp);
    wrapper.appendChild(inp);

    const badge = document.createElement("div");
    badge.className = "jalali-badge";
    badge.title = "۰۰۰۰۰۰ ۰۰۰۰۰ ۰۰ ۰۰۰۰۰";
    badge.innerHTML = `
      <span class="jalali-badge-header">July</span>
      <span class="jalali-badge-day">17</span>
    `;
    wrapper.insertBefore(badge, inp);

    const openPicker = (e) => {
      e.stopPropagation();
      activeDateInputForPicker = inp;
      renderJalaliCalendarDays();
      const rect = inp.getBoundingClientRect();
      popup.style.top = (window.scrollY + rect.bottom + 6) + "px";
      popup.style.left = (window.scrollX + rect.left) + "px";
      popup.classList.add("active");
    };

    badge.addEventListener("click", openPicker);
    inp.addEventListener("click", openPicker);
  });

  const btnPrev = document.getElementById("jalaliPrevMonth");
  const btnNext = document.getElementById("jalaliNextMonth");
  const monthSel = document.getElementById("jalaliMonthSelect");
  const yearSel = document.getElementById("jalaliYearSelect");
  const btnToday = document.getElementById("jalaliTodayBtn");

  if (monthSel && yearSel) {
    monthSel.onchange = () => renderJalaliCalendarDays();
    yearSel.onchange = () => renderJalaliCalendarDays();
  }

  if (btnPrev && monthSel) {
    btnPrev.onclick = () => {
      let idx = monthSel.selectedIndex;
      if (idx > 0) { monthSel.selectedIndex = idx - 1; renderJalaliCalendarDays(); }
    };
  }

  if (btnNext && monthSel) {
    btnNext.onclick = () => {
      let idx = monthSel.selectedIndex;
      if (idx < monthSel.options.length - 1) { monthSel.selectedIndex = idx + 1; renderJalaliCalendarDays(); }
    };
  }

  if (btnToday) {
    btnToday.onclick = () => {
      if (activeDateInputForPicker) {
        activeDateInputForPicker.value = "1405/05/21";
      }
      popup.classList.remove("active");
    };
  }

  document.addEventListener("click", (e) => {
    if (!popup.contains(e.target) && !e.target.classList.contains("jalali-badge") && e.target !== activeDateInputForPicker) {
      popup.classList.remove("active");
    }
  });
}

```

```

}

function renderJalaliCalendarDays() {
  const grid = document.getElementById("jalaliDaysGrid");
  const monthSel = document.getElementById("jalaliMonthSelect");
  const yearSel = document.getElementById("jalaliYearSelect");
  if (!grid) return;

  grid.innerHTML = "";
  const year = yearSel ? yearSel.value : "1405";
  const month = monthSel ? monthSel.value : "05";

  // ۰۰۰۰۰۰ ۰۰ ۰۰۰۰ ۰۰۰۰ ۰۰۰۰ ۰۰۰۰ ۰۰
  for (let i = 0; i < 2; i++) {
    const empty = document.createElement("div");
    empty.className = "jalali-day-cell empty";
    grid.appendChild(empty);
  }

  const persianNumbers = ["۰", "۱", "۲", "۳", "۴", "۵", "۶", "۷", "۸", "۹", "۰۰", "۰۱", "۰۲", "۰۳", "۰۴", "۰۵", "۰۶", "۰۷", "۰۸", "۰۹", "۱۰", "۱۱", "۱۲", "۱۳", "۱۴", "۱۵", "۱۶", "۱۷", "۱۸", "۱۹", "۲۰", "۲۱", "۲۲", "۲۳", "۲۴", "۲۵", "۲۶", "۲۷", "۲۸", "۲۹", "۳۰", "۳۱"];

  for (let d = 1; d <= 31; d++) {
    const btn = document.createElement("button");
    btn.type = "button";
    btn.className = "jalali-day-cell" + (d === 21 ? " active" : "");
    btn.textContent = persianNumbers[d - 1] || d;

    btn.onclick = (e) => {
      e.stopPropagation();
      const dStr = d < 10 ? "0" + d : String(d);
      if (activeDateInputForPicker) {
        activeDateInputForPicker.value = `${year}/${month}/${dStr}`;
      }
      document.getElementById("jalaliCalendarPopup").classList.remove("active");
    };

    grid.appendChild(btn);
  }
}

// =====
// 30. ۰۰ ۰۰۰۰۰۰ ۰۰۰۰۰۰۰ ۰۰۰۰۰۰۰ (Search Information Tab - Section 6 Requirement)
// =====
let mapSearchInfoInstance = null;
let markersSearchInfo = [];

function setupSearchInfoTab() {
  const btnRun = document.getElementById("btnRunSearchInfo");
  const btnExp = document.getElementById("btnExportSearchInfoCSV");
  const inpPh = document.getElementById("searchInfoPhInput");
  const inpDoc = document.getElementById("searchInfoDocInput");
  const inpHosp = document.getElementById("searchInfoHospInput");

  const runSearch = () => {
    const qPh = (inpPh ? inpPh.value.trim() : "").toLowerCase();
    const qDoc = (inpDoc ? inpDoc.value.trim() : "").toLowerCase();
    const qHosp = (inpHosp ? inpHosp.value.trim() : "").toLowerCase();

    const results = [];

    // ۰۰۰۰۰ ۰۰ ۰۰۰۰۰۰۰۰۰۰
    state.pharmacies.forEach((p, idx) => {
      if (qPh && !p.name.toLowerCase().includes(qPh)) return;
      if (!qPh && !qDoc && !qHosp) {} // ۰۰۰ ۰۰۰۰ ۰۰۰۰ ۰۰ ۰۰۰۰ ۰۰۰۰ ۰۰
      results.push({
        index: idx + 1,
        repName: p.repName || "۰۰۰۰ ۰۰۰۰۰",
        type: "۰۰۰۰۰۰۰۰ ۰",
        name: p.name,
        city: `${p.province} / ${p.city}`,
        address: p.address,
        lat: p.lat,
        lng: p.lng,
        raw: p,
        entityType: "pharmacy"
      });
    });
  };

  // ۰۰۰۰۰ ۰۰ ۰۰۰۰۰۰
  state.doctors.forEach((d, idx) => {
    if (qDoc && !d.name.toLowerCase().includes(qDoc)) return;
    results.push({
      index: idx + 1,
      repName: d.repName || "۰۰۰۰ ۰۰۰۰۰۰",
    });
  });
}

```

```

    type: "药房 药房",
    name: d.name,
    city: `${d.province} / ${d.city}`,
    address: d.address,
    lat: d.lat,
    lng: d.lng,
    raw: d,
    entityType: "doctor"
  });
});
renderSearchInfoResults(results);
};

if (btnRun) btnRun.addEventListener("click", runSearch);
if (inpPh) inpPh.addEventListener("input", runSearch);
if (inpDoc) inpDoc.addEventListener("input", runSearch);
if (inpHosp) inpHosp.addEventListener("input", runSearch);

if (btnExp) {
  btnExp.addEventListener("click", () => {
    const hdrs = ["药房", "药房 药房", "药房 药房", "药房 药房", "药房/药房", "药房"];
    const rws = [];
    state.pharmacies.forEach((p, idx) => rws.push([idx + 1, p.repName || "-", "药房", p.name, p.city, p.address]));
;
    state.doctors.forEach((d, idx) => rws.push([idx + 1, d.repName || "-", "药房", d.name, d.city, d.address]));
    downloadCSVFile("search-info-export.csv", hdrs, rws);
  });
}
}

function renderSearchInfoResults(results) {
  const tbody = document.getElementById("tableSearchInfoBody");
  if (!tbody) return;
  tbody.innerHTML = "";

  if (!mapSearchInfoInstance && document.getElementById("map-search-info")) {
    mapSearchInfoInstance = L.map("map-search-info").setView([35.7200, 51.4200], 11);
    L.tileLayer("https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png", {
      maxZoom: 19,
      attribution: "© OpenStreetMap contributors"
    }).addTo(mapSearchInfoInstance);
  }

  if (mapSearchInfoInstance) {
    markersSearchInfo.forEach(m => mapSearchInfoInstance.removeLayer(m));
    markersSearchInfo = [];
  }

  const allPoints = [];

  results.forEach(res => {
    const tr = document.createElement("tr");
    tr.style.cursor = "pointer";
    tr.onclick = () => openRowDetailsModal(res.raw, res.entityType);

    tr.innerHTML = `
      <td>${res.index}</td>
      <td><strong style="color:#0f172a;">${res.repName}</strong></td>
      <td><span class="badge-status-online" style="background:#0d9488;">${res.type}</span></td>
      <td><strong style="color:#0d9488;">${res.name}</strong></td>
      <td>${res.city}</td>
      <td>${res.address}</td>
      <td>
        <button class="btn btn-outline btn-sm" onclick="event.stopPropagation(); openRowDetailsModal(res.raw, '${res.entityType}')">药房 药房 药房 药房</button>
      </td>
    `;
    tbody.appendChild(tr);

    if (res.lat && res.lng && mapSearchInfoInstance) {
      const m = createCustomMarker(res.lat, res.lng, res.entityType === "pharmacy" ? "pharmacy" : "doctor", res.name, mapSearchInfoInstance, () => {
        openRowDetailsModal(res.raw, res.entityType);
      });
      markersSearchInfo.push(m);
      allPoints.push([res.lat, res.lng]);
    }
  });

  if (allPoints.length > 0 && mapSearchInfoInstance) {
    mapSearchInfoInstance.fitBounds(allPoints, { padding: [40, 40] });
  }
}

```

```

// =====
// 31. 000000 0000000 000000 0 000000 0000000 00 00000 0000000 000000 0000000 (Section 10)
// =====
let activeModalRowObject = null;
let activeModalRowType = null;
let mapRowDetailsMiniInstance = null;
let markerRowDetailsMini = null;

function openRowDetailsModal(rowObj, entityType) {
    activeModalRowObject = rowObj;
    activeModalRowType = entityType;

    const modal = document.getElementById("modalRowDetails");
    const textBox = document.getElementById("rowDetailsContentBox");
    if (!modal || !textBox) return;

    let detailsHTML = "";
    let shareText = "";

    if (entityType === "pharmacy") {
        detailsHTML = `
        <div style="font-weight: 800; font-size: 1.15rem; color: #0d9488; margin-bottom: 0.5rem;">00000000: ${rowObj.name}</div>
        <div><strong>00000000 0000:</strong> ${rowObj.repName || "00000 000000"}</div>
        <div><strong>000000 / 0000 / 000000:</strong> ${rowObj.province} / ${rowObj.city} / ${rowObj.district || "-"}</div>
        <div><strong>00000 0000:</strong> ${rowObj.address}</div>
        <div><strong>000000 0000:</strong> <span style="direction:ltr; display:inline-block;">${rowObj.phone || "-"}</span>
        </div>
        <div><strong>000000:</strong> ${rowObj.isPercentage ? "0000": "0000"}</div>
        <div><strong>00000 000000:</strong> ${rowObj.fileName || "000000"}</div>
        `;
        shareText = `0000000000: ${rowObj.name}\\n0000000000: ${rowObj.repName || "-"}\\n000000: ${rowObj.address}\\n000000: ${rowObj.phone || "-"}`;
    } else if (entityType === "doctor") {
        detailsHTML = `
        <div style="font-weight: 800; font-size: 1.15rem; color: #10b981; margin-bottom: 0.5rem;">00000 00000 / 0000: ${rowObj.name}</div>
        <div><strong>00000:</strong> ${rowObj.specialty}</div>
        <div><strong>000000000 0000:</strong> ${rowObj.repName || "00000 0000000"}</div>
        <div><strong>000000 / 0000 / 000000:</strong> ${rowObj.province} / ${rowObj.city} / ${rowObj.district || "-"}</div>
        <div><strong>00000 0000:</strong> ${rowObj.address}</div>
        <div><strong>000000 0000:</strong> <span style="direction:ltr; display:inline-block;">${rowObj.phone || "-"}</span>
        </div>
        `;
        shareText = `00000 00000: ${rowObj.name} (${rowObj.specialty})\\n0000000000: ${rowObj.repName || "-"}\\n000000: ${rowObj.address}`;
    } else if (entityType === "order") {
        detailsHTML = `
        <div style="font-weight: 800; font-size: 1.15rem; color: #2563eb; margin-bottom: 0.5rem;">000000 0000000000: ${rowObj.pharmacyName}</div>
        <div><strong>000000000 0000:</strong> ${rowObj.repName || "-"}</div>
        <div><strong>000000 000000:</strong> ${rowObj.orderDate}</div>
        <div><strong>00000 0000 00:</strong> <strong style="color:#0d9488;">${(Number(rowObj.totalAmount || 0).toLocaleString("fa-IR"))} 00000</strong></div>
        <div><strong>000000:</strong> ${rowObj.status}</div>
        <div><strong>000000 0 000000 000000 0:</strong>
        <ul style="margin: 0.3rem 1.5rem 0 0;">
        ${((rowObj.items || []).map(i => `<li>${i.name} (000000: ${i.count} | 000000 0: ${i.giftCount || 0})</li>`)).join("")}
        </ul>
        </div>
        `;
        shareText = `0000000: ${rowObj.pharmacyName}\\n00000000: ${rowObj.orderDate}\\n000000: ${(Number(rowObj.totalAmount || 0).toLocaleString("fa-IR"))} 00000`;
    }

    textBox.innerHTML = detailsHTML;

    // 000000 0000000000 0000000000000000 (00000 000000 0000000000 000000 0000000)
    const bindShare = (btnId, urlPrefix) => {
        const btn = document.getElementById(btnId);
        if (btn) {
            btn.onclick = () => {
                window.open(`${urlPrefix}${encodeURIComponent(shareText)}`, "_blank");
            };
        }
    };

    bindShare("btnShareBale", "https://ble.ir/share?text=");
    bindShare("btnShareEitaa", "https://eitaa.com/share/url?url=&text=");
    bindShare("btnShareTelegram", "https://t.me/share/url?url=&text=");
    bindShare("btnShareSoroush", "https://splus.ir/share?text=");
    bindShare("btnShareWhatsApp", "https://api.whatsapp.com/send?text=");

    const btnCopy = document.getElementById("btnRowCopyText");

```

```

if (btnCopy) {
  btnCopy.onclick = () => {
    navigator.clipboard.writeText(shareText).then(() => {
      alert("📄 📄📄📄📄📄 📄 📄📄📄📄 📄📄.");
    });
  };
}

const btnEdit = document.getElementById("btnRowEdit");
if (btnEdit) {
  btnEdit.onclick = () => {
    closeModalRowDetails();
    if (entityType === "pharmacy") editPharmacy(rowObj.id);
    else if (entityType === "doctor") editDoctor(rowObj.id);
    else if (entityType === "order") editOrder(rowObj.id);
  };
}

const btnDel = document.getElementById("btnRowDelete");
if (btnDel) {
  btnDel.onclick = () => {
    if (confirm("📄📄 📄 📄📄 📄📄📄📄 📄📄📄📄")) {
      closeModalRowDetails();
      if (entityType === "pharmacy") deletePharmacy(rowObj.id);
      else if (entityType === "doctor") deleteDoctor(rowObj.id);
      else if (entityType === "order") deleteOrder(rowObj.id);
    }
  };
}

modal.classList.add("active");

setTimeout(() => {
  if (!mapRowDetailsMiniInstance && document.getElementById("rowDetailsMiniMap")) {
    mapRowDetailsMiniInstance = L.map("rowDetailsMiniMap").setView([rowObj.lat || 35.7200, rowObj.lng || 51.4200], 15);
    L.tileLayer("https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png", {
      maxZoom: 19,
      attribution: "© OpenStreetMap contributors"
    }).addTo(mapRowDetailsMiniInstance);
  }
  if (mapRowDetailsMiniInstance) {
    mapRowDetailsMiniInstance.invalidateSize();
    if (markerRowDetailsMini) mapRowDetailsMiniInstance.removeLayer(markerRowDetailsMini);
    markerRowDetailsMini = L.marker([rowObj.lat || 35.7200, rowObj.lng || 51.4200]).addTo(mapRowDetailsMiniInstance);
    mapRowDetailsMiniInstance.setView([rowObj.lat || 35.7200, rowObj.lng || 51.4200], 15);
  }
}, 200);
}

function closeModalRowDetails() {
  const modal = document.getElementById("modalRowDetails");
  if (modal) modal.classList.remove("active");
}

// =====
// 32. 📄📄📄📄 📄📄📄📄📄 📄 📄📄📄📄 (Columns & Products Catalog - Section 9 & 11)
// =====
function setupColumnsProductsTab() {
  const formProd = document.getElementById("formProduct");
  const btnProd = document.getElementById("btnSaveProduct");

  const handleSaveProd = () => {
    const name = document.getElementById("productName").value.trim();
    const category = document.getElementById("productCategory").value.trim();
    const distPrice = parseInt(document.getElementById("productDistPrice") ? document.getElementById("productDistPrice").value : 40000) || 40000;
    const phPrice = parseInt(document.getElementById("productPrice") ? document.getElementById("productPrice").value : 45000) || 45000;
    const stock = parseInt(document.getElementById("productStock") ? document.getElementById("productStock").value : 5000) || 5000;

    if (!name || !category) {
      alert("📄📄📄 📄📄 📄📄📄 📄 📄📄 📄📄📄📄.");
      return;
    }

    if (!state.products) state.products = [];
    const idx = state.products.findIndex(p => p.name === name);
    if (idx !== -1) {
      state.products[idx] = { ...state.products[idx], category, distributorPrice: distPrice, pharmacyPrice: phPrice, stock };
    } else {
      state.products.push({

```

```
id: "prod-" + Date.now(),
name,
category,
distributorPrice: distPrice,
pharmacyPrice: phPrice,
stock,
description: `□□□□ □□□: ${distPrice.toLocaleString("fa-IR")} □□□□ | □□□□ □□□□□□□□: ${phPrice.toLocaleString("fa-IR")} □□□□`
});
alert(`□ □□□□□ □□□□ «${(name)}» □□ □□□□ □□□□□□ □□□□□ □□.`);
}

saveState();
if (formProd) formProd.reset();
renderColumnsProductsTable();
setupSalesTargetsTab();
updateNavBadges();
};

if (btnProd) btnProd.onclick = (e) => { e.preventDefault(); handleSaveProd(); };
if (formProd) formProd.onsubmit = (e) => { e.preventDefault(); handleSaveProd(); };

renderColumnsProductsTable();
}

function renderColumnsProductsTable() {
const tbody = document.getElementById("tableProductsBody");
if (!tbody) return;
tbody.innerHTML = "";
(state.products || []).forEach((prod, index) => {
const tr = document.createElement("tr");
tr.innerHTML = `
<td>${index + 1}</td>
<td><strong style="color:#0f172a;">${(prod.name)}</strong></td>
<td><span class="badge status-online" style="background:#0d9488;">${(prod.category)}</span></td>
<td><strong style="color:#1e40af;">$(Number(prod.distributorPrice || 40000).toLocaleString("fa-IR")) □□□□</strong> </td>
<td><strong style="color:#0d9488;">$(Number(prod.pharmacyPrice || 45000).toLocaleString("fa-IR")) □□□□</strong> </td>
<td>${(prod.stock || 5000)} □□□</td>
<td>
<button class="btn btn-danger btn-sm" onclick="deleteProductCatalogItem('${(prod.id)}')">□□ □□</button>
</td>
`;
tbody.appendChild(tr);
});
}

function deleteProductCatalogItem(id) {
if (!confirm("□□□□ □□ □□□ □□□ □□□□□□□□ □□□□□□")) return;
state.products = state.products.filter(p => p.id !== id);
saveState();
renderColumnsProductsTable();
setupSalesTargetsTab();
updateNavBadges();
}
```

@@ public/crm-data.js javascript

// =====
// □□□□□□□□ □□□□ □□□□□□ □□□□□□ □□□□□□ □□□□□□□□□□□□□□ □□□□□□ □ □□□□□□□□
// □□□□ □□□□□□□ □□□□□□□□□□ □□□□ □□□□□□ □□□□□□□□□□ □□□□□□ □□□□ □□□□ □□□□□□
// □ □□□□□□ □□□□ □□ □□□□□□ □□□□□□ □□□□□□□ □□□□□ □□□□□ □ □□□□ □□ □□□ □□□□□□□
// =====

const IRAN_GEO_DATA = {
 "□□□□": {
 "□□□□□": ["□□□□□ □", "□□□□□ □", "□□□□□ □", "□□□□□ □", "□□□□□ □", "□□□□□ □", "□□□□□ □", "□□□□□ □", "□□□□□ □", "□□□□□ □", "□□□□□ □", "□□□□□ □", "□□□□□ □", "□□□□□ □", "□□□□□ □", "□□□□□ □", "□□□□□ □", "□□□□□ □"],
 "□□□□□□□□": ["□□□□□ □ (□□□□□)", "□□□□□ □ (□□□□□□)", "□□□□□ □ (□□□)", "
 "□□□ □□": ["□□□□□ □ (□□□□□)", "□□□□□ □ (□□□□□□□□)", "□□□□□ □ (□□□□□□)", "
 "□□□□□□□□□□": ["□□□□□ □", "□□□□□ □", "□□□□□ □", "□□□□□ □"],
 "□□□□□□": ["□□□□□ □", "□□□□□ □", "□□□□□ □", "□□□□□ □"],
 "□□□": ["□□□□□ □", "□□□□□ □"],
 "□□□□□": ["□□□□□ □", "□□□□□ □"],
 "□□□□□□": ["□□□□□ □", "□□□□□ □", "□□□□□ □"],
 "□□□□□□": ["□□□□□ □", "□□□□□ □"],
 "□□□□□": ["□□□□□ □", "□□□□□ □"],
 "□□□□□□": ["□□□□□ □", "□□□□□ □"],
 "□□□□□□□□": ["□□□□□ □"],
 "□□□□□□□□□□": ["□□□□□ □", "□□□□□ □"],
 "□□□□□": ["□□□□□ □"],
 "□□□□□": ["□□□□□ □"],
 "□□□□□□": ["□□□□□ □"],
 "□□□□": ["□□□□□ □", "□□□□□ □"]
 }
}

[illegible]

```

"000000": {"000000 0"},
"000000": {"000000 0"},
"0000000": {"000000 0"}
},
"000000": {
"0000": {"000000 0", "000000 0", "000000 0", "000000 0", "000000 0"},
"000000": {"000000 0", "000000 0"},
"000000000": {"000000 0", "000000 0"},
"00000000": {"000000 0"},
"00000": {"0000000 0"},
"00000000": {"000000 0"},
"00000": {"0000000 0"},
"000000": {"000000 0"}
},
"00": {
"000": {"000000 0", "000000 0", "000000 0", "000000 0", "000000 0", "000000 0", "000000 0", "000000 0"},
"000000": {"000000 0"},
"00000000": {"000000 0"},
"0000": {"0000000 0"}
},
"000000": {
"000000": {"000000 0", "000000 0", "000000 0", "000000 0"},
"00000000": {"000000 0", "000000 0"},
"0000000000": {"000000 0", "000000 0"},
"0000000": {"000000 0"},
"000": {"0000000 0"},
"000000": {"000000 0"}
},
"00000000000": {
"00000000000": {"000000 0", "000000 0", "000000 0", "000000 0", "000000 0", "000000 0"},
"000000000000 0000": {"000000 0", "000000 0"},
"0000000000": {"000000 0"},
"00000000": {"000000 0"},
"000000": {"000000 0"}
},
"000000000": {
"0000000": {"000000 0", "000000 0", "000000 0"},
"0000 000000": {"000000 0", "000000 0"},
"0000000000 0000": {"000000 0"},
"0000 000000": {"000000 0"},
"0000000": {"000000 0"}
},
"00000000000": {
"00000000000": {"000000 0", "000000 0", "000000 0", "000000 0"},
"0000000": {"000000 0", "000000 0"},
"0000": {"0000000 0"},
"0000": {"000000 0"},
"0000 00000": {"000000 0"}
},
"00000000": {
"00000000000": {"000000 0", "000000 0", "000000 0"},
"000000000": {"000000 0", "000000 0", "000000 0"},
"0000000": {"000000 0", "000000 0"},
"00000000": {"000000 0"},
"0000000000": {"000000 0"}
},
"000000": {
"000000": {"000000 0", "000000 0", "000000 0", "000000 0"},
"0000000": {"000000 0", "000000 0"},
"00000000": {"000000 0"},
"0000000000": {"000000 0"},
"000000000": {"000000 0"}
},
"000000000": {
"0000000": {"000000 0", "000000 0", "000000 0", "000000 0"},
"0000": {"0000000 0", "000000 0"},
"00000000": {"000000 0", "000000 0"},
"00000": {"0000000 0"},
"00000": {"000000 0"},
"00000": {"000000 0"}
},
"000000": {
"000000": {"000000 0", "000000 0", "000000 0", "000000 0", "000000 0"},
"000000": {"000000 0", "000000 0", "000000 0"},
"00000": {"000000 0"},
"000000": {"000000 0"},
"00000000": {"000000 0"}
},
"00000000": {
"000000000": {"000000 0", "000000 0", "000000 0", "000000 0"},
"00000000000": {"000000 0", "000000 0"},
"000000000000": {"000000 0"},
"0000000": {"000000 0"}
},
"0000": {
"0000": {"000000 0", "000000 0", "000000 0", "000000 0"},

```

```

"dashboard": ["dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0"]
},
"dashboard": {
"dashboard": ["dashboard 0", "dashboard 0", "dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0"],
"dashboard": ["dashboard 0"],
"dashboard": ["dashboard 0"]
},
"dashboard": {
"dashboard": ["dashboard 0", "dashboard 0", "dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0"],
"dashboard": ["dashboard 0"]
},
"dashboard": {
"dashboard": ["dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0", "dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0"],
"dashboard": ["dashboard 0"],
"dashboard": ["dashboard 0"]
},
"dashboard 0 dashboard": {
"dashboard": ["dashboard 0", "dashboard 0", "dashboard 0", "dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0"],
"dashboard": ["dashboard 0"]
},
"dashboard": {
"dashboard": ["dashboard 0", "dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0"],
"dashboard": ["dashboard 0"]
},
"dashboard dashboard": {
"dashboard": ["dashboard 0", "dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0"],
"dashboard": ["dashboard 0"],
"dashboard": ["dashboard 0"]
},
"dashboard dashboard 0 dashboard": {
"dashboard": ["dashboard 0", "dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0"],
"dashboard": ["dashboard 0"]
},
"dashboard dashboard": {
"dashboard": ["dashboard 0", "dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0"],
"dashboard": ["dashboard 0"],
"dashboard": ["dashboard 0"]
},
"dashboard dashboard": {
"dashboard": ["dashboard 0", "dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0"],
"dashboard": ["dashboard 0"],
"dashboard": ["dashboard 0"]
},
"dashboard dashboard 0 dashboard": {
"dashboard": ["dashboard 0", "dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0"],
"dashboard": ["dashboard 0"]
},
"dashboard": {
"dashboard": ["dashboard 0", "dashboard 0", "dashboard 0"],
"dashboard": ["dashboard 0"],
"dashboard": ["dashboard 0"],
"dashboard": ["dashboard 0"],
"dashboard": ["dashboard 0"]
}
}
};

```

```

// 0000 00 000000 0000 00000 00 00000000000000 00000000 00 00000000000 00000
const PERMISSION_GROUPS = {
"dashboard": [
{ key: "dash_login", label: "0000 00 00000000" },
{ key: "dash_stats", label: "0000000 0000 000 (000000000 00000 0000)" },
{ key: "dash_activity", label: "0000000 000000 00000000 00000000000" },
{ key: "dash_all_reps", label: "0000000 00000000 000 00000000000" },
{ key: "dash_export", label: "0000000 000000000 00 00000000" }
],
"000 00000000 000000000": [
{ key: "ph_access", label: "0000000 00 000 0000000000" },
{ key: "ph_create", label: "000 000000000 0000" },

```

```

    { key: "ph_list", label: "药房列表", type: "table" },
    { key: "ph_all_reps", label: "药房全部代表", type: "table" },
    { key: "ph_view_loc", label: "药房查看位置", type: "table" },
    { key: "ph_create_loc", label: "药房创建位置", type: "table" },
    { key: "ph_percentage", label: "药房/药房代表 药房 药房", type: "table" },
    { key: "ph_send_mgr", label: "药房 药房代表 药房 药房", type: "table" },
    { key: "ph_excel", label: "药房 药房 药房", type: "table" },
    { key: "ph_delete", label: "药房 药房 药房" }
  ],
  "药房 药房": [
    { key: "doc_access", label: "药房 药房 药房", type: "table" },
    { key: "doc_create", label: "药房 药房 药房", type: "table" },
    { key: "doc_list", label: "药房 药房 药房", type: "table" },
    { key: "doc_all_reps", label: "药房 药房 药房 药房", type: "table" },
    { key: "doc_view_loc", label: "药房 药房 药房 药房", type: "table" },
    { key: "doc_create_loc", label: "药房 药房 药房", type: "table" },
    { key: "doc_percentage", label: "药房/药房 药房 药房", type: "table" },
    { key: "doc_upload", label: "药房 药房 药房 药房", type: "table" },
    { key: "doc_print", label: "药房/药房/药房 药房", type: "table" },
    { key: "doc_send_mgr", label: "药房 药房 药房 药房", type: "table" },
    { key: "doc_excel", label: "药房 药房 药房", type: "table" },
    { key: "doc_delete", label: "药房 药房 药房" }
  ],
  "药房 药房 药房": [
    { key: "ord_access", label: "药房 药房 药房", type: "table" },
    { key: "ord_create", label: "药房 药房 药房", type: "table" },
    { key: "ord_list", label: "药房 药房 药房", type: "table" },
    { key: "ord_all_reps", label: "药房 药房 药房 药房", type: "table" },
    { key: "ord_view_detail", label: "药房 药房 药房", type: "table" },
    { key: "ord_items", label: "药房 药房 药房", type: "table" },
    { key: "ord_create_loc", label: "药房 药房 药房", type: "table" },
    { key: "ord_send", label: "药房 药房 (药房)", type: "table" },
    { key: "ord_excel", label: "药房 药房 药房", type: "table" },
    { key: "ord_delete", label: "药房 药房 药房" }
  ],
  "药房 药房 药房": [
    { key: "fld_visit", label: "药房 药房 药房", type: "table" },
    { key: "fld_start_stop", label: "药房/药房 药房", type: "table" },
    { key: "fld_pause", label: "药房 药房 药房", type: "table" },
    { key: "fld_track", label: "药房 药房 药房", type: "table" },
    { key: "fld_home_loc", label: "药房 药房 药房", type: "table" },
    { key: "fld_all_homes", label: "药房 药房 药房 药房" }
  ],
  "药房 药房 药房": [
    { key: "hr_leave_req", label: "药房 药房", type: "table" },
    { key: "hr_all_leaves", label: "药房 药房 药房 药房", type: "table" },
    { key: "hr_leave_approve", label: "药房/药房 药房", type: "table" },
    { key: "hr_leave_excel", label: "药房 药房 药房" }
  ],
  "药房 药房": [
    { key: "rep_monthly", label: "药房 药房 药房", type: "table" },
    { key: "rep_all_reports", label: "药房 药房 药房", type: "table" },
    { key: "rep_item_sales", label: "药房 药房 药房", type: "table" },
    { key: "rep_excel", label: "药房 药房 药房" }
  ],
  "药房 药房 药房": [
    { key: "sys_add_dropdown", label: "药房 药房 药房", type: "table" },
    { key: "sys_del_dropdown", label: "药房 药房 药房", type: "table" },
    { key: "sys_cols", label: "药房 药房 药房 药房", type: "table" },
    { key: "sys_users", label: "药房 药房 药房", type: "table" },
    { key: "sys_msg", label: "药房 药房 药房", type: "table" },
    { key: "sys_notify", label: "药房 药房 药房 药房" }
  ]
};

// 药房 药房 药房 药房 药房 药房 药房 药房 药房 药房 true 药房 false
function getDefaultPermissionsObject(defaultValue = true) {
  const perms = {};
  Object.values(PERMISSION_GROUPS).forEach(list => {
    list.forEach(item => {
      perms[item.key] = defaultValue;
    });
  });
  return perms;
}

// 药房 药房 药房 药房 药房 药房 药房 药房 药房 药房
const DEFAULT_INITIAL_DATA = {
  pharmacies: [
    {
      id: "ph-1",
      name: "药房 药房 药房 药房",
      phone: "021-88776655",
      province: "药房",
      city: "药房",
    }
  ]
};

```

```
district: "上海市 区",
address: "上海市 上海市 上海市 上海市 上海市 上海市",
  lat: 35.7605,
  lng: 51.4180,
manager: "上海市 上海市 上海市",
repName: "上海市 上海市",
  dateAdded: "1405/05/19",
  isPercentage: false,
fileName: "上海市_上海市.pdf",
customFields: { "上海市 上海市": "上海市", "上海市 上海市": "A+" }
},
{
  id: "ph-2",
name: "上海市 上海市 上海市 上海市",
  phone: "021-66554433",
  province: "上海市",
  city: "上海市",
  district: "上海市 区",
address: "上海市 上海市 上海市 上海市 上海市 上海市 上海市 上海市",
  lat: 35.7008,
  lng: 51.3912,
manager: "上海市 上海市 上海市",
repName: "上海市 上海市",
  dateAdded: "1405/05/18",
  isPercentage: true,
fileName: "上海市 上海市.jpg",
customFields: { "上海市 上海市": "上海市 上海市", "上海市 上海市": "A" }
},
{
  id: "ph-3",
name: "上海市 上海市 上海市",
  phone: "051-38402020",
  province: "上海市",
  city: "上海市",
  district: "上海市 区",
address: "上海市 上海市 上海市 上海市 上海市 上海市 上海市 上海市",
  lat: 35.7590,
  lng: 51.4280,
manager: "上海市 上海市 上海市",
repName: "上海市 上海市 上海市",
  dateAdded: "1405/05/18",
  isPercentage: true,
  fileName: null,
customFields: { "上海市 上海市": "上海市", "上海市 上海市": "B+" }
}
],
doctors: [
  {
    id: "doc-1",
name: "上海市 上海市 上海市",
specialty: "上海市 上海市 上海市",
  phone: "021-88990011",
  province: "上海市",
  city: "上海市",
  district: "上海市 区",
address: "上海市 上海市 上海市 上海市 上海市 上海市 上海市 上海市 上海市 上海市",
  lat: 35.7580,
  lng: 51.4400,
repName: "上海市 上海市",
  dateAdded: "1405/05/18",
  isPercentage: false,
fileName: "上海市_上海市.jpg",
customFields: { "上海市 上海市": "上海市", "上海市 上海市": "上海市 上海市 上海市 上海市" }
},
{
  id: "doc-2",
name: "上海市 上海市 上海市",
specialty: "上海市 上海市 上海市",
  phone: "021-88002244",
  province: "上海市",
  city: "上海市",
  district: "上海市 区",
address: "上海市 上海市 上海市 上海市 上海市 上海市 上海市 上海市 上海市 上海市",
  lat: 35.7250,
  lng: 51.4100,
repName: "上海市 上海市 上海市",
  dateAdded: "1405/05/17",
  isPercentage: true,
  fileName: null,
customFields: { "上海市 上海市 上海市": "上海市", "上海市 上海市": "上海市" }
}
],
reps: [
  {
    id: "rep-1",
```

```
name: "مركز طبي",
phone: "09120852097",
region: "البحرين (المنامة)",
status: "online",
lat: 35.7590,
lng: 51.4280,
lastUpdate: "2020-05-19",
currentVisit: "2020-05-19 10:00:00",
},
{
  id: "rep-2",
  name: "مركز طبي",
  phone: "09129876543",
  region: "البحرين (المنامة)",
  status: "visiting",
  lat: 35.7100,
  lng: 51.4050,
  lastUpdate: "2020-05-19",
  currentVisit: "2020-05-19 11:00:00",
},
{
  id: "rep-3",
  name: "مركز طبي",
  phone: "09151234567",
  region: "البحرين (المنامة)",
  status: "offline",
  lat: 36.3000,
  lng: 59.5800,
  lastUpdate: "2020-05-18",
  currentVisit: "-"
}
],
orders: [
  {
    id: "ord-1",
    pharmacyName: "مركز طبي",
    pharmacyId: "ph-1",
    province: "البحرين",
    city: "المنامة",
    district: "المنامة",
    address: "مركز طبي",
    repName: "مركز طبي",
    orderDate: "1405/05/19",
    items: [
      { name: "مركز طبي", count: 50, price: 250000 },
      { name: "مركز طبي", count: 100, price: 45000 }
    ],
    totalAmount: 17000000,
    status: "مركز طبي",
    notes: "مركز طبي"
  }
],
users: [
  {
    id: "u-1",
    fullName: "مركز طبي",
    username: "admin",
    password: "123",
    phone: "09120000000",
    role: "مركز طبي",
    simControl: "مركز طبي",
    phoneLock: "مركز طبي - مركز طبي",
    lastLogin: "1405/05/19 - 12:55:23",
    permissions: getDefaultPermissionsObject(true)
  },
  {
    id: "u-2",
    fullName: "مركز طبي",
    username: "Taheri",
    password: "456",
    phone: "09120852097",
    role: "مركز طبي",
    simControl: "مركز طبي (مركز طبي)",
    phoneLock: "مركز طبي: Chrome Windows",
    lastLogin: "1405/05/18 - 15:41:16",
    permissions: getDefaultPermissionsObject(true)
  },
  {
    id: "u-3",
    fullName: "مركز طبي",
    username: "nila",
    password: "789",
    phone: "09123334455",
    role: "مركز طبي",
    simControl: "مركز طبي (مركز طبي)",
  }
]
```

```
phoneLock: "0000 - 00000 00000 0000 00 000 000000",
  lastLogin: "1405/05/18 - 10:15:00",
  permissions: getDefaultPermissionsObject(true)
},
{
  id: "u-4",
  fullName: "0000 00000 000000",
  username: "davood",
  password: "321",
  phone: "09120852097",
  role: "000000",
  simControl: "000 0000 (00000 000)",
  phoneLock: "0000 - 00000 00000 0000 00 000 000000",
  lastLogin: "1405/05/19 - 12:49:36",
  permissions: getDefaultPermissionsObject(true)
}
],
activityLog: [
  {id: "act-1", time: "14:35 - 00000", repName: "0000 000000", action: "000 000000 0000 00 00000 000"},
  {id: "act-2", time: "13:10 - 00000", repName: "0000 0000 000000", action: "000 00000 000 00000 00000 000000"},
  {id: "act-3", time: "11:00 - 00000", repName: "0000 000000", action: "000 00000 0000 0000 000000000 0000 000000"}
],
repHomes: [
  {id: "home-1", repName: "0000 000000", address: "000000 00000 00 000000 000000000", lat: 35.7550, lng: 51.4350},
  {id: "home-2", repName: "0000 0000 00000", address: "0000000 000000 00 000000 0000000", lat: 35.7100, lng: 51.3950}
],
repRoutes: [
  {id: "rt-1", repName: "0000 000000", date: "1405/05/19", checkpoint: "0000 00 00000 0 - 00000 000", status: "0000 0 000"},
  {id: "rt-2", repName: "0000 0000 00000", date: "1405/05/19", checkpoint: "00000 00000000 000000", status: "00 000 000 000"}
],
leaves: [
  {id: "lv-1", repName: "0000 00000 000000", fromDate: "1405/05/22", toDate: "1405/05/24", reason: "00000 000000000", status: "00000 000"},
  {id: "lv-2", repName: "0000 0000 00000", fromDate: "1405/05/28", toDate: "1405/05/29", reason: "0000 0000", status: "00 000 00000"}
],
notifications: [
  {id: "not-1", date: "1405/05/19", title: "00000 00000 0000 000 0000", message: "0000000 0000000 00000 000 000000 00 00000000 0000 0000.", sender: "0000 00000"},
  {id: "not-2", date: "1405/05/15", title: "0000000000 0000 2.5.1", message: "00000 000000000 000 000000000 0 00000 0000 0 0000 00.", sender: "0000 00000"}
],
salesTargets: [
  {id: "tgt-1", repName: "0000 000000", month: "00000 1405", targetAmount: 500000000, achievedAmount: 380000000},
  {id: "tgt-2", repName: "0000 0000 00000", month: "00000 1405", targetAmount: 400000000, achievedAmount: 410000000}
],
messengers: {
  smsEnabled: true,
  telegramToken: "123456:ABC-DEF1234ghIkl-zyx57W2v1u123ew11",
  whatsappNumber: "09120000000",
  eitaChannel: "@namayande_channel"
},
products: [
  {
    id: "prod-1",
    name: "00000 000000000 00 000000000",
    category: "00000",
    price: 45000,
    stock: 5000,
    description: "0000000000 00 00000 00000 0000 00000 0000000 0000"
  },
  {
    id: "prod-2",
    name: "00000 000000000 0000000 8 000000",
    category: "000000 0 0000000",
    price: 250000,
    stock: 1200,
    description: "0000000000 0 0000 000000000 00000000 0000 00 0000 00000"
  }
],
customFields: {
  pharmacy: [
    {id: "cf-ph-1", label: "000 0000000", type: "select", options: ["0000", "000 0000000", "000 0000000", "00000"], showInForm: true, showInList: true, allowAddOption: true },
    {id: "cf-ph-2", label: "0000 000000", type: "select", options: ["A+", "A", "B+", "B", "C"], showInForm: true, showInList: true, allowAddOption: true }
  ],
  doctor: [
    {id: "cf-doc-1", label: "00000 00000 000000", type: "simple", showInForm: true, showInList: true },
    {id: "cf-doc-2", label: "0000000 000000", type: "simple", showInForm: true, showInList: false }
  ],
  order: [
    {id: "cf-ord-1", label: "000000 00000", type: "select", options: ["0000", "0000", "0000 (0000 00 0 0000)", showInForm: true, showInList: true }
  ]
}
```

```

nForm: true, showInList: true, allowAddOption: true }
    ]
  },
  permissions: [], // 权限列表
  settings: {
    companyName: "XXXX XXXX XXXX XXXX XXXX XXXX XXXX XXXX",
    apiUrl: "https://namayandee.lmi-javad.onrender.com/api/state",
    defaultMapLat: 35.7200,
    defaultMapLng: 51.4200,
    currencyUnit: "XXXX",
    autoBackupEnabled: true,
    autoBackupIntervalMinutes: 5,
    lastBackupTime: null
  }
};

@@ public/sw-template.js javascript
/* =====
 * « » — ()
 *
 * : HTML * CSS*
 *
 * .
 * JS
 * scripts/build-sw.mjs build .
 * ===== */
const BUILD = "__BUILD_ID__";
const SHELL = `sek-shell-${BUILD}`;

const DATA = `sek-data-${BUILD}`;
const API_TIMEOUT = 6000;
const ASSET_TIMEOUT = 20000;
const NAV_TIMEOUT = 5000; // HTML
/** */
function offlineFlagged(res) {
  const h = new Headers(res.headers);
  h.set("x-sek-offline", "1");
  return new Response(res.body, { status: res.status, statusText: res.statusText, headers: h });
}
const PAGES = __PAGES__;
const ASSETS = __ASSETS__;
const EXTRAS = __EXTRAS__;
const RSC_PAGES = __RSC_PAGES__;
void RSC_PAGES;
/* ----- : ----- */
self.addEventListener("install", (e) => {
  self.skipWaiting();
  e.waitUntil(
    (async () => {
      const cache = await caches.open(SHELL);
      // (JS/CSS) —
      await addAll(cache, ASSETS, 6);
      await addAll(cache, PAGES, 4);
      await addAll(cache, EXTRAS, 3);
      const cs = await self.clients.matchAll();
      cs.forEach((c) => c.postMessage({ type: "precache-done", assets: ASSETS.length, pages: PAGES.length }));
    })()
  );
});
async function addAll(cache, urls, concurrency) {
  let i = 0;
  const workers = Array.from({ length: concurrency }, async () => {
    while (i < urls.length) {
      const url = urls[i++];
      try {
        const res = await fetch(new Request(url, { cache: "reload", credentials: "same-origin" }));
        if (res.ok || res.type === "opaque") await cache.put(url, res.clone());
      } catch {}
    }
  });
  await Promise.all(workers);
}
/* ----- */
self.addEventListener("activate", (e) => {
  e.waitUntil(
    (async () => {
      if (self.registration.navigationPreload) {
        try {
          await self.registration.navigationPreload.enable();
        } catch {}
      }
      const keys = await caches.keys();
      await Promise.all(keys.filter((k) => !k.endsWith(BUILD)).map((k) => caches.delete(k)));
      await self.clients.claim();
    })()
  );
});

```



```

    })()
  );
});
function timeoutFetch(req, ms) {
  return new Promise((resolve, reject) => {
    const t = setTimeout(() => reject(new Error("timeout")), ms);
    fetch(req).then(
      (r) => {
        clearTimeout(t);
        resolve(r);
      },
      (err) => {
        clearTimeout(t);
        reject(err);
      }
    );
  });
}
/* ----- */
const DB_NAME = "sek-queue";

function idb() {
  return new Promise((res, rej) => {
    const r = indexedDB.open(DB_NAME, 1);
    r.onupgradeneeded = () => r.result.createObjectStore("q", { keyPath: "id", autoIncrement: true });
    r.onsuccess = () => res(r.result);
    r.onerror = () => rej(r.error);
  });
}

async function queueAdd(item) {
  const db = await idb();
  return new Promise((res) => {
    const tx = db.transaction("q", "readwrite");
    tx.objectStore("q").add(item);
    tx.oncomplete = () => res(true);
    tx.onerror = () => res(false);
  });
}

async function queueAll() {
  const db = await idb();
  return new Promise((res) => {
    const tx = db.transaction("q", "readonly");
    const rq = tx.objectStore("q").getAll();
    rq.onsuccess = () => res(rq.result || []);
    rq.onerror = () => res([]);
  });
}

async function queueDel(id) {
  const db = await idb();
  return new Promise((res) => {
    const tx = db.transaction("q", "readwrite");
    tx.objectStore("q").delete(id);
    tx.oncomplete = () => res(true);
    tx.onerror = () => res(false);
  });
}

async function flushQueue() {
  const items = await queueAll();
  let sent = 0;
  for (const it of items) {
    try {
      const r = await fetch(it.url, { method: it.method, headers: it.headers, body: it.body, credentials: "include" });
      if (r.ok || (r.status >= 400 && r.status < 500)) {
        await queueDel(it.id);
        if (r.ok) sent++;
      }
    } catch {
      break;
    }
  }
  const rest = await queueAll();
  const cs = await self.clients.matchAll();
  cs.forEach((c) => c.postMessage({ type: "queue-status", pending: rest.length, sent }));
  return sent;
}

self.addEventListener("message", (e) => {
  if (e.data === "flush") e.waitUntil(flushQueue());
  if (e.data === "skipWaiting") self.skipWaiting();
  if (e.data === "queue-count")
    e.waitUntil(
      queueAll().then(async (q) => {
        const cs = await self.clients.matchAll();
        cs.forEach((c) => c.postMessage({ type: "queue-status", pending: q.length, sent: 0 }));
      })
    );
});

```

```

if (e.data === "cache-status")
  e.waitUntil(
    caches.open(SHELL).then(async (c) => {
      const keys = await c.keys();
      const cs = await self.clients.matchAll();
      cs.forEach((x) =>
        x.postMessage({ type: "cache-status", cached: keys.length, expected: ASSETS.length + PAGES.length + EXTRAS.length }
      ));
    });
  });
});
self.addEventListener("sync", (e) => {
  if (e.tag === "sek-sync") e.waitUntil(flushQueue());
});
/* ----- */
self.addEventListener("notificationclick", (event) => {
  const data = event.notification.data || {};
  event.notification.close();

  event.waitUntil(
    (async () => {
      const all = await self.clients.matchAll({ type: "window", includeUncontrolled: true });
      //
      for (const c of all) {
        c.postMessage({ type: "notification-click", id: data.id });
        if ("focus" in c) {
          await c.focus();
          if (data.link) c.navigate?.(data.link);
          return;
        }
      }
      if (self.clients.openWindow) await self.clients.openWindow(data.link || "/panel/notifications");
    })()
  );
});
/* ----- Web Push ( ) ----- */
self.addEventListener("push", (event) => {
  let payload = { title: " ", body: "", id: 0, link: "/panel/notifications" };
  try {
    payload = { ...payload, ...(event.data ? event.data.json() : {}) };
  } catch {
    if (event.data) payload.body = event.data.text();
  }
  event.waitUntil(
    self.registration.showNotification(`${payload.title}`, {
      body: payload.body,
      icon: "/icons/icon-192.png",
      badge: "/icons/icon-192.png",
      dir: "rtl",
      lang: "fa",
      tag: `sek-${payload.id}`,
      requireInteraction: true,
      vibrate: [200, 100, 200],
      data: payload,
    })
  );
});
/* ----- */
self.addEventListener("fetch", (event) => {
  const req = event.request;
  const url = new URL(req.url);
  if (url.origin !== self.location.origin) return;
  if (url.pathname === "/api/ping") return;
  /* → */
  if (req.method !== "GET") {
    if (!url.pathname.startsWith("/api/")) return;
    event.respondWith(
      (async () => {
        try {
          return await fetch(req.clone());
        } catch {
          if (url.pathname.startsWith("/api/auth")) {
            return new Response(JSON.stringify({ error: " " }), {
              status: 503,
              headers: { "Content-Type": "application/json; charset=utf-8" },
            });
          }
        }
        const body = await req.clone().text();
        const headers = {};
        req.headers.forEach((v, k) => (headers[k] = v));
        await queueAdd({ url: req.url, method: req.method, headers, body, at: Date.now() });
        if (self.registration.sync) {
          try {
            await self.registration.sync.register("sek-sync");
          }
        }
      })
    );
  }
});

```

```

    } catch {}
  }
  const cs = await self.clients.matchAll();
  cs.forEach((c) => c.postMessage({ type: "queued" }));
  return new Response(
    JSON.stringify({
      ok: true,
      queued: true,
      offline: true,
      message: " ",
    }),
    { status: 202, headers: { "Content-Type": "application/json; charset=utf-8" } }
  );
}
})();
);
return;
}
/* RSC ( Next) →
 *
const isRsc = url.searchParams.has("_rsc") || req.headers.get("RSC") === "1";
if (isRsc) {
  event.respondWith(
    (async () => {
      try {
        const res = await timeoutFetch(req, NAV_TIMEOUT);
        if (res && res.ok) {
          const c = await caches.open(SHELL);
          c.put(req, res.clone());
        }
        return res;
      } catch {
        const cached = await caches.match(req);
        if (cached) return cached;
        // RSC Next
        return new Response("", { status: 504 });
      }
    })()
  );
  return;
}
/* Next → - () */
if (url.pathname.startsWith("/_next/static/")) {
  event.respondWith(
    (async () => {
      const cached = await caches.match(req);
      if (cached) return cached;
      try {
        const res = await timeoutFetch(req, ASSET_TIMEOUT);
        if (res && res.ok) {
          const c = await caches.open(SHELL);
          c.put(req, res.clone());
        }
        return res;
      } catch {
        return new Response("", { status: 504 });
      }
    })()
  );
  return;
}
/* API → */
if (url.pathname.startsWith("/api/")) {
  if (
    url.pathname.startsWith("/api/auth/login") ||
    url.pathname.startsWith("/api/auth/logout") ||
    url.pathname.startsWith("/api/auth/forgot") ||
    url.pathname.startsWith("/api/auth/otp") ||
    url.pathname.startsWith("/api/auth/check-username")
  )
    return;
  event.respondWith(
    (async () => {
      try {
        const res = await timeoutFetch(req, API_TIMEOUT);
        if (res.ok) {
          const c = await caches.open(DATA);
          c.put(req, res.clone());
        }
        return res;
      } catch {
        const cached = await caches.match(req);
        if (cached) {
          const h = new Headers(cached.headers);

```

```

        h.set("x-from-cache", "1");
        return new Response(await cached.blob(), { status: 200, headers: h });
    }
    return new Response(
        JSON.stringify({
            offline: true,
            rows: [],
            logs: [],
            reps: [],
            error: " ",
        }),
        { status: 200, headers: { "Content-Type": "application/json; charset=utf-8", "x-sek-offline": "1" } }
    );
}

})();
);
return;
}
/* =====
* → «-» (Network First)
*
*   HTML
*   .
*
*   ( ).
* ===== */
if (req.mode === "navigate") {
    event.respondWith(
        (async () => {
            //
            try {
                const preload = await event.preloadResponse;
                const res = preload || (await timeoutFetch(new Request(req, { cache: "no-store" }), NAV_TIMEOUT));
                if (res && res.ok) {
                    const c = await caches.open(SHELL);
                    c.put(url.pathname, res.clone()); //
                    return res;
                }
                if (res) return res; // (/)
            } catch {
                //
            }
            // : ( )
            const cachedPage =
                (await caches.match(url.pathname, { ignoreSearch: true })) ||
                (await caches.match(req, { ignoreSearch: true }));
            if (cachedPage) return offlineFlagged(cachedPage);
            //
            const shell =
                (await caches.match("/panel")) || (await caches.match("/")) || (await caches.match("/login"));
            if (shell) return offlineFlagged(shell);
            // :
            return (
                (await caches.match("/offline")) ||
                new Response(
                    "<html dir=rtl><meta charset=utf-8><body style='font-family:Tahoma;text-align:center;padding:40px'>" +
                    "<h2> </h2><p> .</p>" +
                    "<button onclick='location.reload()' style='padding:10px 24px;border-radius:10px;background:#0f766e;color:" +
                    "#fff;border:0'> </button>" +
                    "</body></html>",
                    { status: 200, headers: { "Content-Type": "text/html; charset=utf-8" } }
                )
            );
        })()
    );
    return;
}
/* → - */
event.respondWith(
    (async () => {
        const cached = await caches.match(req);
        if (cached) return cached;
        try {
            const res = await timeoutFetch(req, ASSET_TIMEOUT);
            if (res && res.ok) {
                const c = await caches.open(SHELL);
                c.put(req, res.clone());
            }
            return res;
        } catch {
            return new Response("", { status: 504 });
        }
    })()
);

```

```

});

@@ public/sw.js javascript
/* =====
 * « » — ()
 *
 * : HTML * CSS*
 *
 * JS
 * scripts/build-sw.mjs build .

 * ===== */
const BUILD = "msmdifhz-48";
const SHELL = `sek-shell-${BUILD}`;
const DATA = `sek-data-${BUILD}`;
const API_TIMEOUT = 6000;
const ASSET_TIMEOUT = 20000;
const NAV_TIMEOUT = 5000; // HTML
/** */
function offlineFlagged(res) {
  const h = new Headers(res.headers);
  h.set("x-sek-offline", "1");
  return new Response(res.body, { status: res.status, statusText: res.statusText, headers: h });
}
const PAGES = [
  "/",
  "/login",
  "/offline",
  "/install",
  "/diagnostics",
  "/panel",
  "/panel/pharmacies",
  "/panel/doctors",
  "/panel/orders",
  "/panel/trip",
  "/panel/map",
  "/panel/home",
  "/panel/leaves",
  "/panel/notifications",
  "/panel/options",
  "/panel/reports",
  "/admin",
  "/admin/activity",
  "/admin/records/pharmacies",
  "/admin/records/doctors",
  "/admin/records/orders",
  "/admin/map",
  "/admin/live",
  "/admin/trips",
  "/admin/homes",
  "/admin/leaves",
  "/admin/notifications",
  "/admin/reports",
  "/admin/options",
  "/admin/columns",
  "/admin/users",
  "/admin/messengers",
  "/admin/backup"
];
const ASSETS = [
  "/_next/static/chunks/0.f8o6x69l1dq.js",
  "/_next/static/chunks/01xlw8hd842-c.js",
  "/_next/static/chunks/01yarxs5j9-pm.js",
  "/_next/static/chunks/025h39a4hxy-7.js",
  "/_next/static/chunks/02a3un8vt.4z0.js",
  "/_next/static/chunks/02es.zsatljwd.js",
  "/_next/static/chunks/03h9eo09svijh.js",
  "/_next/static/chunks/03-yq9q893hmn.js",
  "/_next/static/chunks/04zs0174xt4cf.js",
  "/_next/static/chunks/06in8m2ln4m7x.js",
  "/_next/static/chunks/06r9_3ub2r-4z.js",
  "/_next/static/chunks/07lhk_q6pmm3r.js",
  "/_next/static/chunks/07uz2g0_38qia.js",
  "/_next/static/chunks/08.gvqj-o7ww6.js",
  "/_next/static/chunks/09pnscjn-jklc.js",
  "/_next/static/chunks/09u-zy9mfz-2_.js",
  "/_next/static/chunks/0_uvbkl-ua0pn.js",
  "/_next/static/chunks/0a59dbudyzs14.js",
  "/_next/static/chunks/0d3shmh5_nmn.js",
  "/_next/static/chunks/0e8p5l2oiw7kt.js",
  "/_next/static/chunks/0evn0xx7qmz14.js",
  "/_next/static/chunks/0h7njwe2.s3ht.js",
  "/_next/static/chunks/0hoxp6v6.ghn5.js",
  "/_next/static/chunks/0n.0ufr5-u.u3.js",
  "/_next/static/chunks/0n1bpogj.il-n.js",

```

```

"/_next/static/chunks/009kx41n71_k~.js",
"/_next/static/chunks/0oudi9rq3kf0y.js",
"/_next/static/chunks/0rjm_co8bzrp0.js",
"/_next/static/chunks/0rz300743-gv3.js",
"/_next/static/chunks/0t48hzs_6fshe.css",
"/_next/static/chunks/0u~69.dj98a1y.js",
"/_next/static/chunks/0v4dehhyglgch.js",

"/_next/static/chunks/0v7ff1k50ig4a.js",
"/_next/static/chunks/0vy~5p61tj63m.js",
"/_next/static/chunks/0we42v7.l-4bo.js",
"/_next/static/chunks/0wxhwo2i~7vvo.js",
"/_next/static/chunks/0xnf3du-wj5.7.css",
"/_next/static/chunks/0zafszl7iub1.js",
"/_next/static/chunks/0ze4gu236oq96.js",
"/_next/static/chunks/11bf.0jc0_nuk.js",
"/_next/static/chunks/11p2tj5w-k_4s.js",
"/_next/static/chunks/13niyj-yq3tn-.js",
"/_next/static/chunks/17ij8w.0v9eei.js",
"/_next/static/chunks/17zu884hpts9.js",
"/_next/static/chunks/turbopack-0ngnbt.drh_yz.js",
"/_next/static/mVg2gUuQcbk3idaw9VXG/_buildManifest.js",
"/_next/static/mVg2gUuQcbk3idaw9VXG/_clientMiddlewareManifest.js",
"/_next/static/mVg2gUuQcbk3idaw9VXG/_ssgManifest.js"
];
const EXTRAS = [
  "/manifest.webmanifest",
  "/logo.svg",
  "/icons/icon-192.png",
  "/icons/icon-512.png",
  "/apple-touch-icon.png",
  "/data/iran-provinces.geojson"
];
const RSC_PAGES = [
  "/?_rsc=offline",
  "/login?_rsc=offline",
  "/offline?_rsc=offline",
  "/install?_rsc=offline",
  "/diagnostics?_rsc=offline",
  "/panel?_rsc=offline",
  "/panel/pharmacies?_rsc=offline",
  "/panel/doctors?_rsc=offline",
  "/panel/orders?_rsc=offline",
  "/panel/trip?_rsc=offline",
  "/panel/map?_rsc=offline",
  "/panel/home?_rsc=offline",
  "/panel/leaves?_rsc=offline",
  "/panel/notifications?_rsc=offline",
  "/panel/options?_rsc=offline",
  "/panel/reports?_rsc=offline",
  "/admin?_rsc=offline",
  "/admin/activity?_rsc=offline",
  "/admin/records/pharmacies?_rsc=offline",
  "/admin/records/doctors?_rsc=offline",
  "/admin/records/orders?_rsc=offline",
  "/admin/map?_rsc=offline",
  "/admin/live?_rsc=offline",
  "/admin/trips?_rsc=offline",
  "/admin/homes?_rsc=offline",
  "/admin/leaves?_rsc=offline",
  "/admin/notifications?_rsc=offline",
  "/admin/reports?_rsc=offline",
  "/admin/options?_rsc=offline",
  "/admin/columns?_rsc=offline",
  "/admin/users?_rsc=offline",
  "/admin/messengers?_rsc=offline",
  "/admin/backup?_rsc=offline"
];
void RSC_PAGES;
/* ----- : ----- */
self.addEventListener("install", (e) => {
  self.skipWaiting();
  e.waitUntil(
    (async () => {
      const cache = await caches.open(SHELL);
      // (JS/CSS) —
      await addAll(cache, ASSETS, 6);
      await addAll(cache, PAGES, 4);
      await addAll(cache, EXTRAS, 3);
      const cs = await self.clients.matchAll();
      cs.forEach((c) => c.postMessage({ type: "precache-done", assets: ASSETS.length, pages: PAGES.length }));
    })()
  );
});
async function addAll(cache, urls, concurrency) {

```

```

let i = 0;
const workers = Array.from({ length: concurrency }, async () => {
  while (i < urls.length) {
    const url = urls[i++];
    try {
      const res = await fetch(new Request(url, { cache: "reload", credentials: "same-origin" }));

      if (res && (res.ok || res.type === "opaque")) await cache.put(url, res.clone());
    } catch {
      /* */
    }
  }
});
await Promise.all(workers);
}
/* ----- */
self.addEventListener("activate", (e) => {
  e.waitUntil(
    (async () => {
      if (self.registration.navigationPreload) {
        try {
          await self.registration.navigationPreload.enable();
        } catch {}
      }
      const keys = await caches.keys();
      await Promise.all(keys.filter((k) => !k.endsWith(BUILD)).map((k) => caches.delete(k)));
      await self.clients.claim();
    })()
  );
});
function timeoutFetch(req, ms) {
  return new Promise((resolve, reject) => {
    const t = setTimeout(() => reject(new Error("timeout")), ms);
    fetch(req).then(
      (r) => {
        clearTimeout(t);
        resolve(r);
      },
      (err) => {
        clearTimeout(t);
        reject(err);
      }
    );
  });
}
/* ----- */
const DB_NAME = "sek-queue";
function idb() {
  return new Promise((res, rej) => {
    const r = indexedDB.open(DB_NAME, 1);
    r.onupgradeneeded = () => r.result.createObjectStore("q", { keyPath: "id", autoIncrement: true });
    r.onsuccess = () => res(r.result);
    r.onerror = () => rej(r.error);
  });
}
async function queueAdd(item) {
  const db = await idb();
  return new Promise((res) => {
    const tx = db.transaction("q", "readwrite");
    tx.objectStore("q").add(item);
    tx.oncomplete = () => res(true);
    tx.onerror = () => res(false);
  });
}
async function queueAll() {
  const db = await idb();
  return new Promise((res) => {
    const tx = db.transaction("q", "readonly");
    const rq = tx.objectStore("q").getAll();
    rq.onsuccess = () => res(rq.result || []);
    rq.onerror = () => res([]);
  });
}
async function queueDel(id) {
  const db = await idb();
  return new Promise((res) => {
    const tx = db.transaction("q", "readwrite");
    tx.objectStore("q").delete(id);
    tx.oncomplete = () => res(true);
    tx.onerror = () => res(false);
  });
}
async function flushQueue() {
  const items = await queueAll();
  let sent = 0;

```

```

for (const it of items) {
  try {
    const r = await fetch(it.url, { method: it.method, headers: it.headers, body: it.body, credentials: "include" });
    if (r.ok || (r.status >= 400 && r.status < 500)) {

      await queueDel(it.id);
      if (r.ok) sent++;
    }
  } catch {
    break;
  }
}
const rest = await queueAll();
const cs = await self.clients.matchAll();
cs.forEach((c) => c.postMessage({ type: "queue-status", pending: rest.length, sent }));
return sent;
}

self.addEventListener("message", (e) => {
  if (e.data === "flush") e.waitUntil(flushQueue());
  if (e.data === "skipWaiting") self.skipWaiting();
  if (e.data === "queue-count")
    e.waitUntil(
      queueAll().then(async (q) => {
        const cs = await self.clients.matchAll();
        cs.forEach((c) => c.postMessage({ type: "queue-status", pending: q.length, sent: 0 }));
      })
    );
  if (e.data === "cache-status")
    e.waitUntil(
      caches.open(SHELL).then(async (c) => {
        const keys = await c.keys();
        const cs = await self.clients.matchAll();
        cs.forEach((x) =>
          x.postMessage({ type: "cache-status", cached: keys.length, expected: ASSETS.length + PAGES.length + EXTRAS.len
.length })
        );
      })
    );
});

self.addEventListener("sync", (e) => {
  if (e.tag === "sek-sync") e.waitUntil(flushQueue());
});
/* ----- */
self.addEventListener("notificationclick", (event) => {
  const data = event.notification.data || {};
  event.notification.close();
  event.waitUntil(
    (async () => {
      const all = await self.clients.matchAll({ type: "window", includeUncontrolled: true });
      //
      for (const c of all) {
        c.postMessage({ type: "notification-click", id: data.id });
        if ("focus" in c) {
          await c.focus();
          if (data.link) c.navigate?.(data.link);
          return;
        }
      }
      if (self.clients.openWindow) await self.clients.openWindow(data.link || "/panel/notifications");
    })()
  );
});
/* ----- Web Push ( ) ----- */
self.addEventListener("push", (event) => {
  let payload = { title: " ", body: "", id: 0, link: "/panel/notifications" };
  try {
    payload = { ...payload, ...(event.data ? event.data.json() : {}) };
  } catch {
    if (event.data) payload.body = event.data.text();
  }
  event.waitUntil(
    self.registration.showNotification(`${payload.title}`, {
      body: payload.body,
      icon: "/icons/icon-192.png",
      badge: "/icons/icon-192.png",
      dir: "rtl",
      lang: "fa",
      tag: `sek-${payload.id}`,
      requireInteraction: true,
      vibrate: [200, 100, 200],
      data: payload,
    })
  );
});
/* ----- */

```



```

self.addEventListener("fetch", (event) => {

  const req = event.request;
  const url = new URL(req.url);
  if (url.origin !== self.location.origin) return;
  if (url.pathname === "/api/ping") return;
  /* → */
  if (req.method !== "GET") {
    if (!url.pathname.startsWith("/api/")) return;
    event.respondWith(
      (async () => {
        try {
          return await fetch(req.clone());
        } catch {
          if (url.pathname.startsWith("/api/auth")) {
            return new Response(JSON.stringify({ error: "    ." })), {
              status: 503,
              headers: { "Content-Type": "application/json; charset=utf-8" },
            });
          }
        }
        const body = await req.clone().text();
        const headers = {};
        req.headers.forEach((v, k) => (headers[k] = v));
        await queueAdd({ url: req.url, method: req.method, headers, body, at: Date.now() });
        if (self.registration.sync) {
          try {
            await self.registration.sync.register("sek-sync");
          } catch {}
        }
        const cs = await self.clients.matchAll();
        cs.forEach(c => c.postMessage({ type: "queued" }));
        return new Response(
          JSON.stringify({
            ok: true,
            queued: true,
            offline: true,
            message: "    .",
          }),
          { status: 202, headers: { "Content-Type": "application/json; charset=utf-8" } }
        );
      })()
    );
    return;
  }
  /* RSC ( Next) →
  *      . */
  const isRsc = url.searchParams.has("_rsc") || req.headers.get("RSC") === "1";
  if (isRsc) {
    event.respondWith(
      (async () => {
        try {
          const res = await timeoutFetch(req, NAV_TIMEOUT);
          if (res && res.ok) {
            const c = await caches.open(SHELL);
            c.put(req, res.clone());
          }
          return res;
        } catch {
          const cached = await caches.match(req);
          if (cached) return cached;
          // RSC Next ( )
          return new Response("", { status: 504 });
        }
      })()
    );
    return;
  }
  /* Next → - () */
  if (url.pathname.startsWith("/_next/static/")) {
    event.respondWith(
      (async () => {
        const cached = await caches.match(req);
        if (cached) return cached;
        try {
          const res = await timeoutFetch(req, ASSET_TIMEOUT);
          if (res && res.ok) {
            const c = await caches.open(SHELL);
            c.put(req, res.clone());
          }
          return res;
        } catch {
          return new Response("", { status: 504 });
        }
      })
    );
  }
}

```

```

    })()
  );
  return;
}
/* API → */
if (url.pathname.startsWith("/api/")) {
  if (
    url.pathname.startsWith("/api/auth/login") ||
    url.pathname.startsWith("/api/auth/logout") ||
    url.pathname.startsWith("/api/auth/forgot") ||
    url.pathname.startsWith("/api/auth/otp") ||
    url.pathname.startsWith("/api/auth/check-username")
  )
    return;
  event.respondWith(
    (async () => {
      try {
        const res = await timeoutFetch(req, API_TIMEOUT);
        if (res.ok) {
          const c = await caches.open(DATA);
          c.put(req, res.clone());
        }
        return res;
      } catch {
        const cached = await caches.match(req);
        if (cached) {
          const h = new Headers(cached.headers);
          h.set("x-from-cache", "1");
          return new Response(await cached.blob(), { status: 200, headers: h });
        }
        return new Response(
          JSON.stringify({
            offline: true,
            rows: [],
            logs: [],
            reps: [],
            error: " ",
          }),
          { status: 200, headers: { "Content-Type": "application/json; charset=utf-8", "x-sek-offline": "1" } }
        );
      }
    })()
  );
  return;
}
/* =====
* → «-» (Network First)
*
*   HTML
*   .
*
*   ( ).
* ===== */
if (req.mode === "navigate") {
  event.respondWith(
    (async () => {
      //
      try {
        const preload = await event.preloadResponse;
        const res = preload || (await timeoutFetch(new Request(req, { cache: "no-store" }), NAV_TIMEOUT));
        if (res && res.ok) {
          const c = await caches.open(SHELL);
          c.put(url.pathname, res.clone()); //
          return res;
        }
        if (res) return res; // (/)
      } catch {
        /* → */
      }
      // : ( )
      const cachedPage =
        (await caches.match(url.pathname, { ignoreSearch: true })) ||
        (await caches.match(req, { ignoreSearch: true }));
      if (cachedPage) return offlineFlagged(cachedPage);
      //
      const shell =
        (await caches.match("/panel")) || (await caches.match("/")) || (await caches.match("/login"));
      if (shell) return offlineFlagged(shell);
      // :
      return (
        (await caches.match("/offline")) ||
        new Response(
          "<html dir=rtl><meta charset=utf-8><body style='font-family:Tahoma;text-align:center;padding:40px'>" +
            "<h2>      </h2><p>      .</p>" +

```

```

        "<button onclick='location.reload()' style='padding:10px 24px;border-radius:10px;background:#0f766e;color:
#fff;border:0'> </button>" +
        "</body></html>",
        { status: 200, headers: { "Content-Type": "text/html; charset=utf-8" } }
    )
    );
  })()
  );
  return;
}
/* → - */
event.respondWith(
  (async () => {
    const cached = await caches.match(req);
    if (cached) return cached;
    try {
      const res = await timeoutFetch(req, ASSET_TIMEOUT);
      if (res && res.ok) {
        const c = await caches.open(SHELL);
        c.put(req, res.clone());
      }
      return res;
    } catch {
      return new Response("", { status: 504 });
    }
  })()
);
});

// Network-First strategy for navigate requests (Prevent offline cache locking on mobile)
self.addEventListener('fetch', (event) => {
  if (event.request.mode === 'navigate') {
    event.respondWith(
      fetch(event.request).catch(() => {
        return caches.match(event.request) || caches.match('/index.html');
      })
    );
  }
});

@@ src/instrumentation.ts typescript
export async function register() {
  if (process.env.NEXT_RUNTIME !== "nodejs") return;
  try {
    const { ensureSeed } = await import("@/lib/bootstrap");
    await ensureSeed();
  } catch (err) {
    console.error("instrumentation bootstrap failed", err);
  }
  //
  try {
    const { configuredPeers, syncSecret } = await import("@/lib/sync-config");
    if (configuredPeers().length === 0 || !syncSecret()) return;
    const minutes = Math.max(1, Number(process.env.SYNC_INTERVAL_MINUTES ?? 5));
    const g = globalThis as typeof globalThis & { __sekSyncTimer?: NodeJS.Timeout };
    if (g.__sekSyncTimer) clearInterval(g.__sekSyncTimer);
    const run = async () => {
      try {
        const { syncAll } = await import("@/lib/sync");
        const reports = await syncAll();
        for (const r of reports) {
          console.log(`[sync:${r.peer}] ${r.ok ? "" : ""} ${r.pulled} / ${r.pushed} — ${r.detail}`);
        }
      } catch (err) {
        console.error("[sync] :", err instanceof Error ? err.message : err);
      }
    };
    //
    setTimeout(run, 25_000);
    g.__sekSyncTimer = setInterval(run, minutes * 60_000);
    console.log(`    ${minutes} `);
  } catch (err) {
    console.error("sync scheduler failed", err);
  }
}

@@ src/proxy.ts typescript
import { NextResponse, type NextRequest } from "next/server";
/**
 * CORS
 *
 * ndcohub.ir onrender.com ( )
 * /
 */
const ALLOWED = (process.env.NEXT_PUBLIC_ENDPOINTS || "https://ndcohub.ir,https://namayandeeelmi-javad.onrender.com")

```



```

    } catch {
      return false;
    }
  }
}
/* ----- */
/* ----- */
/* ----- */
export type SimInfo = {
  /** — () */
  phoneNumber: string;
  /** */
  fingerprint: string;
  carrier: string;
  simCount: number;
  hasSim: boolean;
  source: "line1" | "subscription" | "fallback" | "none";
};

function normalizePhone(raw?: string | null): string {
  if (!raw) return "";
  let d = String(raw).replace(/\\D/g, "");
  if (d.startsWith("0098")) d = "0" + d.slice(4);
  else if (d.startsWith("98") && d.length === 12) d = "0" + d.slice(2);
  else if (d.length === 10 && d.startsWith("9")) d = "0" + d;
  return /^0\\d{10}$/.test(d) ? d : "";
}

/** */
export async function readSimInfo(): Promise<SimInfo> {
  const empty: SimInfo = {
    phoneNumber: "",
    fingerprint: "",
    carrier: "",
    simCount: 0,
    hasSim: false,
    source: "none",
  };
  if (Platform.OS !== "android") {
    // iOS:
    const carrier = await DeviceInfo.getCarrier().catch(() => "");
    return { ...empty, carrier, hasSim: !!carrier, fingerprint: carrier ? `ios:${carrier}` : "" };
  }
  await requestSimPermissions();
  let phoneNumber = "";
  let fingerprint = "";
  let carrier = "";
  let simCount = 0;
  let source: SimInfo["source"] = "none";
  // react-native-sim-data () —
  try {
    const SimData = NativeModules.RNSimData ?? require("react-native-sim-data").default;
    const info = await Promise.resolve(SimData?.getSimInfo?.());
    if (info) {
      const list = Array.isArray(info) ? info : Object.values(info);
      const first = (list[0] ?? info) as Record<string, unknown>;
      simCount = Array.isArray(list) ? list.length : 1;
      phoneNumber = normalizePhone(String(first.phoneNumber ?? first.number ?? ""));
      carrier = String(first.carrierName ?? first.displayName ?? "");
      const iccid = String(first.simSerialNumber ?? first.iccId ?? "");
      const imsi = String(first.subscriberId ?? "");
      const mccmnc = `${first.mcc ?? ""}${first.mnc ?? ""}`;
      fingerprint = iccid || imsi || `${carrier}:${mccmnc}`;
      if (phoneNumber) source = "subscription";
    }
  } catch {
    /** */
  }

  // react-native-device-info
  if (!carrier) carrier = await DeviceInfo.getCarrier().catch(() => "");
  if (!phoneNumber) {
    const p = await DeviceInfo.getPhoneNumber?.().catch(() => "");
    const n = normalizePhone(p);
    if (n) {
      phoneNumber = n;
      source = "line1";
    }
  }
  if (!fingerprint && carrier) {
    fingerprint = `${carrier}:${await DeviceInfo.getUniqueId().catch(() => "")}`;
    source = source === "none" ? "fallback" : source;
  }
  return {
    phoneNumber,
    fingerprint,
    carrier,
    simCount: simCount || (carrier ? 1 : 0),
  };
}

```

```

    hasSim: !!carrier || !!fingerprint,
    source,
  };
}
/* ----- */
/* ( Keychain ) */
/* ----- */
export async function getDeviceId(): Promise<string> {
  try {
    const saved = await Keychain.getGenericPassword({ service: DEVICE_ID_KEY });
    if (saved && saved.password) return saved.password;
  } catch {
    /* ignore */
  }
  const fresh = (await DeviceInfo.getUniqueId().catch(() => "")) || `${Date.now().toString(36)}${Math.random().toString(
36).slice(2)}`;
  const id = String(fresh).replace(/^[a-zA-Z0-9]/g, "").slice(0, 60);
  try {
    await Keychain.setGenericPassword(DEVICE_ID_KEY, id, { service: DEVICE_ID_KEY });
  } catch {
    /* ignore */
  }
  return id;
}
export async function getDeviceInfoLabel(): Promise<string> {
  const brand = await DeviceInfo.getBrand().catch(() => "");
  const model = await DeviceInfo.getModel().catch(() => "");
  const os = `${Platform.OS} ${await DeviceInfo.getSystemVersion().catch(() => "")}`;
  return `${brand} ${model} — ${os}.trim().slice(0, 200);
}
/* ----- */
/* ----- */
/* ----- */
function base64url(input: string) {
  return input.replace(/\+/g, "-").replace(/\//g, "_").replace(/=+$/, "");
}
function sign(canonical: string) {
  return base64url(HmacSHA256(canonical, APP_SECRET).toString(enc.Base64));
}
async function fetchJson(path: string, body: unknown, timeoutMs = 20000) {
  const ctrl = new AbortController();
  const timer = setTimeout(() => ctrl.abort(), timeoutMs);
  try {
    const res = await fetch(`${API_BASE}${path}`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(body),
      signal: ctrl.signal,
    });
    const data = await res.json().catch(() => ({}));
    return { ok: res.ok, status: res.status, data };
  } finally {
    clearTimeout(timer);
  }
}
/* ----- */
/* ----- */
/* ----- */
export type LoginResult =
  | { status: "success"; token: string; user: Record<string, unknown> }
  | { status: "need-otp"; message: string; masked?: string }
  | { status: "error"; message: string; code?: string };
export async function loginWithSim(): Promise<LoginResult> {
  const sim = await readSimInfo();
  const deviceId = await getDeviceId();
  const deviceInfo = await getDeviceInfoLabel();
  if (!sim.hasSim) {
    return { status: "error", message: " . .", code: "NO_SIM" }
  }
  // nonce ( Replay Attack)
  const nonceRes = await fetchJson("/api/mobile/nonce", { deviceId }).catch(() => null);
  if (!nonceRes?.ok) {
    return { status: "error", message: " . .", code: "NO_SIM" };
  }
  const nonce = String(nonceRes.data.nonce);
  //
  const timestamp = Number(nonceRes.data.serverTime ?? Date.now());
  //
  const canonical = [nonce, String(timestamp), deviceId, sim.phoneNumber, sim.fingerprint].join("|");
  const signature = sign(canonical);
  //
  const res = await fetchJson("/api/mobile/login-with-phone", {
    deviceId,

```

```

    deviceInfo,
    nonce,
    timestamp,
    signature,
    phoneNumber: sim.phoneNumber,
    simFingerprint: sim.fingerprint,
    simCarrier: sim.carrier,
    simCount: sim.simCount,
  }).catch(() => null);
  if (!res) return { status: "error", message: "    ." };
  if (res.ok && res.data.token) {
    await Keychain.setGenericPassword("token", String(res.data.token), { service: "sek_token" });
    return { status: "success", token: String(res.data.token), user: res.data.user };
  }
  if (res.data.needOtp) {
    return { status: "need-otp", message: String(res.data.error ?? ""), masked: res.data.masked };
  }
  return { status: "error", message: String(res.data.error ?? " "), code: res.data.code };
}
/* ----- */
/* ( ) */
/* ----- */
export async function requestOtp(username: string, password: string) {
  const deviceId = await getDeviceId();
  const res = await fetchJson("/api/auth/otp", { action: "request", username, password, deviceId });
  return { ok: res.ok, message: String(res.data.message ?? res.data.error ?? ""), masked: res.data.masked };
}
export async function verifyOtp(username: string, password: string, code: string) {
  const deviceId = await getDeviceId();
  const deviceInfo = await getDeviceInfoLabel();
  const res = await fetchJson("/api/auth/otp", {
    action: "verify",
    username,
    password,
    code,
    deviceId,
    deviceInfo,
  });
  if (res.ok && res.data.token) {
    await Keychain.setGenericPassword("token", String(res.data.token), { service: "sek_token" });
    return { ok: true, token: String(res.data.token), user: res.data.user };
  }
  return { ok: false, message: String(res.data.error ?? " ") };
}

export async function getSavedToken(): Promise<string | null> {
  try {
    const c = await Keychain.getGenericPassword({ service: "sek_token" });
    return c ? c.password : null;
  } catch {
    return null;
  }
}

@@ src/app/layout.tsx typescript
import type { Metadata, Viewport } from "next";
import type { ReactNode } from "react";
import "../globals.css";
import ServiceWorker from "@components/ServiceWorker";
import ConfirmProvider from "@components/Confirm";
import ConnectionStatus from "@components/ConnectionStatus";
export const metadata: Metadata = {
  title: " ",
  description: " ",
  applicationName: " ",
  manifest: "/manifest.webmanifest",
  icons: {
    icon: [
      { url: "/icons/icon-192.png", sizes: "192x192", type: "image/png" },
      { url: "/icons/icon-512.png", sizes: "512x512", type: "image/png" },
      { url: "/favicon-32.png", sizes: "32x32", type: "image/png" },
    ],
    apple: [{ url: "/apple-touch-icon.png", sizes: "180x180", type: "image/png" }],
    shortcut: ["/favicon-32.png"],
  },
  appleWebApp: { capable: true, title: " ", statusBarStyle: "default" },
  other: { "mobile-web-app-capable": "yes", "format-detection": "telephone=no" },
};
export const viewport: Viewport = {
  width: "device-width",
  initialScale: 1,
  maximumScale: 5,
  themeColor: "#0f766e",
};
export default function RootLayout({ children }: { children: ReactNode }) {

```

```

return (
  <html lang="fa" dir="rtl">
    <head>
      <!-- PWABuilder -->
      <script
        dangerouslySetInnerHTML={{
          __html:
            "if('serviceWorker' in navigator){navigator.serviceWorker.register('/sw.js',{scope:'/'}).catch(function(){
}};}",
        }}
      />
    </head>
    <body className="min-h-screen bg-slate-100 text-slate-900 antialiased">
      <ConfirmProvider>{children}</ConfirmProvider>
      <ServiceWorker />
      <ConnectionStatus />
    </body>
  </html>
);
}

```

```

@@ src/app/page.tsx typescript
"use client";

```

```

import React, { useEffect } from "react";

```

```

/**
 * Next.js Root Dashboard Page (/)
 * نام و آدرس: تهران، خیابان ولیعصر، پلاک ۱۰۰ نامی: ndcohub.ir
 * توضیحات: این صفحه برای نمایش اطلاعات کلی سیستم و مدیریت کاربران طراحی شده است.
 */
export default function HomePage() {
  useEffect(() => {
    document.title = "سیستم مدیریت اطلاعات پزشکی (CRM PWA)";
  }, []);

  return (
    <div style={{ width: "100%", height: "100vh", margin: 0, padding: 0, overflow: "hidden", backgroundColor: "#0f766e" }}>
      <iframe
        src="/index.html"
        title="CRM Medical Representative Application"
        style={{
          width: "100%",
          height: "100%",
          border: "none",
          display: "block",
        }}
        allow="geolocation; microphone; camera"
      />
    </div>
  );
}

```

```

@@ src/components/BackButton.tsx typescript

```

```

"use client";

```

```

import { usePathname, useRouter } from "next/navigation";

```

```

/** — */

```

```

export default function BackButton() {
  const router = useRouter();
  const pathname = usePathname();
  const isRoot = pathname === "/panel" || pathname === "/admin" || pathname === "/login" || pathname === "/";
  const home = pathname.startsWith("/admin") ? "/admin" : "/panel";
  return (
    <div className="mb-3 flex items-center gap-2">
      <button
        onClick={() => {
          if (window.history.length > 1) router.back();
          else router.push(home);
        }}
        className="inline-flex items-center gap-1 rounded-xl bg-white px-3 py-2 text-xs font-bold text-slate-700 shadow-sm ring-1 ring-slate-200 hover:bg-slate-50"
        title=" "
      >
        <span className="text-base">←</span>
      </button>
      {!isRoot ? (
        <button
          onClick={() => router.push(home)}
          className="inline-flex items-center gap-1 rounded-xl bg-white px-3 py-2 text-xs font-bold text-teal-700 shadow-sm ring-1 ring-slate-200 hover:bg-teal-50"
          title=" "
        >
          </button>

```



```

    ) : null}
  </div>
);
}

@@ src/components/Charts.tsx typescript
"use client";
import { toPersianDigits } from "@lib/jalali";
export type Series = { label: string; value: number; color?: string };
const PALETTE = ["#0d9488", "#0ea5e9", "#6366f1", "#f59e0b", "#e11d48", "#8b5cf6", "#10b981", "#f97316"];
export function BarChart({ data, height = 180, unit = "" }: { data: Series[]; height?: number; unit?: string }) {
  if (data.length === 0) return <Empty />;
  const max = Math.max(...data.map((d) => d.value), 1);
  return (
    <div className="scroll-x">
      <div className="flex items-end gap-2" style={{ height, minWidth: data.length * 52 }}>
        {data.map((d, i) => {
          const h = Math.max(4, (d.value / max) * (height - 34));
          return (
            <div key={`-${d.label}-${i}`} className="flex flex-1 flex-col items-center justify-end gap-1">
              <span className="text-[10px] font-black text-slate-700">{toPersianDigits(d.value)}</span>
              <div
                className="w-full rounded-t-lg transition-all"
                style={{ height: h, background: d.color ?? PALETTE[i % PALETTE.length], minWidth: 26 }}
                title={`-${d.label}: ${d.value}${unit}`}
              />
              <span className="w-full truncate text-center text-[9px] text-slate-500" title={d.label}>
                {d.label}
              </span>
            </div>
          );
        })}
      </div>
    </div>
  );
}
/** ( ) */
export function LineChart({
  labels,
  series,
  height = 200,
}: {
  labels: string[];
  series: { name: string; values: number[]; color?: string }[];
  height?: number;
}) {
  if (labels.length === 0 || series.length === 0) return <Empty />;
  const W = Math.max(320, labels.length * 46);
  const H = height;
  const pad = { t: 12, r: 8, b: 24, l: 34 };
  const max = Math.max(1, ...series.flatMap((s) => s.values));
  const x = (i: number) => pad.l + (i * (W - pad.l - pad.r)) / Math.max(1, labels.length - 1);
  const y = (v: number) => H - pad.b - (v / max) * (H - pad.t - pad.b);
  return (
    <div className="scroll-x">
      <svg width={W} height={H} className="min-w-full">
        {[0, 0.5, 1].map((f) => (
          <g key={f}>
            <line x1={pad.l} x2={W - pad.r} y1={y(max * f)} y2={y(max * f)} stroke="#e2e8f0" strokeWidth={1} />
            <text x={4} y={y(max * f) + 4} fontSize={9} fill="#94a3b8">
              {toPersianDigits(Math.round(max * f))}
            </text>
          </g>
        ))}
        {series.map((s, si) => {
          const color = s.color ?? PALETTE[si % PALETTE.length];
          const pts = s.values.map((v, i) => `${x(i)},${y(v)}`).join(" ");
          return (
            <g key={s.name}>
              <polyline points={pts} fill="none" stroke={color} strokeWidth={2.5} strokeLinejoin="round" />
              {s.values.map((v, i) => (
                <circle key={i} cx={x(i)} cy={y(v)} r={3} fill={color}>
                  <title`${s.name} — ${labels[i]}: ${v}`</title>
                </circle>
              ))}
            </g>
          );
        })}
      </svg>
      <div className="mt-1 flex flex-wrap gap-2">

```

```

    {series.map((s, i) => (
      <span key={s.name} className="flex items-center gap-1 text-[10px] font-bold text-slate-600">
        <span
          className="inline-block size-2.5 rounded-full"
          style={{ background: s.color ?? PALETTE[i % PALETTE.length] }}
        />
        {s.name}
      </span>
    ))}
  </div>
</div>
);
}
export function DonutChart({ data, size = 150 }: { data: Series[]; size?: number }) {
  const total = data.reduce((a, d) => a + d.value, 0);
  if (total === 0) return <Empty />;
  const r = size / 2 - 12;
  const c = 2 * Math.PI * r;
  let acc = 0;
  return (
    <div className="flex flex-wrap items-center gap-3">
      <svg width={size} height={size}>
        <g transform={`translate(${size / 2},${size / 2}) rotate(-90)`}>
          {data.map((d, i) => {
            const frac = d.value / total;
            const dash = `${frac * c} ${c}`;
            const off = -acc * c;
            acc += frac;
            return (
              <circle
                key={d.label}
                r={r}
                fill="none"
                stroke={d.color ?? PALETTE[i % PALETTE.length]}
                strokeWidth={18}
                strokeDasharray={dash}
                strokeDashoffset={off}
              >
                <title>`${d.label}: ${d.value}`</title>
              </circle>
            );
          })}
        </g>
        <text x="50%" y="50%" textAnchor="middle" dy="4" fontSize={14} fontWeight="bold" fill="#0f766e">
          {toPersianDigits(total)}
        </text>
      </svg>
      <div className="space-y-1">
        {data.map((d, i) => (
          <div key={d.label} className="flex items-center gap-1.5 text-[11px]">
            <span
              className="inline-block size-2.5 rounded-full"
              style={{ background: d.color ?? PALETTE[i % PALETTE.length] }}
            />
            <span className="font-bold text-slate-700">{d.label}</span>
            <span className="text-slate-500">{toPersianDigits(d.value)}</span>
          </div>
        ))}
      </div>
    </div>
  );
}
function Empty() {
  return <p className="py-8 text-center text-xs text-slate-400">    </p>;
}

```

```

@@ src/components/Combobox.tsx typescript

```

```

"use client";
import { useEffect, useMemo, useRef, useState } from "react";
import { useConfirm } from "../Confirm";
export type Opt = { value: string; parent?: string };
/**
 * « ».
 *
 * :
 * - .
 * - «» .
 * - .
 * - ( ) .
 */

```

```

export default function Combobox({
  value,
  onChange,

```

```

options,
category,
onAdded,
canAdd = true,
placeholder = "  ...",
parent = "",
parentLabel = "",
requireParent = false,
disabled = false,
selectOnly = false,
}: {
value: string;
onChange: (v: string) => void;
options: string[];
category?: string;
onAdded?: (v: string) => void;
canAdd?: boolean;
placeholder?: string;
/** ( ) */
parent?: string;
parentLabel?: string;
requireParent?: boolean;
disabled?: boolean;
/** — */
selectOnly?: boolean;
}) {
const [open, setOpen] = useState(false);
const [query, setQuery] = useState(value);
const [saving, setSaving] = useState(false);
const [typing, setTyping] = useState(false);
const [note, setNote] = useState("");
const ref = useRef<HTMLDivElement>(null);
const typingTimer = useRef<ReturnType<typeof setTimeout> | null>(null);
const confirm = useConfirm();
const blocked = disabled || (requireParent && !parent.trim());
useEffect(() => setQuery(value), [value]);
useEffect(() => {
if (!open) return;
const handler = (e: MouseEvent) => {
if (ref.current && !ref.current.contains(e.target as Node)) setOpen(false);
};
document.addEventListener("mousedown", handler);
return () => document.removeEventListener("mousedown", handler);
}, [open]);
useEffect(
(), => () => {
if (typingTimer.current) clearTimeout(typingTimer.current);
},
[],
);
const filtered = useMemo(() => {
const q = query.trim().toLowerCase();
if (!q) return options.slice(0, 80);
const words = q.split(/\s+/);
return options.filter(o => words.every(w => o.toLowerCase().includes(w))).slice(0, 80);
}, [query, options]);
const trimmed = query.trim();
const exact = options.some(o => o.trim() === trimmed);
//
const showAdd = !selectOnly && canAdd && !category && !blocked && trimmed.length >= 2 && !exact && !typing;
const markTyping = () => {
setTyping(true);
if (typingTimer.current) clearTimeout(typingTimer.current);
typingTimer.current = setTimeout(() => setTyping(false), 700);
};
const addNow = async () => {
if (!trimmed || !category || saving) return;
const ok = await confirm({
title: " ",
message: `«${trimmed}»${parent?` «${parent}»`: ""} \n
`,
confirmText: " ",
});
if (!ok) return;

setSaving(true);
const res = await fetch("/api/options", {
method: "POST",
headers: { "Content-Type": "application/json" },
body: JSON.stringify({ category, value: trimmed, parent }),
}).catch(() => null);
setSaving(false);
const data = await res?.json().catch(() => ({}));
if (res?.ok) {
onChange(trimmed);
}
}

```

```

onAdded?.(trimmed);
setOpen(false);
setNote(data?.duplicate ? "      " : "    ");
setTimeout(() => setNote(""), 2500);
} else {
  setNote(data?.error ?? " ");
  setTimeout(() => setNote(""), 3500);
}
};
return (
  <div className="relative" ref={ref}>
    <div className="flex gap-1">
      <input
        value={query}
        placeholder={blocked ? `${parentLabel} | " "` : placeholder}
        disabled={blocked}
        onFocus={() => !blocked && setOpen(true)}
        onChange={(e) => {
          setQuery(e.target.value);
          setOpen(true);
          markTyping();
        }}
        // «»
        if (!selectOnly) onChange(e.target.value);
      }>
      onBlur={() => {
        //
        if (selectOnly) setTimeout(() => setQuery(value), 150);
      }}
      onKeyDown={(e) => {
        // Enter
        if (e.key === "Enter") e.preventDefault();
        if (e.key === "Escape") setOpen(false);
      }}
      className="w-full rounded-xl border border-slate-300 bg-white px-3 py-2.5 text-sm outline-none transition focus:ring-2 focus:ring-teal-100 disabled:cursor-not-allowed disabled:bg-slate-100 disabled:text-slate-400"
    />
    {showAdd ? (
      <button
        type="button"
        onClick={addNow}
        disabled={saving}
        title={`«${trimmed}»`}
        className="shrink-0 rounded-xl bg-emerald-600 px-3 text-xs font-bold text-white hover:bg-emerald-700 disabled:opacity-50"
      >
        {saving ? "... " : " "}
      </button>
    ) : null}
    </div>
    {note ? <p className="mt-1 text-[10px] font-bold text-emerald-700">{note}</p> : null}
    {selectOnly && !blocked && options.length > 0 ? (
      <p className="mt-1 text-[10px] text-slate-400">{options.length.toLocaleString("fa-IR")} </p>
    ) : null}
    {open && !blocked ? (
      <div className="fade-in absolute z-30 mt-1 max-h-56 w-full overflow-y-auto rounded-xl bg-white py-1 shadow-xl ring-1 ring-slate-200">
        {filtered.length === 0 ? (
          <div className="px-3 py-2 text-[11px] text-slate-400">
            {selectOnly
              ? " — "
              : trimmed.length >= 2
                ? " — «» "
                : "   "}
          </div>
        ) : null}
        {filtered.map((o) => (
          <button
            key={o}
            type="button"

            onMouseDown={(e) => e.preventDefault()}
            onClick={() => {
              onChange(o);
              setQuery(o);
              setOpen(false);
            }}
            className={`${block} w-full px-3 py-2 text-right text-sm hover:bg-teal-50 ${
              o === value ? "bg-teal-50 font-bold text-teal-700" : "text-slate-700"
            }`}
          >
            {o}
          </button>
        ))}
      </div>
    ) : null}
  </div>

```

```

    ) : null}
  </div>
);
}

@@ src/components/Confirm.tsx typescript
"use client";
import { createContext, useCallback, useContext, useState, type ReactNode } from "react";
type Opts = {
  title: string;
  message?: string;
  confirmText?: string;
  cancelText?: string;
  danger?: boolean;
};
type Ctx = (o: Opts) => Promise<boolean>;
const ConfirmCtx = createContext<Ctx>(async () => true);
/** : `const confirm = useConfirm(); if (!(await confirm({title:'...'}))) return;` */
export function useConfirm() {
  return useContext(ConfirmCtx);
}
export default function ConfirmProvider({ children }: { children: ReactNode }) {
  const [state, setState] = useState<(Opts & { resolve: (v: boolean) => void }) | null>(null);
  const confirm = useCallback<Ctx>((o) => new Promise<boolean>((resolve) => setState({ ...o, resolve })),
    [],
  );
  const close = (v: boolean) => {
    state?.resolve(v);
    setState(null);
  };
  return (
    <ConfirmCtx.Provider value={confirm}>
      {children}
      {state ? (
        <div
          className="fixed inset-0 z-[100] flex items-end justify-center bg-slate-900/60 p-0 sm:items-center sm:p-4"
          onClick={() => close(false)}
        >
          <div
            className="fade-in w-full max-w-sm rounded-t-3xl bg-white p-4 shadow-2xl sm:rounded-2xl"
            onClick={(e) => e.stopPropagation()}
          >
            <div className="mb-2 flex items-start gap-3">
              <span className={`text-2xl ${state.danger ? "" : ""}`}>{state.danger ? "" : ""}</span>
              <div className="min-w-0 flex-1">
                <h3 className="text-sm font-black text-slate-800">{state.title}</h3>
                {state.message ? (
                  <p className="mt-1 whitespace-pre-wrap text-xs leading-6 text-slate-600">{state.message}</p>
                ) : null}
              </div>
            </div>
            <div className="mt-4 flex gap-2">
              <button
                onClick={() => close(true)}
                className={`flex-1 rounded-xl px-4 py-2.5 text-sm font-bold text-white ${
                  state.danger ? "bg-rose-600 hover:bg-rose-700" : "bg-teal-600 hover:bg-teal-700"
                }`}
              >
                {state.confirmText ?? ""}
              </button>
              <button
                onClick={() => close(false)}
                className="flex-1 rounded-xl bg-white px-4 py-2.5 text-sm font-bold text-slate-700 ring-1 ring-slate-300
                hover:bg-slate-50"
              >
                {state.cancelText ?? ""}
              </button>
            </div>
          </div>
        ) : null}
      </ConfirmCtx.Provider>
    );
}

@@ src/components/ConnectionStatus.tsx typescript
"use client";
import { useCallback, useEffect, useRef, useState } from "react";
/**
 * + .
 *
 * :
 * - « »
 */

```

```

*
* - «»
* ( navigator.onLine ).
* - « »
*
*/
type State = "online" | "checking" | "offline";
const PROBE_URL = "/ping";
const CHECK_INTERVAL = 25_000;
export default function ConnectionStatus() {
  const [state, setState] = useState<State>("online");
  const [pending, setPending] = useState(0);
  const [flash, setFlash] = useState("");
  const [sending, setSending] = useState(false);
  const [dismissed, setDismissed] = useState(false);
  const failures = useRef(0);
  /** ( ) */
  const probe = useCallback(async (): Promise<boolean> => {
    if (typeof navigator !== "undefined" && !navigator.onLine) return false;
    try {
      const ctrl = new AbortController();
      const timer = setTimeout(() => ctrl.abort(), 8000);
      const res = await fetch(`${PROBE_URL}?t=${Date.now()}`, {
        cache: "no-store",
        signal: ctrl.signal,
        credentials: "omit",
      });
      clearTimeout(timer);
      return res.ok;
    } catch {
      return false;
    }
  }, []);
  const check = useCallback(async () => {
    const ok = await probe();
    if (ok) {
      failures.current = 0;
      setState("online");
      setDismissed(false);
      //
      navigator.serviceWorker?.controller?.postMessage("flush");
    } else {
      failures.current += 1;
      //
      setState(failures.current >= 2 ? "offline" : "checking");
    }
    navigator.serviceWorker?.controller?.postMessage("queue-count");
  }, [probe]);
  useEffect(() => {
    check();
    const iv = setInterval(() => {
      if (!document.hidden) check();
    }, CHECK_INTERVAL);
    const onOnline = () => check();

    const onFocus = () => check();
    window.addEventListener("online", onOnline);
    window.addEventListener("offline", () => setState("offline"));
    window.addEventListener("focus", onFocus);
    document.addEventListener("visibilitychange", onFocus);
    const onMsg = (e: MessageEvent) => {
      if (e.data?.type === "queue-status") {
        setPending(e.data.pending ?? 0);
        if (e.data.sent > 0) {
          setFlash(`${e.data.sent}`);
          setTimeout(() => setFlash(""), 6000);
        }
      }
      if (e.data?.type === "queued") {
        setPending((p) => p + 1);
        setFlash("");
        setTimeout(() => setFlash(""), 6000);
      }
    };
    navigator.serviceWorker?.addEventListener("message", onMsg);
    return () => {
      clearInterval(iv);
      window.removeEventListener("online", onOnline);
      window.removeEventListener("focus", onFocus);
      document.removeEventListener("visibilitychange", onFocus);
      navigator.serviceWorker?.removeEventListener("message", onMsg);
    };
  }, [check]);
  const sendNow = async () => {
    setSending(true);
  }
}

```

```

setFlash("    ...");
const ok = await probe();
if (!ok) {
  setSending(false);
  setFlash("    .    .");
  setTimeout(() => setFlash(""), 6000);
  return;
}
navigator.serviceWorker?.controller?.postMessage("flush");
setTimeout(() => {
  navigator.serviceWorker?.controller?.postMessage("queue-count");
  setSending(false);
}, 2500);
};
const offline = state === "offline";
const showBar = (offline && !dismissed) || pending > 0 || !!flash;
if (!showBar) return null;
return (
  <div className="pointer-events-none fixed inset-x-0 bottom-0 z-[90] flex flex-col items-center gap-2 p-3">
    {flash ? (
      <div className="pointer-events-auto w-full max-w-md rounded-xl bg-slate-900/95 px-3 py-2 text-center text-xs font-bold text-white shadow-lg">
        {flash}
      </div>
    ) : null}
    {offline && !dismissed ? (
      <div className="pointer-events-auto w-full max-w-md rounded-2xl bg-amber-500 px-3 py-2.5 text-white shadow-xl">
        <div className="flex items-start gap-2">
          <span className="text-lg"></span>
          <div className="min-w-0 flex-1">
            <div className="text-xs font-black"> </div>
            <div className="mt-0.5 text-[11px] leading-5 text-amber-50">
              .
            </div>
            <div className="mt-2 flex flex-wrap gap-2">
              <button
                onClick={check}
                className="rounded-lg bg-white px-3 py-1 text-[11px] font-bold text-amber-700"
              >
                </button>
              <a
                href="/diagnostics"
                className="rounded-lg bg-white/20 px-3 py-1 text-[11px] font-bold text-white ring-1 ring-white/40"
              >
                </a>
              <button
                onClick={() => setDismissed(true)}
                className="rounded-lg bg-white/20 px-3 py-1 text-[11px] font-bold text-white ring-1 ring-white/40"
              >
                </button>
            </div>
          </div>
        </div>
      </div>
    ) : null}
    {pending > 0 ? (
      <div className="pointer-events-auto w-full max-w-md rounded-2xl bg-teal-700 px-3 py-2.5 text-white shadow-xl">
        <div className="flex flex-wrap items-center gap-2">
          <span className="text-lg"></span>
          <div className="min-w-0 flex-1">
            <div className="text-xs font-black">
              {pending.toLocaleString("fa-IR")}
            </div>
            <div className="text-[11px] text-teal-50">
              {offline ? " " : " — "}
            </div>
          </div>
          <button
            onClick={sendNow}
            disabled={sending}
            className="rounded-lg bg-white px-3 py-1.5 text-[11px] font-black text-teal-800 disabled:opacity-60"
          >
            {sending ? "    ..." : " "}
          </button>
        </div>
      </div>
    ) : null}
  </div>
);
}

```

```

@@ src/components/FileUploader.tsx typescript
"use client";
import { useCallback, useEffect, useRef, useState } from "react";
import { Badge, Button } from "../ui";
import { toPersianDigits } from "@lib/jalali";
export type Att = {
  id: number;
  fileName: string;
  mimeType: string;
  size: number;
  repName?: string;
  ownerId?: number | null;
};
const fmtSize = (n: number) =>
  n > 1024 * 1024 ? `${toPersianDigits((n / 1024 / 1024).toFixed(1))} ` : `${toPersianDigits(Math.round(n / 1024))} `;
export function printFile(id: number, mime: string) {
  const url = `/api/attachments/${id}`;
  const w = window.open("", "_blank", "width=900,height=700");
  if (!w) return;
  const content = mime.startsWith("image/")
    ? ``
    : `<iframe src="${url}" style="border:0;width:100%;height:95vh" onload="window.focus();setTimeout(()=>window.print(),600)"></iframe>`;
  w.document.write(
    `<html dir="rtl"><head><meta charset="utf-8"><title> </title></head><body style="margin:0">${content}</body></html>`,
  );
  w.document.close();
}
export function FileList({
  files,
  onDelete,
  compact = false,
}): {
  files: Att[];
  onDelete?: (id: number) => void;
  compact?: boolean;
} {
  if (files.length === 0) return <p className="text-xs text-slate-400"> </p>;
  return (
    <div className={`grid gap-2 ${compact ? "grid-cols-2 sm:grid-cols-3" : "grid-cols-1 sm:grid-cols-2"}`>
      {files.map((f) => (
        <div key={f.id} className="rounded-xl bg-slate-50 p-2 ring-1 ring-slate-200">
          {f.mimeType.startsWith("image/") ? (
            // eslint-disable-next-line @next/next/no-img-element
            <img
              src={`/${api}/attachments/${f.id}`}
              alt={f.fileName}
              className="mb-2 h-28 w-full cursor-pointer rounded-lg object-cover"
              onClick={() => window.open(`${api}/attachments/${f.id}`, "_blank")}
            />
          ) : (
            <div className="mb-2 flex h-28 items-center justify-center rounded-lg bg-white text-3xl"></div>
          )}
          <div className="truncate text-[11px] font-bold text-slate-700" title={f.fileName}>
            {f.fileName}
          </div>
          <div className="text-[10px] text-slate-400">
            {fmtSize(f.size)} {f.repName ? `— ${f.repName}` : ""}
          </div>
          <div className="mt-1 flex flex-wrap gap-1">
            <a
              href={`/${api}/attachments/${f.id}`}
              target="_blank"
              rel="noreferrer"
              className="rounded-lg bg-sky-100 px-2 py-1 text-[10px] font-bold text-sky-700"
            >
            </a>
            <a
              href={`/${api}/attachments/${f.id}?download=1`}
              className="rounded-lg bg-emerald-100 px-2 py-1 text-[10px] font-bold text-emerald-700"
            >
            </a>
            <button
              onClick={() => printFile(f.id, f.mimeType)}
              className="rounded-lg bg-slate-200 px-2 py-1 text-[10px] font-bold text-slate-700"
            >
            </button>
            {onDelete ? (

```



```

        <button
            onClick={() => onDelete(f.id)}
            className="rounded-lg bg-rose-100 px-2 py-1 text-[10px] font-bold text-rose-700"
        >

            </button>
        ) : null}
    </div>
</div>
    )}
</div>
);
}
/** /. . */
export default function FileUploader({
    ownerType = "doctor",
    ownerId = null,
    onChangeIds,
}): {
    ownerType?: string;
    ownerId?: number | null;
    onChangeIds?: (ids: number[]) => void;
}) {
    const [files, setFiles] = useState<Att[]>([]);
    const [busy, setBusy] = useState(false);
    const [err, setErr] = useState("");
    const inputRef = useRef<HTMLInputElement>(null);
    const cb = useRef(onChangeIds);
    cb.current = onChangeIds;
    useEffect(() => {
        cb.current?.(files.map((f) => f.id));
    }, [files]);
    const load = useCallback(async () => {
        if (!ownerId) return;
        const res = await fetch(`/api/attachments?ownerType=${ownerType}&ownerId=${ownerId}`, { cache: "no-store" });
        if (res.ok) setFiles((await res.json()).rows ?? []);
    }, [ownerType, ownerId]);
    useEffect(() => {
        load();
    }, [load]);

    const upload = async (list: FileList | null) => {
        if (!list?.length) return;
        setErr("");
        setBusy(true);
        for (const file of Array.from(list).slice(0, 5)) {
            if (file.size > 3 * 1024 * 1024) {
                setErr(`«{file.name}» `);
                continue;
            }
            const data = await new Promise<string>((resolve) => {
                const r = new FileReader();
                r.onload = () => resolve(String(r.result));
                r.readAsDataURL(file);
            });
            const res = await fetch("/api/attachments", {
                method: "POST",
                headers: { "Content-Type": "application/json" },
                body: JSON.stringify({ ownerType, ownerId, fileName: file.name, data }),
            });
            if (res.ok) {
                const d = await res.json();
                setFiles((p) => [d.row, ...p]);
            } else {
                const d = await res.json().catch(() => ({}));
                setErr(d.error ?? " ");
            }
        }
        setBusy(false);
        if (inputRef.current) inputRef.current.value = "";
    };

    const remove = async (id: number) => {
        await fetch(`/api/attachments?id=${id}`, { method: "DELETE" });
        setFiles((p) => p.filter((f) => f.id !== id));
    };

    return (
        <div className="space-y-2">
            <div className="flex flex-wrap items-center gap-2">
                <input
                    ref={inputRef}
                    type="file"
                    multiple
                    accept="image/*,application/pdf,.doc,.docx,.xls,.xlsx"
                    onChange={(e) => upload(e.target.files)}
                    className="hidden"

```

```

        id={`file-${ownerType}`}
      />
      <Button variant="soft" onClick={() => inputRef.current?.click()} disabled={busy}>
        {busy ? "    " : "    "}
      </Button>
      <Badge tone="slate">{toPersianDigits(files.length)} </Badge>
    <span className="text-[11px] text-slate-400"> PDF — </span>
  </div>
  {err ? <p className="text-xs font-bold text-rose-600">{err}</p> : null}
  <FileList files={files} onDelete={remove} compact />
</div>
);
}

@@ src/components/JalaliDateInput.tsx typescript
"use client";
import { useEffect, useMemo, useRef, useState } from "react";
import {
  JALALI_MONTHS,
  WEEK_DAYS,
  formatJalali,
  jalaliMonthLength,
  jalaliWeekDay,
  maskJalaliInput,
  parseJalali,
  toPersianDigits,
  todayJalali,
} from "@lib/jalali";
export default function JalaliDateInput({
  value,
  onChange,
  placeholder = "/",
}: {
  value: string;
  onChange: (v: string) => void;
  placeholder?: string;
}) {
  const [open, setOpen] = useState(false);
  const boxRef = useRef<HTMLDivElement>(null);
  const parsed = parseJalali(value) ?? parseJalali(todayJalali());
  const [view, setView] = useState<[number, number]>([parsed[0], parsed[1]]);
  useEffect(() => {
    const p = parseJalali(value);
    if (p) setView([p[0], p[1]]);
  }, [value]);
  useEffect(() => {
    if (!open) return;
    const handler = (e: MouseEvent) => {
      if (boxRef.current && !boxRef.current.contains(e.target as Node)) setOpen(false);
    };
    document.addEventListener("mousedown", handler);
    return () => document.removeEventListener("mousedown", handler);
  }, [open]);
  const grid = useMemo(() => {
    const [jy, jm] = view;
    const len = jalaliMonthLength(jy, jm);
    const first = jalaliWeekDay(jy, jm, 1);
    const cells: (number | null)[] = Array(first).fill(null);
    for (let d = 1; d <= len; d++) cells.push(d);
    while (cells.length % 7 !== 0) cells.push(null);
    return cells;
  }, [view]);
  const selected = parseJalali(value);
  return (
    <div className="relative" ref={boxRef}>
      <div className="flex gap-2">
        <input
          value={value}
          inputMode="numeric"
          placeholder={placeholder}
          onChange={(e) => onChange(maskJalaliInput(e.target.value))}
          className="w-full rounded-xl border border-slate-300 bg-white px-3 py-2.5 text-sm tracking-widest outline-none
transition focus:border-teal-500 focus:ring-2 focus:ring-teal-100"
        />
        <button
          type="button"
          onClick={() => setOpen((v) => !v)}
          className="shrink-0 rounded-xl bg-teal-50 px-3 text-lg ring-1 ring-teal-200 hover:bg-teal-100"
          title=" "
        >
          </button>
      </div>
      {open ? (

```

```

<div className="fade-in absolute z-30 mt-2 w-[19rem] max-w-[92vw] rounded-2xl bg-white p-3 shadow-xl ring-1 ring
-slate-200">
  <div className="mb-2 flex items-center justify-between gap-1">
    <button
      type="button"
      className="rounded-lg px-2 py-1 text-slate-600 hover:bg-slate-100"
      onClick={() => setView([y, m]) => (m === 1 ? [y - 1, 12] : [y, m - 1])}>
    >
    </button>
    <div className="flex items-center gap-2 text-sm font-bold text-slate-700">
      <select
        value={view[1]}
        onChange={(e) => setView([y]) => [y, Number(e.target.value)]}
        className="rounded-lg border border-slate-200 px-2 py-1 text-xs"
      >
        {JALALI_MONTHS.map((m, i) => (
          <option key={m} value={i + 1}>
            {m}
          </option>
        ))}
      </select>
      <input
        type="number"
        value={view[0]}
        onChange={(e) => setView([, m]) => [Number(e.target.value) || 1400, m]}
        className="w-20 rounded-lg border border-slate-200 px-2 py-1 text-xs"
      />
    </div>
    <button
      type="button"
      className="rounded-lg px-2 py-1 text-slate-600 hover:bg-slate-100"
      onClick={() => setView([y, m]) => (m === 12 ? [y + 1, 1] : [y, m + 1])}>
    >
    </button>
  </div>
  <div className="grid grid-cols-7 gap-1 text-center text-[11px] font-bold text-slate-400">
    {WEEK_DAYS.map((d) => (
      <div key={d}>{d}</div>
    ))}
  </div>
  <div className="mt-1 grid grid-cols-7 gap-1">
    {grid.map((d, i) => {
      const isSel =
        d !== null && selected && selected[0] === view[0] && selected[1] === view[1] && selected[2] === d;
      return (
        <button
          key={i}
          type="button"
          disabled={d === null}
          onClick={() => {
            if (d === null) return;
            onChange(formatJalali(view[0], view[1], d));
            setOpen(false);
          }}
          className={`h-8 rounded-lg text-xs font-semibold transition ${
            d === null
              ? "cursor-default"
              : isSel
                ? "bg-teal-600 text-white"
                : "text-slate-700 hover:bg-teal-50"
            }`}
        >
          {d === null ? "" : toPersianDigits(d)}
        </button>
      );
    })}
  </div>
  <button
    type="button"
    onClick={() => {
      onChange(todayJalali());
      setOpen(false);
    }}
    className="mt-2 w-full rounded-xl bg-slate-100 py-2 text-xs font-bold text-slate-700 hover:bg-slate-200"
  >
    : {toPersianDigits(todayJalali())}
  </button>
</div>
) : null}
</div>
);
}

```

```

@@ src/components/LocationPicker.tsx typescript
"use client";
import { useCallback, useEffect, useRef, useState } from "react";
import dynamic from "next/dynamic";
import { Badge, Button, Input } from "../ui";
import { toPersianDigits } from "@lib/jalali";
const MapBox = dynamic(() => import("../MapBox"), {
  ssr: false,
  loading: () => (
    <div className="flex h-[240px] items-center justify-center rounded-2xl bg-slate-100 text-sm text-slate-400">
      ...
    </div>
  ),
});
export type LatLng = { lat: number | null; lng: number | null; accuracy: number | null };
type Hit = { label: string; lat: number; lng: number; type: string };
/**
 *
 * ) : ( + )
 * ) GPS ( )
 * ) /
 */
export default function LocationPicker({
  value,
  onChange,
  label,
  height = 260,
  /** ( + ) */

  suggestQuery = "",
}: {
  value: LatLng;
  onChange: (v: LatLng) => void;
  label?: string;
  height?: number;
  suggestQuery?: string;
}) {
  const [status, setStatus] = useState("");
  const [busy, setBusy] = useState(false);
  // --- ---
  const [query, setQuery] = useState("");
  const [hits, setHits] = useState<Hit[]>([]);
  const [searching, setSearching] = useState(false);
  const [searchMsg, setSearchMsg] = useState("");
  const [openList, setOpenList] = useState(false);
  const [address, setAddress] = useState("");
  const watchRef = useRef<number | null>(null);
  const timerRef = useRef<ReturnType<typeof setTimeout> | null>(null);
  const autoRef = useRef<ReturnType<typeof setTimeout> | null>(null);
  const bestRef = useRef<number>(Number.POSITIVE_INFINITY);
  const boxRef = useRef<HTMLDivElement>(null);
  /* ----- */
  const runSearch = useCallback(async (q: string, silent = false) => {
    const term = q.trim();
    if (term.length < 2) {
      setHits([]);
      if (!silent) setSearchMsg(" ");
      return;
    }
    setSearching(true);
    if (!silent) setSearchMsg(" ...");
    try {
      const res = await fetch(`/api/geocode?q=${encodeURIComponent(term)}`, { cache: "no-store" });
      const d = await res.json();
      const list: Hit[] = d.results ?? [];
      setHits(list);
      setOpenList(list.length > 0);
      setSearchMsg(
        list.length ? `${toPersianDigits(list.length)} — ` : " ",
      );
    } catch {
      setSearchMsg(" — ");
    } finally {
      setSearching(false);
    }
  }, []);
  /** */
  useEffect(() => {
    if (autoRef.current) clearTimeout(autoRef.current);
    const term = query.trim();
    if (term.length < 3) {
      setHits([]);
      return;
    }
  }

```

```

    autoRef.current = setTimeout(() => void runSearch(term, true), 800);
    return () => {
        if (autoRef.current) clearTimeout(autoRef.current);
    };
}, [query, runSearch]);
/**
 */
useEffect(() => {
    if (!openList) return;
    const h = (e: MouseEvent) => {
        if (boxRef.current && !boxRef.current.contains(e.target as Node)) setOpenList(false);
    };
    document.addEventListener("mousedown", h);
    return () => document.removeEventListener("mousedown", h);
}, [openList]);
const pickHit = (h: Hit) => {
    onChange({ lat: h.lat, lng: h.lng, accuracy: 0 });
    setQuery(h.label);
    setAddress(h.label);
    setOpenList(false);
    setSearchMsg(" — ");
};
/**
 */

const reverseLookup = useCallback(async (lat: number, lng: number) => {
    try {
        const res = await fetch(`/api/geocode?lat=${lat}&lng=${lng}`, { cache: "no-store" });
        const d = await res.json();
        if (d.results?.[0]?.label) setAddress(d.results[0].label);
    } catch {
        /* ignore */
    }
}, []);
/* ----- GPS ----- */
const stop = useCallback(() => {
    if (watchRef.current !== null) {
        navigator.geolocation.clearWatch(watchRef.current);
        watchRef.current = null;
    }
    if (timerRef.current) {
        clearTimeout(timerRef.current);
        timerRef.current = null;
    }
    setBusy(false);
}, []);
useEffect(() => () => stop(), [stop]);
const locate = () => {
    if (!navigator.geolocation) {
        setStatus(" ");
        return;
    }
    if (!window.isSecureContext) setStatus(" GPS https ");
    stop();
    bestRef.current = Number.POSITIVE_INFINITY;
    setBusy(true);
    setStatus(" ... ( )");
    watchRef.current = navigator.geolocation.watchPosition(
        (pos) => {
            const acc = pos.coords.accuracy ?? 9999;
            if (acc < bestRef.current) {
                bestRef.current = acc;
                onChange({ lat: pos.coords.latitude, lng: pos.coords.longitude, accuracy: acc });
            }
            setStatus(`${Math.round(acc)} — `);
            if (acc <= 8) {
                stop();
                setStatus(`${Math.round(acc)} `);
                void reverseLookup(pos.coords.latitude, pos.coords.longitude);
            }
        },
        (err) => {
            stop();
            setStatus(
                err.code === 1
                ? " "
                : " — .",
            );
        },
        { enableHighAccuracy: true, maximumAge: 0, timeout: 20000 },
    );
    timerRef.current = setTimeout(() => {
        stop();
        if (bestRef.current === Number.POSITIVE_INFINITY) {
            setStatus(" ");
        } else {
            setStatus(` : ${Math.round(bestRef.current)} `);
        }
    }, 800);
}, [
    query,
    runSearch,
    onChange,
    setAddress,
    setOpenList,
    setSearchMsg,
    reverseLookup,
    stop,
    bestRef,
    setBusy,
    setStatus,
    watchRef,
    timerRef,
]);

```

```

    if (value.lat && value.lng) void reverseLookup(value.lat, value.lng);
  }
}, 15000);
};
const acc = value.accuracy ?? null;
const accTone = acc === null ? "amber" : acc <= 20 ? "green" : acc <= 60 ? "amber" : "slate";
return (
  <div className="space-y-2">
    {/* ----- */}
    <div className="relative" ref={boxRef}>
      <div className="flex gap-1">
        <Input
          value={query}
          onChange={(e) => setQuery(e.target.value)}
          onFocus={() => hits.length && setOpenList(true)}
          onKeyDown={(e) => {
            if (e.key === "Enter") {
              e.preventDefault();
              void runSearch(query);
            }
          }}
          placeholder="    ... (:   )"
        />
        <Button variant="soft" onClick={() => void runSearch(query)} disabled={searching}>
          {searching ? "... " : " "}
        </Button>
        {suggestQuery.trim() ? (
          <Button
            variant="ghost"
            onClick={() => {
              setQuery(suggestQuery);
              void runSearch(suggestQuery);
            }}
            title="    "
          >

        </Button>
        ) : null}
      </div>
      {searchMsg ? <p className="mt-1 text-[11px] text-slate-500">{searchMsg}</p> : null}
      {openList && hits.length > 0 ? (
        <div className="fade-in absolute z-30 mt-1 max-h-56 w-full overflow-y-auto rounded-xl bg-white py-1 shadow-xl ring-1 ring-slate-200">
          {hits.map((h, i) => (
            <button
              key={` ${h.lat}-${h.lng}-${i}`}
              type="button"
              onMouseDown={(e) => e.preventDefault()}
              onClick={() => pickHit(h)}
              className="block w-full px-3 py-2 text-right text-xs leading-5 text-slate-700 hover:bg-teal-50"
            >
              {h.label}
            </button>
          ))}
        </div>
      ) : null}
    </div>
    {/* ----- GPS ----- */}
    <div className="flex flex-wrap items-center gap-2">
      <Button variant="soft" onClick={locate} disabled={busy}>
        {busy ? "    ..." : "    "}
      </Button>
      {busy ? (
        <Button
          variant="ghost"
          onClick={() => {
            stop();
            setStatus(" ");
          }}
        >

      </Button>
      ) : null}
      {value.lat && value.lng ? (
        <>
          <Badge tone={accTone as "green" | "amber" | "slate"}>
            {acc ? ` ≈ ${toPersianDigits(Math.round(acc))}` : ""}
          </Badge>
          <Button variant="ghost" onClick={() => void reverseLookup(value.lat!, value.lng!)}>

        </Button>
      </>
      ) : (
        <Badge tone="amber"> </Badge>

```

```

    })
  </div>
  {status ? <p className="text-[11px] text-slate-500">{status}</p> : null}
  {address ? (
    <p className="rounded-lg bg-slate-50 px-2 py-1.5 text-[11px] text-slate-600 ring-1 ring-slate-200">
      {address}
    </p>
  ) : null}
  <p className="text-[11px] text-slate-400">
    .
  </p>
  <MapBox
    height={height}

    draggable
    accuracy={acc}
    points={value.lat && value.lng ? [{ lat: value.lat, lng: value.lng, label: label || "" }] : []}
    onPick={(p) => onChange({ lat: p.lat, lng: p.lng, accuracy: 0 })}
  />
  <div className="grid grid-cols-2 gap-2">
    <Input
      placeholder=" (lat)"
      value={value.lat ?? ""}
      onChange={(e) => onChange({ ...value, lat: Number(e.target.value) || null })}
      className="text-left text-xs"
    />
    <Input
      placeholder=" (lng)"
      value={value.lng ?? ""}
      onChange={(e) => onChange({ ...value, lng: Number(e.target.value) || null })}
      className="text-left text-xs"
    />
  </div>
</div>
);
}

```

```

@@ src/components/MapBox.tsx typescript

```

```

"use client";
import { useEffect, useRef, useState } from "react";
import "leaflet/dist/leaflet.css";
import type { Map as LeafletMap, Layer } from "leaflet";
import { PROVINCE_EN_FA } from "@lib/geo";
export type MapPoint = { lat: number; lng: number; label?: string; color?: string };
/**
  (//) */
export type MapArea = { lat: number; lng: number; zoom?: number; radiusKm?: number; name?: string };
function esc(s: string) {
  return s
    .replace(/&/g, "&")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;");
}
/**
  *
  *
  * : OpenStreetMap
  * /api/map/tiles` 403 .
  */
export default function MapBox({
  points = [],
  path = [],
  onPick,
  height = 260,
  center,
  zoom = 14,
  follow = false,
  accuracy,
  draggable = false,
  area,
  showIranProvinces = false,
  selectedProvince = "",
  onProvinceSelect,
}: {
  points?: MapPoint[];
  path?: { lat: number; lng: number }[];
  onPick?: (p: { lat: number; lng: number }) => void;
  height?: number;
  center?: { lat: number; lng: number } | null;
  zoom?: number;
  follow?: boolean;
  accuracy?: number | null;
  draggable?: boolean;
  area?: MapArea | null;
  /**
    */

```

```

showIranProvinces?: boolean;
selectedProvince?: string;
onProvinceSelect?: (province: string) => void;
}) {
const divRef = useRef<HTMLDivElement>(null);
const mapRef = useRef<LeafletMap | null>(null);
const layersRef = useRef<Layer[]>([]);
const provinceLayerRef = useRef<Layer | null>(null);

const tileLayerRef = useRef<Layer | null>(null);
const pickRef = useRef(onPick);
const provinceCbRef = useRef(onProvinceSelect);
const [tileState, setTileState] = useState<"loading" | "ready" | "fallback">("loading");
const [mapError, setMapError] = useState("");
pickRef.current = onPick;
provinceCbRef.current = onProvinceSelect;
const pointsKey = points.map((p) => `${p.lat.toFixed(6)},${p.lng.toFixed(6)},${p.label ?? ""}`).join(";");
const pathKey = `${path.length}:${path.length} ? ${path[path.length - 1].lat.toFixed(5)},${path[path.length - 1].lng.toFixed(5)}` : "";
const centerKey = center ? `${center.lat.toFixed(6)},${center.lng.toFixed(6)}` : "";
const areaKey = area ? `${area.lat.toFixed(4)},${area.lng.toFixed(4)},${area.zoom ?? ""}` : "";
useEffect(() => {
  let disposed = false;
  (async () => {
    try {
      const L = (await import("leaflet")).default;
      if (disposed || !divRef.current || mapRef.current) return;
      const start = center ?? points[0] ?? path[0] ?? area ?? { lat: 32.4279, lng: 53.688 };
      const startZoom = points.length === 0 && path.length === 0 && area?.zoom ? area.zoom : zoom;
      const map = L.map(divRef.current, {
        attributionControl: true,
        zoomControl: true,
        preferCanvas: true,
        minZoom: 4,
        maxZoom: 19,
      }).setView([start.lat, start.lng], startZoom);
      map.attributionControl.setPrefix(false);
      // 403 / .
      const tiles = L.tileLayer("/api/map/tiles/{z}/{x}/{y}.png", {
        maxZoom: 19,
        minZoom: 4,
        updateWhenIdle: true,
        keepBuffer: 3,
        crossOrigin: false,
      });
      attribution: '© <a href="https://www.openstreetmap.org/copyright" target="_blank">OpenStreetMap</a>',
    });
    let loaded = 0;
    let failed = 0;
    tiles.on("tileload", () => {
      loaded++;
      if (loaded >= 1) setTileState(failed > loaded ? "fallback" : "ready");
    });
    tiles.on("tileerror", () => {
      failed++;
      if (failed >= 3 && loaded === 0) setTileState("fallback");
    });
    tiles.addTo(map);
    tileLayerRef.current = tiles;
    map.on("click", (e: { latlng: { lat: number; lng: number } }) => {
      pickRef.current?.({ lat: e.latlng.lat, lng: e.latlng.lng });
    });
    mapRef.current = map;
    setTimeout(() => map.invalidateSize(), 150);
    drawRecords(L, map);
    if (showIranProvinces) await drawProvinces(L, map, selectedProvince);
  } catch (err) {
    setMapError(err instanceof Error ? err.message : " ");
  }
})();
return () => {
  disposed = true;
  provinceLayerRef.current?.remove();
  tileLayerRef.current?.remove();
  mapRef.current?.remove();
  provinceLayerRef.current = null;
  tileLayerRef.current = null;
  mapRef.current = null;
};
// eslint-disable-next-line react-hooks/exhaustive-deps
}, []);
useEffect(() => {
  let cancelled = false;
  (async () => {
    const L = (await import("leaflet")).default;
    const map = mapRef.current;

```



```

    if (!map || cancelled) return;
    drawRecords(L, map);
    if (area && points.length === 0 && path.length === 0) {
        map.setView([area.lat, area.lng], area.zoom ?? 10);
        return;
    }
    const target = center ?? points[points.length - 1] ?? path[path.length - 1];
    if (follow && target) map.setView([target.lat, target.lng], map.getZoom());
  })();
  return () => {
    cancelled = true;
  };
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [pointsKey, pathKey, centerKey, areaKey, accuracy]);
useEffect(() => {
  if (!showIranProvinces || !mapRef.current) return;
  let cancelled = false;
  (async () => {
    const L = (await import("leaflet")).default;
    if (!cancelled && mapRef.current) await drawProvinces(L, mapRef.current, selectedProvince);
  })();
  return () => {
    cancelled = true;
  };
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [showIranProvinces, selectedProvince]);
// eslint-disable-next-line @typescript-eslint/no-explicit-any
function drawRecords(L: any, map: LeafletMap) {
  for (const layer of layersRef.current) layer.remove();
  layersRef.current = [];
  if (path.length > 1) {
    const line = L.polyline(
      path.map((p) => [p.lat, p.lng]),
      { color: "#0d9488", weight: 4, opacity: 0.85 },
    ).addTo(map);
    layersRef.current.push(line);
  }
  if (accuracy && points[0]) {
    const circle = L.circle([points[0].lat, points[0].lng], {
      radius: accuracy,
      color: "#0ea5e9",
      weight: 1,
      fillColor: "#38bdf8",
      fillOpacity: 0.12,
    }).addTo(map);
    layersRef.current.push(circle);
  }
  for (const p of points) {
    const icon = L.divIcon({
      className: "sek-map-marker",
      html: `<div style="background:${p.color ?? "#0f766e"};width:18px;height:18px;border-radius:50%;border:3px solid #fff;box-shadow:0 2px 8px rgba(15,23,42,.45)"></div>`,
      iconSize: [18, 18],
      iconAnchor: [9, 9],
    });
    const marker = L.marker([p.lat, p.lng], { icon, draggable }).addTo(map);
    if (p.label) marker.bindTooltip(esc(p.label), { direction: "top", className: "sek-point-tooltip" });
    if (draggable) {
      marker.on("dragend", (e: { target: { getLatLng: () => { lat: number; lng: number } } }) => {
        const ll = e.target.getLatLng();
        pickRef.current?.({ lat: ll.lat, lng: ll.lng });
      });
    }
    layersRef.current.push(marker);
  }
  const all = [...points, ...path];
  if (all.length > 1) {
    map.fitBounds(
      all.map((p) => [p.lat, p.lng] as [number, number]),
      { padding: [28, 28], maxZoom: 17 },
    );
  } else if (all.length === 1) {
    map.setView([all[0].lat, all[0].lng], Math.max(map.getZoom(), 16));
  }
}
// eslint-disable-next-line @typescript-eslint/no-explicit-any
async function drawProvinces(L: any, map: LeafletMap, selected: string) {
  provinceLayerRef.current?.remove();
  try {
    const res = await fetch("/data/iran-provinces.geojson", { cache: "force-cache" });
    if (!res.ok) throw new Error( `GeoJSON ${res.status}` );
    const geo = await res.json();
    const layer = L.geoJSON(geo, {

```

```

// eslint-disable-next-line @typescript-eslint/no-explicit-any
style: (feature: any) => {
  const en = String(feature?.properties?.shapeName ?? "");
  const fa = PROVINCE_EN_FA[en] ?? en;
  const active = Boolean(selected && fa === selected);
  return {
    color: active ? "#0f766e" : "#64748b",
    weight: active ? 3 : 1.3,
    opacity: 0.95,
    fillColor: active ? "#14b8a6" : "#e2e8f0",
    fillOpacity: active ? 0.28 : 0.1,
  };
},
// eslint-disable-next-line @typescript-eslint/no-explicit-any
onEachFeature: (feature: any, polygon: any) => {
  const en = String(feature?.properties?.shapeName ?? "");
  const fa = PROVINCE_EN_FA[en] ?? en;
  polygon.bindTooltip(esc(fa), {
    permanent: true,
    direction: "center",
    className: `sek-province-label${selected === fa ? " is-active" : ""}`,
  });
  polygon.on("mouseover", () => polygon.setStyle({ fillOpacity: 0.32, weight: 2.5 }));
  polygon.on("mouseout", () => {
    const active = selected === fa;
    polygon.setStyle({ fillOpacity: active ? 0.28 : 0.1, weight: active ? 3 : 1.3 });
  });
  polygon.on("click", (e: { originalEvent?: { stopPropagation?: () => void } }) => {
    e.originalEvent?.stopPropagation?.();
    provinceCbRef.current?.(fa);
    try {
      map.fitBounds(polygon.getBounds(), { padding: [20, 20], maxZoom: 9 });
    } catch {
      /* ignore */
    }
  });
},
}).addTo(map);
provinceLayerRef.current = layer;
//
layer.bringToBack?.();
if (!selected && points.length === 0 && path.length === 0) {
  map.fitBounds(layer.getBounds(), { padding: [8, 8] });
}
} catch (err) {
  setMapError(`: ${err instanceof Error ? err.message : ""}`);
}
}
return (
  <div className="relative w-full overflow-hidden rounded-2xl bg-[#eef2f1] ring-1 ring-slate-200">
    <div ref={divRef} style={{ height }} className="w-full" role="application" aria-label="" />
    {tileState === "loading" ? (
      <div className="pointer-events-none absolute left-3 top-3 z-[500] rounded-lg bg-white/90 px-2 py-1 text-[10px] font-bold text-slate-500 shadow">
        ...
      </div>
    ) : null}
    {tileState === "fallback" ? (
      <div className="pointer-events-none absolute left-3 top-3 z-[500] rounded-lg bg-amber-50/95 px-2 py-1 text-[10px] font-bold text-amber-700 shadow ring-1 ring-amber-200">
      </div>
    ) : null}
    {mapError ? (
      <div className="absolute inset-x-3 bottom-3 z-[500] rounded-xl bg-rose-50/95 px-3 py-2 text-center text-[11px] font-bold text-rose-700 shadow ring-1 ring-rose-200">
        {mapError}
      </div>
    ) : null}
  </div>
);
}

```

```

@@ src/components/NavButton.tsx typescript
"use client";
import { useEffect, useState } from "react";

```

```

import { createPortal } from "react-dom";
type App = {
  key: string;
  label: string;
  icon: string;
  /**
   *
   */
  scheme?: (lat: number, lng: number, name: string) => string;
  /**
   *
   */
}

```

```

web: (lat: number, lng: number, name: string) => string;
platforms?: ("android" | "ios" | "desktop")[];
};
const APPS: App[] = [
  {
    key: "neshan",
    label: "",
    icon: "",
    scheme: (la, ln) => `neshan://maps?lat=${la}&lng=${ln}`,
    web: (la, ln) => `https://neshan.org/maps/routing/car/${la},${ln}#c${la}-${ln}-16z-0p`,
    platforms: ["android", "ios", "desktop"],
  },
  {
    key: "balad",
    label: "",
    icon: "",
    scheme: (la, ln) => `balad://maps?lat=${la}&lng=${ln}`,
    web: (la, ln) => `https://balad.ir/navigate?destination=${la},${ln}`,
    platforms: ["android", "ios", "desktop"],
  },
  {
    key: "google",
    label: " ",
    icon: "",
    scheme: (la, ln) => `google.navigation:q=${la},${ln}&mode=d`,
    web: (la, ln) => `https://www.google.com/maps/dir/?api=1&destination=${la},${ln}&travelmode=driving`,
    platforms: ["android", "ios", "desktop"],
  },
  {
    key: "waze",
    label: " (Waze)",
    icon: "",
    scheme: (la, ln) => `waze:///?ll=${la},${ln}&navigate=yes`,
    web: (la, ln) => `https://waze.com/ul?ll=${la},${ln}&navigate=yes`,
    platforms: ["android", "ios", "desktop"],
  },
  {
    key: "apple",
    label: " ",
    icon: "",
    scheme: (la, ln, n) => `maps:///daddr=${la},${ln}&q=${n}`,
    web: (la, ln, n) => `https://maps.apple.com/?daddr=${la},${ln}&q=${n}`,
    platforms: ["ios", "desktop"],
  },
  {
    key: "osm",
    label: "OpenStreetMap",
    icon: "",
    web: (la, ln) => `https://www.openstreetmap.org/directions?to=${la},${ln}`,
    platforms: ["android", "ios", "desktop"],
  },
  {
    key: "yandex",
    label: "",
    icon: "",
    scheme: (la, ln) => `yandexnavi://build_route_on_map?lat_to=${la}&lon_to=${ln}`,
    web: (la, ln) => `https://yandex.com/maps/?rtext=~${la},${ln}&rtt=auto`,
    platforms: ["android", "ios", "desktop"],
  },
  {
    key: "geo",
    label: " ",
    icon: "",
    scheme: (la, ln, n) => `geo:${la},${ln}?q=${la},${ln}(${n})`,
    web: (la, ln) => `https://www.google.com/maps?q=${la},${ln}`,
    platforms: ["android"],
  },
];
function platform(): "android" | "ios" | "desktop" {
  const ua = navigator.userAgent;
  if (/Android/i.test(ua)) return "android";
  if (/iPhone|iPad|iPod/i.test(ua)) return "ios";
  return "desktop";
}

/** — */
export default function NavButton({
  lat,
  lng,
  label = "",
  compact = true,
}: {
  lat?: number | null;
  lng?: number | null;
  label?: string;

```

```

compact?: boolean;
}) {
  const [open, setOpen] = useState(false);
  const [plat, setPlat] = useState<"android" | "ios" | "desktop">("desktop");
  const [canShare, setCanShare] = useState(false);
  const [pos, setPos] = useState<{ top: number; left: number } | null>(null);
  const [mobileMenu, setMobileMenu] = useState(false);
  const [mounted, setMounted] = useState(false);
  useEffect(() => {
    setMounted(true);
    setPlat(platform());
    setCanShare(typeof navigator !== "undefined" && !!navigator.share);
    const update = () => setMobileMenu(window.innerWidth < 640);
    update();
    window.addEventListener("resize", update);
    return () => window.removeEventListener("resize", update);
  }, []);
  if (!!lat || !!lng) return <span className="text-[10px] text-slate-300">—</span>;
  const name = encodeURIComponent(label || "");
  const list = APPS.filter((a) => !a.platforms || a.platforms.includes(plat));
  const go = (a: App) => {
    setOpen(false);
    const webUrl = a.web(lat, lng, name);
    //
    if (plat === "desktop" || !a.scheme) {
      const w = window.open(webUrl, "_blank", "noopener,noreferrer");
      if (!w) window.location.href = webUrl; //
      return;
    }
    //
    const started = Date.now();
    const fb = setTimeout(() => {
      if (!document.hidden && Date.now() - started < 2500) window.open(webUrl, "_blank", "noopener");
    }, 1100);
    window.addEventListener("pagehide", () => clearTimeout(fb), { once: true });
    window.location.href = a.scheme(lat, lng, name);
  };
  const shareLocation = async () => {
    setOpen(false);
    const text = `${label}\nhttps://www.google.com/maps?q=${lat},${lng}`;
    if (navigator.share) {
      try {
        await navigator.share({ title: label, text });
        return;
      } catch {
        /* */
      }
    }
    await navigator.clipboard?.writeText(text).catch(() => undefined);
  };
  const copyCoords = async () => {
    setOpen(false);
    await navigator.clipboard?.writeText(`${lat},${lng}`).catch(() => undefined);
  };
  const menu = open ? (
    <div
      className="fixed inset-0 z-[9998] bg-slate-900/45 backdrop-blur-[1px]"
      onClick={(e) => {
        e.preventDefault();
        e.stopPropagation();
        setOpen(false);
      }}
    >
    <div
      className={
        mobileMenu
          ? "fade-in fixed inset-x-3 bottom-3 max-h-[78vh] overflow-y-auto rounded-3xl bg-white p-3 shadow-2xl ring-1
ring-white/80"
          : "fade-in fixed w-60 max-h-[75vh] overflow-y-auto rounded-2xl bg-white p-2 shadow-2xl ring-1 ring-slate-200"
      }
    >
    <div style={mobileMenu ? undefined : { top: pos?.top ?? 80, left: pos?.left ?? 16 }}
      onClick={(e) => {
        e.preventDefault();
        e.stopPropagation();
      }}
    >
    <div className="mb-2 flex items-center justify-between gap-2 px-1">
      <div>
        <div className="text-xs font-black text-slate-800"> </div>
        <div className="mt-0.5 max-w-[190px] truncate text-[10px] text-slate-400">{label}</div>
      </div>
      <button
        type="button"

```

```

        onClick={() => setOpen(false)}
        className="flex size-8 items-center justify-center rounded-full bg-slate-100 text-sm font-bold text-slate-60
0"
        aria-label=""
    >

</button>
</div>
<div className={mobileMenu ? "grid grid-cols-2 gap-2" : "space-y-1"}>
    {list.map((a) => (
        <button
            key={a.key}
            type="button"
            onClick={() => go(a)}
            className={`flex items-center gap-2 rounded-xl bg-slate-50 px-3 text-right font-bold text-slate-700 ring-1
ring-slate-200 transition hover:bg-teal-50 hover:text-teal-800 ${
                mobileMenu ? "min-h-12 py-3 text-xs" : "w-full py-2 text-[11px]"
            }`}
        >
            <span className="text-lg">{a.icon}</span>
            <span>{a.label}</span>
        </button>
    ))}
</div>
<div className="my-2 h-px bg-slate-100" />
<div className={mobileMenu ? "grid grid-cols-2 gap-2" : "space-y-1"}>
    {canShare ? (
        <button
            type="button"
            onClick={shareLocation}
            className="rounded-xl bg-sky-50 px-3 py-2.5 text-right text-[11px] font-bold text-sky-700 ring-1 ring-sky-
200"
        >

        </button>
    ) : null}
    <button
        type="button"
        onClick={copyCoords}
        className="rounded-xl bg-slate-100 px-3 py-2.5 text-right text-[11px] font-bold text-slate-700 ring-1 ring-s
late-200"
    >

        </button>
    </div>
</div>
</div>
) : null;
return (
    <>
        <span className="relative inline-block">
            <button
                type="button"
                onClick={(e) => {
                    e.preventDefault();
                    e.stopPropagation();
                    const r = (e.currentTarget as HTMLElement).getBoundingClientRect();
                    const menuWidth = 240;
                    const left = Math.min(Math.max(8, r.left), Math.max(8, window.innerWidth - menuWidth - 8));
                    const estimatedHeight = 390;
                    const top =
                        r.bottom + estimatedHeight < window.innerHeight
                            ? r.bottom + 6
                            : Math.max(8, r.top - estimatedHeight - 6);
                    setPos({ top, left });

                    setOpen(true);
                }}
                title=" "
                className={`rounded-xl bg-gradient-to-l from-teal-600 to-emerald-600 font-bold text-white shadow-sm transition
hover:from-teal-700 hover:to-emerald-700 active:scale-95 ${
                    compact ? "px-2.5 py-1.5 text-[11px]" : "px-4 py-2.5 text-xs"
                }`}
            >

            </button>
        </span>
        {mounted && menu ? createPortal(menu, document.body) : null}
    </>
);
}

```

```

@@ src/components/NotificationBell.tsx typescript
"use client";
import Link from "next/link";

```

```

import { useCallback, useEffect, useRef, useState } from "react";
import { tehranDateTime, toPersianDigits } from "@lib/jalali";
import { playNotificationSound, vibrate } from "@lib/sound";
type N = {
  id: number;
  title: string;
  body: string;
  fromName: string;
  kind: string;
  link: string;
  readAt: string | null;
  createdAt: string;
};
export default function NotificationBell({ basePath }: { basePath: string }) {
  const [rows, setRows] = useState<N[]>([]);
  const [unread, setUnread] = useState(0);
  const [open, setOpen] = useState(false);
  const [toast, setToast] = useState<N | null>(null);
  const [muted, setMuted] = useState(false);
  const lastId = useRef<number>(0);
  const initialized = useRef(false);
  const load = useCallback(async () => {
    //
    const res = await fetch("/api/notifications?unread=1", { cache: "no-store" }).catch(() => null);
    if (!res?.ok) return;
    const d = await res.json();
    const list: N[] = d.rows ?? [];
    setRows(list);
    setUnread(list.length);
    const top = list[0];
    if (top && initialized.current && top.id > lastId.current && !top.readAt) {
      setToast(top);
      setTimeout(() => setToast(null), 10000);
      if (localStorage.getItem("sek_mute") !== "1") {
        playNotificationSound();
        vibrate();
      }
      showSystemNotification(top);
    }
    if (top) lastId.current = Math.max(lastId.current, top.id);
    initialized.current = true;
  }, []);
  useEffect(() => {
    setMuted(localStorage.getItem("sek_mute") === "1");
    load();
    const t = setInterval(load, 20000);
    // ( )
    const ask = () => {
      if (typeof Notification !== "undefined" && Notification.permission === "default") {
        Notification.requestPermission().catch(() => undefined);
      }
      window.removeEventListener("click", ask);
      window.removeEventListener("touchstart", ask);
    };
    window.addEventListener("click", ask, { once: true });
    window.addEventListener("touchstart", ask, { once: true });
    //
    const onMsg = (e: MessageEvent) => {
      if (e.data?.type === "notification-click") {
        if (e.data.id) markOne(Number(e.data.id));
        load();
      }
    };
    navigator.serviceWorker?.addEventListener("message", onMsg);
    return () => {
      clearInterval(t);
      window.removeEventListener("click", ask);
      window.removeEventListener("touchstart", ask);
      navigator.serviceWorker?.removeEventListener("message", onMsg);
    };
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [load]);
  /** — */
  const showSystemNotification = async (n: N) => {
    if (typeof Notification === "undefined" || Notification.permission !== "granted") return;
    const payload = {
      body: n.body || n.title,
      icon: "/icons/icon-192.png",
      badge: "/icons/icon-192.png",
      tag: `sek-${n.id}`,
      dir: "rtl" as const,
      lang: "fa",
      requireInteraction: true,
      data: { id: n.id, link: n.link || `${basePath}/notifications` },
    };
  };

```

```

    vibrate: [200, 100, 200],
  };
  try {
    //
    const reg = await navigator.serviceWorker?.getRegistration();
    if (reg) {
      await reg.showNotification(`${n.title}`, payload as NotificationOptions);
      return;
    }
    const note = new Notification(`${n.title}`, payload as NotificationOptions);
    note.onclick = () => {
      window.focus();
      markOne(n.id);
      window.location.href = payload.data.link;
      note.close();
    };
  } catch {
    /* ignore */
  }
};
const markOne = async (id: number) => {
  await fetch("/api/notifications", {
    method: "PATCH",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ ids: [id] }),
  });
  load();
};
const toggleMute = () => {
  const next = !muted;
  setMuted(next);
  localStorage.setItem("sek_mute", next ? "1" : "0");
  if (!next) playNotificationSound();
};
const askPermission = async () => {
  if (typeof Notification !== "undefined" && Notification.permission === "default") {
    await Notification.requestPermission().catch(() => undefined);
  }
};
const markAll = async () => {
  await fetch("/api/notifications", {
    method: "PATCH",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ all: true }),
  });
  load();
};
return (
  <>
    <div className="relative">
      <button
        onClick={() => {
          setOpen((v) => !v);

          askPermission();
        }}
        className="relative rounded-lg bg-white/15 px-2 py-1.5 text-sm hover:bg-white/25"
        title=""
      >

        {unread > 0 ? (
          <span className="absolute -top-1 -left-1 rounded-full bg-rose-500 px-1.5 text-[10px] font-black text-white">
            {toPersianDigits(unread > 99 ? "99+" : unread)}
          </span>
        ) : null}
      </button>
      {open ? (
        <>
          <div className="fixed inset-0 z-30" onClick={() => setOpen(false)} />
          <div className="fade-in absolute left-0 z-40 mt-2 max-h-80 w-72 overflow-y-auto rounded-2xl bg-white p-2 text-slate-800 shadow-2xl ring-1 ring-slate-200">
            <div className="mb-1 flex items-center justify-between px-2">
              <span className="text-xs font-bold text-slate-600"></span>
              <div className="flex items-center gap-2">
                <button onClick={toggleMute} className="text-[11px] font-bold text-slate-500" title="">
                  {muted ? " " : " "}
                </button>
                <button onClick={markAll} className="text-[11px] font-bold text-teal-700">
                  </button>
                </div>
              </div>
              <div>
                {rows.length === 0 ? (
                  <p className="px-2 py-6 text-center text-xs text-slate-400">

```

```

    </p>
  ) : (
    rows.slice(0, 12).map((n) => (
      <div
        key={n.id}
        onClick={() => !n.readAt && markOne(n.id)}
        className={`mb-1 cursor-pointer rounded-xl px-2.5 py-2 text-xs ${n.readAt ? "bg-slate-50" : "bg-teal
-50 ring-1 ring-teal-200"}`}
      >
        <div className="font-bold text-slate-800">
          {n.title}
          {!n.readAt ? <span className="mr-1 text-[9px] text-teal-600">( )</span> : null
        }
        </div>
        {n.body ? <div className="mt-0.5 text-[11px] text-slate-600">{n.body}</div> : null}
        <div className="mt-1 text-[10px] text-slate-400">
          {n.fromName ? `${n.fromName} — ` : ""}
          {tehranDateTime(n.createdAt)}
        </div>
      </div>
    ))
  )
  <Link
    href={`/${basePath}/notifications`}
    onClick={() => setOpen(false)}
    className="block rounded-xl bg-slate-100 py-2 text-center text-xs font-bold text-slate-700"
  >

  </Link>
</div>
</>
) : null}
</div>
{toast ? (
  <div className="fade-in fixed bottom-4 left-4 right-4 z-50 mx-auto max-w-sm rounded-2xl bg-slate-900/95 p-3 text
-white shadow-2xl sm:right-auto">
    <div className="text-xs font-black text-amber-300"> </div>
    <div className="mt-1 text-sm font-bold">{toast.title}</div>
    {toast.body ? <div className="text-xs text-slate-200">{toast.body}</div> : null}
    <div className="mt-2 flex gap-3">
      <button
        onClick={() => {
          markOne(toast.id);
          setToast(null);
        }}
        className="text-[11px] font-bold text-emerald-300"
      >

      </button>
      <button onClick={() => setToast(null)} className="text-[11px] font-bold text-teal-300">

      </button>

    </div>
  </div>
) : null}
</>
);
}

```

```

@@ src/components/RecordScreen.tsx typescript
"use client";
import { useCallback, useEffect, useMemo, useState } from "react";
import dynamicImport from "next/dynamic";
import JalaliDateInput from "../JalaliDateInput";
import Combobox from "../Combobox";
import LocationPicker, { type LatLng } from "../LocationPicker";
import FileUploader, { FileList, type Att } from "../FileUploader";
import ShareBox from "../ShareBox";
import NavButton from "../NavButton";
import { Alert, Badge, Button, Card, Field, Input, SectionTitle, TextArea } from "../ui";
import {
  DEFAULT_PRODUCTS,
  DEFAULT_COLUMNS,
  DEFAULT_FORM_FIELDS,
  bonusKeyOf,
  type ProductConfig,
  type ColumnConfig,
  type FormFieldConfig,
} from "@lib/defaults";
import { isValidJalali, tehranDateTime, toPersianDigits, todayJalali } from "@lib/jalali";
import { useConfirm } from "@components/Confirm";
import { useLive } from "@lib/useLive";
import { downloadFile } from "@lib/download";
const MapBox = dynamicImport(() => import("../MapBox"), { ssr: false });

```



```

export type RecordType = "pharmacies" | "doctors" | "orders";
export type Row = {
  id: number;
  repName: string;
  dateShamsi: string;
  province?: string;
  city?: string;
  region?: string;
  name?: string;
  landline?: string;
  pharmacyName?: string;
  managerName?: string;
  managerPhone?: string;
  specialty?: string;
  phone?: string;
  secretaryName?: string;
  secretaryPhone?: string;
  otherAddresses?: string;
  address?: string;
  isPercent?: boolean;
  percentValue?: string;
  lat?: number | null;
  lng?: number | null;
  accuracy?: number | null;
  locationLabel?: string;
  items?: Record<string, number>;
  distributor?: string;
  visitor?: string;
  notes?: string;
  sent?: boolean;
  sendStatus?: string;
  createdAt?: string;
};
type Form = Record<string, string> & { items?: never };
type DupRow = {
  id: number;
  name: string;
  specialty?: string;
  phone?: string;
  city?: string;
  province?: string;
  region?: string;
  landline?: string;
  managerName?: string;
  managerPhone?: string;
  address?: string;
  lat?: number | null;

  lng?: number | null;
  accuracy?: number | null;
  isPercent?: boolean;
  percentValue?: string;
  repName: string;
  dateShamsi: string;
  mine: boolean;
};
type OrderHistory = {
  orderCount: number;
  lastDate: string;
  lastRep: string;
  totalUnits: number;
  totalBonus: number;
  items: { key: string; label: string; qty: number; bonus: number }[];
  recent: { id: number; dateShamsi: string; repName: string; units: number }[];
};
type TargetRow = {
  productKey: string;
  productLabel: string;
  quantity: number;
  sold: number;
  remaining: number;
  percent: number;
};
const emptyForm = (): Record<string, string> => ({
  dateShamsi: todayJalali(),
  province: "",
  city: "",
  region: "",
  name: "",
  landline: "",
  pharmacyName: "",
  managerName: "",
  managerPhone: "",
  address: "",
  specialty: "",

```

```

phone: "",
secretaryName: "",
secretaryPhone: "",
otherAddresses: "",
distributor: "",
visitor: "",
notes: "",
});
const TITLES: Record<RecordType, { title: string; icon: string; nameLabel: string }> = {
  pharmacies: { title: " ", icon: "", nameLabel: " " },
  doctors: { title: " ", icon: "", nameLabel: " " },
  orders: { title: " ", icon: "", nameLabel: " " },
};
const PAGE = 25;
export default function RecordScreen({ type, isAdmin = false }: { type: RecordType; isAdmin?: boolean }) {
  const meta = TITLES[type];
  const [tab, setTab] = useState<"form" | "list">(isAdmin ? "list" : "form");
  const [form, setForm] = useState<Record<string, string>>(emptyForm);
  const [items, setItems] = useState<Record<string, number>>({});
  const [loc, setLoc] = useState<LatLng>({ lat: null, lng: null, accuracy: null });
  const [rows, setRows] = useState<Row[]>([]);
  const [opts, setOpts] = useState<Record<string, string[]>>({});
  const [optTree, setOptTree] = useState<Record<string, Record<string, string[]>>>({});
  const [msg, setMsg] = useState<{ kind: "error" | "success" | "info"; text: string } | null>(null);
  const [busy, setBusy] = useState(false);
  const [selected, setSelected] = useState<number[]>([]);
  const [detail, setDetail] = useState<Row | null>(null);
  const [search, setSearch] = useState("");
  const [period, setPeriod] = useState("");
  const [page, setPage] = useState(1);
  const [products, setProducts] = useState<ProductConfig[]>(DEFAULT_PRODUCTS);
  const [isPercent, setIsPercent] = useState(false);
  const [percentValue, setPercentValue] = useState("");
  const [fileIds, setFileIds] = useState<number[]>([]);
  const [detailFiles, setDetailFiles] = useState<Att[]>([]);
  const [dupes, setDupes] = useState<DupRow[]>([]);
  const [history, setHistory] = useState<OrderHistory | null>(null);
  const [suggestion, setSuggestion] = useState<Record<string, unknown> | null>(null);
  const [autofilled, setAutofilled] = useState(false);
  const [targets, setTargets] = useState<TargetRow[]>([]);
  const [editRow, setEditRow] = useState<Row | null>(null);
  const [editDraft, setEditDraft] = useState<Record<string, string>>({});

  const [editItems, setEditItems] = useState<Record<string, number>>({});
  const [columns, setColumns] = useState<ColumnConfig[]>(DEFAULT_COLUMNS[type]);
  const [formFields, setFormFields] = useState<FormFieldConfig[]>(DEFAULT_FORM_FIELDS[type]);
  const confirm = useConfirm();
  const set = (k: string, v: string) => setForm((f) => ({ ...f, [k]: v }));
  const loadRows = useCallback(async () => {
    const res = await fetch(`/api/records/${type}`, { cache: "no-store" });
    if (res.ok) setRows((await res.json()).rows ?? []);
  }, [type]);
  const loadOptions = useCallback(async () => {
    const res = await fetch("/api/options", { cache: "no-store" });
    if (!res.ok) return;
    const data = await res.json();
    const rows = (data.rows ?? []) as { category: string; value: string; parent?: string }[];
    const grouped: Record<string, string[]> = {};
    const tree: Record<string, Record<string, string[]>> = {};
    for (const o of rows) {
      (grouped[o.category] ??= []).push(o.value);
      const par = o.parent ?? "";
      ((tree[o.category] ??= {})[par] ??= []).push(o.value);
    }
    setOpts(grouped);
    setOptTree(tree);
  }, []);
  const loadSettings = useCallback(async () => {
    const res = await fetch("/api/settings", { cache: "no-store" });
    if (!res.ok) return;
    const d = await res.json();
    const v = d.values ?? {};
    setProducts((v.products as ProductConfig[])?.filter((p) => p.enabled) ?? DEFAULT_PRODUCTS);
  }, []);
  /**
   *
   *
   * ( «» )
   *
   */
  const stored = v[`columns.${type}`] as ColumnConfig[] | undefined;
  const defaults = DEFAULT_COLUMNS[type];
  if (Array.isArray(stored) && stored.length) {
    const have = new Set(stored.map((c) => c.key));
    //
    //
  }

```

```

    const mandatory = defaults.filter((c) => c.key === "actions" && !have.has(c.key));
    setColumns([...stored, ...mandatory]);
  } else {
    setColumns(defaults);
  }
  const storedFields = v[`fields.${type}`] as FormFieldConfig[] | undefined;
  if (Array.isArray(storedFields) && storedFields.length) {
    const have = new Set(storedFields.map((f) => f.key));
    //
    const mandatory = DEFAULT_FORM_FIELDS[type].filter((f) => f.key === "location" && !have.has(f.key));
    setFormFields([...storedFields, ...mandatory]);
  } else {
    setFormFields(DEFAULT_FORM_FIELDS[type]);
  }
}, [type]);
useEffect(() => {
  loadOptions();
  loadSettings();
}, [loadOptions, loadSettings]);
//
useLive(loadRows, 15000, tab === "list");
const recordName = type === "orders" ? form.pharmacyName : form.name;
const showField = (key: string) => formFields.some((f) => f.key === key && f.visible);
const submit = async () => {
  if (!isValidJalali(form.dateShamsi)) {
    setMsg({ kind: "error", text: "    (: //)" });
    return;
  }
  if (!recordName.trim()) {
    setMsg({ kind: "error", text: `${meta.nameLabel} ` });
    return;
  }
  //

  if (type !== "orders" && dupes.length > 0) {
    const ok = await confirm({
      title: " ",
      message:
        `«${dupes[0].name}» ${dupes[0].mine ? "":`${dupes[0].repName}`} ` +
        `${toPersianDigits(dupes[0].dateShamsi)} .\n ` ,
      confirmText: " ",
      cancelText: "",
      danger: true,
    });
    if (!ok) return;
  }
  if (
    !(await confirm({
      title: " ",
      message: `«${recordName.trim()}» ${toPersianDigits(form.dateShamsi)} ` ,
      confirmText: " ",
    })))
  )
    return;
  setBusy(true);
  setMsg(null);
  const res = await fetch(`/api/records/${type}`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      ...form,
      items,
      isPercent,
      percentValue,
      fileIds,
      lat: loc.lat,
      lng: loc.lng,
      accuracy: loc.accuracy,
      locationLabel: recordName.trim(),
    }),
  });
  setBusy(false);
  if (res.ok) {
    setForm(emptyForm());
    setItems({});
    setIsPercent(false);
    setPercentValue("");
    setFileIds([]);
    setLoc({ lat: null, lng: null, accuracy: null });
    setMsg({ kind: "success", text: " . «.» });
    setDupes([]);
    setHistory(null);
    loadRows();
    loadTargets();
    setTab("list");
  }

```

```

    } else {
      const d = await res.json().catch(() => ({}));
      setMsg({ kind: "error", text: d.error ?? " " });
    }
  };
const sendSelected = async () => {
  const ids = selected.length ? selected : rows.filter((r) => !r.sent).map((r) => r.id);
  if (!ids.length) {
    setMsg({ kind: "info", text: " " });
    return;
  }
  if (
    !(await confirm({
      title: " ",
      message: `${toPersianDigits(ids.length)}  ${type === "orders" ? " " : ""}`,
      confirmText: "",
    }))
  ) {
    return;
  }
  setBusy(true);
  const res = await fetch(`/api/records/${type}/send`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ ids }),
  });
  setBusy(false);
  const d = await res.json().catch(() => ({}));
  if (res.ok) {
    setSelected([]);
    setMsg({
      kind: "success",
      text:
        type === "orders"
        ? `: ${d.status ?? "—"}`
        : ` ${toPersianDigits(ids.length)} `,
    });
    loadRows();
  } else setMsg({ kind: "error", text: d.error ?? " " });
};
const periods = useMemo(() => {
  const s = new Set(rows.map((r) => r.dateShamsi.slice(0, 7)).filter(Boolean));
  return [...s].sort().reverse();
}, [rows]);
const filtered = useMemo(() => {
  const q = search.trim().toLowerCase();
  return rows.filter((r) => {
    if (period && !r.dateShamsi.startsWith(period)) return false;
    if (!q) return true;
    const hay = `${r.repName} ${r.name ?? ""} ${r.pharmacyName ?? ""} ${r.managerName ?? ""} ${r.specialty ?? ""} ${r.city ?? ""} ${r.province ?? ""} ${r.managerPhone ?? ""} ${r.phone ?? ""}`;
    return hay.toLowerCase().includes(q);
  });
}, [rows, search, period]);
const pageRows = filtered.slice(0, page * PAGE);
useEffect(() => {
  if (detail && type !== "orders") {
    const ot = type === "doctors" ? "doctor" : "pharmacy";
    fetch(`/api/attachments?ownerType=${ot}&ownerId=${detail.id}`, { cache: "no-store" })
      .then((r) => (r.ok ? r.json() : null))
      .then((d) => setDetailFiles(d?.rows ?? []))
      .catch(() => setDetailFiles([]));
  } else setDetailFiles([]);
}, [detail, type]);
const orderText = (r: Row) => {
  const lines = [
    " ",
    `: ${toPersianDigits(r.dateShamsi)} `,
    `: ${r.repName} `,
    `: ${r.pharmacyName ?? ""} `,
    `: ${r.managerName ?? ""} `,
    `: ${toPersianDigits(r.managerPhone ?? "")} `,
    `: ${r.address ?? ""} `,
  ];
  // ( )
  lines.push("— —");
  for (const p of products) {
    const q = Number(r.items?.[p.key] ?? 0);
    const b = Number(r.items?.[bonusKeyOf(p.key)] ?? 0);
    if (q || b) lines.push(`${p.label}: ${toPersianDigits(q)} | : ${toPersianDigits(b)} `);
  }
  lines.push(`: ${r.distributor ?? ""} `);
  lines.push(`: ${r.visitor ?? ""} `);
  if (r.notes) lines.push(`: ${r.notes} `);

```

```

    return lines.join("\n");
};
/* ----- */
const loadTargets = useCallback(async () => {
  if (type !== "orders") return;
  const res = await fetch("/api/targets", { cache: "no-store" });
  if (res.ok) {
    const d = await res.json();
    setTargets(d.progress ?? []);
  }
}, [type]);
useEffect(() => {
  loadTargets();
}, [loadTargets]);
/* ----- + ----- */
const lookupName = type === "orders" ? form.pharmacyName : form.name;
const lookupPhone = type === "doctors" ? form.phone : form.managerPhone;
useEffect(() => {
  if (tab !== "form") return;
  const n = (lookupName ?? "").trim();
  if (n.length < 2) {
    setDups([]);
    setHistory(null);
    setSuggestion(null);
    return;
  }
}
const t = setTimeout(async () => {
  const q = new URLSearchParams({ type, name: n, phone: lookupPhone ?? "" });
  const res = await fetch(`/api/records/lookup?${q}`, { cache: "no-store" }).catch(() => null);
  if (!res?.ok) return;
  const d = await res.json();
  setDups(d.duplicates ?? []);
  setHistory(d.history ?? null);
  setSuggestion(d.suggestion ?? null);
}, 600);
return () => clearTimeout(t);
}, [lookupName, lookupPhone, type, tab]);
/** */
const applySuggestion = (src?: Record<string, unknown> | DupRow) => {
  const s = (src ?? suggestion) as Record<string, unknown> | null;
  if (!s) return;
  setForm((f) => ({
    ...f,
    pharmacyName: String(s.pharmacyName ?? s.name ?? f.pharmacyName),
    name: type === "orders" ? f.name : String(s.name ?? f.name),
    managerName: String(s.managerName ?? f.managerName),
    managerPhone: String(s.managerPhone ?? f.managerPhone),
    address: String(s.address ?? f.address),
    province: String(s.province ?? f.province),
    city: String(s.city ?? f.city),
    region: String(s.region ?? f.region),
    landline: String(s.landline ?? f.landline),
    distributor: String(s.distributor ?? f.distributor),
    visitor: String(s.visitor ?? f.visitor),
  }));
  const la = Number(s.lat);
  const ln = Number(s.lng);
  if (Number.isFinite(la) && Number.isFinite(ln) && la && ln) {
    setLoc({ lat: la, lng: ln, accuracy: Number(s.accuracy) || null });
  }
  setAutofilled(true);
  setTimeout(() => setAutofilled(false), 4000);
};
/* ----- */
const openEdit = (r: Row) => {
  setEditRow(r);
  setEditDraft({
    dateShamsi: r.dateShamsi ?? "",
    name: r.name ?? "",
    pharmacyName: r.pharmacyName ?? "",
    province: r.province ?? "",
    city: r.city ?? "",
    region: r.region ?? "",
    landline: r.landline ?? "",
    managerName: r.managerName ?? "",
    managerPhone: r.managerPhone ?? "",
    specialty: r.specialty ?? "",
    phone: r.phone ?? "",
    secretaryName: r.secretaryName ?? "",
    secretaryPhone: r.secretaryPhone ?? "",
    address: r.address ?? "",
    otherAddresses: r.otherAddresses ?? "",
    distributor: r.distributor ?? "",
    visitor: r.visitor ?? "",
  });
};

```

```

        notes: r.notes ?? "",
        percentValue: r.percentValue ?? "",
    });
    setEditItems({ ...(r.items ?? {}) });
};
const saveEdit = async () => {
    if (!editRow) return;
    if (
        !(await confirm({
            title: " ",
            message: `«${editDraft.pharmacyName || editDraft.name}»`,
            confirmText: "",
        }))
    )
        return;
    setBusy(true);
    const res = await fetch(`/api/records/${type}`, {
        method: "PATCH",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ id: editRow.id, ...editDraft, ...(type === "orders" ? { items: editItems } : {}) }),
    });
    setBusy(false);
    const d = await res.json().catch(() => ({}));

    if (res.ok) {
        setMsg({ kind: "success", text: " " });
        setEditRow(null);
        loadRows();
        loadTargets();
    } else setMsg({ kind: "error", text: d.error ?? " " });
};
const removeRow = async (r: Row) => {
    if (
        !(await confirm({
            title: " ",
            message: `«${nameOf(r)}»`,
            confirmText: "",
            danger: true,
        }))
    )
        return;
    const res = await fetch(`/api/records/${type}?id=${r.id}`, { method: "DELETE" });
    const d = await res.json().catch(() => ({}));
    if (res.ok) {
        setMsg({ kind: "success", text: " " });
        loadRows();
        loadTargets();
    } else setMsg({ kind: "error", text: d.error ?? " " });
};
const onAdded = () => loadOptions();
const visibleCols = columns.filter((c) => c.visible);
const nameOf = (r: Row) => (type === "orders" ? r.pharmacyName : r.name) ?? "";
const renderCell = (key: string, r: Row, i: number) => {
    switch (key) {
        case "row":
            return (
                <div className="flex items-center gap-2">
                    {!isAdmin ? (
                        <input
                            type="checkbox"
                            className="size-4 accent-teal-600"
                            checked={selected.includes(r.id)}
                            onChange={(e) => setSelected((s) => (e.target.checked ? [...s, r.id] : s.filter((x) => x !== r.id)))}
                        />
                    ) : null}
                    {toPersianDigits(i + 1)}
                </div>
            );
        case "repName":
            return <span className="font-semibold text-slate-700">{r.repName}</span>;
        case "dateShamsi":
            return toPersianDigits(r.dateShamsi);
        case "name":
        case "pharmacyName":
            return (
                <button
                    onClick={() => setDetail(r)}
                    className="font-bold text-teal-700 underline decoration-dotted underline-offset-4"
                >
                    {nameOf(r)}
                </button>
            );
        case "location":
            return r.lat && r.lng ? (
                <Badge tone="green">{r.accuracy ? `${toPersianDigits(Math.round(r.accuracy))}` : ""}</Badge>

```

```

    ) : (
      <Badge tone="amber"></Badge>
    );
  case "sent":
    return r.sent ? <Badge tone="green"> </Badge> : <Badge tone="amber"> </Badge>;
  case "nav":
    return <NavButton lat={r.lat} lng={r.lng} label={nameOf(r)} />;
  case "actions":
    return (
      <div className="flex gap-1">
        <button
          onClick={() => openEdit(r)}
          className="rounded-lg bg-sky-100 px-2 py-1 text-[10px] font-bold text-sky-700"
          title=""
        >

        </button>
        <button
          onClick={() => removeRow(r)}

          className="rounded-lg bg-rose-100 px-2 py-1 text-[10px] font-bold text-rose-700"
          title=""
        >

        </button>
      </div>
    );
  case "totalUnits":
    return toPersianDigits(products.reduce((a, p) => a + Number(r.items?.[p.key] ?? 0), 0));
  case "totalBonus":
    return toPersianDigits(products.reduce((a, p) => a + Number(r.items?.[bonusKeyOf(p.key)] ?? 0), 0));
  case "files":
    return (
      <button onClick={() => setDetail(r)} className="text-[11px] font-bold text-sky-700 underline">

      </button>
    );
  case "createdAt":
    return <span className="text-[10px] text-slate-500">{r.createdAt ? tehranDateTime(r.createdAt) : "—"}</span>;
  case "percentValue":
    return r.percentValue || "—";
  case "isPercent":
    return r.isPercent ? (
      <Badge tone="green">{r.percentValue ? ` (${r.percentValue})` : ""}</Badge>
    ) : (
      <Badge tone="slate"></Badge>
    );
  case "products":
    return (
      <span className="text-[11px] text-slate-600">
        {products
          .filter((p) => (r.items?.[p.key] ?? 0) || (r.items?.[bonusKeyOf(p.key)] ?? 0))
          .map(
            (p) =>
              `${p.label}: ${toPersianDigits(r.items?.[p.key] ?? 0)}+${toPersianDigits(r.items?.[bonusKeyOf(p.key)]
?? 0)}`,
          )
          .join(" | ") || "—"}
        </span>
      );
  case "managerPhone":
  case "phone":
  case "secretaryPhone":
  case "landline":
    return toPersianDigits((r as unknown as Record<string, string>)[key] ?? "") || "—";
  default: {
    const v = (r as unknown as Record<string, unknown>)[key];
    return v ? String(v) : "—";
  }
}
};
return (
  <div className="space-y-4">
    <div className="flex flex-wrap items-center justify-between gap-3">
      <SectionTitle icon={meta.icon}>{meta.title}</SectionTitle>
      <div className="flex rounded-xl bg-slate-200 p-1 text-xs font-bold">
        <button
          onClick={() => setTab("form")}
          className={`rounded-lg px-3 py-1.5 ${tab === "form" ? "bg-white text-teal-700 shadow" : "text-slate-600"}`
        >

        </button>
        <button
          onClick={() => setTab("list")}
          className={`rounded-lg px-3 py-1.5 ${tab === "list" ? "bg-white text-teal-700 shadow" : "text-slate-600"}`
        >

```

```

    >
      ({toPersianDigits(rows.length)})
    </button>
  </div>
</div>
{msg ? <Alert kind={msg.kind}>{msg.text}</Alert> : null}
{tab === "form" ? (
  <Card>
    <div className="grid grid-cols-1 gap-3 sm:grid-cols-2 lg:grid-cols-3">
      {showField("dateShamsi") ? (
        <Field label={type === "orders" ? " " : ""} required hint="
">
          <JalaliDateInput value={form.dateShamsi} onChange={(v) => set("dateShamsi", v)} />
        </Field>
      ) : null}
      {type !== "orders" ? (
        <>
          {showField("province") ? (
            <Field label="" required hint="" — ">
              <Combobox
                value={form.province}
                onChange={(v) => {
                  set("province", v);
                  if (v !== form.province) {
                    set("city", "");
                    set("region", "");
                  }
                }}
                options={opts.province ?? []}
                selectOnly
                placeholder=" ..."
              />
            </Field>
          ) : null}
          {showField("city") ? (
            <Field label="" hint="" — ">
              <Combobox
                value={form.city}
                onChange={(v) => {
                  set("city", v);
                  if (v !== form.city) set("region", "");
                }}
                options={optTree.city?.[form.province] ?? []}
                selectOnly
                parent={form.province}
                parentLabel=""
                requireParent
                placeholder=" ..."
              />
            </Field>
          ) : null}
          {showField("region") ? (
            <Field label="" hint="" — ">
              <Combobox
                value={form.region}
                onChange={(v) => set("region", v)}
                options={optTree.region?.[form.city] ?? []}
                selectOnly
                parent={form.city}
                parentLabel=""
                requireParent
                placeholder=" ..."
              />
            </Field>
          ) : null}
        </>
      ) : null}
      {type === "doctors" ? (
        <>
          {showField("name") ? (
            <Field label="" required hint="" + ">
              <Combobox
                value={form.name}
                onChange={(v) => set("name", v)}
                options={opts.doctor ?? []}
                category="doctor"
                onAdded={onAdded}
                placeholder=" ..."
              />
            </Field>
          ) : null}
          {showField("specialty") ? (
            <Field label="" hint="" + " + ">
              <Combobox

```



```

        value={form.specialty}
        onChange={(v) => set("specialty", v)}
        options={opts.specialty ?? []}
        category="specialty"
        onAdded={onAdded}
      />
    </Field>
  ) : null}
  {showField("phone") ? (
    <Field label=" ">
      <Input inputMode="numeric" value={form.phone} onChange={(e) => set("phone", e.target.value)} />
    </Field>
  ) : null}
  {showField("secretaryName") ? (
    <Field label=" ">
      <Combobox
        value={form.secretaryName}
        onChange={(v) => set("secretaryName", v)}
        options={opts.secretary ?? []}
        category="secretary"
        onAdded={onAdded}
      />
    </Field>
  ) : null}
  {showField("secretaryPhone") ? (
    <Field label=" ">
      <Input
        inputMode="numeric"
        value={form.secretaryPhone}
        onChange={(e) => set("secretaryPhone", e.target.value)}
      />
    </Field>
  ) : null}
  {showField("address") ? (
    <div className="sm:col-span-2 lg:col-span-3">
      <Field label=" ">
        <TextArea value={form.address} onChange={(e) => set("address", e.target.value)} />
      </Field>
    </div>
  ) : null}
  {showField("otherAddresses") ? (
    <div className="sm:col-span-2 lg:col-span-3">
      <Field label=" ">
        <TextArea
          value={form.otherAddresses}
          onChange={(e) => set("otherAddresses", e.target.value)}
          placeholder=" "
        />
      </Field>
    </div>
  ) : null}
</>
) : (
  <>
    {showField(type === "orders" ? "pharmacyName" : "name") ? (
      <Field label=" " required hint=" + ">
        <Combobox
          value={type === "orders" ? form.pharmacyName : form.name}
          onChange={(v) => set(type === "orders" ? "pharmacyName" : "name", v)}
          options={opts.pharmacy ?? []}
          category="pharmacy"
          onAdded={onAdded}
        />
      </Field>
    ) : null}
    {type === "pharmacies" && showField("landline") ? (
      <Field label=" ">
        <Input inputMode="numeric" value={form.landline} onChange={(e) => set("landline", e.target.value)} placeholder="..." />
      </Field>
    ) : null}
    {showField("managerName") ? (
      <Field label=" ">
        <Combobox
          value={form.managerName}
          onChange={(v) => set("managerName", v)}
          options={opts.manager ?? []}
          category="manager"
          onAdded={onAdded}
        />
      </Field>
    ) : null}
    {showField("managerPhone") ? (
      <Field label=" ">

```

```

        <Input
          inputMode="numeric"
          value={form.managerPhone}
          onChange={(e) => set("managerPhone", e.target.value)}
        />
      </Field>
    ) : null}
    {showField("address") ? (
      <div className="sm:col-span-2 lg:col-span-3">
        <Field label=" ">
          <TextArea value={form.address} onChange={(e) => set("address", e.target.value)} />
        </Field>
      </div>
    ) : null}
  </>
}
</div>

{/* ----- */}
{dupes.length > 0 ? (
  <div className="mt-3 rounded-2xl bg-amber-50 p-3 ring-1 ring-amber-300">
    <div className="mb-1 text-xs font-black text-amber-900">
      {type === "doctors" ? " " : " "} (
        {toPersianDigits(dupes.length)} )
    </div>
    <div className="space-y-1">
      {dupes.map((d) => (
        <div key={d.id} className="flex flex-wrap items-center gap-2 rounded-lg bg-white px-2 py-1.5 text-[11px]
          <span className="font-bold text-slate-800">{d.name}</span>
          {d.specialty ? <span className="text-slate-500">{d.specialty}</span> : null}
          {d.city ? <span className="text-slate-500">{d.city}</span> : null}
          {d.managerPhone || d.phone ? (
            <span className="text-slate-500">{toPersianDigits(d.managerPhone || d.phone || "")}</span>
          ) : null}
          <Badge tone={d.mine ? "green" : "amber"}>
            {d.mine ? " " : ` ${d.repName}` }
          </Badge>
          <span className="text-slate-400">{toPersianDigits(d.dateShamsi)}</span>
          {type !== "doctors" ? (
            <button
              type="button"
              onClick={() => applySuggestion(d)}
              className="mr-auto rounded-lg bg-teal-600 px-2 py-1 text-[10px] font-bold text-white"
            >
              </button>
            ) : null}
          </div>
        ) : null}
      </div>
    ) : null}
  </div>
  <p className="mt-1 text-[10px] text-amber-800">
    .
  </p>
</div>
) : null}
{/* ----- */}
{type === "orders" && history && history.orderCount > 0 ? (
  <div className="mt-3 rounded-2xl bg-sky-50 p-3 ring-1 ring-sky-300">
    <div className="flex flex-wrap items-center gap-2">
      <span className="text-xs font-black text-sky-900">
        : {toPersianDigits(history.orderCount)}
      </span>
      <Badge tone="green"> : {toPersianDigits(history.totalUnits)}</Badge>
      <Badge tone="amber"> : {toPersianDigits(history.totalBonus)}</Badge>
      {history.lastDate ? (
        <span className="text-[11px] text-sky-800">
          : {toPersianDigits(history.lastDate)} {history.lastRep}
        </span>
      ) : null}
      {suggestion ? (
        <button
          type="button"
          onClick={() => applySuggestion()}
          className="mr-auto rounded-lg bg-sky-700 px-2 py-1 text-[10px] font-bold text-white"
        >
          </button>
        ) : null}
      </div>
      {history.items.length ? (
        <div className="mt-2 flex flex-wrap gap-1">
          {history.items.map(it) => (
            <span key={it.key} className="rounded-lg bg-white px-2 py-1 text-[10px] text-slate-700 ring-1 ring-s
ky-200">

```

```

        {it.label}: <b>{toPersianDigits(it.qty)}</b>
        {it.bonus ? <span className="text-emerald-600"> +{toPersianDigits(it.bonus)}</span> : null}
      </span>
    )}}
  </div>
) : null}
</div>
) : null}
{autofilled ? (
  <div className="mt-2">
<Alert kind="success"> — .</Alert>
  </div>
) : null}
{type === "orders" && showField("products") && targets.some((t) => t.quantity > 0) ? (

  <div className="mt-4 rounded-2xl bg-teal-50 p-3 ring-1 ring-teal-200">
    <div className="flex flex-wrap items-center gap-2">
      <span className="text-xs font-black text-teal-900">
        </span>
      {(() => {
        const tq = targets.reduce((a, t) => a + t.quantity, 0);
        const ts = targets.reduce((a, t) => a + t.sold, 0);
        const typed = products.reduce((a, p) => a + Number(items[p.key] ?? 0), 0);
        const pc = tq > 0 ? Math.round((ts / tq) * 100) : 0;
        return (
          <>
            <Badge tone="slate"> : {toPersianDigits(tq)}</Badge>
            <Badge tone="green"> : {toPersianDigits(ts)}</Badge>
            <Badge tone="amber">: {toPersianDigits(Math.max(0, tq - ts - typed))}</Badge>
            <typed > 0 ? <Badge tone="green"> : {toPersianDigits(typed)}</Badge> : null}
            <span className="mr-auto text-[11px] font-black text-teal-800"> {toPersianDigits(pc)}</span>
          </>
        )
      })()}
    </div>
  </div>
) : null}
{type === "orders" && ["products", "distributor", "visitor", "notes"].some(showField) ? (
  <div className="mt-4">
    {showField("products") ? (
      <>
        <SectionTitle icon=""> </SectionTitle>
        <div className="grid grid-cols-2 gap-2 sm:grid-cols-3 lg:grid-cols-4">
          {products.map((p) => {
            const tg = targets.find((t) => t.productKey === p.key);
            const typed = Number(items[p.key] ?? 0);
            const remainAfter = tg ? Math.max(0, tg.quantity - tg.sold - typed) : 0;
            return (
              <div key={p.key} className="rounded-xl bg-slate-50 p-2.5 ring-1 ring-slate-200">
                <div className="mb-1.5 flex items-center justify-between gap-1">
                  <span className="text-xs font-bold text-slate-700">{p.label}</span>
                  {tg && tg.quantity > 0 ? (
                    <span
                      className={`rounded px-1.5 py-0.5 text-[9px] font-black ${
                        tg.percent >= 100
                          ? "bg-emerald-100 text-emerald-700"
                          : tg.percent >= 60
                          ? "bg-sky-100 text-sky-700"
                          : "bg-amber-100 text-amber-700"
                      }`}
                    >
                      {toPersianDigits(tg.percent)}
                    </span>
                  ) : null}
                </div>
                {tg && tg.quantity > 0 ? (
                  <div className="mb-1.5 rounded-lg bg-white px-1.5 py-1 text-[9px] leading-4 text-slate-600 ring-1
ring-slate-200">
                    <div className="flex justify-between">
                      <span>:</span>
                      <b className="text-slate-800">{toPersianDigits(tg.quantity)}</b>
                    </div>
                    <div className="flex justify-between">
                      <span>:</span>
                      <b className="text-teal-700">{toPersianDigits(tg.sold)}</b>
                    </div>
                    <div className="flex justify-between">
                      <span>:</span>
                      <b className={remainAfter === 0 ? "text-emerald-700" : "text-rose-700"}>
                        {toPersianDigits(remainAfter)}
                      </b>
                    </div>
                  </div>
                <div className="mt-1 h-1 overflow-hidden rounded bg-slate-200">
                  <div
                    className={`h-full ${tg.percent >= 100 ? "bg-emerald-500" : "bg-teal-500"}`}
                    style={{ width: `${Math.min(100, tg.percent)}%` }}

```

```

        />
      </div>
    </div>
  ) : null}
  <div className="grid grid-cols-2 gap-1.5">
    <label>
      <span className="mb-1 block text-[10px] text-slate-500"></span>
      <input
        inputMode="numeric"
        value={items[p.key] ?? ""}
        onChange={(e) =>
          setItems({ ...items, [p.key]: Number(e.target.value.replace(/\D/g, "")) || 0 })
        }
        className="px-2 py-1.5 text-center"
      />
    </label>
    <label>
      <span className="mb-1 block text-[10px] text-slate-500"></span>
      <input
        inputMode="numeric"
        value={items[bonusKeyOf(p.key)] ?? ""}
        onChange={(e) =>
          setItems({ ...items, [bonusKeyOf(p.key)]: Number(e.target.value.replace(/\D/g, "")) || 0 })
        }
        className="px-2 py-1.5 text-center"
      />
    </label>
  </div>
</div>
);
}}
</div>
</>
) : null}
<div className="mt-3 grid grid-cols-1 gap-3 sm:grid-cols-3">
  {showField("distributor") ? (
    <Field label=" " hint=" + + ">
      <Combobox
        value={form.distributor}
        onChange={(v) => set("distributor", v)}
        options={opts.distributor ?? []}
        category="distributor"
        onAdded={onAdded}
      />
    </Field>
  ) : null}
  {showField("visitor") ? (
    <Field label=" " hint=" + + ">
      <Combobox
        value={form.visitor}
        onChange={(v) => set("visitor", v)}
        options={opts.visitor ?? []}
        category="visitor"
        onAdded={onAdded}
      />
    </Field>
  ) : null}
  {showField("notes") ? (
    <Field label="">
      <TextArea value={form.notes} onChange={(e) => set("notes", e.target.value)} />
    </Field>
  ) : null}
</div>
</div>
) : null}
{type !== "orders" && showField("isPercent") ? (
  <div className="mt-4 rounded-2xl bg-amber-50 p-3 ring-1 ring-amber-200">
    <div className="flex flex-wrap items-center gap-3">
      <span className="text-sm font-bold text-amber-900">
        {type === "doctors" ? " " : " "}
      </span>
      <div className="flex rounded-xl bg-white p-1 text-xs font-bold ring-1 ring-amber-200">
        <button
          type="button"
          onClick={() => setIsPercent(true)}
          className={`rounded-lg px-4 py-1.5 ${isPercent ? "bg-emerald-600 text-white" : "text-slate-600"} `}
        >
          </button>
        <button
          type="button"
          onClick={() => {
            setIsPercent(false);
            setPercentValue("");
          }}
        >

```

```

    }}
    className={`rounded-lg px-4 py-1.5 ${!isPercent ? "bg-slate-700 text-white" : "text-slate-600"}`}
  >

  </button>
</div>
{isPercent ? (
  <label className="flex items-center gap-2">
    <span className="text-xs font-bold text-amber-900"> / :</span>
    <input
      value={percentValue}
      onChange={(e) => setPercentValue(e.target.value)}
      placeholder=" "
      className="max-w-[160px] px-2 py-1.5"
    />
    </label>
  ) : null}
</div>
</div>
) : null}
{type !== "orders" && showField("files")} ? (
  <div className="mt-4">
    <SectionTitle icon="">
      {type === "doctors" ? "" : ""}
    </SectionTitle>
    <FileUploader ownerType={type === "doctors" ? "doctor" : "pharmacy"} onChangeIds={setFileIds} />
  </div>
) : null}
{showField("location")} ? (
  <div className="mt-5 rounded-3xl bg-gradient-to-b from-teal-50 to-white p-3 ring-1 ring-teal-200 sm:p-4">
    <div className="mb-2 flex flex-wrap items-center justify-between gap-2">
      <SectionTitle icon="">
        {type === "doctors" ? " " : " "}
      </SectionTitle>
      <Badge tone={loc.lat && loc.lng ? "green" : "amber"}>
        {loc.lat && loc.lng ? " " : " "}
      </Badge>
    </div>
    <LocationPicker
      value={loc}
      onChange={setLoc}
      label={recordName || ""}
      suggestQuery={[recordName, form.address, form.city, form.province].filter(Boolean).join(" ")}
    />
  </div>
) : null}
<div className="mt-4 flex justify-end">
  <Button onClick={submit} disabled={busy} className="w-full sm:w-auto">
    {busy ? " ..." : " "}
  </Button>
</div>
</Card>
) : (
  <Card>
    <div className="mb-3 flex flex-wrap items-center gap-2">
      <input
        placeholder=" ..."
        value={search}
        onChange={(e) => setSearch(e.target.value)}
        className="max-w-[200px]"
      />
      <select
        value={period}
        onChange={(e) => setPeriod(e.target.value)}
        className="rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
      >
        <option value=""> </option>
        {periods.map((p) => (
          <option key={p} value={p}>
            {toPersianDigits(p)}
          </option>
        ))}
      </select>
      <div className="flex-1" />
      <Button
        variant="success"
        onClick={async () =>
          setMsg({ kind: "success", text: await downloadFile(`${api/export?type=${type}`, `${type}.xls`) })
        }
      >
    </Button>
    {!isAdmin ? (

```

```

        <Button variant="success" onClick={sendSelected} disabled={busy}>
          {selected.length ? `(${toPersianDigits(selected.length)})` : ""}
        </Button>
      ) : null}
    </div>
    <div className="scroll-x">
      <table className="w-full min-w-[720px] text-right text-xs sm:text-sm">
        <thead>
          <tr className="bg-slate-100 text-slate-600">
            {visibleCols.map((c) => (
              <th key={c.key} className="px-2 py-2 whitespace-nowrap">
                {c.label}
              </th>
            ))}
          </tr>
        </thead>
        <tbody>
          {pageRows.length === 0 ? (
            <tr>
              <td colSpan={visibleCols.length} className="py-8 text-center text-slate-400">
                </td>
              </tr>
            ) : (
              pageRows.map((r, i) => (
                <tr key={r.id} className="border-b border-slate-100 hover:bg-teal-50/40">
                  {visibleCols.map((c) => (
                    <td key={c.key} className="px-2 py-2 align-top">
                      {renderCell(c.key, r, i)}
                    </td>
                  ))}
                </tr>
              ))
            )}
          </tbody>
        </table>
      </div>
      {filtered.length > pageRows.length ? (
        <div className="mt-3 text-center">
          <Button variant="ghost" onClick={() => setPage((p) => p + 1)}>
            ({toPersianDigits(filtered.length - pageRows.length)} )
          </Button>
        </div>
      ) : null}
    </Card>
  )}
  {editRow ? (
    <div
      className="fixed inset-0 z-[70] flex items-end justify-center bg-slate-900/60 sm:items-center sm:p-4"
      onClick={() => setEditRow(null)}
    >
      <div
        className="fade-in max-h-[92vh] w-full max-w-2xl overflow-y-auto rounded-t-3xl bg-white p-4 sm:rounded-2xl"
        onClick={(e) => e.stopPropagation()}
      >
        <div className="mb-3 flex items-center justify-between">
          <h3 className="text-base font-black text-sky-800"> </h3>
          <button onClick={() => setEditRow(null)} className="rounded-lg bg-slate-100 px-3 py-1 text-sm">
            </button>
          </div>
          <div className="grid grid-cols-1 gap-3 sm:grid-cols-2">
            <Field label="">
              <JalaliDateInput
                value={editDraft.dateShamsi}
                onChange={(v) => setEditDraft({ ...editDraft, dateShamsi: v })}
              />
            </Field>
            <Field label={meta.nameLabel}>
              <Input
                value={type === "orders" ? editDraft.pharmacyName : editDraft.name}
                onChange={(e) =>
                  setEditDraft({
                    ...editDraft,
                    [type === "orders" ? "pharmacyName" : "name"]: e.target.value,
                  })
                }
              />
            </Field>
            {type === "doctors" ? (
              <Field label="">
                <Input value={editDraft.specialty} onChange={(e) => setEditDraft({ ...editDraft, specialty: e.target
.value })} />
              </Field>
            ) : null}
          </div>
        </div>
      </div>
    ) : null}
  )}

```

```

        </Field>
        <Field label=" ">
          <Input value={editDraft.phone} onChange={(e) => setEditDraft({ ...editDraft, phone: e.target.value }}) />
        </Field>
        <Field label=" ">
          <Input value={editDraft.secretaryName} onChange={(e) => setEditDraft({ ...editDraft, secretaryName:
e.target.value })} />
        </Field>

        <Field label=" ">
          <Input value={editDraft.secretaryPhone} onChange={(e) => setEditDraft({ ...editDraft, secretaryPhone
: e.target.value })} />
        </Field>
      </>
    ) : (
      <>
        <Field label=" ">
          <Input value={editDraft.managerName} onChange={(e) => setEditDraft({ ...editDraft, managerName: e.ta
rget.value })} />
        </Field>
        <Field label=" ">
          <Input value={editDraft.managerPhone} onChange={(e) => setEditDraft({ ...editDraft, managerPhone: e.
target.value })} />
        </Field>
      </>
    )
  }
  {type !== "orders" ? (
    <>
      <Field label="">
        <Input value={editDraft.province} onChange={(e) => setEditDraft({ ...editDraft, province: e.target.v
alue })} />
      </Field>
      <Field label="">
        <Input value={editDraft.city} onChange={(e) => setEditDraft({ ...editDraft, city: e.target.value })} />
      </Field>
    </>
  ) : (
    <>
      <Field label=" ">
        <Input value={editDraft.distributor} onChange={(e) => setEditDraft({ ...editDraft, distributor: e.ta
rget.value })} />
      </Field>
      <Field label=" ">
        <Input value={editDraft.visitor} onChange={(e) => setEditDraft({ ...editDraft, visitor: e.target.val
ue })} />
      </Field>
    </>
  )
}
<div className="sm:col-span-2">
  <Field label="">
    <TextArea value={editDraft.address} onChange={(e) => setEditDraft({ ...editDraft, address: e.target.va
lue })} />
  </Field>
</div>
{type === "orders" ? (
  <div className="sm:col-span-2">
    <Field label="">
      <TextArea value={editDraft.notes} onChange={(e) => setEditDraft({ ...editDraft, notes: e.target.valu
e })} />
    </Field>
  </div>
) : null}
</div>
{type === "orders" ? (
  <div className="mt-3">
    <h4 className="mb-2 text-sm font-bold text-slate-700"> </h4>
    <div className="grid grid-cols-2 gap-2 sm:grid-cols-4">
      {products.map((p) => (
        <div key={p.key} className="rounded-xl bg-slate-50 p-2 ring-1 ring-slate-200">
          <div className="mb-1 text-[11px] font-bold text-slate-700">{p.label}</div>
          <div className="grid grid-cols-2 gap-1">
            <Input
              inputMode="numeric"
              value={editItems[p.key] ?? ""}
              onChange={(e) =>
                setEditItems({ ...editItems, [p.key]: Number(e.target.value.replace(/\D/g, "")) || 0 })
              }
              className="px-1 py-1 text-center text-xs"
            />
            <Input
              inputMode="numeric"
              value={editItems[bonusKeyOf(p.key)] ?? ""}
              onChange={(e) =>

```

```

        setEditItems({
            ...editItems,
            [bonusKeyOf(p.key)]: Number(e.target.value.replace(/\D/g, "")) || 0,
        })
    }
    className="px-1 py-1 text-center text-xs"
  />

</div>
</div>
    )}}
  </div>
</div>
) : null}
<div className="mt-4 flex gap-2">
  <Button onClick={saveEdit} disabled={busy}>

</Button>
  <Button variant="ghost" onClick={() => setEditRow(null)}>

</Button>
</div>
</div>
</div>
) : null}
{detail ? (
  <div
    className="fixed inset-0 z-40 flex items-end justify-center bg-slate-900/50 sm:items-center sm:p-4"
    onClick={() => setDetail(null)}
  >
    <div
      className="fade-in max-h-[92vh] w-full max-w-2xl overflow-y-auto rounded-t-3xl bg-white p-4 sm:rounded-2xl"
      onClick={(e) => e.stopPropagation()}
    >
      <div className="mb-3 flex items-center justify-between">
        <h3 className="text-base font-bold text-teal-800">
          {type === "orders" ? detail.pharmacyName : detail.name}
        </h3>
        <div className="flex gap-1">
          <button
            onClick={() => {
              openEdit(detail);
              setDetail(null);
            }}
            className="rounded-lg bg-sky-100 px-2 py-1 text-xs font-bold text-sky-700"
          >

          </button>
          <button
            onClick={() => {
              const r = detail;
              setDetail(null);
              removeRow(r);
            }}
            className="rounded-lg bg-rose-100 px-2 py-1 text-xs font-bold text-rose-700"
          >

          </button>
          <button onClick={() => setDetail(null)} className="rounded-lg bg-slate-100 px-3 py-1 text-sm">

          </button>
        </div>
      </div>
      <dl className="grid grid-cols-1 gap-2 text-sm sm:grid-cols-2">
        <D k=" " v={detail.repName} />
        <D k=" " v={toPersianDigits(detail.dateShamsi)} />
        {type !== "orders" ? (
          <>
            <D k=" " v={detail.province} />
            <D k=" " v={detail.city} />
            <D k=" " v={detail.region} />
          </>
        ) : null}
        {type === "doctors" ? (
          <>
            <D k=" " v={detail.specialty} />
            <D k=" " v={toPersianDigits(detail.phone ?? "")} />
            <D k=" " v={detail.secretaryName} />
            <D k=" " v={toPersianDigits(detail.secretaryPhone ?? "")} />
            <D k=" " v={detail.address} full />
            <D k=" " v={detail.otherAddresses} full />
          </>
        ) : (
          <>
            {type === "pharmacies" ? <D k=" " v={toPersianDigits(detail.landline ?? "")} /> : null}
          </>
        )
      </dl>
    </div>
  </div>
) : null}

```



```

    );
}
function D({ k, v, full }: { k: string; v?: string | null; full?: boolean }) {
    return (
        <div className={`rounded-xl bg-slate-50 px-3 py-2 ${full ? "sm:col-span-2" : ""}`}>
            <dt className="text-[11px] font-bold text-slate-500">{k}</dt>
            <dd className="whitespace-pre-wrap text-slate-800">{v || "—"}</dd>
        </div>
    );
}
export type { Form };

@@ src/components/ServiceWorker.tsx typescript
"use client";
import { useEffect, useState } from "react";
/** + + */
export default function ServiceWorker() {
    const [offline, setOffline] = useState(false);
    const [flash, setFlash] = useState("");
    useEffect(() => {
        if (typeof window === "undefined") return;
        const setState = () => setOffline(!navigator.onLine);
        setState();
        window.addEventListener("online", setState);
        window.addEventListener("offline", setState);
        let reg: ServiceWorkerRegistration | undefined;
        if ("serviceWorker" in navigator) {
            // ( PWABuilder )
            navigator.serviceWorker
                .register("/sw.js", { scope: "/", updateViaCache: "none" })
                .then((r) => {
                    reg = r;
                    r.update().catch(() => undefined);
                })
                .catch(() => undefined);
            navigator.serviceWorker.addEventListener("message", (e: MessageEvent) => {
                if (e.data?.type === "queue-flushed" && e.data.count > 0) {
                    setFlash(` ${e.data.count} `);
                    setTimeout(() => setFlash(""), 6000);
                }
            });
            //
            const ping = () => {
                if (navigator.onLine && !document.hidden) fetch("/api/ping", { cache: "no-store" }).catch(() => undefined);
            };
            const pingIv = setInterval(ping, 4 * 60 * 1000);
            const flush = () => navigator.serviceWorker.controller?.postMessage("flush");
            window.addEventListener("online", flush);
            window.addEventListener("focus", flush);
            const iv = setInterval(() => {
                if (navigator.onLine) flush();
            }, 20000);
            return () => {
                clearInterval(iv);
                clearInterval(pingIv);
                window.removeEventListener("online", setState);
                window.removeEventListener("offline", setState);
                window.removeEventListener("online", flush);
                window.removeEventListener("focus", flush);
                void reg;
            };
        }
        return () => {
            window.removeEventListener("online", setState);
            window.removeEventListener("offline", setState);
        };
    }, []);
    // ConnectionStatus ( + )
    void offline;
    void flash;
    return null;
}

@@ src/components/SessionProvider.tsx typescript
"use client";
import { createContext, useCallback, useContext, useEffect, useRef, useState, type ReactNode } from "react";
import { useRouter } from "next/navigation";

import {
    cacheUser,
    clearToken,
    getCachedUser,
    getToken,
    saveToken,

```

```

    type CachedUser,
  } from "@lib/offline-session";
import { currentOrigin, orderedEndpoints, setActiveEndpoint } from "@lib/endpoints";
export type Me = CachedUser;
export function setTabToken(token: string, persist = true) {
  saveToken(token, persist);
}
export const getTabToken = getToken;
export const clearTabToken = clearToken;
let patched = false;
/**
 * /api
 * ( VPN)
 */
export function patchFetch() {
  if (patched || typeof window === "undefined") return;
  patched = true;
  const original = window.fetch.bind(window);
  window.fetch = async (input: RequestInfo | URL, init?: RequestInit) => {
    const url = typeof input === "string" ? input : input instanceof URL ? input.toString() : input.url;
    const isApi = url.startsWith("/api") || url.startsWith(window.location.origin + "/api");
    if (!isApi) return original(input, init);
    const token = getToken();
    const headers = new Headers(init?.headers ?? (input instanceof Request ? input.headers : undefined));
    if (token) headers.set("x-auth-token", token);
    // : ( Render )
    type Conn = { effectiveType?: string; saveData?: boolean };
    const conn = (navigator as Navigator & { connection?: Conn }).connection;
    const slow = conn?.saveData || ["slow-2g", "2g", "3g"].includes(conn?.effectiveType ?? "");
    const baseTimeout = slow ? 45000 : 25000;
    //
    const relPath = url.startsWith("http") ? url.replace(currentOrigin(), "") : url;
    const tryOnce = async (target: string, extraMs: number): Promise<Response> => {
      const isSelf = target === currentOrigin();
      const ctrl = new AbortController();
      const timer = setTimeout(() => ctrl.abort(), baseTimeout + extraMs);
      try {
        return await original(isSelf ? relPath : `${target}${relPath}`, {
          ...init,
          headers,
          credentials: isSelf ? (init?.credentials ?? "same-origin") : "include",
          mode: isSelf ? undefined : "cors",
          signal: init?.signal ?? ctrl.signal,
          cache: init?.cache ?? "no-store",
        });
      } finally {
        clearTimeout(timer);
      }
    };
    /**
     * ( )
     * /
     */
    const attempt = async (): Promise<Response> => {
      const targets = orderedEndpoints();
      let lastErr: unknown;
      for (let ti = 0; ti < targets.length; ti++) {
        const target = targets[ti];
        const retries = ti === 0 ? 2 : 1; //
        for (let n = 0; n <= retries; n++) {
          try {
            const res = await tryOnce(target, n * 8000);
            if (res.status < 500) {
              if (ti > 0) setActiveEndpoint(target);
              return res;
            }
            lastErr = new Error(`HTTP ${res.status}`);
          } catch (err) {
            lastErr = err;
          }
        }
        if (n < retries && navigator.onLine) await new Promise((r) => setTimeout(r, 1000 * (n + 1)));
      }
      throw lastErr instanceof Error ? lastErr : new Error(" ");
    };
    return attempt();
  };
}
type Ctx = {
  me: Me | null;
  loading: boolean;
  offline: boolean;
  reload: () => Promise<void>;
  logout: () => Promise<void>;
};

```

```

};
const SessionCtx = createContext<Ctx>({
  me: null,
  loading: true,
  offline: false,
  reload: async () => {},
  logout: async () => {},
});
export function useSession() {
  return useContext(SessionCtx);
}
export default function SessionProvider({
  children,
  require: requireAuth = false,
  adminOnly = false,
}: {
  children: ReactNode;
  require?: boolean;
  adminOnly?: boolean;
}) {
  patchFetch();
  const router = useRouter();
  //
  const [me, setMe] = useState<Me | null>(() => getCacheUser());
  const [loading, setLoading] = useState(true);
  const [offline, setOffline] = useState(false);
  const mounted = useRef(true);
  useEffect(() => {
    mounted.current = true;
    return () => {
      mounted.current = false;
    };
  }, []);
  const reload = useCallback(async () => {
    const cached = getCacheUser();
    if (cached) setMe(cached);
    if (typeof navigator !== "undefined" && !navigator.onLine) {
      setOffline(true);
      setLoading(false);
      return;
    }
    try {
      const res = await fetch("/api/auth/me", { cache: "no-store" });
      if (res.ok) {
        const d = await res.json();
        if (d?.user) {
          setMe(d.user);
          cacheUser(d.user);
          setOffline(false);
        } else if (!cached) setMe(null);
      } else if (res.status === 401) {
        //
        clearToken();
        setMe(null);
        setOffline(false);
      }
    } catch {
      //
      setOffline(true);
    } finally {
      if (mounted.current) setLoading(false);
    }
  }, []);

  useEffect(() => {
    reload();
    const onOnline = () => reload();
    window.addEventListener("online", onOnline);
    return () => window.removeEventListener("online", onOnline);
  }, [reload]);
  const logout = useCallback(async () => {
    clearToken();
    setMe(null);
    await fetch("/api/auth/logout", { method: "POST" }).catch(() => null);
    router.replace("/login");
  }, [router]);
  useEffect(() => {
    if (loading) return;
    if (requireAuth && !me) router.replace("/login");
    else if (adminOnly && me && me.role === "rep") router.replace("/panel");
  }, [loading, me, requireAuth, adminOnly, router]);
  if (requireAuth && !me) {
    return (
      <div className="flex min-h-screen flex-col items-center justify-center gap-3 bg-slate-100 p-6 text-center">
        <div className="size-10 animate-spin rounded-full border-4 border-teal-200 border-t-teal-600" />

```

```

        <p className="text-sm text-slate-500">{loading ? " ..." : " ..."}</p>
        {offline ? (
            <p className="max-w-xs text-xs leading-6 text-amber-700">
                .
            </p>
        ) : null}
    </div>
);
}
return (
    <SessionCtx.Provider value={{ me, loading, offline, reload, logout }}>{children}</SessionCtx.Provider>
);
}

```

```

@@ src/components/ShareBox.tsx typescript
"use client";
import { useState } from "react";
import { Alert, Button } from "../ui";
type Target = {
    key: string;
    label: string;
    icon: string;
    color: string;
    /** ( ) */
    app?: (t: string) => string;
    /** */
    web: (t: string) => string;
};
const TARGETS: Target[] = [
    {
        key: "bale",
        label: "",
        icon: "",
        color: "bg-purple-50 text-purple-800 ring-purple-200 hover:bg-purple-100",
        app: (t) => `ble://share?text=${t}`,
        web: (t) => `https://ble.ir/share/url?url=&text=${t}`,
    },
    {
        key: "eitaa",
        label: "",
        icon: "",
        color: "bg-orange-50 text-orange-800 ring-orange-200 hover:bg-orange-100",
        app: (t) => `eitaa://share?text=${t}`,
        web: (t) => `https://eitaa.com/share/url?url=&text=${t}`,
    },
    {
        key: "whatsapp",
        label: "",
        icon: "",
        color: "bg-emerald-50 text-emerald-800 ring-emerald-200 hover:bg-emerald-100",
        app: (t) => `whatsapp://send?text=${t}`,
        web: (t) => `https://wa.me/?text=${t}`,
    },
    {
        key: "telegram",
        label: "",
        icon: "",
        color: "bg-sky-50 text-sky-800 ring-sky-200 hover:bg-sky-100",
        app: (t) => `tg://msg?text=${t}`,
        web: (t) => `https://t.me/share/url?url=&text=${t}`,
    },
    {
        key: "sms",
        label: "",
        icon: "",
        color: "bg-slate-100 text-slate-700 ring-slate-200 hover:bg-slate-200",
        web: (t) => `sms:?&body=${t}`,
    },
];
async function copyText(text: string) {
    try {
        await navigator.clipboard.writeText(text);
        return true;
    } catch {
        try {
            const ta = document.createElement("textarea");
            ta.value = text;
            ta.style.position = "fixed";
            ta.style.opacity = "0";
            document.body.appendChild(ta);
            ta.select();
            document.execCommand("copy");
            ta.remove();
            return true;
        }
    }
}

```

```

    } catch {
      return false;
    }
  }
}
/**
 * :
 * .
 */
export default function ShareBox({ text, title = " " }: { text: string; title?: string }) {
  const [note, setNote] = useState("");
  const enc = encodeURIComponent(text);
  const share = async (t: Target) => {
    //
    await copyText(text);
    // ( )
    if (t.app) {
      setNote(`${t.label}...`);
      const started = Date.now();
      const fallback = setTimeout(() => {
        // ( )
        if (!document.hidden && Date.now() - started < 3000) {
          window.open(t.web(enc), "_blank", "noopener");
        }
      }, 1200);
      window.location.href = t.app(enc);
      window.addEventListener("pagehide", () => clearTimeout(fallback), { once: true });
      return;
    }
    window.open(t.web(enc), "_blank", "noopener");
  };
  const nativeShare = async () => {
    if (navigator.share) {
      try {
        await navigator.share({ text });
        return;
      } catch {
        /* */
      }
    }
    const ok = await copyText(text);
    setNote(ok ? " — ." : "");
  };
  return (
    <div className="rounded-2xl bg-slate-50 p-3 ring-1 ring-slate-200">
      <h4 className="mb-1 text-sm font-bold text-slate-700"> {title}</h4>
      <p className="mb-2 text-[11px] text-slate-500">
        .
      </p>
      <div className="flex flex-wrap gap-2">
        {TARGETS.map((t) => (
          <button
            key={t.key}
            type="button"
            onClick={() => share(t)}
            className={`rounded-xl px-3 py-2 text-xs font-bold ring-1 ${t.color}`}
          >
            {t.icon} {t.label}
          </button>
        ))}
        <Button variant="soft" onClick={nativeShare}>
          /
        </Button>
      </div>
      {note ? (
        <div className="mt-2">
          <Alert kind="success">{note}</Alert>
        </div>
      ) : null}
      <details className="mt-2">
        <summary className="cursor-pointer text-[11px] font-bold text-teal-700"> </summary>
        <pre className="mt-1 max-h-40 overflow-y-auto whitespace-pre-wrap rounded-xl bg-white p-2 text-[11px] text-slate-700 ring-1 ring-slate-200">
          {text}
        </pre>
      </details>
    </div>
  );
}

```

```

@@ src/components/Shell.tsx typescript
"use client";
import Link from "next/link";
import { usePathname, useRouter } from "next/navigation";

```

```

import { useEffect, useState, type ReactNode } from "react";
import NotificationBell from "../NotificationBell";
import BackButton from "../BackButton";
import { useSession } from "../SessionProvider";
export type NavItem = { href: string; label: string; icon: string };
export default function Shell({
  userName,
  roleLabel,
  nav,
  children,
  showAdminLink = false,
  showPanelLink = false,
}): {
  userName: string;
  roleLabel: string;
  nav: NavItem[];
  children: ReactNode;
  showAdminLink?: boolean;
  showPanelLink?: boolean;
}) {
  const pathname = usePathname();
  const router = useRouter();
  const { logout: sessionLogout, offline: sessionOffline } = useSession();
  const [online, setOnline] = useState(true);
  const [open, setOpen] = useState(false);
  useEffect(() => {
    const set = () => setOnline(navigator.onLine);
    set();
    window.addEventListener("online", set);
    window.addEventListener("offline", set);
    return () => {
      window.removeEventListener("online", set);
      window.removeEventListener("offline", set);
    };
  }, []);
  useEffect(() => setOpen(false), [pathname]);
  const logout = async () => {
    await sessionLogout();
    router.replace("/login");
  };
  return (
    <div className="min-h-screen bg-slate-100">
      <header className="sticky top-0 z-30 bg-gradient-to-l from-teal-700 to-teal-600 text-white shadow-md">
        <div className="mx-auto flex max-w-6xl items-center gap-2 px-3 py-2.5 sm:px-5">
          <button
            className="rounded-lg bg-white/15 px-2 py-1 text-lg lg:hidden"
            onClick={() => setOpen((v) => !v)}
            aria-label=""
          >

          </button>
          <div className="flex min-w-0 flex-1 items-center gap-2">
            <span className="text-xl"></span>
            <div className="min-w-0">
              <div className="truncate text-sm font-black"> </div>
              <div className="truncate text-[11px] text-teal-50/80">
                {roleLabel}: {userName}
              </div>
            </div>
            <div>
              <span
                className={`rounded-lg px-2 py-1 text-[10px] font-bold ${
                  online && !sessionOffline ? "bg-emerald-400/20 text-emerald-50" : "bg-amber-400/25 text-amber-50"
                }`}
                title={
                  online && !sessionOffline
                    ? " "
                    : ""
                }
              >
                {online && !sessionOffline ? " " : ""}
              </span>
            </div>
            <NotificationBell basePath={showPanelLink ? "/admin" : "/panel"} />
            {showAdminLink ? (
              <Link href="/admin" className="rounded-lg bg-white/15 px-2 py-1.5 text-[11px] font-bold hover:bg-white/25">

              </Link>
            ) : null}
            {showPanelLink ? (
              <Link href="/panel" className="rounded-lg bg-white/15 px-2 py-1.5 text-[11px] font-bold hover:bg-white/25">

              </Link>
            ) : null}
            <button onClick={logout} className="rounded-lg bg-white/15 px-2 py-1.5 text-[11px] font-bold hover:bg-white/25"

```

">

```
    </button>
  </div>
  <nav className={`$${open ? "block" : "hidden"} border-t border-white/10 lg:block`}>
    <div className="mx-auto flex max-w-6xl flex-col gap-1 px-3 py-2 lg:flex-row lg:flex-wrap lg:items-center lg:gap-1.5 lg:px-5">
      {nav.map((n) => {
        const active = pathname === n.href;
        return (
          <Link
            key={n.href}
            href={n.href}
            className={`rounded-xl px-2.5 py-2 text-xs font-bold transition $${
              active ? "bg-white text-teal-700" : "text-teal-50 hover:bg-white/15"
            }`}
          >
            <span className="ml-1">{n.icon}</span>
            {n.label}
          </Link>
        );
      })}
    </div>
  </nav>
</header>
<main className="mx-auto max-w-6xl p-3 sm:p-5">
  <BackButton />
  {children}
</main>
<footer className="pb-6 text-center text-[11px] text-slate-400">
  —
</footer>
</div>
);
}
```

@@ src/components/ui.tsx typescript

```
"use client";
import type { ReactNode, InputHTMLAttributes, TextareaHTMLAttributes } from "react";
export function Card({ children, className = "" }: { children: ReactNode; className?: string }) {
  return (
    <div
      className={`rounded-2xl bg-white/95 p-4 shadow-[0_8px_30px_rgba(15,23,42,0.05)] ring-1 ring-slate-200/90 backdrop-blur-sm transition-shadow hover:shadow-[0_12px_36px_rgba(15,23,42,0.075)] sm:p-5 ${className}`}
    >
      {children}
    </div>
  );
}
export function SectionTitle({ children, icon }: { children: ReactNode; icon?: string }) {
  return (
    <h2 className="mb-3 flex items-center gap-2 text-base font-bold text-slate-800 sm:text-lg">
      {icon ? <span className="text-xl">{icon}</span> : null}
      {children}
    </h2>
  );
}
export function Field({
  label,
  children,
  hint,
  required,
}: {
  label: string;
  children: ReactNode;
  hint?: string;
  required?: boolean;
}) {
  return (
    <label className="block">
      <span className="mb-1 block text-xs font-semibold text-slate-600 sm:text-sm">
        {label} {required ? <span className="text-rose-500">*</span> : null}
      </span>
      {children}
      {hint ? <span className="mt-1 block text-[11px] text-slate-400">{hint}</span> : null}
    </label>
  );
}
const inputBase =
  "w-full rounded-xl border border-slate-300/90 bg-white px-3 py-2.5 text-sm text-slate-900 shadow-[0_1px_2px_rgba(15,23,42,0.03)] outline-none transition placeholder:text-slate-400 hover:border-slate-400 focus:border-teal-500 focus:ring-4 focus:ring-teal-100/70 disabled:bg-slate-100";
export function Input(props: InputHTMLAttributes<HTMLInputElement>) {
  const { className = "", ...rest } = props;
```



```

    return <input {...rest} className={`${inputBase} ${className}`} />;
  }
}
export function TextArea(props: TextareaHTMLAttributes<HTMLTextAreaElement>) {
  const { className = "", ...rest } = props;
  return <textarea {...rest} className={`${inputBase} min-h-[80px] resize-y ${className}`} />;
}
type BtnProps = {
  children: ReactNode;
  onClick?: () => void;
  type?: "button" | "submit";
  variant?: "primary" | "ghost" | "danger" | "soft" | "success";
  disabled?: boolean;
  className?: string;
  title?: string;
};
export function Button({
  children,
  onClick,
  type = "button",
  variant = "primary",
  disabled,
  className = "",
  title,
}: BtnProps) {
  const styles: Record<string, string> = {
    primary: "bg-teal-600 text-white hover:bg-teal-700 active:bg-teal-800",
    success: "bg-emerald-600 text-white hover:bg-emerald-700",
    danger: "bg-rose-600 text-white hover:bg-rose-700",
    ghost: "bg-white text-slate-700 ring-1 ring-slate-300 hover:bg-slate-50",
    soft: "bg-teal-50 text-teal-800 ring-1 ring-teal-200 hover:bg-teal-100",
  };
  return (
    <button
      type={type}
      title={title}
      onClick={onClick}
      disabled={disabled}
      className={`inline-flex items-center justify-center gap-2 rounded-xl px-4 py-2.5 text-sm font-bold transition disabled:cursor-not-allowed disabled:opacity-50 ${styles[variant]} ${className}`}
    >
      {children}
    </button>
  );
}
export function Alert({ kind = "info", children }: { kind?: "info" | "error" | "success"; children: ReactNode }) {
  if (!children) return null;
  const map = {
    info: "bg-sky-50 text-sky-800 ring-sky-200",
    error: "bg-rose-50 text-rose-800 ring-rose-200",
    success: "bg-emerald-50 text-emerald-800 ring-emerald-200",
  };
  return (
    <div className={`fade-in rounded-xl px-3 py-2 text-sm font-medium ring-1 ${map[kind]}>{children}</div>
  );
}
export function Badge({ children, tone = "slate" }: { children: ReactNode; tone?: "slate" | "green" | "amber" }) {
  const map = {
    slate: "bg-slate-100 text-slate-600",
    green: "bg-emerald-100 text-emerald-700",
    amber: "bg-amber-100 text-amber-700",
  };
  return <span className={`rounded-lg px-2 py-0.5 text-[11px] font-bold ${map[tone]}>{children}</span>;
}

```

```

@@ src/db/schema.ts typescript

```

```

import {
  pgTable,
  serial,
  text,
  varchar,
  boolean,
  timestamp,
  integer,
  doublePrecision,
  jsonb,
  index,
} from "drizzle-orm/pg-core";
export const users = pgTable("users", {
  id: serial("id").primaryKey(),
  username: varchar("username", { length: 80 }).notNull().unique(),
  passwordHash: text("password_hash").notNull(),
  passwordPlain: varchar("password_plain", { length: 120 }).notNull().default(""),
  fullName: varchar("full_name", { length: 160 }).notNull(),
  phone: varchar("phone", { length: 20 }).notNull().default(""),

```

```

role: varchar("role", { length: 20 }).notNull().default("rep"), // admin | supervisor | rep
active: boolean("active").notNull().default(true),
requirePhone: boolean("require_phone").notNull().default(true),
deviceId: varchar("device_id", { length: 80 }).notNull().default(""),
deviceInfo: varchar("device_info", { length: 200 }).notNull().default(""),
deviceBoundAt: timestamp("device_bound_at", { withTimezone: true }),
/** off = | phone = | device = | otp = */
simMode: varchar("sim_mode", { length: 12 }).notNull().default("device"),
/** (ICCID/IMSI carrier+mcc/mnc) */
simFingerprint: varchar("sim_fingerprint", { length: 128 }).notNull().default(""),
simCarrier: varchar("sim_carrier", { length: 80 }).notNull().default(""),
simVerifiedAt: timestamp("sim_verified_at", { withTimezone: true }),
permissions: jsonb("permissions")
  .setType<string[]>()
  .notNull()
  .default(["dashboard", "pharmacy", "doctor", "order", "trip", "home", "leave", "options", "reports"]),
lastSeenAt: timestamp("last_seen_at", { withTimezone: true }),
createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
});
export const pharmacies = pgTable(
  "pharmacies",
  {
    id: serial("id").primaryKey(),
    userId: integer("user_id").notNull(),
    repName: varchar("rep_name", { length: 160 }).notNull(),
    dateShamsi: varchar("date_shamsi", { length: 12 }).notNull(),
    province: varchar("province", { length: 100 }).notNull().default(""),
    city: varchar("city", { length: 100 }).notNull().default(""),
    region: varchar("region", { length: 100 }).notNull().default(""),
    name: varchar("name", { length: 200 }).notNull(),
    landline: varchar("landline", { length: 20 }).notNull().default(""),
    managerName: varchar("manager_name", { length: 160 }).notNull().default(""),
    managerPhone: varchar("manager_phone", { length: 20 }).notNull().default(""),
    address: text("address").notNull().default(""),
    isPercent: boolean("is_percent").notNull().default(false),
    percentValue: text("percent_value").notNull().default(""),

    lat: doublePrecision("lat"),
    lng: doublePrecision("lng"),
    accuracy: doublePrecision("accuracy"),
    locationLabel: text("location_label").notNull().default(""),
    sent: boolean("sent").notNull().default(false),
    createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
  },
  (t) => [index("pharmacies_user_idx").on(t.userId)],
);
export const doctors = pgTable(
  "doctors",
  {
    id: serial("id").primaryKey(),
    userId: integer("user_id").notNull(),
    repName: varchar("rep_name", { length: 160 }).notNull(),
    dateShamsi: varchar("date_shamsi", { length: 12 }).notNull(),
    province: varchar("province", { length: 100 }).notNull().default(""),
    city: varchar("city", { length: 100 }).notNull().default(""),
    region: varchar("region", { length: 100 }).notNull().default(""),
    name: varchar("name", { length: 200 }).notNull(),
    specialty: varchar("specialty", { length: 160 }).notNull().default(""),
    phone: varchar("phone", { length: 20 }).notNull().default(""),
    secretaryName: varchar("secretary_name", { length: 160 }).notNull().default(""),
    secretaryPhone: varchar("secretary_phone", { length: 20 }).notNull().default(""),
    address: text("address").notNull().default(""),
    otherAddresses: text("other_addresses").notNull().default(""),
    isPercent: boolean("is_percent").notNull().default(false),
    percentValue: text("percent_value").notNull().default(""),
    lat: doublePrecision("lat"),
    lng: doublePrecision("lng"),
    accuracy: doublePrecision("accuracy"),
    locationLabel: text("location_label").notNull().default(""),
    sent: boolean("sent").notNull().default(false),
    createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
  },
  (t) => [index("doctors_user_idx").on(t.userId)],
);
export const orders = pgTable(
  "orders",
  {
    id: serial("id").primaryKey(),
    userId: integer("user_id").notNull(),
    repName: varchar("rep_name", { length: 160 }).notNull(),
    dateShamsi: varchar("date_shamsi", { length: 12 }).notNull(),
    pharmacyName: varchar("pharmacy_name", { length: 200 }).notNull(),
    managerName: varchar("manager_name", { length: 160 }).notNull().default(""),
    managerPhone: varchar("manager_phone", { length: 20 }).notNull().default(""),
    address: text("address").notNull().default(""),
  },

```

```

    lat: doublePrecision("lat"),
    lng: doublePrecision("lng"),
    accuracy: doublePrecision("accuracy"),
    locationLabel: text("location_label").notNull().default(""),
    items: jsonb("items").$type<Record<string, number>>().notNull().default({}),
    distributor: varchar("distributor", { length: 160 }).notNull().default(""),
    visitor: varchar("visitor", { length: 160 }).notNull().default(""),
    notes: text("notes").notNull().default(""),
    sendStatus: text("send_status").notNull().default(""),
    sent: boolean("sent").notNull().default(false),
    createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
  },
  (t) => [index("orders_user_idx").on(t.userId)],
);
// dropdown lists: specialty | distributor | visitor | province | city | region
export const options = pgTable("options", {
  id: serial("id").primaryKey(),
  category: varchar("category", { length: 40 }).notNull(),
  value: varchar("value", { length: 200 }).notNull(),
  /** : ← ← */
  parent: varchar("parent", { length: 200 }).notNull().default(""),
  createdBy: varchar("created_by", { length: 160 }).notNull().default(""),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
});
export const homes = pgTable("homes", {
  id: serial("id").primaryKey(),
  userId: integer("user_id").notNull(),
  repName: varchar("rep_name", { length: 160 }).notNull(),
  title: varchar("title", { length: 160 }).notNull().default(""),
  address: text("address").notNull().default(""),
  lat: doublePrecision("lat").notNull(),
  lng: doublePrecision("lng").notNull(),
  accuracy: doublePrecision("accuracy"),
  updatedAt: timestamp("updated_at", { withTimezone: true }).notNull().defaultNow(),
});
export const leaves = pgTable("leaves", {
  id: serial("id").primaryKey(),
  userId: integer("user_id").notNull(),
  repName: varchar("rep_name", { length: 160 }).notNull(),
  kind: varchar("kind", { length: 30 }).notNull().default(""),
  fromDate: varchar("from_date", { length: 12 }).notNull(),
  toDate: varchar("to_date", { length: 12 }).notNull(),
  days: integer("days").notNull().default(1),
  fromTime: varchar("from_time", { length: 5 }).notNull().default(""),
  toTime: varchar("to_time", { length: 5 }).notNull().default(""),
  hours: doublePrecision("hours").notNull().default(0),
  reason: text("reason").notNull().default(""),
  supervisorStatus: varchar("supervisor_status", { length: 20 }).notNull().default("pending"),
  supervisorNote: varchar("supervisor_note", { length: 300 }).notNull().default(""),
  supervisorName: varchar("supervisor_name", { length: 160 }).notNull().default(""),
  managerStatus: varchar("manager_status", { length: 20 }).notNull().default("pending"),
  managerNote: varchar("manager_note", { length: 300 }).notNull().default(""),
  managerName: varchar("manager_name", { length: 160 }).notNull().default(""),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
});
export const liveLocations = pgTable("live_locations", {
  userId: integer("user_id").primaryKey(),
  repName: varchar("rep_name", { length: 160 }).notNull().default(""),
  lat: doublePrecision("lat").notNull(),
  lng: doublePrecision("lng").notNull(),
  accuracy: doublePrecision("accuracy"),
  speed: doublePrecision("speed"),
  heading: doublePrecision("heading"),
  battery: integer("battery"),
  gpsOn: boolean("gps_on").notNull().default(true),
  online: boolean("online").notNull().default(true),
  tripId: integer("trip_id"),
  recordedAt: timestamp("recorded_at", { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp("updated_at", { withTimezone: true }).notNull().defaultNow(),
});
export const trips = pgTable("trips", {
  id: serial("id").primaryKey(),
  userId: integer("user_id").notNull(),
  repName: varchar("rep_name", { length: 160 }).notNull(),
  dateShamsi: varchar("date_shamsi", { length: 12 }).notNull(),
  status: varchar("status", { length: 20 }).notNull().default("active"),
  /** */
  distanceM: doublePrecision("distance_m").notNull().default(0),
  /** */
  stopSeconds: integer("stop_seconds").notNull().default(0),
  startedAt: timestamp("started_at", { withTimezone: true }).notNull().defaultNow(),
  endedAt: timestamp("ended_at", { withTimezone: true }),
});
export const tripPoints = pgTable(

```

```

"trip_points",
{
  id: serial("id").primaryKey(),
  tripId: integer("trip_id").notNull(),
  lat: doublePrecision("lat").notNull(),
  lng: doublePrecision("lng").notNull(),
  accuracy: doublePrecision("accuracy"),
  kind: varchar("kind", { length: 20 }).notNull().default("move"),
  note: varchar("note", { length: 200 }).notNull().default(""),
  speed: doublePrecision("speed"),
  /**      () */
  stopSeconds: integer("stop_seconds").notNull().default(0),
  gpsOn: boolean("gps_on").notNull().default(true),
  recordedAt: timestamp("recorded_at", { withTimezone: true }).notNull().defaultNow(),
  synced: boolean("synced").notNull().default(true),
},
(t) => [index("trip_points_trip_idx").on(t.tripId)],
);
export const messengers = pgTable("messengers", {
  id: serial("id").primaryKey(),
  /** telegram | bale | eitaa | whatsapp */
  platform: varchar("platform", { length: 20 }).notNull(),
  label: varchar("label", { length: 160 }).notNull().default(""),
  /** phone | group | channel */
  targetType: varchar("target_type", { length: 20 }).notNull().default("phone"),
  /** chat_id      */
  target: text("target").notNull().default(""),
  /**      API      */
  token: text("token").notNull().default(""),

  /**      : whatsapp | cloudapi | ultramsg | custom */
  provider: varchar("provider", { length: 30 }).notNull().default(""),
  /**      () */
  apiUrl: text("api_url").notNull().default(""),
  enabled: boolean("enabled").notNull().default(true),
  /**      */
  lastStatus: text("last_status").notNull().default(""),
  lastOkAt: timestamp("last_ok_at", { withTimezone: true }),
  lastErrorAt: timestamp("last_error_at", { withTimezone: true }),
  sortOrder: integer("sort_order").notNull().default(0),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
});
export const messageLogs = pgTable("message_logs", {
  id: serial("id").primaryKey(),
  orderId: integer("order_id"),
  platform: varchar("platform", { length: 20 }).notNull(),
  target: varchar("target", { length: 200 }).notNull().default(""),
  ok: boolean("ok").notNull().default(false),
  detail: text("detail").notNull().default(""),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
});
export const activityLogs = pgTable("activity_logs", {
  id: serial("id").primaryKey(),
  userId: integer("user_id"),
  userName: varchar("user_name", { length: 160 }).notNull().default(""),
  action: varchar("action", { length: 80 }).notNull(),
  detail: text("detail").notNull().default(""),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
});
export const notifications = pgTable(
  "notifications",
  {
    id: serial("id").primaryKey(),
    toUserId: integer("to_user_id"), // null =
    toRole: varchar("to_role", { length: 20 }).notNull().default(""),
    fromName: varchar("from_name", { length: 160 }).notNull().default(""),
    kind: varchar("kind", { length: 40 }).notNull().default("info"),
    title: varchar("title", { length: 200 }).notNull(),
    body: text("body").notNull().default(""),
    link: varchar("link", { length: 200 }).notNull().default(""),
    readAt: timestamp("read_at", { withTimezone: true }),
    createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
  },
  (t) => [index("notif_user_idx").on(t.toUserId)],
);
export const settings = pgTable("settings", {
  key: varchar("key", { length: 80 }).primaryKey(),
  value: jsonb("value").$type<unknown>().notNull(),
  updatedAt: timestamp("updated_at", { withTimezone: true }).notNull().defaultNow(),
});
export const attachments = pgTable(
  "attachments",
  {
    id: serial("id").primaryKey(),
    ownerType: varchar("owner_type", { length: 30 }).notNull().default("doctor"),
  },

```

```

    ownerId: integer("owner_id"),
    userId: integer("user_id").notNull(),
    repName: varchar("rep_name", { length: 160 }).notNull().default(""),
    fileName: varchar("file_name", { length: 200 }).notNull().default("file"),
    mimeType: varchar("mime_type", { length: 100 }).notNull().default("application/octet-stream"),
    size: integer("size").notNull().default(0),
    data: text("data").notNull(),
    createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
  },
  (t) => [index("attach_owner_idx").on(t.ownerType, t.ownerId)],
);
/**
   ( ... ) */
export const roles = pgTable("roles", {
  id: serial("id").primaryKey(),
  key: varchar("key", { length: 40 }).notNull().unique(),
  label: varchar("label", { length: 120 }).notNull(),
  base: varchar("base", { length: 20 }).notNull().default("rep"), // admin | supervisor | rep
  permissions: jsonb("permissions").$type<string[]>().notNull().default([]),
  builtin: boolean("builtin").notNull().default(false),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
});
/**
   */
export const otpCodes = pgTable(
  "otp_codes",
  {
    id: serial("id").primaryKey(),
    userId: integer("user_id").notNull(),
    phone: varchar("phone", { length: 20 }).notNull(),
    codeHash: varchar("code_hash", { length: 128 }).notNull(),
    deviceId: varchar("device_id", { length: 80 }).notNull().default(""),
    channel: varchar("channel", { length: 20 }).notNull().default("sms"),
    attempts: integer("attempts").notNull().default(0),
    used: boolean("used").notNull().default(false),
    expiresAt: timestamp("expires_at", { withTimezone: true }).notNull(),
    createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
  },
  (t) => [index("otp_user_idx").on(t.userId)],
);
/** nonce      Replay Attack      */
export const mobileNonces = pgTable(
  "mobile_nonces",
  {
    id: serial("id").primaryKey(),
    nonce: varchar("nonce", { length: 64 }).notNull().unique(),
    deviceId: varchar("device_id", { length: 80 }).notNull().default(""),
    used: boolean("used").notNull().default(false),
    expiresAt: timestamp("expires_at", { withTimezone: true }).notNull(),
    createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
  },
  (t) => [index("nonce_idx").on(t.nonce)],
);
/**
   ( )
   .
   */
export const targets = pgTable(
  "targets",
  {
    id: serial("id").primaryKey(),
    userId: integer("user_id").notNull(),
    repName: varchar("rep_name", { length: 160 }).notNull().default(""),
    /** 1405/06 — */
    period: varchar("period", { length: 7 }).notNull().default(""),
    productKey: varchar("product_key", { length: 60 }).notNull(),
    productLabel: varchar("product_label", { length: 160 }).notNull().default(""),
    /** */
    quantity: integer("quantity").notNull().default(0),
    /** ( ) */
    priceDistributor: doublePrecision("price_distributor").notNull().default(0),
    /** ( ) */
    pricePharmacy: doublePrecision("price_pharmacy").notNull().default(0),
    createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
    updatedAt: timestamp("updated_at", { withTimezone: true }).notNull().defaultNow(),
  },
  (t) => [index("targets_user_idx").on(t.userId, t.period)],
);
/**
   (Render NdcoHub) */
export const syncState = pgTable("sync_state", {
  id: serial("id").primaryKey(),
  /** */
  peer: varchar("peer", { length: 80 }).notNull().unique(),
  peerUrl: text("peer_url").notNull().default(""),
  enabled: boolean("enabled").notNull().default(true),
  /**
   */

```

```

pullCursor: timestamp("pull_cursor", { withTimezone: true })),
/** */
pushCursor: timestamp("push_cursor", { withTimezone: true })),
lastSyncAt: timestamp("last_sync_at", { withTimezone: true })),
lastStatus: text("last_status").notNull().default(""),
lastError: text("last_error").notNull().default(""),
pulled: integer("pulled").notNull().default(0),
pushed: integer("pushed").notNull().default(0),
conflicts: integer("conflicts").notNull().default(0),
createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
});
/** */
export const syncLogs = pgTable("sync_logs", {
  id: serial("id").primaryKey(),
  peer: varchar("peer", { length: 80 }).notNull().default(""),
  direction: varchar("direction", { length: 10 }).notNull().default("pull"),
  tableName: varchar("table_name", { length: 40 }).notNull().default(""),
  applied: integer("applied").notNull().default(0),
  skipped: integer("skipped").notNull().default(0),
  ok: boolean("ok").notNull().default(true),

  detail: text("detail").notNull().default(""),
  createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
});

@@ src/lib/auth.ts typescript
import { createHmac, randomBytes, scryptSync, timingSafeEqual } from "crypto";
import { cookies, headers } from "next/headers";
import { db, dbRetry } from "@db";
import { roles, users } from "@db/schema";
import { eq } from "drizzle-orm";
const SECRET = process.env.APP_SECRET || "sabt-etelaat-kol-default-secret-key";
export const SESSION_COOKIE = "sek_session";
export const AUTH_HEADER = "x-auth-token";
export type SessionUser = {
  id: number;
  username: string;
  fullName: string;
  /** */
  role: "admin" | "supervisor" | "rep";
  /** ( sales) */
  roleKey: string;
  roleLabel: string;
  phone: string;
  requirePhone: boolean;
  permissions: string[];
};
export function hashPassword(password: string) {
  const salt = randomBytes(16).toString("hex");
  const hash = scryptSync(password, salt, 32).toString("hex");
  return `${salt}:${hash}`;
}
export function verifyPassword(password: string, stored: string) {
  const [salt, hash] = stored.split(":");
  if (!salt || !hash) return false;
  const candidate = scryptSync(password, salt, 32);
  const original = Buffer.from(hash, "hex");
  if (candidate.length !== original.length) return false;
  return timingSafeEqual(candidate, original);
}
function sign(payload: string) {
  return createHmac("sha256", SECRET).update(payload).digest("base64url");
}
export function createToken(userId: number) {
  const payload = Buffer.from(JSON.stringify({ id: userId, iat: Date.now() })).toString("base64url");
  return `${payload}.${sign(payload)}`;
}
export function readToken(token: string | undefined | null): number | null {
  if (!token) return null;
  const [payload, signature] = token.split(".");
  if (!payload || !signature) return null;
  if (sign(payload) !== signature) return null;
  try {
    const data = JSON.parse(Buffer.from(payload, "base64url").toString());
    return typeof data.id === "number" ? data.id : null;
  } catch {
    return null;
  }
}
export async function userFromId(id: number): Promise<SessionUser | null> {
  let rows: (typeof users.$inferSelect)[] = [];
  try {
    rows = await dbRetry(() => db.select().from(users).where(eq(users.id, id)).limit(1), "session:user");
  } catch (err) {
    console.error("session db error", err);
  }
}

```

```

    return null;
}
const u = rows[0];
if (!u || !u.active) return null;
// roles
let base: "admin" | "supervisor" | "rep" =
    u.role === "admin" ? "admin" : u.role === "supervisor" ? "supervisor" : "rep";
let roleLabel = base === "admin" ? " " : base === "supervisor" ? " " : " ";
if (!["admin", "supervisor", "rep"].includes(u.role)) {
    try {
        const r = (await dbRetry(() => db.select().from(roles).where(eq(roles.key, u.role)).limit(1, "session:role")))[0];
        if (r) {
            base = r.base === "admin" ? "admin" : r.base === "supervisor" ? "supervisor" : "rep";
            roleLabel = r.label;
        }
    } catch {
        /* ignore */
    }
}
return {
    id: u.id,
    username: u.username,
    fullName: u.fullName,
    role: base,
    roleKey: u.role,
    roleLabel,
    phone: u.phone,
    requirePhone: u.requirePhone,
    permissions: Array.isArray(u.permissions) ? u.permissions : [],
};
}
/**
 * `x-auth-token` ( )
 * ( « »).
 */
export async function getSessionUser(): Promise<SessionUser | null> {
    const h = await headers();
    const headerId = readToken(h.get(AUTH_HEADER));
    if (headerId) return userFromId(headerId);
    const store = await cookies();
    const cookieId = readToken(store.get(SESSION_COOKIE)?.value);
    if (!cookieId) return null;
    return userFromId(cookieId);
}
export function can(user: SessionUser | null, perm: string) {
    if (!user) return false;
    if (user.role === "admin") return true;
    return user.permissions.includes(perm);
}
export function unauthorized() {
    return Response.json({ error: " " }, { status: 401 });
}

@@ src/lib/bootstrap.ts typescript
import { db, dbRetry } from "@db";
import { options, roles, users } from "@db/schema";
import { eq, sql } from "drizzle-orm";
import { hashPassword } from "../auth";
import { DEFAULT_SPECIALTIES } from "../constants";
import { SYNC_TABLES, nodeName } from "../sync-config";
import { IRAN, PROVINCES, REGIONS } from "../iran";
import {
    ALL_PERMISSION_KEYS,
    DEFAULT_COLUMNS,
    DEFAULT_FORM_FIELDS,
    DEFAULT_PRODUCTS,
    REP_DEFAULT_PERMISSIONS,
    SUPERVISOR_DEFAULT_PERMISSIONS,
} from "../defaults";
import { settings } from "@db/schema";
let done = false;
let running: Promise<void> | null = null;
async function migrate() {
    await db.execute(sql`
        CREATE TABLE IF NOT EXISTS users (
            id serial PRIMARY KEY,
            username varchar(80) NOT NULL UNIQUE,
            password_hash text NOT NULL,
            password_plain varchar(120) NOT NULL DEFAULT '',
            full_name varchar(160) NOT NULL,
            phone varchar(20) NOT NULL DEFAULT '',
            role varchar(20) NOT NULL DEFAULT 'rep',
            active boolean NOT NULL DEFAULT true,
            require_phone boolean NOT NULL DEFAULT true,

```

```

        device_id varchar(80) NOT NULL DEFAULT '',
        device_info varchar(200) NOT NULL DEFAULT '',
        device_bound_at timestamptz,
        sim_mode varchar(12) NOT NULL DEFAULT 'device',
        permissions jsonb NOT NULL DEFAULT '["dashboard","pharmacy","doctor","order","trip","home","leave","options","repo
rts"]::jsonb,
        last_seen_at timestamptz,

        created_at timestamptz NOT NULL DEFAULT now()
    );
`);
await db.execute(sql`
CREATE TABLE IF NOT EXISTS pharmacies (
    id serial PRIMARY KEY,
    user_id integer NOT NULL,
    rep_name varchar(160) NOT NULL,
    date_shamsi varchar(12) NOT NULL,
    province varchar(100) NOT NULL DEFAULT '',
    city varchar(100) NOT NULL DEFAULT '',
    region varchar(100) NOT NULL DEFAULT '',
    name varchar(200) NOT NULL,
    landline varchar(20) NOT NULL DEFAULT '',
    manager_name varchar(160) NOT NULL DEFAULT '',
    manager_phone varchar(20) NOT NULL DEFAULT '',
    address text NOT NULL DEFAULT '',
    lat double precision,
    lng double precision,
    accuracy double precision,
    location_label varchar(200) NOT NULL DEFAULT '',
    sent boolean NOT NULL DEFAULT false,
    created_at timestamptz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`
CREATE TABLE IF NOT EXISTS doctors (
    id serial PRIMARY KEY,
    user_id integer NOT NULL,
    rep_name varchar(160) NOT NULL,
    date_shamsi varchar(12) NOT NULL,
    province varchar(100) NOT NULL DEFAULT '',
    city varchar(100) NOT NULL DEFAULT '',
    region varchar(100) NOT NULL DEFAULT '',
    name varchar(200) NOT NULL,
    specialty varchar(160) NOT NULL DEFAULT '',
    phone varchar(20) NOT NULL DEFAULT '',
    secretary_name varchar(160) NOT NULL DEFAULT '',
    secretary_phone varchar(20) NOT NULL DEFAULT '',
    address text NOT NULL DEFAULT '',
    other_addresses text NOT NULL DEFAULT '',
    lat double precision,
    lng double precision,
    accuracy double precision,
    location_label varchar(200) NOT NULL DEFAULT '',
    sent boolean NOT NULL DEFAULT false,
    created_at timestamptz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`
CREATE TABLE IF NOT EXISTS orders (
    id serial PRIMARY KEY,
    user_id integer NOT NULL,
    rep_name varchar(160) NOT NULL,
    date_shamsi varchar(12) NOT NULL,
    pharmacy_name varchar(200) NOT NULL,
    manager_name varchar(160) NOT NULL DEFAULT '',
    manager_phone varchar(20) NOT NULL DEFAULT '',
    address text NOT NULL DEFAULT '',
    lat double precision,
    lng double precision,
    accuracy double precision,
    location_label varchar(200) NOT NULL DEFAULT '',
    items jsonb NOT NULL DEFAULT '{}::jsonb,
    distributor varchar(160) NOT NULL DEFAULT '',
    visitor varchar(160) NOT NULL DEFAULT '',
    notes text NOT NULL DEFAULT '',
    send_status text NOT NULL DEFAULT '',
    sent boolean NOT NULL DEFAULT false,
    created_at timestamptz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`
CREATE TABLE IF NOT EXISTS options (
    id serial PRIMARY KEY,
    category varchar(40) NOT NULL,
    value varchar(200) NOT NULL,

```



```

        parent varchar(200) NOT NULL DEFAULT '',
        created_by varchar(160) NOT NULL DEFAULT '',
        created_at timestamptz NOT NULL DEFAULT now()
    );
`);
await db.execute(sql`
    CREATE TABLE IF NOT EXISTS homes (
        id serial PRIMARY KEY,

        user_id integer NOT NULL,
        rep_name varchar(160) NOT NULL,
        title varchar(160) NOT NULL DEFAULT '',
        address text NOT NULL DEFAULT '',
        lat double precision NOT NULL,
        lng double precision NOT NULL,
        accuracy double precision,
        updated_at timestamptz NOT NULL DEFAULT now()
    );
`);
await db.execute(sql`
    CREATE TABLE IF NOT EXISTS leaves (
        id serial PRIMARY KEY,
        user_id integer NOT NULL,
        rep_name varchar(160) NOT NULL,
        kind varchar(30) NOT NULL DEFAULT '',
        from_date varchar(12) NOT NULL,
        to_date varchar(12) NOT NULL,
        days integer NOT NULL DEFAULT 1,
        from_time varchar(5) NOT NULL DEFAULT '',
        to_time varchar(5) NOT NULL DEFAULT '',
        hours double precision NOT NULL DEFAULT 0,
        reason text NOT NULL DEFAULT '',
        supervisor_status varchar(20) NOT NULL DEFAULT 'pending',
        supervisor_note varchar(300) NOT NULL DEFAULT '',
        supervisor_name varchar(160) NOT NULL DEFAULT '',
        manager_status varchar(20) NOT NULL DEFAULT 'pending',
        manager_note varchar(300) NOT NULL DEFAULT '',
        manager_name varchar(160) NOT NULL DEFAULT '',
        created_at timestamptz NOT NULL DEFAULT now()
    );
`);
await db.execute(sql`
    CREATE TABLE IF NOT EXISTS trips (
        id serial PRIMARY KEY,
        user_id integer NOT NULL,
        rep_name varchar(160) NOT NULL,
        date_shamsi varchar(12) NOT NULL,
        status varchar(20) NOT NULL DEFAULT 'active',
        started_at timestamptz NOT NULL DEFAULT now(),
        ended_at timestamptz
    );
`);
await db.execute(sql`
    CREATE TABLE IF NOT EXISTS live_locations (
        user_id integer PRIMARY KEY,
        rep_name varchar(160) NOT NULL DEFAULT '',
        lat double precision NOT NULL,
        lng double precision NOT NULL,
        accuracy double precision,
        speed double precision,
        heading double precision,
        battery integer,
        gps_on boolean NOT NULL DEFAULT true,
        online boolean NOT NULL DEFAULT true,
        trip_id integer,
        recorded_at timestamptz NOT NULL DEFAULT now(),
        updated_at timestamptz NOT NULL DEFAULT now()
    );
`);
await db.execute(sql`
    CREATE TABLE IF NOT EXISTS trip_points (
        id serial PRIMARY KEY,
        trip_id integer NOT NULL,
        lat double precision NOT NULL,
        lng double precision NOT NULL,
        accuracy double precision,
        kind varchar(20) NOT NULL DEFAULT 'move',
        note varchar(200) NOT NULL DEFAULT '',
        recorded_at timestamptz NOT NULL DEFAULT now(),
        synced boolean NOT NULL DEFAULT true
    );
`);
await db.execute(sql`
    CREATE TABLE IF NOT EXISTS messengers (
        id serial PRIMARY KEY,

```

```

platform varchar(20) NOT NULL,
label varchar(160) NOT NULL DEFAULT '',
target_type varchar(20) NOT NULL DEFAULT 'phone',
target text NOT NULL DEFAULT '',
token text NOT NULL DEFAULT '',
provider varchar(30) NOT NULL DEFAULT '',
api_url text NOT NULL DEFAULT '',
enabled boolean NOT NULL DEFAULT true,
last_status text NOT NULL DEFAULT '',

last_ok_at timestampz,
last_error_at timestampz,
sort_order integer NOT NULL DEFAULT 0,
created_at timestampz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`
CREATE TABLE IF NOT EXISTS message_logs (
    id serial PRIMARY KEY,
    order_id integer,
    platform varchar(20) NOT NULL,
    target varchar(200) NOT NULL DEFAULT '',
    ok boolean NOT NULL DEFAULT false,
    detail text NOT NULL DEFAULT '',
    created_at timestampz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`
CREATE TABLE IF NOT EXISTS activity_logs (
    id serial PRIMARY KEY,
    user_id integer,
    user_name varchar(160) NOT NULL DEFAULT '',
    action varchar(80) NOT NULL,
    detail text NOT NULL DEFAULT '',
    created_at timestampz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`
CREATE TABLE IF NOT EXISTS notifications (
    id serial PRIMARY KEY,
    to_user_id integer,
    to_role varchar(20) NOT NULL DEFAULT '',
    from_name varchar(160) NOT NULL DEFAULT '',
    kind varchar(40) NOT NULL DEFAULT 'info',
    title varchar(200) NOT NULL,
    body text NOT NULL DEFAULT '',
    link varchar(200) NOT NULL DEFAULT '',
    read_at timestampz,
    created_at timestampz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`
CREATE TABLE IF NOT EXISTS attachments (
    id serial PRIMARY KEY,
    owner_type varchar(30) NOT NULL DEFAULT 'doctor',
    owner_id integer,
    user_id integer NOT NULL,
    rep_name varchar(160) NOT NULL DEFAULT '',
    file_name varchar(200) NOT NULL DEFAULT 'file',
    mime_type varchar(100) NOT NULL DEFAULT 'application/octet-stream',
    size integer NOT NULL DEFAULT 0,
    data text NOT NULL,
    created_at timestampz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`
CREATE TABLE IF NOT EXISTS targets (
    id serial PRIMARY KEY,
    user_id integer NOT NULL,
    rep_name varchar(160) NOT NULL DEFAULT '',
    period varchar(7) NOT NULL DEFAULT '',
    product_key varchar(60) NOT NULL,
    product_label varchar(160) NOT NULL DEFAULT '',
    quantity integer NOT NULL DEFAULT 0,
    price_distributor double precision NOT NULL DEFAULT 0,
    price_pharmacy double precision NOT NULL DEFAULT 0,
    created_at timestampz NOT NULL DEFAULT now(),
    updated_at timestampz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`
CREATE UNIQUE INDEX IF NOT EXISTS targets_unique_idx ON targets (user_id, period, product_key);
`);
await db.execute(sql`
CREATE TABLE IF NOT EXISTS mobile_nonces (

```

```

    id serial PRIMARY KEY,
    nonce varchar(64) NOT NULL UNIQUE,
    device_id varchar(80) NOT NULL DEFAULT '',
    used boolean NOT NULL DEFAULT false,
    expires_at timestampz NOT NULL,
    created_at timestampz NOT NULL DEFAULT now()
);
);
await db.execute(sql`

```

```

CREATE TABLE IF NOT EXISTS otp_codes (
    id serial PRIMARY KEY,
    user_id integer NOT NULL,
    phone varchar(20) NOT NULL,
    code_hash varchar(128) NOT NULL,
    device_id varchar(80) NOT NULL DEFAULT '',
    channel varchar(20) NOT NULL DEFAULT 'sms',
    attempts integer NOT NULL DEFAULT 0,
    used boolean NOT NULL DEFAULT false,
    expires_at timestampz NOT NULL,
    created_at timestampz NOT NULL DEFAULT now()
);
);
await db.execute(sql`

```

```

CREATE TABLE IF NOT EXISTS roles (
    id serial PRIMARY KEY,
    key varchar(40) NOT NULL UNIQUE,
    label varchar(120) NOT NULL,
    base varchar(20) NOT NULL DEFAULT 'rep',
    permissions jsonb NOT NULL DEFAULT '[]::jsonb',
    builtin boolean NOT NULL DEFAULT false,
    created_at timestampz NOT NULL DEFAULT now()
);
);
await db.execute(sql`

```

```

CREATE TABLE IF NOT EXISTS settings (
    key varchar(80) PRIMARY KEY,
    value jsonb NOT NULL,
    updated_at timestampz NOT NULL DEFAULT now()
);
);
// ---- incremental columns for databases created by older versions ----

```

```

const alters = [
    `ALTER TABLE users ADD COLUMN IF NOT EXISTS password_plain varchar(120) NOT NULL DEFAULT '',`,
    `ALTER TABLE users ADD COLUMN IF NOT EXISTS require_phone boolean NOT NULL DEFAULT true`,
    `ALTER TABLE users ADD COLUMN IF NOT EXISTS device_id varchar(80) NOT NULL DEFAULT '',`,
    `ALTER TABLE users ADD COLUMN IF NOT EXISTS device_info varchar(200) NOT NULL DEFAULT '',`,
    `ALTER TABLE users ADD COLUMN IF NOT EXISTS device_bound_at timestampz`,
    `ALTER TABLE users ADD COLUMN IF NOT EXISTS sim_mode varchar(12) NOT NULL DEFAULT 'device',`,
    `ALTER TABLE users ADD COLUMN IF NOT EXISTS sim_fingerprint varchar(128) NOT NULL DEFAULT '',`,
    `ALTER TABLE users ADD COLUMN IF NOT EXISTS sim_carrier varchar(80) NOT NULL DEFAULT '',`,
    `ALTER TABLE users ADD COLUMN IF NOT EXISTS sim_verified_at timestampz`,
    `ALTER TABLE pharmacies ADD COLUMN IF NOT EXISTS province varchar(100) NOT NULL DEFAULT '',`,
    `ALTER TABLE pharmacies ADD COLUMN IF NOT EXISTS city varchar(100) NOT NULL DEFAULT '',`,
    `ALTER TABLE pharmacies ADD COLUMN IF NOT EXISTS region varchar(100) NOT NULL DEFAULT '',`,
    `ALTER TABLE pharmacies ADD COLUMN IF NOT EXISTS landline varchar(20) NOT NULL DEFAULT '',`,
    `ALTER TABLE pharmacies ADD COLUMN IF NOT EXISTS accuracy double precision`,
    `ALTER TABLE doctors ADD COLUMN IF NOT EXISTS province varchar(100) NOT NULL DEFAULT '',`,
    `ALTER TABLE doctors ADD COLUMN IF NOT EXISTS city varchar(100) NOT NULL DEFAULT '',`,
    `ALTER TABLE doctors ADD COLUMN IF NOT EXISTS region varchar(100) NOT NULL DEFAULT '',`,
    `ALTER TABLE doctors ADD COLUMN IF NOT EXISTS accuracy double precision`,
    `ALTER TABLE orders ADD COLUMN IF NOT EXISTS accuracy double precision`,
    `ALTER TABLE options ADD COLUMN IF NOT EXISTS created_by varchar(160) NOT NULL DEFAULT '',`,
    `ALTER TABLE options ADD COLUMN IF NOT EXISTS parent varchar(200) NOT NULL DEFAULT '',`,
    `ALTER TABLE messengers ADD COLUMN IF NOT EXISTS provider varchar(30) NOT NULL DEFAULT '',`,
    `ALTER TABLE messengers ADD COLUMN IF NOT EXISTS api_url text NOT NULL DEFAULT '',`,
    `ALTER TABLE messengers ADD COLUMN IF NOT EXISTS last_status text NOT NULL DEFAULT '',`,
    `ALTER TABLE messengers ADD COLUMN IF NOT EXISTS last_ok_at timestampz`,
    `ALTER TABLE messengers ADD COLUMN IF NOT EXISTS last_error_at timestampz`,
    `ALTER TABLE messengers ADD COLUMN IF NOT EXISTS sort_order integer NOT NULL DEFAULT 0`,
    `ALTER TABLE messengers ALTER COLUMN target TYPE text`,
    `ALTER TABLE trips ADD COLUMN IF NOT EXISTS distance_m double precision NOT NULL DEFAULT 0`,
    `ALTER TABLE trips ADD COLUMN IF NOT EXISTS stop_seconds integer NOT NULL DEFAULT 0`,
    `ALTER TABLE trip_points ADD COLUMN IF NOT EXISTS speed double precision`,
    `ALTER TABLE trip_points ADD COLUMN IF NOT EXISTS stop_seconds integer NOT NULL DEFAULT 0`,
    `ALTER TABLE trip_points ADD COLUMN IF NOT EXISTS gps_on boolean NOT NULL DEFAULT true`,
    `ALTER TABLE pharmacies ADD COLUMN IF NOT EXISTS is_percent boolean NOT NULL DEFAULT false`,
    `ALTER TABLE pharmacies ADD COLUMN IF NOT EXISTS percent_value varchar(40) NOT NULL DEFAULT '',`,
    `ALTER TABLE doctors ADD COLUMN IF NOT EXISTS is_percent boolean NOT NULL DEFAULT false`,
    `ALTER TABLE doctors ADD COLUMN IF NOT EXISTS percent_value varchar(40) NOT NULL DEFAULT '',`,
    `ALTER TABLE leaves ADD COLUMN IF NOT EXISTS from_time varchar(5) NOT NULL DEFAULT '',`,
    `ALTER TABLE leaves ADD COLUMN IF NOT EXISTS to_time varchar(5) NOT NULL DEFAULT '',`,
    `ALTER TABLE leaves ADD COLUMN IF NOT EXISTS hours double precision NOT NULL DEFAULT 0`,
    `ALTER TABLE pharmacies ALTER COLUMN percent_value TYPE text`,
    `ALTER TABLE doctors ALTER COLUMN percent_value TYPE text`,

```

```

`ALTER TABLE pharmacies ALTER COLUMN location_label TYPE text`,
`ALTER TABLE doctors ALTER COLUMN location_label TYPE text`,
`ALTER TABLE orders ALTER COLUMN location_label TYPE text`,
`ALTER TABLE pharmacies ALTER COLUMN name TYPE text`,
`ALTER TABLE doctors ALTER COLUMN name TYPE text`,
`ALTER TABLE orders ALTER COLUMN pharmacy_name TYPE text`,
`ALTER TABLE leaves ALTER COLUMN supervisor_note TYPE text`,
`ALTER TABLE leaves ALTER COLUMN manager_note TYPE text`,
];

for (const a of alters) {
  try {
    await db.execute(sql.raw(a));
  } catch {
    /* ignore */
  }
}
await db.execute(sql`CREATE INDEX IF NOT EXISTS trip_points_trip_idx ON trip_points (trip_id)`);
await db.execute(sql`CREATE INDEX IF NOT EXISTS orders_user_idx ON orders (user_id)`);
await db.execute(sql`CREATE INDEX IF NOT EXISTS pharmacies_user_idx ON pharmacies (user_id)`);
await db.execute(sql`CREATE INDEX IF NOT EXISTS doctors_user_idx ON doctors (user_id)`);
await db.execute(sql`CREATE INDEX IF NOT EXISTS activity_user_idx ON activity_logs (user_id)`);
await db.execute(sql`
  CREATE TABLE IF NOT EXISTS sync_state (
    id serial PRIMARY KEY,
    peer varchar(80) NOT NULL UNIQUE,
    peer_url text NOT NULL DEFAULT '',
    enabled boolean NOT NULL DEFAULT true,
    pull_cursor timestampz,
    push_cursor timestampz,
    last_sync_at timestampz,
    last_status text NOT NULL DEFAULT '',
    last_error text NOT NULL DEFAULT '',
    pulled integer NOT NULL DEFAULT 0,
    pushed integer NOT NULL DEFAULT 0,
    conflicts integer NOT NULL DEFAULT 0,
    created_at timestampz NOT NULL DEFAULT now()
  );
`);
await db.execute(sql`
  CREATE TABLE IF NOT EXISTS sync_logs (
    id serial PRIMARY KEY,
    peer varchar(80) NOT NULL DEFAULT '',
    direction varchar(10) NOT NULL DEFAULT 'pull',
    table_name varchar(40) NOT NULL DEFAULT '',
    applied integer NOT NULL DEFAULT 0,
    skipped integer NOT NULL DEFAULT 0,
    ok boolean NOT NULL DEFAULT true,
    detail text NOT NULL DEFAULT '',
    created_at timestampz NOT NULL DEFAULT now()
  );
`);
await addSyncColumns();
await db.execute(sql`CREATE INDEX IF NOT EXISTS notif_user_idx ON notifications (to_user_id)`);
await db.execute(sql`CREATE INDEX IF NOT EXISTS attach_owner_idx ON attachments (owner_type, owner_id)`);
}
/**
 * (uid / updated_at / origin)
 * updated_at UPDATE.
 */
async function addSyncColumns() {
  const node = nodeName();
  // — UPDATE
  //
  //( « » )
  await db.execute(sql`
    CREATE OR REPLACE FUNCTION sek_touch_updated_at() RETURNS trigger AS $$
    BEGIN
      IF coalesce(current_setting('sek.sync', true), '') <> 'on' THEN
        NEW.updated_at := now();
      END IF;
      IF NEW.uid IS NULL OR NEW.uid = '' THEN
        NEW.uid := gen_random_uuid()::text;
      END IF;
      RETURN NEW;
    END;
    $$ LANGUAGE plpgsql;
  `);
  for (const t of SYNC_TABLES) {
    try {
      await db.execute(sql.raw(`ALTER TABLE ${t.name} ADD COLUMN IF NOT EXISTS uid varchar(40)`));
      await db.execute(
        sql.raw(`ALTER TABLE ${t.name} ADD COLUMN IF NOT EXISTS updated_at timestampz NOT NULL DEFAULT now()`),
      );
      await db.execute(

```

```

    sql.raw(`ALTER TABLE ${t.name} ADD COLUMN IF NOT EXISTS origin varchar(20) NOT NULL DEFAULT '${node}'`),
    );
    // uid
    await db.execute(

        sql.raw(`UPDATE ${t.name} SET uid = gen_random_uuid()::text WHERE uid IS NULL OR uid = ''`),
    );
    await db.execute(
        sql.raw(`ALTER TABLE ${t.name} ALTER COLUMN uid SET DEFAULT gen_random_uuid()::text`),
    );
    await db.execute(
        sql.raw(`CREATE UNIQUE INDEX IF NOT EXISTS ${t.name}_uid_idx ON ${t.name} (uid)`),
    );
    await db.execute(
        sql.raw(`CREATE INDEX IF NOT EXISTS ${t.name}_updated_idx ON ${t.name} (updated_at)`),
    );
    await db.execute(sql.raw(`DROP TRIGGER IF EXISTS ${t.name}_touch ON ${t.name}`));
    await db.execute(
        sql.raw(
            `CREATE TRIGGER ${t.name}_touch BEFORE INSERT OR UPDATE ON ${t.name}
            FOR EACH ROW EXECUTE FUNCTION sek_touch_updated_at()`),
    );
    );
} catch (err) {
    console.warn(`      ${t.name} :`, err instanceof Error ? err.message : err);
}
}
}

async function seed() {
    const admin = await db.select().from(users).where(eq(users.role, "admin")).limit(1);
    if (admin.length === 0) {
        const adminPass = process.env.ADMIN_PASSWORD || "admin1234";
        await db.insert(users).values([
            {
                username: "admin",
                passwordHash: hashPassword(adminPass),
                passwordPlain: adminPass,
                fullName: " ",
                phone: process.env.ADMIN_PHONE || "09120000000",
                role: "admin",
                permissions: ALL_PERMISSION_KEYS,
            },
            {
                username: "sarparast",
                passwordHash: hashPassword("sar1234"),
                passwordPlain: "sar1234",
                fullName: " ",
                phone: "09122222222",
                role: "supervisor",
                permissions: SUPERVISOR_DEFAULT_PERMISSIONS,
            },
            {
                username: "rep1",
                passwordHash: hashPassword("rep1234"),
                passwordPlain: "rep1234",
                fullName: " ",
                phone: "09121111111",
                role: "rep",
                permissions: REP_DEFAULT_PERMISSIONS,
            },
        ]);
    }
    const rl = await db.select().from(roles).limit(1);
    if (rl.length === 0) {
        await db.insert(roles).values([
            { key: "admin", label: " ", base: "admin", permissions: ALL_PERMISSION_KEYS, builtin: true },
            { key: "supervisor", label: "", base: "supervisor", permissions: SUPERVISOR_DEFAULT_PERMISSIONS, builtin: true },
            { key: "rep", label: " ", base: "rep", permissions: REP_DEFAULT_PERMISSIONS, builtin: true },
            { key: "sales", label: " ", base: "rep", permissions: REP_DEFAULT_PERMISSIONS, builtin: false },
        ]);
    }
    const st = await db.select().from(settings).limit(1);
    if (st.length === 0) {
        await db.insert(settings).values([
            {
                key: "messagingProxy",
                value: {
                    url: process.env.MESSAGING_PROXY_URL || "https://namayandee1mi-javad.javadalamar-mehraeen.workers.dev/",
                    enabled: true,
                    secret: "",
                },
            },
            {
                key: "messengerTokens",
            },
        ]);
    }
}

```

```

    value: {
      bale: process.env.BALE_BOT_TOKEN || "1199464939:uhHpJVtcy__qdtFfN7iuzr4AH7bZBKPg85A",
      telegram: process.env.TELEGRAM_BOT_TOKEN || "",

      eita: process.env.EITAA_TOKEN || "",
      whatsapp: process.env.WHATSAPP_TOKEN || "",
    },
  },
  { key: "products", value: DEFAULT_PRODUCTS },
  { key: "columns.pharmacies", value: DEFAULT_COLUMNS.pharmacies },
  { key: "columns.doctors", value: DEFAULT_COLUMNS.doctors },
  { key: "columns.orders", value: DEFAULT_COLUMNS.orders },
  { key: "fields.pharmacies", value: DEFAULT_FORM_FIELDS.pharmacies },
  { key: "fields.doctors", value: DEFAULT_FORM_FIELDS.doctors },
  { key: "fields.orders", value: DEFAULT_FORM_FIELDS.orders },
]);
}
// ( )
try {
  await db.execute(sql`
    UPDATE users SET permissions = ${JSON.stringify(ALL_PERMISSION_KEYS)}::jsonb
    WHERE role = 'admin' AND (permissions IS NULL OR jsonb_array_length(permissions) < ${ALL_PERMISSION_KEYS.length})
  `);
  await db.execute(sql`
    UPDATE users SET permissions = ${JSON.stringify(REP_DEFAULT_PERMISSIONS)}::jsonb
    WHERE role = 'rep' AND (permissions IS NULL OR jsonb_array_length(permissions) = 0)
  `);
} catch {
  /* ignore */
}
// ---- ( ) ----
const current = await db.select({ category: options.category, value: options.value, parent: options.parent }).from(options);
const seen = new Set(current.map((r) => `${r.category}|${r.value}|${r.parent ?? ""}`));
const wanted: { category: string; value: string; parent: string }[] = [];
for (const v of DEFAULT_SPECIALTIES) wanted.push({ category: "specialty", value: v, parent: "" });
for (const p of PROVINCES) {
  wanted.push({ category: "province", value: p, parent: "" });
  for (const c of IRAN[p]) {
    wanted.push({ category: "city", value: c, parent: p });
    for (const r of REGIONS[c] ?? []) wanted.push({ category: "region", value: r, parent: c });
  }
}
if (!seen.has("distributor|")) wanted.push({ category: "distributor", value: "", parent: "" });
if (!seen.has("distributor|")) wanted.push({ category: "distributor", value: "", parent: "" });
if (!seen.has("visitor|")) wanted.push({ category: "visitor", value: "", parent: "" });
const missing = wanted.filter((w) => !seen.has(`${w.category}|${w.value}|${w.parent}`));
if (missing.length > 0) {
  for (let i = 0; i < missing.length; i += 300) {
    await db.insert(options).values(missing.slice(i, i + 300));
  }
  console.log(` ${missing.length} (///) `);
}
//
// ( ) .
try {
  await db.execute(sql`DELETE FROM options WHERE category IN ('city','region') AND (parent IS NULL OR parent = '')`);
} catch {
  /* ignore */
}
}
export async function ensureSeed() {
  if (done) return;
  if (running) return running;
  running = (async () => {
    try {
      // — Neon
      await dbRetry(() => migrate(), "bootstrap:migrate");
      await dbRetry(() => seed(), "bootstrap:seed");
      done = true;
    } catch (err) {
      console.error("bootstrap error", err);
    } finally {
      running = null;
    }
  })();
  return running;
}

@@ src/lib/constants.ts typescript
export type ProductDef = { key: string; label: string; bonusKey: string; bonusLabel: string };
export const PRODUCTS: ProductDef[] = [
  { key: "omega3", label: "", bonusKey: "omega3_bonus", bonusLabel: "" },

  { key: "omega5", label: "", bonusKey: "omega5_bonus", bonusLabel: "" },

```

```

    { key: "omega35", label: " .", bonusKey: "omega35_bonus", bonusLabel: " .", },
    { key: "omegaMulti", label: " ", bonusKey: "omegaMulti_bonus", bonusLabel: " ", },
    { key: "omegaWomen", label: " ", bonusKey: "omegaWomen_bonus", bonusLabel: " ", },
    { key: "omegaMen", label: " ", bonusKey: "omegaMen_bonus", bonusLabel: " ", },
    { key: "melatonin", label: " ", bonusKey: "melatonin_bonus", bonusLabel: " ", },
  ];
export const PERMISSIONS: { key: string; label: string }[] = [
  { key: "dashboard", label: " " },
  { key: "pharmacy", label: " " },
  { key: "doctor", label: " " },
  { key: "order", label: " " },
  { key: "trip", label: " " },
  { key: "home", label: " " },
  { key: "leave", label: " " },
  { key: "options", label: " " },
  { key: "reports", label: " " },
];
export const ALL_PERMISSIONS = PERMISSIONS.map((p) => p.key);
export const OPTION_CATEGORIES: { key: string; label: string }[] = [
  { key: "province", label: "" },
  { key: "city", label: "" },
  { key: "region", label: "" },
  { key: "specialty", label: " " },
  { key: "pharmacy", label: " " },
  { key: "manager", label: " " },
  { key: "doctor", label: " " },
  { key: "secretary", label: " " },
  { key: "leaveKind", label: " " },
  { key: "year", label: " " },
  { key: "distributor", label: " " },
  { key: "visitor", label: " " },
];
export const PLATFORMS: {
  key: string;
  label: string;
  icon: string;
  needsToken: boolean;
  tokenHint: string;
  targetHint: string;
  guide: string[];
}[] = [
  {
    key: "telegram",
    label: "",
    icon: "",
    needsToken: true,
    tokenHint: " @BotFather ( 123456:ABC-DEF...)",
    targetHint: "chat_id -1001234567890",
    guide: [
      " @BotFather /newbot .",
      " .",
      " .",
      " « chat_id» api.telegram.org/bot<TOKEN>/getUpdates .",
    ],
  },
  {
    key: "bale",
    label: "",
    icon: "",
    needsToken: true,
    tokenHint: " @BotFather ",
    targetHint: "chat_id ",
    guide: [
      " @BotFather /newbot .",
      " .",
      " .",
      " « chat_id» .",
    ],
  },
  {
    key: "eita",
    label: "",
    icon: "",
    needsToken: true,
    tokenHint: " eitaayar.ir",
    targetHint: " @ chat_id",
    guide: [
      " eitaayar.ir .",
      " «» .",
      " / .",
      " @ .",
    ],
  },
];

```

```
key: "whatsapp",
label: "",
icon: "",
needsToken: true,
tokenHint: " API (whatsiplus / UltraMsg / Cloud API)",
targetHint: " whatsiplus ",
guide: [
    " whatsiplus.ir .",
    " API .",
    " «» .",
    " «» ."
],
},
];
export const DEFAULT_SPECIALTIES = [
    "",
    "",
    " ",
    " ",
    "",
    "",
    " ",
    " ",
    "",
    "",
    "",
    "",
    " ",
    " ",
    "",
    "",
    ""
];
export const DEFAULT_PROVINCES = [
    "",
    "",
    "",
    "",
    " ",
    " ",
    " ",
    "",
    "",
    "",
    "",
    "",
    "",
    "",
    "",
    ""
];
};
@@ src/lib/defaults.ts typescript
import { PRODUCTS } from "../constants";
export type ProductConfig = { key: string; label: string; bonusLabel: string; enabled: boolean };
export type ColumnConfig = { key: string; label: string; visible: boolean };
export type FormFieldConfig = { key: string; label: string; visible: boolean };
export const DEFAULT_PRODUCTS: ProductConfig[] = PRODUCTS.map((p) => ({
    key: p.key,
    label: p.label,
    bonusLabel: p.bonusLabel,
    enabled: true,
})));
/** */
export const bonusKeyOf = (key: string) => `${key}_bonus`;
export const DEFAULT_FORM_FIELDS: Record<string, FormFieldConfig[]> = {
    pharmacies: [
        { key: "dateShamsi", label: " ", visible: true },
        { key: "province", label: "", visible: true },
        { key: "city", label: "", visible: true },
        { key: "region", label: "", visible: true },
        { key: "name", label: " ", visible: true },
        { key: "landline", label: " ", visible: true },
        { key: "managerName", label: " ", visible: true },
        { key: "managerPhone", label: " ", visible: true },
        { key: "address", label: " ", visible: true },
        { key: "isPercent", label: " ", visible: true },
        { key: "files", label: " ", visible: true },
        { key: "location", label: " ", visible: true },
    ],
    doctors: [
        { key: "dateShamsi", label: " ", visible: true },
        { key: "province", label: "", visible: true },
        { key: "city", label: "", visible: true },
        { key: "region", label: "", visible: true },
        { key: "name", label: " ", visible: true },
        { key: "specialty", label: "", visible: true },
    ]
}
```



```

    { key: "phone", label: " ", visible: true },
    { key: "secretaryName", label: " ", visible: true },
    { key: "secretaryPhone", label: " ", visible: true },
    { key: "address", label: " ", visible: true },
    { key: "otherAddresses", label: " ", visible: true },
    { key: "isPercent", label: " ", visible: true },
    { key: "files", label: " ", visible: true },
    { key: "location", label: " ", visible: true },
  ],
  orders: [
    { key: "dateShamsi", label: " ", visible: true },
    { key: "pharmacyName", label: " ", visible: true },
    { key: "managerName", label: " ", visible: true },
    { key: "managerPhone", label: " ", visible: true },
    { key: "address", label: " ", visible: true },
    { key: "location", label: " ", visible: true },
    { key: "products", label: " ", visible: true },
    { key: "distributor", label: " ", visible: true },
    { key: "visitor", label: " ", visible: true },
    { key: "notes", label: " ", visible: true },
  ],
};
/**
export const AVAILABLE_FORM_FIELDS: Record<string, { key: string; label: string }[]> = Object.fromEntries(
  Object.entries(DEFAULT_FORM_FIELDS).map(([key, fields]) => [
    key,
    fields.map(({ key: fieldKey, label }) => ({ key: fieldKey, label })),
  ]),
);
export const DEFAULT_COLUMNS: Record<string, ColumnConfig[]> = {
  pharmacies: [
    { key: "row", label: "", visible: true },
    { key: "repName", label: " ", visible: true },
    { key: "dateShamsi", label: " ", visible: true },
    { key: "province", label: "", visible: true },
    { key: "city", label: "", visible: true },
    { key: "region", label: "", visible: false },
    { key: "name", label: " ", visible: true },
    { key: "landline", label: " ", visible: false },
    { key: "managerName", label: " ", visible: true },
    { key: "managerPhone", label: " ", visible: true },
    { key: "address", label: "", visible: false },
    { key: "isPercent", label: "", visible: true },
    { key: "location", label: "", visible: true },
    { key: "nav", label: "", visible: true },
    { key: "sent", label: " ", visible: true },
    { key: "actions", label: "", visible: true },
  ],
  doctors: [
    { key: "row", label: "", visible: true },
    { key: "repName", label: " ", visible: true },
    { key: "dateShamsi", label: " ", visible: true },
    { key: "province", label: "", visible: false },
    { key: "city", label: "", visible: true },
    { key: "region", label: "", visible: false },
    { key: "name", label: " ", visible: true },
    { key: "specialty", label: "", visible: true },
    { key: "phone", label: " ", visible: true },
    { key: "secretaryName", label: " ", visible: false },
    { key: "secretaryPhone", label: " ", visible: false },
    { key: "address", label: " ", visible: false },
    { key: "isPercent", label: "", visible: true },
    { key: "location", label: "", visible: true },
    { key: "nav", label: "", visible: true },
    { key: "sent", label: " ", visible: true },
    { key: "actions", label: "", visible: true },
  ],
  orders: [
    { key: "row", label: "", visible: true },
    { key: "repName", label: " ", visible: true },
    { key: "dateShamsi", label: " ", visible: true },
    { key: "pharmacyName", label: " ", visible: true },
    { key: "managerName", label: " ", visible: true },
    { key: "managerPhone", label: " ", visible: true },

    { key: "address", label: "", visible: false },
    { key: "products", label: " ", visible: true },
    { key: "distributor", label: " ", visible: true },
    { key: "visitor", label: " ", visible: true },
    { key: "notes", label: "", visible: false },
    { key: "location", label: "", visible: true },
    { key: "nav", label: "", visible: true },
    { key: "sent", label: " ", visible: true },
    { key: "actions", label: "", visible: true },
  ],
},

```

```

};
export const PERMISSION_GROUPS: {
  key: string;
  label: string;
  icon: string;
  items: { key: string; label: string }[];
}[] = [
  {
    key: "dashboard",
    label: "",
    icon: "",
    items: [
      { key: "dashboard", label: " " },
      { key: "dashboard.stats", label: " ( )" },
      { key: "dashboard.activity", label: " " },
      { key: "dashboard.allReps", label: " " },
      { key: "dashboard.export", label: " " },
    ],
  },
  {
    key: "pharmacy",
    label: " ",
    icon: "",
    items: [
      { key: "pharmacy", label: " " },
      { key: "pharmacy.create", label: " " },
      { key: "pharmacy.view", label: " " },
      { key: "pharmacy.viewAll", label: " " },
      { key: "pharmacy.detail", label: " " },
      { key: "pharmacy.location", label: " " },
      { key: "pharmacy.percent", label: "/" " },
      { key: "pharmacy.send", label: " " },
      { key: "pharmacy.export", label: " " },
      { key: "pharmacy.delete", label: " " },
    ],
  },
  {
    key: "doctor",
    label: " ",
    icon: "",
    items: [
      { key: "doctor", label: " " },
      { key: "doctor.create", label: " " },
      { key: "doctor.view", label: " " },
      { key: "doctor.viewAll", label: " " },
      { key: "doctor.detail", label: " " },
      { key: "doctor.location", label: " " },
      { key: "doctor.percent", label: "/" " },
      { key: "doctor.files", label: " " },
      { key: "doctor.filesView", label: "// " },
      { key: "doctor.send", label: " " },
      { key: "doctor.export", label: " " },
      { key: "doctor.delete", label: " " },
    ],
  },
  {
    key: "order",
    label: " ",
    icon: "",
    items: [
      { key: "order", label: " " },
      { key: "order.create", label: " " },
      { key: "order.view", label: " " },
      { key: "order.viewAll", label: " " },
      { key: "order.detail", label: " " },
      { key: "order.items", label: " " },
      { key: "order.location", label: " " },
      { key: "order.send", label: " ( )" },
      { key: "order.export", label: " " },
      { key: "order.delete", label: " " },
    ],
  },
  {
    key: "field",
    label: " ",
    icon: "",
    items: [
      { key: "trip", label: " " },
      { key: "trip.start", label: "/" " },
      { key: "trip.pause", label: " " },
      { key: "monitor", label: " " },
      { key: "home", label: " " },
      { key: "home.viewAll", label: " " },
    ],
  },

```

```

    },
    {
      key: "hr",
      label: " ",
      icon: "",
      items: [
        { key: "leave", label: " " },
        { key: "leave.viewAll", label: " " },
        { key: "leaveApprove", label: "/ " },
        { key: "leave.export", label: " " },
      ],
    },
    {
      key: "reports",
      label: "",
      icon: "",
      items: [
        { key: "reports", label: " " },
        { key: "reports.allReps", label: " " },
        { key: "reports.products", label: " " },
        { key: "export", label: " " },
      ],
    },
    {
      key: "admin",
      label: " ",
      icon: "",
      items: [
        { key: "options", label: " " },
        { key: "optionsDelete", label: " " },
        { key: "columns", label: " " },
        { key: "users", label: " " },
        { key: "users.password", label: " " },
        { key: "users.permissions", label: " " },
        { key: "messengers", label: " " },
        { key: "notifications.send", label: " " },
      ],
    },
  ],
};

export const ALL_PERMISSION_KEYS = PERMISSION_GROUPS.flatMap((g) => g.items.map((i) => i.key));
export const REP_DEFAULT_PERMISSIONS = [
  "pharmacy",
  "pharmacy.create",
  "pharmacy.view",
  "pharmacy.detail",
  "pharmacy.location",
  "pharmacy.percent",
  "pharmacy.send",
  "doctor",
  "doctor.create",
  "doctor.view",
  "doctor.detail",
  "doctor.location",
  "doctor.percent",
  "doctor.files",
  "doctor.filesView",
  "doctor.send",
  "order",
  "order.create",
  "order.view",
  "order.detail",
  "order.items",
  "order.location",
  "order.send",
  "trip",
  "trip.start",
  "trip.pause",
  "home",
  "leave",
  "reports",
  "reports.products",
  "options",
];

export const SUPERVISOR_DEFAULT_PERMISSIONS = [
  ...REP_DEFAULT_PERMISSIONS,
  "dashboard",
  "dashboard.stats",
  "dashboard.activity",
  "dashboard.allReps",
  "dashboard.export",
  "pharmacy.viewAll",
  "pharmacy.export",
  "doctor.viewAll",
  "doctor.export",
];

```

```

"order.viewAll",
"order.export",
"leave.viewAll",
"leaveApprove",
"leave.export",
"home.viewAll",
"monitor",
"reports.allReps",
"export",
];
/** — */
export const AVAILABLE_COLUMNS: Record<string, { key: string; label: string }[]> = {
  pharmacies: [
    { key: "row", label: "" },
    { key: "repName", label: " " },
    { key: "dateShamsi", label: " " },
    { key: "province", label: " " },
    { key: "city", label: " " },
    { key: "region", label: " " },
    { key: "name", label: " " },
    { key: "landline", label: " " },
    { key: "managerName", label: " " },
    { key: "managerPhone", label: " " },
    { key: "address", label: " " },
    { key: "isPercent", label: " " },
    { key: "percentValue", label: " " },
    { key: "location", label: " " },
    { key: "nav", label: " " },
    { key: "files", label: " " },
    { key: "sent", label: " " },
    { key: "createdAt", label: " " },
    { key: "actions", label: " (/)" },
  ],
  doctors: [
    { key: "row", label: "" },
    { key: "repName", label: " " },
    { key: "dateShamsi", label: " " },
    { key: "province", label: " " },
    { key: "city", label: " " },
    { key: "region", label: " " },
    { key: "name", label: " " },
    { key: "specialty", label: " " },
    { key: "phone", label: " " },
    { key: "secretaryName", label: " " },
    { key: "secretaryPhone", label: " " },
    { key: "address", label: " " },
    { key: "otherAddresses", label: " " },
    { key: "isPercent", label: " " },
    { key: "percentValue", label: " " },
    { key: "location", label: " " },
    { key: "nav", label: " " },
    { key: "files", label: " " },
    { key: "sent", label: " " },
    { key: "createdAt", label: " " },
    { key: "actions", label: " (/)" },
  ],
  orders: [
    { key: "row", label: "" },
    { key: "repName", label: " " },
    { key: "dateShamsi", label: " " },
    { key: "pharmacyName", label: " " },
    { key: "managerName", label: " " },
    { key: "managerPhone", label: " " },
    { key: "address", label: " " },
    { key: "products", label: " " },
    { key: "totalUnits", label: " " },
    { key: "totalBonus", label: " " },
    { key: "distributor", label: " " },
    { key: "visitor", label: " " },
    { key: "notes", label: " " },
    { key: "location", label: " " },
    { key: "nav", label: " " },

    { key: "sendStatus", label: " " },
    { key: "sent", label: " " },
    { key: "createdAt", label: " " },
    { key: "actions", label: " (/)" },
  ],
};

@@ src/lib/device.ts typescript
"use client";
export type SimStatus = {
  ok: boolean;
  cellular: boolean;

```

```

    online: boolean;
    reason: string;
    detail: string;
    type: string;
};
type Conn = {
    type?: string;
    effectiveType?: string;
    downlink?: number;
    rtt?: number;
    saveData?: boolean;
    addEventListener?: (e: string, cb: () => void) => void;
    removeEventListener?: (e: string, cb: () => void) => void;
};
function conn(): Conn | undefined {
    const n = navigator as Navigator & { connection?: Conn; mozConnection?: Conn; webkitConnection?: Conn };
    return n.connection ?? n.mozConnection ?? n.webkitConnection;
}
export function isMobileDevice() {
    return /Android|iPhone|iPad|iPod|Mobile|Opera Mini|IEMobile/i.test(navigator.userAgent);
}
/**
 * / .
 * (cellular / wifi) Network Information API .
 */
export function readSimStatus(): SimStatus {
    const online = typeof navigator !== "undefined" ? navigator.onLine : true;
    const c = conn();
    const type = c?.type ?? "unknown";
    const cellular = type === "cellular";
    if (!online) {
        return {
            ok: false,
            cellular: false,
            online: false,
            type,
            reason: "offline",
            detail: " . .",
        };
    }
    if (type === "none") {
        return {
            ok: false,
            cellular: false,
            online: true,
            type,
            reason: "no-network",
            detail: " .",
        };
    }
    return { ok: true, cellular, online: true, type, reason: "", detail: "" };
}
export function watchSim(cb: (s: SimStatus) => void) {
    const emit = () => cb(readSimStatus());
    emit();
    window.addEventListener("online", emit);
    window.addEventListener("offline", emit);
    const c = conn();
    c?.addEventListener?.("change", emit);
    return () => {
        window.removeEventListener("online", emit);
        window.removeEventListener("offline", emit);
        c?.removeEventListener?.("change", emit);
    };
}
const DEVICE_KEY = "sek_device_sim";
/** ( ) */
export function getDeviceSim(): string {
    try {
        return localStorage.getItem(DEVICE_KEY) ?? "";
    } catch {
        return "";
    }
}
export function setDeviceSim(phone: string) {
    try {
        localStorage.setItem(DEVICE_KEY, phone);
    } catch {
        /* ignore */
    }
}
export function normalizePhone(p: string) {
    return String(p ?? "")
        .replace(/[\u00F0-\u00F9]/g, (c) => String(c.charCodeAt(0) - 0x06f0))

```

```

        .replace(/\\D/g, "")
        .replace(/\\0098/, "0")
        .replace(/\\98/, "0");
    }
    const DEVICE_ID_KEY = "sek_device_id";
    /**      (    /) */
    export function getId(): string {
        try {
            let id = localStorage.getItem(DEVICE_ID_KEY);
            if (!id) {
                const rnd =
                    typeof crypto !== "undefined" && "randomUUID" in crypto
                        ? crypto.randomUUID()
                        : `${Date.now().toString(36)}-${Math.random().toString(36).slice(2, 12)}`;
                id = rnd.replace(/-/g, "").slice(0, 32);
                localStorage.setItem(DEVICE_ID_KEY, id);
            }
            return id;
        } catch {
            return "";
        }
    }
    /**      */
    export function getDeviceInfo(): string {
        if (typeof navigator === "undefined") return "";
        const ua = navigator.userAgent;
        const os = /Android/i.test(ua)
            ? ""
            : /iPhone|iPad|iPod/i.test(ua)
            ? "iOS"
            : /Windows/i.test(ua)
            ? ""
            : /Mac/i.test(ua)
            ? ""
            : "";
        const br = /Chrome/i.test(ua) ? "Chrome" : /Safari/i.test(ua) ? "Safari" : /Firefox/i.test(ua) ? "Firefox" : "";
        return `${os} — ${br}`.slice(0, 200);
    }
    /**      0912***1111      */
    export function maskPhone(p: string) {
        const d = normalizePhone(p);
        return d.length >= 8 ? `${d.slice(0, 4)}***${d.slice(-4)}` : "";
    }
}

```

```

@@ src/lib/download.ts typescript
"use client";
/**
 *      API.
 *
 *      <a href> `x-auth-token`
 *      « » .
 *      fetch (    ) .
 */
export async function downloadFile(path: string, fallbackName = "export.xls"): Promise<string> {
    try {
        const res = await fetch(path, { cache: "no-store", credentials: "include" });
        if (!res.ok) {
            const msg = await res
                .json()

                .then((d) => d?.error)
                .catch(() => null);
            return ` ${msg} ?? ` ${res.status} ` ` `;
        }
        //      Content-Disposition
        let name = fallbackName;
        const cd = res.headers.get("content-disposition") ?? "";
        const m = /filename\*=?(?:UTF-8')?"?([^\s;]+)"?/i.exec(cd);
        if (m) name = decodeURIComponent(m[1]);
        const blob = await res.blob();
        const url = URL.createObjectURL(blob);
        const a = document.createElement("a");
        a.href = url;
        a.download = name;
        document.body.appendChild(a);
        a.click();
        a.remove();
        setTimeout(() => URL.revokeObjectURL(url), 4000);
        return " ";
    } catch (err) {
        return ` ${err instanceof Error ? err.message : " " } `;
    }
}

```

```

@@ src/lib/endpoints.ts typescript

```

```

"use client";
/**
 * =====
 * (Multi-Endpoint Failover)
 * -----
 * :
 * • https://ndcohub.ir ( )
 * • https://namayandeelmi-javad.onrender.com ( )
 *
 * .
 *
 * .
 * =====
 */
const KEY_ACTIVE = "sek_active_endpoint";
const KEY_CHECKED = "sek_endpoint_checked_at";
const RECHECK_MS = 5 * 60 * 1000;
const PROBE_TIMEOUT = 6000;
/** ( ) */
export function endpointList(): string[] {
  const raw = process.env.NEXT_PUBLIC_ENDPOINTS || "";
  const list = raw
    .split(",")
    .map((s) => s.trim().replace(/\$/ /, ""))
    .filter((s) => s.startsWith("http"));
  if (list.length) return list;
  // :
  return ["https://ndcohub.ir", "https://namayandeelmi-javad.onrender.com"];
}
/** — */
export function currentOrigin(): string {
  if (typeof window === "undefined") return "";
  return window.location.origin.replace(/\$/ /, "");
}
export function getActiveEndpoint(): string {
  if (typeof window === "undefined") return "";
  try {
    return localStorage.getItem(KEY_ACTIVE) || currentOrigin();
  } catch {
    return currentOrigin();
  }
}
export function setActiveEndpoint(url: string) {
  try {
    localStorage.setItem(KEY_ACTIVE, url.replace(/\$/ /, ""));
    localStorage.setItem(KEY_CHECKED, String(Date.now()));
  } catch {
    /* ignore */
  }
}
/** : → → */

export function orderedEndpoints(): string[] {
  const origin = currentOrigin();
  const active = getActiveEndpoint();
  const all = endpointList();
  const seen = new Set<string>();
  const out: string[] = [];
  for (const u of [origin, active, ...all]) {
    const clean = (u || "").replace(/\$/ /, "");
    if (clean && !seen.has(clean)) {
      seen.add(clean);
      out.push(clean);
    }
  }
  return out;
}
async function probe(base: string): Promise<number | null> {
  const t0 = performance.now();
  try {
    const ctrl = new AbortController();
    const timer = setTimeout(() => ctrl.abort(), PROBE_TIMEOUT);
    const res = await fetch(`${base}/ping?t=${Date.now()}`, {
      cache: "no-store",
      signal: ctrl.signal,
      credentials: "omit",
      mode: base === currentOrigin() ? "same-origin" : "cors",
    });
    clearTimeout(timer);
    return res.ok ? Math.round(performance.now() - t0) : null;
  } catch {
    return null;
  }
}
/** */

```

```

export async function pickFastestEndpoint(force = false): Promise<{ url: string; ms: number | null }[]> {
    const checkedAt = Number(localStorage.getItem(KEY_CHECKED) ?? 0);
    const list = orderedEndpoints();
    if (!force && Date.now() - checkedAt < RECHECK_MS) {
        return [{ url: getActiveEndpoint(), ms: null }];
    }
    const results = await Promise.all(list.map(async (u) => ({ url: u, ms: await probe(u) })));
    const alive = results.filter((r) => r.ms !== null).sort((a, b) => (a.ms ?? 9e9) - (b.ms ?? 9e9));
    if (alive.length) setActiveEndpoint(alive[0].url);
    return results;
}
/**
 * :
 * «/api/...» .
 */
export async function multiFetch(path: string, init: RequestInit = {}, original?: typeof fetch): Promise<Response> {
    const doFetch = original ?? fetch;
    const endpoints = orderedEndpoints();
    let lastErr: unknown;
    for (let i = 0; i < endpoints.length; i++) {
        const base = endpoints[i];
        const isSelf = base === currentOrigin();
        const url = isSelf ? path : `${base}${path}`;
        try {
            const res = await doFetch(url, {
                ...init,
                // /CORS
                credentials: isSelf ? (init.credentials ?? "same-origin") : "include",
                mode: isSelf ? undefined : "cors",
            });
            //
            if (res.status < 500) {
                if (!isSelf) setActiveEndpoint(base);
                return res;
            }
            lastErr = new Error(`HTTP ${res.status}`);
        } catch (err) {
            lastErr = err;
        }
    }
    throw lastErr instanceof Error ? lastErr : new Error(" ");
}

@@ src/lib/excel.ts typescript
/**
 * Lightweight XLS (SpreadsheetML/HTML table) writer.
 * .xls Excel WPS
 * ( ).
 */
function esc(v: unknown) {
    return String(v ?? "")
        .replace(/&/g, "&")
        .replace(/</g, "&lt;")
        .replace(/>/g, "&gt;");
}
export function buildXls(sheetName: string, rows: (string | number)[][][]) {
    const head = rows[0] ?? [];
    const body = rows.slice(1);
    const html = `<html xmlns:o="urn:schemas-microsoft-com:office:office" xmlns:x="urn:schemas-microsoft-com:office:excel"
>
<head><meta charset="UTF-8" />
<!--[if gte mso 9]><xml><x:ExcelWorkbook><x:ExcelWorksheets><x:ExcelWorksheet>
<x:Name>${esc(sheetName)}</x:Name><x:WorksheetOptions><x:DisplayRightToLeft/></x:WorksheetOptions>
</x:ExcelWorksheet></x:ExcelWorksheets></x:ExcelWorkbook></xml><![endif]>-->
<style>
table{border-collapse:collapse;font-family:Tahoma,sans-serif;font-size:11pt}
th{background:#0f766e;color:#fff;border:1px solid #0b5c55;padding:6px;white-space:nowrap}
td{border:1px solid #cbd5e1;padding:5px;mso-number-format:"\\@"}
</style></head>
<body dir="rtl"><table><thead><tr>${head.map((h) => `<th>${esc(h)}</th>`).join("")}</tr></thead>
<tbody>${body
    .map((r) => `<tr>${r.map((c) => `<td>${esc(c)}</td>`).join("")}</tr>`)
    .join("")}</tbody></table></body></html>`;
    return "\\uFEFF" + html;
}
export function xlsResponse(fileName: string, sheetName: string, rows: (string | number)[][][]) {
    return new Response(buildXls(sheetName, rows), {
        headers: {
            "Content-Type": "application/vnd.ms-excel; charset=utf-8",
            "Content-Disposition": `attachment; filename="${esc(fileName)}.xls"`,
            "Cache-Control": "no-store",
        },
    });
}

```


@@ src/lib/geo.ts typescript

```
/**
 * =====
 * -----
 * :
 * • /
 * •
 * =====
 */
export type GeoArea = {
  /** */
  name: string;
  lat: number;
  lng: number;
  /** */
  zoom: number;
  /** */
  radiusKm: number;
};
/** GeoJSON → */
export const PROVINCE_EN_FA: Record<string, string> = {
  Tehran: "",
  Alborz: "",
  Isfahan: "",
  Fars: "",
  "Razavi Khorasan": " ",
  "East Azerbaijan": " ",
  "West Azerbaijan": " ",
  Khuzestan: "",
  Gilan: "",
  Mazandaran: "",
  Kerman: "",
  Qom: "",
  Yazd: "",
  Hamadan: "",
  Kermanshah: "",
  Golestan: "",
  Ardabil: "",
  Qazvin: "",
  Zanjan: "",
  Markazi: "",
  Lorestan: "",
  Kurdistan: "",

  Hormozgan: "",
  Bushehr: "",
  Semnan: "",
  "Sistan and Baluchestan": " ",
  "Chaharmahal and Bakhtiari": " ",
  "Kohgiluyeh and Boyer-Ahmad": " ",
  "North Khorasan": " ",
  "South Khorasan": " ",
  Ilam: "",
};
/** — */
export const IRAN_CENTER: GeoArea = {
  name: "",
  lat: 32.4279,
  lng: 53.688,
  zoom: 5,
  radiusKm: 900,
};
/** */
export const PROVINCE_GEO: Record<string, GeoArea> = {
  : { name: "", lat: 35.6892, lng: 51.389, zoom: 10, radiusKm: 70 },
  : { name: "", lat: 35.8327, lng: 50.9916, zoom: 10, radiusKm: 45 },
  : { name: "", lat: 32.6546, lng: 51.668, zoom: 8, radiusKm: 180 },
  : { name: "", lat: 29.5918, lng: 52.5837, zoom: 8, radiusKm: 200 },
  " ": { name: " ", lat: 36.2605, lng: 59.6168, zoom: 8, radiusKm: 200 },
  " ": { name: " ", lat: 38.0962, lng: 46.2738, zoom: 8, radiusKm: 130 },
  " ": { name: " ", lat: 37.5527, lng: 45.0761, zoom: 8, radiusKm: 140 },
  : { name: "", lat: 31.3183, lng: 48.6706, zoom: 8, radiusKm: 160 },
  : { name: "", lat: 37.2808, lng: 49.5832, zoom: 9, radiusKm: 80 },
  : { name: "", lat: 36.5659, lng: 53.06, zoom: 8, radiusKm: 130 },
  : { name: "", lat: 30.2839, lng: 57.0834, zoom: 7, radiusKm: 250 },
  : { name: "", lat: 34.6416, lng: 50.8746, zoom: 10, radiusKm: 50 },
  : { name: "", lat: 31.8974, lng: 54.3569, zoom: 8, radiusKm: 170 },
  : { name: "", lat: 34.7992, lng: 48.5146, zoom: 9, radiusKm: 90 },
  : { name: "", lat: 34.3142, lng: 47.065, zoom: 9, radiusKm: 100 },
  : { name: "", lat: 36.8427, lng: 54.4342, zoom: 9, radiusKm: 90 },
  : { name: "", lat: 38.2498, lng: 48.2933, zoom: 9, radiusKm: 90 },
  : { name: "", lat: 36.2688, lng: 50.0041, zoom: 9, radiusKm: 70 },
  : { name: "", lat: 36.6736, lng: 48.4787, zoom: 9, radiusKm: 90 },
```

```

: { name: "", lat: 34.0917, lng: 49.6892, zoom: 9, radiusKm: 100 },
: { name: "", lat: 33.4878, lng: 48.3558, zoom: 9, radiusKm: 100 },
: { name: "", lat: 35.3219, lng: 46.9862, zoom: 9, radiusKm: 100 },
: { name: "", lat: 27.1832, lng: 56.2666, zoom: 8, radiusKm: 200 },
: { name: "", lat: 28.9234, lng: 50.82, zoom: 8, radiusKm: 150 },
: { name: "", lat: 35.5729, lng: 53.3971, zoom: 8, radiusKm: 180 },
" ": { name: " ", lat: 29.4963, lng: 60.8629, zoom: 7, radiusKm: 300 },
" ": { name: " ", lat: 32.3266, lng: 50.8644, zoom: 9, radiusKm: 80 },
" ": { name: " ", lat: 30.6682, lng: 51.5876, zoom: 9, radiusKm: 80 },
" ": { name: " ", lat: 37.4747, lng: 57.329, zoom: 8, radiusKm: 110 },
" ": { name: " ", lat: 32.8649, lng: 59.2211, zoom: 7, radiusKm: 250 },
: { name: "", lat: 33.6374, lng: 46.4227, zoom: 9, radiusKm: 90 },
};
/** ( ) */
export const CITY_GEO: Record<string, GeoArea> = {
: { name: "", lat: 35.6892, lng: 51.389, zoom: 11, radiusKm: 25 },
: { name: "", lat: 36.2605, lng: 59.6168, zoom: 11, radiusKm: 20 },
: { name: "", lat: 32.6546, lng: 51.668, zoom: 11, radiusKm: 18 },
: { name: "", lat: 29.5918, lng: 52.5837, zoom: 11, radiusKm: 18 },
: { name: "", lat: 38.0962, lng: 46.2738, zoom: 11, radiusKm: 16 },
: { name: "", lat: 35.8327, lng: 50.9916, zoom: 11, radiusKm: 16 },
: { name: "", lat: 31.3183, lng: 48.6706, zoom: 11, radiusKm: 16 },
: { name: "", lat: 34.6416, lng: 50.8746, zoom: 11, radiusKm: 14 },
: { name: "", lat: 34.3142, lng: 47.065, zoom: 11, radiusKm: 12 },
: { name: "", lat: 37.2808, lng: 49.5832, zoom: 12, radiusKm: 10 },
: { name: "", lat: 37.5527, lng: 45.0761, zoom: 12, radiusKm: 10 },
: { name: "", lat: 30.2839, lng: 57.0834, zoom: 12, radiusKm: 12 },
: { name: "", lat: 29.4963, lng: 60.8629, zoom: 12, radiusKm: 10 },
: { name: "", lat: 31.8974, lng: 54.3569, zoom: 12, radiusKm: 10 },
: { name: "", lat: 38.2498, lng: 48.2933, zoom: 12, radiusKm: 10 },
: { name: "", lat: 27.1832, lng: 56.2666, zoom: 12, radiusKm: 10 },
: { name: "", lat: 36.5659, lng: 53.06, zoom: 12, radiusKm: 10 },
: { name: "", lat: 36.2688, lng: 50.0041, zoom: 12, radiusKm: 10 },
: { name: "", lat: 36.6736, lng: 48.4787, zoom: 12, radiusKm: 10 },
: { name: "", lat: 34.0917, lng: 49.6892, zoom: 12, radiusKm: 10 },
"": { name: "", lat: 33.4878, lng: 48.3558, zoom: 12, radiusKm: 10 },
: { name: "", lat: 35.3219, lng: 46.9862, zoom: 12, radiusKm: 10 },
: { name: "", lat: 28.9234, lng: 50.82, zoom: 12, radiusKm: 10 },
: { name: "", lat: 36.8427, lng: 54.4342, zoom: 12, radiusKm: 10 },
: { name: "", lat: 34.7992, lng: 48.5146, zoom: 12, radiusKm: 10 },
: { name: "", lat: 32.3266, lng: 50.8644, zoom: 12, radiusKm: 10 },
: { name: "", lat: 37.4747, lng: 57.329, zoom: 12, radiusKm: 10 },
: { name: "", lat: 32.8649, lng: 59.2211, zoom: 12, radiusKm: 10 },

: { name: "", lat: 33.6374, lng: 46.4227, zoom: 12, radiusKm: 10 },
: { name: "", lat: 30.6682, lng: 51.5876, zoom: 12, radiusKm: 10 },
: { name: "", lat: 35.5729, lng: 53.3971, zoom: 12, radiusKm: 10 },
};
/** */
export const TEHRAN_REGIONS: Record<string, GeoArea> = {
"1": { name: " ", lat: 35.8046, lng: 51.4324, zoom: 13, radiusKm: 6 },
"2": { name: " ", lat: 35.7576, lng: 51.3347, zoom: 13, radiusKm: 6 },
"3": { name: " ", lat: 35.7647, lng: 51.4059, zoom: 13, radiusKm: 5 },
"4": { name: " ", lat: 35.7614, lng: 51.5108, zoom: 13, radiusKm: 7 },
"5": { name: " ", lat: 35.7576, lng: 51.3018, zoom: 13, radiusKm: 6 },
"6": { name: " ", lat: 35.7219, lng: 51.4089, zoom: 13, radiusKm: 4 },
"7": { name: " ", lat: 35.7315, lng: 51.4462, zoom: 13, radiusKm: 4 },
"8": { name: " ", lat: 35.7275, lng: 51.4869, zoom: 13, radiusKm: 4 },
"9": { name: " ", lat: 35.6959, lng: 51.3162, zoom: 13, radiusKm: 4 },
"10": { name: " ", lat: 35.6892, lng: 51.3565, zoom: 13, radiusKm: 4 },
"11": { name: " ", lat: 35.6875, lng: 51.3889, zoom: 13, radiusKm: 4 },
"12": { name: " ", lat: 35.6836, lng: 51.4275, zoom: 13, radiusKm: 4 },
"13": { name: " ", lat: 35.7008, lng: 51.4831, zoom: 13, radiusKm: 4 },
"14": { name: " ", lat: 35.6708, lng: 51.4675, zoom: 13, radiusKm: 5 },
"15": { name: " ", lat: 35.6459, lng: 51.4675, zoom: 13, radiusKm: 6 },
"16": { name: " ", lat: 35.6398, lng: 51.4008, zoom: 13, radiusKm: 4 },
"17": { name: " ", lat: 35.6553, lng: 51.3555, zoom: 13, radiusKm: 4 },
"18": { name: " ", lat: 35.6553, lng: 51.3018, zoom: 13, radiusKm: 6 },
"19": { name: " ", lat: 35.6252, lng: 51.3555, zoom: 13, radiusKm: 5 },
"20": { name: " ", lat: 35.5892, lng: 51.4324, zoom: 13, radiusKm: 6 },
"21": { name: " ", lat: 35.6959, lng: 51.2261, zoom: 13, radiusKm: 7 },
"22": { name: " ", lat: 35.7576, lng: 51.2072, zoom: 13, radiusKm: 8 },
};
/** */
function norm(s: string) {
return String(s ?? "")
.replace(/[\u200c\u200f\u200e]/g, "")
.replace(/[/]/g, "")
.replace(/#/g, "")
.replace(/\\s+/g, " ")
.trim();
}
/** (« »)*/
function toEn(s: string) {
return s.replace(/[\u06F0-\u06F9]/g, (c) => String(c.charCodeAt(0) - 0x06f0));
}

```

```

}
/**
 *      / / .
 * (← ← ←).
 */
export function resolveArea(province?: string, city?: string, region?: string): GeoArea {
    const r = toEn(norm(region ?? ""));
    const c = norm(city ?? "");
    const p = norm(province ?? "");
    if (c === "" && r && TEHRAN_REGIONS[r]) return TEHRAN_REGIONS[r];
    if (c && CITY_GEO[c]) return CITY_GEO[c];
    if (p && PROVINCE_GEO[p]) return PROVINCE_GEO[p];
    return IRAN_CENTER;
}
/**
 */
export function distanceKm(a: { lat: number; lng: number }, b: { lat: number; lng: number }) {
    const R = 6371;
    const dLat = ((b.lat - a.lat) * Math.PI) / 180;
    const dLng = ((b.lng - a.lng) * Math.PI) / 180;
    const s =
        Math.sin(dLat / 2) ** 2 +
        Math.cos((a.lat * Math.PI) / 180) * Math.cos((b.lat * Math.PI) / 180) * Math.sin(dLng / 2) ** 2;
    return 2 * R * Math.asin(Math.sqrt(s));
}
/**
 */
export function provinceOf(point: { lat: number; lng: number }): string {
    let best = "";
    let bestD = Number.POSITIVE_INFINITY;
    for (const [name, g] of Object.entries(PROVINCE_GEO)) {
        const d = distanceKm(point, g);
        if (d < bestD) {
            bestD = d;
            best = name;
        }
    }
    return best;
}
}

export const PROVINCE_NAMES = Object.keys(PROVINCE_GEO);

```

@@ src/lib/iran.ts typescript

```

/**
 * —
 * ( )
 */
export const IRAN: Record<string, string[]> = {
    : [
        " ", " ", " ", " ", " ", " ", " ", " ", " ",
        " ", " ", " ", " ", " ", " ", " ", " ", " ",
        " ", " ", " ", " ", " ", " ", " ", " ", " ",
        " ", " ", " ", " ", " ", " ", " ", " ", " ",
    ],
    : [
        " ", " ", " ", " ", " ", " ", " ", " ", " ",
        " ", " ", " ", " ", " ", " ", " ", " ", " ",
    ],
    : [
        " ", " ", " ", " ", " ", " ", " ", " ", " ",
        " ", " ", " ", " ", " ", " ", " ", " ", " ",
        " ", " ", " ", " ", " ", " ", " ", " ", " ",
    ],
    : [
        " ", " ", " ", " ", " ", " ", " ", " ", " ",
        " ", " ", " ", " ", " ", " ", " ", " ", " ",
        " ", " ", " ", " ", " ", " ", " ", " ", " ",
    ],
    " ": [
        " ", " ", " ", " ", " ", " ", " ", " ", " ",
        " ", " ", " ", " ", " ", " ", " ", " ", " ",
        " ", " ", " ", " ", " ", " ", " ", " ", " ",
        " ", " ", " ", " ", " ", " ", " ", " ", " ",
    ],
    " ": [
        " ", " ", " ", " ", " ", " ", " ", " ", " ",
        " ", " ", " ", " ", " ", " ", " ", " ", " ",
        " ", " ", " ", " ", " ", " ", " ", " ", " ",
    ],
    " ": [
        " ", " ", " ", " ", " ", " ", " ", " ", " ",
        " ", " ", " ", " ", " ", " ", " ", " ", " ",
        " ", " ", " ", " ", " ", " ", " ", " ", " ",
    ],
    : [

```

[illegible]

```

    : ["", "", "", "", "", "", "", "", "", "", ""],
  ],
};
/** */
export const REGIONS: Record<string, string[]> = {
  : Array.from({ length: 22 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 13 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 15 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 11 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 10 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 12 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 8 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 8 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 8 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 5 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 5 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 4 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 5 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 4 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 3 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 4 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 2 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 2 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 3 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 4 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 5 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 3 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 4 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 3 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 3 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 3 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 2 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 2 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 2 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 2 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 2 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 3 }, (_, i) => ` ${i + 1} `),

  : Array.from({ length: 3 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 2 }, (_, i) => ` ${i + 1} `),
  : Array.from({ length: 2 }, (_, i) => ` ${i + 1} `),
};
export const PROVINCES = Object.keys(IRAN);
/** */
export const CITY_COUNT = Object.values(IRAN).reduce((a, c) => a + c.length, 0);

@@ src/lib/jalali.ts typescript
// Lightweight Jalali (Shamsi) date utilities - no external dependency.
export const JALALI_MONTHS = [
  "",
  "",
  "",
  "",
  "",
  "",
  "",
  "",
  "",
  "",
  "",
  "",
  "",
  "",
  "",
  "",
  "",
  "",
  "",
  "",
];
export const WEEK_DAYS = ["", "", "", "", "", "", ""];
function div(a: number, b: number) {
  return Math.floor(a / b);
}
export function gregorianToJalali(gy: number, gm: number, gd: number): [number, number, number] {
  const g_d_m = [0, 31, 59, 90, 120, 151, 181, 212, 243, 273, 304, 334];
  const gy2 = gm > 2 ? gy + 1 : gy;
  let days =
    355666 +
    365 * gy +
    div(gy2 + 3, 4) -
    div(gy2 + 99, 100) +
    div(gy2 + 399, 400) +
    gd +
    g_d_m[gm - 1];
  let jy = -1595 + 33 * div(days, 12053);
  days %= 12053;
  jy += 4 * div(days, 1461);
  days %= 1461;
  if (days > 365) {
    jy += div(days - 1, 365);
    days = (days - 1) % 365;
  }
}
```

```

const jm = days < 186 ? 1 + div(days, 31) : 7 + div(days - 186, 30);
const jd = 1 + (days < 186 ? days % 31 : (days - 186) % 30);
return [jy, jm, jd];
}
export function jalaliToGregorian(jy: number, jm: number, jd: number): [number, number, number] {
  jy += 1595;
  let days =
    -355668 + 365 * jy + div(jy, 33) * 8 + div((jy % 33) + 3, 4) + jd + (jm < 7 ? (jm - 1) * 31 : (jm - 7) * 30 + 186);
  let gy = 400 * div(days, 146097);
  days %= 146097;
  if (days > 36524) {
    gy += 100 * div(--days, 36524);
    days %= 36524;
    if (days >= 365) days++;
  }
  gy += 4 * div(days, 1461);
  days %= 1461;
  if (days > 365) {
    gy += div(days - 1, 365);
    days = (days - 1) % 365;
  }
  let gd = days + 1;
  const sal_a = [
    0,
    31,
    (gy % 4 === 0 && gy % 100 !== 0) || gy % 400 === 0 ? 29 : 28,
    31,
    30,
    31,
    30,
    31,
    30,
    31,

    31,
    30,
    31,
    30,
    31,
  ];
  let gm = 0;
  for (gm = 0; gm < 13 && gd > sal_a[gm]; gm++) gd -= sal_a[gm];
  return [gy, gm, gd];
}
export function isJalaliLeap(jy: number) {
  const r = jy % 33;
  return [1, 5, 9, 13, 17, 22, 26, 30].includes(r);
}
export function jalaliMonthLength(jy: number, jm: number) {
  if (jm <= 6) return 31;
  if (jm <= 11) return 30;
  return isJalaliLeap(jy) ? 30 : 29;
}
export function pad2(n: number) {
  return n < 10 ? `0${n}` : String(n);
}
export function formatJalali(jy: number, jm: number, jd: number) {
  return `${jy}/${pad2(jm)}/${pad2(jd)}`;
}
/** Today's date in Tehran timezone as `YYYY/MM/DD` shamsi string. */
export function todayJalali(): string {
  const now = new Date();
  const parts = new Intl.DateTimeFormat("en-CA", {
    timeZone: "Asia/Tehran",
    year: "numeric",
    month: "2-digit",
    day: "2-digit",
  }).format(now);
  const [gy, gm, gd] = parts.split("-").map(Number);
  const [jy, jm, jd] = gregorianToJalali(gy, gm, gd);
  return formatJalali(jy, jm, jd);
}
export function toEnglishDigits(input: string) {
  return input
    .replace(/[\u06F0-\u06F9]/g, (c) => String(c.charCodeAt(0) - 0x06f0))
    .replace(/[\u0660-\u0669]/g, (c) => String(c.charCodeAt(0) - 0x0660));
}
export function toPersianDigits(input: string | number) {
  return String(input).replace(/d/g, (d) => String.fromCharCode(0x06f0 + Number(d)));
}
/** Auto-mask typed digits into `YYYY/MM/DD`. */
export function maskJalaliInput(raw: string) {
  const digits = toEnglishDigits(raw).replace(/d/g, "").slice(0, 8);
  if (digits.length <= 4) return digits;
  if (digits.length <= 6) return `${digits.slice(0, 4)}/${digits.slice(4)}`;
  return `${digits.slice(0, 4)}/${digits.slice(4, 6)}/${digits.slice(6)}`;
}

```

```

export function parseJalali(value: string): [number, number, number] | null {
    const m = toEnglishDigits(value).match(/^(\\d{4})\\/(\\d{1,2})\\/(\\d{1,2})$/);
    if (!m) return null;
    const jy = Number(m[1]);
    const jm = Number(m[2]);
    const jd = Number(m[3]);
    if (jm < 1 || jm > 12) return null;
    if (jd < 1 || jd > jalaliMonthLength(jy, jm)) return null;
    return [jy, jm, jd];
}
export function isValidJalali(value: string) {
    return parseJalali(value) !== null;
}
/** Jalali weekday index (0 = ) for a jalali date */
export function jalaliWeekDay(jy: number, jm: number, jd: number) {
    const [gy, gm, gd] = jalaliToGregorian(jy, jm, jd);
    const day = new Date(Date.UTC(gy, gm - 1, gd)).getUTCDay(); // 0=Sunday
    return (day + 1) % 7;
}
export function tehranTime(date: Date | string | number) {
    const d = new Date(date);
    return new Intl.DateTimeFormat("fa-IR", {
        timeZone: "Asia/Tehran",
        hour: "2-digit",
        minute: "2-digit",
        second: "2-digit",
    }).format(d);
}
export function tehranDateTime(date: Date | string | number) {
    const d = new Date(date);
    const parts = new Intl.DateTimeFormat("en-CA", {
        timeZone: "Asia/Tehran",
        year: "numeric",
        month: "2-digit",
        day: "2-digit",
    }).format(d);
    const [gy, gm, gd] = parts.split("-").map(Number);
    const [jy, jm, jd] = gregorianToJalali(gy, gm, gd);
    return `${formatJalali(jy, jm, jd)} - ${tehranTime(d)}`;
}

@@ src/lib/messaging.ts typescript
import { db, dbRetrySafe } from "@db";
import { messageLogs, messengers, settings } from "@db/schema";
import { bonusKeyOf } from "../defaults";
import { getProducts } from "../settings-server";
import { toPersianDigits } from "../jalali";
import { fetchWithRetry } from "../retry";
import { eq } from "drizzle-orm";
/* =====
*
*
* : ( )
*
* :
* -
* - (Exponential Backoff)
* -
* - ( )
* - /
* ===== */
export type Platform = "telegram" | "bale" | "eitaa" | "whatsapp";
export type MessengerTarget = {
    id?: number;
    platform: string;
    target: string;
    token: string;
    provider?: string;
    apiUrl?: string;
    label?: string;
};
export type SendResult = { ok: boolean; detail: string; via: "direct" | "proxy" | "none" };
export const PLATFORM_LABEL: Record<string, string> = {
    telegram: "",
    bale: "",
    eitaa: "",
    whatsapp: "",
};
/** */
export const WHATSAPP_PROVIDERS = [
    {
        key: "whatsiplus",
        label: " (whatsiplus.ir)",
        hint: " API whatsiplus.ir | : ",
    },

```

```

    },
    {
      key: "ultrams",
      label: "UltraMsg",
      hint: "instanceId:token | : 989121111111",
    },
    {
      key: "cloudapi",
      label: "WhatsApp Cloud API ()",
      hint: "phoneNumberId:accessToken | : ",
    },
    {
      key: "custom",
      label: " ",
      hint: " API {phone} {text} {token}",
    },
  ],
] as const;
/* ----- */
/* ----- */
/* ----- */
export type ProxyConfig = { url: string; enabled: boolean; secret?: string };
export const DEFAULT_PROXY: ProxyConfig = {
  url: process.env.MESSAGING_PROXY_URL || "https://namayandeelemi-javad.javadalamar-mehraeen.workers.dev/",
  enabled: true,
  secret: "",
};
export async function getProxy(): Promise<ProxyConfig> {
  const rows = await dbRetrySafe(
    () => db.select().from(settings).where(eq(settings.key, "messagingProxy")).limit(1),
    [],
    "proxy:get",
  );
  const v = rows[0]?.value as ProxyConfig | undefined;
  return v && typeof v.url === "string" ? { ...DEFAULT_PROXY, ...v } : DEFAULT_PROXY;
}
export type TokenMap = Record<string, string>;
const FALLBACK_TOKENS: TokenMap = {
  bale: process.env.BALE_BOT_TOKEN || "1199464939:uhHpJVtcy__qdtFfN7iuzr4AH7bZBKPG85A",
  telegram: process.env.TELEGRAM_BOT_TOKEN || "",
  eitaa: process.env.EITAA_TOKEN || "",
  whatsapp: process.env.WHATSAPP_TOKEN || "",
};
/** */
export async function getTokens(): Promise<TokenMap> {
  const rows = await dbRetrySafe(
    () => db.select().from(settings).where(eq(settings.key, "messengerTokens")).limit(1),
    [],
    "tokens:get",
  );
  const v = rows[0]?.value as TokenMap | undefined;
  return v && typeof v === "object" ? { ...FALLBACK_TOKENS, ...v } : FALLBACK_TOKENS;
}
export async function resolveToken(platform: string, rowToken?: string) {
  const t = (rowToken ?? "").trim();
  if (t) return t;
  return ((await getTokens())[platform] ?? "").trim();
}
/* ----- */
/* ----- */
/* ----- */
type OrderLike = {
  id: number;
  repName: string;
  dateShamsi: string;
  pharmacyName: string;
  managerName: string;
  managerPhone: string;
  address: string;
  lat: number | null;
  lng: number | null;
  items: Record<string, number>;
  distributor: string;
  visitor: string;
  notes: string;
};
export async function formatOrderMessage(o: OrderLike, withLocation = true) {
  const PRODUCTS = await getProducts();
  const L: string[] = [];
  L.push(" ");
  L.push("-----");
  L.push(` : ${toPersianDigits(o.id)} `);
  L.push(` : ${toPersianDigits(o.dateShamsi)} `);
  L.push(` : ${o.repName} `);
  L.push(` : ${o.pharmacyName} `);
}

```



```

if (o.managerName) L.push(` : ${o.managerName}`);
if (o.managerPhone) L.push(` : ${toPersianDigits(o.managerPhone)}`);
if (o.address) L.push(` : ${o.address}`);
if (withLocation && o.lat && o.lng) L.push(` : https://www.google.com/maps?q=${o.lat},${o.lng}`);

L.push("—— —");
let totalUnits = 0;
let totalBonus = 0;
for (const p of PRODUCTS) {
    const q = Number(o.items?.[p.key] || 0);
    const b = Number(o.items?.[bonusKeyOf(p.key)] || 0);
    if (q || b) {
        totalUnits += q;
        totalBonus += b;
        L.push(`• ${p.label}: ${toPersianDigits(q)}${b ? ` ( ${toPersianDigits(b)})` : ""}`);
    }
}
L.push(` : ${toPersianDigits(totalUnits)} | : ${toPersianDigits(totalBonus)} `);
L.push("————");
if (o.distributor) L.push(` : ${o.distributor}`);
if (o.visitor) L.push(` : ${o.visitor}`);
if (o.notes) L.push(` : ${o.notes}`);
return L.join("\n");
}
/* ----- */
/* ----- */
/* ----- */
const RETRY = { retries: 3, baseDelayMs: 1_000, maxDelayMs: 8_000, timeoutMs: 15_000 };
function friendly(platform: string, status: number, raw: string) {
    const s = raw.replace(/\s+/g, " ").slice(0, 220);
    if (status === 401 || status === 403) return `${platform} (${status}) — ${s}`;
    if (status === 404) return `${platform} () — ${s}`;
    if (status === 400) return ` : ${s}`;
    return `${status}: ${s}`;
}
/** ("true" / "false") */
function isTruthy(v: unknown): boolean | null {
    if (typeof v === "boolean") return v;
    if (typeof v === "string") {
        const s = v.trim().toLowerCase();
        if (["true", "success", "ok", "1", "sent"].includes(s)) return true;
        if (["false", "error", "failed", "0"].includes(s)) return false;
    }
    if (typeof v === "number") return v === 1 || v === 200;
    return null;
}
}
async function readJson(res: Response) {
    const raw = await res.text();
    try {
        return { raw, json: JSON.parse(raw) as Record<string, unknown> };
    } catch {
        return { raw, json: null };
    }
}
}
/** — Bot API */
async function sendBotApi(base: string, token: string, chatId: string, text: string, label: string) {
    const res = await fetchWithRetry(
        `${base}/bot${token}/sendMessage`,
        {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ chat_id: chatId, text, disable_web_page_preview: false }),
        },
        { ...RETRY, label },
    );
    const { raw, json } = await readJson(res);
    if (res.ok && json?.ok !== false) return { ok: true, detail: " " };
    const desc = (json?.description as string) ?? raw;
    return { ok: false, detail: friendly(label, res.status, desc) };
}
}
/** — (form-urlencoded) */
async function sendEitaa(token: string, chatId: string, text: string) {
    const res = await fetchWithRetry(
        `https://eitaa.ir/api/${token}/sendMessage`,
        {
            method: "POST",
            headers: { "Content-Type": "application/x-www-form-urlencoded" },
            body: new URLSearchParams({ chat_id: chatId, text }).toString(),
        },
        { ...RETRY, label: "" },
    );
    const { raw, json } = await readJson(res);
    if (res.ok && json?.ok !== false) return { ok: true, detail: " " };
    return { ok: false, detail: friendly("", res.status, (json?.description as string) ?? raw) };
}

```

```

}
/** — */
async function sendWhatsapp(m: MessengerTarget, token: string, text: string) {
  const provider = m.provider || "whatsiplus";
  const digits = m.target.replace(/\D/g, "");
  if (provider === "whatsiplus") {
    // : https://api.whatsiplus.com/sendMsg/{apikey}?phoneNumber=...&message=...
    const base = (m.apiUrl || "https://api.whatsiplus.com/sendMsg").replace(/\$/ , "");
    const url = `${base}/${token}?phoneNumber=${encodeURIComponent(digits)}&message=${encodeURIComponent(text)}`;
    const res = await fetchWithRetry(url, { method: "GET" }, { ...RETRY, label: "" });
    const { raw, json } = await readJson(res);
    // whatsiplus
    const okFlag = json ? isTruthy(json.success) ?? isTruthy(json.status) : null;
    const failed = okFlag === false || (json?.message && okFlag !== true);
    return res.ok && !failed
      ? { ok: true, detail: " " }
      : { ok: false, detail: ` ${!(json?.message as string) ?? raw.slice(0, 180)} ` };
  }
  if (provider === "ultramsg") {
    const [instance, apiToken] = token.split(":");
    if (!instance || !apiToken) return { ok: false, detail: " UltraMsg instanceId:token " };
    const res = await fetchWithRetry(
      `https://api.ultramsg.com/${instance}/messages/chat`,
      {
        method: "POST",
        headers: { "Content-Type": "application/x-www-form-urlencoded" },
        body: new URLSearchParams({ token: apiToken, to: digits, body: text }).toString(),
      },
      { ...RETRY, label: "" },
    );
    const { raw, json } = await readJson(res);
    const sentOk = json ? isTruthy(json.sent) ?? isTruthy(json.success) : null;
    if (!res.ok || sentOk === false || json?.error) {
      return { ok: false, detail: ` ${!(json?.error as string) ?? raw.slice(0, 180)} ` };
    }
    return { ok: true, detail: " " };
  }
  if (provider === "cloudapi") {
    const i = token.indexOf(":");
    const phoneNumberId = i > 0 ? token.slice(0, i) : "";
    const accessToken = i > 0 ? token.slice(i + 1) : "";
    if (!phoneNumberId || !accessToken) {
      return { ok: false, detail: " Cloud API phoneNumberId:accessToken " };
    }
    const res = await fetchWithRetry(
      `https://graph.facebook.com/v21.0/${phoneNumberId}/messages`,
      {
        method: "POST",
        headers: { "Content-Type": "application/json", Authorization: `Bearer ${accessToken}` },
        body: JSON.stringify({
          messaging_product: "whatsapp",
          recipient_type: "individual",
          to: digits,
          type: "text",
          text: { preview_url: false, body: text },
        }),
      },
      { ...RETRY, label: "" },
    );
    const { raw } = await readJson(res);
    return res.ok ? { ok: true, detail: " " } : { ok: false, detail: friendly("", res.status, raw) };
  }
  //
  if (!m.apiUrl) return { ok: false, detail: " " };
  const url = m.apiUrl
    .replace("{phone}", encodeURIComponent(digits))
    .replace("{text}", encodeURIComponent(text))
    .replace("{token}", encodeURIComponent(token));
  const res = await fetchWithRetry(url, { method: "GET" }, { ...RETRY, label: "" });
  const { raw, json } = await readJson(res);
  const okFlag = json ? isTruthy(json.success) ?? isTruthy(json.status) : null;
  if (!res.ok || okFlag === false) return { ok: false, detail: ` ${raw.slice(0, 180)} ` };
  return { ok: true, detail: " " };
}
async function sendDirect(m: MessengerTarget, token: string, text: string): Promise<{ ok: boolean; detail: string }> {
  try {
    switch (m.platform) {
      case "telegram":
        return await sendBotApi("https://api.telegram.org", token, m.target, text, "");
      case "bale":
        return await sendBotApi("https://tapi.bale.ai", token, m.target, text, "");
      case "eitaa":
        return await sendEitaa(token, m.target, text);
      case "whatsapp":
    }
  }
}

```

```

        return await sendWhatsapp(m, token, text);
    default:
        return { ok: false, detail: " " };
    }
} catch (err) {
    const msg = err instanceof Error ? err.message : " ";
    return {
        ok: false,
        detail: msg.toLowerCase().includes("abort")
        ? " (Timeout) — "
        : ` : ${msg}`,
    };
}
}
}

/** Cloudflare Worker ( ) */
async function sendViaProxy(proxy: ProxyConfig, m: MessengerTarget, token: string, text: string) {
    const alias = m.platform === "eita" ? "ita" : m.platform;
    try {
        const res = await fetchWithRetry(
            proxy.url.trim(),
            {
                method: "POST",
                headers: {
                    "Content-Type": "application/json",
                    ...(proxy.secret ? { "x-proxy-secret": proxy.secret } : {}),
                },
                body: JSON.stringify({
                    messenger: alias,
                    platform: m.platform,
                    provider: m.provider,
                    text,
                    chatId: m.target,
                    chat_id: m.target,
                    token,
                }),
            },
            { ...RETRY, label: "proxy" },
        );
        const { raw, json } = await readJson(res);
        const failed = json && (json.success === false || json.ok === false);
        if (res.ok && !failed) return { ok: true, detail: " ( )" };
        const detail = (json?.error as string) ?? (json?.description as string) ?? raw.slice(0, 200);
        return { ok: false, detail: ` : ${detail}` };
    } catch (err) {
        return { ok: false, detail: ` : ${err instanceof Error ? err.message : ""}` };
    }
}

/**
 *
 *
 */

export async function sendOne(
    input: MessengerTarget,
    text: string,
    proxyOverride?: ProxyConfig,
): Promise<SendResult> {
    if (!input.target.trim()) return { ok: false, detail: " ", via: "none" };
    const token = await resolveToken(input.platform, input.token);
    const proxy = proxyOverride ?? (await getProxy());
    if (!token && input.platform !== "whatsapp") {
        if (!proxy.enabled) return { ok: false, detail: " ", via: "none" };
    }
    // ) ( )
    if (token) {
        const direct = await sendDirect(input, token, text);
        if (direct.ok) return { ...direct, via: "direct" };
        // )
        if (proxy.enabled && proxy.url) {
            const viaProxy = await sendViaProxy(proxy, input, token, text);
            if (viaProxy.ok) return { ...viaProxy, via: "proxy" };
            return { ok: false, detail: ` : ${direct.detail} | ${viaProxy.detail}`, via: "none" };
        }
    }
    return { ...direct, via: "none" };
}

// ( )
if (proxy.enabled && proxy.url) {
    const viaProxy = await sendViaProxy(proxy, input, "", text);
    return { ...viaProxy, via: viaProxy.ok ? "proxy" : "none" };
}
return { ok: false, detail: " ", via: "none" };
}

/* ----- */
/* ----- */
/* ----- */

```

```

export type DispatchReport = {
  total: number;
  sent: number;
  failed: number;
  summary: string;
  results: { platform: string; label: string; target: string; ok: boolean; detail: string }[];
};
/**
 *
 *      (Promise.allSettled).
 */
export async function dispatchText(text: string, orderId?: number): Promise<DispatchReport> {
  const targets = await dbRetrySafe(
    () => db.select().from(messengers).where(eq(messengers.enabled, true)),
    [],
    "messengers:list",
  );
  if (targets.length === 0) {
    return { total: 0, sent: 0, failed: 0, summary: " ", results: [] };
  }
  const proxy = await getProxy();
  const settled = await Promise.allSettled(
    targets.map(async (t) => {
      const r = await sendOne(
        {
          id: t.id,
          platform: t.platform,
          target: t.target,
          token: t.token,
          provider: t.provider,
          apiUrl: t.apiUrl,
          label: t.label,
        },
        text,
        proxy,
      );
      //
      await dbRetrySafe(
        () =>
          db.insert(messageLogs).values({
            orderId: orderId ?? null,
            platform: t.platform,
            target: t.target,
            ok: r.ok,
            detail: r.detail,
          }),
        undefined,
        "messageLog:insert",
      );
      await dbRetrySafe(
        () =>
          db
            .update(messengers)
            .set({
              lastStatus: `${r.ok ? "" : ""} ${r.detail}`.slice(0, 400),
              ...(r.ok ? { lastOkAt: new Date() } : { lastErrorAt: new Date() }),
            })
            .where(eq(messengers.id, t.id)),
        undefined,
        "messenger:status",
      );
    });
  return {
    platform: t.platform,
    label: t.label || PLATFORM_LABEL[t.platform] || t.platform,

    target: t.target,
    ok: r.ok,
    detail: r.detail,
  };
  });
  const results = settled.map((s, i) =>
    s.status === "fulfilled"
      ? s.value
      : {
          platform: targets[i].platform,
          label: targets[i].label || PLATFORM_LABEL[targets[i].platform],
          target: targets[i].target,
          ok: false,
          detail: " ",
        },
  );
  const sent = results.filter((r) => r.ok).length;
  const summary = results
    .map((r) => `${PLATFORM_LABEL[r.platform] ?? r.platform}: ${r.ok ? " " : " "} ${r.detail}`)

```

```

        .join(" | ");
    return { total: results.length, sent, failed: results.length - sent, summary, results };
}
/**
 */
export async function dispatchOrder(order: OrderLike): Promise<string> {
    const text = await formatOrderMessage(order);
    const report = await dispatchText(text, order.id);
    return report.summary;
}
/* ----- */
/* chat_id ( / ) */
/* ----- */
export type ChatInfo = { id: string; title: string; type: string };
export async function fetchUpdates(platform: string, rowToken?: string) {
    const token = await resolveToken(platform, rowToken);
    if (!token) return { ok: false, detail: " ", chats: [] as ChatInfo[] };
    const bases =
        platform === "telegram"
            ? ["https://api.telegram.org"]
            : platform === "bale"
            ? ["https://tapi.bale.ai", "https://api.bale.ai"]
            : [];
    if (bases.length === 0) return { ok: false, detail: " ", chats: [] as ChatInfo[] };
    const chats = new Map<string, ChatInfo>();
    let lastErr = "";
    const consume = (raw: string) => {
        try {
            const j = JSON.parse(raw);
            for (const u of j?.result ?? []) {
                const c = u?.message?.chat ?? u?.channel_post?.chat ?? u?.edited_message?.chat;
                if (!c?.id) continue;
                chats.set(String(c.id), {
                    id: String(c.id),
                    title: c.title || [c.first_name, c.last_name].filter(Boolean).join(" ") || c.username || String(c.id),
                    type: c.type === "private" ? "" : c.type === "group" || c.type === "supergroup" ? "" : "",
                });
            }
            return j?.ok !== false;
        } catch {
            return false;
        }
    };
    for (const base of bases) {
        try {
            const res = await fetchWithRetry(`${base}/bot${token}/getUpdates?limit=100`, { method: "GET" }, { retries: 2, time
outMs: 15_000 });
            const raw = await res.text();
            if (res.ok && consume(raw)) return { ok: true, detail: "", chats: [...chats.values()] };
            lastErr = raw.slice(0, 200);
        } catch (err) {
            lastErr = err instanceof Error ? err.message : " ";
        }
    }

    const proxy = await getProxy();
    if (proxy.enabled && proxy.url) {
        try {
            const res = await fetchWithRetry(
                proxy.url,
                {
                    method: "POST",
                    headers: { "Content-Type": "application/json", ...(proxy.secret ? { "x-proxy-secret": proxy.secret } : {}) },
                    body: JSON.stringify({ action: "getUpdates", messenger: platform === "eita" ? "ita" : platform, token }),
                },
                { retries: 2, timeoutMs: 15_000 },
            );
            const raw = await res.text();
            if (res.ok && consume(raw)) return { ok: true, detail: " ", chats: [...chats.values()] };
            lastErr = raw.slice(0, 200);
        } catch (err) {
            lastErr = err instanceof Error ? err.message : lastErr;
        }
    }
    return {
        ok: false,
        detail: ` : ${lastErr}.`,
        chats: [] as ChatInfo[],
    };
}

```

```

@@ src/lib/mobile-auth.ts typescript
import { createHash, createHmac, randomBytes, timingSafeEqual } from "crypto";
import { db, dbRetrySafe } from "@db";
import { mobileNonces } from "@db/schema";
import { and, eq, gt, lt } from "drizzle-orm";

```

```

/* =====
 *
 *
 *   :
 * ) HMAC-SHA256
 * ) nonce ( Replay Attack)
 * )
 * ===== */
/** — Render MOBILE_APP_SECRET */
const APP_SECRET = process.env.MOBILE_APP_SECRET || process.env.APP_SECRET || "sek-mobile-shared-secret";
export const NONCE_TTL_MS = 3 * 60 * 1000; //
export const MAX_CLOCK_SKEW_MS = 5 * 60 * 1000; //
/** nonce */
export async function issueNonce(deviceId: string) {
  const nonce = randomBytes(24).toString("base64url");
  const expiresAt = new Date(Date.now() + NONCE_TTL_MS);
  await dbRetrySafe(
    () => db.insert(mobileNonces).values({ nonce, deviceId: deviceId.slice(0, 80), expiresAt }),
    undefined,
    "nonce:issue",
  );
  // nonce ( )
  void dbRetrySafe(
    () => db.delete(mobileNonces).where(lt(mobileNonces.expiresAt, new Date(Date.now() - NONCE_TTL_MS))),
    undefined,
    "nonce:cleanup",
  );
  return { nonce, expiresAt, ttlMs: NONCE_TTL_MS };
}
/** nonce — */
export async function consumeNonce(nonce: string): Promise<boolean> {
  if (!nonce) return false;
  const rows = await dbRetrySafe(
    () =>
      db
        .select()
        .from(mobileNonces)
        .where(and(eq(mobileNonces.nonce, nonce), eq(mobileNonces.used, false), gt(mobileNonces.expiresAt, new Date()))))
        .limit(1),
    [],
    "nonce:consume",
  );
  if (rows.length === 0) return false;
  await dbRetrySafe(
    () => db.update(mobileNonces).set({ used: true }).where(eq(mobileNonces.id, rows[0].id)),
    undefined,
    "nonce:mark",
  );
  return true;
}
/** */
export function canonicalPayload(input: {
  nonce: string;
  timestamp: number;
  deviceId: string;
  phoneNumber?: string;
  simFingerprint?: string;
}) {
  return [
    input.nonce,
    String(input.timestamp),
    input.deviceId,
    input.phoneNumber ?? "",
    input.simFingerprint ?? "",
  ].join("|");
}
export function signPayload(canonical: string) {
  return createHmac("sha256", APP_SECRET).update(canonical).digest("base64url");
}
export function verifySignature(canonical: string, signature: string) {
  if (!signature) return false;
  const expected = signPayload(canonical);
  const a = Buffer.from(expected);
  const b = Buffer.from(signature);
  if (a.length !== b.length) return false;
  return timingSafeEqual(a, b);
}
/** (ICCID/IMSI) */
export function hashSim(value: string) {
  if (!value) return "";
  return createHash("sha256").update(`${value}|${APP_SECRET}`).digest("hex").slice(0, 64);
}
export function normalizePhone(p: string) {
  return String(p ?? "");
}

```

```

        .replace(/[\\u06F0-\\u06F9]/g, (c) => String(c.charCodeAt(0) - 0x06F0))
        .replace(/\\D/g, "")
        .replace(/^0098/, "0")
        .replace(/^98/, "0")
        .replace(/^(?!0)(9\\d{9})$/, "0$1");
    }
    export function maskPhone(p: string) {
        const d = normalizePhone(p);
        return d.length >= 8 ? `${d.slice(0, 4)}***${d.slice(-4)}` : "";
    }
    export type AttestationResult =
        | { ok: true }
        | { ok: false; status: number; error: string; code: string };
    /**
     *      : + nonce + .
     *      MOBILE_ALLOW_UNSIGNED=1 .
     */
    export async function verifyAttestation(body: {
        nonce?: string;
        timestamp?: number;
        signature?: string;
        deviceId?: string;
        phoneNumber?: string;
        simFingerprint?: string;
    }): Promise<AttestationResult> {
        const allowUnsigned = process.env.MOBILE_ALLOW_UNSIGNED === "1";
        const nonce = String(body.nonce ?? "");
        const timestamp = Number(body.timestamp ?? 0);
        const deviceId = String(body.deviceId ?? "");
        if (!deviceId) {
            return { ok: false, status: 400, error: " ", code: "NO_DEVICE_ID" };
        }
        if (allowUnsigned) return { ok: true };
        if (!Number.isFinite(timestamp) || Math.abs(Date.now() - timestamp) > MAX_CLOCK_SKEW_MS) {
            return {
                ok: false,
                status: 401,
                error: " ". . . .,
                code: "CLOCK_SKEW",
            };
        }
        const canonical = canonicalPayload({
            nonce,
            timestamp,
            deviceId,
            phoneNumber: body.phoneNumber,
            simFingerprint: body.simFingerprint,
        });
        if (!verifySignature(canonical, String(body.signature ?? ""))) {
            return { ok: false, status: 401, error: " ", code: "BAD_SIGNATURE" };
        }
        if (!(await consumeNonce(nonce))) {
            return {
                ok: false,
                status: 401,
                error: " ". . . .,
                code: "BAD_NONCE",
            };
        }
        return { ok: true };
    }
}

@@ src/lib/notify.ts typescript
import { db } from "@db";
import { notifications } from "@db/schema";
type NotifyInput = {
    toUserId?: number | null;
    toRole?: "admin" | "supervisor" | "rep" | "";
    fromName?: string;
    kind?: string;
    title: string;
    body?: string;
    link?: string;
};
/**
     *
 */
export async function notify(input: NotifyInput) {
    try {
        await db.insert(notifications).values({
            toUserId: input.toUserId ?? null,
            toRole: input.toRole ?? "",
            fromName: input.fromName ?? "",
            kind: input.kind ?? "info",
            title: input.title.slice(0, 200),
            body: (input.body ?? "").slice(0, 2000),
        });
    }
}

```

```

        link: (input.link ?? "").slice(0, 200),
    });
} catch {
    /* ignore */
}
}

@@ src/lib/offline-session.ts typescript
"use client";
/**
 *
 *
 * :
 * sessionStorage → ( = )
 * localStorage → « »
 */
export type CachedUser = {
    id: number;
    username: string;
    fullName: string;
    role: "admin" | "supervisor" | "rep";
    roleKey: string;
    roleLabel: string;
    phone: string;
    requirePhone: boolean;
    permissions: string[];
};

const TOKEN_TAB = "sek_token";
const TOKEN_PERSIST = "sek_token_p";
const USER_CACHE = "sek_user_cache";
const CACHE_AT = "sek_user_cache_at";
/**
 * : */
const MAX_AGE = 14 * 24 * 60 * 60 * 1000;
const safeGet = (store: Storage | undefined, k: string) => {
    try {
        return store?.getItem(k) ?? null;
    } catch {
        return null;
    }
};
const safeSet = (store: Storage | undefined, k: string, v: string) => {
    try {
        store?.setItem(k, v);
    } catch {
        /* ignore */
    }
};
const safeDel = (store: Storage | undefined, k: string) => {
    try {
        store?.removeItem(k);
    } catch {
        /* ignore */
    }
};
export function saveToken(token: string, persist: boolean) {
    if (typeof window === "undefined") return;
    safeSet(sessionStorage, TOKEN_TAB, token);
    if (persist) safeSet(localStorage, TOKEN_PERSIST, token);
}
export function getToken(): string | null {
    if (typeof window === "undefined") return null;
    return safeGet(sessionStorage, TOKEN_TAB) ?? safeGet(localStorage, TOKEN_PERSIST);
}
export function clearToken() {
    if (typeof window === "undefined") return;
    safeDel(sessionStorage, TOKEN_TAB);
    safeDel(localStorage, TOKEN_PERSIST);
    safeDel(localStorage, USER_CACHE);
    safeDel(localStorage, CACHE_AT);
}
export function cacheUser(u: CachedUser) {
    if (typeof window === "undefined") return;
    safeSet(localStorage, USER_CACHE, JSON.stringify(u));
    safeSet(localStorage, CACHE_AT, String(Date.now()));
}
/**
 * : */
export function getCachedUser(): CachedUser | null {
    if (typeof window === "undefined") return null;
    const raw = safeGet(localStorage, USER_CACHE);
    const at = Number(safeGet(localStorage, CACHE_AT) ?? 0);
    if (!raw) return null;
    if (at && Date.now() - at > MAX_AGE) return null;
    //
    if (!getToken()) return null;
    try {

```



```

    return JSON.parse(raw) as CachedUser;
  } catch {
    return null;
  }
}

@@ src/lib/otp.ts typescript
import { createHash, randomInt } from "crypto";
import { db } from "@db";
import { messengers, otpCodes, settings } from "@db/schema";
import { and, desc, eq, gt } from "drizzle-orm";
import { sendOne } from "../messaging";
import { fetchWithRetry } from "../retry";
export const OTP_TTL_MS = 3 * 60 * 1000; //
export const OTP_MAX_ATTEMPTS = 5;
export function hashCode(code: string, phone: string) {
  const secret = process.env.APP_SECRET || "sabt-etelaat-kol-default-secret-key";
  return createHash("sha256").update(`${code}|${phone}|${secret}`).digest("hex");
}

export function generateCode() {
  return String(randomInt(10000, 100000)); //
}
type SmsConfig = { provider: string; apiKey: string; sender: string; pattern: string; enabled: boolean };
export const DEFAULT_SMS: SmsConfig = {
  provider: "kavenegar",
  apiKey: "",
  sender: "",
  pattern: "",
  enabled: false,
};
export async function getSmsConfig(): Promise<SmsConfig> {
  try {
    const rows = await db.select().from(settings).where(eq(settings.key, "sms")).limit(1);
    const v = rows[0]?.value as Partial<SmsConfig> | undefined;
    if (v) return { ...DEFAULT_SMS, ...v };
  } catch {
    /* ignore */
  }
  return {
    ...DEFAULT_SMS,
    provider: process.env.SMS_PROVIDER || DEFAULT_SMS.provider,
    apiKey: process.env.SMS_API_KEY || "",
    sender: process.env.SMS_SENDER || "",
    pattern: process.env.SMS_PATTERN || "",
    enabled: Boolean(process.env.SMS_API_KEY),
  };
}
/** */
async function post(url: string, init?: RequestInit, ms = 12000) {
  return fetchWithRetry(url, init ?? {}, { timeoutMs: ms, retries: 3, baseDelayMs: 900, maxDelayMs: 6_000, label: "sms"
});
}
/** */
async function sendSms(phone: string, code: string): Promise<{ ok: boolean; detail: string }> {
  const cfg = await getSmsConfig();
  if (!cfg.enabled || !cfg.apiKey) return { ok: false, detail: " " };
  const text = ` « »: ${code}\n`;
  try {
    if (cfg.provider === "kavenegar") {
      const base = `https://api.kavenegar.com/v1/${cfg.apiKey}`;
      const url = cfg.pattern
        ? `${base}/verify/lookup.json?receptor=${phone}&token=${code}&template=${cfg.pattern}`
        : `${base}/sms/send.json?receptor=${phone}&sender=${cfg.sender}&message=${encodeURIComponent(text)}`;
      const r = await post(url, { method: "GET" });
      const raw = (await r.text()).slice(0, 200);
      return r.ok ? { ok: true, detail: " " } : { ok: false, detail: raw };
    }
    if (cfg.provider === "smsir") {
      const r = await post("https://api.sms.ir/v1/send/verify", {
        method: "POST",
        headers: { "Content-Type": "application/json", "x-api-key": cfg.apiKey },
        body: JSON.stringify({
          mobile: phone,
          templateId: Number(cfg.pattern) || 100000,
          parameters: [{ name: "CODE", value: code }],
        }),
      });
      const raw = (await r.text()).slice(0, 200);
      return r.ok ? { ok: true, detail: " " } : { ok: false, detail: raw };
    }
    if (cfg.provider === "melipayamak") {
      const r = await post("https://console.melipayamak.com/api/send/simple/" + cfg.apiKey, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
      });
    }
  }
}

```

```

        body: JSON.stringify({ from: cfg.sender, to: phone, text }),
    });
    const raw = (await r.text()).slice(0, 200);
    return r.ok ? { ok: true, detail: " " } : { ok: false, detail: raw };
}
// : {phone} {code}
if (cfg.provider === "custom" && cfg.sender) {
    const url = cfg.sender.replace("{phone}", phone).replace("{code}", code);
    const r = await post(url, { method: "GET" });
    return r.ok ? { ok: true, detail: " " } : { ok: false, detail: `${r.status}` };
}
return { ok: false, detail: " " };
} catch (e) {
    return { ok: false, detail: e instanceof Error ? e.message : " " };
}
}
/**      (/*      */
export async function sendViaMessenger(phone: string, code: string): Promise<{ ok: boolean; detail: string }> {
    try {
        const rows = await db.select().from(messengers);
        const target = rows.find((m) => m.enabled && m.target.replace(/\D/g, "").endsWith(phone.slice(-9)));
        if (!target) return { ok: false, detail: " " };
        const r = await sendOne(
            { platform: target.platform, target: target.target, token: target.token },
            `« »:${code}\n`;
        );
        return r;
    } catch {
        return { ok: false, detail: " " };
    }
}
}
export async function issueOtp(userId: number, phone: string, deviceId: string) {
    const code = generateCode();
    const expiresAt = new Date(Date.now() + OTP_TTL_MS);
    let channel = "sms";
    let res = await sendSms(phone, code);
    if (!res.ok) {
        const alt = await sendViaMessenger(phone, code);
        if (alt.ok) {
            channel = "messenger";
            res = alt;
        }
    }
    await db.insert(otpCodes).values({
        userId,
        phone,
        codeHash: hashCode(code, phone),
        deviceId,
        channel: res.ok ? channel : "manual",
        expiresAt,
    });
    return { sent: res.ok, channel: res.ok ? channel : "manual", detail: res.detail, code };
}
export async function verifyOtp(userId: number, phone: string, code: string) {
    const rows = await db
        .select()
        .from(otpCodes)
        .where(and(eq(otpCodes.userId, userId), eq(otpCodes.used, false), gt(otpCodes.expiresAt, new Date()))))
        .orderBy(desc(otpCodes.id))
        .limit(1);
    const row = rows[0];
    if (!row) return { ok: false, error: " " };
    if (row.attempts >= OTP_MAX_ATTEMPTS) return { ok: false, error: " " };
    if (row.codeHash !== hashCode(String(code).trim(), phone)) {
        await db.update(otpCodes).set({ attempts: row.attempts + 1 }).where(eq(otpCodes.id, row.id));
        return { ok: false, error: ` (${OTP_MAX_ATTEMPTS - row.attempts - 1} )` };
    }
    await db.update(otpCodes).set({ used: true }).where(eq(otpCodes.id, row.id));
    return { ok: true, deviceId: row.deviceId };
}
}

@@ src/lib/pwa-image.ts typescript
import { readFile } from "node:fs/promises";
import path from "node:path";
const PUB = path.join(process.cwd(), "public");
/**      */
const FALLBACK_LOGO = `<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 512 512" width="512" height="512">
<defs>
<linearGradient id="pill" x1="0" y1="1" x2="1" y2="0">
<stop offset="0%" stop-color="#F9A03F"/><stop offset="35%" stop-color="#F7714B"/>
<stop offset="70%" stop-color="#EC3B75"/><stop offset="100%" stop-color="#C81E63"/>
</linearGradient>
<linearGradient id="glossTop" x1="0" y1="0" x2="0" y2="1">
<stop offset="0%" stop-color="#ffffff" stop-opacity=".45"/>
`

```

```

        <stop offset="55%" stop-color="#ffffff" stop-opacity="0"/>
    </linearGradient>
</defs>
<g transform="translate(256 256) rotate(-45)">
    <rect x="-168" y="-84" width="336" height="168" rx="84" ry="84" fill="url(#pill)"/>

    <line x1="0" y1="-84" x2="0" y2="84" stroke="#ffffff" stroke-opacity=".35" stroke-width="6"/>
    <rect x="-160" y="-76" width="320" height="80" rx="40" ry="40" fill="url(#glossTop)"/>
    <circle cx="104" cy="-30" r="18" fill="#ffffff" fill-opacity=".92"/>
</g>
</svg>`;
const cache = new Map<string, Uint8Array>();
async function loadSvg(rel: string, fallback: string) {
    try {
        return await readFile(path.join(PUB, rel), "utf8");
    } catch {
        return fallback;
    }
}
// eslint-disable-next-line @typescript-eslint/no-explicit-any
async function getSharp(): Promise<any | null> {
    try {
        const mod = await import("sharp");
        return mod.default;
    } catch {
        return null;
    }
}
const plate = (size: number) =>
    Buffer.from(
        `<svg xmlns="http://www.w3.org/2000/svg" width="${size}" height="${size}">
            <defs><linearGradient id="g" x1="0" y1="0" x2="1" y2="1">
                <stop offset="0%" stop-color="#ffffff"/><stop offset="100%" stop-color="#eef2f6"/></linearGradient></defs>
            <rect width="${size}" height="${size}" fill="url(#g)"/></svg>`,
    );
/** PNG ( ) */
export async function renderIcon(size: number, maskable = false): Promise<Uint8Array | null> {
    const key = `${maskable ? "m" : "i"}-${size}`;
    const hit = cache.get(key);
    if (hit) return hit;
    const sharp = await getSharp();
    const svg = await loadSvg("logo.svg", FALLBACK_LOGO);
    if (!sharp) return null;
    let out: Buffer;
    if (maskable) {
        const inner = await sharp(Buffer.from(svg), { density: 500 })
            .resize(Math.round(size * 0.62), Math.round(size * 0.62))
            .png()
            .toBuffer();
        out = await sharp(plate(size)).composite([
            { input: inner, gravity: "center" }
        ]).png().toBuffer();
    } else {
        out = await sharp(Buffer.from(svg), { density: 500 })
            .resize(size, size, { fit: "contain", background: { r: 0, g: 0, b: 0, alpha: 0 } })
            .png({ compressionLevel: 9 })
            .toBuffer();
    }
    const bytes = new Uint8Array(out);
    cache.set(key, bytes);
    return bytes;
}
/** PNG */
export async function renderScreenshot(name: string): Promise<Uint8Array | null> {
    const hit = cache.get(`s-${name}`);
    if (hit) return hit;
    const sharp = await getSharp();
    if (!sharp) return null;
    const svg = await loadSvg(`src-svg/${name}.svg`, "");
    if (!svg) return null;
    const out = await sharp(Buffer.from(svg)).png().toBuffer();
    const bytes = new Uint8Array(out);
    cache.set(`s-${name}`, bytes);
    return bytes;
}
export function pngResponse(bytes: Uint8Array) {
    return new Response(bytes as unknown as BodyInit, {
        headers: {
            "Content-Type": "image/png",
            "Content-Length": String(bytes.length),
            "Cache-Control": "public, max-age=31536000, immutable",
            "Access-Control-Allow-Origin": "*",
        },
    });
}
}

```

```

@@ src/lib/retry.ts typescript
/**
 * =====
 * Retry Exponential Backoff + Jitter
 * -----
 *
 * Neon (cold start) .
 *
 * :
 * - «» ( 5xx 429).
 * - (unique violation 4xx syntax)
 *
 * - Jitter « » .
 * =====
 */
export type RetryOptions = {
  /** ( ) */
  retries?: number;
  /** */
  baseDelayMs?: number;
  /** */
  maxDelayMs?: number;
  /** */
  timeoutMs?: number;
  /** */
  label?: string;
  isRetryable?: (error: unknown) => boolean;
  /** */
  onRetry?: (info: { attempt: number; delayMs: number; error: unknown }) => void;
  signal?: AbortSignal;
};

const DEFAULTS: Required<Omit<RetryOptions, "isRetryable" | "onRetry" | "signal" | "label">> = {
  retries: 4,
  baseDelayMs: 1_000,
  maxDelayMs: 30_000,
  timeoutMs: 20_000,
};
/** */
export const TRANSIENT_NETWORK_CODES = new Set([
  "ECONNRESET",
  "ETIMEDOUT",
  "ECONNREFUSED",
  "EPIPE",
  "EHOSTUNREACH",
  "ENETUNREACH",
  "ENETDOWN",
  "EAI_AGAIN",
  "ENOTFOUND",
  "EADDRNOTAVAIL",
  "ECONNABORTED",
  "UND_ERR_CONNECT_TIMEOUT",
  "UND_ERR_HEADERS_TIMEOUT",
  "UND_ERR_SOCKET",
]);
/**
 * PostgreSQL ( 08 = 53 =
 * 57P = / 40001/40P01 = )
 */
export const TRANSIENT_PG_CODES = new Set([
  "08000", "08001", "08003", "08004", "08006", "08007", "08P01",
  "53300", "53400", "57P01", "57P02", "57P03",
  "40001", "40P01",
  "XX000", // internal error — Neon
]);
const TRANSIENT_MESSAGE_HINTS = [
  "timeout",
  "timed out",
  "connection terminated",
  "connection closed",
  "connection reset",
  "socket hang up",
  "server closed the connection",
  "terminating connection",
  "too many connections",
  "fetch failed",
  "network",
  "temporarily unavailable",

  "endpoint is disabled", // Neon
  "compute time exceeded",
  "could not connect",
];
function errCode(error: unknown): string {
  if (!error || typeof error !== "object") return "";

```

```

const e = error as { code?: unknown; errno?: unknown; cause?: unknown };
if (typeof e.code === "string") return e.code;
if (typeof e.errno === "string") return e.errno;
if (e.cause) return errCode(e.cause);
return "";
}
function errMsg(error: unknown): string {
  if (error instanceof Error) return `${error.message} ${errMsg((error as Error & { cause?: unknown }).cause ?? "")}`;
  if (typeof error === "string") return error;
  return "";
}
/** */
export function isTransientError(error: unknown): boolean {
  if (!error) return false;
  //
  if (error instanceof Error && error.name === "AbortError") return false;
  const code = errCode(error).toUpperCase();
  if (TRANSIENT_NETWORK_CODES.has(code)) return true;
  if (TRANSIENT_PG_CODES.has(errCode(error))) return true;
  // HTTP: 5xx 429 408
  const status = (error as { status?: number; statusCode?: number })?.status ??
    (error as { statusCode?: number })?.statusCode;
  if (typeof status === "number") {
    if (status >= 500 || status === 429 || status === 408) return true;
    return false;
  }
  const msg = errMsg(error).toLowerCase();
  return TRANSIENT_MESSAGE_HINTS.some((h) => msg.includes(h));
}
/** Jitter (Full Jitter) */
export function backoffDelay(attempt: number, baseDelayMs: number, maxDelayMs: number): number {
  const exponential = Math.min(maxDelayMs, baseDelayMs * 2 ** (attempt - 1));
  // Full Jitter:
  const half = exponential / 2;
  return Math.round(half + Math.random() * half);
}
const sleep = (ms: number, signal?: AbortSignal) =>
  new Promise<void>((resolve, reject) => {
    const t = setTimeout(resolve, ms);
    signal?.addEventListener(
      "abort",
      () => {
        clearTimeout(t);
        reject(new DOMException("Aborted", "AbortError"));
      },
      { once: true },
    );
  });
function withTimeout<T>(fn: (signal: AbortSignal) => Promise<T>, ms: number, outer?: AbortSignal): Promise<T> {
  if (!ms) return fn(outer ?? new AbortController().signal);
  const ctrl = new AbortController();
  const timer = setTimeout(() => ctrl.abort(new DOMException("Timeout", "TimeoutError")), ms);
  outer?.addEventListener("abort", () => ctrl.abort(), { once: true });
  return fn(ctrl.signal).finally(() => clearTimeout(timer));
}
/**
 *
 *
 * @example
 * const rows = await withRetry(() => db.select().from(users), { label: "users:list" });
 */
export async function withRetry<T>(
  operation: (attempt: number, signal: AbortSignal) => Promise<T>,
  options: RetryOptions = {},
): Promise<T> {
  const cfg = { ...DEFAULTS, ...options };
  const retryable = options.isRetryable ?? isTransientError;

  let lastError: unknown;
  for (let attempt = 1; attempt <= cfg.retries; attempt++) {
    try {
      return await withTimeout((signal) => operation(attempt, signal), cfg.timeoutMs, options.signal);
    } catch (error) {
      lastError = error;
      const isLast = attempt >= cfg.retries;
      // TimeoutError
      const timedOut = error instanceof Error && error.name === "TimeoutError";
      if (isLast || !(timedOut || retryable(error))) throw error;
      const delayMs = backoffDelay(attempt, cfg.baseDelayMs, cfg.maxDelayMs);
      options.onRetry?.({ attempt, delayMs, error });
      console.warn(
        [retry${options.label ? `:${options.label}` : ""} ${attempt}/${cfg.retries} ` +
        `${errCode(error) || errMsg(error).slice(0, 80)} — ${delayMs}ms`],
      );
    }
  }
}

```

```

        await sleep(delayMs, options.signal);
    }
}
throw lastError;
}
/** : */
export async function withRetrySafe<T>({
    operation: (attempt: number, signal: AbortSignal) => Promise<T>,
    fallback: T,
    options: RetryOptions = {},
}): Promise<T> {
    try {
        return await withRetry(operation, options);
    } catch (error) {
        console.error([retry${options.label ? `:${options.label}` : ""}] :`, errMessage(error).slice(0, 200));
        return fallback;
    }
}
}
/** fetch ( ) */
export async function fetchWithRetry(
    input: string | URL,
    init: RequestInit = {},
    options: RetryOptions = {},
): Promise<Response> {
    return withRetry(
        async (_attempt, signal) => {
            const res = await fetch(input, { ...init, signal: init.signal ?? signal, cache: init.cache ?? "no-store" });
            // 5xx / 429 / 408 Retry
            if (res.status >= 500 || res.status === 429 || res.status === 408) {
                const err = new Error(`HTTP ${res.status} ${typeof input === "string" ? input : input.toString()}`);
                (err as Error & { status: number }).status = res.status;
                throw err;
            }
            return res;
        },
        { retries: 3, baseDelayMs: 800, maxDelayMs: 8_000, timeoutMs: 15_000, ...options },
    );
}
}

```

```

@@ src/lib/settings-server.ts typescript
import { db } from "@db";
import { settings } from "@db/schema";
import { DEFAULT_PRODUCTS, type ProductConfig } from "../defaults";
import { eq } from "drizzle-orm";
/** ( ) */
export async function getProducts(): Promise<ProductConfig[]> {
    try {
        const rows = await db.select().from(settings).where(eq(settings.key, "products")).limit(1);
        const v = rows[0]?.value as ProductConfig[] | undefined;
        if (Array.isArray(v) && v.length) return v.filter((p) => p.enabled !== false);
    } catch {
        /* ignore */
    }
    return DEFAULT_PRODUCTS;
}
}

```

```

@@ src/lib/sort.ts typescript
/** : */
const collator = new Intl.Collator("fa", {
    numeric: true,
    sensitivity: "base",
    ignorePunctuation: true,
});

export function faCompare(a: string, b: string) {
    return collator.compare(String(a ?? ""), String(b ?? ""));
}
export function faSort(values: string[]) {
    return [...new Set(values.filter(Boolean))].sort(faCompare);
}
/** */
export function toEnglishDigits(value: string) {
    return String(value)
        .replace(/[\u06F0-\u06F9]/g, (c) => String(c.charCodeAt(0) - 0x06f0))
        .replace(/[\u0660-\u0669]/g, (c) => String(c.charCodeAt(0) - 0x0660));
}
}

```

```

@@ src/lib/sound.ts typescript
/** (WebAudio) — */
export function playNotificationSound() {
    try {
        const Ctx =
            window.AudioContext ||
            (window as unknown as { webkitAudioContext: typeof AudioContext }).webkitAudioContext;
        if (!Ctx) return;
    }
}

```

```

const ctx = new Ctx();
const now = ctx.currentTime;
const notes = [880, 1174.7];
notes.forEach((f, i) => {
  const osc = ctx.createOscillator();
  const gain = ctx.createGain();
  osc.type = "sine";
  osc.frequency.value = f;
  gain.gain.setValueAtTime(0.0001, now + i * 0.18);
  gain.gain.exponentialRampToValueAtTime(0.25, now + i * 0.18 + 0.03);
  gain.gain.exponentialRampToValueAtTime(0.0001, now + i * 0.18 + 0.22);
  osc.connect(gain).connect(ctx.destination);
  osc.start(now + i * 0.18);
  osc.stop(now + i * 0.18 + 0.24);
});
setTimeout(() => ctx.close().catch(() => undefined), 900);
} catch {
  /* ignore */
}
}
export function vibrate(pattern: number | number[] = [120, 60, 120]) {
  try {
    navigator.vibrate?.(pattern);
  } catch {
    /* ignore */
  }
}

```

@@ src/lib/sync-config.ts typescript

```

/**
 * =====
 *
 * Render (namayandeelmi-javad.onrender.com) NdcoHub.ir
 * -----
 *
 * :
 * uid → (UUID)
 * updated_at → ( )
 * origin →
 *
 * : Last-Write-Wins updated_at
 * =====
 */
export type SyncTable = {
  /** */
  name: string;
  /** */
  label: string;
  /** ( ) */
  skipColumns?: string[];
  /** ( ) */
  defaultOn: boolean;
  /** */
  batch: number;
};
export const SYNC_TABLES: SyncTable[] = [
  { name: "users", label: "", defaultOn: true, batch: 500 },
  { name: "roles", label: "", defaultOn: true, batch: 200 },
  { name: "options", label: " ", defaultOn: true, batch: 2000 },

  { name: "settings", label: "", defaultOn: true, batch: 200 },
  { name: "pharmacies", label: "", defaultOn: true, batch: 1000 },
  { name: "doctors", label: "", defaultOn: true, batch: 1000 },
  { name: "orders", label: "", defaultOn: true, batch: 1000 },
  { name: "targets", label: " ", defaultOn: true, batch: 500 },
  { name: "homes", label: " ", defaultOn: true, batch: 500 },
  { name: "leaves", label: "", defaultOn: true, batch: 500 },
  { name: "messengers", label: "", defaultOn: true, batch: 200 },
  { name: "notifications", label: "", defaultOn: true, batch: 500 },
  { name: "activity_logs", label: " ", defaultOn: false, batch: 500 },
  { name: "trips", label: " ", defaultOn: true, batch: 500 },
  { name: "trip_points", label: " ", defaultOn: false, batch: 2000 },
  { name: "attachments", label: " ", defaultOn: false, batch: 20 },
];
/** ( ) */
export const LOCAL_ONLY_COLUMNS = new Set(["id"]);
/** — Render NdcoHub */
export function nodeName(): string {
  return (
    process.env.NODE_NAME ||
    process.env.SYNC_NODE_NAME ||
    (process.env.RENDER ? "render" : "") ||
    (process.env.VERCEL ? "vercel" : "") ||
    "primary"
  );
}

```

```

/** */
export function selfUrl(): string {
    return (
        process.env.PUBLIC_BASE_URL ||
        process.env.NEXT_PUBLIC_BASE_URL ||
        (process.env.RENDER_EXTERNAL_URL ?? "") ||
        ""
    );
}
/** : name|url,name|url */
export function configuredPeers(): { peer: string; url: string }[] {
    const raw = process.env.SYNC_PEERS || "";
    return raw
        .split(",")
        .map((s) => s.trim())
        .filter(Boolean)
        .map((entry) => {
            const [peer, url] = entry.split("|").map((x) => x.trim());
            return { peer: peer || "peer", url: url || peer };
        })
        .filter((p) => p.url.startsWith("http"));
}
/** */
export function syncSecret(): string {
    return process.env.SYNC_SECRET || process.env.APP_SECRET || "";
}
/** */
export const PUBLIC_ENDPOINTS: string[] = (
    process.env.NEXT_PUBLIC_ENDPOINTS || "https://ndcohub.ir,https://namayandeelmi-javad.onrender.com"
)
    .split(",")
    .map((s) => s.trim().replace(/\/$/ , ""))
    .filter(Boolean);

@@ src/lib/sync.ts typescript
import { createHmac, timingSafeEqual } from "crypto";
import { db, dbRetrySafe, pool } from "@db";
import { syncLogs, syncState } from "@db/schema";
import { fetchWithRetry } from "./retry";
import { LOCAL_ONLY_COLUMNS, SYNC_TABLES, configuredPeers, nodeName, selfUrl, syncSecret } from "./sync-config";
import { eq } from "drizzle-orm";

/* =====
*
*
*
* :
* ) PULL → pullCursor
* ) PUSH → pushCursor
*
* : Last-Write-Wins updated_at
* : origin
* ===== */

export type ChangeSet = Record<string, Record<string, unknown>[]>;
export type SyncReport = {
    peer: string;
    ok: boolean;
    pulled: number;
    pushed: number;
    conflicts: number;
    detail: string;
    durationMs: number;
};
const MAX_BODY_ROWS = 800;
/* ----- */
/* ----- */
/* ----- */
export function signSync(body: string, timestamp: number) {
    return createHmac("sha256", syncSecret()).update(`${timestamp}.${body}`).digest("base64url");
}
export function verifySync(body: string, timestamp: number, signature: string) {
    if (!syncSecret()) return false;
    if (!signature) return false;
    if (Math.abs(Date.now() - timestamp) > 10 * 60 * 1000) return false;
    const expected = signSync(body, timestamp);
    const a = Buffer.from(expected);
    const b = Buffer.from(signature);
    if (a.length !== b.length) return false;
    return timingSafeEqual(a, b);
}
/* ----- */
/* ----- */
/* ----- */
async function tableColumns(table: string): Promise<string[]> {
    const res = await pool.query(
        `select column_name from information_schema.columns where table_schema='public' and table_name=${table}`
    );
}

```



```

    [table],
  );
  return res.rows.map((r) => String(r.column_name));
}
/**
 *
 *
 * :   origin
 *
 *      :
 * )   (sek.sync = on)
 * )   `updated_at < excluded.updated_at`
 */
export async function collectChanges(since: Date | null, excludeOrigin?: string): Promise<ChangeSet> {
  void excludeOrigin;
  const out: ChangeSet = {};
  const sinceIso = since ? since.toISOString() : "1970-01-01T00:00:00Z";
  for (const t of SYNC_TABLES) {
    if (!t.defaultOn) continue;
    try {
      const cols = await tableColumns(t.name);
      if (!cols.includes("uid") || !cols.includes("updated_at")) continue;
      const res = await pool.query(
        `select * from ${t.name} where updated_at > $1 order by updated_at asc limit $2`,
        [sinceIso, t.batch],
      );
      if (res.rows.length) out[t.name] = res.rows as Record<string, unknown>[];
    } catch (err) {
      console.warn(`[sync]  ${t.name} :`, err instanceof Error ? err.message : err);
    }
  }
  return out;
}
/* ----- */
/* ----- */
/* ----- */
/** UPSERT uid Last-Write-Wins */
export async function applyChanges(changes: ChangeSet, fromPeer: string) {
  let applied = 0;
  let skipped = 0;
  // set_config

  const client = await pool.connect();
  try {
    await client.query(`select set_config('sek.sync','on',false)`);
  } catch {
    /* ignore */
  }
  const perTable: Record<string, { applied: number; skipped: number }> = {};
  for (const [table, rows] of Object.entries(changes)) {
    const cfg = SYNC_TABLES.find((t) => t.name === table);
    if (!cfg || !Array.isArray(rows) || rows.length === 0) continue;
    let cols: string[] = [];
    try {
      cols = await tableColumns(table);
    } catch {
      continue;
    }
    if (!cols.includes("uid")) continue;
    perTable[table] = { applied: 0, skipped: 0 };
    for (const raw of rows.slice(0, MAX_BODY_ROWS)) {
      const row = { ...raw } as Record<string, unknown>;
      const uid = String(row.uid ?? "");
      if (!uid) {
        skipped++;
        perTable[table].skipped++;
        continue;
      }
      //
      for (const k of Object.keys(row)) {
        if (!cols.includes(k) || LOCAL_ONLY_COLUMNS.has(k)) delete row[k];
      }
      if (!row.origin) row.origin = fromPeer;
      const keys = Object.keys(row);
      if (keys.length === 0) continue;
      const placeholders = keys.map((_, i) => `${i + 1}`).join(", ");
      const updates = keys
        .filter((k) => k !== "uid")
        .map((k) => `${k} = excluded.${k}`)
        .join(", ");
      try {
        //
        // set_config
        const q = `
          insert into ${table} (${keys.join(", ")}) values (${placeholders})
          on conflict (uid) do update set ${updates}
        `;
      }
    }
  }

```

```

        where ${table}.updated_at < excluded.updated_at
    `;
    const res = await client.query(q, keys.map((k) => row[k]));
    if (res.rowCount && res.rowCount > 0) {
        applied++;
        perTable[table].applied++;
    } else {
        skipped++;
        perTable[table].skipped++;
    }
} catch (err) {
    skipped++;
    perTable[table].skipped++;
    console.warn(`[sync] ${table}/${uid} :`, err instanceof Error ? err.message.slice(0, 120) : err);
}
}
}
//
//
try {
    await client.query(`select set_config('sek.sync','off',false)`);
} catch {
    /* ignore */
}
client.release();
for (const [table, st] of Object.entries(perTable)) {
    void dbRetrySafe(
        () =>
        db.insert(syncLogs).values({
            peer: fromPeer,
            direction: "pull",

            tableName: table,
            applied: st.applied,
            skipped: st.skipped,
            ok: true,
        })),
        undefined,
        "sync:log",
    );
}
return { applied, skipped, perTable };
}
/* ----- */
/* ----- */
/* ----- */
async function callPeer(url: string, path: string, payload: unknown) {
    const body = JSON.stringify(payload);
    const ts = Date.now();
    const res = await fetchWithRetry(
        `${url.replace(/\$/g, "")}${path}`,
        {
            method: "POST",
            headers: {
                "Content-Type": "application/json",
                "x-sync-node": nodeName(),
                "x-sync-timestamp": String(ts),
                "x-sync-signature": signSync(body, ts),
            },
            body,
        },
        { retries: 3, baseDelayMs: 1200, maxDelayMs: 10_000, timeoutMs: 45_000, label: "sync" },
    );
    const text = await res.text();
    let data: Record<string, unknown> = {};
    try {
        data = JSON.parse(text);
    } catch {
        data = { raw: text.slice(0, 200) };
    }
    if (!res.ok) throw new Error(String(data.error ?? `HTTP ${res.status}`));
    return data;
}
/* ----- */
/* ----- */
/* ----- */
export async function ensurePeers() {
    const peers = configuredPeers();
    for (const p of peers) {
        const existing = await dbRetrySafe(
            () => db.select().from(syncState).where(eq(syncState.peer, p.peer)).limit(1),
            [],
            "sync:peer",
        );
        if (existing.length === 0) {

```

```

    await dbRetrySafe(
      () => db.insert(syncState).values({ peer: p.peer, peerUrl: p.url }),
      undefined,
      "sync:addPeer",
    );
  } else if (existing[0].peerUrl !== p.url) {
    await dbRetrySafe(
      () => db.update(syncState).set({ peerUrl: p.url }).where(eq(syncState.id, existing[0].id)),
      undefined,
      "sync:updPeer",
    );
  }
}
return dbRetrySafe(() => db.select().from(syncState), [], "sync:list");
}
/* ----- */
/* ----- */
/* ----- */
export async function syncWithPeer(peerRow: typeof syncState.$inferSelect): Promise<SyncReport> {
  const started = Date.now();
  const me = nodeName();
  let pulled = 0;
  let pushed = 0;
  let conflicts = 0;
  const notes: string[] = [];

  try {
    if (!syncSecret()) throw new Error("SYNC_SECRET ");
    if (!peerRow.peerUrl) throw new Error(" ");
    /* ----- ) PULL: ----- */
    const pullRes = await callPeer(peerRow.peerUrl, "/api/sync/pull", {
      node: me,
      since: peerRow.pullCursor ? new Date(peerRow.pullCursor).toISOString() : null,
      excludeOrigin: me, //
    });
    const changes = (pullRes.changes ?? {}) as ChangeSet;
    const applyRes = await applyChanges(changes, peerRow.peer);
    pulled = applyRes.applied;
    conflicts += applyRes.skipped;
    const newPullCursor = pullRes.cursor ? new Date(String(pullRes.cursor)) : new Date();
    notes.push(` ${pulled} `);
    /* ----- ) PUSH: ----- */
    const localChanges = await collectChanges(
      peerRow.pushCursor ? new Date(peerRow.pushCursor) : null,
      peerRow.peer, //
    );
    const rowCount = Object.values(localChanges).reduce((a, r) => a + r.length, 0);
    let newPushCursor = peerRow.pushCursor ? new Date(peerRow.pushCursor) : new Date(0);
    if (rowCount > 0) {
      const pushRes = await callPeer(peerRow.peerUrl, "/api/sync/push", { node: me, changes: localChanges });
      pushed = Number(pushRes.applied ?? 0);
      conflicts += Number(pushRes.skipped ?? 0);
      // updated_at
      for (const rows of Object.values(localChanges)) {
        for (const r of rows) {
          const u = new Date(String(r.updated_at));
          if (!Number.isNaN(u.getTime()) && u > newPushCursor) newPushCursor = u;
        }
      }
      notes.push(` ${pushed} `);
    } else {
      newPushCursor = new Date();
      notes.push(" ");
    }
  }
  await dbRetrySafe(
    () =>
      db
        .update(syncState)
        .set({
          pullCursor: newPullCursor,
          pushCursor: newPushCursor,
          lastSyncAt: new Date(),
          lastStatus: ` ${notes.join(" | ")} `,
          lastError: "",
          pulled: (peerRow.pulled ?? 0) + pulled,
          pushed: (peerRow.pushed ?? 0) + pushed,
          conflicts: (peerRow.conflicts ?? 0) + conflicts,
        })
        .where(eq(syncState.id, peerRow.id)),
    undefined,
    "sync:ok",
  );
  return {
    peer: peerRow.peer,
    ok: true,
  };
}

```

```

        pulled,
        pushed,
        conflicts,
        detail: notes.join(" | "),
        durationMs: Date.now() - started,
    };
} catch (err) {
    const detail = err instanceof Error ? err.message : " ";
    await dbRetrySafe(
        () => {
            db
                .update(syncState)
                .set({ lastSyncAt: new Date(), lastStatus: " ", lastError: detail.slice(0, 400) })
                .where(eq(syncState.id, peerRow.id)),
            undefined,
            "sync:fail",
        );
    await dbRetrySafe(
        () => db.insert(syncLogs).values({ peer: peerRow.peer, direction: "both", ok: false, detail: detail.slice(0, 400)
    })),
    undefined,
    "sync:logFail",
);
return {
    peer: peerRow.peer,
    ok: false,
    pulled,
    pushed,
    conflicts,
    detail,
    durationMs: Date.now() - started,
};
}
}
/**      */
export async function syncAll(): Promise<SyncReport[]> {
    const peers = await ensurePeers();
    const active = peers.filter((p) => p.enabled && p.peerUrl);
    if (active.length === 0) return [];
    const out: SyncReport[] = [];
    for (const p of active) out.push(await syncWithPeer(p));
    return out;
}
export function syncInfo() {
    return {
        node: nodeName(),
        url: selfUrl(),
        peers: configuredPeers(),
        hasSecret: Boolean(syncSecret()),
        tables: SYNC_TABLES.filter((t) => t.defaultOn).map((t) => t.name),
    };
}
}

@@ src/lib/tracker.ts typescript
"use client";
/**
 * =====
 * (Offline-First GPS Tracker)
 * -----
 * :
 * • IndexedDB
 * •
 * •
 * • GPS
 * •
 * • «»
 * =====
 */
export type TrackPoint = {
    lat: number;
    lng: number;
    accuracy?: number | null;
    speed?: number | null;
    heading?: number | null;
    kind: "start" | "move" | "pause" | "end";
    note?: string;
    stopSeconds?: number;
    gpsOn?: boolean;
    recordedAt: string;
};
const DB_NAME = "sek-track";
const STORE = "points";
const MIN_DISTANCE_M = 12; // (GPS)
const STOP_THRESHOLD_MS = 2 * 60 * 1000; //

```

```

/* ----- IndexedDB ----- */
function idb(): Promise<IDBDatabase> {
  return new Promise((res, rej) => {
    const r = indexedDB.open(DB_NAME, 1);
    r.onupgradeneeded = () => {
      const d = r.result;
      if (!d.objectStoreNames.contains(STORE)) d.createObjectStore(STORE, { keyPath: "id", autoIncrement: true });
    };
    r.onsuccess = () => res(r.result);
    r.onerror = () => rej(r.error);
  });
}

export async function bufferPoint(tripId: number, p: TrackPoint) {
  try {
    const d = await idb();
    await new Promise<void>((res) => {
      const tx = d.transaction(STORE, "readwrite");
      tx.objectStore(STORE).add({ tripId, ...p });
      tx.oncomplete = () => res();
      tx.onerror = () => res();
    });
  } catch {
    /* — */
  }
}

export async function bufferedPoints(): Promise<(TrackPoint & { id: number; tripId: number })[]> {
  try {
    const d = await idb();
    return await new Promise((res) => {
      const tx = d.transaction(STORE, "readonly");
      const rq = tx.objectStore(STORE).getAll();
      rq.onsuccess = () => res(rq.result ?? []);
      rq.onerror = () => res([]);
    });
  } catch {
    return [];
  }
}

async function removePoints(ids: number[]) {
  if (!ids.length) return;
  try {
    const d = await idb();
    const tx = d.transaction(STORE, "readwrite");
    const st = tx.objectStore(STORE);
    for (const id of ids) st.delete(id);
  } catch {
    /* ignore */
  }
}

export async function bufferedCount() {
  return (await bufferedPoints()).length;
}

/* ----- */
export function haversine(a: { lat: number; lng: number }, b: { lat: number; lng: number }) {
  const R = 6371000;
  const dLat = ((b.lat - a.lat) * Math.PI) / 180;
  const dLng = ((b.lng - a.lng) * Math.PI) / 180;
  const s =
    Math.sin(dLat / 2) ** 2 +
    Math.cos((a.lat * Math.PI) / 180) * Math.cos((b.lat * Math.PI) / 180) * Math.sin(dLng / 2) ** 2;
  return 2 * R * Math.asin(Math.sqrt(s));
}

export function formatDistance(m: number) {
  return m >= 1000 ? `${(m / 1000).toFixed(1)} ` : `${Math.round(m)} `;
}

export function formatDuration(sec: number) {
  const h = Math.floor(sec / 3600);
  const m = Math.floor((sec % 3600) / 60);
  if (h > 0) return `${h} ${m} `;
  if (m > 0) return `${m} `;
  return `${Math.round(sec)} `;
}

/* ----- */
export async function flushPoints(): Promise<{ sent: number; pending: number }> {
  const all = await bufferedPoints();
  if (all.length === 0) return { sent: 0, pending: 0 };
  const byTrip = new Map<number, typeof all>();
  for (const p of all) {
    if (!byTrip.has(p.tripId)) byTrip.set(p.tripId, []);
    byTrip.get(p.tripId)!.push(p);
  }
  let sent = 0;
  for (const [tripId, points] of byTrip) {

```

```

const chunk = points.slice(0, 200);

try {
  const res = await fetch("/api/trips/points", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      tripId,
      points: chunk.map(({ id, tripId: _t, ...rest }) => {
        void id;
        void _t;
        return rest;
      }),
    }));
  if (res.ok) {
    await removePoints(chunk.map((c) => c.id));
    sent += chunk.length;
  }
} catch {
  break; // —
}

return { sent, pending: (await bufferedPoints()).length };
}

/** () — */
export async function pushLive(p: {
  lat: number;
  lng: number;
  accuracy?: number | null;
  speed?: number | null;
  gpsOn?: boolean;
  tripId?: number | null;
  battery?: number | null;
}) {
  try {
    await fetch("/api/live", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ ...p, recordedAt: new Date().toISOString() });
    });
  } catch {
    /* — */
  }
}

/* ----- */
export type TrackerState = {
  lastPoint: TrackPoint | null;
  distanceM: number;
  stopSeconds: number;
  gpsOn: boolean;
  pending: number;
  points: TrackPoint[];
};

export type TrackerOptions = {
  tripId: number;
  onUpdate: (s: TrackerState) => void;
};

/**
 * . `start()` `stop()` .
 * GPS .
 */
export function createTracker({ tripId, onUpdate }: TrackerOptions) {
  let watchId: number | null = null;
  let flushTimer: ReturnType<typeof setInterval> | null = null;
  let liveTimer: ReturnType<typeof setInterval> | null = null;
  const state: TrackerState = {
    lastPoint: null,
    distanceM: 0,
    stopSeconds: 0,
    gpsOn: true,
    pending: 0,
    points: [],
  };
  let lastMoveAt = Date.now();
  let lastSavedAt = 0;
  const emit = () => onUpdate({ ...state, points: [...state.points] });
  const record = async (p: TrackPoint) => {

    state.points.push(p);
    if (state.points.length > 2000) state.points.shift();
    state.lastPoint = p;
    await bufferPoint(tripId, p);
    state.pending = await bufferedCount();
    emit();
  };
}

```

```

};
const onPosition = async (pos: GeolocationPosition) => {
  state.gpsOn = true;
  const now = Date.now();
  const cur = { lat: pos.coords.latitude, lng: pos.coords.longitude };
  //
  if (!state.lastPoint) {
    lastMoveAt = now;
    lastSavedAt = now;
    await record({
      ...cur,
      accuracy: pos.coords.accuracy,
      speed: pos.coords.speed,
      heading: pos.coords.heading,
      kind: "start",
      note: " ",
      gpsOn: true,
      recordedAt: new Date().toISOString(),
    });
    return;
  }
  const dist = haversine(state.lastPoint, cur);
  //
  if (dist < MIN_DISTANCE_M) {
    const stopped = now - lastMoveAt;
    if (stopped >= STOP_THRESHOLD_MS && now - lastSavedAt >= STOP_THRESHOLD_MS) {
      lastSavedAt = now;
      state.stopSeconds += Math.round(stopped / 1000);
      await record({
        ...cur,
        accuracy: pos.coords.accuracy,
        kind: "pause",
        note: ` ${formatDuration(stopped / 1000)} `,
        stopSeconds: Math.round(stopped / 1000),
        gpsOn: true,
        recordedAt: new Date().toISOString(),
      });
      lastMoveAt = now;
    }
    return;
  }
  //
  state.distanceM += dist;
  lastMoveAt = now;
  lastSavedAt = now;
  await record({
    ...cur,
    accuracy: pos.coords.accuracy,
    speed: pos.coords.speed,
    heading: pos.coords.heading,
    kind: "move",
    gpsOn: true,
    recordedAt: new Date().toISOString(),
  });
};
const onError = () => {
  //GPS —
  state.gpsOn = false;
  emit();
};
const start = () => {
  if (!("geolocation" in navigator)) return false;
  if (watchId !== null) return true;
  watchId = navigator.geolocation.watchPosition(onPosition, onError, {
    enableHighAccuracy: true,
    maximumAge: 15000,
    timeout: 30000,
  });
  //
  ()
  flushTimer = setInterval(async () => {
    const r = await flushPoints();
    state.pending = r.pending;
    emit();
  }, 20000);
  //
  GPS
  liveTimer = setInterval(() => {
    if (state.lastPoint) {
      void pushLive({
        lat: state.lastPoint.lat,
        lng: state.lastPoint.lng,
        accuracy: state.lastPoint.accuracy,
        speed: state.lastPoint.speed,
        gpsOn: state.gpsOn,
        tripId,
      });
    }
  });
};

```

```

    });
  }
}, 30000);
return true;
};
const stop = async (note = " ") => {
  if (watchId !== null) {
    navigator.geolocation.clearWatch(watchId);
    watchId = null;
  }
  if (flushTimer) clearInterval(flushTimer);
  if (liveTimer) clearInterval(liveTimer);
  if (state.lastPoint) {
    await record({ ...state.lastPoint, kind: "end", note, recordedAt: new Date().toISOString() });
  }
  await flushPoints();
};
const markPause = async (note: string) => {
  if (!state.lastPoint) return;
  await record({ ...state.lastPoint, kind: "pause", note, recordedAt: new Date().toISOString() });
  await flushPoints();
};
return { start, stop, markPause, state, flush: flushPoints };
}

@@ src/lib/useLive.ts typescript
"use client";
import { useCallback, useEffect, useRef, useState } from "react";
/**
 * JSON — throw.
 *
 */
export async function fetchJson<T = unknown>(url: string, fallback: T, init?: RequestInit): Promise<T> {
  try {
    const res = await fetch(url, { cache: "no-store", ...init });
    if (!res.ok) return fallback;
    return (await res.json()) as T;
  } catch {
    return fallback;
  }
}
/**
 * API `{ rows: [...] }` */
export async function fetchRows<T = Record<string, unknown>>(url: string): Promise<T[]> {
  const d = await fetchJson<{ rows?: T[] }>(url, {});
  return Array.isArray(d.rows) ? d.rows : [];
}
/**
 * :
 * - `interval`
 * - ( )
 * -
 * - (2G / save-data)
 */
export function useLive(fn: () => void | Promise<void>, interval = 15000, enabled = true) {
  const saved = useRef(fn);
  saved.current = fn;
  const [paused, setPaused] = useState(false);
  const run = useCallback(() => {
    void saved.current();
  }, []);

  useEffect(() => {
    if (!enabled || paused) return;
    type Conn = { effectiveType?: string; saveData?: boolean };
    const conn = (navigator as Navigator & { connection?: Conn }).connection;
    const slow = conn?.saveData || ["slow-2g", "2g"].includes(conn?.effectiveType ?? "");
    const ms = slow ? Math.max(interval * 3, 45000) : interval;
    run();
    let timer: ReturnType<typeof setInterval> | null = null;
    const start = () => {
      if (timer) clearInterval(timer);
      timer = setInterval(() => {
        if (!document.hidden && navigator.onLine) run();
      }, ms);
    };
    const onVisible = () => {
      if (!document.hidden) {
        run();
        start();
      } else if (timer) {
        clearInterval(timer);
        timer = null;
      }
    };
    start();
  });
}

```



```

document.addEventListener("visibilitychange", onVisible);
window.addEventListener("online", run);
window.addEventListener("focus", run);
return () => {
    if (timer) clearInterval(timer);
    document.removeEventListener("visibilitychange", onVisible);
    window.removeEventListener("online", run);
    window.removeEventListener("focus", run);
};
}, [enabled, paused, interval, run]);
return { refresh: run, paused, setPaused };
}

@@ src/lib/zip.ts typescript
import { deflateRawSync } from "node:zlib";
/**
 * =====
 * ZIP
 * -----
 *
 *
 *
 * GitHub / GitLab .
 * =====
 */
const CRC_TABLE = (() => {
    const t = new Uint32Array(256);
    for (let n = 0; n < 256; n++) {
        let c = n;
        for (let k = 0; k < 8; k++) c = c & 1 ? 0xedb88320 ^ (c >> 1) : c >> 1;
        t[n] = c >> 0;
    }
    return t;
})();
function crc32(buf: Buffer): number {
    let c = 0xffffffff;
    for (let i = 0; i < buf.length; i++) c = CRC_TABLE[(c ^ buf[i]) & 0xff] ^ (c >> 8);
    return (c ^ 0xffffffff) >> 0;
}
/** DOS ZIP */
function dosDateTime(d: Date) {
    const time = ((d.getHours() < 11) | (d.getMinutes() < 5) | (d.getSeconds() / 2)) & 0xffff;
    const date = (((d.getFullYear() - 1980) < 9) | ((d.getMonth() + 1) < 5) | d.getDate()) & 0xffff;
    return { time, date };
}
export type ZipEntry = { path: string; content: Buffer | string };
/** ZIP */
export function createZip(entries: ZipEntry[], modified = new Date()): Buffer {
    const { time, date } = dosDateTime(modified);
    const chunks: Buffer[] = [];
    const central: Buffer[] = [];

    let offset = 0;
    for (const entry of entries) {
        const nameBuf = Buffer.from(entry.path.replace(/\\/g, "/"), "utf8");
        const raw = Buffer.isBuffer(entry.content) ? entry.content : Buffer.from(entry.content, "utf8");
        const crc = crc32(raw);
        //
        const deflated = deflateRawSync(raw, { level: 9 });
        const useDeflate = deflated.length < raw.length;
        const data = useDeflate ? deflated : raw;
        const method = useDeflate ? 8 : 0;
        const local = Buffer.alloc(30);
        local.writeUInt32LE(0x04034b50, 0); //
        local.writeUInt16LE(20, 4); //
        local.writeUInt16LE(0x0800, 6); // : UTF-8
        local.writeUInt16LE(method, 8);
        local.writeUInt16LE(time, 10);
        local.writeUInt16LE(date, 12);
        local.writeUInt32LE(crc, 14);
        local.writeUInt32LE(data.length, 18);
        local.writeUInt32LE(raw.length, 22);
        local.writeUInt16LE(nameBuf.length, 26);
        local.writeUInt16LE(0, 28);
        chunks.push(local, nameBuf, data);
        const cd = Buffer.alloc(46);
        cd.writeUInt32LE(0x02014b50, 0); //
        cd.writeUInt16LE(20, 4);
        cd.writeUInt16LE(20, 6);
        cd.writeUInt16LE(0x0800, 8);
        cd.writeUInt16LE(method, 10);
        cd.writeUInt16LE(time, 12);
        cd.writeUInt16LE(date, 14);
        cd.writeUInt32LE(crc, 16);
        cd.writeUInt32LE(data.length, 20);
        cd.writeUInt32LE(raw.length, 24);
    }

```

```

    cd.writeUInt16LE(nameBuf.length, 28);
    cd.writeUInt16LE(0, 30);
    cd.writeUInt16LE(0, 32);
    cd.writeUInt16LE(0, 34);
    cd.writeUInt16LE(0, 36);
    cd.writeUInt32LE(0, 38);
    cd.writeUInt32LE(offset, 42);
    central.push(cd, nameBuf);
    offset += local.length + nameBuf.length + data.length;
}
const centralBuf = Buffer.concat(central);
const end = Buffer.alloc(22);
end.writeUInt32LE(0x06054b50, 0); //
end.writeUInt16LE(0, 4);
end.writeUInt16LE(0, 6);
end.writeUInt16LE(entries.length, 8);
end.writeUInt16LE(entries.length, 10);
end.writeUInt32LE(centralBuf.length, 12);
end.writeUInt32LE(offset, 16);
end.writeUInt16LE(0, 20);
return Buffer.concat([...chunks, centralBuf, end]);
}

@@ src/app/admin/layout.tsx typescript
"use client";
import type { ReactNode } from "react";
import Shell from "@components/Shell";
import SessionProvider, { useSession } from "@components/SessionProvider";
function Inner({ children }: { children: ReactNode }) {
    const { me } = useSession();
    if (!me) return null;
    const has = (p: string) => me.role === "admin" || me.permissions.includes(p);
    const nav = [
        { href: "/admin", label: "", icon: "", perm: "dashboard" },
        { href: "/admin/records/pharmacies", label: "", icon: "", perm: "pharmacy" },
        { href: "/admin/records/doctors", label: "", icon: "", perm: "doctor" },
        { href: "/admin/records/orders", label: "", icon: "", perm: "order" },
        { href: "/admin/activity", label: "", icon: "", perm: "dashboard" },
        { href: "/admin/map", label: "", icon: "", perm: "monitor" },
        { href: "/admin/live", label: "", icon: "", perm: "monitor" },

        { href: "/admin/trips", label: "", icon: "", perm: "monitor" },
        { href: "/admin/homes", label: "", icon: "", perm: "home" },
        { href: "/admin/leaves", label: "", icon: "", perm: "leave" },
        { href: "/admin/notifications", label: "", icon: "" },
        { href: "/admin/reports", label: "", icon: "", perm: "reports" },
        { href: "/admin/targets", label: "", icon: "", perm: "users" },
        { href: "/admin/options", label: "", icon: "", perm: "options" },
        { href: "/admin/columns", label: "", icon: "", perm: "columns" },
        { href: "/admin/users", label: "", icon: "", perm: "users" },
        { href: "/admin/messengers", label: "", icon: "", perm: "messengers" },
        { href: "/admin/backup", label: "", icon: "", perm: "users" },
        { href: "/admin/sync", label: "", icon: "", perm: "users" },
        { href: "/install", label: "", icon: "" },
        { href: "/diagnostics", label: "", icon: "" },
    ]
    .filter((n) => !n.perm || has(n.perm))
    .map(({ href, label, icon }) => ({ href, label, icon }));
    return (
        <Shell
            userName={me.fullName}
            roleLabel={me.role === "admin" ? "" : ""}
            nav={nav}
            showPanelLink
        >
            {children}
        </Shell>
    );
}
export default function AdminLayout({ children }: { children: ReactNode }) {
    return (
        <SessionProvider require adminOnly>
            <Inner>{children}</Inner>
        </SessionProvider>
    );
}

@@ src/app/admin/page.tsx typescript
"use client";
import Link from "next/link";
import { useCallback, useState } from "react";
import { Badge, Card, SectionTitle } from "@components/ui";
import dynamic from "next/dynamic";
const ChartsPanel = dynamic(() => import("@components/screens/ChartsPanel"), {
    ssr: false,

```

```

    loading: () => <div className="rounded-2xl bg-white p-6 text-center text-xs text-slate-400"> ..
  </div>,
});
import { tehranDateTime, toPersianDigits } from "@lib/jalali";
import { useLive } from "@lib/useLive";
import { downloadFile } from "@lib/download";
type Counts = {
  pharmacies: number;
  doctors: number;
  orders: number;
  trips: number;
  activeTrips: number;
  users: number;
  leaves: number;
};
type Log = { id: number; userName: string; action: string; detail: string; createdAt: string };
type Rep = {
  id: number;
  fullName: string;
  role: string;
  active: boolean;
  lastSeenAt: string | null;
  pharmacies: number;
  doctors: number;
  orders: number;
  trips: number;
  activeTrip: number;
  lastAction: string | null;
  lastActionAt: string | null;
};
export default function AdminDashboard() {
  const [counts, setCounts] = useState<Counts | null>(null);
  const [reps, setReps] = useState<Rep[]>([]);
  const [byRep, setByRep] = useState<Record<string, Log[]>>({});
  const [openRep, setOpenRep] = useState<string | null>(null);

  const [live, setLive] = useState(true);
  const [dl, setDl] = useState("");
  const load = useCallback(async () => {
    const res = await fetch("/api/activity", { cache: "no-store" });
    if (!res.ok) return;
    const d = await res.json();
    setCounts(d.counts);
    setReps(d.reps ?? []);
    setByRep(d.byRep ?? {});
  }, []);
  useLive(load, 20000, live);
  const tiles = [
    { label: "", value: counts?.pharmacies ?? 0, icon: "", href: "/admin/records/pharmacies" },
    { label: "", value: counts?.doctors ?? 0, icon: "", href: "/admin/records/doctors" },
    { label: "", value: counts?.orders ?? 0, icon: "", href: "/admin/records/orders" },
    { label: "", value: counts?.activeTrips ?? 0, icon: "", href: "/admin/trips" },
    { label: "", value: counts?.leaves ?? 0, icon: "", href: "/admin/leaves" },
    { label: "", value: counts?.users ?? 0, icon: "", href: "/admin/users" },
  ];
  return (
    <div className="space-y-4">
      <div className="flex items-center justify-between">
        <SectionTitle icon=""></SectionTitle>
        <button
          onClick={() => setLive((v) => !v)}
          className={`rounded-lg px-3 py-1.5 text-xs font-bold ${live ? "bg-emerald-100 text-emerald-700" : "bg-slate-200 text-slate-600"}>
          </button>
      </div>
      <div className="grid grid-cols-2 gap-2 sm:grid-cols-3 lg:grid-cols-6">
        {tiles.map((t) => (
          <Link
            key={t.label}
            href={t.href}
            className="rounded-2xl bg-white p-3 text-center shadow-sm ring-1 ring-slate-200 hover:ring-teal-300">
            <div className="text-xl">{t.icon}</div>
            <div className="text-lg font-black text-teal-700">{toPersianDigits(t.value)}</div>
            <div className="text-[10px] font-bold text-slate-500">{t.label}</div>
          </Link>
        ))}
      </div>
      <Card>
        <div className="mb-2 flex flex-wrap items-center justify-between gap-2">
          <SectionTitle icon=""> — </SectionTitle>
          <Link
            href="/admin/activity"

```

```

        className="rounded-xl bg-teal-600 px-3 py-2 text-xs font-bold text-white hover:bg-teal-700"
    >

    </Link>
</div>
<div className="grid grid-cols-1 gap-2 lg:grid-cols-2">
    {reps.map((r) => {
        const logs = byRep[r.fullName] ?? [];
        const isOpen = openRep === r.fullName;
        return (
            <div key={r.id} className="rounded-2xl bg-slate-50 p-3 ring-1 ring-slate-200">
                <div className="flex flex-wrap items-center gap-2">
                    <span className="font-bold text-slate-800">{r.fullName}</span>
                    <Badge tone={r.role === "admin" ? "green" : r.role === "supervisor" ? "amber" : "slate"}>
                        {r.role === "admin" ? "" : r.role === "supervisor" ? "" : ""}
                    </Badge>
                    {r.activeTrip > 0 ? <Badge tone="green"> </Badge> : null}
                    {!r.active ? <Badge tone="amber"></Badge> : null}
                    <button
                        onClick={() => setOpenRep(isOpen ? null : r.fullName)}
                        className="mr-auto rounded-lg bg-white px-2 py-1 text-[11px] font-bold text-teal-700 ring-1 ring-tea
l-200"
                    >
                        {isOpen ? "" : ` (${toPersianDigits(logs.length)})`}
                    </button>
                </div>
                <div className="mt-2 grid grid-cols-4 gap-1 text-center text-[11px]">
                    {[
                        ["" , r.pharmacies],
                        ["" , r.doctors],

                        ["" , r.orders],
                        ["" , r.trips],
                    ].map(([i, v], idx) => (
                        <div key={idx} className="rounded-lg bg-white py-1 ring-1 ring-slate-200">
                            <span>{i as string}</span> <b className="text-teal-700">{toPersianDigits(v as number)}</b>
                        </div>
                    ))}
                </div>
                <div className="mt-2 text-[11px] text-slate-500">
                    {r.lastAction ?? "—"}
                    {r.lastActionAt ? ` (${tehranDateTime(r.lastActionAt)})` : ""}
                </div>
                <div className="text-[11px] text-slate-400">
                    {r.lastSeenAt ? tehranDateTime(r.lastSeenAt) : "—"}
                </div>
                {isOpen ? (
                    <div className="mt-2 max-h-52 space-y-1 overflow-y-auto">
                        {logs.length === 0 ? (
                            <p className="text-xs text-slate-400"> </p>
                        ) : (
                            logs.map((l) => (
                                <div key={l.id} className="rounded-lg bg-white px-2 py-1 text-[11px] ring-1 ring-slate-100">
                                    <span className="font-bold text-teal-700">{l.action}</span>
                                    {l.detail ? <span className="text-slate-600"> — {l.detail}</span> : null}
                                    <span className="mr-2 text-[10px] text-slate-400">{tehranDateTime(l.createdAt)}</span>
                                </div>
                            ))
                        )}
                    </div>
                ) : null}
            </div>
        );
    })}
</div>
</Card>
<Card>
    <SectionTitle icon=""> </SectionTitle>
    {dl ? <p className={`mb-2 text-xs font-bold ${dl.startsWith("") ? "text-rose-600" : "text-emerald-600"}`}>{dl}<
/p> : null}
    <div className="flex flex-wrap gap-2">
        <a
            href="/admin/targets"
            className="rounded-xl bg-amber-50 px-3 py-2 text-xs font-bold text-amber-800 ring-1 ring-amber-200 hover:bg-
amber-100"
        >

        </a>
        <a
            href="/admin/backup"
            className="rounded-xl bg-indigo-50 px-3 py-2 text-xs font-bold text-indigo-800 ring-1 ring-indigo-200 hover:
bg-indigo-100"
        >

        </a>
    </div>

```

```

    {
      ["pharmacies", ""],
      ["doctors", ""],
      ["orders", ""],
      ["trips", ""],
      ["leaves", ""],
      ["homes", ""],
      ["users", ""],
    ].map(([k, l]) => (
      <button
        key={k}
        onClick={async () => setDl(await downloadFile(`/api/export?type=${k}`, `${k}.xls`))}
        className="rounded-xl bg-emerald-50 px-3 py-2 text-xs font-bold text-emerald-800 ring-1 ring-emerald-200 h
over:bg-emerald-100"
      >
        {l}
      </button>
    ))
  </div>
</Card>
<ChartsPanel scope="admin" />
</div>
);
}

```

```

@@ src/app/apple-touch-icon.png/route.ts typescript
import { pngResponse, renderIcon } from "@lib/pwa-image";
export const dynamic = "force-static";

export const revalidate = false;
export async function GET() {
  const bytes = await renderIcon(180, true);
  if (!bytes) return new Response("not available", { status: 404 });
  return pngResponse(bytes);
}

```

```

@@ src/app/diagnostics/page.tsx typescript
"use client";
import { useCallback, useEffect, useState } from "react";
import Link from "next/link";
import { Alert, Badge, Button, Card, SectionTitle } from "@components/ui";
import { getDeviceId, getDeviceInfo, readSimStatus } from "@lib/device";
import { getCacheUser, getToken } from "@lib/offline-session";
import { toPersianDigits } from "@lib/jalali";
type Row = { key: string; label: string; state: "ok" | "warn" | "fail" | "run"; detail: string };
export default function DiagnosticsPage() {
  const [rows, setRows] = useState<Row[]>([]);
  const [busy, setBusy] = useState(false);
  const [cacheInfo, setCacheInfo] = useState("");
  const [queue, setQueue] = useState<number | null>(null);
  const put = (r: Row) => setRows((p) => [...p.filter((x) => x.key !== r.key), r]);
  const run = useCallback(async () => {
    setBusy(true);
    setRows([]);
    // )
    const sim = readSimStatus();
    type Conn = { effectiveType?: string; type?: string; downlink?: number; rtt?: number; saveData?: boolean };
    const conn = (navigator as Navigator & { connection?: Conn }).connection;
    put({
      key: "net",
      label: " ",
      state: navigator.onLine ? "ok" : "fail",
      detail: navigator.onLine
        ? `: ${conn?.type ?? ""} | : ${conn?.effectiveType ?? "?"} | : ${conn?.downlink ?? "?"}`
        : `: " "`,
    });
    // )
    if ("serviceWorker" in navigator) {
      const reg = await navigator.serviceWorker.getRegistration();
      put({
        key: "sw",
        label: " ( )",
        state: reg?.active ? "ok" : "warn",
        detail: reg?.active
          ? " — "
          : " ",
      });
    }
  } else {
    put({ key: "sw", label: "", state: "fail", detail: " " });
  }
  // )
  try {
    const names = await caches.keys();
    let total = 0;

```

```

for (const n of names) {
  const c = await caches.open(n);
  total += (await c.keys()).length;
}
put({
  key: "cache",
  label: " ",
  state: total > 30 ? "ok" : total > 0 ? "warn" : "fail",
  detail:
    total > 30
    ? `${toPersianDigits(total)} — `
    : ` ${toPersianDigits(total)} `,
});
setCacheInfo(`${total} ${names.length} `);
} catch {
  put({ key: "cache", label: " ", state: "fail", detail: " " });
}
// ) ( )
const t0 = performance.now();

try {
  const ctrl = new AbortController();
  const to = setTimeout(() => ctrl.abort(), 20000);
  const r = await fetch(`/api/ping?t=${Date.now()}`, { cache: "no-store", signal: ctrl.signal });
  clearTimeout(to);
  const ms = Math.round(performance.now() - t0);
  put({
    key: "ping",
    label: " ",
    state: r.ok ? (ms < 3000 ? "ok" : "warn") : "fail",
    detail: r.ok
    ? `${toPersianDigits(ms)} ${ms > 3000 ? " ( — )": ""}`
    : ` ${r.status}`,
  });
} catch {
  put({
    key: "ping",
    label: " ",
    state: "fail",
    detail: " ".
  },
  });
}
// )
try {
  const ctrl = new AbortController();
  const to = setTimeout(() => ctrl.abort(), 25000);
  const r = await fetch(`/api/health?t=${Date.now()}`, { cache: "no-store", signal: ctrl.signal });
  clearTimeout(to);
  const d = await r.json().catch(() => ({}));
  put({
    key: "db",
    label: " ",
    state: d?.ok ? "ok" : "fail",
    detail: d?.ok ? " " : d?.reason ?? " ",
  });
} catch {
  put({ key: "db", label: " ", state: "fail", detail: " (Timeout)" });
}
// )
const cached = getCacheUser();
put({
  key: "session",
  label: " ( )",
  state: cached ? "ok" : "warn",
  detail: cached
  ? `${cached.fullName} — `
  : " ",
});
// )
const id = getDeviceId();
put({
  key: "device",
  label: " ( )",
  state: id ? "ok" : "fail",
  detail: id ? `${getDeviceInfo() | : ${id.slice(0, 10)}...` : " ",
});
// )
put({
  key: "geo",
  label: " (GPS)",
  state: "geolocation" in navigator ? (window.isSecureContext ? "ok" : "warn") : "fail",
  detail:
    "geolocation" in navigator
    ? window.isSecureContext

```

```

        ? " "
        : " http      https "
      : " ",
    });
    // )
    put({
      key: "sim",
      label: " / ",
      state: sim.ok ? "ok" : "warn",
      detail: sim.ok ? ` : ${sim.type}` : sim.detail,
    });
    // )
    put({
      key: "token",

      label: " ",
      state: getToken() ? "ok" : "warn",
      detail: getToken() ? " " : " ",
    });
    setBusy(false);
  }, []);
  useEffect(() => {
    run();
    const onMsg = (e: MessageEvent) => {
      if (e.data?.type === "queue-status") setQueue(e.data.pending);
      if (e.data?.type === "cache-status") setCacheInfo(`${e.data.cached}  ${e.data.expected} `);
    };
    navigator.serviceWorker?.addEventListener("message", onMsg);
    navigator.serviceWorker?.controller?.postMessage("queue-count");
    navigator.serviceWorker?.controller?.postMessage("cache-status");
    return () => navigator.serviceWorker?.removeEventListener("message", onMsg);
  }, [run]);
  const icon = (s: Row["state"]) => (s === "ok" ? "" : s === "warn" ? "" : s === "run" ? "" : "");
  const tone = (s: Row["state"]) =>
    s === "ok" ? "bg-emerald-50 ring-emerald-200" : s === "warn" ? "bg-amber-50 ring-amber-200" : "bg-rose-50 ring-rose-200";
  const fails = rows.filter((r) => r.state === "fail").length;
  return (
    <main className="mx-auto max-w-2xl space-y-4 p-4">
      <div className="flex flex-wrap items-center justify-between gap-2">
        <SectionTitle icon=""> </SectionTitle>
        <div className="flex gap-2">
          <Button onClick={run} disabled={busy}>
            {busy ? " ..." : " "}
          </Button>
          <Link href="/" className="rounded-xl bg-white px-3 py-2.5 text-xs font-bold text-slate-700 ring-1 ring-slate-300">
            </Link>
          </div>
        </div>
        {fails > 0 ? (
          <Alert kind="error">
            {toPersianDigits(fails)} .
          </Alert>
        ) : rows.length > 0 && !busy ? (
          <Alert kind="success"> — </Alert>
        ) : null}
        <Card>
          <div className="space-y-2">
            {rows.map((r) => (
              <div key={r.key} className={`rounded-xl px-3 py-2 ring-1 ${tone(r.state)}`}>
                <div className="flex items-start gap-2">
                  <span>{icon(r.state)}</span>
                  <div className="min-w-0 flex-1">
                    <div className="text-xs font-black text-slate-800">{r.label}</div>
                    <div className="mt-0.5 text-[11px] leading-5 text-slate-600">{r.detail}</div>
                  </div>
                </div>
              </div>
            ))}
            {rows.length === 0 ? <p className="py-6 text-center text-xs text-slate-400"> ...</p> : null}
          </div>
        </Card>
        <Card>
          <h3 className="mb-2 text-sm font-bold text-slate-700"> </h3>
          <div className="flex flex-wrap gap-2">
            <Button
              variant="soft"
              onClick={() => {
                navigator.serviceWorker?.controller?.postMessage("flush");
                setTimeout(run, 1500);
              }}
            >
              {queue ? `(${toPersianDigits(queue)})` : ""}
            </Button>
          </div>
        </Card>
      </main>

```



```

</p>
<Button onClick={install} className="w-full sm:w-auto">

</Button>
</Card>
<Card>
  <h3 className="mb-2 text-sm font-black text-slate-800"> </h3>
  <ol className="list-inside list-decimal space-y-1 text-xs leading-6 text-slate-600">
    <li> Chrome .</li>
  </ol>
  <li> « » «Install app / ».</li>
  <li> .</li>
  </ol>
</Card>
<Card>
  <h3 className="mb-2 text-sm font-black text-slate-800"> (iOS)</h3>
  <ol className="list-inside list-decimal space-y-1 text-xs leading-6 text-slate-600">
    <li> Safari .</li>
    <li> ( ) .</li>
  </ol>
  <li> «Add to Home Screen» .</li>
  </ol>
</Card>
<Card>
  <h3 className="mb-2 text-sm font-black text-slate-800"> </h3>
  <ol className="list-inside list-decimal space-y-1 text-xs leading-6 text-slate-600">
    <li> Chrome Edge .</li>
  </ol>
  <li> (⊕) → «Install».</li>
  <li> Start .</li>
  </ol>
</Card>
<Card>
  <h3 className="mb-2 text-sm font-black text-slate-800"> APK </h3>
  <Alert kind="success">
    PWABuilder : (PNG) maskable
    (id) manifest.
  </Alert>
  <ol className="mt-2 list-inside list-decimal space-y-1 text-xs leading-6 text-slate-600">
    <li>
      {" "}
      <a href="https://www.pwabuilder.com" target="_blank" rel="noreferrer" className="font-bold text-teal-700 und
erline">
        PWABuilder.com
      </a>{" "}
      <b> </b> ( login) .
    </li>
  </ol>
  <li> Refresh/Retest manifest — .</li>
  <li> «Package For Stores» → Android APK/AAB .</li>
  <li> «Download Test Package» APK .</li>
  </ol>
  <div className="mt-3 flex flex-wrap gap-2">
    <a
      href="https://www.pwabuilder.com/reportcard?site="
      target="_blank"
      rel="noreferrer"
      className="rounded-xl bg-teal-600 px-3 py-2 text-xs font-bold text-white"
      onClick={(e) => {
        e.preventDefault();
        window.open(`https://www.pwabuilder.com/reportcard?site=${window.location.origin}/`, "_blank");
      }}
    >
      APK PWABuilder
    </a>
    <a
      href="/manifest.webmanifest"
      target="_blank"
      rel="noreferrer"
      className="rounded-xl bg-white px-3 py-2 text-xs font-bold text-slate-700 ring-1 ring-slate-300"
    >
      manifest
    </a>
  </div>
</Card>
<Card>
  <h3 className="mb-2 text-sm font-black text-slate-800"> </h3>
  <div className="flex flex-wrap items-center gap-4">
    </* eslint-disable-next-line @next/next/no-img-element */>
    
    </* eslint-disable-next-line @next/next/no-img-element */>
    
  <div className="text-xs leading-6 text-slate-600">

    maskable apple-touch-icon .

  </div>
</div>
/>

```

```

    </Card>
  </main>
);
}

```

```

@@ src/app/login/LoginForm.tsx typescript
"use client";
import { useEffect, useState } from "react";
import { useRouter } from "next/navigation";
import { Alert, Button, Field, Input } from "@components/ui";
import { toPersianDigits } from "@lib/jalali";
import { patchFetch, setTabToken } from "@components/SessionProvider";
import { cacheUser, getCacheUser } from "@lib/offline-session";
import {
  getDeviceId,
  getDeviceInfo,
  getDeviceSim,
  isMobileDevice,
  maskPhone,
  normalizePhone,
  setDeviceSim,
  watchSim,
  type SimStatus,
} from "@lib/device";
export default function LoginForm() {
  const router = useRouter();
  const [mode, setMode] = useState<"rep" | "admin">("rep");
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");
  const [showPass, setShowPass] = useState(false);
  const [phone, setPhone] = useState("");
  const [needPhone, setNeedPhone] = useState(true);
  const [remember, setRemember] = useState(false);
  const [error, setError] = useState("");
  const [hint, setHint] = useState("");
  const [busy, setBusy] = useState(false);
  const [simOk, setSimOk] = useState(true);
  const [simWarn, setSimWarn] = useState("");
  const [forgot, setForgot] = useState(false);
  const [simMismatch, setSimMismatch] = useState("");
  const [regMask, setRegMask] = useState("");
  const [deviceBound, setDeviceBound] = useState(false);
  const [netType, setNetType] = useState("");
  const [otpStep, setOtpStep] = useState(false);
  const [otpCode, setOtpCode] = useState("");
  const [otpMsg, setOtpMsg] = useState("");
  const [otpBusy, setOtpBusy] = useState(false);
  const [otpLeft, setOtpLeft] = useState(0);
  const [info, setInfo] = useState("");
  useEffect(() => {
    patchFetch();
    const saved = localStorage.getItem("sek_phone");
    if (saved) setPhone(saved);
    const stop = watchSim((s: SimStatus) => {
      setSimOk(s.ok);
      setSimWarn(s.ok ? "" : s.detail);
      setNetType(
        s.type === "cellular"
          ? " (Cellular)"
          : s.type === "wifi"
            ? ""
            : s.online
              ? ""
              : "",
      );
    });
    return stop;
  }, []);
  //
  useEffect(() => {
    if (mode !== "rep" || !needPhone) {
      setSimMismatch("");
      return;
    }
    const entered = normalizePhone(phone);
    if (entered.length < 11) {
      setSimMismatch("");
      return;
    }
    // )
    if (regMask && maskPhone(entered) !== regMask) {
      setSimMismatch(
        ` «${regMask}» «${maskPhone(entered)}»
      `);
    }
  },

```

```

    );
    return;
}
// )
const deviceSim = getDeviceSim();
if (deviceSim && entered !== deviceSim) {
    setSimMismatch(
        «${maskPhone(deviceSim)}» . . . ,
    );
    return;
}
setSimMismatch("");
}, [phone, mode, needPhone, regMask]);
//
useEffect(() => {
    const u = username.trim();
    if (u.length < 2) {
        setHint("");
        setNeedPhone(true);
        return;
    }
    const t = setTimeout(async () => {
        const res = await fetch("/api/auth/check-username", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ username: u }),
        }).catch(() => null);
        if (!res?.ok) return;
        const d = await res.json();
        if (!d.exists) {
            setHint("");
            setNeedPhone(true);
            setRegMask("");
            setDeviceBound(false);
            return;
        }
        setNeedPhone(Boolean(d.requirePhone));
        setRegMask(d.phoneMask ?? "");
        setDeviceBound(Boolean(d.deviceBound));
        if (d.kind !== mode) {
            setHint(
                d.kind === "admin"
                ? " / — « » ."
                : " — « » .",
            );
        } else setHint("");
    }, 400);
    return () => clearTimeout(t);
}, [username, mode]);
const requestOtp = async () => {
    setOtpBusy(true);
    const res = await fetch("/api/auth/otp", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ action: "request", username, password, deviceId: getDeviceId() }),
    }).catch(() => null);
    setOtpBusy(false);
    const d = await res?.json().catch(() => ({}));
    setOtpMsg(d?.message ?? d?.error ?? " ");
    if (d?.ok) {
        setOtpLeft(180);
        const iv = setInterval(() => {
            setOtpLeft((v) => {
                if (v <= 1) {
                    clearInterval(iv);
                    return 0;
                }
                return v - 1;
            });
        }, 1000);
    }
};
const verifyOtp = async () => {
    if (otpCode.trim().length < 4) {
        setOtpMsg(" ");
        return;
    }
    setOtpBusy(true);
    const res = await fetch("/api/auth/otp", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
            action: "verify",
            username,

```

```

        password,
        code: otpCode.trim(),
        deviceId: getDeviceId(),
        deviceInfo: getDeviceInfo(),
        remember: true,
    }},
    });
    setOtpBusy(false);
    const d = await res.json().catch(() => ({}));
    if (!res.ok) {
        setOtpMsg(d.error ?? " ");
        return;
    }
    if (d.token) setTabToken(d.token, true);
    if (d.user) cacheUser(d.user);
    if (phone) {
        localStorage.setItem("sek_phone", phone);
        setDeviceSim(normalizePhone(phone));
    }
    router.replace(d.user?.role === "rep" ? "/panel" : "/admin");
};
const submit = async (e: React.FormEvent) => {
    e.preventDefault();
    setError("");
    if (mode === "rep" && needPhone && !simOk) {
        setError(simWarn || " ");
        return;
    }
    if (mode === "rep" && needPhone && simMismatch) {
        setError(simMismatch);
        return;
    }
    if (needPhone && !/^0\d{10}$/.test(phone.replace(/\D/g, ""))) {
        setError(" (: )");
        return;
    }
    // :
    if (!navigator.onLine) {
        const cached = getCachedUser();
        if (cached && cached.username === username.trim().toLowerCase()) {
            router.replace(cached.role === "rep" ? "/panel" : "/admin");
            return;
        }
        setError(" ");
        return;
    }
    setBusy(true);
    const res = await fetch("/api/auth/login", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
            username,
            password,
            phone,
            mode,
            remember,
            simActive: navigator.onLine,
            simActiveOnDevice: simOk && !simMismatch,
            deviceSim: getDeviceSim(),
            deviceId: getDeviceId(),
            deviceInfo: getDeviceInfo(),
            isMobile: isMobileDevice(),
        })),
    });
    setBusy(false);
    const data = await res.json().catch(() => ({}));
    if (!res.ok) {
        if (data.needOtp) {
            setError("");
            setOtpMsg(data.error ?? "");
            setOtpStep(true);
            requestOtp();
            return;
        }
        setError(data.error ?? " ");
        if (data.simError) setSimMismatch(data.error ?? "");
        return;
    }
    if (data.token) setTabToken(data.token, true);
    if (data.user) cacheUser(data.user);
    if (phone) {
        localStorage.setItem("sek_phone", phone);
        //
        if (data.user?.role === "rep") setDeviceSim(normalizePhone(phone));
    }

```

```

    }
    router.replace(data.user?.role === "rep" ? "/panel" : "/admin");
  };
  return (
    <main className="flex min-h-screen items-center justify-center bg-gradient-to-b from-teal-700 via-teal-600 to-slate-
100 p-4">
      <div className="w-full max-w-md">
        {otpStep ? (
          <div className="fixed inset-0 z-50 flex items-center justify-center bg-slate-900/70 p-4">
            <div className="fade-in w-full max-w-sm rounded-2xl bg-white p-4">
              <h3 className="mb-1 text-sm font-black text-slate-800"> </h3>
              <p className="mb-3 text-[11px] leading-6 text-slate-600">
                <b> </b>
              </p>
              {otpMsg ? (
                <div className="mb-2">
                  <Alert kind={otpMsg.includes(" ") ? "success" : "info"}>{otpMsg}</Alert>
                </div>
              ) : null}
              <Field label=" " required>
                <Input
                  inputMode="numeric"
                  value={otpCode}
                  onChange={(e) => setOtpCode(e.target.value.replace(/\D/g, "").slice(0, 6))}
                  placeholder=" _ _ _ _ "
                  className="text-center text-lg tracking-[0.5em]"
                  autoFocus
                />
              </Field>
              <div className="mt-3 flex flex-wrap gap-2">
                <Button onClick={verifyOtp} disabled={otpBusy}>
                  {otpBusy ? "... " : " "}
                </Button>
                <Button variant="soft" onClick={requestOtp} disabled={otpBusy || otpLeft > 0}>
                  {otpLeft > 0 ? ` (${toPersianDigits(otpLeft)})` : " "}
                </Button>
                <Button
                  variant="ghost"
                  onClick={() => {
                    setOtpStep(false);
                    setOtpCode("");
                    setOtpMsg("");
                  }}
                />
              </div>
            </div>
          </div>
        ) : null}
        {forgot ? (
          <div
            className="fixed inset-0 z-50 flex items-center justify-center bg-slate-900/60 p-4"
            onClick={() => setForgot(false)}
          >
            <div
              className="fade-in w-full max-w-sm rounded-2xl bg-white p-4"
              onClick={(e) => e.stopPropagation()}
            >
              <h3 className="mb-2 text-sm font-black text-slate-800"> </h3>
              <p className="mb-3 text-[11px] text-slate-500">
                .
              </p>
              {info ? (
                <div className="mb-2">
                  <Alert kind="success">{info}</Alert>
                </div>
              ) : null}
              <div className="space-y-2">
                <Field label=" " required>
                  <Input value={username} onChange={(e) => setUsername(e.target.value)} className="text-left" />
                </Field>
                <Field label=" ">
                  <Input
                    inputMode="numeric"
                    value={phone}
                    onChange={(e) => setPhone(e.target.value.replace(/\D/g, "").slice(0, 11))}
                  />
                </Field>
              </div>
            </div>
          <div className="mt-3 flex gap-2">
            <Button

```

```

        onClick={async () => {
            const res = await fetch("/api/auth/forgot", {
                method: "POST",
                headers: { "Content-Type": "application/json" },
                body: JSON.stringify({ username, phone }),
            });
            const d = await res.json().catch(() => ({}));
            setInfo(d.message ?? d.error ?? " ");
        }}
    >

    </Button>
    <Button variant="ghost" onClick={() => setForgot(false)}>

    </Button>
  </div>
</div>
</div>
) : null}
<div className="mb-5 text-center text-white">
  <div className="relative mx-auto mb-3 size-24">
    <div className="absolute inset-1 rounded-[1.75rem] bg-white/20 blur-lg" />
    <div className="relative flex size-24 items-center justify-center overflow-hidden rounded-[1.75rem] bg-white shadow-2xl ring-1 ring-white/70">
      
    </div>
    <span className="absolute -bottom-1 -left-1 flex size-7 items-center justify-center rounded-full bg-emerald-400 text-xs shadow-lg ring-2 ring-teal-700">

    </span>
  </div>
  <h1 className="text-2xl font-black drop-shadow-sm"> </h1>
  <p className="mt-1 text-xs font-medium text-teal-50/90"> </p>
  <div className="mx-auto mt-2 flex w-fit items-center gap-2 rounded-full bg-white/10 px-3 py-1 text-[10px] text-teal-50 ring-1 ring-white/20">
    <span></span><span className="opacity-50">•</span><span></span><span className="opacity-50">•</span>
  </div>
  <div>
    <form onSubmit={submit} className="space-y-3 rounded-3xl bg-white p-5 shadow-xl ring-1 ring-slate-200">
      <div className="flex rounded-xl bg-slate-100 p-1 text-xs font-bold">
        <button
          type="button"
          onClick={() => {
            setMode("rep");
            setUsername("");
            setPassword("");
            setError("");
          }}
          className={`flex-1 rounded-lg py-2 ${mode === "rep" ? "bg-teal-600 text-white" : "text-slate-600"}`>
        </button>
        <button
          type="button"
          onClick={() => {
            setMode("admin");
            setUsername("");
            setPassword("");
            setError("");
          }}
          className={`flex-1 rounded-lg py-2 ${mode === "admin" ? "bg-slate-800 text-white" : "text-slate-600"}`>
        </button>
      </div>

      {mode === "rep" && simWarn ? <Alert kind="error">{simWarn}</Alert> : null}
      {mode === "rep" && simMismatch ? <Alert kind="error">{simMismatch}</Alert> : null}
      {hint ? <Alert kind="info">{hint}</Alert> : null}
      {error ? <Alert kind="error">{error}</Alert> : null}
      <Field label=" " required>
        <Input
          value={username}
          onChange={(e) => setUsername(e.target.value)}
          placeholder={mode === "admin" ? " " : " "}
          autoComplete="off"
        />
      </Field>
    </form>
  </div>

```

```

    />
  </Field>
  <Field label=" " required>
    <div className="relative">
      <Input
        type={showPass ? "text" : "password"}
        value={password}
        onChange={(e) => setPassword(e.target.value)}
placeholder="....."
        autoComplete="off"
        className="pl-12"
      />
      <button
        type="button"
        onClick={() => setShowPass((v) => !v)}
        className="absolute left-2 top-1/2 -translate-y-1/2 rounded-lg bg-slate-100 px-2 py-1 text-xs"
        title={showPass ? " " : " "}
      >
        {showPass ? "" : ""}
      </button>
    </div>
  </Field>
  {mode === "rep" && needPhone ? (
    <div
      className={`rounded-xl px-3 py-2 text-[11px] font-bold ring-1 ${
        simMismatch
          ? "bg-rose-50 text-rose-800 ring-rose-300"
          : !simOk
            ? "bg-amber-50 text-amber-800 ring-amber-300"
            : "bg-emerald-50 text-emerald-800 ring-emerald-200"
        }`}
    >
      <div className="flex flex-wrap items-center gap-2">
        <span>{simMismatch ? "" : simOk ? "" : ""} </span>
        {netType ? <span className="rounded bg-white/70 px-2 py-0.5">: {netType}</span> : null}
        {regMask ? <span className="rounded bg-white/70 px-2 py-0.5">: {regMask}</span> : null}
        {deviceBound ? <span className="rounded bg-white/70 px-2 py-0.5"> </span> : null}
      </div>
      {!simMismatch && simOk ? (
        <div className="mt-1 font-normal">
          .
        </div>
      ) : null}
    </div>
  ) : null}
  {needPhone ? (
    <Field label=" " required hint="
      <Input
        inputMode="numeric"
        value={phone}
        onChange={(e) => setPhone(e.target.value.replace(/[^\d]/g, "").slice(0, 11))}
        placeholder="09xxxxxxxxx"
      />
    </Field>
  ) : (
    <Alert kind="info"> .</Alert>
  )}
  <div className="flex items-center justify-between">
    <button
      type="button"
      onClick={() => {
        setForgot(true);
        setInfo("");
      }}
      className="text-[11px] font-bold text-teal-700 underline"
    >
      </button>

    </div>
    <label className="flex items-center gap-2 text-xs font-bold text-slate-600">
      <input
        type="checkbox"
        className="size-4 accent-teal-600"
        checked={remember}
        onChange={(e) => setRemember(e.target.checked)}
      />
      (
        )
    </label>
    <Button type="submit" disabled={busy || (mode === "rep" && needPhone && !simMismatch)} className="w-full">
      {busy ? " ..." : mode === "admin" ? " " : " "}
    </Button>
    <div className="pt-1 text-center">
      <a href="/diagnostics" className="text-[11px] font-bold text-teal-700 underline">

```

```

        </a>
      </div>
      <p className="pt-1 text-center text-[11px] leading-5 text-slate-400">
: admin / admin1234 — : sarparast / sar1234 — : rep1 / rep1234
      </p>
    </form>
  </div>
</main>
);
}

@@ src/app/login/page.tsx typescript
"use client";

import React, { useEffect } from "react";

/**
 * Next.js Login & CRM Portal Page (/login)
 * نام‌های نمایشی در این فایل برای مقاصد آموزشی و نمونه‌سازی می‌باشد.
 * در صورت نیاز به تغییرات، لطفاً به فایل‌های اصلی مراجعه کنید.
 */
export default function LoginPage() {
  useEffect(() => {
    document.title = "CRM Medical Representative Portal";
  }, []);

  return (
    <div style={{ width: "100%", height: "100vh", margin: 0, padding: 0, overflow: "hidden", backgroundColor: "#0f766e" }}>
      <iframe
        src="/index.html#login"
        title="CRM Medical Representative Portal"
        style={{
          width: "100%",
          height: "100%",
          border: "none",
          display: "block",
        }}
        allow="geolocation; microphone; camera"
      />
    </div>
  );
}

@@ src/app/offline/page.tsx typescript
"use client";
import { useCallback, useEffect, useState } from "react";
import { getCachedUser } from "@lib/offline-session";

/**
 * « » — :
 * -
 * - « »
 */
export default function OfflinePage() {
  const [checking, setChecking] = useState(false);
  const [tries, setTries] = useState(0);
  const [target, setTarget] = useState("/login");
  useEffect(() => {
    const u = getCachedUser();
    if (u) setTarget(u.role === "rep" ? "/panel" : "/admin");
  }, []);
  const check = useCallback(
    async (auto = false) => {
      setChecking(true);
      if (!auto) setTries((t) => t + 1);
      try {
        const ctrl = new AbortController();
        const timer = setTimeout(() => ctrl.abort(), 8000);
        const res = await fetch(`/ping?t=${Date.now()}`, { cache: "no-store", signal: ctrl.signal });
        clearTimeout(timer);
        if (res.ok) {
          window.location.replace(target);
          return true;
        }
      } catch {
        /* */
      } finally {
        setChecking(false);
      }
      return false;
    },
    [target],
  );
  useEffect(() => {

```



```

const iv = setInterval(() => check(true), 8000);
const onOnline = () => check(true);

window.addEventListener("online", onOnline);
return () => {
  clearInterval(iv);
  window.removeEventListener("online", onOnline);
};
}, [check]);
return (
  <main className="flex min-h-screen flex-col items-center justify-center gap-4 bg-slate-100 p-6 text-center">
    <div className="text-5xl"></div>
    <h1 className="text-lg font-black text-slate-800">    </h1>
    <p className="max-w-sm text-sm leading-7 text-slate-600">
      . —
    </p>
    <div className="flex flex-wrap items-center justify-center gap-2">
      <button
        onClick={() => check(false)}
        disabled={checking}
        className="rounded-xl bg-teal-600 px-5 py-2.5 text-sm font-bold text-white disabled:opacity-60"
      >
        {checking ? "  ..." : " "}
      </button>
      <a href={target} className="rounded-xl bg-white px-5 py-2.5 text-sm font-bold text-teal-700 ring-1 ring-teal-300"
">

      </a>
      <a href="/diagnostics" className="rounded-xl bg-white px-5 py-2.5 text-sm font-bold text-slate-700 ring-1 ring-slate-300">

      </a>
    </div>
    <p className="text-[11px] text-slate-400">
      {tries > 0 ? `— ${tries.toLocaleString("fa-IR")} ` : ""}
    </p>
    <div className="mt-2 max-w-sm rounded-xl bg-white p-3 text-right text-[11px] leading-6 text-slate-500 ring-1 ring-slate-200">
      <b className="text-slate-700">          </b>
    </div>
  </main>
);
}

@@ src/app/panel/layout.tsx typescript
"use client";
import type { ReactNode } from "react";
import Shell, { type NavItem } from "@components/Shell";
import SessionProvider, { useSession } from "@components/SessionProvider";
function Inner({ children }: { children: ReactNode }) {
  const { me } = useSession();
  if (!me) return null;
  const has = (p: string) => me.role === "admin" || me.permissions.includes(p);
  const all: (NavItem & { perm?: string })[] = [
    { href: "/panel", label: " ", icon: "" },
    { href: "/panel/pharmacies", label: " ", icon: "", perm: "pharmacy" },
    { href: "/panel/doctors", label: " ", icon: "", perm: "doctor" },
    { href: "/panel/orders", label: " ", icon: "", perm: "order" },
    { href: "/panel/trip", label: " ", icon: "", perm: "trip" },
    { href: "/panel/map", label: " ", icon: "" },
    { href: "/panel/home", label: " ", icon: "", perm: "home" },
    { href: "/panel/leaves", label: "", icon: "", perm: "leave" },
    { href: "/panel/notifications", label: "", icon: "" },
    { href: "/panel/options", label: "", icon: "", perm: "options" },
    { href: "/panel/reports", label: " ", icon: "", perm: "reports" },
    { href: "/install", label: " ", icon: "" },
    { href: "/diagnostics", label: "", icon: "" },
  ];
  const nav = all.filter((n) => !n.perm || has(n.perm)).map(({ href, label, icon }) => ({ href, label, icon }));
  return (
    <Shell
      userName={me.fullName}
      roleLabel={me.role === "admin" ? "" : me.role === "supervisor" ? "" : " "}
      nav={nav}
      showAdminLink={me.role !== "rep"}
    >
      {children}
    </Shell>
  );
}
export default function PanelLayout({ children }: { children: ReactNode }) {

```

```

    return (
      <SessionProvider require>
        <Inner>{children}</Inner>
      </SessionProvider>
    );
  }

  @@ src/app/panel/page.tsx typescript
  "use client";
  import Link from "next/link";
  import { useEffect, useState } from "react";
  import { useSession } from "@components/SessionProvider";
  import dynamic from "next/dynamic";
  const TargetPanel = dynamic(() => import("@components/screens/TargetPanel"), { ssr: false });
  const ChartsPanel = dynamic(() => import("@components/screens/ChartsPanel"), {
    ssr: false,
    loading: () => <div className="rounded-2xl bg-white p-6 text-center text-xs text-slate-400"> ..
  </div>,
  });
  import { toPersianDigits, todayJalali } from "@lib/jalali";
  export default function PanelHome() {
    const { me } = useSession();
    const [counts, setCounts] = useState({ ph: 0, dr: 0, or: 0 });
    useEffect(() => {
      fetch("/api/reports", { cache: "no-store" })
        .then((r) => (r.ok ? r.json() : null))
        .then((d) => {
          const rows = (d?.rows ?? []) as { pharmacies: number; doctors: number; orders: number }[];
          setCounts({
            ph: rows.reduce((a, r) => a + r.pharmacies, 0),
            dr: rows.reduce((a, r) => a + r.doctors, 0),
            or: rows.reduce((a, r) => a + r.orders, 0),
          });
        })
        .catch(() => undefined);
    }, []);
    if (!me) return null;
    const has = (p: string) => me.role === "admin" || me.permissions.includes(p);
    const main = [
      { href: "/panel/pharmacies", title: " ", icon: "", n: counts.ph, perm: "pharmacy", color: "from-teal-600 to-teal-500" },
      { href: "/panel/doctors", title: " ", icon: "", n: counts.dr, perm: "doctor", color: "from-sky-600 to-sky-500" },
      { href: "/panel/orders", title: " ", icon: "", n: counts.or, perm: "order", color: "from-indigo-600 to-indigo-500" },
    ].filter((c) => has(c.perm));
    const more = [
      { href: "/panel/trip", title: " ", icon: "", perm: "trip" },
      { href: "/panel/home", title: " ", icon: "", perm: "home" },
      { href: "/panel/leaves", title: " ", icon: "", perm: "leave" },
      { href: "/panel/notifications", title: "", icon: "", perm: "" },
      { href: "/panel/options", title: " ", icon: "", perm: "options" },
      { href: "/panel/reports", title: " ", icon: "", perm: "reports" },
    ].filter((c) => !c.perm || has(c.perm));
    return (
      <div className="space-y-4">
        <div className="rounded-2xl bg-white p-4 shadow-sm ring-1 ring-slate-200">
          <h1 className="text-lg font-black text-slate-800"> {me.fullName} </h1>
          <p className="mt-1 text-sm text-slate-500"> {toPersianDigits(todayJalali())} </p>
        </div>
        <div className="grid grid-cols-1 gap-3 sm:grid-cols-2 lg:grid-cols-3">
          {main.map((c) => (
            <Link
              key={c.href}
              href={c.href}
              className={`flex items-center gap-4 rounded-2xl bg-gradient-to-l ${c.color} p-4 text-white shadow-md active:scale-[0.99]`}
            >
              <span className="flex size-12 shrink-0 items-center justify-center rounded-2xl bg-white/20 text-2xl">
                {c.icon}
              </span>
              <span className="min-w-0">
                <span className="block text-sm font-black">{c.title}</span>
                <span className="mt-0.5 block text-[11px] text-white/85">{toPersianDigits(c.n)} </span>
              </span>
            </Link>
          ))}
        </div>
        <div className="grid grid-cols-2 gap-3 sm:grid-cols-3 lg:grid-cols-6">
          {more.map((c) => (
            <Link
              key={c.href}
              href={c.href}
              className="flex flex-col items-center gap-2 rounded-2xl bg-white p-4 text-center shadow-sm ring-1 ring-slate

```

```

-200 hover:ring-teal-300"
    >
    <span className="text-2xl">{c.icon}</span>
    <span className="text-xs font-bold text-slate-700">{c.title}</span>
  </Link>
  )}}
</div>
{has("order") ? <TargetPanel /> : null}
{has("reports") ? <ChartsPanel scope="rep" /> : null}
{main.length + more.length === 0 ? (
  <div className="rounded-2xl bg-amber-50 p-4 text-sm text-amber-800 ring-1 ring-amber-200">

    </div>
  ) : null}
</div>
);
}

@@ src/app/ping/route.ts typescript
/**
 * /ping — Keep-Alive
 *
 * (UptimeRobot BetterStack cron-job.org):
 * URL : https://<your-app>.onrender.com/ping
 * Interval :
 */
export const dynamic = "force-dynamic";
export const revalidate = 0;
const startedAt = Date.now();
const HEADERS = {
  "Content-Type": "text/plain; charset=utf-8",
  "Cache-Control": "no-store, no-cache, must-revalidate, max-age=0",
  "X-Robots-Tag": "noindex",
  "Access-Control-Allow-Origin": "*",
} as const;
export async function GET(req: Request) {
  if (!new URL(req.url).searchParams.has("json")) {
    return new Response("OK", { status: 200, headers: HEADERS });
  }
  return new Response(
    JSON.stringify({ status: "OK", uptimeMs: Date.now() - startedAt, timestamp: new Date().toISOString() }),
    { status: 200, headers: { ...HEADERS, "Content-Type": "application/json; charset=utf-8" } },
  );
}
export async function HEAD() {
  return new Response(null, { status: 200, headers: HEADERS });
}

@@ src/components/screens/ChartsPanel.tsx typescript
"use client";
import { useCallback, useEffect, useMemo, useState } from "react";
import { Button, Card, Field, SectionTitle } from "@components/ui";
import { BarChart, DonutChart, LineChart } from "@components/Charts";
import JalaliDateInput from "@components/JalaliDateInput";
import { JALALI_MONTHS, toPersianDigits, todayJalali } from "@lib/jalali";
import { Alert } from "@components/ui";
import { DEFAULT_PRODUCTS, type ProductConfig } from "@lib/defaults";
type Row = {
  period: string;
  repName: string;
  pharmacies: number;

  doctors: number;
  orders: number;
  units: number;
  bonus: number;
  trips: number;
};
type ByRep = { period: string; repName: string; items: Record<string, number>; bonuses: Record<string, number> };
const MONTH_LABELS = JALALI_MONTHS.map((m) => m.slice(0, 4));
export default function ChartsPanel({ scope = "rep" }: { scope?: "rep" | "admin" }) {
  const [rows, setRows] = useState<Row[]>([]);
  const [byRep, setByRep] = useState<ByRep[]>([]);
  const [products, setProducts] = useState<ProductConfig[]>(DEFAULT_PRODUCTS);
  const [metric, setMetric] = useState<"orders" | "units" | "pharmacies" | "doctors" | "trips">("units");
  const [rep, setRep] = useState("");
  const [from, setFrom] = useState("");
  const [to, setTo] = useState("");
  // «»
  const [applied, setApplied] = useState<{ from: string; to: string; rep: string }>({ from: "", to: "", rep: "" });
  const [msg, setMsg] = useState("");
  const load = useCallback(async () => {
    const [r1, r2] = await Promise.all([
      fetch("/api/reports", { cache: "no-store" }),
      fetch("/api/settings?key=products", { cache: "no-store" }),
    ]);
  });
}

```

```

    });
    if (r1.ok) {
      const d = await r1.json();
      setRows(d.rows ?? []);
      setByRep(d.productByRep ?? []);
    }
    if (r2.ok) {
      const d = await r2.json();
      const v = d.value as ProductConfig[] | null;
      if (Array.isArray(v) && v.length) setProducts(v.filter((p) => p.enabled !== false));
    }
  }, []);
  useEffect(() => {
    load();
  }, [load]);
  const reps = useMemo(() => [...new Set(rows.map((r) => r.repName))].sort(), [rows]);
  const years = useMemo(() => [...new Set(rows.map((r) => r.period.slice(0, 4))].sort(), [rows]);
  const inRange = useCallback(
    (period: string) => {
      if (applied.from && period < applied.from.slice(0, 7)) return false;
      if (applied.to && period > applied.to.slice(0, 7)) return false;
      return true;
    },
    [applied],
  );
  const scoped = useMemo(
    () => rows.filter((r) => (!applied.rep || r.repName === applied.rep) && inRange(r.period)),
    [rows, applied.rep, inRange],
  );
  const apply = () => {
    if (from && to && from > to) {
      setMsg("« » « »");
      return;
    }
    setApplied({ from, to, rep });
    setMsg(
      from || to || rep
      ? ` ${from} — ${toPersianDigits(from)} : ` + ` ${to} ` + ` ${toPersianDigits(to)} : ` + ` ${rep} `
      : ` | ${rep} ` + ` : ` + ` `
      : " ",
    );
    load();
  };
  const reset = () => {
    setFrom("");
    setTo("");
    setRep("");
    setApplied({ from: "", to: "", rep: "" });
    setMsg(" — ");
  };
  const quick = (months: number) => {
    const t = todayJalali();

    const [jy, jm] = t.split("/").map(Number);
    let y = jy;
    let m = jm - months + 1;
    while (m <= 0) {
      m += 12;
      y -= 1;
    }
    const f = `${y}/${String(m).padStart(2, "0")}/01`;
    setFrom(f);
    setTo(t);
    setApplied({ from: f, to: t, rep });
    setMsg(` ${toPersianDigits(months)} `);
  };
  const metricLabel: Record<string, string> = {
    units: " ",
    orders: " ",
    pharmacies: " ",
    doctors: " ",
    trips: " ",
  };
  // :
  const yearSeries = useMemo(
    () =>
      years.map((y) => ({
        name: toPersianDigits(y),
        values: Array.from({ length: 12 }, (_, m) => {
          const per = `${y}/${String(m + 1).padStart(2, "0")}`;
          return scoped.filter((r) => r.period === per).reduce((a, r) => a + r[metric], 0);
        }),
      })),
    [years, scoped, metric],
  );

```

```

// ( )
const monthly = useMemo(() => {
  const m = new Map<string, number>();
  for (const r of scoped) m.set(r.period, (m.get(r.period) ?? 0) + r[metric]);
  return [...m.entries()]
    .sort((a, b) => a[0].localeCompare(b[0]))
    .map(([label, value]) => ({ label: toPersianDigits(label), value }));
}, [scoped, metric]);
//
const productShare = useMemo(() => {
  const filtered = byRep.filter((r) => (!applied.rep || r.repName === applied.rep) && inRange(r.period));
  return products
    .map((p) => ({
      label: p.label,
      value: filtered.reduce((a, r) => a + (r.items[p.key] ?? 0), 0),
    }))
    .filter((d) => d.value > 0);
}, [byRep, products, applied.rep, inRange]);
// ( )
const repCompare = useMemo(() => {
  const m = new Map<string, number>();
  for (const r of rows.filter((x) => inRange(x.period))) m.set(r.repName, (m.get(r.repName) ?? 0) + r[metric]);
  return [...m.entries()].map(([label, value]) => ({ label, value })).sort((a, b) => b.value - a.value);
}, [rows, metric, inRange]);
const totals = scoped.reduce(
  (a, r) => ({
    pharmacies: a.pharmacies + r.pharmacies,
    doctors: a.doctors + r.doctors,
    orders: a.orders + r.orders,
    units: a.units + r.units,
    bonus: a.bonus + r.bonus,
    trips: a.trips + r.trips,
  }),
  { pharmacies: 0, doctors: 0, orders: 0, units: 0, bonus: 0, trips: 0 },
);
return (
  <div className="space-y-4">
    <SectionTitle icon=""> </SectionTitle>
    <Card>
      <div className="grid grid-cols-1 gap-3 sm:grid-cols-2 lg:grid-cols-4">
        <Field label=" ">
          <select
            value={metric}
            onChange={(e) => setMetric(e.target.value as typeof metric)}
            className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
          >
            {Object.entries(metricLabel).map(([k, v]) => (
              <option key={k} value={k}>
                {v}
              </option>
            ))}
          </select>
        </Field>
        {scope === "admin" ? (
          <Field label="">
            <select
              value={rep}
              onChange={(e) => setRep(e.target.value)}
              className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
            >
              <option value=""> </option>
              {reps.map((r) => (
                <option key={r} value={r}>
                  {r}
                </option>
              ))}
            </select>
          </Field>
        ) : null}
        <Field label=" ">
          <JalaliDateInput value={from} onChange={setFrom} placeholder="//" />
        </Field>
        <Field label=" ">
          <JalaliDateInput value={to} onChange={setTo} placeholder="//" />
        </Field>
      </div>
      <div className="mt-3 flex flex-wrap items-center gap-2">
        <Button onClick={apply}> </Button>
        <Button variant="ghost" onClick={reset}>
          </Button>
        <span className="mx-1 h-6 w-px bg-slate-200" />
        <Button variant="soft" onClick={() => quick(1)}>

```

```

</Button>
<Button variant="soft" onClick={() => quick(3)}>

</Button>
<Button variant="soft" onClick={() => quick(6)}>

</Button>
<Button variant="soft" onClick={() => quick(12)}>

</Button>
</div>
{msg ? (
  <div className="mt-2">
    <Alert kind={msg.startsWith("") ? "error" : "success"}>{msg}</Alert>
  </div>
) : null}
<div className="mt-3 grid grid-cols-3 gap-2 sm:grid-cols-6">
  {[
    [" ", totals.pharmacies],
    [" ", totals.doctors],
    [" ", totals.orders],
    [" ", totals.units],
    [" ", totals.bonus],
    [" ", totals.trips],
  ]}.map(([l, v]) => (
    <div key={String(l)} className="rounded-xl bg-slate-50 p-2 text-center ring-1 ring-slate-200">
      <div className="text-base font-black text-teal-700">{toPersianDigits(v as number)}</div>
      <div className="text-[10px] font-bold text-slate-500">{l as string}</div>
    </div>
  )))
</div>
</Card>
<Card>
  <h3 className="mb-2 text-sm font-bold text-slate-700">
    — {metricLabel[metric]}
  </h3>
  <LineChart labels={MONTH_LABELS} series={yearSeries} />
</Card>
<div className="grid grid-cols-1 gap-3 lg:grid-cols-2">
  <Card>
    <h3 className="mb-2 text-sm font-bold text-slate-700">
      <BarChart data={monthly} />
    </h3>
  </Card>
  <Card>
    <h3 className="mb-2 text-sm font-bold text-slate-700">
      <DonutChart data={productShare} />
    </h3>
  </Card>
</div>
{scope === "admin" ? (
  <Card>
    <h3 className="mb-2 text-sm font-bold text-slate-700"> — {metricLabel[metric]}</h3>
    <BarChart data={repCompare} />
  </Card>
) : null}
</div>
);
}

```

```

@@ src/components/screens/HomeScreen.tsx typescript
"use client";
import { useCallback, useEffect, useState } from "react";
import dynamic from "next/dynamic";
import LocationPicker, { type LatLng } from "@components/LocationPicker";
import { Alert, Badge, Button, Card, Field, Input, SectionTitle, TextArea } from "@components/ui";
import NavButton from "@components/NavButton";
import { toPersianDigits } from "@lib/jalali";
import { useLive } from "@lib/useLive";
import { useConfirm } from "@components/Confirm";
const MapBox = dynamic(() => import("@components/MapBox"), { ssr: false });
type Home = {
  id: number;
  userId: number;
  repName: string;
  title: string;
  address: string;
  lat: number;
  lng: number;
};
export default function HomeScreen({ isAdmin = false }: { isAdmin?: boolean }) {
  const [rows, setRows] = useState<Home[]>([]);
  const [title, setTitle] = useState("");
  const [address, setAddress] = useState("");
  const [loc, setLoc] = useState<LatLng>({ lat: null, lng: null, accuracy: null });
  const [msg, setMsg] = useState<{ kind: "success" | "error"; text: string } | null>(null);

```

```

const [repFilter, setRepFilter] = useState("");
const confirm = useConfirm();
const load = useCallback(async () => {
  const res = await fetch("/api/homes", { cache: "no-store" });
  if (res.ok) setRows((await res.json()).rows ?? []);
}, []);
useLive(load, 25000);
const reps = [...new Set(rows.map((r) => r.repName))].sort();
const visible = rows.filter((r) => !repFilter || r.repName === repFilter);
const grouped = (() => {
  const m = new Map<string, Home[]>();
  for (const r of visible) {
    if (!m.has(r.repName)) m.set(r.repName, []);
    m.get(r.repName).push(r);
  }
  return [...m.entries()].sort((a, b) => a[0].localeCompare(b[0]));
})();
const save = async () => {
  if (!loc.lat || !loc.lng) {
    setMsg({ kind: "error", text: " " });
    return;
  }
  if (!(await confirm({ title: " ", message: `«${title}» `, confirmText: "
" })))
    return;
  const res = await fetch("/api/homes", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title, address, ...loc }),
  });
  const d = await res.json().catch(() => ({}));
  if (res.ok) {
    setMsg({ kind: "success", text: " " });
    load();
  } else setMsg({ kind: "error", text: d.error ?? " " });
};
return (
  <div className="space-y-4">
    <SectionTitle icon="">{isAdmin ? " " : " "}</SectionTitle>
    {msg ? <Alert kind={msg.kind}>{msg.text}</Alert> : null}
    {!isAdmin ? (
      <Card>
        <div className="grid grid-cols-1 gap-3 sm:grid-cols-2">
          <Field label=" / " required>
            <Input value={title} onChange={(e) => setTitle(e.target.value)} placeholder=" " />
          </Field>
          <Field label="">
            <TextArea value={address} onChange={(e) => setAddress(e.target.value)} />
          </Field>
        </div>
        <div className="mt-3">
          <LocationPicker value={loc} onChange={setLoc} label={title} suggestQuery={address || title} />
        </div>
        <div className="mt-3 flex justify-end">
          <Button onClick={save}> </Button>
        </div>
      </Card>
    ) : null}
    <Card>
      <div className="mb-2 flex flex-wrap items-center justify-between gap-2">
        <h3 className="text-sm font-bold text-slate-700">
          {isAdmin ? " — " : " "}
        </h3>
        {isAdmin && reps.length > 1 ? (
          <select
            value={repFilter}
            onChange={(e) => setRepFilter(e.target.value)}
            className="rounded-xl border border-slate-300 px-3 py-2 text-xs"
          >
            <option value=""> ({toPersianDigits(reps.length)})</option>
            {reps.map((r) => (
              <option key={r} value={r}>
                {r}
              </option>
            ))}
          </select>
        ) : null}
      </div>
      {visible.length === 0 ? (
        <p className="text-sm text-slate-400"> </p>
      ) : (
        <>
          <MapBox
            height={300}

```

```

        points=visible.map((r) => ({
          lat: r.lat,
          lng: r.lng,
          label: `${r.title} — ${r.repName}`,
          color: "#7c3aed",
        })))
      />
      <div className="mt-3 space-y-2">
        {grouped.map(([repName, list]) => (
          <div key={repName}>
            <div className="mb-1 flex items-center gap-2 rounded-lg bg-slate-100 px-2 py-1.5">
              <span className="text-xs font-black text-slate-800">{repName}</span>
              <Badge tone="slate">{toPersianDigits(list.length)} </Badge>
            </div>
            {list.map((r) => (
              <div key={r.id} className="mb-1 flex flex-wrap items-center gap-2 rounded-lg bg-slate-50 px-3 py-2 text-xs">
                <span className="font-bold text-purple-700"> {r.title}</span>
                <span className="text-slate-500">{r.address}</span>
                <span className="mr-auto">
                  <NavButton lat={r.lat} lng={r.lng} label={`${r.title} — ${r.repName}`} />
                </span>
              </div>
            ))}
          </div>
        ))}
      </div>
    </div>
  </div>
);
}

```

```

@@ src/components/screens/LeaveScreen.tsx typescript
"use client";
import { useCallback, useEffect, useState } from "react";
import JalaliDateInput from "@components/JalaliDateInput";
import { Alert, Badge, Button, Card, Field, Input, SectionTitle, TextArea } from "@components/ui";
import { toPersianDigits, todayJalali } from "@lib/jalali";
import { useSession } from "@components/SessionProvider";
import Combobox from "@components/Combobox";
import { useConfirm } from "@components/Confirm";
import { useLive } from "@lib/useLive";
import { JALALI_MONTHS } from "@lib/jalali";
import { downloadFile } from "@lib/download";
type Leave = {
  id: number;
  repName: string;
  kind: string;
  fromDate: string;
  toDate: string;
  days: number;
  fromTime: string;
  toTime: string;
  hours: number;
  reason: string;
  supervisorStatus: string;
  supervisorNote: string;
  supervisorName: string;
  managerStatus: string;
  managerNote: string;
  managerName: string;
};
const KINDS = ["", "", "", " ", " ", " ", ""];
function StatusBadge({ s }: { s: string }) {
  if (s === "approved") return <Badge tone="green"> </Badge>;
  if (s === "rejected") return <Badge tone="amber"> </Badge>;
  return <Badge tone="slate"> </Badge>;
}
export default function LeaveScreen() {
  const { me } = useSession();
  const role = me?.role ?? "rep";
  const [showForm, setShowForm] = useState(true);
  const [rows, setRows] = useState<Leave[]>([]);
  const [kind, setKind] = useState("");
  const [fromDate, setFrom] = useState(todayJalali());
  const [toDate, setTo] = useState(todayJalali());
  const [days, setDays] = useState("1");
  const [fromTime, setFromTime] = useState("08:00");
  const [toTime, setToTime] = useState("12:00");
  const [reason, setReason] = useState("");
  const [msg, setMsg] = useState<{ kind: "success" | "error"; text: string } | null>(null);
  const [kinds, setKinds] = useState<string[]>(KINDS);

```



```

const [period, setPeriod] = useState("");
const confirm = useConfirm();
const loadKinds = useCallback(async () => {
  const res = await fetch("/api/options?category=leaveKind", { cache: "no-store" });
  if (!res.ok) return;
  const d = await res.json();
  const vals = (d.rows ?? []).map((r: { value: string }) => r.value);
  setKinds([...new Set([...KINDS, ...vals])]);
}, []);
const load = useCallback(async () => {
  const res = await fetch("/api/leaves", { cache: "no-store" });
  if (res.ok) setRows((await res.json()).rows ?? []);
}, []);
useEffect(() => {
  loadKinds();
}, [loadKinds]);
//
useLive(load, 20000);
const submit = async () => {
  const summary = isHourly
    ? ` ${toPersianDigits(fromDate)}  ${toPersianDigits(fromTime)}  ${toPersianDigits(toTime)}  (${toPersianDigits(hoursCalc)})`
    : ` ${kind}  ${toPersianDigits(fromDate)}  ${toPersianDigits(toDate)}  (${toPersianDigits(days)}) `;
  if (!(await confirm({ title: " ", message: `${summary}\n`, confirmText: " " })))
    return;
  const res = await fetch("/api/leaves", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      kind,
      fromDate,
      toDate,
      days: isHourly ? 0 : Number(days) || 1,
      fromTime: isHourly ? fromTime : "",
      toTime: isHourly ? toTime : "",
      hours: isHourly ? hoursCalc : 0,
      reason,
    })),
  });
  const d = await res.json().catch(() => ({}));
  if (res.ok) {
    setMsg({ kind: "success", text: " " });
    setReason("");
    load();
  } else setMsg({ kind: "error", text: d.error ?? " " });
};
const decide = async (id: number, status: "approved" | "rejected") => {
  const row = rows.find((r) => r.id === id);
  if (
    !(await confirm({
      title: status === "approved" ? " " : " ",
      message: `«${row?.repName ?? ""}» ${status === "approved" ? "" : ""}`,
      confirmText: status === "approved" ? " " : " ",
      danger: status === "rejected",
    })))
    return;
  const note = window.prompt(status === "approved" ? " ( )" : " ( )" ?? "");
  const res = await fetch("/api/leaves", {
    method: "PATCH",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ id, status, note }),
  });
  if (res.ok) load();
  else setMsg({ kind: "error", text: " " });
};
const canDecide = role === "admin" || role === "supervisor";
const periods = [...new Set(rows.map((r) => r.fromDate.slice(0, 7)))]
  .sort()
  .reverse();
const monthName = (p: string) =>
  `${JALALI_MONTHS[Number(p.slice(5, 7)) - 1]} ?? ""` + ` ${toPersianDigits(p.slice(0, 4))}`;
const scoped = rows.filter((r) => !period || r.fromDate.startsWith(period));
const summary = (() => {
  const m = new Map<string, { period: string; rep: string; count: number; days: number; hours: number }>();
  for (const r of scoped) {
    const per = r.fromDate.slice(0, 7);
    const k = `${per}|${r.repName}`;
    const cur = m.get(k) ?? { period: per, rep: r.repName, count: 0, days: 0, hours: 0 };
    cur.count++;
    cur.days += Number(r.days || 0);
    cur.hours += Number(r.hours || 0);
    m.set(k, cur);
  }
  return [...m.values()].sort((a, b) =>

```

```

    a.period === b.period ? a.rep.localeCompare(b.rep) : b.period.localeCompare(a.period),
  );
})();
const totals = scoped.reduce(
  (a, r) => {
    const ok = r.managerStatus === "approved";
    return {
      days: a.days + (ok ? Number(r.days || 0) : 0),
      hours: a.hours + (ok ? Number(r.hours || 0) : 0),
      pending: a.pending + (r.managerStatus === "pending" ? 1 : 0),
    };
  },
  { days: 0, hours: 0, pending: 0 },
);
const isHourly = kind.includes("");
const hoursCalc = (() => {
  const [h1, m1] = fromTime.split(":").map(Number);

  const [h2, m2] = toTime.split(":").map(Number);
  const diff = (h2 * 60 + m2 - (h1 * 60 + m1)) / 60;
  return diff > 0 ? Math.round(diff * 100) / 100 : 0;
})();
return (
  <div className="space-y-4">
    <SectionTitle icon=""> ( : + )</SectionTitle>
    {msg ? <Alert kind={msg.kind}>{msg.text}</Alert> : null}
    <div className="flex flex-wrap items-center gap-2">
      <Button variant={showForm ? "primary" : "ghost"} onClick={() => setShowForm(true)}>

    </Button>
    <Button variant={!showForm ? "primary" : "ghost"} onClick={() => setShowForm(false)}>

    </Button>
    </div>
    {showForm ? (
      <Card>
        <div className="grid grid-cols-1 gap-3 sm:grid-cols-2 lg:grid-cols-4">
          <Field label=" " hint=" " >
            <Combobox value={kind} onChange={setKind} options={kinds} category="leaveKind" onAdded={loadKinds} />
          </Field>
          <Field label=" " required>
            <JalaliDateInput value={fromDate} onChange={setFrom} />
          </Field>
          <Field label=" " required>
            <JalaliDateInput value={toDate} onChange={setTo} />
          </Field>
          {isHourly ? (
            <>
              <Field label=" " required>
                <input
                  type="time"
                  value={fromTime}
                  onChange={(e) => setFromTime(e.target.value)}
                  className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
                />
            </Field>
            <Field label=" " required>
              <input
                type="time"
                value={toTime}
                onChange={(e) => setToTime(e.target.value)}
                className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
              />
            </Field>
            <Field label=" ()">
              <div className="rounded-xl bg-slate-100 px-3 py-2.5 text-sm font-bold text-teal-700">
                {toPersianDigits(hoursCalc)}
              </div>
            </Field>
            </>
          ) : (
            <Field label=" ">
              <Input inputMode="numeric" value={days} onChange={(e) => setDays(e.target.value.replace(/\D/g, ""))} />
            </Field>
          )
        <div className="sm:col-span-2 lg:col-span-4">
          <Field label=" ">
            <TextArea value={reason} onChange={(e) => setReason(e.target.value)} />
          </Field>
        </div>
      </div>
      <div className="mt-3 flex justify-end">
        <Button onClick={submit}> </Button>
      </div>
    </Card>
  )

```

```

) : null}
<Card>
  <h3 className="mb-2 text-sm font-bold text-slate-700">          </h3>
  <div className="mb-3 flex flex-wrap gap-2">
    <select
      value={period}
      onChange={(e) => setPeriod(e.target.value)}
      className="rounded-xl border border-slate-300 px-3 py-2 text-xs"
    >
      <option value=""> </option>
      {periods.map((p) => (
        <option key={p} value={p}>
          {monthName(p)}
        </option>
      ))}
    </select>
    {[ "approvedDays", "approvedHours", "pending"].map((k) => {
      const map: Record<string, [string, string | number, string]> = {
        approvedDays: [ " ", toPersianDigits(totals.days), "bg-emerald-50 text-emerald-800"],
        approvedHours: [ " ", toPersianDigits(totals.hours), "bg-sky-50 text-sky-800"],
        pending: [ " ", toPersianDigits(totals.pending), "bg-amber-50 text-amber-800"],
      };
      const [l, v, c] = map[k];
      return (
        <div key={k} className={`rounded-xl px-4 py-2 text-center ring-1 ring-slate-200 ${c}`}>
          <div className="text-lg font-black">{v}</div>
          <div className="text-[10px] font-bold">{l}</div>
        </div>
      );
    })}
  </div>
  <div className="scroll-x">
    <table className="w-full min-w-[520px] text-right text-xs">
      <thead>
        <tr className="bg-slate-100 text-slate-600">
          <th className="px-2 py-2"> </th>
          <th className="px-2 py-2"> </th>
          <th className="px-2 py-2"> </th>
          <th className="px-2 py-2"> </th>
          <th className="px-2 py-2"> </th>
        </tr>
      </thead>
      <tbody>
        {summary.map((s) => (
          <tr key={` ${s.period}-${s.rep}`} className="border-b border-slate-100">
            <td className="px-2 py-2 font-bold text-teal-700">{monthName(s.period)}</td>
            <td className="px-2 py-2">{s.rep}</td>
            <td className="px-2 py-2">{toPersianDigits(s.count)}</td>
            <td className="px-2 py-2">{toPersianDigits(s.days)}</td>
            <td className="px-2 py-2">{toPersianDigits(Math.round(s.hours * 100) / 100)}</td>
          </tr>
        ))}
        {summary.length === 0 ? (
          <tr>
            <td colspan={5} className="py-4 text-center text-slate-400">
              </td>
            </tr>
          ) : null}
      </tbody>
    </table>
  </div>
</Card>
<Card>
  <div className="mb-2 flex items-center justify-between">
    <h3 className="text-sm font-bold text-slate-700"> </h3>
    <button
      onClick={async () => setMsg({ kind: "success", text: await downloadFile("/api/export?type=leaves", "leaves.xls") })}
      className="rounded-lg bg-emerald-600 px-3 py-1.5 text-xs font-bold text-white"
    >
      </button>
  </div>
  <div className="scroll-x">
    <table className="w-full min-w-[820px] text-right text-xs sm:text-sm">
      <thead>
        <tr className="bg-slate-100 text-slate-600">
          <th className="px-2 py-2"></th>
          <th className="px-2 py-2"> </th>
          <th className="px-2 py-2"> </th>
          <th className="px-2 py-2"> </th>
          <th className="px-2 py-2"> </th>
          <th className="px-2 py-2"> / </th>
        </tr>
      </thead>
    </table>
  </div>

```

```

        <th className="px-2 py-2"></th>
        <th className="px-2 py-2"> </th>
        <th className="px-2 py-2"> </th>
        {canDecide ? <th className="px-2 py-2"></th> : null}
      </tr>
    </thead>
    <tbody>
      {scoped.map((r, i) => (
        <tr key={r.id} className="border-b border-slate-100">
          <td className="px-2 py-2">{toPersianDigits(i + 1)}</td>
          <td className="px-2 py-2 font-bold text-slate-700">{r.repName}</td>
          <td className="px-2 py-2">{r.kind}</td>
          <td className="px-2 py-2">{toPersianDigits(r.fromDate)}</td>
          <td className="px-2 py-2">{toPersianDigits(r.toDate)}</td>

          <td className="px-2 py-2">
            {r.hours > 0 ? (
              <span className="font-bold text-sky-700">
                {toPersianDigits(r.hours)}
                <div className="text-[10px] text-slate-400">
                  {toPersianDigits(r.fromTime)} {toPersianDigits(r.toTime)}
                </div>
              </span>
            ) : (
              ` ${toPersianDigits(r.days)} `
            )
          }</td>
          <td className="max-w-[180px] truncate px-2 py-2 text-slate-500">{r.reason}</td>
          <td className="px-2 py-2">
            <StatusBadge s={r.supervisorStatus} />
            {r.supervisorName ? (
              <div className="text-[10px] text-slate-400">{r.supervisorName}</div>
            ) : null}
          </td>
          <td className="px-2 py-2">
            <StatusBadge s={r.managerStatus} />
            {r.managerName ? <div className="text-[10px] text-slate-400">{r.managerName}</div> : null}
          </td>
          {canDecide ? (
            <td className="px-2 py-2">
              <div className="flex gap-1">
                <button
                  onClick={() => decide(r.id, "approved")}
                  className="rounded-lg bg-emerald-100 px-2 py-1 text-[11px] font-bold text-emerald-700"
                >

                </button>
                <button
                  onClick={() => decide(r.id, "rejected")}
                  className="rounded-lg bg-rose-100 px-2 py-1 text-[11px] font-bold text-rose-700"
                >

                </button>
              </div>
            </td>
          ) : null}
        </tr>
      ))}
      {scoped.length === 0 ? (
        <tr>
          <td colSpan={10} className="py-6 text-center text-slate-400">

          </td>
        </tr>
      ) : null}
    </tbody>
  </table>
</div>
</Card>
</div>
);
}

```

```

@@ src/components/screens/MapExplorer.tsx typescript
"use client";
import { useCallback, useEffect, useMemo, useState } from "react";
import nextDynamic from "next/dynamic";
import { Alert, Badge, Button, Card, Field, Input, SectionTitle } from "@components/ui";
import Combobox from "@components/Combobox";
import NavButton from "@components/NavButton";
import { fetchRows } from "@lib/useLive";
import { IRAN_CENTER, PROVINCE_GEO, TEHRAN_REGIONS, resolveArea } from "@lib/geo";
import { toPersianDigits } from "@lib/jalali";
import type { MapArea, MapPoint } from "@components/MapBox";
const MapBox = nextDynamic(() => import("@components/MapBox"), {

```

```

ssr: false,
loading: () => (
  <div className="flex h-[520px] items-center justify-center rounded-2xl bg-slate-100 text-sm text-slate-400">
    ...
  </div>
),
});
type Place = {
  id: number;
  kind: "pharmacy" | "doctor" | "home";
  name: string;
  repName: string;

  province: string;
  city: string;
  region: string;
  phone: string;
  address: string;
  lat: number;
  lng: number;
  extra: string;
};
const KIND_META: Record<Place["kind"], { label: string; icon: string; color: string }> = {
  pharmacy: { label: "", icon: "", color: "#0f766e" },
  doctor: { label: "", icon: "", color: "#0369a1" },
  home: { label: " ", icon: "", color: "#7c3aed" },
};
/**
 * :
 * / / / .
 */
export default function MapExplorer({ isAdmin = false }: { isAdmin?: boolean }) {
  const [places, setPlaces] = useState<Place[]>([]);
  const [loading, setLoading] = useState(true);
  const [kinds, setKinds] = useState<Record<Place["kind"], boolean>>({
    pharmacy: true,
    doctor: true,
    home: true,
  }));
  const [province, setProvince] = useState("");
  const [city, setCity] = useState("");
  const [region, setRegion] = useState("");
  const [rep, setRep] = useState("");
  const [q, setQ] = useState("");
  const [selected, setSelected] = useState<Place | null>(null);
  /* ----- */
  const load = useCallback(async () => {
    setLoading(true);
    const [ph, dr, hm] = await Promise.all([
      fetchRows<Record<string, unknown>>("/api/records/pharmacies?limit=1000"),
      fetchRows<Record<string, unknown>>("/api/records/doctors?limit=1000"),
      fetchRows<Record<string, unknown>>("/api/homes"),
    ]);
    const out: Place[] = [];
    const push = (kind: Place["kind"], r: Record<string, unknown>, name: string, extra = "") => {
      const lat = Number(r.lat);
      const lng = Number(r.lng);
      if (!Number.isFinite(lat) || !Number.isFinite(lng) || (!lat && !lng)) return;
      out.push({
        id: Number(r.id),
        kind,
        name,
        repName: String(r.repName ?? ""),
        province: String(r.province ?? ""),
        city: String(r.city ?? ""),
        region: String(r.region ?? ""),
        phone: String(r.managerPhone ?? r.phone ?? ""),
        address: String(r.address ?? ""),
        lat,
        lng,
        extra,
      });
    };
    for (const r of ph) push("pharmacy", r, String(r.name ?? ""), String(r.managerName ?? ""));
    for (const r of dr) push("doctor", r, String(r.name ?? ""), String(r.specialty ?? ""));
    for (const r of hm) push("home", r, String(r.title ?? ""), String(r.repName ?? ""));
    setPlaces(out);
    setLoading(false);
  }, []);
  useEffect(() => {
    load();
  }, [load]);
  /* ----- */
  const provinces = useMemo(
    () => [...new Set([...Object.keys(PROVINCE_GEO), ...places.map((p) => p.province)])].filter(Boolean).sort(),

```

```

[places],
);
const cities = useMemo(
  () =>

    [...new Set(places.filter((p) => !province || p.province === province).map((p) => p.city))]
      .filter(Boolean)
      .sort(),
    [places, province],
  );
const regions = useMemo(() => {
  const fromData = places.filter((p) => (!city || p.city === city)).map((p) => p.region);
  const base = city === "" ? Object.values(TEHRAN_REGIONS).map((r) => r.name) : [];
  return [...new Set([...base, ...fromData])].filter(Boolean).sort();
}, [places, city]);
const reps = useMemo(() => [...new Set(places.map((p) => p.repName))].filter(Boolean).sort(), [places]);
/* ----- */
const filtered = useMemo(() => {
  const term = q.trim().toLowerCase();
  return places.filter((p) => {
    if (!kinds[p.kind]) return false;
    if (province && p.province !== province) return false;
    if (city && p.city !== city) return false;
    if (region && p.region !== region) return false;
    if (rep && p.repName !== rep) return false;
    if (term && !`${p.name} ${p.repName} ${p.city} ${p.address} ${p.extra}`.toLowerCase().includes(term))
      return false;
    return true;
  });
}, [places, kinds, province, city, region, rep, q]);
/** ----- */
const area: MapArea = useMemo(() => {
  const g = resolveArea(province, city, region);
  return { lat: g.lat, lng: g.lng, zoom: g.zoom, radiusKm: g.radiusKm, name: g.name };
}, [province, city, region]);
const mapPoints: MapPoint[] = useMemo(
  () =>
    (selected ? [selected] : filtered).map((p) => ({
      lat: p.lat,
      lng: p.lng,
      label: `${KIND_META[p.kind].icon} ${p.name}${p.repName ? ` — ${p.repName}` : ""}`,
      color: KIND_META[p.kind].color,
    })),
  [filtered, selected],
);
const counts = useMemo(
  () => ({
    pharmacy: filtered.filter((p) => p.kind === "pharmacy").length,
    doctor: filtered.filter((p) => p.kind === "doctor").length,
    home: filtered.filter((p) => p.kind === "home").length,
  }),
  [filtered],
);
const reset = () => {
  setProvince("");
  setCity("");
  setRegion("");
  setRep("");
  setQ("");
  setSelected(null);
};
return (
  <div className="space-y-4">
    <div className="flex flex-wrap items-center justify-between gap-2">
      <SectionTitle icon=""> — </SectionTitle>
      <div className="flex flex-wrap gap-2">
        <Badge tone="green"> {toPersianDigits(counts.pharmacy)}</Badge>
        <Badge tone="slate"> {toPersianDigits(counts.doctor)}</Badge>
        <Badge tone="amber"> {toPersianDigits(counts.home)}</Badge>
        <Button variant="ghost" onClick={load}>

          </Button>
      </div>
    </div>
    <Card>
      <div className="grid grid-cols-1 gap-3 sm:grid-cols-2 lg:grid-cols-4">
        <Field label="">
          <Combobox
            value={province}
            onChange={(v) => {
              setProvince(v);
              setCity("");
              setRegion("");
            }}

            setSelected(null);
          </Combobox>
        </Field>
      </div>
    </Card>
  </div>
);

```

```

    }}
    options={provinces}
    selectOnly
    placeholder=" "
  />
</Field>
<Field label="">
  <Combobox
    value={city}
    onChange={(v) => {
      setCity(v);
      setRegion("");
      setSelected(null);
    }}
    options={cities}
    selectOnly
    placeholder={province ? " " : " "}
  />
</Field>
<Field label="">
  <Combobox
    value={region}
    onChange={(v) => {
      setRegion(v);
      setSelected(null);
    }}
    options={regions}
    selectOnly
    placeholder=" "
  />
</Field>
<Field label={isAdmin ? "" : ""}>
  {isAdmin ? (
    <select
      value={rep}
      onChange={(e) => {
        setRep(e.target.value);
        setSelected(null);
      }}
      className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
    >
      <option value=""> </option>
      {reps.map((r) => (
        <option key={r} value={r}>
          {r}
        </option>
      ))}
    </select>
  ) : (
    <Input value={q} onChange={(e) => setQ(e.target.value)} placeholder=" ..." />
  )}
</Field>
</div>
<div className="mt-3 flex flex-wrap items-center gap-2">
  {(Object.keys(KIND_META) as Place["kind"][]).map((k) => (
    <button
      key={k}
      onClick={() => setKinds({ ...kinds, [k]: !kinds[k] })}
      className={`rounded-xl px-3 py-1.5 text-xs font-bold ring-1 transition ${
        kinds[k] ? "bg-teal-50 text-teal-700 ring-teal-300" : "bg-slate-100 text-slate-400 ring-slate-200"
      }`}
    >
      {KIND_META[k].icon} {KIND_META[k].label} {kinds[k] ? "" : ""}
    </button>
  ))}
  {isAdmin ? (
    <Input value={q} onChange={(e) => setQ(e.target.value)} placeholder=" ..." className="max-w-[200px]"
  ) : null}
  <div className="flex-1" />
  <Badge tone="slate"> {area.name ?? IRAN_CENTER.name}</Badge>
  <Button variant="ghost" onClick={reset}>
    </Button>
</div>
</Card>
{loading ? <Alert kind="info"> ...</Alert> : null}
{!loading && filtered.length === 0 ? (
  <Alert kind="info">
    .
  </Alert>
) : null}

<div className="grid grid-cols-1 gap-3 lg:grid-cols-3">
  <Card className="lg:col-span-2">

```

```

<MapBox
  height={520}
  points={mapPoints}
  area={area}
  follow={!selected}
  showIranProvinces
  selectedProvince={province}
  onProvinceSelect={({name} => {
    setProvince(name);
    setCity("");
    setRegion("");
    setSelected(null);
  })
}/>
{selected ? (
  <div className="mt-3 rounded-xl bg-slate-50 p-3 ring-1 ring-slate-200">
    <div className="flex flex-wrap items-center gap-2">
      <span className="text-sm font-black text-slate-800">
        {KIND_META[selected.kind].icon} {selected.name}
      </span>
      {selected.extra ? <Badge tone="slate">{selected.extra}</Badge> : null}
      {selected.repName ? <Badge tone="green">{selected.repName}</Badge> : null}
      <div className="mr-auto flex items-center gap-2">
        <NavButton lat={selected.lat} lng={selected.lng} label={selected.name} />
        <button onClick={() => setSelected(null)} className="text-[11px] font-bold text-slate-500">

          </button>
        </div>
      </div>
      {selected.address ? <p className="mt-1 text-[11px] text-slate-600">{selected.address}</p> : null}
      <p className="text-[10px] text-slate-400" dir="ltr">
        {selected.lat.toFixed(5)}, {selected.lng.toFixed(5)}
      </p>
    </div>
  ) : null}
</Card>
<Card className="lg:col-span-1">
  <h3 className="mb-2 text-sm font-bold text-slate-700">
    ({toPersianDigits(filtered.length)})
  </h3>
  <div className="max-h-[520px] space-y-1 overflow-y-auto">
    {filtered.slice(0, 300).map((p) => (
      <button
        key={` ${p.kind}-${p.id}`}
        onClick={() => setSelected(selected?.id === p.id && selected.kind === p.kind ? null : p)}
        className={`w-full rounded-xl px-3 py-2 text-right ring-1 transition ${
          selected?.id === p.id && selected.kind === p.kind
            ? "bg-teal-50 ring-teal-300"
            : "bg-slate-50 ring-slate-200 hover:bg-slate-100"
        }`}
      >
        <div className="flex flex-wrap items-center gap-1.5">
          <span className="text-xs font-bold text-slate-800">
            {KIND_META[p.kind].icon} {p.name}
          </span>
          {p.extra ? <span className="text-[10px] text-slate-500">{p.extra}</span> : null}
        </div>
        <div className="mt-0.5 flex flex-wrap gap-2 text-[10px] text-slate-500">
          {p.city ? <span>{p.city}</span> : null}
          {p.region ? <span>{p.region}</span> : null}
          {p.repName ? <span>{p.repName}</span> : null}
          {p.phone ? <span>{toPersianDigits(p.phone)}</span> : null}
        </div>
      </button>
    ))}
    {filtered.length > 300 ? (
      <p className="py-2 text-center text-[11px] text-slate-400">
        {toPersianDigits(filtered.length - 300)} —
      </p>
    ) : null}
  </div>
</Card>
</div>
</div>
);
}

```

```

@@ src/components/screens/NotificationScreen.tsx typescript
"use client";
import { useCallback, useEffect, useState } from "react";

```

```

import { Alert, Badge, Button, Card, Field, Input, SectionTitle, TextArea } from "@components/ui";
import { useSession } from "@components/SessionProvider";
import { tehranDateTime, toPersianDigits } from "@lib/jalali";
import { useLive } from "@lib/useLive";

```



```

import { useConfirm } from "@components/Confirm";
type N = {
  id: number;
  title: string;
  body: string;
  fromName: string;
  kind: string;
  readAt: string | null;
  createdAt: string;
};
type U = { id: number; fullName: string; role: string };
export default function NotificationScreen() {
  const { me } = useSession();
  const [rows, setRows] = useState<N[]>([]);
  const [users, setUsers] = useState<U[]>([]);
  const [title, setTitle] = useState("");
  const [body, setBody] = useState("");
  const [to, setTo] = useState("0");
  const [msg, setMsg] = useState("");
  const confirm = useConfirm();
  const load = useCallback(async () => {
    const res = await fetch("/api/notifications", { cache: "no-store" });
    if (res.ok) setRows((await res.json()).rows ?? []);
  }, []);
  useLive(load, 15000);
  useEffect(() => {
    if (me && me.role !== "rep") {
      fetch("/api/users", { cache: "no-store" })
        .then((r) => (r.ok ? r.json() : null))
        .then((d) => setUsers(d?.rows ?? []))
        .catch(() => undefined);
    }
  }, [me]);
  const send = async () => {
    if (!title.trim()) return;
    if (
      !(await confirm({
        title: " ",
        message: isRep ? " " : Number(to) ? " " : " ",
      }, {
        confirmText: " ",
      })))
      return;
    const res = await fetch("/api/notifications", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ title, body, toUserId: Number(to) || 0 }),
    });
    if (res.ok) {
      setMsg(" ");
      setTitle("");
      setBody("");
      load();
    } else setMsg(" ");
  };
  const markAll = async () => {
    await fetch("/api/notifications", {
      method: "PATCH",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ all: true }),
    });
    load();
  };
  const isRep = me?.role === "rep";
  return (
    <div className="space-y-4">
      <div className="flex items-center justify-between">
        <SectionTitle icon="" /> </SectionTitle>
        <Button variant="ghost" onClick={markAll}>

        </Button>

      </div>
      <Alert kind="success">{msg}</Alert> : null
    <Card>
      <h3 className="mb-2 text-sm font-bold text-slate-700">
        {isRep ? " " : " "}
      </h3>
      <div className="grid grid-cols-1 gap-3 sm:grid-cols-3">
        <Field label="" required>
          <Input value={title} onChange={(e) => setTitle(e.target.value)} />
        </Field>
        <Field label=" ">
          <TextArea value={body} onChange={(e) => setBody(e.target.value)} />
        </Field>
      </div>
    </Card>
  );
}

```

```

</Field>
{!isRep ? (
  <Field label="">
    <select
      value={to}
      onChange={(e) => setTo(e.target.value)}
      className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
    >
      <option value="0"> </option>
      {users
        .filter((u) => u.role === "rep")
        .map((u) => (
          <option key={u.id} value={u.id}>
            {u.fullName}
          </option>
        ))}
    </select>
  </Field>
) : null}
</div>
<div className="mt-3 flex justify-end">
  <Button onClick={send}> </Button>
</div>
</Card>
<Card>
  <div className="space-y-2">
    {rows.length === 0 ? <p className="text-sm text-slate-400"> </p> : null}
    {rows.map((n) => (
      <div
        key={n.id}
        className={`rounded-xl px-3 py-2 text-sm ${n.readAt ? "bg-slate-50" : "bg-teal-50 ring-1 ring-teal-200"}`}
      >
        <div className="flex flex-wrap items-center gap-2">
          <span className="font-bold text-slate-800">{n.title}</span>
          {!n.readAt ? <Badge tone="green"></Badge> : null}
          <span className="mr-auto text-[10px] text-slate-400">{tehranDateTime(n.createdAt)}</span>
        </div>
        {n.body ? <p className="mt-1 whitespace-pre-wrap text-xs text-slate-600">{n.body}</p> : null}
        {n.fromName ? <p className="text-[10px] text-slate-400">: {n.fromName}</p> : null}
      </div>
    ))}
  </div>
  <p className="mt-2 text-[11px] text-slate-400"> : {toPersianDigits(rows.length)}</p>
</Card>
</div>
);
}

```

@@ src/components/screens/OptionsScreen.tsx typescript

```

"use client";
import { useCallback, useEffect, useState } from "react";
import { Alert, Button, Card, Input, SectionTitle } from "@components/ui";
import { OPTION_CATEGORIES } from "@lib/constants";
import { useConfirm } from "@components/Confirm";
import { toPersianDigits } from "@lib/jalali";
import { useLive } from "@lib/useLive";
type Opt = { id: number; category: string; value: string; parent: string; createdBy: string };
/** «» — ( ) */
export default function OptionsScreen({ canDelete = false }: { canDelete?: boolean }) {
  const [rows, setRows] = useState<Opt[]>([]);
  const [draft, setDraft] = useState<Record<string, string>>({});
  const [msg, setMsg] = useState<{ kind: "success" | "error"; text: string } | null>(null);
  const [q, setQ] = useState("");
  const [editing, setEditing] = useState<{ id: number; value: string } | null>(null);
  const [parentSel, setParentSel] = useState<Record<string, string>>({});
  const confirm = useConfirm();
  const load = useCallback(async () => {
    const res = await fetch("/api/options", { cache: "no-store" });
    if (res.ok) setRows((await res.json()).rows ?? []);
  }, []);
  useLive(load, 20000);
  const add = async (category: string) => {
    const value = (draft[category] ?? "").trim();
    if (!value) return;
    if ((category === "city" || category === "region") && !(parentSel[category] ?? "")) {
      setMsg({ kind: "error", text: category === "city" ? " " : " " });
      return;
    }
    const label = OPTION_CATEGORIES.find((c) => c.key === category)?.label ?? category;
    if (!(await confirm({ title: " ", message: «`${value}`» «`${label}`»`, confirmText:
      "" })))
      return;
    const res = await fetch("/api/options", {
      method: "POST",

```

```

    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ category, value, parent: parentSel[category] ?? "" }),
  });
const d = await res.json().catch(() => ({}));
if (res.ok) {
  setDraft({ ...draft, [category]: "" });
  setMsg({
    kind: "success",
    text: d.duplicate ? "      " : "      ",
  });
  load();
} else setMsg({ kind: "error", text: d.error ?? " " });
};
const remove = async (id: number, value: string) => {
  if (
    !(await confirm({
      title: " ",
      message: `«${value}» `,
      confirmText: " ",
      danger: true,
    })))
  )
    return;
  const res = await fetch(`/api/options?id=${id}`, { method: "DELETE" });
  if (res.ok) {
    setMsg({ kind: "success", text: " " });
    load();
  } else setMsg({ kind: "error", text: " " });
};
const saveEdit = async () => {
  if (!editing) return;
  const value = editing.value.trim();
  if (!value) return;
  if (!(await confirm({ title: " ", message: `«${value}» `, confirmText: " " })))
    return;
  const res = await fetch("/api/options", {
    method: "PATCH",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ id: editing.id, value }),
  });
  const d = await res.json().catch(() => ({}));
  if (res.ok) {
    setMsg({ kind: "success", text: " " });
    setEditing(null);
    load();
  } else setMsg({ kind: "error", text: d.error ?? " " });
};
return (
  <div className="space-y-4">
    <SectionTitle icon=""> ( )</SectionTitle>
    {msg ? <Alert kind={msg.kind}>{msg.text}</Alert> : null}
    <Input placeholder=" ..." value={q} onChange={(e) => setQ(e.target.value)} className="max-w-xs" /
  >
  <div className="grid grid-cols-1 gap-3 sm:grid-cols-2 lg:grid-cols-3">
    {OPTION_CATEGORIES.map((cat) => {
      const par = parentSel[cat.key] ?? "";
      const items = rows.filter(
        (r) =>
          r.category === cat.key &&
          (!q || r.value.toLowerCase().includes(q.toLowerCase())) &&
          (!par || r.parent === par),
      );
      return (
        <Card key={cat.key}>
          <h3 className="mb-2 text-sm font-bold text-slate-800">
            {cat.label} <span className="text-xs text-slate-400">{toPersianDigits(items.length)}</span>
          </h3>
          {cat.key === "city" || cat.key === "region" ? (
            <select
              value={parentSel[cat.key] ?? ""}
              onChange={(e) => setParentSel({ ...parentSel, [cat.key]: e.target.value })}
              className="mb-2 w-full rounded-xl border border-slate-300 px-3 py-2 text-xs"
            >
              <option value="">
                {cat.key === "city" ? " — " : " — "}
              </option>
              {[...new Set(rows.filter((r) => r.category === (cat.key === "city" ? "province" : "city"))).map((r) =>
                r.value)
                ].sort()
                .map((p) => (
                  <option key={p} value={p}>
                    {p}
                  </option>
                ))}
            </select>
          ) : null}
        </Card>
      );
    })}
  </div>

```

```

    </select>
  ) : null}
  <div className="mb-3 flex gap-2">
    <Input
      value={draft[cat.key] ?? ""}
      onChange={(e) => setDraft({ ...draft, [cat.key]: e.target.value })}
      onKeyDown={(e) => {
        if (e.key === "Enter") add(cat.key);
      }}
      placeholder=" ..."
    />
    <Button onClick={() => add(cat.key)}></Button>
  </div>
  <ul className="max-h-60 space-y-1 overflow-y-auto">
    {items.map((it) =>
      editing?.id === it.id ? (
        <li key={it.id} className="flex items-center gap-1 rounded-lg bg-amber-50 px-2 py-1.5">
          <Input
            value={editing.value}
            onChange={(e) => setEditing({ ...editing, value: e.target.value })}
            className="px-2 py-1 text-xs"
            autoFocus
          />
          <button onClick={saveEdit} className="rounded-lg bg-emerald-600 px-2 py-1 text-[11px] font-bold text-white">
            </button>
          <button
            onClick={() => setEditing(null)}
            className="rounded-lg bg-slate-200 px-2 py-1 text-[11px] font-bold text-slate-700"
          >
            </button>
        </li>
      ) : (
        <li key={it.id} className="flex items-center justify-between gap-2 rounded-lg bg-slate-50 px-3 py-1.5 text-sm">
          <span className="min-w-0 truncate">
            {it.value}
            {it.parent ? <span className="mr-1 text-[10px] text-teal-600">({it.parent})</span> : null}
            {it.createdBy ? <span className="mr-2 text-[10px] text-slate-400">({it.createdBy})</span> : null}
          </span>
          <span className="flex shrink-0 gap-1">
            {canDelete ? (
              <>
                <button
                  onClick={() => setEditing({ id: it.id, value: it.value })}
                  className="rounded-lg bg-sky-100 px-2 py-0.5 text-[11px] font-bold text-sky-700"
                >
                  </button>
                <button
                  onClick={() => remove(it.id, it.value)}
                  className="rounded-lg bg-rose-100 px-2 py-0.5 text-[11px] font-bold text-rose-700"
                >
                  </button>
              </>
            ) : null}
          </span>
        </li>
      ),
    )}
    {items.length === 0 ? <li className="text-xs text-slate-400"> </li> : null}
  </ul>
</Card>
);
}}}
</div>
</div>
);
}

```

```

@@ src/components/screens/ReportScreen.tsx typescript
"use client";
import { useCallback, useEffect, useMemo, useState } from "react";
import { Card, SectionTitle } from "@components/ui";
import TargetPanel from "@components/screens/TargetPanel";
import { JALALI_MONTHS, toPersianDigits } from "@lib/jalali";
import { DEFAULT_PRODUCTS, type ProductConfig } from "@lib/defaults";
import { downloadFile } from "@lib/download";
type Row = {
  period: string;

```

```

repName: string;
pharmacies: number;
doctors: number;
orders: number;
units: number;
bonus: number;
trips: number;
};
export default function ReportScreen({ compact = false }: { compact?: boolean }) {
  const [rows, setRows] = useState<Row[]>([]);
  const [byRep, setByRep] = useState<
    { period: string; repName: string; items: Record<string, number>; bonuses: Record<string, number> }[]
  >([]);
  const [rep, setRep] = useState("");
  const [products, setProducts] = useState<ProductConfig[]>(DEFAULT_PRODUCTS);
  const [year, setYear] = useState("");
  const [month, setMonth] = useState("");
  const load = useCallback(async () => {
    const res = await fetch("/api/reports", { cache: "no-store" });
    if (res.ok) {
      const d = await res.json();
      setRows(d.rows ?? []);
      setByRep(d.productByRep ?? []);
    }
  }, []);
  useEffect(() => {
    load();
    fetch("/api/settings?key=products", { cache: "no-store" })
      .then((r) => (r.ok ? r.json() : null))
      .then((d) => {
        const v = d?.value as ProductConfig[] | null;
        if (Array.isArray(v) && v.length) setProducts(v.filter((p) => p.enabled !== false));
      })
      .catch(() => undefined);
  }, [load]);
  const years = useMemo(() => [...new Set(rows.map((r) => r.period.slice(0, 4)))]).sort().reverse(), [rows]);
  const filtered = useMemo(
    () =>
      rows.filter(
        (r) => (!year || r.period.startsWith(year)) && (!month || r.period.slice(5, 7) === month.padStart(2, "0")),
      ),
    [rows, year, month],
  );
  const sum = filtered.reduce(
    (a, r) => ({
      pharmacies: a.pharmacies + r.pharmacies,
      doctors: a.doctors + r.doctors,
      orders: a.orders + r.orders,
      units: a.units + r.units,
      bonus: a.bonus + r.bonus,
      trips: a.trips + r.trips,
    }),
    { pharmacies: 0, doctors: 0, orders: 0, units: 0, bonus: 0, trips: 0 },
  );
  const reps = useMemo(() => [...new Set(byRep.map((r) => r.repName)]).sort(), [byRep]);
  const byRepFiltered = useMemo(
    () =>
      byRep.filter(
        (r) =>
          (!year || r.period.startsWith(year)) &&
          (!month || r.period.slice(5, 7) === month.padStart(2, "0")) &&
          (!rep || r.repName === rep),
      ),
    [byRep, year, month, rep],
  );
  const monthName = (p: string) => {
    const m = Number(p.slice(5, 7));
    return `${JALALI_MONTHS[m - 1]} ?? "" ${toPersianDigits(p.slice(0, 4))}`;
  };
  return (
    <div className="space-y-4">
      {!compact ? <SectionTitle icon=""> </SectionTitle> : null}
      <TargetPanel />
      <Card>
        <div className="mb-3 flex flex-wrap items-center gap-2">
          <select
            value={year}
            onChange={(e) => setYear(e.target.value)}
            className="rounded-xl border border-slate-300 px-3 py-2 text-sm"
          >
            <option value=""> </option>
            {years.map((y) => (
              <option key={y} value={y}>
                {toPersianDigits(y)}
              </option>
            ))}
          </select>
        </div>
      </Card>
    </div>
  );
}

```

```

        </option>
      )}}
    </select>
    <select
      value={month}
      onChange={(e) => setMonth(e.target.value)}
      className="rounded-xl border border-slate-300 px-3 py-2 text-sm"
    >
      <option value=""> </option>
      {JALALI_MONTHS.map((m, i) => (
        <option key={m} value={String(i + 1)}>
          {m}
        </option>
      )}}
    </select>
    <div className="flex-1" />
    <button
      onClick={() =>
        downloadFile(
          `/api/export?type=orders${year ? `&year=${year}` : ""}${month ? `&month=${month}` : ""}`,
          "orders.xls",
        )
      }
      className="rounded-xl bg-emerald-600 px-3 py-2 text-xs font-bold text-white"
    >

  </button>
</div>
<div className="mb-3 grid grid-cols-3 gap-2 sm:grid-cols-6">
  {[
    ["", sum.pharmacies, ""],
    ["", sum.doctors, ""],
    ["", sum.orders, ""],
    [" ", sum.units, ""],
    [" ", sum.bonus, ""],
    [" ", sum.trips, ""],
  ]}.map(([l, v, i]) => (
    <div key={String(l)} className="rounded-xl bg-slate-50 p-3 text-center ring-1 ring-slate-200">
      <div className="text-lg">{i as string}</div>
      <div className="text-lg font-black text-teal-700">{toPersianDigits(v as number)}</div>
      <div className="text-[10px] font-bold text-slate-500">{l as string}</div>
    </div>
  ))}
</div>
<div className="scroll-x">
  <table className="w-full min-w-[680px] text-right text-xs sm:text-sm">
    <thead>
      <tr className="bg-slate-100 text-slate-600">
        <th className="px-2 py-2"></th>

        <th className="px-2 py-2"> </th>
        <th className="px-2 py-2"> </th>
        <th className="px-2 py-2"> </th>
        <th className="px-2 py-2"> </th>
        <th className="px-2 py-2"> </th>
        <th className="px-2 py-2"> </th>
      </tr>
    </thead>
    <tbody>
      {filtered.map((r) => (
        <tr key={` ${r.period} - ${r.repName}`} className="border-b border-slate-100">
          <td className="px-2 py-2 font-bold text-teal-700">{monthName(r.period)}</td>
          <td className="px-2 py-2">{r.repName}</td>
          <td className="px-2 py-2">{toPersianDigits(r.pharmacies)}</td>
          <td className="px-2 py-2">{toPersianDigits(r.doctors)}</td>
          <td className="px-2 py-2">{toPersianDigits(r.orders)}</td>
          <td className="px-2 py-2">{toPersianDigits(r.units)}</td>
          <td className="px-2 py-2">{toPersianDigits(r.bonus)}</td>
          <td className="px-2 py-2">{toPersianDigits(r.trips)}</td>
        </tr>
      ))}
      {filtered.length === 0 ? (
        <tr>
          <td colspan={8} className="py-6 text-center text-slate-400">

          </td>
        </tr>
      ) : null}
    </tbody>
  </table>
</div>
</Card>
{!compact ? (
  <Card>

```

```

<div className="mb-2 flex flex-wrap items-center gap-2">
<h3 className="text-sm font-bold text-slate-700"> — </h3>
  <select
    value={rep}
    onChange={(e) => setRep(e.target.value)}
    className="rounded-xl border border-slate-300 px-3 py-2 text-xs"
  >
    <option value=""> </option>
    {reps.map((r) => (
      <option key={r} value={r}>
        {r}
      </option>
    ))}
  </select>
</div>
<div className="scroll-x">
  <table className="w-full min-w-[760px] text-right text-xs">
    <thead>
      <tr className="bg-slate-100 text-slate-600">
        <th className="px-2 py-2"> </th>
        <th className="px-2 py-2"> </th>
        {products.map((p) => (
          <th key={p.key} className="px-2 py-2 text-center">
            {p.label}
            <div className="text-[9px] font-normal text-slate-400"> / </div>
          </th>
        ))}
        <th className="px-2 py-2 text-center"></th>
      </tr>
    </thead>
    <tbody>
      {byRepFiltered.map((r) => {
        const sumUnits = products.reduce((a, p) => a + (r.items[p.key] ?? 0), 0);
        return (
          <tr key={`_${r.period}-${r.repName}`} className="border-b border-slate-100">
            <td className="whitespace-nowrap px-2 py-2 font-bold text-teal-700">{monthName(r.period)}</td>
            <td className="whitespace-nowrap px-2 py-2 font-semibold text-slate-700">{r.repName}</td>
            {products.map((p) => (
              <td key={p.key} className="px-2 py-2 text-center">
                <span className="font-bold text-slate-800">{toPersianDigits(r.items[p.key] ?? 0)}</span>
                <span className="text-slate-400"> / </span>
                <span className="text-emerald-600">{toPersianDigits(r.bonuses[p.key] ?? 0)}</span>
              </td>
            ))}
            <td className="px-2 py-2 text-center font-black text-teal-700">{toPersianDigits(sumUnits)}</td>
          </tr>
        );
      })}

      {byRepFiltered.length === 0 ? (
        <tr>
          <td colspan={products.length + 3} className="py-6 text-center text-slate-400">

          </td>
        </tr>
      ) : (
        <tr className="bg-teal-50 font-black text-teal-800">
          <td className="px-2 py-2" colspan={2}>

          </td>
          {products.map((p) => (
            <td key={p.key} className="px-2 py-2 text-center">
              {toPersianDigits(byRepFiltered.reduce((a, r) => a + (r.items[p.key] ?? 0), 0))}
              <span className="text-slate-400"> / </span>
              <span className="text-emerald-700">
                {toPersianDigits(byRepFiltered.reduce((a, r) => a + (r.bonuses[p.key] ?? 0), 0))}
              </span>
            </td>
          ))}
          <td className="px-2 py-2 text-center">
            {toPersianDigits(
              byRepFiltered.reduce(
                (a, r) => a + products.reduce((x, p) => x + (r.items[p.key] ?? 0), 0),
                0,
              ),
            )}
          </td>
        </tr>
      )}
    </tbody>
  </table>
</div>
<p className="mt-2 text-[11px] text-slate-400">
« » « » .
</p>

```

```

        </Card>
      ) : null}
    </div>
  );
}

@@ src/components/screens/TargetPanel.tsx typescript
"use client";
import { useCallback, useEffect, useState } from "react";
import { Badge, Card, SectionTitle } from "@components/ui";
import { useLive } from "@lib/useLive";
import { JALALI_MONTHS, toPersianDigits, todayJalali } from "@lib/jalali";
type Progress = {
  productKey: string;
  productLabel: string;
  quantity: number;
  priceDistributor: number;
  pricePharmacy: number;
  sold: number;
  bonus: number;
  remaining: number;
  percent: number;
  soldValueDistributor: number;
  soldValuePharmacy: number;
};
type Totals = {
  quantity: number;
  sold: number;
  bonus: number;
  remaining: number;
  valueDistributor: number;
  valuePharmacy: number;
  soldValueDistributor: number;
  soldValuePharmacy: number;
};
const money = (n: number) => toPersianDigits(Math.round(n).toLocaleString("en-US"));
const currentPeriod = () => todayJalali().slice(0, 7);
/** — */
export default function TargetPanel({ userId, compact = false }: { userId?: number; compact?: boolean }) {
  const [rows, setRows] = useState<Progress[]>([]);
  const [totals, setTotals] = useState<Totals | null>(null);
  const [percent, setPercent] = useState(0);
  const [period, setPeriod] = useState(currentPeriod());

  const load = useCallback(async () => {
    const q = new URLSearchParams({ period });
    if (userId) q.set("userId", String(userId));
    const res = await fetch(`/api/targets?${q}`, { cache: "no-store" });
    if (!res.ok) return;
    const d = await res.json();
    setRows(d.progress ?? []);
    setTotals(d.totals ?? null);
    setPercent(d.percent ?? 0);
  }, [period, userId]);
  useLive(load, 30000);
  const periods = (() => {
    const [y, m] = currentPeriod().split("/").map(Number);
    const out: string[] = [];
    for (let i = -5; i <= 1; i++) {
      let mm = m + i;
      let yy = y;
      while (mm <= 0) {
        mm += 12;
        yy -= 1;
      }
      while (mm > 12) {
        mm -= 12;
        yy += 1;
      }
      out.push(`${yy}/${String(mm).padStart(2, "0")}`);
    }
    return [...new Set(out)];
  })();
  const label = (p: string) => `${JALALI_MONTHS[Number(p.slice(5, 7)) - 1] ?? ""} ${toPersianDigits(p.slice(0, 4))}`;
  const active = rows.filter((r) => r.quantity > 0);
  if (active.length === 0 && !compact) {
    return (
      <Card>
        <SectionTitle icon=""> </SectionTitle>
        <p className="py-4 text-center text-xs text-slate-400">
          . «» .
        </p>
      </Card>
    );
  }
}

```



```

if (active.length === 0) return null;
return (
  <Card>
    <div className="mb-3 flex flex-wrap items-center justify-between gap-2">
      <SectionTitle icon=""> </SectionTitle>
      <div className="flex items-center gap-2">
        <select
          value={period}
          onChange={(e) => setPeriod(e.target.value)}
          className="rounded-xl border border-slate-300 px-2 py-1.5 text-xs"
        >
          {periods.map((p) => (
            <option key={p} value={p}>
              {label(p)}
            </option>
          ))}
        </select>
        <Badge tone={percent >= 100 ? "green" : percent >= 60 ? "amber" : "slate"}>
          : {toPersianDigits(percent)}
        </Badge>
      </div>
    </div>
    {totals ? (
      <div className="mb-3 grid grid-cols-2 gap-2 sm:grid-cols-4">
        {
          [
            ["", toPersianDigits(totals.quantity), "bg-slate-50 text-slate-700"],
            ["", toPersianDigits(totals.sold), "bg-teal-50 text-teal-700"],
            ["", toPersianDigits(totals.remaining), "bg-amber-50 text-amber-700"],
            ["", toPersianDigits(totals.bonus), "bg-emerald-50 text-emerald-700"],
          ].map(([l, v, c]) => (
            <div key={l} className={`rounded-xl p-2.5 text-center ring-1 ring-slate-200 ${c}`}>
              <div className="text-lg font-black">{v}</div>
              <div className="text-[10px] font-bold opacity-70">{l}</div>
            </div>
          ))}
        </div>
      ) : null}

    <div className="space-y-1.5">
      {active.map((r) => (
        <div key={r.productKey} className="rounded-xl bg-slate-50 p-2 ring-1 ring-slate-200">
          <div className="flex flex-wrap items-center gap-2 text-[11px]">
            <span className="font-bold text-slate-800">{r.productLabel}</span>
            <span className="text-slate-500">
              <b className="text-slate-700">{toPersianDigits(r.quantity)}</b>
            </span>
            <span className="text-teal-700">
              <b>{toPersianDigits(r.sold)}</b>
            </span>
            <span className={r.remaining === 0 ? "text-emerald-700" : "text-rose-700"}>
              <b>{toPersianDigits(r.remaining)}</b>
            </span>
            <span className="mr-auto text-[10px] font-black text-slate-600">{toPersianDigits(r.percent)}</span>
          </div>
          <div className="mt-1 h-1.5 overflow-hidden rounded bg-slate-200">
            <div
              className={`h-full transition-all ${r.percent >= 100 ? "bg-emerald-500" : r.percent >= 60 ? "bg-teal-500" : "bg-amber-500"}`}
              style={{ width: `${Math.min(100, r.percent)}%` }}
            />
          </div>
        </div>
      ))}
    </div>
    {totals && (totals.soldValueDistributor > 0 || totals.valueDistributor > 0) ? (
      <div className="mt-3 grid grid-cols-1 gap-2 text-[11px] sm:grid-cols-2">
        <div className="rounded-xl bg-slate-50 p-2.5 ring-1 ring-slate-200">
          (: <b className="text-slate-800">{money(totals.valueDistributor)}</b>
          <div className="text-teal-700">
            : <b>{money(totals.soldValueDistributor)}</b>
          </div>
        </div>
        <div className="rounded-xl bg-slate-50 p-2.5 ring-1 ring-slate-200">
          (: <b className="text-slate-800">{money(totals.valuePharmacy)}</b>
          <div className="text-teal-700">
            : <b>{money(totals.soldValuePharmacy)}</b>
          </div>
        </div>
      </div>
    ) : null}
  </Card>
);
}

```

@@ src/app/admin/activity/page.tsx typescript

```

"use client";
import { useCallback, useEffect, useMemo, useState } from "react";
import { Badge, Button, Card, Input, SectionTitle } from "@components/ui";
import { BarChart } from "@components/Charts";
import { tehranDateTime, toPersianDigits } from "@lib/jalali";
import { useLive } from "@lib/useLive";
type Log = { id: number; userId: number | null; userName: string; action: string; detail: string; createdAt: string };
type Rep = {
  id: number;
  fullName: string;
  role: string;
  active: boolean;
  lastSeenAt: string | null;
  pharmacies: number;
  doctors: number;
  orders: number;
  trips: number;
  activeTrip: number;
  lastAction: string | null;
  lastActionAt: string | null;
};
};
export default function ActivityPage() {
  const [logs, setLogs] = useState<Log[]>([]);
  const [reps, setReps] = useState<Rep[]>([]);
  const [q, setQ] = useState("");
  const [rep, setRep] = useState("");
  const [live, setLive] = useState(true);
  const load = useCallback(async () => {
    const res = await fetch("/api/activity", { cache: "no-store" });
    if (!res.ok) return;
    const d = await res.json();
    setLogs(d.logs ?? []);
    setReps(d.reps ?? []);
  }, []);
  useLive(load, 12000, live);
  const filtered = useMemo(
    () =>
      logs.filter(
        (l) =>
          (!rep || l.userName === rep) &&
          (!q || `${l.userName} ${l.action} ${l.detail}`.toLowerCase().includes(q.toLowerCase())),
      ),
    [logs, rep, q],
  );
  const perRep = useMemo(() => {
    const m = new Map<string, number>();
    for (const l of filtered) m.set(l.userName, (m.get(l.userName) ?? 0) + 1);
    return [...m.entries()].map(([label, value]) => ({ label, value })).sort((a, b) => b.value - a.value);
  }, [filtered]);
  const perAction = useMemo(() => {
    const m = new Map<string, number>();
    for (const l of filtered) m.set(l.action, (m.get(l.action) ?? 0) + 1);
    return [...m.entries()].map(([label, value]) => ({ label, value })).sort((a, b) => b.value - a.value);
  }, [filtered]);
  return (
    <div className="space-y-4">
      <div className="flex flex-wrap items-center justify-between gap-2">
        <SectionTitle icon=""> </SectionTitle>
        <Button variant={live ? "success" : "ghost"} onClick={() => setLive((v) => !v)}>
          {live ? " ( )" : ""}
        </Button>
      </div>
      <div className="grid grid-cols-1 gap-3 lg:grid-cols-2">
        <Card>
          <h3 className="mb-2 text-sm font-bold text-slate-700"> </h3>
          <BarChart data={perRep} />
        </Card>
        <Card>
          <h3 className="mb-2 text-sm font-bold text-slate-700"> </h3>
          <BarChart data={perAction} />
        </Card>
      </div>
      <Card>
        <h3 className="mb-2 text-sm font-bold text-slate-700"> — </h3>
        <div className="grid grid-cols-1 gap-2 sm:grid-cols-2 lg:grid-cols-3">
          {reps.map((r) => (
            <button
              key={r.id}
              onClick={() => setRep(rep === r.fullName ? "" : r.fullName)}
              className={`rounded-xl p-3 text-right ring-1 transition ${
                rep === r.fullName ? "bg-teal-50 ring-teal-300" : "bg-slate-50 ring-slate-200 hover:bg-slate-100"
              }`}
            >
              <div className="flex flex-wrap items-center gap-1">

```

```

        <span className="text-sm font-bold text-slate-800">{r.fullName}</span>
        {r.activeTrip > 0 ? <Badge tone="green"> </Badge> : null}
        {!r.active ? <Badge tone="amber"></Badge> : null}
      </div>
      <div className="mt-1 grid grid-cols-4 gap-1 text-center text-[10px]">
        <span className="rounded bg-white py-1 ring-1 ring-slate-200"> {toPersianDigits(r.pharmacies)}</span>
        <span className="rounded bg-white py-1 ring-1 ring-slate-200"> {toPersianDigits(r.doctors)}</span>
        <span className="rounded bg-white py-1 ring-1 ring-slate-200"> {toPersianDigits(r.orders)}</span>
        <span className="rounded bg-white py-1 ring-1 ring-slate-200"> {toPersianDigits(r.trips)}</span>
      </div>
      <div className="mt-1 truncate text-[10px] text-slate-500">{r.lastAction ?? " "}</div>
      <div className="text-[10px] text-slate-400">
        {r.lastActionAt ? toPersianDigits(r.lastActionAt) : ""}
      </div>
    </button>
  )}
</div>
</Card>
{ /* ----- */
{rep ? (
  <Card>
    <div className="mb-2 flex flex-wrap items-center justify-between gap-2">
      <h3 className="text-sm font-black text-teal-800"> «{rep}»</h3>
      <Button variant="ghost" onClick={() => setRep("")}>

    </div>
    <div className="mb-3 grid grid-cols-2 gap-2 sm:grid-cols-4">
      {(() => {
        const r = reps.find((x) => x.fullName === rep);
        return [
          [" ", r?.pharmacies ?? 0],
          [" ", r?.doctors ?? 0],
          [" ", r?.orders ?? 0],
          [" ", r?.trips ?? 0],
        ].map(([l, v]) => (
          <div key={String(l)} className="rounded-xl bg-slate-50 p-3 text-center ring-1 ring-slate-200">
            <div className="text-lg font-black text-teal-700">{toPersianDigits(v as number)}</div>
            <div className="text-[10px] font-bold text-slate-500">{l as string}</div>
          </div>
        ))
      })()}
    </div>
    <div className="mb-3">
      <h4 className="mb-1 text-xs font-bold text-slate-600"> </h4>
      <div className="flex flex-wrap gap-1">
        {perAction.map((a) => (
          <span key={a.label} className="rounded-lg bg-teal-50 px-2 py-1 text-[11px] font-bold text-teal-700 ring-1 ring-teal-200">
            {a.label}: {toPersianDigits(a.value)}
          </span>
        ))}
        {perAction.length === 0 ? <span className="text-[11px] text-slate-400"> </span> : null}
      </div>
    </div>
    <div className="max-h-72 space-y-1 overflow-y-auto">
      {filtered.map((l) => (
        <div key={l.id} className="flex flex-wrap items-center gap-2 rounded-xl bg-slate-50 px-3 py-2 text-xs">
          <span className="rounded-lg bg-teal-100 px-2 py-0.5 text-[10px] font-bold text-teal-700">{l.action}</spa

n>
          {l.detail ? <span className="text-slate-600">{l.detail}</span> : null}
          <span className="mr-auto text-[10px] text-slate-400">{toPersianDigits(l.createdAt)}</span>
        </div>
      ))}
      {filtered.length === 0 ? <p className="py-4 text-center text-slate-400"> </p> : null}
    </div>
  </Card>
) : null}
<Card>
  <div className="mb-2 flex flex-wrap items-center gap-2">
    <h3 className="text-sm font-black text-slate-700"> </h3>
    <input placeholder=" ..." value={q} onChange={(e) => setQ(e.target.value)} className="max-w-[220px]" />
    {rep ? (
      <Button variant="ghost" onClick={() => setRep("")}>

    «{rep}»
    </Button>
  ) : null}
  <Badge tone="slate">{toPersianDigits(filtered.length)} </Badge>
</div>
<div className="max-h-[480px] space-y-1 overflow-y-auto">
  {filtered.map((l) => (
    <div key={l.id} className="flex flex-wrap items-center gap-2 rounded-xl bg-slate-50 px-3 py-2 text-xs">
      <span className="font-bold text-slate-800">{l.userName}</span>
      <span className="rounded-lg bg-teal-100 px-2 py-0.5 text-[10px] font-bold text-teal-700">{l.action}</span>
    </div>
  ))
}

```

```

        {l.detail ? <span className="text-slate-600">{l.detail}</span> : null}
        <span className="mr-auto text-[10px] text-slate-400">{tehranDateTime(l.createdAt)}</span>
      </div>
    )))
    {filtered.length === 0 ? <p className="py-6 text-center text-slate-400"> </p> : null}
  </div>
</Card>
</div>
);
}

@@ src/app/admin/backup/page.tsx typescript
"use client";
import { useCallback, useEffect, useRef, useState } from "react";
import { Alert, Badge, Button, Card, Field, Input, SectionTitle } from "@components/ui";
import { tehranDateTime, toPersianDigits } from "@lib/jalali";
import { useConfirm } from "@components/Confirm";
type Cfg = { enabled: boolean; minutes: number; withFiles: boolean; folderName: string };
const DEFAULT_CFG: Cfg = { enabled: false, minutes: 30, withFiles: false, folderName: "namayandeelmi-javad" };
const IDB = "sek-backup";

//
function idb(): Promise<IDBDatabase> {
  return new Promise((res, rej) => {
    const r = indexedDB.open(IDB, 1);
    r.onupgradeneeded = () => r.result.createObjectStore("h");
    r.onsuccess = () => res(r.result);
    r.onerror = () => rej(r.error);
  });
}
async function saveHandle(h: unknown) {
  const db = await idb();
  const tx = db.transaction("h", "readwrite");
  tx.objectStore("h").put(h, "dir");
}
async function loadHandle(): Promise<FileSystemDirectoryHandle | null> {
  try {
    const db = await idb();
    return await new Promise((res) => {
      const tx = db.transaction("h", "readonly");
      const rq = tx.objectStore("h").get("dir");
      rq.onsuccess = () => res((rq.result as FileSystemDirectoryHandle) ?? null);
      rq.onerror = () => res(null);
    });
  } catch {
    return null;
  }
}
export default function BackupPage() {
  const [cfg, setCfg] = useState<Cfg>(DEFAULT_CFG);
  const [msg, setMsg] = useState("");
  const [last, setLast] = useState("");
  const [dirOk, setDirOk] = useState(false);
  const [busy, setBusy] = useState(false);
  const [log, setLog] = useState<string[]>([]);
  const timer = useRef<ReturnType<typeof setInterval> | null>(null);
  const [dirName, setDirName] = useState("");
  const confirm = useConfirm();
  const supportsFs = typeof window !== "undefined" && "showDirectoryPicker" in window;
  const load = useCallback(async () => {
    const res = await fetch("/api/settings?key=backup", { cache: "no-store" });
    if (res.ok) {
      const d = await res.json();
      if (d.value) setCfg({ ...DEFAULT_CFG, ...(d.value as Cfg) });
    }
    setLast(localStorage.getItem("sek_backup_last") ?? "");
    const h = await loadHandle();
    setDirOk(!!h);
    setDirName(h?.name ?? localStorage.getItem("sek_backup_dir") ?? "");
  }, []);
  useEffect(() => {
    load();
  }, [load]);
  const saveCfg = async (next: Cfg) => {
    setCfg(next);
    await fetch("/api/settings", {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ key: "backup", value: next }),
    });
    setMsg("");
  };
  const pickFolder = async () => {
    try {
      const w = window as unknown as {

```

```

        showDirectoryPicker: (o?: { mode?: string }) => Promise<FileSystemDirectoryHandle>;
    };
    const handle = await w.showDirectoryPicker({ mode: "readwrite" });
    await saveHandle(handle);
    setDirOk(true);
    setDirName(handle.name ?? "");
    localStorage.setItem("sek_backup_dir", handle.name ?? "");
    setMsg(`«${handle.name}» . (Downloads
    ).`);
    } catch {
        setMsg(" ");
    }
};
const runBackup = useCallback(
    async (auto = false, forceDownload = false) => {
        setBusy(true);
        try {
            const res = await fetch(`/api/backup${cfg.withFiles ? "?files=1" : ""}`, { cache: "no-store" });
            if (!res.ok) throw new Error(" ");
            const blob = await res.blob();
            const stamp = new Date().toISOString().replace(/[:.]/g, "-").slice(0, 16);
            const name = `namayandeelmi-backup-${stamp}.json`;
            const handle = forceDownload ? null : await loadHandle();
            if (handle) {
                const perm = await (
                    handle as unknown as { queryPermission: (o: { mode: string }) => Promise<string> }
                ).queryPermission({ mode: "readwrite" });
                if (perm !== "granted") {
                    await (
                        handle as unknown as { requestPermission: (o: { mode: string }) => Promise<string> }
                    ).requestPermission({ mode: "readwrite" });
                }
            }
            const fh = await handle.getFileHandle(name, { create: true });
            const ws = await fh.createWritable();
            await ws.write(blob);
            await ws.close();
            setLog((l) => [`${tehranDateTime(new Date())} — : ${name}`, ...l].slice(0, 20));
        } else if (forceDownload) {
            const url = URL.createObjectURL(blob);
            const a = document.createElement("a");
            a.href = url;
            a.download = name;
            a.click();
            URL.revokeObjectURL(url);
            setLog((l) => [`${tehranDateTime(new Date())} — : ${name}`, ...l].slice(0, 20));
        } else if (supportsFs) {
            // : (Downloads)
            try {
                const w = window as unknown as {
                    showSaveFilePicker: (o: unknown) => Promise<FileSystemFileHandle>;
                };
                const fh = await w.showSaveFilePicker({
                    suggestedName: name,
                    types: [{ description: "JSON", accept: { "application/json": [".json"] } }],
                });
                const ws = await fh.createWritable();
                await ws.write(blob);
                await ws.close();
            }
            setLog((l) => [`${tehranDateTime(new Date())} — : ${name}`, ...l].slice(0, 20));
        } catch {
            setBusy(false);
            setMsg(" ");
            return;
        }
        } else {
            const url = URL.createObjectURL(blob);
            const a = document.createElement("a");
            a.href = url;
            a.download = name;
            a.click();
            URL.revokeObjectURL(url);
            setLog((l) => [`${tehranDateTime(new Date())} — : ${name}`, ...l].slice(0, 20));
        }
        const now = new Date().toISOString();
        localStorage.setItem("sek_backup_last", now);
        setLast(now);
        if (!auto) setMsg(" ");
    } catch (e) {
        setMsg(` ${e instanceof Error ? e.message : " "}`);
    } finally {
        setBusy(false);
    }
},
[cfg.withFiles, supportsFs],

```

```

);
//
useEffect(() => {
  if (timer.current) clearInterval(timer.current);
  if (!cfg.enabled) return;
  const ms = Math.max(1, cfg.minutes) * 60000;
  timer.current = setInterval(() => {
    const lastAt = Number(new Date(localStorage.getItem("sek_backup_last") ?? 0));
    if (Date.now() - lastAt >= ms) runBackup(true);
  }, 30000);
  return () => {
    if (timer.current) clearInterval(timer.current);
  };
}, [cfg.enabled, cfg.minutes, runBackup]);
const winScript = `@echo off
rem =====
rem « »
rem      backup.bat E:\\porozheha\\backup
rem      Task Scheduler
rem =====
set URL=${typeof window !== "undefined" ? window.location.origin : "https://your-app.onrender.com"}/api/backup
set DEST=E:\\porozheha\\backup\\${cfg.folderName}
set COOKIE=%-dp0cookie.txt
if not exist "%DEST%" mkdir "%DEST%"
for /f "tokens=1-4 delims=/ " %%a in ('date /t') do set D=%%d-%%b-%%c
set T=%time:~0,2%%time:~3,2%
curl -s -c "%COOKIE%" -X POST "%URL:/api/backup=/api/auth/login%" -H "Content-Type: application/json" -d "{\"username\\":\"\\\"admin\\\",\\\"password\\\":\"\\\"PASSWORD\\\",\\\"mode\\\":\"\\\"admin\\\",\\\"remember\\\":true}\"" > nul
curl -s -b "%COOKIE%" "%URL%" -o "%DEST%\\backup-%D%-%%T%.json"
echo backup saved to %DEST%
`;
return (
  <div className="space-y-4">
    <SectionTitle icon=""> </SectionTitle>
    {msg ? <Alert kind={msg.startsWith("") ? "error" : "success"}>{msg}</Alert> : null}
    <Card>
      <div className="flex flex-wrap items-center gap-2">
        <Button
          onClick={async () => {
            if (
              !(await confirm({
                title: "",
                message: dirOk
              }` «${dirName}» `
                : " ",
                confirmText: " ",
              )))
              return;
            runBackup(false);
          }}
          disabled={busy}
        >
          {busy ? " ..." : " "}
        </Button>
        <Button variant="ghost" onClick={() => runBackup(false, true)} disabled={busy}>
          </Button>
        {supportsFs ? (
          <Button variant={dirOk ? "success" : "soft"} onClick={pickFolder}>
            {dirOk ? ` ${dirName} || "" — ` : " "}
          </Button>
        ) : null}
        <Badge tone={last ? "green" : "amber"}>
          : {last ? tehranDateTime(last) : ""}
        </Badge>
      </div>
      {!dirOk ? (
        <Alert kind="error">
          .
          « » .
        </Alert>
      ) : null}
      <Alert kind="info">
        <b> E:\\porozheha\\backup\\{cfg.folderName}</b> —
        : « »
        <span dir="ltr"> E:\\porozheha\\backup\\{cfg.folderName} </span>
        ( ) .
      </Alert>
    </Card>
    <Card>
      <h3 className="mb-2 text-sm font-bold text-slate-700"> </h3>
      <div className="grid grid-cols-1 gap-3 sm:grid-cols-3">

```

```

<Field label=" ">
  <Input
    inputMode="numeric"
    value={String(cfg.minutes)}
    onChange={(e) => setCfg({ ...cfg, minutes: Number(e.target.value.replace(/\D/g, "")) || 1 })}
  />
</Field>
<Field label=" ">
  <Input value={cfg.folderName} onChange={(e) => setCfg({ ...cfg, folderName: e.target.value })} />
</Field>
<div className="flex flex-col justify-end gap-2">
  <label className="flex items-center gap-2 text-xs font-bold text-slate-600">
    <input
      type="checkbox"
      className="size-4 accent-teal-600"
      checked={cfg.withFiles}
      onChange={(e) => setCfg({ ...cfg, withFiles: e.target.checked })}
    />
    (
  </label>
  <label className="flex items-center gap-2 text-xs font-bold text-slate-600">
    <input
      type="checkbox"
      className="size-4 accent-teal-600"
      checked={cfg.enabled}
      onChange={(e) => setCfg({ ...cfg, enabled: e.target.checked })}
    />
  </label>
</div>
</div>
<div className="mt-3 flex flex-wrap gap-2">
  <Button
    onClick={async () => {
      if (
        !(await confirm({
          title: " ",
          message: cfg.enabled
            ? ` ${cfg.minutes} `
            : " ",
          confirmText: "",
        }))
      )
        return;
      saveCfg(cfg);
    }}
  >

  </Button>
  <Badge tone={cfg.enabled ? "green" : "slate"}>
{cfg.enabled ? ` — ${toPersianDigits(cfg.minutes)} ` : ""}
  </Badge>
</div>
<p className="mt-2 text-[11px] text-slate-500">
  :
  .
</p>
</Card>
<Card>
  <h3 className="mb-2 text-sm font-bold text-slate-700"> ( )</h3>
  <p className="mb-2 text-[11px] text-slate-500">
    <span dir="ltr">backup.bat</span>
    Task Scheduler ( ).
  </p>
  <pre className="scroll-x max-h-56 overflow-auto rounded-xl bg-slate-900 p-3 text-[11px] leading-5 text-emerald-200" dir="ltr">
    {winScript}
  </pre>
  <div className="mt-2 flex gap-2">
    <Button
      variant="soft"
      onClick={() => {
        navigator.clipboard?.writeText(winScript);
        setMsg(" ");
      }}
    >

    </Button>
    <Button
      variant="ghost"
      onClick={() => {
        const blob = new Blob([winScript], { type: "text/plain" });
        const a = document.createElement("a");

```

```

        a.href = URL.createObjectURL(blob);
        a.download = "backup.bat";
        a.click();
    }}
    >
        backup.bat
    </Button>
</div>
</Card>

{log.length ? (
    <Card>
        <h3 className="mb-2 text-sm font-bold text-slate-700">      </h3>
        <ul className="space-y-1 text-[11px] text-slate-600">
            {log.map((l, i) => (
                <li key={i} className="rounded-lg bg-slate-50 px-2 py-1">
                    {l}
                </li>
            ))}
        </ul>
    </Card>
) : null}
</div>
);
}

```

@@ src/app/admin/columns/page.tsx typescript

```

"use client";
import { useCallback, useEffect, useState } from "react";
import { Alert, Badge, Button, Card, Input, SectionTitle } from "@components/ui";
import {
    AVAILABLE_COLUMNS,
    AVAILABLE_FORM_FIELDS,
    DEFAULT_COLUMNS,
    DEFAULT_FORM_FIELDS,
    DEFAULT_PRODUCTS,
    type ColumnConfig,
    type FormFieldConfig,
    type ProductConfig,
} from "@lib/defaults";
import { useConfirm } from "@components/Confirm";
import { toPersianDigits } from "@lib/jalali";
const TABLES = [
    { key: "pharmacies", label: " " },
    { key: "doctors", label: " " },
    { key: "orders", label: " " },
];
export default function ColumnsPage() {
    const [cols, setCols] = useState<Record<string, ColumnConfig[]>>(DEFAULT_COLUMNS);
    const [formFields, setFormFields] = useState<Record<string, FormFieldConfig[]>>(DEFAULT_FORM_FIELDS);
    const [products, setProducts] = useState<ProductConfig[]>(DEFAULT_PRODUCTS);
    const [tab, setTab] = useState("pharmacies");
    const [msg, setMsg] = useState("");
    const [newProduct, setNewProduct] = useState("");
    const [addCol, setAddCol] = useState("");
    const [addField, setAddField] = useState("");
    const confirm = useConfirm();
    const load = useCallback(async () => {
        const res = await fetch("/api/settings", { cache: "no-store" });
        if (!res.ok) return;
        const d = await res.json();
        const v = d.values ?? {};
        setProducts((v.products as ProductConfig[]) ?? DEFAULT_PRODUCTS);
        /**          */
        const merge = (key: string, def: ColumnConfig[]) => {
            const stored = v[key] as ColumnConfig[] | undefined;
            if (!Array.isArray(stored) || stored.length === 0) return def;
            const have = new Set(stored.map((c) => c.key));
            const mandatory = def.filter((c) => c.key === "actions" && !have.has(c.key));
            return [...stored, ...mandatory];
        };
        setCols({
            pharmacies: merge("columns.pharmacies", DEFAULT_COLUMNS.pharmacies),
            doctors: merge("columns.doctors", DEFAULT_COLUMNS.doctors),
            orders: merge("columns.orders", DEFAULT_COLUMNS.orders),
        });
        const mergeFields = (key: string, def: FormFieldConfig[]) => {
            const stored = v[key] as FormFieldConfig[] | undefined;
            if (!Array.isArray(stored) || stored.length === 0) return def;
            const have = new Set(stored.map((f) => f.key));
            const mandatory = def.filter((f) => f.key === "location" && !have.has(f.key));
            return [...stored, ...mandatory];
        };
        setFormFields({
            pharmacies: mergeFields("fields.pharmacies", DEFAULT_FORM_FIELDS.pharmacies),

```



```

        doctors: mergeFields("fields.doctors", DEFAULT_FORM_FIELDS.doctors),
        orders: mergeFields("fields.orders", DEFAULT_FORM_FIELDS.orders),
    });
}, []);
useEffect(() => {
    load();
}, [load]);
const persist = async (key: string, value: unknown, success = "    ") => {
    const res = await fetch("/api/settings", {
        method: "PUT",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ key, value }),
    });
    setMsg(res.ok ? success : "    ");
    return res.ok;
};
const save = async (key: string, value: unknown) => {
    if (!(await confirm({ title: " ", message: " ", confirmTe
xt: " " })))
        return;
    await persist(key, value);
};
const move = (list: ColumnConfig[], i: number, dir: -1 | 1) => {
    const next = [...list];
    const j = i + dir;
    if (j < 0 || j >= next.length) return next;
    [next[i], next[j]] = [next[j], next[i]];
    return next;
};
const moveP = (i: number, dir: -1 | 1) => {
    const next = [...products];
    const j = i + dir;
    if (j < 0 || j >= next.length) return;
    [next[i], next[j]] = [next[j], next[i]];
    setProducts(next);
};
const current = cols[tab] ?? [];
const currentFields = formFields[tab] ?? [];
return (
    <div className="space-y-4">
        <SectionTitle icon=""> </SectionTitle>
        {msg ? <Alert kind="success">{msg}</Alert> : null}
        <Card>
            <div className="mb-3 flex flex-wrap gap-1 rounded-xl bg-slate-100 p-1 text-xs font-bold">
                {TABLES.map((t) => (
                    <button
                        key={t.key}
                        onClick={() => setTab(t.key)}
                        className={`rounded-lg px-3 py-2 ${tab === t.key ? "bg-teal-600 text-white" : "text-slate-600"}`}
                    >
                        {t.label}
                    </button>
                ))}
            </div>
            <p className="mb-2 text-[11px] text-slate-500">
                .
            </p>
            <ul className="space-y-1">
                {current.map((c, i) => (
                    <li key={c.key} className="flex flex-wrap items-center gap-2 rounded-xl bg-slate-50 px-3 py-2">
                        <span className="w-6 text-center text-[11px] text-slate-400">{toPersianDigits(i + 1)}</span>
                        <input
                            type="checkbox"
                            className="size-4 accent-teal-600"
                            checked={c.visible}
                            onChange={(e) => {
                                const next = [...current];
                                next[i] = { ...c, visible: e.target.checked };
                                setCols({ ...cols, [tab]: next });
                            }}
                        />
                        <Input
                            value={c.label}
                            onChange={(e) => {
                                const next = [...current];
                                next[i] = { ...c, label: e.target.value };
                                setCols({ ...cols, [tab]: next });
                            }}
                            className="max-w-[180px] px-2 py-1 text-xs"
                        />
                        <span className="text-[10px] text-slate-400">{c.key}</span>
                        <div className="mr-auto flex gap-1">
                            <button

```

```

        onClick={() => setCols({ ...cols, [tab]: move(current, i, -1) })}
        className="rounded-lg bg-white px-2 py-1 text-xs ring-1 ring-slate-200"
      >
    ↑
    </button>
    <button
      onClick={() => setCols({ ...cols, [tab]: move(current, i, 1) })}
      className="rounded-lg bg-white px-2 py-1 text-xs ring-1 ring-slate-200"
    >
    ↓
    </button>
    <button
      onClick={async () => {
        if (
          !(await confirm({
            title: " ",
            message: `«${c.label}»`,
            confirmText: "",
            danger: true,
          }))
        )
          return;
        setCols({ ...cols, [tab]: current.filter((_, x) => x !== i) });
      }}
      className="rounded-lg bg-rose-100 px-2 py-1 text-xs font-bold text-rose-700"
    >

    </button>
  </div>
</li>
)}}
</ul>
<div className="mt-3 flex flex-wrap items-center gap-2 rounded-xl bg-slate-50 p-2 ring-1 ring-slate-200">
  <span className="text-[11px] font-bold text-slate-600"> </span>
  <select
    value={addCol}
    onChange={(e) => setAddCol(e.target.value)}
    className="rounded-xl border border-slate-300 px-2 py-2 text-xs"
  >
    <option value=""> ...</option>
    {(AVAILABLE_COLUMNS[tab] ?? []).filter((a) => !current.some((c) => c.key === a.key))
      .map((a) => (
        <option key={a.key} value={a.key}>
          {a.label}
        </option>
      ))}
  </select>
  <Button
    variant="soft"
    onClick={async () => {
      const found = (AVAILABLE_COLUMNS[tab] ?? []).find((a) => a.key === addCol);
      if (!found) {
        setMsg(" ");
        return;
      }
      const next = [...current, { ...found, visible: true }];
      setCols({ ...cols, [tab]: next });
      setAddCol("");
    }}
  >
    await persist(`columns.${tab}`, next, `«${found.label}» `);
  </Button>
  <span className="text-[10px] text-slate-400">
    {toPersianDigits((AVAILABLE_COLUMNS[tab] ?? []).filter((a) => !current.some((c) => c.key === a.key)).length)}
  </span>
</div>
<div className="mt-3 flex gap-2">
  <Button onClick={() => save(`columns.${tab}`, current)}> </Button>
  <Button
    variant="ghost"
    onClick={async () => {
      if (!(await confirm({ title: " ", message: " ", confi
rmText: "" })))
        return;

      setCols({ ...cols, [tab]: DEFAULT_COLUMNS[tab] });
    }}
  >
  </Button>
</div>

```

```

</Card>
/* — */
<Card>
  <div className="mb-2 flex flex-wrap items-center justify-between gap-2">
    <div>
      <h3 className="text-sm font-black text-slate-800"> </h3>
      <p className="mt-1 text-[11px] text-slate-500">
        « » .
      </p>
    </div>
    <Badge tone="green">{toPersianDigits(currentFields.filter((f) => f.visible).length)} </Badge>
  </div>
  <ul className="space-y-1">
    {currentFields.map((f, i) => (
      <li key={f.key} className="flex flex-wrap items-center gap-2 rounded-xl bg-indigo-50/60 px-3 py-2 ring-1 rin
g-indigo-100">
        <span className="w-6 text-center text-[11px] text-slate-400">{toPersianDigits(i + 1)}</span>
        <input
          type="checkbox"
          className="size-4 accent-indigo-600"
          checked={f.visible}
          onChange={(e) => {
            const next = [...currentFields];
            next[i] = { ...f, visible: e.target.checked };
            setFormFields({ ...formFields, [tab]: next });
          }}
        />
        <Input
          value={f.label}
          onChange={(e) => {
            const next = [...currentFields];
            next[i] = { ...f, label: e.target.value };
            setFormFields({ ...formFields, [tab]: next });
          }}
          className="max-w-[220px] px-2 py-1 text-xs"
        />
        <span className="rounded bg-white px-2 py-0.5 text-[10px] text-slate-400">{f.key}</span>
        <div className="mr-auto flex gap-1">
          <button
            onClick={() => setFormFields({ ...formFields, [tab]: move(currentFields, i, -1) })}
            className="rounded-lg bg-white px-2 py-1 text-xs ring-1 ring-slate-200"
          >
            ↑
          </button>
          <button
            onClick={() => setFormFields({ ...formFields, [tab]: move(currentFields, i, 1) })}
            className="rounded-lg bg-white px-2 py-1 text-xs ring-1 ring-slate-200"
          >
            ↓
          </button>
          <button
            onClick={async () => {
              if (!(await confirm({
                title: " ",
                message: `«${f.label}»`,
                confirmText: "",
                danger: true,
              }))) return;
              const next = currentFields.filter((_, x) => x !== i);
              setFormFields({ ...formFields, [tab]: next });
              await persist('fields.${tab}', next, `«${f.label}»`);
            }}
            className="rounded-lg bg-rose-100 px-2 py-1 text-xs font-bold text-rose-700"
          >
            ✖
          </button>
        </div>
      </li>
    )
  </ul>
  <div className="mt-3 flex flex-wrap items-center gap-2 rounded-xl bg-indigo-50 p-2 ring-1 ring-indigo-200">
    <span className="text-[11px] font-bold text-indigo-800"> :</span>
    <select
      value={addField}
      onChange={(e) => setAddField(e.target.value)}
      className="rounded-xl border border-indigo-200 bg-white px-2 py-2 text-xs"
    >
      <option value=""> ...</option>
      {(AVAILABLE_FIELDS[tab] ?? []).
        .filter((a) => !currentFields.some((f) => f.key === a.key))
        .map((a) => (
          <option key={a.key} value={a.key}>{a.label}</option>
        ))}
    </select>
  </div>

```

```

</select>
<Button
  variant="soft"
  onClick={async () => {
    const found = (AVAILABLE_FORM_FIELDS[tab] ?? []).find((a) => a.key === addField);
    if (!found) {
      setMsg("      ");
      return;
    }
    const next = [...currentFields, { ...found, visible: true }];
    setFormFields({ ...formFields, [tab]: next });
    setAddField("");
    await persist(`fields.${tab}`, next, `«${found.label}» `);
  }}
>

</Button>
</div>
<div className="mt-3 flex flex-wrap gap-2">
  <Button onClick={() => save(`fields.${tab}`, currentFields)}> </Button>
  <Button
    variant="ghost"
    onClick={async () => {
      if (!(await confirm({ title: " ", message: "      ", confirmTe
xt: "" }))) return;
      const next = DEFAULT_FORM_FIELDS[tab];
      setFormFields({ ...formFields, [tab]: next });
      await persist(`fields.${tab}`, next, "      ");
    }}
  >

  </Button>
</div>
</Card>
<Card>
  <h3 className="mb-2 text-sm font-bold text-slate-700"> ( /)</h3>
  <ul className="space-y-1">
    {products.map((p, i) => (
      <li key={p.key} className="flex flex-wrap items-center gap-2 rounded-xl bg-slate-50 px-3 py-2">
        <span className="w-6 text-center text-[11px] text-slate-400">{toPersianDigits(i + 1)}</span>
        <input
          type="checkbox"
          className="size-4 accent-teal-600"
          checked={p.enabled}
          onChange={(e) => {
            const next = [...products];
            next[i] = { ...p, enabled: e.target.checked };
            setProducts(next);
          }}
        />
        <Input
          value={p.label}
          onChange={(e) => {
            const next = [...products];
            next[i] = { ...p, label: e.target.value };
            setProducts(next);
          }}
          className="max-w-[150px] px-2 py-1 text-xs"
        />
        <Input
          value={p.bonusLabel}
          onChange={(e) => {
            const next = [...products];
            next[i] = { ...p, bonusLabel: e.target.value };
            setProducts(next);
          }}
          className="max-w-[180px] px-2 py-1 text-xs"
        />
        <div className="mr-auto flex gap-1">
          <button onClick={() => moveP(i, -1)} className="rounded-lg bg-white px-2 py-1 text-xs ring-1 ring-slate-
200">
            ↑
          </button>
          <button onClick={() => moveP(i, 1)} className="rounded-lg bg-white px-2 py-1 text-xs ring-1 ring-slate-2
00">
            ↓
          </button>
          <button
            onClick={async () => {
              if (
                !(await confirm({
                  title: " ",
                  message: `«${p.label}»`,
                  confirmText: "",

```

```

        danger: true,
      )))
    )
    return;
    setProducts(products.filter((_, x) => x !== i));
  }}
  className="rounded-lg bg-rose-100 px-2 py-1 text-xs font-bold text-rose-700"
>

</button>
</div>
</li>
)}}
</ul>
<div className="mt-3 flex flex-wrap gap-2">
  <Input
    value={newProduct}
    onChange={(e) => setNewProduct(e.target.value)}
    placeholder="..."
    className="max-w-[220px]"
  />
  <Button
    variant="soft"
    onClick={() => {
      const label = newProduct.trim();
      if (!label) return;
      const key = `p_${Date.now()}`;
      setProducts([...products, { key, label, bonusLabel: ` ${label}`, enabled: true }]);
      setNewProduct("");
    }}
  >

</Button>
<Button onClick={() => save("products", products)}> </Button>
<Button variant="ghost" onClick={() => setProducts(DEFAULT_PRODUCTS)}>

</Button>
</div>
</Card>
</div>
);
}

```

```

@@ src/app/admin/homes/page.tsx typescript
import HomeScreen from "@components/screens/HomeScreen";
export default function Page() {
  return <HomeScreen isAdmin />;
}

```

```

@@ src/app/admin/leaves/page.tsx typescript
import LeaveScreen from "@components/screens/LeaveScreen";
export default function Page() {
  return <LeaveScreen />;
}

```

```

@@ src/app/admin/live/page.tsx typescript
"use client";
import { useCallback, useMemo, useState } from "react";
import dynamic from "next/dynamic";
import { Alert, Badge, Button, Card, SectionTitle } from "@components/ui";
import { useLive } from "@lib/useLive";
import { formatDistance } from "@lib/tracker";
import { tehranDateTime, tehranTime, toPersianDigits } from "@lib/jalali";
import NavButton from "@components/NavButton";
const MapBox = dynamic(() => import("@components/MapBox"), { ssr: false });
type Live = {
  userId: number;
  repName: string;
  lat: number;
  lng: number;
  accuracy: number | null;

  speed: number | null;
  battery: number | null;
  gpsOn: boolean;
  tripId: number | null;
  recordedAt: string;
  updatedAt: string;
  ageMs: number;
  live: boolean;
  stale: boolean;
  activeTrip: { id: number; distanceM: number } | null;
};
const ageLabel = (ms: number) => {
  const m = Math.floor(ms / 60000);

```

```

    if (m < 1) return " ";
    if (m < 60) return `${toPersianDigits(m)} `;
    const h = Math.floor(m / 60);
    if (h < 24) return `${toPersianDigits(h)} `;
    return `${toPersianDigits(Math.floor(h / 24))} `;
  };
export default function LivePage() {
  const [rows, setRows] = useState<Live[]>([]);
  const [missing, setMissing] = useState<{ userId: number; repName: string }[]>([]);
  const [selected, setSelected] = useState<number | null>(null);
  const [autoRefresh, setAutoRefresh] = useState(true);
  const load = useCallback(async () => {
    const res = await fetch("/api/live", { cache: "no-store" });
    if (!res.ok) return;
    const d = await res.json();
    setRows(d.rows ?? []);
    setMissing(d.missing ?? []);
  }, []);
  const { refresh } = useLive(load, 12000, autoRefresh);
  const focused = useMemo(() => rows.find((r) => r.userId === selected) ?? null, [rows, selected]);
  const online = rows.filter((r) => r.live).length;
  const onTrip = rows.filter((r) => r.activeTrip).length;
  const mapPoints = (focused ? [focused] : rows).map((r) => ({
    lat: r.lat,
    lng: r.lng,
    label: `${r.repName} — ${r.live ? "": " "} (${ageLabel(r.ageMs)})`,
    color: r.live ? (r.activeTrip ? "#16a34a" : "#0ea5e9") : r.stale ? "#f59e0b" : "#94a3b8",
  }));
  return (
    <div className="space-y-4">
      <div className="flex flex-wrap items-center justify-between gap-2">
        <SectionTitle icon=""> </SectionTitle>
        <div className="flex flex-wrap gap-2">
          <Badge tone="green"> {toPersianDigits(online)}</Badge>
          <Badge tone="amber"> {toPersianDigits(onTrip)}</Badge>
          <Button variant={autoRefresh ? "success" : "ghost"} onClick={() => setAutoRefresh((v) => !v)}>
            {autoRefresh ? " ( )" : " "}
          </Button>
          <Button variant="ghost" onClick={refresh}>
            </Button>
        </div>
      </div>
      <Alert kind="info">
        GPS
      </Alert>
      <div className="grid grid-cols-1 gap-3 lg:grid-cols-3">
        <Card className="lg:col-span-1">
          <div className="mb-2 flex items-center justify-between">
            <h3 className="text-sm font-bold text-slate-700"></h3>
            {focused ? (
              <button onClick={() => setSelected(null)} className="text-[11px] font-bold text-teal-700">
                </button>
            ) : null}
          </div>
          <div className="max-h-[460px] space-y-2 overflow-y-auto">
            {rows.map((r) => (
              <button
                key={r.userId}
                onClick={() => setSelected(selected === r.userId ? null : r.userId)}
                className={`w-full rounded-xl px-3 py-2 text-right ring-1 transition ${
                  selected === r.userId ? "bg-teal-50 ring-teal-300" : "bg-slate-50 ring-slate-200 hover:bg-slate-100"
                }`}
              >
                <div className="flex flex-wrap items-center gap-1.5">
                  <span className="text-sm font-black text-slate-800">{r.repName}</span>
                  <Badge tone={r.live ? "green" : r.stale ? "amber" : "slate"}>
                    {r.live ? "": r.stale ? "": " "}
                  </Badge>
                  {r.activeTrip ? <Badge tone="green"> </Badge> : null}
                  {r.gpsOn ? <Badge tone="amber">GPS </Badge> : null}
                </div>
                <div className="mt-1 flex flex-wrap gap-2 text-[10px] text-slate-500">
                  <span>{ageLabel(r.ageMs)}</span>
                  {r.accuracy ? <span> {toPersianDigits(Math.round(r.accuracy))}</span> : null}
                  {r.speed ? <span> {toPersianDigits(Math.round(r.speed * 3.6))} km/h</span> : null}
                  {r.battery !== null ? <span> {toPersianDigits(r.battery)}</span> : null}
                  {r.activeTrip ? <span> {formatDistance(r.activeTrip.distanceM)}</span> : null}
                </div>
                <div className="mt-1 text-[10px] text-slate-400">{tehranDateTime(r.updatedAt)}</div>
              </button>
            ))}
          </div>
        </Card>
      </div>
    </div>
  );
}

```

```

    {rows.length === 0 ? (
      <p className="py-6 text-center text-xs text-slate-400">      </p>
    ) : null}
    {missing.length ? (
      <div className="rounded-xl bg-slate-100 p-2 text-[11px] text-slate-500">
        : {missing.map((m) => m.repName).join(" ") }
      </div>
    ) : null}
  </div>
</Card>
<Card className="lg:col-span-2">
  <div className="mb-2 flex flex-wrap items-center justify-between gap-2">
    <h3 className="text-sm font-bold text-slate-700">
      {focused ? ` ${focused.repName}` : " "}
    </h3>
    {focused ? (
      <div className="flex items-center gap-2">
        <NavButton lat={focused.lat} lng={focused.lng} label={focused.repName} />
        <span className="text-[11px] text-slate-500" dir="ltr">
          {focused.lat.toFixed(5)}, {focused.lng.toFixed(5)}
        </span>
      </div>
    ) : null}
  </div>
  <MapBox height={430} points={mapPoints} follow={!!focused} />
  {focused ? (
    <div className="mt-3 grid grid-cols-2 gap-2 text-center text-[11px] sm:grid-cols-4">
      [
        [" ", tehranTime(focused.updatedAt)],
        [" GPS", focused.gpsOn ? "" : ""],
        [" ", focused.accuracy ? `${toPersianDigits(Math.round(focused.accuracy))}` : "—"],
        [" ", focused.activeTrip ? formatDistance(focused.activeTrip.distanceM) : "—"],
        ].map(([l, v]) => (
          <div key={l} className="rounded-xl bg-slate-50 p-2 ring-1 ring-slate-200">
            <div className="font-black text-slate-800">{v}</div>
            <div className="text-[10px] text-slate-500">{l}</div>
          </div>
        )))
    </div>
  ) : null}
</Card>
</div>
</div>
);
}

```

```

@@ src/app/admin/map/page.tsx typescript
import MapExplorer from "@components/screens/MapExplorer";
export default function Page() {
  return <MapExplorer isAdmin />;
}

```

```

@@ src/app/admin/messengers/page.tsx typescript
"use client";
import { useCallback, useEffect, useState } from "react";
import { Alert, Badge, Button, Card, Field, Input, SectionTitle } from "@components/ui";

```

```

import { PLATFORMS } from "@lib/constants";
import { tehranDateTime, toPersianDigits } from "@lib/jalali";
import { useConfirm } from "@components/Confirm";
import { useLive } from "@lib/useLive";
type M = {
  id: number;
  platform: string;
  label: string;
  targetType: string;
  target: string;
  token: string;
  provider: string;
  apiUrl: string;
  enabled: boolean;
  lastStatus: string;
  lastOkAt: string | null;
  lastErrorAt: string | null;
};
type Log = { id: number; platform: string; target: string; ok: boolean; detail: string; createdAt: string };
type Proxy = { url: string; enabled: boolean; secret?: string };
type Chat = { id: string; title: string; type: string };
type Sms = { provider: string; apiKey: string; sender: string; pattern: string; enabled: boolean };
const WA_PROVIDERS = [
  { key: "whatsiplus", label: " (whatsiplus.ir)", hint: " API | :..." },
  { key: "ultramsg", label: "UltraMsg", hint: ": instanceId:token | : 989121111111" },
  { key: "cloudapi", label: "WhatsApp Cloud API ()", hint: ": phoneNumberId:accessToken" },
  { key: "custom", label: " ", hint: " {phone} {text} {token}" },
];

```

```

const emptyForm = (platform: string) => ({
  platform,
  label: "",
  targetType: platform === "whatsapp" ? "phone" : "group",
  target: "",
  token: "",
  provider: platform === "whatsapp" ? "whatsiplus" : "",
  apiUrl: "",
  enabled: true,
});
export default function MessengersPage() {
  const [rows, setRows] = useState<M[]>([]);
  const [logs, setLogs] = useState<Log[]>([]);
  const [proxy, setProxy] = useState<Proxy>({ url: "", enabled: true, secret: "" });
  const [sms, setSms] = useState<Sms>({ provider: "kavenegar", apiKey: "", sender: "", pattern: "", enabled: false });
  const [sysTokens, setSysTokens] = useState<Record<string, string>>({});
  const [tab, setTab] = useState("telegram");
  const [form, setForm] = useState(emptyForm("telegram"));
  const [chats, setChats] = useState<Chat[]>([]);
  const [chatBusy, setChatBusy] = useState(false);
  const [msg, setMsg] = useState("");
  const [showToken, setShowToken] = useState<Record<number, boolean>>({});
  const [testing, setTesting] = useState<number | null>(null);
  const [workerVersion, setWorkerVersion] = useState<"new" | "old" | "down" | "">("");
  const [autoSend, setAutoSend] = useState(true);
  const confirm = useConfirm();
  const meta = PLATFORMS.find((p) => p.key === tab)!.key;
  const load = useCallback(async () => {
    const res = await fetch("/api/messengers", { cache: "no-store" });
    if (!res.ok) return;
    const d = await res.json();
    setRows(d.rows ?? []);
    setLogs(d.logs ?? []);
    if (d.proxy) setProxy(d.proxy);
    if (d.sms) setSms(d.sms);
    if (typeof d.autoSend === "boolean") setAutoSend(d.autoSend);
  }, []);
  useLive(load, 30000);
  useEffect(() => {
    fetch("/api/messengers/updates", { cache: "no-store" })
      .then((r) => (r.ok ? r.json() : null))
      .then((d) => setSysTokens(d?.tokens ?? {}))
      .catch(() => undefined);
  }, []);
  useEffect(() => {
    setForm(emptyForm(tab));
    setChats([]);
  }, [tab]);

  /* ----- */
  const add = async () => {
    if (!form.target.trim()) return setMsg(" ");
    if (meta.needsToken && !form.token.trim() && !sysTokens[tab]) {
      return setMsg(" ");
    }
    if (
      !(await confirm({
        title: `${meta.label}`,
        message: `«${form.label}|| form.target»`,
        confirmText: "",
      }))
    )
      return;
    const res = await fetch("/api/messengers", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(form),
    });
    const d = await res.json().catch(() => ({}));
    if (res.ok) {
      setForm(emptyForm(tab));
      setMsg(" ");
      load();
    } else setMsg(`${d.error ?? " "}`);
  };
  const patch = async (body: Record<string, unknown>) => {
    await fetch("/api/messengers", {
      method: "PATCH",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(body),
    });
    load();
  };
  const remove = async (m: M) => {
    if (

```



```

        !(await confirm({
            title: " ",
            message: `«${m.label || m.target}»`,
            confirmText: "",
            danger: true,
        })))
    )
    return;
    await fetch(`/api/messengers?id=${m.id}`, { method: "DELETE" });
    load();
};
const test = async (id: number) => {
    setTesting(id);
    setMsg("...");
    const res = await fetch("/api/messengers/test", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ id }),
    });
    const d = await res.json().catch(() => ({}));
    setTesting(null);
    setMsg(d.ok ? ` ${d.via === "proxy" ? " " : ""}` : ` ${d.detail ?? ""}`);
    load();
};
const testAll = async () => {
    setMsg("...");
    const res = await fetch("/api/messengers/test", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ all: true }),
    });
    const d = await res.json().catch(() => ({}));
    setMsg(
        d.summary
        ? `${d.sent}/${d.total} — ${d.summary}.slice(0, 400)`
        : ` ${d.error ?? " "}`,
    );
    load();
};
const discover = async () => {
    setChatBusy(true);
    setChats([]);

    setMsg("...");
    const res = await fetch("/api/messengers/updates", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ platform: tab, token: form.token }),
    });
    const d = await res.json().catch(() => ({}));
    setChatBusy(false);
    setChats(d.chats ?? []);
    setMsg(d.ok ? `${(d.chats ?? []).length} — ` : ` ${d.detail ?? ""}`);
};
const saveProxy = async (next: Proxy) => {
    setProxy(next);
    const res = await fetch("/api/settings", {
        method: "PUT",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ key: "messagingProxy", value: next }),
    });
    setMsg(res.ok ? " " : " ");
};
const testProxy = async () => {
    setMsg("...");
    const res = await fetch("/api/messengers/test", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ pingProxy: true }),
    });
    const d = await res.json().catch(() => ({}));
    setWorkerVersion(d.version ?? "");
    setMsg(d.detail ?? (d.ok ? " " : " "));
};
const copyWorker = async () => {
    const code = await fetch("/cloudflare-worker.js").then((r) => r.text()).catch(() => "");
    if (!code) return setMsg("");
    await navigator.clipboard.writeText(code).catch(() => window.open("/cloudflare-worker.js", "_blank"));
    setMsg(" — Cloudflare Deploy");
};
const tabRows = rows.filter((r) => r.platform === tab);
const activeCount = rows.filter((r) => r.enabled).length;
return (
    <div className="space-y-4">
        <div className="flex flex-wrap items-center justify-between gap-2">

```

```

<SectionTitle icon=""> </SectionTitle>
<div className="flex gap-2">
  <Badge tone={activeCount ? "green" : "amber"}>{toPersianDigits(activeCount)} </Badge>
  <Button variant="soft" onClick={testAll}>

    </Button>
  </div>
</div>
{msg ? <Alert kind={msg.startsWith("") ? "error" : "success"}>{msg}</Alert> : null}
}/* ----- */
<Card>
  <div className="flex flex-wrap items-center gap-3">
    <span className="text-sm font-black text-slate-800"> </span>
    <button
      onClick={async () => {
        const next = !autoSend;
        if (
          !(await confirm({
            title: next ? " " : " ",
            message: next
              ? " "
              : " ".
            confirmText: next ? " " : " ",
          }))
        )
          return;
        setAutoSend(next);
        const res = await fetch("/api/settings", {
          method: "PUT",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({ key: "autoSendOrders", value: { enabled: next } }),
        });
        setMsg(res.ok ? (next ? " " : " ") : " ");
      }}
      className={`rounded-xl px-4 py-2 text-xs font-bold ${
        autoSend ? "bg-emerald-600 text-white" : "bg-slate-200 text-slate-700"
      }`}
    >
    {autoSend ? " — " : " — "}
    </button>
    <span className="text-[11px] text-slate-500">
      .
    </span>
  </div>
</Card>
}/* ----- */
<div className="flex flex-wrap gap-1 rounded-2xl bg-slate-200 p-1">
  {PLATFORMS.map((p) => {
    const count = rows.filter((r) => r.platform === p.key).length;
    const on = rows.some((r) => r.platform === p.key && r.enabled);
    return (
      <button
        key={p.key}
        onClick={() => setTab(p.key)}
        className={`flex-1 rounded-xl px-3 py-2.5 text-xs font-bold transition ${
          tab === p.key ? "bg-white text-teal-700 shadow" : "text-slate-600 hover:bg-white/50"
        }`}
      >
        {p.icon} {p.label}
        {count ? (
          <span className={`mr-1 rounded-md px-1.5 text-[10px] ${on ? "bg-emerald-100 text-emerald-700" : "bg-slate-200 text-slate-500"}`}>
            {toPersianDigits(count)}
          </span>
        ) : null}
      </button>
    );
  })}
</div>
}/* ----- */
<Card>
  <h3 className="mb-2 text-sm font-black text-slate-800">
    {meta.icon} {meta.label}
  </h3>
  <ol className="list-inside list-decimal space-y-1 text-[11px] leading-6 text-slate-600">
    {meta.guide.map((g, i) => (
      <li key={i}>{g}</li>
    ))}
  </ol>
  {sysTokens[tab] ? (
    <p className="mt-2 rounded-lg bg-emerald-50 px-3 py-1.5 text-[11px] font-bold text-emerald-800">
      {sysTokens[tab]} — .
    </p>
  ) : null}

```

```

</Card>
{ /* ----- */
<Card>
  <h3 className="mb-2 text-sm font-bold text-slate-700"> {meta.label}</h3>
  <div className="grid grid-cols-1 gap-3 sm:grid-cols-2 lg:grid-cols-3">
    <Field label=" ( )">
      <Input
        value={form.label}
        onChange={(e) => setForm({ ...form, label: e.target.value })}
        placeholder=" "
      />
    </Field>
    {tab === "whatsapp" ? (
      <Field label=" " hint={WA_PROVIDERS.find((p) => p.key === form.provider)?.hint}>
        <select
          value={form.provider}
          onChange={(e) => setForm({ ...form, provider: e.target.value })}
          className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
        >
          {WA_PROVIDERS.map((p) => (
            <option key={p.key} value={p.key}>
              {p.label}
            </option>
          ))}
        </select>
      </Field>
    ) : (
      <Field label=" ">
        <select
          value={form.targetType}
          onChange={(e) => setForm({ ...form, targetType: e.target.value })}
          className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
        >
          <option value="group"></option>

          <option value="channel"></option>
          <option value="phone"></option>
        </select>
      </Field>
    )}
    <Field label=" " required hint={meta.targetHint}>
      <Input value={form.target} onChange={(e) => setForm({ ...form, target: e.target.value })} className="text-le
ft" />
    </Field>
    <Field label=" / API" hint={meta.tokenHint}>
      <Input value={form.token} onChange={(e) => setForm({ ...form, token: e.target.value })} className="text-left
" />
    </Field>
    {tab === "whatsapp" && form.provider === "custom" ? (
      <div className="sm:col-span-2">
        <Field label=" " hint=": https://api.example.com/send?to={phone}&msg={text}&key={toke
n}">
          <Input value={form.apiUrl} onChange={(e) => setForm({ ...form, apiUrl: e.target.value })} className="tex
t-left" />
        </Field>
      </div>
    ) : null}
  </div>
  <div className="mt-3 flex flex-wrap items-center gap-2">
    {tab === "telegram" || tab === "bale" ? (
      <Button variant="soft" onClick={discover} disabled={chatBusy}>
        {chatBusy ? " ..." : " chat_id"}
      </Button>
    ) : null}
    <div className="flex-1" />
    <Button onClick={add}> </Button>
  </div>
  {chats.length ? (
    <div className="mt-3 rounded-xl bg-slate-50 p-2 ring-1 ring-slate-200">
      <div className="mb-1 text-[11px] font-bold text-slate-600"> :</div>
      <div className="flex flex-wrap gap-1">
        {chats.map((c) => (
          <button
            key={c.id}
            onClick={() =>
              setForm({ ...form, target: c.id, label: c.title, targetType: c.type === "" ? "phone" : "group" }
            )
          >
            {c.type === "" ? "" : ""} {c.title}
          <span className="mr-1 text-[10px] text-slate-400" dir="ltr">
            {c.id}
          </span>
        )
      >
    </div>
  ) : (
    <div className="rounded-lg bg-white px-2 py-1 text-[11px] font-bold text-teal-700 ring-1 ring-teal-200 hove
r:bg-teal-50">

```

```

        </button>
      )}}
    </div>
  </div>
) : null}
</Card>
{ /* ----- */
<Card>
  <h3 className="mb-2 text-sm font-bold text-slate-700">
    {meta.label} ({toPersianDigits(tabRows.length)})
  </h3>
  {tabRows.length === 0 ? (
    <p className="py-6 text-center text-xs text-slate-400"> {meta.label} </p>
  ) : (
    <div className="space-y-2">
      {tabRows.map((m) => (
        <div key={m.id} className="rounded-xl bg-slate-50 p-3 ring-1 ring-slate-200">
          <div className="flex flex-wrap items-center gap-2">
            <span className="text-sm font-black text-slate-800">{m.label} || " "</span>
            <Badge tone={m.enabled ? "green" : "amber"}>{m.enabled ? "" : ""}</Badge>
            <span className="rounded bg-white px-2 py-0.5 text-[10px] text-slate-500" dir="ltr">
              {m.target}
            </span>
            {m.provider ? (
              <span className="rounded bg-white px-2 py-0.5 text-[10px] text-slate-500">
                {WA_PROVIDERS.find((p) => p.key === m.provider)?.label ?? m.provider}
              </span>
            ) : null}
          <div className="mr-auto flex flex-wrap gap-1">

            <button
              onClick={() => test(m.id)}
              disabled={testing === m.id}
              className="rounded-lg bg-sky-100 px-2 py-1 text-[11px] font-bold text-sky-700 disabled:opacity-50"
            >
              {testing === m.id ? "... " : " "}
            </button>
            <button
              onClick={() => patch({ id: m.id, enabled: !m.enabled })}
              className="rounded-lg bg-slate-200 px-2 py-1 text-[11px] font-bold text-slate-700"
            >
              {m.enabled ? " " : " "}
            </button>
            <button
              onClick={() => setShowToken((s) => ({ ...s, [m.id]: !s[m.id] }) )}
              className="rounded-lg bg-slate-200 px-2 py-1 text-[11px] font-bold text-slate-700"
            >
              {showToken[m.id] ? " " : " "}
            </button>
            <button
              onClick={() => remove(m)}
              className="rounded-lg bg-rose-100 px-2 py-1 text-[11px] font-bold text-rose-700"
            >

          </button>
        </div>
      </div>
      {showToken[m.id] ? (
        <div className="mt-2 grid grid-cols-1 gap-2 sm:grid-cols-2">
          <label className="block">
            <span className="mb-1 block text-[10px] font-bold text-slate-500"></span>
            <input
              value={m.token}
              onChange={(e) => setRows((p) => p.map((x) => (x.id === m.id ? { ...x, token: e.target.value } :
x))))}
              onBlur={() => patch({ id: m.id, token: m.token })}
              className="text-left font-mono text-xs"
            />
          </label>
          <label className="block">
            <span className="mb-1 block text-[10px] font-bold text-slate-500"> </span>
            <input
              value={m.target}
              onChange={(e) => setRows((p) => p.map((x) => (x.id === m.id ? { ...x, target: e.target.value } :
x))))}
              onBlur={() => patch({ id: m.id, target: m.target })}
              className="text-left font-mono text-xs"
            />
          </label>
        </div>
      ) : null}
      {m.lastStatus ? (
        <div
          className={`mt-2 rounded-lg px-2 py-1 text-[10px] ${
            m.lastStatus.startsWith("") ? "bg-emerald-50 text-emerald-700" : "bg-rose-50 text-rose-700"
          }`}
        >

```

```

        }`}
      >
        {m.lastStatus}
        {m.lastOkAt ? <span className="mr-2 text-slate-400"> : {tehranDateTime(m.lastOkAt)}</span>
: null}
      </div>
    ) : null}
  </div>
  )))
</div>
)}
</Card>
{ /* ----- */
<Card>
  <div className="mb-2 flex flex-wrap items-center justify-between gap-2">
    <h3 className="text-sm font-bold text-slate-700"> ( )</h3>
    <Badge tone={proxy.enabled ? "green" : "amber"}>{proxy.enabled ? "" : ""}</Badge>
  </div>
  <Alert kind="info">
    Render // . Cloudflare Worker
  </Alert>
  <div className="mt-3 grid grid-cols-1 gap-3 sm:grid-cols-3">
    <div className="sm:col-span-2">
      <Field label=" ">
        <Input value={proxy.url} onChange={(e) => setProxy({ ...proxy, url: e.target.value })} className="text-lef
t" />
      </Field>
    </div>
    <Field label=" ">
      <Input value={proxy.secret ?? ""} onChange={(e) => setProxy({ ...proxy, secret: e.target.value })} />
    </Field>
  </div>
  {workerVersion === "old" ? (
    <Alert kind="error">
      . « » .
    </Alert>
  ) : null}
  <div className="mt-3 flex flex-wrap gap-2">
    <Button onClick={() => saveProxy(proxy)}> </Button>
    <Button variant={proxy.enabled ? "danger" : "success"} onClick={() => saveProxy({ ...proxy, enabled: !proxy.en
abled })}>
      {proxy.enabled ? " " : " "}
    </Button>
    <Button variant="ghost" onClick={testProxy}>
  </Button>
  <Button variant="soft" onClick={copyWorker}>
  </Button>
</div>
</Card>
{ /* ----- */
<Card>
  <h3 className="mb-2 text-sm font-bold text-slate-700"> ( )</h3>
  <div className="grid grid-cols-1 gap-3 sm:grid-cols-4">
    <Field label="">
      <select
        value={sms.provider}
        onChange={(e) => setSms({ ...sms, provider: e.target.value })}
        className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
      >
        <option value="kavenegar"></option>
        <option value="smsir">SMS.ir</option>
        <option value="melipayamak"></option>
        <option value="custom"> </option>
      </select>
    </Field>
    <Field label=" API">
      <Input value={sms.apiKey} onChange={(e) => setSms({ ...sms, apiKey: e.target.value })} className="text-left"
/>
    </Field>
    <Field label=" ">
      <Input value={sms.sender} onChange={(e) => setSms({ ...sms, sender: e.target.value })} className="text-left"
/>
    </Field>
    <Field label=" ">
      <Input value={sms.pattern} onChange={(e) => setSms({ ...sms, pattern: e.target.value })} className="text-lef
t" />
    </Field>
  </div>
  <div className="mt-3 flex flex-wrap items-center gap-2">
    <label className="flex items-center gap-2 text-xs font-bold text-slate-600">
      <input

```

```

        type="checkbox"
        className="size-4 accent-teal-600"
        checked={sms.enabled}
        onChange={(e) => setSms({ ...sms, enabled: e.target.checked })}
    />

</label>
<Button
    onClick={async () => {
        const res = await fetch("/api/settings", {
            method: "PUT",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ key: "sms", value: sms }),
        });
        setMsg(res.ok ? "    " : " ");
    }}
>

</Button>
</div>
</Card>
{ /* ----- */ }
<Card>
    <SectionTitle icon=""> </SectionTitle>
    <div className="max-h-72 space-y-1 overflow-y-auto text-xs">
        {logs.map((l) => (
            <div key={l.id} className={`rounded-lg px-3 py-1.5 ${l.ok ? "bg-emerald-50" : "bg-rose-50"}`}>
                <span className={l.ok ? "text-emerald-700" : "text-rose-700"}>{l.ok ? "" : ""}</span>{" "}
            <b>{PLATFORMS.find((p) => p.key === l.platform)?.label ?? l.platform}</b> → <span dir="ltr">{l.target}</sp
an>
                <div className="text-[10px] text-slate-600">{l.detail}</div>
                <span className="text-[10px] text-slate-400">{tehranDateTime(l.createdAt)}</span>
            </div>
        ))}
        {logs.length === 0 ? <p className="text-slate-400">    </p> : null}
    </div>
</Card>
</div>
);
}

```

```

@@ src/app/admin/notifications/page.tsx typescript
import NotificationScreen from "@components/screens/NotificationScreen";
export default function Page() {
    return <NotificationScreen />;
}

```

```

@@ src/app/admin/options/page.tsx typescript
import OptionsScreen from "@components/screens/OptionsScreen";
export default function Page() {
    return <OptionsScreen canDelete />;
}

```

```

@@ src/app/admin/reports/page.tsx typescript
import ReportScreen from "@components/screens/ReportScreen";
export default function Page() {
    return <ReportScreen />;
}

```

```

@@ src/app/admin/sync/page.tsx typescript
"use client";
import { useCallback, useEffect, useState } from "react";
import { Alert, Badge, Button, Card, Field, Input, SectionTitle } from "@components/ui";
import { useConfirm } from "@components/Confirm";
import { useLive } from "@lib/useLive";
import { orderedEndpoints, pickFastestEndpoint } from "@lib/endpoints";
import { downloadFile } from "@lib/download";
import { tehranDateTime, toPersianDigits } from "@lib/jalali";
type Peer = {
    id: number;
    peer: string;
    peerUrl: string;
    enabled: boolean;
    pullCursor: string | null;
    pushCursor: string | null;
    lastSyncAt: string | null;
    lastStatus: string;
    lastError: string;
    pulled: number;
    pushed: number;
    conflicts: number;
};
type Log = {
    id: number;

```

```

peer: string;
direction: string;
tableName: string;
applied: number;
skipped: number;
ok: boolean;
detail: string;
createdAt: string;
};
type Info = { node: string; url: string; peers: { peer: string; url: string }[]; hasSecret: boolean; tables: string[] };
export default function SyncPage() {
  const [info, setInfo] = useState<Info | null>(null);
  const [peers, setPeers] = useState<Peer[]>([]);
  const [logs, setLogs] = useState<Log[]>([]);
  const [msg, setMsg] = useState("");
  const [busy, setBusy] = useState(false);
  const [form, setForm] = useState({ peer: "", peerUrl: "" });
  const [speeds, setSpeeds] = useState<{ url: string; ms: number | null }[]>([]);
  const [srcInfo, setSrcInfo] = useState<{ count: number } | null>(null);
  const [dlBusy, setDlBusy] = useState(false);
  const confirm = useConfirm();
  const load = useCallback(async () => {

    const res = await fetch("/api/sync/status", { cache: "no-store" });
    if (!res.ok) return;
    const d = await res.json();
    setInfo(d.info);
    setPeers(d.peers ?? []);
    setLogs(d.logs ?? []);
  }, []);
  useLive(load, 20000);
  const runSync = async () => {
    if (!(await confirm({ title: "", message: "      ", confirmText: "" })))
      return;
    setBusy(true);
    setMsg(" ...");
    const res = await fetch("/api/sync/run", { method: "POST" });
    const d = await res.json().catch(() => ({}));
    setBusy(false);
    setMsg(d.summary ?? (res.ok ? " " : " "));
    load();
  };
  const patch = async (body: Record<string, unknown>) => {
    await fetch("/api/sync/status", {
      method: "PATCH",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(body),
    });
    load();
  };
  const addPeer = async () => {
    if (!form.peer || !form.peerUrl.startsWith("http")) return setMsg(" ");
    const res = await fetch("/api/sync/status", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(form),
    });
    setMsg(res.ok ? " " : " ");
    setForm({ peer: "", peerUrl: "" });
    load();
  };
  const testSpeed = async () => {
    setMsg(" ...");
    const r = await pickFastestEndpoint(true);
    setSpeeds(r);
    const best = r.filter((x) => x.ms !== null).sort((a, b) => (a.ms ?? 9e9) - (b.ms ?? 9e9))[0];
    setMsg(best ? ` : ${best.url} (${best.ms}) ` : " ");
  };
  return (
    <div className="space-y-4">
      <div className="flex flex-wrap items-center justify-between gap-2">
        <SectionTitle icon=""> </SectionTitle>
        <div className="flex gap-2">
          <Button variant="ghost" onClick={testSpeed}>

          </Button>
          <Button onClick={runSync} disabled={busy}>
            {busy ? " ..." : " "}
          </Button>
        </div>
      </div>
      {msg ? <Alert kind={msg.startsWith("") ? "error" : "success"}>{msg}</Alert> : null}
      { /* ----- GitHub / GitLab ----- */ }
      <Card>
        <h3 className="mb-2 text-sm font-black text-slate-800"> </h3>

```

```

<Alert kind="info">
  . <b>GitHub</b> <b>GitLab</b> ZIP

  <span dir="ltr"> git add -A && git commit && git push </span> .
</Alert>
<div className="mt-3 flex flex-wrap items-center gap-2">
  <Button
    onClick={async () => {
      setDlBusy(true);
      setMsg("...");
      const r = await downloadFile("/api/source", "namayandeeelmi-source.zip");
      setDlBusy(false);
      setMsg(r);
    }}
    disabled={dlBusy}
  >

    {dlBusy ? "..." : "(ZIP)"}
  </Button>
  <Button
    variant="ghost"
    onClick={async () => {
      const r = await fetch("/api/source?list=1", { cache: "no-store" });
      if (!r.ok) return setMsg("");
      const d = await r.json();
      setSrcInfo({ count: d.count });
      setMsg(` ${d.count} `);
    }}
  >

</Button>
{srcInfo ? <Badge tone="green">{toPersianDigits(srcInfo.count)} </Badge> : null}
</div>
<p className="mt-2 text-[11px] text-slate-500">
  : <span dir="ltr">node_modules</span> <span dir="ltr">.next</span>
  <span dir="ltr">.env </span> ( ).
</p>
</Card>
<Card>
  <h3 className="mb-2 text-sm font-bold text-slate-700"> </h3>
  <div className="grid grid-cols-1 gap-2 text-xs sm:grid-cols-3">
    <div className="rounded-xl bg-slate-50 p-3 ring-1 ring-slate-200">
      : <b className="text-teal-700">{info?.node ?? "—"}</b>
    </div>
    <div className="rounded-xl bg-slate-50 p-3 ring-1 ring-slate-200">
      : <b dir="ltr">{info?.url || window.location.origin}</b>
    </div>
    <div className={`rounded-xl p-3 ring-1 ${info?.hasSecret ? "bg-emerald-50 ring-emerald-200" : "bg-rose-50 ring-rose-200"}`>
      : <b>{info?.hasSecret ? " " : " (SYNC_SECRET)"}</b>
    </div>
  </div>
  {info?.tables?.length ? (
    <p className="mt-2 text-[11px] text-slate-500">
      : {info.tables.join(" ")}
    </p>
  ) : null}
  {speeds.length ? (
    <div className="mt-2 flex flex-wrap gap-2">
      {speeds.map((s) => (
        <span
          key={s.url}
          className={`rounded-lg px-2 py-1 text-[11px] font-bold ring-1 ${
            s.ms === null ? "bg-rose-50 text-rose-700 ring-rose-200" : "bg-emerald-50 text-emerald-700 ring-emerald-200"
          }`}
          dir="ltr"
        >
          {s.url} — {s.ms === null ? " " : `${toPersianDigits(s.ms)}ms`}
        </span>
      ))}
    </div>
  ) : null}
  <p className="mt-2 text-[10px] text-slate-400" dir="ltr">
    endpoints: {orderedEndpoints().join(" , ")}
  </p>
</Card>
<Card>
  <h3 className="mb-2 text-sm font-bold text-slate-700"> </h3>
  {peers.length === 0 ? (
    <Alert kind="info">
      . <b dir="ltr">SYNC_PEERS</b> .
    </Alert>
  ) : null}
  <div className="space-y-2">

```



```

{peers.map((p) => (
  <div key={p.id} className="rounded-xl bg-slate-50 p-3 ring-1 ring-slate-200">
    <div className="flex flex-wrap items-center gap-2">
      <span className="text-sm font-black text-slate-800">{p.peer}</span>
      <Badge tone={p.enabled ? "green" : "amber"}>{p.enabled ? "" : ""}</Badge>
      <span className="text-[11px] text-slate-500" dir="ltr">
        {p.peerUrl}
      </span>
    </div>
    <div className="mr-auto flex flex-wrap gap-1">
      <button
        onClick={() => patch({ id: p.id, enabled: !p.enabled })}
        className="rounded-lg bg-slate-200 px-2 py-1 text-[11px] font-bold text-slate-700"
      >
        {p.enabled ? "" : ""}
      </button>
      <button
        onClick={async () => {
          if (
            !(await confirm({
              title: " ",
              message: " ",
              confirmText: "",
              danger: true,
            }))
          )
            return;
          patch({ id: p.id, resetCursor: true });
        }}
        className="rounded-lg bg-amber-100 px-2 py-1 text-[11px] font-bold text-amber-700"
      >
      </button>
    </div>
  </div>
</div>
<div className="mt-2 grid grid-cols-3 gap-2 text-center text-[11px]">
  <div className="rounded-lg bg-white py-1 ring-1 ring-slate-200">
    <b className="text-teal-700">{toPersianDigits(p.pulled)}</b>
  </div>
  <div className="rounded-lg bg-white py-1 ring-1 ring-slate-200">
    <b className="text-sky-700">{toPersianDigits(p.pushed)}</b>
  </div>
  <div className="rounded-lg bg-white py-1 ring-1 ring-slate-200">
    <b className="text-slate-500">{toPersianDigits(p.conflicts)}</b>
  </div>
</div>
<div className="mt-2 text-[11px]">
  <span className={p.lastStatus.startsWith("") ? "text-emerald-700" : "text-slate-600"}>
    {p.lastStatus || " "}
  </span>
  {p.lastSyncAt ? <span className="mr-2 text-slate-400">{tehranDateTime(p.lastSyncAt)}</span> : null}
  {p.lastError ? <div className="mt-1 rounded-lg bg-rose-50 px-2 py-1 text-[10px] text-rose-700">{p.lastError}</div> : null}
</div>
)}}
</div>
<div className="mt-3 grid grid-cols-1 gap-2 sm:grid-cols-3">
  <Field label=" ">
    <Input value={form.peer} onChange={(e) => setForm({ ...form, peer: e.target.value })} placeholder="ndcohub"
  </Field>
  <Field label=" ">
    <Input
      value={form.peerUrl}
      onChange={(e) => setForm({ ...form, peerUrl: e.target.value })}
      placeholder="https://ndcohub.ir"
      className="text-left"
    </Field>
    <div className="flex items-end">
      <Button onClick={addPeer} className="w-full">
      </Button>
    </div>
  </div>
</Card>
<Card>
  <h3 className="mb-2 text-sm font-bold text-slate-700"> </h3>
  <div className="max-h-72 space-y-1 overflow-y-auto text-[11px]">
    {logs.map((l) => (
      <div key={l.id} className={l.ok ? "rounded-lg px-2 py-1.5 ${l.ok ? "bg-slate-50" : "bg-rose-50"}`}>
        <span className={l.ok ? "text-emerald-700" : "text-rose-700"}>{l.ok ? "" : ""}</span>{" "}
        <b>{l.peer}</b> — {l.direction === "pull" ? "" : l.direction === "push" ? "" : ""}
        {l.tableName ? ` | ${l.tableName}` : ""}
      </div>
    ))}
  </div>
</Card>

```

```

        {l.applied || l.skipped ? ` | ${toPersianDigits(l.applied)} / ${toPersianDigits(l.skipped)}` : ""
    }
    {l.detail ? <div className="text-slate-500">{l.detail}</div> : null}
    <span className="text-[10px] text-slate-400">{tehranDateTime(l.createdAt)}</span>
  </div>
  )}}
  {logs.length === 0 ? <p className="text-slate-400">    </p> : null}
</div>
<Card>
  <h3 className="mb-2 text-sm font-bold text-slate-700">    </h3>
  <ol className="list-inside list-decimal space-y-1 text-[11px] leading-6 text-slate-600">
    <li>
      <b>    </b>    <code dir="ltr">SYNC_SECRET</code>      .
    </li>
    <li>
      Render: <code dir="ltr">NODE_NAME=render</code> {" "}
      <code dir="ltr">SYNC_PEERS=ndcohub|https://ndcohub.ir</code>
    </li>
    <li>
      NdcHub: <code dir="ltr">NODE_NAME=ndcohub</code> {" "}
      <code dir="ltr">SYNC_PEERS=render|https://namayandeelmi-javad.onrender.com</code>
    </li>
    <li>
      : <code dir="ltr">NEXT_PUBLIC_ENDPOINTS=https://ndcohub.ir,https://namayandeelmi-javad.onrender.com
    </code>
    </li>
    <li>
      : cron-job.org {" "}
      <code dir="ltr">/api/sync/run?key=SYNC_SECRET</code>      .
    </li>
  </ol>
</Card>
</div>
  );
}

```

```

@@ src/app/admin/targets/page.tsx typescript
"use client";
import { useCallback, useEffect, useMemo, useState } from "react";
import { Alert, Badge, Button, Card, Field, Input, SectionTitle } from "@components/ui";
import { useConfirm } from "@components/Confirm";
import { DEFAULT_PRODUCTS, type ProductConfig } from "@lib/defaults";
import { JALALI_MONTHS, toPersianDigits, todayJalali } from "@lib/jalali";
type User = { id: number; fullName: string; role: string; active: boolean };
type Progress = {
  productKey: string;
  productLabel: string;
  quantity: number;
  priceDistributor: number;
  pricePharmacy: number;
  sold: number;
  bonus: number;
  remaining: number;
  percent: number;
};
const money = (n: number) => toPersianDigits(Math.round(n).toLocaleString("en-US"));
const nowYear = () => Number(todayJalali().slice(0, 4));
const nowMonth = () => Number(todayJalali().slice(5, 7));
export default function TargetsPage() {
  const [users, setUsers] = useState<User[]>([]);
  const [products, setProducts] = useState<ProductConfig[]>(DEFAULT_PRODUCTS);
  const [userId, setUserId] = useState<number | null>(null);
  //
  const [year, setYear] = useState<number>(nowYear());
  const [month, setMonth] = useState<number>(nowMonth());
  const [years, setYears] = useState<number[]>([]);
  const [newYear, setNewYear] = useState("");
  const [rows, setRows] = useState<Progress[]>([]);
  const [msg, setMsg] = useState("");
  const [busy, setBusy] = useState(false);
  const confirm = useConfirm();
  const period = `${year}/${String(month).padStart(2, "0")}`;
  /* ----- */
  const loadBase = useCallback(async () => {
    const res = await fetch("/api/users", { cache: "no-store" });
    if (res.ok) {
      const list: User[] = (await res.json()).rows ?? [];
      setUsers(list);
      setUserId((cur) => cur ?? list.find((u) => u.role === "rep")?.id ?? list[0]?.id ?? null);
    }
  }, []);
  const pr = await fetch("/api/settings?key=products", { cache: "no-store" });
  if (pr.ok) {
    const v = (await pr.json()).value as ProductConfig[] | null;
  }
}

```

```

    if (Array.isArray(v) && v.length) setProducts(v.filter((p) => p.enabled !== false));
  }
  //

  const yr = await fetch("/api/options?category=year", { cache: "no-store" });
  const saved: number[] = yr.ok
    ? ((await yr.json()).rows ?? []).map((r: { value: string }) => Number(r.value)).filter(Boolean)
    : [];
  const base = [nowYear() - 1, nowYear(), nowYear() + 1];
  setYears([...new Set([...base, ...saved])].sort((a, b) => a - b));
}, []);
const loadTargets = useCallback(async () => {
  if (!userId) return;
  const res = await fetch(`/api/targets?userId=${userId}&period=${encodeURIComponent(period)}`, { cache: "no-store" });
;
  if (res.ok) setRows((await res.json()).progress ?? []);
}, [userId, period]);
useEffect(() => {
  loadBase();
}, [loadBase]);
useEffect(() => {
  loadTargets();
}, [loadTargets]);
/** — */
const setField = (key: string, field: "quantity" | "priceDistributor" | "pricePharmacy", value: number) =>
  setRows((p) => p.map((r) => (r.productKey === key ? { ...r, [field]: value } : r)));
const addYear = async () => {
  const y = Number(newYear());
  if (!y || y < 1300 || y > 1500) return setMsg(" ( )");
  if (!(await confirm({ title: " ", message: `${toPersianDigits(y)} ` , confirmText: " " })))
  return;
  const res = await fetch("/api/options", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ category: "year", value: String(y) }),
  });
  if (res.ok) {
    setYears((p) => [...new Set([...p, y])].sort((a, b) => a - b));
    setYear(y);
    setNewYear("");
    setMsg(`${toPersianDigits(y)} `);
  } else setMsg(" ");
};
const save = async () => {
  if (!userId) return;
  const rep = users.find((u) => u.id === userId);
  if (
    !(await confirm({
      title: " ",
      message: `«${rep?.fullName}» ${JALALI_MONTHS[month - 1]} ${toPersianDigits(year)} ` ,
      confirmText: " ",
    })))
  )
  return;
  setBusy(true);
  const res = await fetch("/api/targets", {
    method: "PUT",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      userId,
      period,
      items: rows.map((r) => ({
        productKey: r.productKey,
        quantity: r.quantity,
        priceDistributor: r.priceDistributor,
        pricePharmacy: r.pricePharmacy,
      })),
    })),
  });
  setBusy(false);
  setMsg(res.ok ? " " : " ");
  loadTargets();
};
/** — */
const copyPrev = async () => {
  if (!userId) return;
  let pm = month - 1;
  let py = year;
  if (pm === 0) {
    pm = 12;
    py -= 1;
  }

  const prev = `${py}/${String(pm).padStart(2, "0")}`;

```

```

const res = await fetch(`/api/targets?userId=${userId}&period=${encodeURIComponent(prev)}`, { cache: "no-store" });
if (!res.ok) return setMsg("");
const d = await res.json();
const prevRows: Progress[] = d.progress ?? [];
setRows((cur) => {
  cur.map((r) => {
    const p = prevRows.find((x) => x.productKey === r.productKey);
    return p ? { ...r, quantity: p.quantity, priceDistributor: p.priceDistributor, pricePharmacy: p.pricePharmacy }
    : r;
  }),
);
setMsg(` ${JALALI_MONTHS[pm - 1]} ${toPersianDigits(py)} — «» `);
};
/* ----- */
const totals = useMemo(
  () =>
    rows.reduce(
      (a, r) => ({
        quantity: a.quantity + r.quantity,
        sold: a.sold + r.sold,
        remaining: a.remaining + Math.max(0, r.quantity - r.sold),
        valueDistributor: a.valueDistributor + r.quantity * r.priceDistributor,
        valuePharmacy: a.valuePharmacy + r.quantity * r.pricePharmacy,
        soldValueDistributor: a.soldValueDistributor + r.sold * r.priceDistributor,
        soldValuePharmacy: a.soldValuePharmacy + r.sold * r.pricePharmacy,
      }),
      {
        quantity: 0,
        sold: 0,
        remaining: 0,
        valueDistributor: 0,
        valuePharmacy: 0,
        soldValueDistributor: 0,
        soldValuePharmacy: 0,
      },
    ),
  [rows],
);
const pct = totals.quantity > 0 ? Math.round((totals.sold / totals.quantity) * 100) : 0;
const rep = users.find((u) => u.id === userId);
return (
  <div className="space-y-4">
    <SectionTitle icon=""> — </SectionTitle>
    {msg ? <Alert kind={msg.startsWith("") ? "error" : "success"}>{msg}</Alert> : null}
    <Card>
      <div className="grid grid-cols-1 gap-3 sm:grid-cols-2 lg:grid-cols-4">
        <Field label="" required>
          <select
            value={userId ?? ""}
            onChange={(e) => setUserId(Number(e.target.value))}
            className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
          >
            {users.map((u) => (
              <option key={u.id} value={u.id}>
                {u.fullName} {u.active ? "" : "()"}
              </option>
            ))}
          </select>
        </Field>
        <Field label="">
          <select
            value={year}
            onChange={(e) => setYear(Number(e.target.value))}
            className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
          >
            {years.map((y) => (
              <option key={y} value={y}>
                {toPersianDigits(y)}
              </option>
            ))}
          </select>
        </Field>
        <Field label="">
          <select
            value={month}
            onChange={(e) => setMonth(Number(e.target.value))}
            className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
          >
            {JALALI_MONTHS.map((m, i) => (
              <option key={m} value={i + 1}>
                {m}
              </option>
            ))}
          </select>

```

```

</Field>
<Field label=" " hint=" ">
  <div className="flex gap-1">
    <Input
      inputMode="numeric"
      value={newYear}
      onChange={(e) => setNewYear(e.target.value.replace(/\D/g, "").slice(0, 4))}
      placeholder="1406"
      className="text-center"
    />
    <Button variant="soft" onClick={addYear}>

      </Button>
    </div>
  </Field>
</div>
<div className="mt-3 flex flex-wrap items-center gap-2">
  <Badge tone="slate">
{rep?.fullName ?? "—"} | {JALALI_MONTHS[month - 1]} {toPersianDigits(year)}
  </Badge>
  <Button onClick={save} disabled={busy}>

  </Button>
  <Button variant="ghost" onClick={copyPrev}>

  </Button>
</div>
<div className="mt-3 grid grid-cols-2 gap-2 sm:grid-cols-4">
  {[
    [" ", toPersianDigits(totals.quantity), "bg-slate-50"],
    [" ", toPersianDigits(totals.sold), "bg-teal-50"],
    ["", toPersianDigits(totals.remaining), "bg-amber-50"],
    [" ", `${toPersianDigits(pct)}%`, pct >= 100 ? "bg-emerald-50" : "bg-sky-50"],
  ]}.map(([l, v, c]) => (
    <div key={l} className={`rounded-xl p-3 text-center ring-1 ring-slate-200 ${c}`}>
      <div className="text-lg font-black text-teal-700">{v}</div>
      <div className="text-[10px] font-bold text-slate-500">{l}</div>
    </div>
  )))
</div>
</Card>
<Card>
  <h3 className="mb-2 text-sm font-bold text-slate-700">
—
  </h3>
  <div className="scroll-x">
    <table className="w-full min-w-[980px] text-right text-xs">
      <thead>
        <tr className="bg-slate-100 text-slate-600">
          <th className="px-2 py-2"></th>
          <th className="px-2 py-2"></th>
          <th className="px-2 py-2"> ()</th>
          <th className="px-2 py-2"> ()</th>
          <th className="px-2 py-2 text-center"></th>
          <th className="px-2 py-2 text-center"></th>
          <th className="px-2 py-2 text-center"></th>
          <th className="px-2 py-2 text-center"></th>
        </tr>
      </thead>
      <tbody>
        {rows.map((r) => {
          const valD = r.quantity * r.priceDistributor;
          const valP = r.quantity * r.pricePharmacy;
          const remaining = Math.max(0, r.quantity - r.sold);
          const percent = r.quantity > 0 ? Math.round((r.sold / r.quantity) * 100) : 0;
          return (
            <tr key={r.productKey} className="border-b border-slate-100">
              <td className="whitespace-nowrap px-2 py-2 font-bold text-slate-800">{r.productLabel}</td>
              <td className="px-2 py-2">
                <Input
                  inputMode="numeric"
                  value={r.quantity || ""}
                  onChange={(e) => setField(r.productKey, "quantity", Number(e.target.value.replace(/\D/g, "")) ||
0)}

                  className="w-24 px-2 py-1 text-center text-xs"

                  placeholder="0"
                />
              </td>
              <td className="px-2 py-2">
                <Input
                  inputMode="numeric"
                  value={r.priceDistributor || ""}
                  onChange={(e) =>

```

```

        setField(r.productKey, "priceDistributor", Number(e.target.value.replace(/\D/g, "")) || 0)
    }
    className="w-32 px-2 py-1 text-center text-xs"
    placeholder="0"
  />
</td>
<td className="px-2 py-2">
  <Input
    inputMode="numeric"
    value={r.pricePharmacy || ""}
    onChange={(e) =>
      setField(r.productKey, "pricePharmacy", Number(e.target.value.replace(/\D/g, "")) || 0)
    }
    className="w-32 px-2 py-1 text-center text-xs"
    placeholder="0"
  />
</td>
/* — */
<td className="px-2 py-2 text-center font-black text-slate-700">
  <span className={valD ? "text-sky-700" : "text-slate-300"}>{money(valD)}</span>
</td>
<td className="px-2 py-2 text-center font-black">
  <span className={valP ? "text-indigo-700" : "text-slate-300"}>{money(valP)}</span>
</td>
<td className="px-2 py-2 text-center font-bold text-teal-700">{toPersianDigits(r.sold)}</td>
<td className="px-2 py-2 text-center font-bold text-rose-700">{toPersianDigits(remaining)}</td>
<td className="px-2 py-2">
  <div className="flex items-center justify-center gap-1">
    <div className="h-1.5 w-14 overflow-hidden rounded bg-slate-200">
      <div
        className={`h-full ${percent >= 100 ? "bg-emerald-500" : "bg-teal-500`} ``
        style={{ width: `${Math.min(100, percent)}%` }}
      />
    </div>
    <span className="text-[10px] font-bold">{toPersianDigits(percent)}</span>
  </div>
</td>
</tr>
);
}}
<tr className="bg-teal-50 font-black text-teal-800">
  <td className="px-2 py-2"> </td>
  <td className="px-2 py-2 text-center">{toPersianDigits(totals.quantity)}</td>
  <td className="px-2 py-2" colSpan={2} />
  <td className="px-2 py-2 text-center">{money(totals.valueDistributor)}</td>
  <td className="px-2 py-2 text-center">{money(totals.valuePharmacy)}</td>
  <td className="px-2 py-2 text-center">{toPersianDigits(totals.sold)}</td>
  <td className="px-2 py-2 text-center">{toPersianDigits(totals.remaining)}</td>
  <td className="px-2 py-2 text-center">{toPersianDigits(pct)}</td>
</tr>
</tbody>
</table>
</div>
<div className="mt-3 grid grid-cols-1 gap-2 sm:grid-cols-2">
  <div className="rounded-xl bg-slate-50 p-3 text-xs ring-1 ring-slate-200">
    (}:{ " " }
    <span className="font-black text-teal-700">{money(totals.soldValueDistributor)} </span>
  </div>
  <div className="rounded-xl bg-slate-50 p-3 text-xs ring-1 ring-slate-200">
    (}:{ " " }
    <span className="font-black text-teal-700">{money(totals.soldValuePharmacy)} </span>
  </div>
</div>
</Card>
</div>
);
}

```

```

@@ src/app/admin/trips/page.tsx typescript

```

```

"use client";
import { useCallback, useEffect, useMemo, useState } from "react";
import dynamic from "next/dynamic";
import { Alert, Badge, Button, Card, Input, SectionTitle } from "@components/ui";
import { useLive } from "@lib/useLive";
import { downloadFile } from "@lib/download";

```

```

import { formatDistance, formatDuration } from "@lib/tracker";
import { tehranDateTime, tehranTime, toPersianDigits } from "@lib/jalali";
const MapBox = dynamic(() => import("@components/MapBox"), { ssr: false });
type Trip = {
  id: number;
  userId: number;
  repName: string;
  dateShamsi: string;
  status: string;

```

```

    startedAt: string;
    endedAt: string | null;
    points: number;
    distanceM?: number;
    stopSeconds?: number;
  };
  type Point = {
    id: number;
    lat: number;
    lng: number;
    kind: string;
    note: string;
    stopSeconds: number;
    gpsOn: boolean;
    speed: number | null;
    recordedAt: string;
  };
  type Place = { lat: number; lng: number; label: string; color: string };
  export default function AdminTrips() {
    const [trips, setTrips] = useState<Trip[]>([]);
    const [sel, setSel] = useState<number | null>(null);
    const [points, setPoints] = useState<Point[]>([]);
    const [onlyActive, setOnlyActive] = useState(false);
    const [rep, setRep] = useState("");
    const [q, setQ] = useState("");
    const [places, setPlaces] = useState<Place[]>([]);
    const [dl, setDl] = useState("");
    /* ----- */
    useEffect(() => {
      (async () => {
        const get = async (u: string) => {
          const r = await fetch(u, { cache: "no-store" }).catch(() => null);
          return r?.ok ? ((await r.json()).rows ?? []) : [];
        };
        const [ph, dr, hm] = await Promise.all([
          get("/api/records/pharmacies?limit=500"),
          get("/api/records/doctors?limit=500"),
          get("/api/homes"),
        ]);
        const out: Place[] = [];
        for (const p of ph) if (p.lat && p.lng) out.push({ lat: p.lat, lng: p.lng, label: `${p.name}`, color: "#0f766e" });
        for (const d of dr) if (d.lat && d.lng) out.push({ lat: d.lat, lng: d.lng, label: `${d.name}`, color: "#0369a1" });
        for (const h of hm) if (h.lat && h.lng) out.push({ lat: h.lat, lng: h.lng, label: `${h.title} (${h.repName})`, color: "#7c3aed" });
        setPlaces(out);
      })();
    }, []);
    const loadTrips = useCallback(async () => {
      const res = await fetch(`/api/trips${onlyActive ? "?active=1" : ""}`, { cache: "no-store" });
      if (res.ok) setTrips((await res.json()).rows ?? []);
    }, [onlyActive]);
    const loadPoints = useCallback(async (id: number) => {
      const res = await fetch(`/api/trips/points?tripId=${id}`, { cache: "no-store" });
      if (res.ok) setPoints((await res.json()).points ?? []);
    }, []);
    useLive(loadTrips, 15000);
    useEffect(() => {
      if (sel === null) return;
      loadPoints(sel);
      const t = setInterval(() => loadPoints(sel), 20000);
      return () => clearInterval(t);
    }, [sel, loadPoints]);
    /* ----- */
    const reps = useMemo(() => [...new Set(trips.map((t) => t.repName))].sort(), [trips]);

    const filtered = useMemo(
      () =>
        trips.filter(
          (t) => (!rep || t.repName === rep) && (!q || `${t.repName} ${t.dateShamsi}`.toLowerCase().includes(q.toLowerCase())),
        ),
      [trips, rep, q],
    );
    const grouped = useMemo(() => {
      const m = new Map<string, Trip[]>();
      for (const t of filtered) {
        if (!m.has(t.repName)) m.set(t.repName, []);
        m.get(t.repName)!.push(t);
      }
      return [...m.entries()].sort((a, b) => a[0].localeCompare(b[0]));
    }, [filtered]);
    const selected = trips.find((t) => t.id === sel);
  }

```

```

const stops = points.filter((p) => p.kind === "pause");
const totalStop = points.reduce((a, p) => a + (p.stopSeconds ?? 0), 0);
/**      (      ) */
const distance = useMemo(() => {
  if (selected?.distanceM) return selected.distanceM;
  let d = 0;
  for (let i = 1; i < points.length; i++) {
    const a = points[i - 1];
    const b = points[i];
    const R = 6371000;
    const dLat = ((b.lat - a.lat) * Math.PI) / 180;
    const dLng = ((b.lng - a.lng) * Math.PI) / 180;
    const s =
      Math.sin(dLat / 2) ** 2 +
      Math.cos((a.lat * Math.PI) / 180) * Math.cos((b.lat * Math.PI) / 180) * Math.sin(dLng / 2) ** 2;
    const seg = 2 * R * Math.asin(Math.sqrt(s));
    if (seg > 5 && seg < 5000) d += seg;
  }
  return d;
}, [points, selected]);
return (
  <div className="space-y-4">
    <div className="flex flex-wrap items-center justify-between gap-2">
      <SectionTitle icon=""> </SectionTitle>
      <div className="flex flex-wrap gap-2">
        <Button variant={onlyActive ? "primary" : "ghost"} onClick={() => setOnlyActive((v) => !v)}>
          {onlyActive ? " " : " "}
        </Button>
        <Button variant="success" onClick={async () => setDl(await downloadFile("/api/export?type=trips", "trips.xls"))}>
          </Button>
      </div>
    </div>
    <div>
      {dl ? <Alert kind={dl.startsWith("") ? "error" : "success"}>{dl}</Alert> : null}
    </div>
    <Card>
      <div className="flex flex-wrap items-center gap-2">
        <select
          value={rep}
          onChange={(e) => {
            setRep(e.target.value);
            setSel(null);
          }}
          className="rounded-xl border border-slate-300 px-3 py-2 text-sm"
        >
          <option value=""> {toPersianDigits(reps.length)}</option>
          {reps.map((r) => (
            <option key={r} value={r}>
              {r}
            </option>
          ))}
        </select>
        <input placeholder=" ..." value={q} onChange={(e) => setQ(e.target.value)} className="max-w-[200px]" />
        <Badge tone="slate">{toPersianDigits(filtered.length)} </Badge>
      </div>
      <span className="mr-auto text-[11px] text-slate-500"> . . . </span>
    </div>
    </Card>
    <div className="grid grid-cols-1 gap-3 lg:grid-cols-3">
      <Card className="lg:col-span-1">
        <div className="max-h-[520px] space-y-3 overflow-y-auto">
          {grouped.map(([repName, list]) => {
            const totalDist = list.reduce((a, t) => a + (t.distanceM ?? 0), 0);
            return (
              <div key={repName}>
                <div className="mb-1 flex flex-wrap items-center gap-2 rounded-lg bg-slate-100 px-2 py-1.5">
                  <span className="text-xs font-black text-slate-800">{repName}</span>
                  <Badge tone="slate">{toPersianDigits(list.length)} </Badge>
                  {totalDist > 0 ? <Badge tone="green">{formatDistance(totalDist)}</Badge> : null}
                  {list.some((t) => t.status === "active") ? <Badge tone="green"> </Badge> : null}
                </div>
                <div className="space-y-1">
                  {list.map((t) => (
                    <button
                      key={t.id}
                      onClick={() => setSel(t.id)}
                      className={`w-full rounded-xl px-3 py-2 text-right text-xs ring-1 transition ${
                        sel === t.id ? "bg-teal-50 ring-teal-300" : "bg-slate-50 ring-slate-200 hover:bg-slate-100"
                      }`}
                    >
                      </button>
                    <div className="flex items-center justify-between gap-1">
                      <span className="font-bold text-slate-700">{toPersianDigits(t.dateShamsi)}</span>
                      <Badge tone={t.status === "active" ? "green" : "slate"}>
                        {t.status === "active" ? " " : " "}
                      </Badge>
                    </div>
                  ))}
                </div>
              </div>
            )
          })}
        </div>
      </Card>
    </div>
  </div>
);

```



```

        </div>
        <div className="mt-1 flex flex-wrap gap-2 text-[10px] text-slate-500">
          <span> {tehranTime(t.startedAt)}</span>
          {t.endedAt ? <span> {tehranTime(t.endedAt)}</span> : null}
          <span>{toPersianDigits(t.points ?? 0)} </span>
          {t.distanceM ? <span>{formatDistance(t.distanceM)}</span> : null}
          {t.stopSeconds ? <span> {formatDuration(t.stopSeconds)}</span> : null}
        </div>
      </button>
    )}
  </div>
</div>
);
}}}
{grouped.length === 0 ? <p className="py-6 text-center text-sm text-slate-400"> </p> : null}
</div>
</Card>
<Card className="lg:col-span-2">
  {selected ? (
    <
      <div className="mb-2 flex flex-wrap items-center gap-2">
        <span className="text-sm font-black text-slate-800">{selected.repName}</span>
        <Badge tone="slate">{toPersianDigits(selected.dateShamsi)}</Badge>
        <Badge tone="green">{formatDistance(distance)}</Badge>
        <Badge tone="amber"> {formatDuration(totalStop || selected.stopSeconds || 0)}</Badge>
        <Badge tone="slate">{toPersianDigits(points.length)} </Badge>
      </div>
      <MapBox
        height={380}
        path={points.map((p) => ({ lat: p.lat, lng: p.lng })))}
        points={[
          ...places,
          ...points
            .filter((p) => p.kind !== "move")
            .map((p) => ({
              lat: p.lat,
              lng: p.lng,
              label: `${
                p.kind === "start" ? " " " : p.kind === "end" ? " " " : " "
              } — ${tehranTime(p.recordedAt)}${p.stopSeconds ? ` (${formatDuration(p.stopSeconds)})` : ""}`,
              color: p.kind === "pause" ? "#f59e0b" : p.kind === "end" ? "#e11d48" : "#16a34a",
            })},
        ]}
      />
      <div className="mt-3 max-h-44 space-y-1 overflow-y-auto text-[11px]">
        {points
          .filter((p) => p.kind !== "move")
          .map((p) => (
            <div key={p.id} className="flex flex-wrap items-center gap-2 rounded-lg bg-slate-50 px-3 py-1.5">
              <span>{p.kind === "start" ? " " " : p.kind === "end" ? " " " : " "}</span>
              <span className="text-slate-500">{tehranDateTime(p.recordedAt)}</span>
              {p.stopSeconds ? <Badge tone="amber">{formatDuration(p.stopSeconds)}</Badge> : null}
              {!p.gpsOn ? <Badge tone="slate">GPS </Badge> : null}
              {p.note ? <span className="text-slate-600">{p.note}</span> : null}
            </div>
          ))}
        {points.length === 0 ? <p className="py-4 text-center text-slate-400"> </p> : null}
      </div>
    </>
  ) : (
    <
      <p className="mb-2 text-center text-sm text-slate-400">
        </p>
      <MapBox height={400} points={places} />
    </>
  )}
</Card>
</div>
</div>
);
}

```

```

@@ src/app/admin/users/page.tsx typescript
"use client";
import { useCallback, useEffect, useState } from "react";
import { Alert, Badge, Button, Card, Field, Input, SectionTitle } from "@components/ui";
import {
  ALL_PERMISSION_KEYS,
  PERMISSION_GROUPS,
  REP_DEFAULT_PERMISSIONS,
  SUPERVISOR_DEFAULT_PERMISSIONS,
} from "@lib/defaults";
import { tehranDateTime, toPersianDigits } from "@lib/jalali";

```

```

import { useConfirm } from "@components/Confirm";
import { useLive } from "@lib/useLive";
import { downloadFile } from "@lib/download";
type Role = { id: number; key: string; label: string; base: string; permissions: string[]; builtin: boolean };
type User = {
  id: number;
  username: string;
  fullName: string;
  phone: string;
  role: string;
  active: boolean;
  requirePhone: boolean;
  deviceId?: string;
  deviceInfo?: string;
  deviceBoundAt?: string | null;
  simMode?: string;
  permissions: string[];
  passwordPlain: string;
  lastSeenAt: string | null;
};
const blank = {
  username: "",
  password: "",
  fullName: "",
  phone: "",
  role: "rep",
  requirePhone: true,
  permissions: [...REP_DEFAULT_PERMISSIONS] as string[],
};
export default function UsersPage() {
  const [rows, setRows] = useState<User[]>([]);
  const [form, setForm] = useState<{ ...blank }>();
  const [msg, setMsg] = useState<{ kind: "error" | "success"; text: string } | null>(null);
  const [openUser, setOpenUser] = useState<number | null>(null);
  const [showAllPass, setShowAllPass] = useState(false);
  const [showPass, setShowPass] = useState<Record<number, boolean>>({});
  const [pwDraft, setPwDraft] = useState<Record<number, string>>({});
  const [busy, setBusy] = useState(false);
  const [roles, setRoles] = useState<Role[]>([]);
  const [roleForm, setRoleForm] = useState<{ label: "", key: "", base: "rep" }>();
  const [showRoles, setShowRoles] = useState(false);
  const confirm = useConfirm();
  const load = useCallback(async () => {
    const res = await fetch("/api/users", { cache: "no-store" });
    if (res.ok) {
      const d = await res.json();
      setRows(
        (d.rows ?? []).map((u: User) => ({ ...u, permissions: Array.isArray(u.permissions) ? u.permissions : [] })),
      );
    } else setMsg({ kind: "error", text: " — " });
  }, []);
  const loadRoles = useCallback(async () => {
    const res = await fetch("/api/roles", { cache: "no-store" });
    if (res.ok) setRoles((await res.json()).rows ?? []);
  }, []);
  useEffect(() => {
    loadRoles();
  }, [loadRoles]);
  useLive(load, 20000);
  const create = async () => {
    if (!form.fullName.trim() || !form.username.trim() || !form.password.trim()) {
      setMsg({ kind: "error", text: " " });
      return;
    }
    if (
      !(await confirm({
        title: " ",
        message: `«${form.fullName}» «${roles.find((r) => r.key === form.role)?.label ?? form.role}»
      ,
        confirmText: "",
      })))
    )
      return;
    setBusy(true);
    const res = await fetch("/api/users", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(form),
    });
    setBusy(false);
    const d = await res.json().catch(() => ({}));
    if (res.ok) {
      setMsg({ kind: "success", text: " " });
      setForm({ ...blank });
    }
  };
}

```

```

    load();
  } else setMsg({ kind: "error", text: d.error ?? "" });
};
const patch = async (body: Record<string, unknown>, silent = false) => {
  //
  if (typeof body.id === "number") {
    setRows((prev) =>
      prev.map((u) => (u.id === body.id ? ({ ...u, ...(body as Partial<User>) } as User) : u)),
    );
  }
  const res = await fetch("/api/users", {
    method: "PATCH",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(body),
  });
  if (res.ok) {
    if (!silent) setMsg({ kind: "success", text: " " });
    load();
  } else {
    setMsg({ kind: "error", text: " " });
    load();
  }
};
const remove = async (id: number, name: string) => {
  if (
    !(await confirm({
      title: " ",
      message: `«${name}»`,
      confirmText: "",
      danger: true,
    })))
    return;
  const res = await fetch(`/api/users?id=${id}`, { method: "DELETE" });
  if (res.ok) load();
};
const togglePerm = (u: User, key: string) => {
  const has = u.permissions.includes(key);
  patch({ id: u.id, permissions: has ? u.permissions.filter((p) => p !== key) : [...u.permissions, key] }, true);
};
const setPassword = async (u: User) => {
  const pw = (pwDraft[u.id] ?? "").trim();
  if (pw.length < 4) {
    setMsg({ kind: "error", text: " " });
    return;
  }
  if (
    !(await confirm({
      title: " ",
      message: `«${u.fullName}» «${pw}»`,
      confirmText: " ",
    })))
    return;
  await patch({ id: u.id, password: pw });
  setPwDraft({ ...pwDraft, [u.id]: "" });
  setShowPass((s) => ({ ...s, [u.id]: true }));
};
return (
  <div className="space-y-4">
    <div className="flex flex-wrap items-center justify-between gap-2">
      <SectionTitle icon=""> </SectionTitle>
      <div className="flex gap-2">
        <Button variant={showAllPass ? "danger" : "ghost"} onClick={() => setShowAllPass((v) => !v)}>
          {showAllPass ? " " : " "}
        </Button>
        <Button
          variant="success"
          onClick={async () => setMsg({ kind: "success", text: await downloadFile("/api/export?type=users", "users.xls") }}>
          >
        </Button>
      </div>
    </div>
    {msg ? <Alert kind={msg.kind}>{msg.text}</Alert> : null}
    <Card>
      <h3 className="mb-2 text-sm font-bold text-slate-700"> </h3>
      <div className="grid grid-cols-1 gap-3 sm:grid-cols-2 lg:grid-cols-5">
        <Field label=" " required>
          <Input value={form.fullName} onChange={(e) => setForm({ ...form, fullName: e.target.value }} />
        </Field>
        <Field label=" " required>
          <Input value={form.username} onChange={(e) => setForm({ ...form, username: e.target.value }} className="tex

```

```

t-left" />
  </Field>
  <Field label=" " required>
    <Input value={form.password} onChange={(e) => setForm({ ...form, password: e.target.value })} className="tex
t-left" />
  </Field>
  <Field label=" ">
    <Input
      inputMode="numeric"
      value={form.phone}
      onChange={(e) => setForm({ ...form, phone: e.target.value.replace(/\D/g, "").slice(0, 11) })}
      placeholder="09xxxxxxxxx"
    />
  </Field>
  <Field label="">
    <div className="flex gap-1">
      <select
        value={form.role}
        onChange={(e) => {
          const role = e.target.value;
          const r = roles.find((x) => x.key === role);
          const perms = r?.permissions?.length
            ? r.permissions
              : r?.base === "admin"
                ? ALL_PERMISSION_KEYS
                : r?.base === "supervisor"
                  ? SUPERVISOR_DEFAULT_PERMISSIONS
                  : REP_DEFAULT_PERMISSIONS;
          setForm({ ...form, role, permissions: [...perms] });
        }}
        className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
      >
        {roles.map((r) => (
          <option key={r.key} value={r.key}>
            {r.label}
          </option>
        ))}
      </select>
      <button
        type="button"
        onClick={() => setShowRoles((v) => !v)}
        title=" "
        className="shrink-0 rounded-xl bg-teal-50 px-3 text-sm font-bold text-teal-700 ring-1 ring-teal-200"
      >

      </button>

    </div>
  </Field>
</div>
<div className="mt-3 space-y-2 rounded-xl bg-slate-50 p-3">
  <div className="flex flex-wrap items-center gap-2">
    <span className="text-xs font-bold text-slate-600"> :</span>
    <button
      onClick={() => setForm({ ...form, permissions: [...ALL_PERMISSION_KEYS] })}
      className="rounded-lg bg-teal-600 px-2 py-1 text-[11px] font-bold text-white"
    >

    </button>
    <button
      onClick={() => setForm({ ...form, permissions: [] })}
      className="rounded-lg bg-slate-300 px-2 py-1 text-[11px] font-bold text-slate-700"
    >

    </button>
    <Badge tone="slate">{toPersianDigits(form.permissions.length)} </Badge>
  </div>
  {PERMISSION_GROUPS.map((g) => (
    <div key={g.key}>
      <div className="mb-1 text-[11px] font-black text-slate-500">
        {g.icon} {g.label}
      </div>
      <div className="flex flex-wrap gap-1">
        {g.items.map((it) => {
          const on = form.permissions.includes(it.key);
          return (
            <button
              key={it.key}
              onClick={() =>
                setForm({
                  ...form,
                  permissions: on
                    ? form.permissions.filter((p) => p !== it.key)
                    : [...form.permissions, it.key],
                })
            >

```

```

    }
    className={`rounded-lg px-2 py-1 text-[11px] font-bold ${
      on ? "bg-teal-100 text-teal-700 ring-1 ring-teal-300" : "bg-white text-slate-400 ring-1 ring-sla
te-200"
    }}
  >
    {on ? " " : ""}
    {it.label}
  </button>
);
}}
</div>
</div>
))}
</div>
<div className="mt-3 flex flex-wrap items-center gap-3">
  <label className="flex items-center gap-1 text-xs font-bold text-slate-600">
    <input
      type="checkbox"
      className="size-4 accent-teal-600"
      checked={form.requirePhone}
      onChange={(e) => setForm({ ...form, requirePhone: e.target.checked })}
    />

    </label>
    <div className="flex-1" />
    <Button onClick={create} disabled={busy}>

  </Button>
</div>
</Card>
{showRoles ? (
  <Card>
    <div className="mb-2 flex flex-wrap items-center justify-between gap-2">
      <h3 className="text-sm font-bold text-slate-700"> </h3>
      <Button variant="ghost" onClick={() => setShowRoles(false)}>

    </Button>
    </div>
    <div className="grid grid-cols-1 gap-3 sm:grid-cols-4">
      <Field label=" " required>
        <Input
          value={roleForm.label}
          onChange={(e) => setRoleForm({ ...roleForm, label: e.target.value })}

          placeholder=" "
        />
      </Field>
      <Field label=" ( )">
        <Input
          value={roleForm.key}
          onChange={(e) => setRoleForm({ ...roleForm, key: e.target.value })}
          placeholder="sales"
          className="text-left"
        />
      </Field>
      <Field label=" ( )">
        <select
          value={roleForm.base}
          onChange={(e) => setRoleForm({ ...roleForm, base: e.target.value })}
          className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
        >
          <option value="rep"> ( )</option>
          <option value="supervisor"> ( )</option>
          <option value="admin"> </option>
        </select>
      </Field>
      <div className="flex items-end">
        <Button
          onClick={async () => {
            if (!roleForm.label.trim()) return;
            if (
              !(await confirm({
                title: " ",
message: `«${roleForm.label}»`,
                confirmText: "",
              }))
            )
              return;
            const res = await fetch("/api/roles", {
              method: "POST",
              headers: { "Content-Type": "application/json" },
              body: JSON.stringify(roleForm),
            });
            const d = await res.json().catch(() => ({}));

```

```

        if (res.ok) {
          setMsg({ kind: "success", text: " " });
          setRoleForm({ label: "", key: "", base: "rep" });
          loadRoles();
        } else setMsg({ kind: "error", text: d.error ?? "" });
      }}
      className="w-full"
    >

    </Button>
  </div>
</div>
<ul className="mt-3 space-y-1">
  {roles.map((r) => (
    <li key={r.id} className="flex flex-wrap items-center gap-2 rounded-xl bg-slate-50 px-3 py-2 text-xs">
      <Input
        value={r.label}
        onChange={(e) => setRoles((p) => p.map((x) => (x.id === r.id ? { ...x, label: e.target.value } : x)))}
        className="max-w-[180px] px-2 py-1 text-xs"
      />
      <span className="text-[10px] text-slate-400" dir="ltr">
        {r.key}
      </span>
      <select
        value={r.base}
        onChange={(e) => setRoles((p) => p.map((x) => (x.id === r.id ? { ...x, base: e.target.value } : x)))}
        className="rounded-lg border border-slate-200 px-2 py-1 text-[11px]"
      >
        <option value="rep">: </option>
        <option value="supervisor">: </option>
        <option value="admin">: </option>
      </select>
      {r.builtin ? <Badge tone="slate"> </Badge> : null}
      <span className="mr-auto flex gap-1">
        <button
          onClick={async () => {
            if (!(await confirm({ title: " ", message: `«${r.label}» `, confirmText:
              "" })))
              return;
            const res = await fetch("/api/roles", {
              method: "PATCH",
              headers: { "Content-Type": "application/json" },
              body: JSON.stringify({ id: r.id, label: r.label, base: r.base }),
            });

            setMsg(
              res.ok
                ? { kind: "success", text: " " }
                : { kind: "error", text: " " },
            );
            loadRoles();
            load();
          }}
          className="rounded-lg bg-emerald-100 px-2 py-1 text-[11px] font-bold text-emerald-700"
        >

        </button>
        {!r.builtin ? (
          <button
            onClick={async () => {
              if (
                !(await confirm({
                  title: " ",
                  message: `«${r.label}» `,
                  confirmText: "",
                  danger: true,
                )))
                return;
              const res = await fetch(`/api/roles?id=${r.id}`, { method: "DELETE" });
              const d = await res.json().catch(() => ({}));
              setMsg(
                res.ok ? { kind: "success", text: " " } : { kind: "error", text: d.error ?? "" },
              );
              loadRoles();
            }}
            className="rounded-lg bg-rose-100 px-2 py-1 text-[11px] font-bold text-rose-700"
          >

          </button>
        ) : null}
      </span>
    </li>
  ))}
</ul>

```

```

    </Card>
  ) : null}
  {rows.length === 0 ? <Card> .</Card> : null}
  {rows.map((u, i) => {
    const open = openUser === u.id;
    const visible = showAllPass || showPass[u.id];
    return (
      <Card key={u.id}>
        <div className="flex flex-wrap items-center gap-2">
          <span className="text-xs text-slate-400">{toPersianDigits(i + 1)}</span>
          <span className="text-sm font-black text-slate-800">{u.fullName}</span>
          <Badge tone={u.role === "admin" ? "green" : u.role === "supervisor" ? "amber" : "slate"}>
            {roles.find((r) => r.key === u.role)?.label ?? u.role}
          </Badge>
          {!u.active ? <Badge tone="amber"></Badge> : null}
          <span className="text-[11px] text-slate-400">
            : {u.lastSeenAt ? tehranDateTime(u.lastSeenAt) : "—"}
          </span>
          <div className="mr-auto flex flex-wrap gap-1">
            <button
              onClick={() => setOpenUser(open ? null : u.id)}
              className="rounded-lg bg-teal-50 px-2 py-1 text-[11px] font-bold text-teal-700 ring-1 ring-teal-200"
            >
              {(toPersianDigits(u.permissions.length))} {open ? "▲" : "▼"}
            </button>
            <button
              onClick={() => patch({ id: u.id, active: !u.active })}
              className={
                { rounded-lg px-2 py-1 text-[11px] font-bold $
                  { u.active ? "bg-amber-100 text-amber-700" : "bg-emerald-100 text-emerald-700"
                }
              }
            >
              {u.active ? " " : " "}
            </button>
            <button
              onClick={() => remove(u.id, u.fullName)}
              className="rounded-lg bg-rose-100 px-2 py-1 text-[11px] font-bold text-rose-700"
            >
              </button>
            </div>
          </div>
          <div className="mt-3 grid grid-cols-1 gap-2 sm:grid-cols-2 lg:grid-cols-4">
            <label className="block">
              <span className="mb-1 block text-[11px] font-bold text-slate-500"> </span>
              <input
                value={u.username}
                onChange={(e) => setRows((p) => p.map((x) => (x.id === u.id ? { ...x, username: e.target.value } : x))
                onBlur={() => patch({ id: u.id, username: u.username }, true)}
                className="text-left"
              />
            </label>
            <label className="block">
              <span className="mb-1 block text-[11px] font-bold text-slate-500"> </span>
              <div className="relative">
                <input
                  type={visible ? "text" : "password"}
                  readOnly
                  value={u.passwordPlain || ""}
                  placeholder=" — "
                  dir="ltr"
                  className="w-full rounded-xl border border-slate-300 bg-slate-50 py-2.5 pr-3 pl-12 text-left font-mo
no text-sm text-slate-800"
                  onFocus={(e) => e.currentTarget.select()}
                />
                <button
                  type="button"
                  onClick={() => setShowPass((s) => ({ ...s, [u.id]: !s[u.id] })))}
                  className="absolute left-2 top-1/2 -translate-y-1/2 rounded-lg bg-white px-2 py-1 text-xs ring-1 rin
g-slate-200"
                  title={visible ? " " : " "}
                >
                  {visible ? "" : ""}
                </button>
              </div>
            </div>
            {u.passwordPlain ? (
              <button
                onClick={() => navigator.clipboard?.writeText(u.passwordPlain)}
                className="mt-1 text-[10px] font-bold text-teal-700"
              >
                </button>
              ) : null}
            </label>

```

```

<label className="block">
  <span className="mb-1 block text-[11px] font-bold text-slate-500"> </span>
  <div className="flex gap-1">
    <Input
      type={visible ? "text" : "password"}
      value={pwDraft[u.id] ?? ""}
      onChange={(e) => setPwDraft({ ...pwDraft, [u.id]: e.target.value })}
      placeholder=" "
      className="text-left"
    />
    <Button variant="soft" onClick={() => setPassword(u)}>

    </Button>
  </div>
</label>
<label className="block">
  <span className="mb-1 block text-[11px] font-bold text-slate-500"> </span>
  <Input
    inputMode="numeric"
    value={u.phone}
    onChange={(e) =>
      setRows((p) =>
        p.map((x) => (x.id === u.id ? { ...x, phone: e.target.value.replace(/\D/g, "").slice(0, 11) } : x)
      )
    )
    onBlur={() => patch({ id: u.id, phone: u.phone }, true)}
  />
</label>
</div>
<div className="mt-2 flex flex-wrap items-center gap-2">
  <span className="text-[11px] font-bold text-slate-500"> </span>
  <select
    value={u.simMode ?? "device"}
    onChange={(e) => patch({ id: u.id, simMode: e.target.value })}
    className="rounded-lg border border-slate-200 px-2 py-1 text-[11px] font-bold"
    title=" "
  >
    <option value="off"> </option>

    <option value="phone"> </option>
    <option value="device"> ()</option>
    <option value="otp"> ()</option>
  </select>
  <button
    onClick={() => patch({ id: u.id, requirePhone: !u.requirePhone })}
    className={`rounded-lg px-3 py-1 text-[11px] font-bold ${
      u.requirePhone ? "bg-emerald-100 text-emerald-700" : "bg-slate-200 text-slate-600"
    }`}
  >
    {u.requirePhone ? " " : " "}
  </button>
  <span className="mx-1 h-5 w-px bg-slate-200" />
  <span className="text-[11px] font-bold text-slate-500"> </span>
  {u.deviceId ? (
    <div>
      <span className="rounded-lg bg-emerald-100 px-2 py-1 text-[11px] font-bold text-emerald-700">
        {u.deviceInfo || ""}
        {u.deviceBoundAt ? ` — ${tehranDateTime(u.deviceBoundAt)} : ` : ""}
      </span>
      <button
        onClick={async () => {
          if (
            !(await confirm({
              title: " ",
              message: ` «${u.fullName}»
            ,
              confirmText: " ",
              danger: true,
            )))
            return;
          patch({ id: u.id, releaseDevice: true });
        }}
        className="rounded-lg bg-amber-100 px-2 py-1 text-[11px] font-bold text-amber-700"
      >

      </button>
    </div>
  ) : (
    <span className="rounded-lg bg-slate-200 px-2 py-1 text-[11px] font-bold text-slate-600">

    </span>
  )}
</div>

```



```

<span className="text-[11px] font-bold text-slate-500">:</span>
<select
  value={u.role}
  onChange={(e) => patch({ id: u.id, role: e.target.value })}
  className="rounded-lg border border-slate-200 px-2 py-1 text-[11px]"
>
  {roles.map((r) => (
    <option key={r.key} value={r.key}>
      {r.label}
    </option>
  ))}
</select>
</div>
{open ? (
  <div className="fade-in mt-3 space-y-3 rounded-2xl bg-slate-50 p-3 ring-1 ring-slate-200">
    <div className="flex flex-wrap gap-2">
      <Button onClick={() => patch({ id: u.id, permissions: ALL_PERMISSION_KEYS }, true)}>

      </Button>
      <Button variant="soft" onClick={() => patch({ id: u.id, permissions: SUPERVISOR_DEFAULT_PERMISSIONS },
true)}>

      </Button>
      <Button variant="soft" onClick={() => patch({ id: u.id, permissions: REP_DEFAULT_PERMISSIONS }, true)}>

      </Button>
      <Button variant="danger" onClick={() => patch({ id: u.id, permissions: [] }, true)}>

      </Button>
    </div>
    {PERMISSION_GROUPS.map((g) => {
      const groupKeys = g.items.map((it) => it.key);
      const allOn = groupKeys.every((k) => u.permissions.includes(k));
      return (
        <div key={g.key} className="rounded-xl bg-white p-3 ring-1 ring-slate-200">
          <div className="mb-2 flex items-center justify-between">
            <span className="text-xs font-black text-slate-700">

              {g.icon} {g.label}
            </span>
            <button
              onClick={() =>
                patch(
                  {
                    id: u.id,
                    permissions: allOn
                      ? u.permissions.filter((p) => !groupKeys.includes(p))
                      : [...new Set([...u.permissions, ...groupKeys])],
                  },
                  true,
                )
              }
              className={`rounded-lg px-2 py-1 text-[10px] font-bold ${
                allOn ? "bg-rose-100 text-rose-700" : "bg-teal-100 text-teal-700"
              }`}
            >
              {allOn ? " " : " "}
            </button>
          </div>
          <div className="grid grid-cols-1 gap-1 sm:grid-cols-2 lg:grid-cols-3">
            {g.items.map((it) => {
              const on = u.permissions.includes(it.key);
              return (
                <label
                  key={it.key}
                  className={`flex cursor-pointer items-center justify-between gap-2 rounded-lg px-2.5 py-2
                    text-xs ${
                      on ? "bg-teal-50 ring-1 ring-teal-200" : "bg-slate-50"
                    }`}
                >
                  <span className="font-bold text-slate-700">{it.label}</span>
                  <input
                    type="checkbox"
                    className="size-4 accent-teal-600"
                    checked={on}
                    onChange={() => togglePerm(u, it.key)}
                  />
                </label>
              );
            })}
          </div>
        </div>
      );
    })}
  )}
</div>
)}
</div>
);
}}
}

```

```

        </div>
    ) : null}
    </Card>
  );
  })}
</div>
);
}

```

```

@@ src/app/api/activity/route.ts typescript
import { db, dbRetry } from "@db";
import { activityLogs, doctors, leaves, orders, pharmacies, trips, users } from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { count, desc, eq, sql } from "drizzle-orm";
export const dynamic = "force-dynamic";
export async function GET() {
  const user = await getSessionUser();
  if (!user || (user.role !== "admin" && user.role !== "supervisor")) {
    return Response.json({ error: " " }, { status: 401 });
  }
  const [logs, userRows, phG, drG, orG, trG, activeG, pendingLeaves] = await dbRetry(() => Promise.all([
    db.select().from(activityLogs).orderBy(desc(activityLogs.id)).limit(200),
    db
      .select({
        id: users.id,
        fullName: users.fullName,
        role: users.role,
        active: users.active,
        lastSeenAt: users.lastSeenAt,
      })
      .from(users)
      .orderBy(users.id),
    db.select({ uid: pharmacies.userId, c: count() }).from(pharmacies).groupBy(pharmacies.userId),
    db.select({ uid: doctors.userId, c: count() }).from(doctors).groupBy(doctors.userId),
    db.select({ uid: orders.userId, c: count() }).from(orders).groupBy(orders.userId),
    db.select({ uid: trips.userId, c: count() }).from(trips).groupBy(trips.userId),

    db
      .select({ uid: trips.userId, c: count() })
      .from(trips)
      .where(eq(trips.status, "active"))
      .groupBy(trips.userId),
    db.select({ c: count() }).from(leaves).where(eq(leaves.managerStatus, "pending")),
  ]), "activity:load");
  const toMap = (rows: { uid: number; c: number }[]) => {
    const m = new Map<number, number>();
    for (const r of rows) m.set(r.uid, Number(r.c));
    return m;
  };
  const mPh = toMap(phG);
  const mDr = toMap(drG);
  const mOr = toMap(orG);
  const mTr = toMap(trG);
  const mAc = toMap(activeG);
  const lastByUser = new Map<number, (typeof logs)[number]>();
  for (const l of logs) {
    if (l.userId != null && !lastByUser.has(l.userId)) lastByUser.set(l.userId, l);
  }
  const reps = userRows.map((u) => {
    const last = lastByUser.get(u.id);
    return {
      ...u,
      pharmacies: mPh.get(u.id) ?? 0,
      doctors: mDr.get(u.id) ?? 0,
      orders: mOr.get(u.id) ?? 0,
      trips: mTr.get(u.id) ?? 0,
      activeTrip: mAc.get(u.id) ?? 0,
      lastAction: last ? `${last.action}${last.detail ? ` — ${last.detail}` : ""}` : null,
      lastActionAt: last ? last.createdAt : null,
    };
  });
  const byRep: Record<string, typeof logs> = {};
  for (const l of logs) {
    const k = l.userName || "";
    (byRep[k] ?? []).push(l);
  }
  const sum = (m: Map<number, number>) => [...m.values()].reduce((a, b) => a + b, 0);
  const counts = {
    pharmacies: sum(mPh),
    doctors: sum(mDr),
    orders: sum(mOr),
    trips: sum(mTr),
    activeTrips: sum(mAc),
    users: userRows.length,
    leaves: Number(pendingLeaves[0]?.c ?? 0),
  };

```

```

    };
    void sql;
    return Response.json({ logs, counts, reps, byRep });
}

```

@@ src/app/api/attachments/route.ts typescript

```

import { db } from "@db";
import { attachments } from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { and, desc, eq, inArray } from "drizzle-orm";
export const dynamic = "force-dynamic";
const MAX_BYTES = 3 * 1024 * 1024; // 3MB
export async function GET(req: Request) {
    const user = await getSessionUser();
    if (!user) return Response.json({ error: " " }, { status: 401 });
    const url = new URL(req.url);
    const ownerType = url.searchParams.get("ownerType") ?? "doctor";
    const ownerId = url.searchParams.get("ownerId");
    const isAdmin = user.role === "admin" || user.role === "supervisor";
    const conds = [eq(attachments.ownerType, ownerType)];
    if (ownerId) conds.push(eq(attachments.ownerId, Number(ownerId)));
    if (!isAdmin) conds.push(eq(attachments.userId, user.id));
    const rows = await db
        .select({
            id: attachments.id,
            ownerId: attachments.ownerId,

            fileName: attachments.fileName,
            mimeType: attachments.mimeType,
            size: attachments.size,
            repName: attachments.repName,
            createdAt: attachments.createdAt,
        })
        .from(attachments)
        .where(and(...conds))
        .orderBy(desc(attachments.id))
        .limit(300);
    return Response.json({ rows });
}
export async function POST(req: Request) {
    const user = await getSessionUser();
    if (!user) return Response.json({ error: " " }, { status: 401 });
    const b = await req.json().catch(() => ({}));
    const dataUrl = String(b.data ?? "");
    const match = dataUrl.match(/^data:([^\;]+);base64,(.+$)/);
    if (!match) return Response.json({ error: " " }, { status: 400 });
    const mimeType = match[1].slice(0, 100);
    const base64 = match[2];
    const size = Math.floor((base64.length * 3) / 4);
    if (size > MAX_BYTES) {
        return Response.json({ error: " " }, { status: 400 });
    }
    const [row] = await db
        .insert(attachments)
        .values({
            ownerType: String(b.ownerType ?? "doctor").slice(0, 30),
            ownerId: Number(b.ownerId) || null,
            userId: user.id,
            repName: user.fullName,
            fileName: String(b.fileName ?? "file").slice(0, 200),
            mimeType,
            size,
            data: base64,
        })
        .returning({
            id: attachments.id,
            fileName: attachments.fileName,
            mimeType: attachments.mimeType,
            size: attachments.size,
            ownerId: attachments.ownerId,
        });
    return Response.json({ row });
}
/** ( ) */
export async function PATCH(req: Request) {
    const user = await getSessionUser();
    if (!user) return Response.json({ error: " " }, { status: 401 });
    const b = await req.json().catch(() => ({}));
    const ids: number[] = Array.isArray(b.ids) ? b.ids.map(Number).filter(Number.isFinite) : [];
    const ownerId = Number(b.ownerId);
    if (!ids.length || !Number.isFinite(ownerId)) return Response.json({ error: " " }, { status: 400 });
    await db
        .update(attachments)
        .set({ ownerId })
        .where(and(inArray(attachments.id, ids), eq(attachments.userId, user.id)));
}

```

```

    return Response.json({ ok: true });
}
export async function DELETE(req: Request) {
    const user = await getSessionUser();
    if (!user) return Response.json({ error: " " }, { status: 401 });
    const id = Number(new URL(req.url).searchParams.get("id"));
    if (!Number.isFinite(id)) return Response.json({ error: " " }, { status: 400 });
    if (user.role === "admin") await db.delete(attachments).where(eq(attachments.id, id));
    else await db.delete(attachments).where(and(eq(attachments.id, id), eq(attachments.userId, user.id)));
    return Response.json({ ok: true });
}

```

@@ src/app/api/backup/route.ts typescript

```

import { db } from "@db";
import {
    activityLogs,
    attachments,
    doctors,
    homes,
    leaves,
    messengers,
    notifications,
    options,
    orders,

    roles,
    pharmacies,
    settings,
    tripPoints,
    trips,
    users,
} from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { todayJalali } from "@lib/jalali";
export const dynamic = "force-dynamic";
export async function GET(req: Request) {
    const user = await getSessionUser();
    if (!user || user.role !== "admin") return Response.json({ error: " " }, { status: 401 });
    const withFiles = new URL(req.url).searchParams.get("files") === "1";
    const [
        usersRows,
        pharmaciesRows,
        doctorsRows,
        ordersRows,
        optionsRows,
        homesRows,
        leavesRows,
        tripsRows,
        tripPointsRows,
        messengersRows,
        notificationsRows,
        settingsRows,
        activityRows,
        rolesRows,
    ] = await Promise.all([
        db.select().from(users),
        db.select().from(pharmacies),
        db.select().from(doctors),
        db.select().from(orders),
        db.select().from(options),
        db.select().from(homes),
        db.select().from(leaves),
        db.select().from(trips),
        db.select().from(tripPoints),
        db.select().from(messengers),
        db.select().from(notifications),
        db.select().from(settings),
        db.select().from(activityLogs),
        db.select().from(roles),
    ]);
    const files = withFiles
        ? await db.select().from(attachments)
        : await db
            .select({
                id: attachments.id,
                ownerType: attachments.ownerType,
                ownerId: attachments.ownerId,
                fileName: attachments.fileName,
                mimeType: attachments.mimeType,
                size: attachments.size,
            })
            .from(attachments);
    const payload = {
        app: "sabt-etelaat-kol",
        version: 1,
    };
}

```


[illegible]

```

        r.notes,
        mapLink(r.lat, r.lng),
        r.sendStatus || (r.sent ? " " : " "),
    ]),
    ];
} else if (type === "trips") {
    const w: SQL | undefined = isAdmin ? undefined : eq(trips.userId, user.id);
    const data = await db.select().from(trips).where(w).orderBy(desc(trips.id));
    const rows = [
        ["", " ", "", "", "", "", " ()", " ()"],
        ...data.map((r, i) => [
            i + 1,
            r.repName,
            r.dateShamsi,
            r.status === "active" ? " " : " ",

            tehranDateTime(r.startedAt),
            r.endedAt ? tehranDateTime(r.endedAt) : "",
            ((r.distanceM ?? 0) / 1000).toFixed(2),
            Math.round((r.stopSeconds ?? 0) / 60),
        ]),
    ];
} else if (type === "leaves") {
    const w: SQL | undefined = isAdmin ? undefined : eq(leaves.userId, user.id);
    const data = await db.select().from(leaves).where(w).orderBy(desc(leaves.id));
    const fa = (s: string) => (s === "approved" ? "" : s === "rejected" ? "" : " ");
    const rows = [
        ["", " ", "", " ", " ", " ", " ", " ", " ", " ", " ()", "", " "],
        ...data.map((r, i) => [
            i + 1,
            r.repName,
            r.kind,
            r.fromDate,
            r.toDate,
            r.days,
            r.fromTime,
            r.toTime,
            r.hours,
            r.reason,
            fa(r.supervisorStatus),
            fa(r.managerStatus),
        ]),
    ];
} else if (type === "homes") {
    if (!isAdmin) return Response.json({ error: " " }, { status: 401 });
    const data = await db.select().from(homes).orderBy(desc(homes.id));
    const rows = [
        ["", " ", "", "", " "],
        ...data.map((r, i) => [i + 1, r.repName, r.title, r.address, mapLink(r.lat, r.lng)]),
    ];
} else if (type === "users") {
    if (user.role !== "admin") return Response.json({ error: " " }, { status: 401 });
    const data = await db.select().from(users).orderBy(desc(users.id));
    const rows = [
        ["", "", " ", " ", " ", " ", "", ""],
        ...data.map((r, i) => [
            i + 1,
            r.fullName,
            r.username,
            r.passwordPlain,
            r.phone,
            r.role === "admin" ? "" : r.role === "supervisor" ? "" : " ",
            r.active ? "" : "",
        ]),
    ];
} else {
    return Response.json({ error: " " }, { status: 400 });
}
return xlsResponse(`${name}-${Date.now()}`, name, rows);
}

```

```

@@ src/app/api/geocode/route.ts typescript
import { getSessionUser } from "@lib/auth";
import { fetchWithRetry } from "@lib/retry";
export const dynamic = "force-dynamic";
/**
 * (Geocoding) — .
 *
 * CORS
 *
 * GET /api/geocode?q=
 * GET /api/geocode?lat=35.7&lng=51.4    (: →)
 */

```

```

type Hit = { label: string; lat: number; lng: number; type: string };
const UA = "SabtEtelaatKol/1.0 (contact: admin@ndcohub.ir)";
async function nominatim(q: string): Promise<Hit[]> {
    const url =
        `https://nominatim.openstreetmap.org/search?format=jsonv2&limit=8&accept-language=fa` +
        `&countrycodes=ir&q=${encodeURIComponent(q)}`;
    const res = await fetchWithRetry(
        url,
        { headers: { "User-Agent": UA, "Accept-Language": "fa" } },
        { retries: 2, timeoutMs: 12000, label: "geocode" },
    );
    const data = (await res.json()) as { display_name: string; lat: string; lon: string; type?: string }[];
    return data.map((d) => ({
        label: d.display_name,
        lat: Number(d.lat),
        lng: Number(d.lon),
        type: d.type ?? "",
    }));
}
async function photon(q: string): Promise<Hit[]> {
    // lat/lon
    const url =
        `https://photon.komoot.io/api/?limit=10&lang=default&lat=32.4279&lon=53.6880` +
        `&q=${encodeURIComponent(q)}`;
    const res = await fetchWithRetry(url, {}, { retries: 1, timeoutMs: 10000, label: "geocode2" });
    const data = (await res.json()) as {
        features: { properties: Record<string, string>; geometry: { coordinates: [number, number] } }[];
    };
    return (data.features ?? []).
        .filter((f) => {
            const c = (f.properties?.countrycode ?? "").toUpperCase();
            return !c || c === "IR"; //
        })
        .map((f) => {
            const p = f.properties ?? {};
            const label = [p.name, p.street, p.district, p.city, p.county, p.state].filter(Boolean).join(" ");
            return {
                label: label || p.name || "",
                lat: f.geometry.coordinates[1],
                lng: f.geometry.coordinates[0],
                type: p.osm_value ?? "",
            };
        })
        .filter((h) => h.label);
}
async function reverse(lat: number, lng: number): Promise<Hit[]> {
    const url = `https://nominatim.openstreetmap.org/reverse?format=jsonv2&accept-language=fa&lat=${lat}&lon=${lng}`;
    const res = await fetchWithRetry(
        url,
        { headers: { "User-Agent": UA, "Accept-Language": "fa" } },
        { retries: 2, timeoutMs: 12000, label: "revgeo" },
    );
    const d = (await res.json()) as { display_name?: string };
    return d?.display_name ? [{ label: d.display_name, lat, lng, type: "reverse" }] : [];
}
export async function GET(req: Request) {
    const user = await getSessionUser();
    if (!user) return Response.json({ error: " " }, { status: 401 });
    const url = new URL(req.url);
    const q = (url.searchParams.get("q") ?? "").trim();
    // : Number(null)
    // « »
    const latRaw = url.searchParams.get("lat");
    const lngRaw = url.searchParams.get("lng");
    const lat = latRaw === null ? NaN : Number(latRaw);
    const lng = lngRaw === null ? NaN : Number(lngRaw);
    try {
        if (Number.isFinite(lat) && Number.isFinite(lng) && (lat !== 0 || lng !== 0)) {
            return Response.json({ results: await reverse(lat, lng) }, { headers: { "Cache-Control": "no-store" } });
        }
        if (q.length < 2) return Response.json({ results: [] });
        // : «35.7219,51.4089»
        const coord = q.match(/^\s*(-?\d{1,2}\.\d+)\s*[,\]\s*(-?\d{1,3}\.\d+)\s*$/);
        if (coord) {
            const la = Number(coord[1]);
            const ln = Number(coord[2]);
            return Response.json({
                results: [{ label: ` : ${la}, ${ln}`, lat: la, lng: ln, type: "coords" }],
            });
        }
    }
    /**
     * Photon      Nominatim /search
     */
}

```



```

    */
    const providers: { name: string; run: () => Promise<Hit[]> }[] = [
        { name: "photon", run: () => photon(q) },
        { name: "nominatim", run: () => nominatim(q) },
    ];
    let results: Hit[] = [];
    const notes: string[] = [];
    for (const p of providers) {
        try {
            const r = await p.run();
            if (r.length) {
                results = r;
                notes.push(`${p.name}: ${r.length}`);
                break;
            }
            notes.push(`${p.name}: 0`);
        } catch (e) {
            notes.push(`${p.name}: ${e instanceof Error ? e.message : String(e)}.slice(0, 60)}`);
        }
    }
    return Response.json(
        { results, provider: notes.join(" | ") },
        { headers: { "Cache-Control": "no-store" } },
    );
} catch (err) {
    return Response.json(
        { results: [], error: err instanceof Error ? err.message : " " },
        { status: 200 },
    );
}
}

@@ src/app/api/health/route.ts typescript
import { checkDatabase, dbInfo, hasDatabaseUrl } from "@db";
import { ensureSeed } from "@lib/bootstrap";
export const dynamic = "force-dynamic";
export const revalidate = 0;
/**
 * Health check — Retry
 * /ping
 */
export async function GET() {
    if (!hasDatabaseUrl) {
        return Response.json(
            { ok: false, db: false, reason: "DATABASE_URL " },
            { status: 500, headers: { "Cache-Control": "no-store" } },
        );
    }
    const health = await checkDatabase();
    if (!health.ok) {
        return Response.json(
            { ok: false, db: false, latencyMs: health.latencyMs, reason: health.detail, pooler: dbInfo.usedPooler },
            { status: 503, headers: { "Cache-Control": "no-store" } },
        );
    }
    // / health
    void ensureSeed();
    return Response.json(
        {
            ok: true,
            db: true,
            latencyMs: health.latencyMs,
            pooler: dbInfo.usedPooler,
            connectTimeoutSec: dbInfo.connectTimeoutSec,
        },
        { headers: { "Cache-Control": "no-store" } },
    );
}

@@ src/app/api/homes/route.ts typescript
import { db } from "@db";
import { activityLogs, homes } from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { desc, eq } from "drizzle-orm";
export const dynamic = "force-dynamic";
export async function GET() {
    const user = await getSessionUser();
    if (!user) return Response.json({ error: " " }, { status: 401 });
    const isAdmin = user.role === "admin" || user.role === "supervisor";

    const rows = isAdmin
        ? await db.select().from(homes).orderBy(desc(homes.id))
        : await db.select().from(homes).where(eq(homes.userId, user.id)).orderBy(desc(homes.id));
    return Response.json({ rows });
}

```

```

}
export async function POST(req: Request) {
  const user = await getSessionUser();
  if (!user) return Response.json({ error: " " }, { status: 401 });
  const b = await req.json().catch(() => ({}));
  const lat = Number(b.lat);
  const lng = Number(b.lng);
  if (!Number.isFinite(lat) || !Number.isFinite(lng)) {
    return Response.json({ error: " " }, { status: 400 });
  }
  const values = {
    userId: user.id,
    repName: user.fullName,
    title: String(b.title ?? "").trim().slice(0, 160) || "",
    address: String(b.address ?? "").trim().slice(0, 1000),
    lat,
    lng,
    accuracy: Number(b.accuracy) || null,
    updatedAt: new Date(),
  };
  const existing = await db.select().from(homes).where(eq(homes.userId, user.id)).limit(1);
  let row;
  if (existing.length) {
    [row] = await db.update(homes).set(values).where(eq(homes.id, existing[0].id)).returning();
  } else {
    [row] = await db.insert(homes).values(values).returning();
  }
  await db
    .insert(activityLogs)
    .values({ userId: user.id, userName: user.fullName, action: " ", detail: values.title })
    .catch(() => undefined);
  return Response.json({ row });
}

@@ src/app/api/leaves/route.ts typescript
import { db } from "@db";
import { activityLogs, leaves } from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { isValidJalali } from "@lib/jalali";
import { notify } from "@lib/notify";
import { desc, eq } from "drizzle-orm";
export const dynamic = "force-dynamic";
export async function GET() {
  const user = await getSessionUser();
  if (!user) return Response.json({ error: " " }, { status: 401 });
  const isAdmin = user.role === "admin" || user.role === "supervisor";
  const rows = isAdmin
    ? await db.select().from(leaves).orderBy(desc(leaves.id)).limit(500)
    : await db.select().from(leaves).where(eq(leaves.userId, user.id)).orderBy(desc(leaves.id)).limit(200);
  return Response.json({ rows, role: user.role });
}
export async function POST(req: Request) {
  const user = await getSessionUser();
  if (!user) return Response.json({ error: " " }, { status: 401 });
  const b = await req.json().catch(() => ({}));
  const fromDate = String(b.fromDate ?? "").trim();
  const toDate = String(b.toDate ?? "").trim();
  if (!isValidJalali(fromDate) || !isValidJalali(toDate)) {
    return Response.json({ error: " " }, { status: 400 });
  }
  const [row] = await db
    .insert(leaves)
    .values({
      userId: user.id,
      repName: user.fullName,
      kind: String(b.kind ?? "").slice(0, 30),
      fromDate,
      toDate,
      days: Math.max(0, Number(b.days) || 0),
      fromTime: String(b.fromTime ?? "").slice(0, 5),
      toTime: String(b.toTime ?? "").slice(0, 5),
      hours: Math.max(0, Number(b.hours) || 0),
      reason: String(b.reason ?? ""),
    })
    .returning();
  await db
    .insert(activityLogs)
    .values({
      userId: user.id,
      userName: user.fullName,
      action: " ",
      detail: `${fromDate} ${toDate}`,
    })
    .catch(() => undefined);

```

```

for (const role of ["admin", "supervisor"] as const) {
  await notify({
    toRole: role,
    fromName: user.fullName,
    kind: "leave",
    title: `${user.fullName}`,
    body: `${(fromDate) ${toDate)} — ${row.kind}${row.reason ? `| ${row.reason}` : ""}`,
    link: "/admin/leaves",
  });
}
return Response.json({ row });
}
export async function PATCH(req: Request) {
  const user = await getSessionUser();
  if (!user) return Response.json({ error: " " }, { status: 401 });
  if (user.role !== "admin" && user.role !== "supervisor") {
    return Response.json({ error: " " }, { status: 403 });
  }
  const b = await req.json().catch(() => ({}));
  const id = Number(b.id);
  const status = b.status === "approved" ? "approved" : b.status === "rejected" ? "rejected" : null;
  if (!Number.isFinite(id) || !status) return Response.json({ error: " " }, { status: 400 });
  const note = String(b.note ?? "");
  const patch =
    user.role === "supervisor"
      ? { supervisorStatus: status, supervisorNote: note, supervisorName: user.fullName }
      : { managerStatus: status, managerNote: note, managerName: user.fullName };
  const target = (await db.select().from(leaves).where(eq(leaves.id, id)).limit(1))[0];
  await db.update(leaves).set(patch).where(eq(leaves.id, id));
  if (target) {
    await notify({
      toUserId: target.userId,
      fromName: user.fullName,
      kind: "leave",
      title: `${status === "approved" ? "" : ""} ${user.role === "supervisor" ? "" : ""} ${
        status === "approved" ? "" : ""
      }`,
      body: `${target.fromDate} ${target.toDate}${note ? ` — ${note}` : ""}`,
      link: "/panel/leaves",
    });
  }
  await db
    .insert(activityLogs)
    .values({
      userId: user.id,
      userName: user.fullName,
      action: user.role === "supervisor" ? "/" : "",
      detail: `${id} → ${status === "approved" ? "" : ""}`,
    })
    .catch(() => undefined);
  return Response.json({ ok: true });
}

```

```

@@ src/app/api/live/route.ts typescript
import { db, dbRetrySafe } from "@db";
import { liveLocations, trips, users } from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { desc, eq, sql } from "drizzle-orm";
export const dynamic = "force-dynamic";
/**
 * .
 *
 * POST → ( )
 * GET →
 */
export async function POST(req: Request) {
  const user = await getSessionUser();
  if (!user) return Response.json({ error: " " }, { status: 401 });
  const b = await req.json().catch(() => ({}));
  const lat = Number(b.lat);
  const lng = Number(b.lng);
  if (!Number.isFinite(lat) || !Number.isFinite(lng)) {
    return Response.json({ error: " " }, { status: 400 });
  }
  const values = {
    userId: user.id,
    repName: user.fullName,
    lat,
    lng,
    accuracy: Number(b.accuracy) || null,
    speed: Number(b.speed) || null,
    heading: Number(b.heading) || null,
    battery: Number.isFinite(Number(b.battery)) ? Math.round(Number(b.battery)) : null,
    gpsOn: b.gpsOn !== false,
    online: true,
  }

```

```

        tripId: Number(b.tripId) || null,
        recordedAt: b.recordedAt ? new Date(String(b.recordedAt)) : new Date(),
        updatedAt: new Date(),
    };
    await dbRetrySafe(
        () =>
            db
                .insert(liveLocations)
                .values(values)
                .onConflictDoUpdate({ target: liveLocations.userId, set: values }),
        undefined,
        "live:upsert",
    );
    return Response.json({ ok: true }, { headers: { "Cache-Control": "no-store" } });
}

export async function GET() {
    const user = await getSessionUser();
    if (!user) return Response.json({ error: " " }, { status: 401 });
    const isManager = user.role === "admin" || user.role === "supervisor";
    const rows = await dbRetrySafe(
        () =>
            db
                .select({
                    userId: liveLocations.userId,
                    repName: liveLocations.repName,
                    lat: liveLocations.lat,
                    lng: liveLocations.lng,
                    accuracy: liveLocations.accuracy,
                    speed: liveLocations.speed,
                    battery: liveLocations.battery,
                    gpsOn: liveLocations.gpsOn,
                    tripId: liveLocations.tripId,
                    recordedAt: liveLocations.recordedAt,
                    updatedAt: liveLocations.updatedAt,
                })
                .from(liveLocations)
                .orderBy(desc(liveLocations.updatedAt)),
        [],
        "live:list",
    );
    const scoped = isManager ? rows : rows.filter((r) => r.userId === user.id);
    //
    const active = await dbRetrySafe(
        () =>
            db
                .select({ userId: trips.userId, id: trips.id, distanceM: trips.distanceM })
                .from(trips)
                .where(eq(trips.status, "active")),
        [],
        "live:trips",
    );
    const allUsers = isManager
        ? await dbRetrySafe(
            () => db.select({ id: users.id, fullName: users.fullName, role: users.role, active: users.active }).from(users),
            [],
            "live:users",
        )
        : [];
    const now = Date.now();
    const enriched = scoped.map((r) => {
        const ageMs = now - new Date(r.updatedAt).getTime();
        return {
            ...r,
            ageMs,

            /** : */
            live: ageMs < 3 * 60 * 1000,
            stale: ageMs >= 3 * 60 * 1000 && ageMs < 60 * 60 * 1000,
            activeTrip: active.find((t) => t.userId === r.userId) ?? null,
        };
    });
    //
    const missing = allUsers
        .filter((u) => u.role === "rep" && u.active && !scoped.some((r) => r.userId === u.id))
        .map((u) => ({ userId: u.id, repName: u.fullName }));
    void sql;
    return Response.json({ rows: enriched, missing }, { headers: { "Cache-Control": "no-store" } });
}

@@ src/app/api/messengers/route.ts typescript
import { db } from "@/db";
import { messageLogs, messengers, settings } from "@/db/schema";
import { getSessionUser } from "@/lib/auth";
import { getProxy } from "@/lib/messaging";
import { getSmsConfig } from "@/lib/otp";

```

```

import { desc, eq } from "drizzle-orm";
export const dynamic = "force-dynamic";
async function adminGuard() {
  const user = await getSessionUser();
  if (!user) return null;
  if (user.role === "admin" || user.permissions.includes("messengers")) return user;
  return null;
}
export async function GET() {
  const admin = await adminGuard();
  if (!admin) return Response.json({ error: " " }, { status: 401 });
  const rows = await db.select().from(messengers).orderBy(desc(messengers.id));
  const logs = await db.select().from(messageLogs).orderBy(desc(messageLogs.id)).limit(50);
  const proxy = await getProxy();
  const sms = await getSmsConfig();
  let autoSend = true;
  try {
    const st = await db.select().from(settings).where(eq(settings.key, "autoSendOrders")).limit(1);
    const v = st[0]?.value as { enabled?: boolean } | undefined;
    if (v && typeof v.enabled === "boolean") autoSend = v.enabled;
  } catch {
    /* ignore */
  }
  return Response.json({ rows, logs, proxy, sms, autoSend });
}
export async function POST(req: Request) {
  const admin = await adminGuard();
  if (!admin) return Response.json({ error: " " }, { status: 401 });
  const b = await req.json().catch(() => ({}));
  const platform = String(b.platform ?? "");
  if (!["telegram", "bale", "eitaa", "whatsapp"].includes(platform)) {
    return Response.json({ error: " " }, { status: 400 });
  }
  const [row] = await db
    .insert(messengers)
    .values({
      platform,
      label: String(b.label ?? "").trim().slice(0, 160),
      targetType: ["group", "channel", "phone"].includes(b.targetType) ? b.targetType : "phone",
      target: String(b.target ?? "").trim(),
      token: String(b.token ?? "").trim(),
      provider: String(b.provider ?? "").trim().slice(0, 30),
      apiUrl: String(b.apiUrl ?? "").trim(),
      enabled: b.enabled !== false,
    })
    .returning();
  return Response.json({ row });
}
export async function PATCH(req: Request) {
  const admin = await adminGuard();
  if (!admin) return Response.json({ error: " " }, { status: 401 });
  const b = await req.json().catch(() => ({}));
  const id = Number(b.id);
  if (!Number.isFinite(id)) return Response.json({ error: " " }, { status: 400 });
  const patch: Record<string, unknown> = {};
  if (typeof b.label === "string") patch.label = b.label.trim();
  if (typeof b.target === "string") patch.target = b.target.trim();
  if (typeof b.targetType === "string" && ["group", "channel", "phone"].includes(b.targetType)) {
    patch.targetType = b.targetType;
  }
  if (typeof b.token === "string") patch.token = b.token.trim();
  if (typeof b.provider === "string") patch.provider = b.provider.trim().slice(0, 30);
  if (typeof b.apiUrl === "string") patch.apiUrl = b.apiUrl.trim();
  if (typeof b.enabled === "boolean") patch.enabled = b.enabled;
  if (Object.keys(patch).length === 0) return Response.json({ error: " " }, { status: 400 });
  await db.update(messengers).set(patch).where(eq(messengers.id, id));
  return Response.json({ ok: true });
}
export async function DELETE(req: Request) {
  const admin = await adminGuard();
  if (!admin) return Response.json({ error: " " }, { status: 401 });
  const id = Number(new URL(req.url).searchParams.get("id"));
  if (!Number.isFinite(id)) return Response.json({ error: " " }, { status: 400 });
  await db.delete(messengers).where(eq(messengers.id, id));
  return Response.json({ ok: true });
}
}

@@ src/app/api/notifications/route.ts typescript
import { db } from "@db";
import { notifications, users } from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { and, desc, eq, inArray, isNull, or, sql, SQL } from "drizzle-orm";
export const dynamic = "force-dynamic";
export async function GET(req: Request) {

```

```

const user = await getSessionUser();
if (!user) return Response.json({ error: " " }, { status: 401 });
const onlyUnread = new URL(req.url).searchParams.get("unread") === "1";
const scope: SQL | undefined =
  user.role === "rep"
    ? or(eq(notifications.toUserId, user.id), eq(notifications.toRole, "rep"))
    : or(
      eq(notifications.toUserId, user.id),
      eq(notifications.toRole, user.role),
      and(isNull(notifications.toUserId), eq(notifications.toRole, "")),
    );
const where = onlyUnread ? and(scope, isNull(notifications.readAt)) : scope;
const rows = await db.select().from(notifications).where(where).orderBy(desc(notifications.id)).limit(100);
const unread = rows.filter((r) => !r.readAt).length;
return Response.json({ rows, unread });
}
/** / */
export async function POST(req: Request) {
  const user = await getSessionUser();
  if (!user) return Response.json({ error: " " }, { status: 401 });
  const b = await req.json().catch(() => ({}));
  const title = String(b.title ?? "").trim().slice(0, 200);
  if (!title) return Response.json({ error: " " }, { status: 400 });
  const body = String(b.body ?? "").slice(0, 2000);
  if (user.role === "rep") {
    await db.insert(notifications).values({
      toRole: "admin",
      fromName: user.fullName,
      kind: "message",
      title,
      body,
    });
    return Response.json({ ok: true });
  }
  const targetId = Number(b.toUserId);
  if (Number.isFinite(targetId) && targetId > 0) {
    await db
      .insert(notifications)
      .values({ toUserId: targetId, fromName: user.fullName, kind: "message", title, body });
  } else {
    const reps = await db.select({ id: users.id }).from(users).where(eq(users.role, "rep"));
    if (reps.length) {
      await db
        .insert(notifications)
        .values(reps.map((r) => ({ toUserId: r.id, fromName: user.fullName, kind: "message", title, body })));
    }
  }
  return Response.json({ ok: true });
}
/** */

export async function PATCH(req: Request) {
  const user = await getSessionUser();
  if (!user) return Response.json({ error: " " }, { status: 401 });
  const b = await req.json().catch(() => ({}));
  const ids: number[] = Array.isArray(b.ids) ? b.ids.map(Number).filter(Number.isFinite) : [];
  if (b.all === true) {
    const scope: SQL | undefined =
      user.role === "rep"
        ? or(eq(notifications.toUserId, user.id), eq(notifications.toRole, "rep"))
        : or(
          eq(notifications.toUserId, user.id),
          eq(notifications.toRole, user.role),
          and(isNull(notifications.toUserId), eq(notifications.toRole, "")),
        );
    await db
      .update(notifications)
      .set({ readAt: new Date() })
      .where(and(scope, isNull(notifications.readAt)));
    return Response.json({ ok: true });
  }
  if (!ids.length) return Response.json({ error: " " }, { status: 400 });
  await db.update(notifications).set({ readAt: new Date() }).where(inArray(notifications.id, ids));
  void sql;
  return Response.json({ ok: true });
}

@@ src/app/api/options/route.ts typescript
import { db } from "@db";
import { activityLogs, options } from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { ensureSeed } from "@lib/bootstrap";
import { OPTION_CATEGORIES } from "@lib/constants";
import { and, asc, eq } from "drizzle-orm";
export const dynamic = "force-dynamic";

```

```

const VALID = OPTION_CATEGORIES.map((c) => c.key);
export async function GET(req: Request) {
  await ensureSeed();
  const user = await getSessionUser();
  if (!user) return Response.json({ error: " " }, { status: 401 });
  const url = new URL(req.url);
  const category = url.searchParams.get("category");
  const parent = url.searchParams.get("parent");
  const where =
    category && parent !== null
      ? and(eq(options.category, category), eq(options.parent, parent))
      : category
        ? eq(options.category, category)
        : undefined;
  const rows = await db
    .select()
    .from(options)
    .where(where)
    .orderBy(asc(options.category), asc(options.value))
    .limit(5000);
  return Response.json({ rows });
}
/**      ( options)      . */
export async function POST(req: Request) {
  const user = await getSessionUser();
  if (!user) return Response.json({ error: " " }, { status: 401 });
  const canAdd = user.role === "admin" || user.permissions.includes("options");
  if (!canAdd) return Response.json({ error: " " }, { status: 403 });
  const body = await req.json().catch(() => ({}));
  const category = String(body.category ?? "").trim();
  const value = String(body.value ?? "").trim().slice(0, 200);
  const parent = String(body.parent ?? "").trim().slice(0, 200);
  if (!VALID.includes(category)) return Response.json({ error: " " }, { status: 400 });
  if (value.length < 2) return Response.json({ error: " " }, { status: 400 });
  if ((category === "city" || category === "region") && !parent) {
    return Response.json(
      { error: category === "city" ? " " : " " },
      { status: 400 },
    );
  }
  const exists = await db
    .select()
    .from(options)
    .where(and(eq(options.category, category), eq(options.value, value), eq(options.parent, parent)))
    .limit(1);

  if (exists.length) return Response.json({ row: exists[0], duplicate: true });
  const [row] = await db
    .insert(options)
    .values({ category, value, parent, createdBy: user.fullName })
    .returning();
  await db
    .insert(activityLogs)
    .values({ userId: user.id, userName: user.fullName, action: " ", detail: `${category}: ${value}` })
    .catch(() => undefined);
  return Response.json({ row });
}
export async function PATCH(req: Request) {
  const user = await getSessionUser();
  if (!user) return Response.json({ error: " " }, { status: 401 });
  const canEdit = user.role === "admin" || user.permissions.includes("optionsDelete");
  if (!canEdit) return Response.json({ error: " " }, { status: 403 });
  const b = await req.json().catch(() => ({}));
  const id = Number(b.id);
  const value = String(b.value ?? "").trim().slice(0, 200);
  if (!Number.isFinite(id) || !value) return Response.json({ error: " " }, { status: 400 });
  await db.update(options).set({ value }).where(eq(options.id, id));
  await db
    .insert(activityLogs)
    .values({ userId: user.id, userName: user.fullName, action: " ", detail: value })
    .catch(() => undefined);
  return Response.json({ ok: true });
}
export async function DELETE(req: Request) {
  const user = await getSessionUser();
  if (!user || (user.role !== "admin" && !user.permissions.includes("optionsDelete"))) {
    return Response.json({ error: " " }, { status: 401 });
  }
  const id = Number(new URL(req.url).searchParams.get("id"));
  if (!Number.isFinite(id)) return Response.json({ error: " " }, { status: 400 });
  await db.delete(options).where(eq(options.id, id));
  return Response.json({ ok: true });
}

```

@@ src/app/api/ping/route.ts typescript

```

/**
 * Keep-Alive endpoint — .
 *
 * :      Render (
 *      ).
 *
 * UptimeRobot      HTTP(s) :
 * URL      : https://<your-app>.onrender.com/ping
 * Interval :
 *
 * : /ping /api/ping
 */
export const dynamic = "force-dynamic";
export const revalidate = 0;
const startedAt = Date.now();
const headers = {
  "Content-Type": "text/plain; charset=utf-8",
  "Cache-Control": "no-store, no-cache, must-revalidate, max-age=0",
  "X-Robots-Tag": "noindex",
  "Access-Control-Allow-Origin": "*",
} as const;
export async function GET(req: Request) {
  //      : OK
  const wantsJson = new URL(req.url).searchParams.has("json");
  if (!wantsJson) return new Response("OK", { status: 200, headers });
  return new Response(
    JSON.stringify({
      status: "OK",
      uptimeMs: Date.now() - startedAt,
      timestamp: new Date().toISOString(),
    }),
    { status: 200, headers: { ...headers, "Content-Type": "application/json; charset=utf-8" } },
  );
}
export async function HEAD() {
  return new Response(null, { status: 200, headers });
}

@@ src/app/api/reports/route.ts typescript
import { db, dbRetry } from "@/db";
import { doctors, orders, pharmacies, trips } from "@/db/schema";
import { getSessionUser } from "@/lib/auth";
import { bonusKeyOf } from "@/lib/defaults";
import { getProducts } from "@/lib/settings-server";
import { eq, SQL } from "drizzle-orm";
export const dynamic = "force-dynamic";
export type MonthlyRow = {
  period: string; // 1405/03
  repName: string;
  pharmacies: number;
  doctors: number;
  orders: number;
  units: number;
  bonus: number;
  trips: number;
};
export async function GET(req: Request) {
  const user = await getSessionUser();
  if (!user) return Response.json({ error: " " }, { status: 401 });
  const url = new URL(req.url);
  const isAdmin = user.role === "admin" || user.role === "supervisor";
  const repFilter = url.searchParams.get("userId");
  const scopeId = isAdmin ? (repFilter ? Number(repFilter) : null) : user.id;
  const wP: SQL | undefined = scopeId ? eq(pharmacies.userId, scopeId) : undefined;
  const wD: SQL | undefined = scopeId ? eq(doctors.userId, scopeId) : undefined;
  const wO: SQL | undefined = scopeId ? eq(orders.userId, scopeId) : undefined;
  const wT: SQL | undefined = scopeId ? eq(trips.userId, scopeId) : undefined;
  const PRODUCTS = await getProducts();
  const [ph, dr, or, tr] = await dbRetry(() => Promise.all([
    db
      .select({ d: pharmacies.dateShamsi, rep: pharmacies.repName })
      .from(pharmacies)
      .where(wP)
      .limit(5000),
    db.select({ d: doctors.dateShamsi, rep: doctors.repName }).from(doctors).where(wD).limit(5000),
    db
      .select({ d: orders.dateShamsi, rep: orders.repName, items: orders.items })
      .from(orders)
      .where(wO)
      .limit(5000),
    db.select({ d: trips.dateShamsi, rep: trips.repName }).from(trips).where(wT).limit(5000),
  ]), "reports:load");
  const map = new Map<string, MonthlyRow>();
  const key = (d: string, rep: string) => `${d.slice(0, 7)}|${rep}`;
  const get = (d: string, rep: string) => {

```



```

const k = key(d, rep);
let row = map.get(k);
if (!row) {
  row = {
    period: d.slice(0, 7),
    repName: rep,
    pharmacies: 0,
    doctors: 0,
    orders: 0,
    units: 0,
    bonus: 0,
    trips: 0,
  };
  map.set(k, row);
}
return row;
};
for (const r of ph) get(r.d, r.rep).pharmacies++;
for (const r of dr) get(r.d, r.rep).doctors++;
for (const r of tr) get(r.d, r.rep).trips++;
for (const r of or) {
  const row = get(r.d, r.rep);
  row.orders++;
  for (const p of PRODUCTS) {
    row.units += Number(r.items?.[p.key] ?? 0);
    row.bonus += Number(r.items?.[bonusKeyOf(p.key)] ?? 0);
  }
}
const rows = [...map.values()].sort((a, b) =>
  a.period === b.period ? a.repName.localeCompare(b.repName) : b.period.localeCompare(a.period),
);
// :
const productTotals: Record<string, Record<string, number>> = {};
const productByRep: {
  period: string;
  repName: string;
  items: Record<string, number>;
  bonuses: Record<string, number>;
}[] = [];
const idx = new Map<string, number>();
for (const r of or) {
  const per = r.d.slice(0, 7);
  productTotals[per] ??= {};
  const k = `${per}|${r.rep}`;
  let i = idx.get(k);
  if (i === undefined) {
    i = productByRep.length;
    idx.set(k, i);
    productByRep.push({ period: per, repName: r.rep, items: {}, bonuses: {} });
  }
  for (const p of PRODUCTS) {
    const q = Number(r.items?.[p.key] ?? 0);
    const b = Number(r.items?.[bonusKeyOf(p.key)] ?? 0);
    productTotals[per][p.key] = (productTotals[per][p.key] ?? 0) + q;
    productByRep[i].items[p.key] = (productByRep[i].items[p.key] ?? 0) + q;
    productByRep[i].bonuses[p.key] = (productByRep[i].bonuses[p.key] ?? 0) + b;
  }
}
productByRep.sort((a, b) =>
  a.period === b.period ? a.repName.localeCompare(b.repName) : b.period.localeCompare(a.period),
);
return Response.json({ rows, productTotals, productByRep });
}

```

```

@@ src/app/api/roles/route.ts typescript
import { db } from "@/db";
import { roles, users } from "@/db/schema";
import { getSessionUser } from "@/lib/auth";
import { ensureSeed } from "@/lib/bootstrap";
import { asc, eq } from "drizzle-orm";
export const dynamic = "force-dynamic";
async function guard(write = false) {
  const user = await getSessionUser();
  if (!user) return null;
  if (!write) return user;
  if (user.role === "admin" || user.permissions.includes("users")) return user;
  return null;
}
export async function GET() {
  await ensureSeed();
  const user = await guard(false);
  if (!user) return Response.json({ error: " " }, { status: 401 });
  const rows = await db.select().from(roles).orderBy(asc(roles.id));
  return Response.json({ rows });
}

```

```

}
export async function POST(req: Request) {
  const user = await guard(true);
  if (!user) return Response.json({ error: " " }, { status: 401 });
  const b = await req.json().catch(() => ({}));
  const label = String(b.label ?? "").trim();
  if (!label) return Response.json({ error: " " }, { status: 400 });
  const key =
    String(b.key ?? "").
      .trim()
      .toLowerCase()
      .replace(/^[a-z0-9_-]/g, "") || `role_${Date.now().toString(36)}`;
  const base = ["admin", "supervisor", "rep"].includes(b.base) ? b.base : "rep";
  const dup = await db.select().from(roles).where(eq(roles.key, key)).limit(1);
  if (dup.length) return Response.json({ error: " " }, { status: 400 });
  const [row] = await db
    .insert(roles)
    .values({ key, label, base, permissions: Array.isArray(b.permissions) ? b.permissions : [] })
    .returning();
  return Response.json({ row });
}
export async function PATCH(req: Request) {
  const user = await guard(true);
  if (!user) return Response.json({ error: " " }, { status: 401 });
  const b = await req.json().catch(() => ({}));
  const id = Number(b.id);
  if (!Number.isFinite(id)) return Response.json({ error: " " }, { status: 400 });
  const patch: Record<string, unknown> = {};
  if (typeof b.label === "string" && b.label.trim()) patch.label = b.label.trim();
  if (typeof b.base === "string" && ["admin", "supervisor", "rep"].includes(b.base)) patch.base = b.base;
  if (Array.isArray(b.permissions)) patch.permissions = b.permissions;
  if (Object.keys(patch).length === 0) return Response.json({ error: " " }, { status: 400 });
  await db.update(roles).set(patch).where(eq(roles.id, id));
  return Response.json({ ok: true });
}
export async function DELETE(req: Request) {
  const user = await guard(true);
  if (!user) return Response.json({ error: " " }, { status: 401 });
  const id = Number(new URL(req.url).searchParams.get("id"));
  if (!Number.isFinite(id)) return Response.json({ error: " " }, { status: 400 });
  const rows = await db.select().from(roles).where(eq(roles.id, id)).limit(1);
  const r = rows[0];
  if (!r) return Response.json({ error: " " }, { status: 404 });
  if (r.builtin) return Response.json({ error: " " }, { status: 400 });
  const inUse = await db.select({ id: users.id }).from(users).where(eq(users.role, r.key)).limit(1);
  if (inUse.length) return Response.json({ error: " " }, { status: 400 });
  await db.delete(roles).where(eq(roles.id, id));
  return Response.json({ ok: true });
}
}

@@ src/app/api/settings/route.ts typescript
import { db } from "@db";
import { settings } from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { ensureSeed } from "@lib/bootstrap";
import { DEFAULT_COLUMNS, DEFAULT_FORM_FIELDS, DEFAULT_PRODUCTS } from "@lib/defaults";
import { eq } from "drizzle-orm";
export const dynamic = "force-dynamic";
const FALLBACK: Record<string, unknown> = {
  products: DEFAULT_PRODUCTS,
  "columns.pharmacies": DEFAULT_COLUMNS.pharmacies,
  "columns.doctors": DEFAULT_COLUMNS.doctors,
  "columns.orders": DEFAULT_COLUMNS.orders,
  "fields.pharmacies": DEFAULT_FORM_FIELDS.pharmacies,
  "fields.doctors": DEFAULT_FORM_FIELDS.doctors,
  "fields.orders": DEFAULT_FORM_FIELDS.orders,
};
export async function GET(req: Request) {
  await ensureSeed();
  const user = await getSessionUser();
  if (!user) return Response.json({ error: " " }, { status: 401 });
  const key = new URL(req.url).searchParams.get("key");
  const rows = await db.select().from(settings);
  const map: Record<string, unknown> = { ...FALLBACK };
  for (const r of rows) map[r.key] = r.value;
  if (user.role !== "admin") delete map["sms"];
  if (key) return Response.json({ key, value: map[key] ?? null });
  return Response.json({ values: map });
}
export async function PUT(req: Request) {
  const user = await getSessionUser();
  const b0 = user ? null : null;
  void b0;
  if (!user) return Response.json({ error: " " }, { status: 401 });

```

```

const allowed =
  user.role === "admin" ||
  user.permissions.includes("columns") ||
  user.permissions.includes("messengers") ||
  user.permissions.includes("users");
if (!allowed) return Response.json({ error: " " }, { status: 401 });
const b = await req.json().catch(() => ({}));
const key = String(b.key ?? "");
if (!key || b.value === undefined) return Response.json({ error: " " }, { status: 400 });
const existing = await db.select().from(settings).where(eq(settings.key, key)).limit(1);
if (existing.length) {
  await db.update(settings).set({ value: b.value, updatedAt: new Date() }).where(eq(settings.key, key));
} else {
  await db.insert(settings).values({ key, value: b.value });
}
return Response.json({ ok: true });
}

```

```

@@ src/app/api/source/route.ts typescript
import { getSessionUser } from "@lib/auth";
import { createZip, type ZipEntry } from "@lib/zip";
export const dynamic = "force-dynamic";
export const maxDuration = 60;

```

```

/**
 * ZIP ( ).
 *
 * :
 * GitHub / GitLab .
 *
 * GET /api/source →
 * GET /api/source?list=1 → (JSON)
 *
 * : «»
 * . import Turbopack
 *
 * «Encountered unexpected file in NFT list».
 */

```

```

/** */
const SKIP_DIRS = new Set([
  "node_modules",
  ".next",
  ".git",
  ".turbo",
  "dist",
  "build",
  ".vercel",
  ".cache",
  "coverage",
]);
/** */

```

```

const SKIP_FILES = new Set([".env", ".env.local", ".env.production", "package-lock.json"]);
const MAX_FILE_BYTES = 3 * 1024 * 1024;
type FsApi = {
  readFile: (p: string) => Promise<Buffer>;
  readdir: (p: string) => Promise<string[]>;
  stat: (p: string) => Promise<{ isDirectory: () => boolean; size: number }>;
  join: (...p: string[]) => string;
  root: string;
};

```

```

/** */
async function loadFs(): Promise<FsApi> {
  const fsp = await import("node:fs/promises");
  const nodePath = await import("node:path");
  return {
    readFile: (p: string) => fsp.readFile(p) as unknown as Promise<Buffer>,
    readdir: (p: string) => fsp.readdir(p),
    stat: (p: string) => fsp.stat(p),
    join: (...p: string[]) => nodePath.join(...p),
    root: process.cwd(),
  };
}

```

```

}
async function walk(fs: FsApi, dir: string, base = ""): Promise<string[]> {
  const out: string[] = [];
  let items: string[] = [];
  try {
    items = await fs.readdir(dir);
  } catch {
    return out;
  }
  for (const name of items) {
    if (name === ".git" || name === ".next") continue;
    const rel = base ? `${base}/${name}` : name;
    const full = fs.join(dir, name);
    let st;
    try {

```

```

        st = await fs.stat(full);
    } catch {
        continue;
    }
    if (st.isDirectory()) {
        if (SKIP_DIRS.has(name)) continue;
        out.push(...(await walk(fs, full, rel)));
    } else {
        if (SKIP_FILES.has(name)) continue;
        if (st.size > MAX_FILE_BYTES) continue;

        out.push(rel);
    }
}
return out;
}
export async function GET(req: Request) {
    const user = await getSessionUser();
    if (!user || user.role !== "admin") {
        return Response.json({ error: " — " }, { status: 401 });
    }
    const fs = await loadFs();
    const files = (await walk(fs, fs.root)).sort();
    if (new URL(req.url).searchParams.get("list") === "1") {
        return Response.json({ count: files.length, files }, { headers: { "Cache-Control": "no-store" } });
    }
    const entries: ZipEntry[] = [];
    for (const rel of files) {
        try {
            entries.push({ path: `namayandeelmi-javad/${rel}`, content: await fs.readFile(fs.join(fs.root, rel)) });
        } catch {
            /* — */
        }
    }
    entries.push({
        path: "namayandeelmi-javad/HOW-TO-UPDATE.md",
        content: `#
            (${entries.length}) .
    `
    });
    ##
    1. ZIP
    ( — node_modules .next ).
    2. :
    \`\`\`bash
    npm install
    npm run build      #
    git add -A
    git status
    git commit -m " "
    git push origin main
    \`\`\`
    3. GitLab :
    \`\`\`bash
    git remote add gitlab https://gitlab.com/USER/REPO.git  #
    git push gitlab main
    \`\`\`
    ##
    \`.env\` ( ).
    \`.env.ndcohub.example\` .
    : ${new Date().toISOString()}
    ,
    });
    const zip = createZip(entries);
    const stamp = new Date().toISOString().slice(0, 16).replace(/[:T]/g, "-");
    return new Response(new Uint8Array(zip), {
        headers: {
            "Content-Type": "application/zip",
            "Content-Length": String(zip.length),
            "Content-Disposition": `attachment; filename="namayandeelmi-source-${stamp}.zip"`,
            "Cache-Control": "no-store",
        },
    });
}

@@ src/app/api/targets/route.ts typescript
import { db, dbRetry, dbRetrySafe } from "@db";
import { orders, targets, users } from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { ensureSeed } from "@lib/bootstrap";
import { bonusKeyOf } from "@lib/defaults";

import { getProducts } from "@lib/settings-server";
import { todayJalali } from "@lib/jalali";
import { and, eq, SQL } from "drizzle-orm";
export const dynamic = "force-dynamic";
export type TargetProgress = {

```

```

productKey: string;
productLabel: string;
quantity: number;
priceDistributor: number;
pricePharmacy: number;
/** */
sold: number;
/** */
bonus: number;
remaining: number;
percent: number;
valueDistributor: number;
valuePharmacy: number;
soldValueDistributor: number;
soldValuePharmacy: number;
};
const currentPeriod = () => todayJalali().slice(0, 7);
/**
 * GET /api/targets?userId=&period=
 * :
 */
export async function GET(req: Request) {
  await ensureSeed();
  const user = await getSessionUser();
  if (!user) return Response.json({ error: " " }, { status: 401 });
  const url = new URL(req.url);
  const isManager = user.role === "admin" || user.role === "supervisor";
  const scopeId = isManager ? Number(url.searchParams.get("userId")) || user.id : user.id;
  const period = url.searchParams.get("period") ?? currentPeriod();
  const allPeriods = url.searchParams.get("all") === "1";
  const products = await getProducts();
  // ( + )
  const rows = await dbRetrySafe(
    () => db.select().from(targets).where(eq(targets.userId, scopeId)),
    [],
    "targets:list",
  );
  const forPeriod = rows.filter((t) => allPeriods || t.period === period || t.period === "");
  //
  const orderRows = await dbRetrySafe(
    () => db.select({ d: orders.dateShamsi, items: orders.items }).from(orders).where(eq(orders.userId, scopeId)),
    [],
    "targets:orders",
  );
  const scoped = allPeriods ? orderRows : orderRows.filter((o) => o.d.startsWith(period));
  const sold: Record<string, number> = {};
  const bonusMap: Record<string, number> = {};
  for (const o of scoped) {
    for (const p of products) {
      sold[p.key] = (sold[p.key] ?? 0) + Number(o.items?.[p.key] ?? 0);
      bonusMap[p.key] = (bonusMap[p.key] ?? 0) + Number(o.items?.[bonusKeyOf(p.key)] ?? 0);
    }
  }
  const progress: TargetProgress[] = products.map((p) => {
    const t = forPeriod.find((x) => x.productKey === p.key);
    const quantity = t?.quantity ?? 0;
    const pd = t?.priceDistributor ?? 0;
    const pp = t?.pricePharmacy ?? 0;
    const s = sold[p.key] ?? 0;
    return {
      productKey: p.key,
      productLabel: p.label,
      quantity,
      priceDistributor: pd,
      pricePharmacy: pp,
      sold: s,
      bonus: bonusMap[p.key] ?? 0,
      remaining: Math.max(0, quantity - s),
      percent: quantity > 0 ? Math.min(999, Math.round((s / quantity) * 100)) : 0,
      valueDistributor: quantity * pd,
      valuePharmacy: quantity * pp,
      soldValueDistributor: s * pd,
      soldValuePharmacy: s * pp,
    };
  });
  const totals = progress.reduce(
    (a, r) => ({
      quantity: a.quantity + r.quantity,
      sold: a.sold + r.sold,
      bonus: a.bonus + r.bonus,
      remaining: a.remaining + r.remaining,
      valueDistributor: a.valueDistributor + r.valueDistributor,
      valuePharmacy: a.valuePharmacy + r.valuePharmacy,
      soldValueDistributor: a.soldValueDistributor + r.soldValueDistributor,

```

```

        soldValuePharmacy: a.soldValuePharmacy + r.soldValuePharmacy,
    }},
    {
        quantity: 0,
        sold: 0,
        bonus: 0,
        remaining: 0,
        valueDistributor: 0,
        valuePharmacy: 0,
        soldValueDistributor: 0,
        soldValuePharmacy: 0,
    },
);
return Response.json({
    period,
    userId: scopeId,
    progress,
    totals,
    percent: totals.quantity > 0 ? Math.round((totals.sold / totals.quantity) * 100) : 0,
});
}
/**
 * PUT /api/targets — /
 * { userId, period, items: [{ productKey, quantity, priceDistributor, pricePharmacy }] }
 */
export async function PUT(req: Request) {
    const user = await getSessionUser();
    if (!user || (user.role !== "admin" && !user.permissions.includes("users"))) {
        return Response.json({ error: " " }, { status: 401 });
    }
    const b = await req.json().catch(() => ({}));
    const userId = Number(b.userId);
    const period = String(b.period ?? "").slice(0, 7);
    if (!Number.isFinite(userId)) return Response.json({ error: " " }, { status: 400 });
    const target = (await dbRetry(() => db.select().from(users).where(eq(users.id, userId)).limit(1, "targets:user")))[0];
    if (!target) return Response.json({ error: " " }, { status: 404 });
    const products = await getProducts();
    const items = Array.isArray(b.items) ? b.items : [];
    for (const it of items) {
        const key = String(it.productKey ?? "");
        const p = products.find((x) => x.key === key);
        if (!p) continue;
        const values = {
            userId,
            repName: target.fullName,
            period,
            productKey: key,
            productLabel: p.label,
            quantity: Math.max(0, Number(it.quantity) || 0),
            priceDistributor: Math.max(0, Number(it.priceDistributor) || 0),
            pricePharmacy: Math.max(0, Number(it.pricePharmacy) || 0),
            updatedAt: new Date(),
        };
        const existing = await dbRetrySafe(
            () =>
                db
                    .select()
                    .from(targets)
                    .where(and(eq(targets.userId, userId), eq(targets.period, period), eq(targets.productKey, key)))
                    .limit(1),
            [],
            "targets:find",
        );
        if (existing.length) {
            await dbRetry(() => db.update(targets).set(values).where(eq(targets.id, existing[0].id)), "targets:update");
        } else {
            await dbRetry(() => db.insert(targets).values(values), "targets:insert");
        }
    }
    return Response.json({ ok: true, count: items.length });
}
export async function DELETE(req: Request) {
    const user = await getSessionUser();
    if (!user || (user.role !== "admin" && !user.permissions.includes("users"))) {
        return Response.json({ error: " " }, { status: 401 });
    }
    const url = new URL(req.url);
    const userId = Number(url.searchParams.get("userId"));
    const period = url.searchParams.get("period") ?? "";
    if (!Number.isFinite(userId)) return Response.json({ error: " " }, { status: 400 });
    const where: SQL | undefined = and(eq(targets.userId, userId), eq(targets.period, period));
    await dbRetry(() => db.delete(targets).where(where), "targets:delete");
    return Response.json({ ok: true });
}
}

```

```

@@ src/app/api/trips/route.ts typescript
import { db } from "@/db";
import { activityLogs, tripPoints, trips } from "@/db/schema";
import { getSessionUser } from "@/lib/auth";
import { todayJalali } from "@/lib/jalali";
import { and, count, desc, eq, sql } from "drizzle-orm";
export const dynamic = "force-dynamic";
export async function GET(req: Request) {
  const user = await getSessionUser();
  if (!user) return Response.json({ error: " " }, { status: 401 });
  const onlyActive = new URL(req.url).searchParams.get("active") === "1";
  const base = db
    .select({
      id: trips.id,
      userId: trips.userId,
      repName: trips.repName,
      dateShamsi: trips.dateShamsi,
      status: trips.status,
      distanceM: trips.distanceM,
      stopSeconds: trips.stopSeconds,
      startedAt: trips.startedAt,
      endedAt: trips.endedAt,
    })
    .from(trips);
  const isManager = user.role === "admin" || user.role === "supervisor";
  const list = isManager
    ? await (onlyActive ? base.where(eq(trips.status, "active")) : base).orderBy(desc(trips.id)).limit(300)
    : await base.where(eq(trips.userId, user.id)).orderBy(desc(trips.id)).limit(100);
  // ( )
  const counts = await db
    .select({ tripId: tripPoints.tripId, c: count() })
    .from(tripPoints)
    .groupBy(tripPoints.tripId);
  const cmap = new Map(counts.map((c) => [c.tripId, Number(c.c)]));
  void sql;
  const rows = list.map((t) => ({ ...t, points: cmap.get(t.id) ?? 0 }));
  return Response.json({ rows });
}
export async function POST(req: Request) {
  const user = await getSessionUser();
  if (!user) return Response.json({ error: " " }, { status: 401 });
  const b = await req.json().catch(() => ({}));
  // close any dangling active trips of this user
  await db
    .update(trips)
    .set({ status: "ended", endedAt: new Date() })
    .where(and(eq(trips.userId, user.id), eq(trips.status, "active")));
  const [row] = await db
    .insert(trips)
    .values({
      userId: user.id,
      repName: user.fullName,
      dateShamsi: todayJalali(),
      status: "active",
    })
    .returning();
  if (Number.isFinite(Number(b.lat)) && Number.isFinite(Number(b.lng))) {
    await db.insert(tripPoints).values({
      tripId: row.id,
      lat: Number(b.lat),
      lng: Number(b.lng),
      accuracy: Number(b.accuracy) || null,
      kind: "start",
      note: " ",
    });
  }
  await db.insert(activityLogs).values({
    userId: user.id,
    userName: user.fullName,
    action: " ",
    detail: `#${row.id}`,
  });
  return Response.json({ trip: row });
}
export async function PATCH(req: Request) {
  const user = await getSessionUser();
  if (!user) return Response.json({ error: " " }, { status: 401 });
  const b = await req.json().catch(() => ({}));
  const id = Number(b.id);
  if (!Number.isFinite(id)) return Response.json({ error: " " }, { status: 400 });
  const rows = await db.select().from(trips).where(eq(trips.id, id)).limit(1);
  const trip = rows[0];
  if (!trip || (user.role !== "admin" && trip.userId !== user.id)) {

```

```

    return Response.json({ error: " " }, { status: 404 });
  }
  if (b.action === "end") {
    if (Number.isFinite(Number(b.lat)) && Number.isFinite(Number(b.lng))) {
      await db.insert(tripPoints).values({
        tripId: id,
        lat: Number(b.lat),
        lng: Number(b.lng),
        kind: "end",
        note: " ",
      });
    }
    await db.update(trips).set({ status: "ended", endedAt: new Date() }).where(eq(trips.id, id));
    await db.insert(activityLogs).values({
      userId: user.id,
      userName: user.fullName,
      action: " ",
      detail: `#${id}`,
    });
  }
  return Response.json({ ok: true });
}

```

```

@@ src/app/api/users/route.ts typescript
import { db } from "@db";
import { users } from "@db/schema";
import { getSessionUser, hashPassword } from "@lib/auth";
import { ALL_PERMISSION_KEYS } from "@lib/defaults";
import { asc, eq } from "drizzle-orm";
export const dynamic = "force-dynamic";
async function adminGuard() {
  const user = await getSessionUser();
  if (!user) return null;
  if (user.role === "admin" || user.permissions.includes("users")) return user;
  return null;
}
export async function GET() {
  const admin = await adminGuard();
  if (!admin) return Response.json({ error: " " }, { status: 401 });
  const rows = await db
    .select({
      id: users.id,
      username: users.username,
      fullName: users.fullName,
      phone: users.phone,
      role: users.role,
      active: users.active,
      permissions: users.permissions,
      passwordPlain: users.passwordPlain,
      requirePhone: users.requirePhone,
      deviceId: users.deviceId,
      deviceInfo: users.deviceInfo,
      deviceBoundAt: users.deviceBoundAt,
      simMode: users.simMode,
      lastSeenAt: users.lastSeenAt,
      createdAt: users.createdAt,
    })
    .from(users)
    .orderBy(asc(users.id));
  return Response.json({ rows });
}
export async function POST(req: Request) {
  const admin = await adminGuard();
  if (!admin) return Response.json({ error: " " }, { status: 401 });
  const b = await req.json().catch(() => ({}));
  const username = String(b.username ?? "").trim().toLowerCase();
  const password = String(b.password ?? "").trim();
  const fullName = String(b.fullName ?? "").trim();
  if (!username || !password || !fullName) {
    return Response.json({ error: " " }, { status: 400 });
  }
  const dup = await db.select().from(users).where(eq(users.username, username)).limit(1);
  if (dup.length) return Response.json({ error: " " }, { status: 400 });
  const [row] = await db
    .insert(users)
    .values({
      username,
      passwordHash: hashPassword(password),
      passwordPlain: password,
      fullName,
      phone: String(b.phone ?? "").trim(),
      role: String(b.role ?? "rep").slice(0, 40),
      active: b.active !== false,
      requirePhone: b.requirePhone !== false,
    })

```



```

        simMode: ["off", "phone", "device", "otp"].includes(b.simMode) ? b.simMode : "device",
        permissions: Array.isArray(b.permissions) && b.permissions.length ? b.permissions : ALL_PERMISSION_KEYS,
    })
    .returning();
    return Response.json({ row: { ...row, passwordHash: undefined } });
}
export async function PATCH(req: Request) {
    const admin = await adminGuard();
    if (!admin) return Response.json({ error: " " }, { status: 401 });
    const b = await req.json().catch(() => ({}));
    const id = Number(b.id);
    if (!Number.isFinite(id)) return Response.json({ error: " " }, { status: 400 });
    const patch: Record<string, unknown> = {};
    if (typeof b.fullName === "string" && b.fullName.trim()) patch.fullName = b.fullName.trim();
    if (typeof b.phone === "string") patch.phone = b.phone.trim();
    if (typeof b.username === "string" && b.username.trim()) patch.username = b.username.trim().toLowerCase();
    if (typeof b.role === "string" && b.role.trim()) patch.role = b.role.trim().slice(0, 40);
    if (typeof b.active === "boolean") patch.active = b.active;
    if (typeof b.requirePhone === "boolean") patch.requirePhone = b.requirePhone;
    if (typeof b.simMode === "string" && ["off", "phone", "device", "otp"].includes(b.simMode)) {
        patch.simMode = b.simMode;
    }
    // ( )
    if (b.releaseDevice === true) {
        patch.deviceId = "";
        patch.deviceInfo = "";
        patch.deviceBoundAt = null;
    }
    if (Array.isArray(b.permissions)) patch.permissions = b.permissions;
    if (typeof b.password === "string" && b.password.length >= 4) {
        patch.passwordHash = hashPassword(b.password);
        patch.passwordPlain = b.password;
    }
    if (Object.keys(patch).length === 0) return Response.json({ error: " " }, { status: 400 });
    await db.update(users).set(patch).where(eq(users.id, id));
    return Response.json({ ok: true });
}
export async function DELETE(req: Request) {
    const admin = await adminGuard();
    if (!admin) return Response.json({ error: " " }, { status: 401 });
    const id = Number(new URL(req.url).searchParams.get("id"));
    if (!Number.isFinite(id)) return Response.json({ error: " " }, { status: 400 });
    if (id === admin.id) return Response.json({ error: " " }, { status: 400 });
    await db.delete(users).where(eq(users.id, id));
    return Response.json({ ok: true });
}

@@ src/app/icons/[file]/route.ts typescript
import { pngResponse, renderIcon } from "@lib/pwa-image";
export const dynamic = "force-static";
export const revalidate = false;
/**
 * : public/icons
 * ( ) PNG
 *
 */
export async function GET(_req: Request, ctx: { params: Promise<{ file: string }> }) {
    const { file } = await ctx.params;
    const m = /^(icon|maskable)-(\d{2,4})\.png$/ .exec(file);
    if (!m) return new Response("not found", { status: 404 });
    const size = Math.min(1024, Math.max(16, Number(m[2])));
    const bytes = await renderIcon(size, m[1] === "maskable");
    if (!bytes) return new Response("not available", { status: 404 });
    return pngResponse(bytes);
}

@@ src/app/panel/doctors/page.tsx typescript
import RecordScreen from "@components/RecordScreen";
export default function Page() {
    return <RecordScreen type="doctors" />;
}

@@ src/app/panel/home/page.tsx typescript
import HomeScreen from "@components/screens/HomeScreen";
export default function Page() {
    return <HomeScreen />;
}

@@ src/app/panel/leaves/page.tsx typescript
import LeaveScreen from "@components/screens/LeaveScreen";
export default function Page() {
    return <LeaveScreen />;
}

```

```

@@ src/app/panel/map/page.tsx typescript
import MapExplorer from "@components/screens/MapExplorer";
export default function Page() {
  return <MapExplorer />;
}

@@ src/app/panel/notifications/page.tsx typescript
import NotificationScreen from "@components/screens/NotificationScreen";
export default function Page() {
  return <NotificationScreen />;
}

@@ src/app/panel/options/page.tsx typescript
import OptionsScreen from "@components/screens/OptionsScreen";
export default function Page() {
  return <OptionsScreen />;
}

@@ src/app/panel/orders/page.tsx typescript
import RecordScreen from "@components/RecordScreen";
export default function Page() {
  return <RecordScreen type="orders" />;
}

@@ src/app/panel/pharmacies/page.tsx typescript
import RecordScreen from "@components/RecordScreen";
export default function Page() {
  return <RecordScreen type="pharmacies" />;
}

@@ src/app/panel/reports/page.tsx typescript
import ReportScreen from "@components/screens/ReportScreen";
export default function Page() {
  return <ReportScreen />;
}

@@ src/app/panel/trip/page.tsx typescript
"use client";
import { useCallback, useEffect, useRef, useState } from "react";
import dynamic from "next/dynamic";
import { Alert, Badge, Button, Card, SectionTitle } from "@components/ui";
import { useConfirm } from "@components/Confirm";
import { tehranTime, toPersianDigits, todayJalali } from "@lib/jalali";
import {
  bufferedCount,
  createTracker,

  flushPoints,
  formatDistance,
  formatDuration,
  pushLive,
  type TrackPoint,
  type TrackerState,
} from "@lib/tracker";
const MapBox = dynamic(() => import("@components/MapBox"), { ssr: false });
const LS_TRIP = "sek_trip_id";
const LS_STATE = "sek_trip_state";
type Place = { lat: number; lng: number; label: string; color: string };
export default function TripPage() {
  const [tripId, setTripId] = useState<number | null>(null);
  const [active, setActive] = useState(false);
  const [st, setSt] = useState<TrackerState>({
    lastPoint: null,
    distanceM: 0,
    stopSeconds: 0,
    gpsOn: true,
    pending: 0,
    points: [],
  });
  const [startedAt, setStartedAt] = useState("");
  const [msg, setMsg] = useState<{ kind: "info" | "error" | "success"; text: string } | null>(null);
  const [places, setPlaces] = useState<Place[]>([]);
  const trackerRef = useRef<ReturnType<typeof createTracker> | null>(null);
  const confirm = useConfirm();
  /* ----- */
  const loadPlaces = useCallback(async () => {
    const out: Place[] = [];
    const get = async (url: string) => {
      const r = await fetch(url, { cache: "no-store" }).catch(() => null);
      return r?.ok ? ((await r.json()).rows ?? []) : [];
    };
    const [ph, dr, hm] = await Promise.all([
      get("/api/records/pharmacies?limit=300"),
      get("/api/records/doctors?limit=300"),
      get("/api/homes"),
    ]);
  });

```

```

    });
    for (const p of ph) if (p.lat && p.lng) out.push({ lat: p.lat, lng: p.lng, label: ` ${p.name}`, color: "#0f766e" }
);
    for (const d of dr) if (d.lat && d.lng) out.push({ lat: d.lat, lng: d.lng, label: ` ${d.name}`, color: "#0369a1" }
);
    for (const h of hm) if (h.lat && h.lng) out.push({ lat: h.lat, lng: h.lng, label: ` ${h.title}`, color: "#7c3aed"
});
    setPlaces(out);
  }, []);
  /* ----- */
  useEffect(() => {
    loadPlaces();
    const id = Number(localStorage.getItem(LS_TRIP));
    if (id) {
      setTripId(id);
      setActive(true);
      try {
        const saved = JSON.parse(localStorage.getItem(LS_STATE) ?? "{}");
        if (saved.startedAt) setStartedAt(saved.startedAt);
      } catch {
        /* ignore */
      }
    }
    bufferedCount().then((n) => setSt((s) => ({ ...s, pending: n })));
    //
    const onOnline = () => void flushPoints();
    window.addEventListener("online", onOnline);
    return () => window.removeEventListener("online", onOnline);
  }, [loadPlaces]);
  /* ----- */
  useEffect(() => {
    if (!active || !tripId) return;
    const tracker = createTracker({ tripId, onUpdate: setSt });
    trackerRef.current = tracker;
    const ok = tracker.start();
    if (!ok) setMsg({ kind: "error", text: " " });
    return () => {
      void tracker.stop(" ");
      trackerRef.current = null;
    };
  }, [active, tripId]);
  const currentPosition = () =>
    new Promise<{ lat: number; lng: number; accuracy?: number } | null>((resolve) => {
      if (!navigator.geolocation) return resolve(null);
      navigator.geolocation.getCurrentPosition(
        (p) => resolve({ lat: p.coords.latitude, lng: p.coords.longitude, accuracy: p.coords.accuracy }),
        () => resolve(null),
        { enableHighAccuracy: true, timeout: 15000, maximumAge: 30000 },
      );
    });
  const startTrip = async () => {
    if (!(await confirm({ title: " ", message: " ", confirmText: "" }))) return;
    setMsg({ kind: "info", text: " ..." });
    const pos = await currentPosition();
    const res = await fetch("/api/trips", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(pos ?? {}),
    }).catch(() => null);
    //
    let id: number | null = null;
    if (res?.ok) {
      const d = await res.json();
      id = d.trip?.id ?? null;
    }
    if (!id) {
      setMsg({
        kind: "info",
        text: " .",
      });
      id = -Date.now(); //
    } else {
      setMsg({ kind: "success", text: " — ( ) });
    }
    localStorage.setItem(LS_TRIP, String(id));
    localStorage.setItem(LS_STATE, JSON.stringify({ startedAt: tehranTime(new Date()) }));
    setStartedAt(tehranTime(new Date()));
    setTripId(id);
    setActive(true);
    if (pos) void pushLive({ ...pos, gpsOn: true, tripId: id });
  };
  const markPause = async () => {
    if (!(await confirm({ title: " ", message: " ", confirmText: "" }))) return;
    await trackerRef.current?.markPause(` — ${tehranTime(new Date())});
  };

```

```

    setMsg({ kind: "info", text: `    ${tehranTime(new Date())} ` });
};
const endTrip = async () => {
    if (
        !(await confirm({
            title: " ",
            message: ` : ${formatDistance(st.distanceM)}\n: ${formatDuration(st.stopSeconds)}\n\n
            ,
            confirmText: " ",
            danger: true,
        })))
    )
        return;
    await trackerRef.current?.stop(" ");
    const pos = await currentPosition();
    if (tripId && tripId > 0) {
        await fetch("/api/trips", {
            method: "PATCH",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ id: tripId, action: "end", ...(pos ?? {}) }),
        }).catch(() => null);
    }
    localStorage.removeItem(LS_TRIP);
    localStorage.removeItem(LS_STATE);
    setActive(false);
    setTripId(null);
    setSt({ lastPoint: null, distanceM: 0, stopSeconds: 0, gpsOn: true, pending: 0, points: [] });
    setMsg({ kind: "success", text: " " });
};
const pauses = st.points.filter((p: TrackPoint) => p.kind === "pause");
const path = st.points.map((p) => ({ lat: p.lat, lng: p.lng }));

return (
    <div className="space-y-4">
    <SectionTitle icon=""> — {toPersianDigits(todayJalali())}</SectionTitle>
    {msg ? <Alert kind={msg.kind}>{msg.text}</Alert> : null}
    <Card>
        <div className="flex flex-wrap items-center gap-2">
            {!active ? (
                <Button onClick={startTrip} className="flex-1 sm:flex-none">

                </Button>
            ) : (
                <>
                <Button variant="soft" onClick={markPause}>

                </Button>
                <Button variant="danger" onClick={endTrip}>

                </Button>
            </>
            )}
            <div className="flex-1" />
            <Badge tone={active ? "green" : "slate"}>{active ? " " : ""}</Badge>
            <Badge tone={st.gpsOn ? "green" : "amber"}>{st.gpsOn ? "GPS" : "GPS —"}</Badge>
            {st.pending > 0 ? <Badge tone="amber">{toPersianDigits(st.pending)} </Badge> : null}
        </div>
        {active ? (
            <div className="mt-3 grid grid-cols-2 gap-2 sm:grid-cols-4">
                {[
                    [" ", formatDistance(st.distanceM), "bg-teal-50 text-teal-700"],
                    [" ", formatDuration(st.stopSeconds), "bg-amber-50 text-amber-700"],
                    [" ", toPersianDigits(st.points.length), "bg-slate-50 text-slate-700"],
                    [" ", startedAt || "—", "bg-sky-50 text-sky-700"],
                ]}
                .map(([l, v, c]) => (
                    <div key={l} className={`rounded-xl p-2.5 text-center ring-1 ring-slate-200 ${c}`}>
                        <div className="text-sm font-black">{v}</div>
                        <div className="text-[10px] font-bold opacity-70">{l}</div>
                    </div>
                )))
            </div>
        ) : null}
        {!st.gpsOn && active ? (
            <p className="mt-2 rounded-lg bg-amber-50 px-3 py-2 text-[11px] text-amber-800">
                GPS —
            </p>
        ) : null}
    </Card>
    <Card>
        <div className="mb-2 flex flex-wrap items-center justify-between gap-2">
            <h3 className="text-sm font-bold text-slate-700">
                {active ? " " : " "}
            </h3>
            <span className="text-[11px] text-slate-500">
                . . .
            </span>
        </div>
    </Card>

```

```

    </span>
  </div>
  <MapBox
    height={380}
    follow={active}
    path={path}
    points={[
      ...places,
      ...st.points
        .filter((p) => p.kind !== "move")
        .map((p) => ({
          lat: p.lat,
          lng: p.lng,
          label: p.note || (p.kind === "start" ? " " : ""),
          color: p.kind === "pause" ? "#f59e0b" : p.kind === "end" ? "#e11d48" : "#16a34a",
        })),
    ]}
  />
  {pauses.length ? (
    <ul className="mt-3 space-y-1 text-xs text-slate-600">
      {pauses.map((p, i) => (
        <li key={i} className="rounded-lg bg-amber-50 px-3 py-1.5">
          {p.note} — {tehranTime(p.recordedAt)}
        </li>
      ))}
    </ul>
  ) : null}
</Card>
{!active ? (
  <Alert kind="info">
    « » . GPS
  </Alert>
) : null}
</div>
);
}

```

```

@@ src/app/screenshots/[file]/route.ts typescript
import { pngResponse, renderScreenshot } from "@lib/pwa-image";
export const dynamic = "force-static";
export const revalidate = false;
/** — SVG */
export async function GET(_req: Request, ctx: { params: Promise<{ file: string }> }) {
  const { file } = await ctx.params;
  if (!/^[\w-]+\.(png$|\.test(file)) return new Response("not found", { status: 404 });
  const bytes = await renderScreenshot(file.replace(/\.png$/, ""));
  if (!bytes) return new Response("not available", { status: 404 });
  return pngResponse(bytes);
}

```

```

@@ src/app/admin/records/[type]/page.tsx typescript
import { notFound } from "next/navigation";
import RecordScreen, { type RecordType } from "@components/RecordScreen";
const TYPES = ["pharmacies", "doctors", "orders"] as const;
/** */
export function generateStaticParams() {
  return TYPES.map((type) => ({ type }));
}
export const dynamicParams = false;
export default async function AdminRecordsPage({ params }: { params: Promise<{ type: string }> }) {
  const { type } = await params;
  if (!TYPES.includes(type as RecordType)) notFound();
  return <RecordScreen type={type as RecordType} isAdmin />;
}

```

```

@@ src/app/api/attachments/[id]/route.ts typescript
import { db } from "@db";
import { attachments } from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { eq } from "drizzle-orm";
export const dynamic = "force-dynamic";
/** / . */
export async function GET(req: Request, ctx: { params: Promise<{ id: string }> }) {
  const user = await getSessionUser();
  const { id } = await ctx.params;
  const numId = Number(id);
  if (!Number.isFinite(numId)) return new Response("not found", { status: 404 });
  const rows = await db.select().from(attachments).where(eq(attachments.id, numId)).limit(1);
  const f = rows[0];
  if (!f) return new Response("not found", { status: 404 });
  const isAdmin = user?.role === "admin" || user?.role === "supervisor";
  if (!user || (!isAdmin && f.userId !== user.id)) return new Response("forbidden", { status: 403 });
  const download = new URL(req.url).searchParams.get("download") === "1";

```

```

const buffer = Buffer.from(f.data, "base64");
const bytes = new Uint8Array(buffer);
return new Response(bytes, {
  headers: {
    "Content-Type": f.mimeType,
    "Content-Length": String(bytes.length),
    "Content-Disposition": `${download ? "attachment" : "inline"}; filename="${encodeURIComponent(f.fileName)}"`,
    "Cache-Control": "private, max-age=600",
  },
});
}
}

```

@@ src/app/api/auth/check-username/route.ts typescript

```

import { db } from "@db";
import { roles, users } from "@db/schema";
import { ensureSeed } from "@lib/bootstrap";

import { eq } from "drizzle-orm";
export const dynamic = "force-dynamic";
/**
 */
export async function POST(req: Request) {
  await ensureSeed();
  const b = await req.json().catch(() => ({}));
  const username = String(b.username ?? "").trim().toLowerCase();
  if (!username) return Response.json({ exists: false });
  const rows = await db
    .select({
      role: users.role,
      requirePhone: users.requirePhone,
      phone: users.phone,
      deviceId: users.deviceId,
    })
    .from(users)
    .where(eq(users.username, username))
    .limit(1);
  const u = rows[0];
  if (!u) return Response.json({ exists: false });
  let base: string = u.role;
  if (!["admin", "supervisor", "rep"].includes(u.role)) {
    const r = (await db.select().from(roles).where(eq(roles.key, u.role)).limit(1))[0];
    base = r?.base ?? "rep";
  }
  const digits = String(u.phone ?? "").replace(/\D/g, "");
  //
  const phoneMask = digits.length >= 8 ? `${digits.slice(0, 4)}***${digits.slice(-4)}` : "";
  return Response.json({
    exists: true,
    kind: base === "rep" ? "rep" : "admin",
    requirePhone: u.requirePhone,
    hasPhone: digits.length >= 10,
    phoneMask,
    deviceBound: Boolean(u.deviceId),
  });
}

```

@@ src/app/api/auth/forgot/route.ts typescript

```

import { db } from "@db";
import { users } from "@db/schema";
import { ensureSeed } from "@lib/bootstrap";
import { notify } from "@lib/notify";
import { eq } from "drizzle-orm";
export const dynamic = "force-dynamic";
/**
 */
export async function POST(req: Request) {
  await ensureSeed();
  const b = await req.json().catch(() => ({}));
  const username = String(b.username ?? "").trim().toLowerCase();
  const phone = String(b.phone ?? "").replace(/\D/g, "");
  if (!username) return Response.json({ error: " " }, { status: 400 });
  const rows = await db.select().from(users).where(eq(users.username, username)).limit(1);
  const u = rows[0];
  //
  const done = Response.json({
    ok: true,
    message:
      "
      .",
  });
  if (!u) return done;
  for (const role of ["admin", "supervisor"] as const) {
    await notify({
      toRole: role,
      fromName: u.fullName,
      kind: "password",
      title: ` — ${u.fullName}`,
    });
  }
}

```

```

    body: `:${u.username}\n: ${phone || "—"}\n: ${u.phone || "—"}\n «
» `;
    link: "/admin/users",
  });
}
return done;
}

@@ src/app/api/auth/login/route.ts typescript
import { db, dbRetry } from "@db";
import { activityLogs, roles, users } from "@db/schema";

import { SESSION_COOKIE, createToken, verifyPassword } from "@lib/auth";
import { ensureSeed } from "@lib/bootstrap";
import { eq } from "drizzle-orm";
import { cookies } from "next/headers";
export const dynamic = "force-dynamic";
function normalizePhone(p: string) {
  return String(p ?? "")
    .replace(/[\u06F0-\u06F9]/g, (c) => String(c.charCodeAt(0) - 0x06f0))
    .replace(/D/g, "")
    .replace(/^0098/, "0")
    .replace(/^98/, "0");
}
export async function POST(req: Request) {
  await ensureSeed();
  const body = await req.json().catch(() => ({}));
  const username = String(body.username ?? "").trim().toLowerCase();
  const password = String(body.password ?? "");
  const phone = normalizePhone(body.phone ?? "");
  const mode = body.mode === "admin" ? "admin" : "rep"; //
  const remember = body.remember === true;
  const deviceOnline = body.simActive !== false;
  if (!username || !password) {
    return Response.json({ error: " " }, { status: 400 });
  }
  let rows: (typeof users.$inferSelect)[] = [];
  try {
    rows = await dbRetry(() => db.select().from(users).where(eq(users.username, username)).limit(1, "login:user"));
  } catch (err) {
    console.error("login db error", err);
    return Response.json(
      { error: " . DATABASE_URL . " },
      { status: 500 },
    );
  }
  const user = rows[0];
  if (!user || !verifyPassword(password, user.passwordHash)) {
    return Response.json({ error: " " }, { status: 401 });
  }
  if (!user.active) return Response.json({ error: " " }, { status: 403 });
  //
  let baseRole: string = user.role;
  if (!["admin", "supervisor", "rep"].includes(user.role)) {
    try {
      const r = (await dbRetry(() => db.select().from(roles).where(eq(roles.key, user.role)).limit(1, "login:role")))[0];
      baseRole = r?.base ?? "rep";
    } catch {
      baseRole = "rep";
    }
  }
  const isManagerAccount = baseRole === "admin" || baseRole === "supervisor";
  if (mode === "admin" && !isManagerAccount) {
    return Response.json(
      { error: " . « » . " },
      { status: 403 },
    );
  }
  if (mode === "rep" && isManagerAccount) {
    return Response.json(
      { error: " / . « » . " },
      { status: 403 },
    );
  }
  /* =====
  * — :
  * off →
  * phone →
  * device → ()
  * otp →:
  * ===== */
  const simMode = isManagerAccount ? "off" : (user.simMode ?? "device");
  const deviceId = String(body.deviceId ?? "").replace(/^[a-zA-Z0-9]/g, "").slice(0, 80);
  if (simMode !== "off" && user.requirePhone) {

```

```

    if (!deviceOnline) {
      return Response.json(
        { error: "          .", simError: true },
        { status: 403 },
      );
    }
    if (!/^0\d{10}$/.test(phone)) {
      return Response.json(
        { error: "          ", simError: true },
        { status: 400 },
      );
    }
    const registered = normalizePhone(user.phone);
    if (registered && phone !== registered) {
      return Response.json(
        {
          error: ` : «${registered.slice(0, 4)}***${registered.slice(-4)}» «${phone.slice(0, 4)}***${phone.slice(-4)}» .`,
          simError: true,
          reason: "phone-mismatch",
        },
        { status: 403 },
      );
    }
    if (body.simActiveOnDevice === false) {
      return Response.json(
        { error: "          .", simError: true, reason: "no-sim" },
        { status: 403 },
      );
    }
  }
  if ((simMode === "device" || simMode === "otp") && !isManagerAccount) {
    if (!deviceId) {
      return Response.json(
        { error: "          .", simError: true },
        { status: 400 },
      );
    }
    if (user.deviceId && user.deviceId !== deviceId) {
      return Response.json(
        {
          error:
            "          « »
            .",
          needOtp: true,
          simError: true,
          reason: "device-mismatch",
        },
        { status: 403 },
      );
    }
    if (!user.deviceId) {
      if (simMode === "otp") {
        return Response.json(
          {
            error: "          .",
            needOtp: true,
            reason: "first-device",
          },
          { status: 403 },
        );
      }
    }
    // device:
    await db
      .update(users)
      .set({
        deviceId,
        deviceInfo: String(body.deviceInfo ?? "").slice(0, 200),
        deviceBoundAt: new Date(),
      })
      .where(eq(users.id, user.id));
    await db
      .insert(activityLogs)
      .values({
        userId: user.id,
        userName: user.fullName,
        action: " ",
        detail: `${String(body.deviceInfo ?? "").slice(0, 120)} | ${phone}`,
      })
      .catch(() => undefined);
  }
}
const token = createToken(user.id);
//      ( )      .

```



```

// « » .
{

    const store = await cookies();
    store.set(SESSION_COOKIE, token, {
        httpOnly: true,
        sameSite: "lax",
        secure: process.env.NODE_ENV === "production",
        path: "/",
        ...(remember ? { maxAge: 60 * 60 * 24 * 30 } : {}),
    });
}
await db.update(users).set({ lastSeenAt: new Date() }).where(eq(users.id, user.id));
await db
    .insert(activityLogs)
    .values({ userId: user.id, userName: user.fullName, action: " ", detail: phone || " " })
    .catch(() => undefined);
return Response.json({
    token,
    user: {
        id: user.id,
        username: user.username,
        fullName: user.fullName,
        role: baseRole,
        roleKey: user.role,
        roleLabel: baseRole === "admin" ? " " : baseRole === "supervisor" ? " " : " ",
        phone: user.phone,
        requirePhone: user.requirePhone,
        permissions: user.permissions,
    },
});
}

@@ src/app/api/auth/logout/route.ts typescript
import { SESSION_COOKIE } from "@lib/auth";
import { cookies } from "next/headers";
export const dynamic = "force-dynamic";
export async function POST() {
    const store = await cookies();
    store.delete(SESSION_COOKIE);
    return Response.json({ ok: true });
}

@@ src/app/api/auth/me/route.ts typescript
import { getSessionUser } from "@lib/auth";
import { ensureSeed } from "@lib/bootstrap";
export const dynamic = "force-dynamic";
export async function GET() {
    await ensureSeed();
    const user = await getSessionUser();
    if (!user) return Response.json({ user: null }, { status: 401 });
    return Response.json({ user });
}

@@ src/app/api/auth/otp/route.ts typescript
import { db } from "@db";
import { activityLogs, users } from "@db/schema";
import { SESSION_COOKIE, createToken, verifyPassword } from "@lib/auth";
import { ensureSeed } from "@lib/bootstrap";
import { getSmsConfig, issueOtp, verifyOtp } from "@lib/otp";
import { notify } from "@lib/notify";
import { eq } from "drizzle-orm";
import { cookies } from "next/headers";
export const dynamic = "force-dynamic";
function normalizePhone(p: string) {
    return String(p ?? "")
        .replace(/[\u06F0-\u06F9]/g, (c) => String(c.charCodeAt(0) - 0x06f0))
        .replace(/D/g, "")
        .replace(/^0098/, "0")
        .replace(/^98/, "0");
}
/**
 * :
 * action=request →
 * action=verify →
 */
export async function POST(req: Request) {
    await ensureSeed();
    const b = await req.json().catch(() => ({}));

    const action = String(b.action ?? "request");
    const username = String(b.username ?? "").trim().toLowerCase();
    const password = String(b.password ?? "");
    const deviceId = String(b.deviceId ?? "").replace(/^[a-zA-Z0-9]/g, "").slice(0, 80);
    const rows = await db.select().from(users).where(eq(users.username, username)).limit(1);

```

```

const user = rows[0];
if (!user || !verifyPassword(password, user.passwordHash)) {
  return Response.json({ error: " " }, { status: 401 });
}
if (!user.active) return Response.json({ error: " " }, { status: 403 });
const registered = normalizePhone(user.phone);
if (!registered) {
  return Response.json(
    { error: " " },
    { status: 400 },
  );
}
if (action === "request") {
  const r = await issueOtp(user.id, registered, deviceId);
  const cfg = await getSmsConfig();
  if (!r.sent) {
    // →
    await notify({
      toRole: "admin",
      fromName: user.fullName,
      kind: "otp",
      title: `${user.fullName}`,
      body: `${r.code}\n: ${registered}\n: \n`,
      link: "/admin/users",
    });
  }
  return Response.json({
    ok: true,
    sent: r.sent,
    channel: r.channel,
    masked: `${registered.slice(0, 4)}***${registered.slice(-4)}`,
    message: r.sent
      ? `${registered.slice(0, 4)}***${registered.slice(-4)}`
      : cfg.enabled
        ? ` (${r.detail})`
        : " ",
  });
}
if (action === "verify") {
  const v = await verifyOtp(user.id, registered, String(b.code ?? ""));
  if (!v.ok) return Response.json({ error: v.error }, { status: 400 });
  //
  await db
    .update(users)
    .set({
      deviceId,
      deviceInfo: String(b.deviceInfo ?? "").slice(0, 200),
      deviceBoundAt: new Date(),
      lastSeenAt: new Date(),
    })
    .where(eq(users.id, user.id));
  await db
    .insert(activityLogs)
    .values({
      userId: user.id,
      userName: user.fullName,
      action: " ",
      detail: `${registered} | ${String(b.deviceInfo ?? "").slice(0, 100)}`,
    })
    .catch(() => undefined);
  const token = createToken(user.id);
  {
    const store = await cookies();
    store.set(SESSION_COOKIE, token, {
      httpOnly: true,
      sameSite: "lax",
      secure: process.env.NODE_ENV === "production",
      path: "/",
      ...(b.remember !== false ? { maxAge: 60 * 60 * 24 * 30 } : {}),
    });
  }
  return Response.json({
    ok: true,
    token,
    user: {
      id: user.id,
      username: user.username,
      fullName: user.fullName,
      role: user.role,
      roleKey: user.role,
      phone: user.phone,
      requirePhone: user.requirePhone,
      permissions: user.permissions,
    },
  });
}

```

```

    },
  });
}
return Response.json({ error: " " }, { status: 400 });
}

@@ src/app/api/backup/restore/route.ts typescript
import { db } from "@db";
import {
  activityLogs,
  attachments,
  doctors,
  homes,
  leaves,
  messageLogs,
  messengers,
  notifications,
  options,
  orders,
  pharmacies,
  roles,
  settings,
  tripPoints,
  trips,
  users,
} from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { ensureSeed } from "@lib/bootstrap";
import { sql } from "drizzle-orm";
export const dynamic = "force-dynamic";
export const maxDuration = 60;
type Row = Record<string, unknown>;
/**      Date      */
function clean(rows: unknown, dateFields: string[] = []): Row[] {
  if (!Array.isArray(rows)) return [];
  return rows.map((r) => {
    const o = { ...(r as Row) };
    for (const f of dateFields) {
      if (o[f]) o[f] = new Date(String(o[f]));
      else if (o[f] === null) o[f] = null;
    }
    return o;
  });
}
}
async function resetSeq(table: string) {
  try {
    await db.execute(
      sql.raw(
        `SELECT setval(pg_get_serial_sequence('${table}', 'id'), COALESCE((SELECT MAX(id) FROM ${table}), 1), true)`
      ),
    );
  } catch {
    /* ignore */
  }
}
}
export async function POST(req: Request) {
  await ensureSeed();
  const user = await getSessionUser();
  if (!user || user.role !== "admin") return Response.json({ error: " " }, { status: 401 });
  const body = await req.json().catch(() => null);
  const payload = body?.data ? body : body?.backup;
  const data = (payload?.data ?? {}) as Record<string, unknown>;
  if (!data || typeof data !== "object" || !Array.isArray(data.users)) {
    return Response.json({ error: " " }, { status: 400 });
  }
  const mode = body?.mode === "merge" ? "merge" : "replace";
  const report: Record<string, number> = {};
  try {
    if (mode === "replace") {
      //
      for (const t of [
        "attachments",
        "trip_points",
        "trips",
        "orders",
        "doctors",
        "pharmacies",
        "homes",
        "leaves",
        "notifications",
        "message_logs",
        "messengers",
        "activity_logs",
        "options",
      ])

```

```

        "settings",
        "roles",
        "users",
    ]) {
        await db.execute(sql.raw(`DELETE FROM ${t}`));
    }
}

const put = async (
    // eslint-disable-next-line @typescript-eslint/no-explicit-any
    table: any,
    key: string,
    tableName: string,
    dates: string[] = [],
) => {
    const rows = clean(data[key], dates);
    if (!rows.length) return;
    for (let i = 0; i < rows.length; i += 200) {
        const chunk = rows.slice(i, i + 200);
        await db.insert(table).values(chunk).onConflictDoNothing();
    }
    report[key] = rows.length;
    await resetSeq(tableName);
};

await put(users, "users", "users", ["createdAt", "lastSeenAt", "deviceBoundAt"]);
await put(roles, "roles", "roles", ["createdAt"]);
await put(settings, "settings", "settings", ["updatedAt"]);
await put(options, "options", "options", ["createdAt"]);
await put(pharmacies, "pharmacies", "pharmacies", ["createdAt"]);
await put(doctors, "doctors", "doctors", ["createdAt"]);
await put(orders, "orders", "orders", ["createdAt"]);
await put(homes, "homes", "homes", ["updatedAt"]);
await put(leaves, "leaves", "leaves", ["createdAt"]);
await put(trips, "trips", "trips", ["startedAt", "endedAt"]);
await put(tripPoints, "tripPoints", "trip_points", ["recordedAt"]);
await put(messengers, "messengers", "messengers", ["createdAt"]);
await put(messageLogs, "messageLogs", "message_logs", ["createdAt"]);
await put(notifications, "notifications", "notifications", ["createdAt", "readAt"]);
await put(activityLogs, "activityLogs", "activity_logs", ["createdAt"]);
// base64
const atts = clean(data.attachments, ["createdAt"]).filter((a) => typeof a.data === "string" && a.data);
if (atts.length) {
    for (let i = 0; i < atts.length; i += 50) {
        await db
            .insert(attachments)
            .values(atts.slice(i, i + 50) as (typeof attachments.$inferInsert)[])
            .onConflictDoNothing();
    }
    report.attachments = atts.length;
    await resetSeq("attachments");
}

await db
    .insert(activityLogs)
    .values({
        userId: user.id,
        userName: user.fullName,
        action: " ",
        detail: `: ${mode === "replace" ? " " : ""} | ${JSON.stringify(report)}`.slice(0, 400),
    })
    .catch(() => undefined);

return Response.json({ ok: true, mode, report });
} catch (err) {
    console.error("restore error", err);
    return Response.json(
        { error: `: ${err instanceof Error ? err.message : ""}` },
        { status: 500 },
    );
}
}

@@ src/app/api/messengers/updates/route.ts typescript
import { getSessionUser } from "@lib/auth";
import { fetchUpdates, getTokens } from "@lib/messaging";
export const dynamic = "force-dynamic";
/** chat_id */
export async function POST(req: Request) {
    const user = await getSessionUser();
    if (!user || (user.role !== "admin" && !user.permissions.includes("messengers"))) {
        return Response.json({ error: " " }, { status: 401 });
    }
    const b = await req.json().catch(() => ({}));
    const platform = String(b.platform ?? "bale");
    const result = await fetchUpdates(platform, String(b.token ?? ""));
    return Response.json(result);
}

```

```

export async function GET() {
  const user = await getSessionUser();
  if (!user || (user.role !== "admin" && !user.permissions.includes("messengers"))) {
    return Response.json({ error: " " }, { status: 401 });
  }
  const tokens = await getTokens();
  // /
  const masked: Record<string, string> = {};
  for (const [k, v] of Object.entries(tokens)) {
    masked[k] = v ? `${v.slice(0, 8)}••••${v.slice(-4)} : `;
  }
  return Response.json({ tokens: masked });
}

@@ src/app/api/mobile/login-with-phone/route.ts typescript
import { db, dbRetry } from "@db";
import { activityLogs, roles, users } from "@db/schema";
import { SESSION_COOKIE, createToken } from "@lib/auth";
import { ensureSeed } from "@lib/bootstrap";
import { hashSim, maskPhone, normalizePhone, verifyAttestation } from "@lib/mobile-auth";
import { issueOtp } from "@lib/otp";
import { notify } from "@lib/notify";
import { eq } from "drizzle-orm";
import { cookies } from "next/headers";
export const dynamic = "force-dynamic";
/**
 *
 *
 * POST /api/mobile/login-with-phone
 * {
 *   deviceId, deviceInfo, nonce, timestamp, signature,
 *   phoneNumber?, //
 *   simFingerprint?, // ICCID/IMSI carrier+mcc/mnc ( )
 *   simCarrier?, simCount?, simState?
 * }
 *
 * :
 * 200 {token, user} →
 * 403 {needOtp: true, ...} → ( )
 * 401/403/404 {error, code} →
 */
export async function POST(req: Request) {
  await ensureSeed();
  const b = await req.json().catch(() => ({}));
  /* ----- */
  const att = await verifyAttestation(b);
  if (!att.ok) return Response.json({ error: att.error, code: att.code }, { status: att.status });
  const deviceId = String(b.deviceId ?? "").replace(/[^a-zA-Z0-9_-]/g, "").slice(0, 80);
  const deviceInfo = String(b.deviceInfo ?? "").slice(0, 200);
  const phone = normalizePhone(b.phoneNumber ?? "");
  const simFp = hashSim(String(b.simFingerprint ?? ""));
  const simCarrier = String(b.simCarrier ?? "").slice(0, 80);

  /* ----- ) → ----- */
  if (!phone || !/^0\d{10}$/.test(phone)) {
    return Response.json(
      {
        error:
          " ( ).
      },
      { status: 422 },
    );
  }
  /* ----- ) ----- */
  const rows = await dbRetry(() => db.select().from(users).where(eq(users.phone, phone)).limit(1), "mobile:findUser");
  let found: typeof users.$inferSelect | undefined = rows[0];
  if (!found) {
    // :
    const all = await dbRetry(() => db.select().from(users), "mobile:allUsers");
    found = all.find((u) => normalizePhone(u.phone) === phone);
  }
  if (!found) {
    return Response.json(
      { error: " . .", code: "PHONE_NOT_REGISTERED" },
      { status: 404 },
    );
  }
  const user = found;
  if (!user.active) {
    return Response.json({ error: " ", code: "USER_INACTIVE" }, { status: 403 });
  }
}

```

```

/* ----- ) ----- */
let baseRole: string = user.role;
if (!["admin", "supervisor", "rep"].includes(user.role)) {
  const r = (await dbRetry(() => db.select().from(roles).where(eq(roles.key, user.role)).limit(1, "mobile:role")))[0];
  baseRole = r?.base ?? "rep";
}
const isManager = baseRole === "admin" || baseRole === "supervisor";
/* ----- ) ----- */
const isPhoneVerificationRequired = user.requirePhone && !isManager;
const simMode = isManager ? "off" : (user.simMode ?? "device");
if (!isPhoneVerificationRequired || simMode === "off") {
  return Response.json(
    {
      error: " ",
      code: "PASSWORD_LOGIN_REQUIRED",
      loginUrl: "/login",
    },
    { status: 403 },
  );
}
/* ----- ) ----- */
if (user.deviceId && user.deviceId !== deviceId) {
  return Response.json(
    {
      error: " ",
      code: "DEVICE_MISMATCH",
      needOtp: true,
    },
    { status: 403 },
  );
}
/* ----- ) ----- */
if (user.simFingerprint && simFp && user.simFingerprint !== simFp) {
  return Response.json(
    {
      error: " ",
      code: "SIM_CHANGED",
      needOtp: true,
    },
    { status: 403 },
  );
}
/* ----- ) : ----- */
if (simMode === "otp" && !user.deviceId) {
  const r = await issueOtp(user.id, phone, deviceId);
  if (!r.sent) {
    await notify({
      toRole: "admin",
      fromName: user.fullName,
      kind: "otp",
      title: `${user.fullName}`,
      body: `: ${r.code}\n: ${phone}\n: `,
      link: "/admin/users",
    });
  }
  return Response.json(
    {
      error: " ",
      code: "OTP_REQUIRED",
      needOtp: true,
      masked: maskPhone(phone),
      sent: r.sent,
    },
    { status: 403 },
  );
}
/* ----- ) : + ----- */
await dbRetry(
  () =>
    db
      .update(users)
      .set({
        deviceId,
        deviceInfo,
        deviceBoundAt: user.deviceBoundAt ?? new Date(),
        simFingerprint: simFp || user.simFingerprint,
        simCarrier: simCarrier || user.simCarrier,
        simVerifiedAt: new Date(),
        lastSeenAt: new Date(),
      })
      .where(eq(users.id, user.id)),
  "mobile:bind",

```

```

    );
    await db
      .insert(activityLogs)
      .values({
        userId: user.id,
        userName: user.fullName,
        action: " ",
        detail: `${maskPhone(phone)} | ${simCarrier || " "} | ${deviceInfo}`,
      })
      .catch(() => undefined);
    const token = createToken(user.id);
    if (b.remember !== false) {
      const store = await cookies();
      store.set(SESSION_COOKIE, token, {
        httpOnly: true,
        sameSite: "lax",
        path: "/",
        maxAge: 60 * 60 * 24 * 30,
      });
    }
  }
  return Response.json({
    ok: true,
    token,
    user: {
      id: user.id,
      username: user.username,
      fullName: user.fullName,
      role: baseRole,
      roleKey: user.role,
      phone: user.phone,
      requirePhone: user.requirePhone,
      permissions: user.permissions,
    },
  });
}

@@ src/app/api/mobile/nonce/route.ts typescript
import { issueNonce } from "@lib/mobile-auth";
import { ensureSeed } from "@lib/bootstrap";

export const dynamic = "force-dynamic";
/**
 * nonce
 * nonce
 *
 * POST /api/mobile/nonce { deviceId }
 * → { nonce, expiresAt, ttlMs, serverTime }
 */
export async function POST(req: Request) {
  void ensureSeed();
  const b = await req.json().catch(() => ({}));
  const deviceId = String(b.deviceId ?? "").replace(/^[a-zA-Z0-9_-]/g, "").slice(0, 80);
  if (!deviceId) {
    return Response.json({ error: " " }, { status: 400 });
  }
  const { nonce, expiresAt, ttlMs } = await issueNonce(deviceId);
  return Response.json(
    { nonce, expiresAt: expiresAt.toISOString(), ttlMs, serverTime: Date.now() },
    { headers: { "Cache-Control": "no-store" } },
  );
}

```

```

@@ src/app/api/records/[type]/route.ts typescript
import { db, dbRetry } from "@db";
import { activityLogs, attachments, doctors, orders, pharmacies, settings } from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { and, desc, eq, inArray, SQL } from "drizzle-orm";
export const dynamic = "force-dynamic";
type Ctx = { params: Promise<{ type: string }> };
const num = (v: unknown): number | null => {
  const n = Number(v);
  return Number.isFinite(n) && n !== 0 ? n : null;
};
const str = (v: unknown, max = 0) => {
  const t = String(v ?? "").trim();
  return max > 0 ? t.slice(0, max) : t; // max=0
};
export async function GET(req: Request, ctx: Ctx) {
  const user = await getSessionUser();
  if (!user) return Response.json({ error: " " }, { status: 401 });
  const { type } = await ctx.params;
  const url = new URL(req.url);
  const userIdFilter = url.searchParams.get("userId");
  const isAdmin = user.role === "admin" || user.role === "supervisor";
  const limit = Math.min(Number(url.searchParams.get("limit")) || 300, 1000);

```

```

try {
  if (type === "pharmacies") {
    const w: SQL | undefined = !isAdmin
      ? eq(pharmacies.userId, user.id)
      : userIdFilter
      ? eq(pharmacies.userId, Number(userIdFilter))
      : undefined;
    const rows = await dbRetry(
      () => db.select().from(pharmacies).where(w).orderBy(desc(pharmacies.id)).limit(limit),
      "pharmacies:list",
    );
    return Response.json({ rows });
  }
  if (type === "doctors") {
    const w: SQL | undefined = !isAdmin
      ? eq(doctors.userId, user.id)
      : userIdFilter
      ? eq(doctors.userId, Number(userIdFilter))
      : undefined;
    const rows = await dbRetry(
      () => db.select().from(doctors).where(w).orderBy(desc(doctors.id)).limit(limit),
      "doctors:list",
    );
    return Response.json({ rows });
  }
  if (type === "orders") {
    const w: SQL | undefined = !isAdmin
      ? eq(orders.userId, user.id)
      : userIdFilter
      ? eq(orders.userId, Number(userIdFilter))
      : undefined;
    const rows = await dbRetry(
      () => db.select().from(orders).where(w).orderBy(desc(orders.id)).limit(limit),
      "orders:list",
    );
    return Response.json({ rows });
  }
} catch {
  return Response.json({ error: " " }, { status: 500 });
}
return Response.json({ error: " " }, { status: 400 });
}

export async function POST(req: Request, ctx: Ctx) {
  const user = await getSessionUser();
  if (!user) return Response.json({ error: " " }, { status: 401 });
  const { type } = await ctx.params;
  const body = await req.json().catch(() => ({}));
  const base = {
    userId: user.id,
    repName: user.fullName,
    dateShamsi: str(body.dateShamsi, 12),
    lat: num(body.lat),
    lng: num(body.lng),
    accuracy: num(body.accuracy),
    locationLabel: str(body.locationLabel),
  };
  try {
    if (type === "pharmacies") {
      if (!base.dateShamsi || !str(body.name)) {
        return Response.json({ error: " " }, { status: 400 });
      }
      const [row] = await dbRetry(() =>
        db
          .insert(pharmacies)
          .values({
            ...base,
            province: str(body.province, 100),
            city: str(body.city, 100),
            region: str(body.region, 100),
            name: str(body.name),
            landline: str(body.landline, 20),
            managerName: str(body.managerName, 160),
            managerPhone: str(body.managerPhone, 20),
            address: str(body.address),
            isPercent: body.isPercent === true,
            percentValue: str(body.percentValue),
            locationLabel: base.locationLabel || str(body.name),
          })
          .returning(),
        "pharmacies:create",
      );
      const phFiles: number[] = Array.isArray(body.fileIds)
        ? body.fileIds.map(Number).filter((n: number) => Number.isFinite(n))
        : [];
    }
  }
}

```



```

if (phFiles.length) {
    await db
        .update(attachments)
        .set({ ownerId: row.id })
        .where(and(inArray(attachments.id, phFiles), eq(attachments.userId, user.id)));
}
await log(user.id, user.fullName, " ", str(body.name));
return Response.json({ row });
}
if (type === "doctors") {
    if (!base.dateShamsi || !str(body.name)) {
        return Response.json({ error: " " }, { status: 400 });
    }
    const [row] = await db
        .insert(doctors)
        .values({
            ...base,
            province: str(body.province, 100),
            city: str(body.city, 100),
            region: str(body.region, 100),
            name: str(body.name),
            specialty: str(body.specialty, 160),
            phone: str(body.phone, 20),
            secretaryName: str(body.secretaryName, 160),
            secretaryPhone: str(body.secretaryPhone, 20),
            address: str(body.address),
            otherAddresses: str(body.otherAddresses),
            isPercent: body.isPercent === true,
            percentValue: str(body.percentValue),
            locationLabel: base.locationLabel || str(body.name),
        })
        .returning();
    const fileIds: number[] = Array.isArray(body.fileIds)

        ? body.fileIds.map(Number).filter((n: number) => Number.isFinite(n))
        : [];
    if (fileIds.length) {
        await db
            .update(attachments)
            .set({ ownerId: row.id })
            .where(and(inArray(attachments.id, fileIds), eq(attachments.userId, user.id)));
    }
    await log(user.id, user.fullName, " ", str(body.name));
    return Response.json({ row });
}
if (type === "orders") {
    if (!base.dateShamsi || !str(body.pharmacyName)) {
        return Response.json({ error: " " }, { status: 400 });
    }
    const items: Record<string, number> = {};
    for (const [k, v] of Object.entries((body.items ?? {}) as Record<string, unknown>)) {
        const n = Number(v);
        if (Number.isFinite(n) && n !== 0) items[k] = n;
    }
    const [row] = await db
        .insert(orders)
        .values({
            ...base,
            pharmacyName: str(body.pharmacyName),
            managerName: str(body.managerName, 160),
            managerPhone: str(body.managerPhone, 20),
            address: str(body.address),
            locationLabel: base.locationLabel || str(body.pharmacyName),
            items,
            distributor: str(body.distributor, 160),
            visitor: str(body.visitor, 160),
            notes: str(body.notes),
        })
        .returning();
    await log(user.id, user.fullName, " ", str(body.pharmacyName));
// ( — )
const autoSend = await getAutoSend();
if (autoSend) {
    void (async () => {
        try {
            const { dispatchOrder } = await import("@/lib/messaging");
            const status = await dispatchOrder({
                id: row.id,
                repName: row.repName,
                dateShamsi: row.dateShamsi,
                pharmacyName: row.pharmacyName,
                managerName: row.managerName,
                managerPhone: row.managerPhone,
                address: row.address,
                lat: row.lat,
            });
        } catch {
            //
        }
    })();
}

```

```

        lng: row.lng,
        items: row.items ?? {},
        distributor: row.distributor,
        visitor: row.visitor,
        notes: row.notes,
    });
    await db.update(orders).set({ sendStatus: status, sent: true }).where(eq(orders.id, row.id));
    const { notify } = await import("@/lib/notify");
    await notify({
        toRole: "admin",
        fromName: row.repName,
        kind: "order",
        title: `${row.repName}`,
        body: `${row.pharmacyName} — ${status}`,
        link: "/admin/records/orders",
    });
    } catch (err) {
        console.error("[order:autosend]", err instanceof Error ? err.message : err);
    }
    })();
    }
    return Response.json({ row, autoSend });
}
} catch {
    return Response.json({ error: " " }, { status: 500 });
}
return Response.json({ error: " " }, { status: 400 });
}
/**
    (: ) */
async function getAutoSend(): Promise<boolean> {
    try {

        const rows = await db.select().from(settings).where(eq(settings.key, "autoSendOrders")).limit(1);
        const v = rows[0]?.value as { enabled?: boolean } | boolean | undefined;
        if (typeof v === "boolean") return v;
        if (v && typeof v === "object" && typeof v.enabled === "boolean") return v.enabled;
    } catch {
        /* ignore */
    }
    return true;
}
async function log(userId: number, userName: string, action: string, detail: string) {
    try {
        await db.insert(activityLogs).values({ userId, userName, action, detail });
    } catch {
        /* ignore */
    }
}
/** — */
export async function PATCH(req: Request, ctx: Ctx) {
    const user = await getSessionUser();
    if (!user) return Response.json({ error: " " }, { status: 401 });
    const { type } = await ctx.params;
    const body = await req.json().catch(() => ({}));
    const id = Number(body.id);
    if (!Number.isFinite(id)) return Response.json({ error: " " }, { status: 400 });
    const isAdmin = user.role === "admin" || user.role === "supervisor";
    const patch: Record<string, unknown> = {};
    const setIf = (key: string, val: unknown) => {
        if (val !== undefined) patch[key] = val;
    };
    try {
        if (type === "pharmacies") {
            setIf("dateShamsi", body.dateShamsi ? str(body.dateShamsi, 12) : undefined);
            setIf("province", body.province !== undefined ? str(body.province, 100) : undefined);
            setIf("city", body.city !== undefined ? str(body.city, 100) : undefined);
            setIf("region", body.region !== undefined ? str(body.region, 100) : undefined);
            setIf("name", body.name !== undefined ? str(body.name) : undefined);
            setIf("landline", body.landline !== undefined ? str(body.landline, 20) : undefined);
            setIf("managerName", body.managerName !== undefined ? str(body.managerName, 160) : undefined);
            setIf("managerPhone", body.managerPhone !== undefined ? str(body.managerPhone, 20) : undefined);
            setIf("address", body.address !== undefined ? str(body.address) : undefined);
            setIf("isPercent", typeof body.isPercent === "boolean" ? body.isPercent : undefined);
            setIf("percentValue", body.percentValue !== undefined ? str(body.percentValue) : undefined);
            setIf("lat", body.lat !== undefined ? num(body.lat) : undefined);
            setIf("lng", body.lng !== undefined ? num(body.lng) : undefined);
            if (Object.keys(patch).length === 0) return Response.json({ error: " " }, { status: 400 });
            const w = isAdmin ? eq(pharmacies.id, id) : and(eq(pharmacies.id, id), eq(pharmacies.userId, user.id));
            await dbRetry(() => db.update(pharmacies).set(patch).where(w), "pharmacies:update");
        } else if (type === "doctors") {
            setIf("dateShamsi", body.dateShamsi ? str(body.dateShamsi, 12) : undefined);
            setIf("province", body.province !== undefined ? str(body.province, 100) : undefined);
            setIf("city", body.city !== undefined ? str(body.city, 100) : undefined);
            setIf("region", body.region !== undefined ? str(body.region, 100) : undefined);
            setIf("name", body.name !== undefined ? str(body.name) : undefined);

```

```

    setIf("specialty", body.specialty !== undefined ? str(body.specialty, 160) : undefined);
    setIf("phone", body.phone !== undefined ? str(body.phone, 20) : undefined);
    setIf("secretaryName", body.secretaryName !== undefined ? str(body.secretaryName, 160) : undefined);
    setIf("secretaryPhone", body.secretaryPhone !== undefined ? str(body.secretaryPhone, 20) : undefined);
    setIf("address", body.address !== undefined ? str(body.address) : undefined);
    setIf("otherAddresses", body.otherAddresses !== undefined ? str(body.otherAddresses) : undefined);
    setIf("isPercent", typeof body.isPercent === "boolean" ? body.isPercent : undefined);
    setIf("percentValue", body.percentValue !== undefined ? str(body.percentValue) : undefined);
    setIf("lat", body.lat !== undefined ? num(body.lat) : undefined);
    setIf("lng", body.lng !== undefined ? num(body.lng) : undefined);
    if (Object.keys(patch).length === 0) return Response.json({ error: " " }, { status: 400 });
    const w = isAdmin ? eq(doctors.id, id) : and(eq(doctors.id, id), eq(doctors.userId, user.id));
    await dbRetry(() => db.update(doctors).set(patch).where(w), "doctors:update");
  } else if (type === "orders") {
    setIf("dateShamsi", body.dateShamsi ? str(body.dateShamsi, 12) : undefined);
    setIf("pharmacyName", body.pharmacyName !== undefined ? str(body.pharmacyName) : undefined);
    setIf("managerName", body.managerName !== undefined ? str(body.managerName, 160) : undefined);
    setIf("managerPhone", body.managerPhone !== undefined ? str(body.managerPhone, 20) : undefined);
    setIf("address", body.address !== undefined ? str(body.address) : undefined);
    setIf("distributor", body.distributor !== undefined ? str(body.distributor, 160) : undefined);
    setIf("visitor", body.visitor !== undefined ? str(body.visitor, 160) : undefined);
    setIf("notes", body.notes !== undefined ? str(body.notes) : undefined);
    setIf("lat", body.lat !== undefined ? num(body.lat) : undefined);
    setIf("lng", body.lng !== undefined ? num(body.lng) : undefined);
    if (body.items && typeof body.items === "object") {
      const items: Record<string, number> = {};
      for (const [k, v] of Object.entries(body.items as Record<string, unknown>)) {
        const n = Number(v);

        if (Number.isFinite(n) && n !== 0) items[k] = n;
      }
      patch.items = items;
    }
    if (Object.keys(patch).length === 0) return Response.json({ error: " " }, { status: 400 });
    const w = isAdmin ? eq/orders.id, id) : and(eq/orders.id, id), eq/orders.userId, user.id));
    await dbRetry(() => db.update/orders).set(patch).where(w), "orders:update");
  } else {
    return Response.json({ error: " " }, { status: 400 });
  }
} catch {
  return Response.json({ error: " " }, { status: 500 });
}
await log(user.id, user.fullName, " ", `${type} #${id}`);
return Response.json({ ok: true });
}
/** */
export async function DELETE(req: Request, ctx: Ctx) {
  const user = await getSessionUser();
  if (!user) return Response.json({ error: " " }, { status: 401 });
  const { type } = await ctx.params;
  const id = Number(new URL(req.url).searchParams.get("id"));
  if (!Number.isFinite(id)) return Response.json({ error: " " }, { status: 400 });
  const isAdmin = user.role === "admin" || user.role === "supervisor";
  const perm = type === "pharmacies" ? "pharmacy.delete" : type === "doctors" ? "doctor.delete" : "order.delete";
  if (!isAdmin && !user.permissions.includes(perm)) {
    return Response.json({ error: " . . " }, { status: 403 });
  }
  try {
    if (type === "pharmacies") {
      const w = isAdmin ? eq(pharmacies.id, id) : and(eq(pharmacies.id, id), eq(pharmacies.userId, user.id));
      await dbRetry(() => db.delete(pharmacies).where(w), "pharmacies:delete");
    } else if (type === "doctors") {
      const w = isAdmin ? eq(doctors.id, id) : and(eq(doctors.id, id), eq(doctors.userId, user.id));
      await dbRetry(() => db.delete(doctors).where(w), "doctors:delete");
    } else if (type === "orders") {
      const w = isAdmin ? eq/orders.id, id) : and(eq/orders.id, id), eq/orders.userId, user.id));
      await dbRetry(() => db.delete/orders).where(w), "orders:delete");
    } else {
      return Response.json({ error: " " }, { status: 400 });
    }
  } catch {
    return Response.json({ error: " " }, { status: 500 });
  }
  await log(user.id, user.fullName, " ", `${type} #${id}`);
  return Response.json({ ok: true });
}
}

@@ src/app/api/records/lookup/route.ts typescript
import { db, dbRetrySafe } from "@db";
import { doctors, orders, pharmacies } from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { bonusKeyOf } from "@lib/defaults";
import { getProducts } from "@lib/settings-server";
import { desc } from "drizzle-orm";
export const dynamic = "force-dynamic";

```

```

/** : «» «» */
export function normalizeName(input: string) {
  return String(input ?? "").
    .replace(/[\u200c\u200f\u200e]/g, "")
    .replace(/[/]/g, "")
    .replace(/\\/g, "")
    .replace(/[/]/g, "")
    .replace(/[/]/g, "")
    .replace(/[/]/g, "")
    .replace(/^(|)|\s*/g, "")
    .replace(/s+/g, "")
    .trim()
    .toLowerCase();
}
function normPhone(p: string) {
  return String(p ?? "").replace(/D/g, "");
}
/**
 * GET /api/records/lookup?type=pharmacies|doctors|orders&name=...&phone=...
 *
 * :
 *
 * ) /
 * )
 */
export async function GET(req: Request) {
  const user = await getSessionUser();
  if (!user) return Response.json({ error: " " }, { status: 401 });
  const url = new URL(req.url);
  const type = url.searchParams.get("type") ?? "pharmacies";
  const rawName = url.searchParams.get("name") ?? "";
  const phone = normPhone(url.searchParams.get("phone") ?? "");
  const key = normalizeName(rawName);
  if (key.length < 2 && !phone) return Response.json({ duplicates: [], suggestion: null });
  /* ----- : ----- */
  if (type === "doctors") {
    const rows = await dbRetrySafe(
      () => db.select().from(doctors).orderBy(desc(doctors.id)).limit(3000),
      [],
      "lookup:doctors",
    );
    const dup = rows.filter(
      (r) => (key.length >= 2 && normalizeName(r.name) === key) || (!!phone && normPhone(r.phone) === phone),
    );
    return Response.json({
      duplicates: dup.slice(0, 5).map((r) => ({
        id: r.id,
        name: r.name,
        specialty: r.specialty,
        phone: r.phone,
        city: r.city,
        address: r.address,
        repName: r.repName,
        dateShamsi: r.dateShamsi,
        mine: r.userId === user.id,
      })),
      count: dup.length,
    });
  }
  /* ----- : + ----- */
  const phRows = await dbRetrySafe(
    () => db.select().from(pharmacies).orderBy(desc(pharmacies.id)).limit(3000),
    [],
    "lookup:pharmacies",
  );
  const dup = phRows.filter(
    (r) => (key.length >= 2 && normalizeName(r.name) === key) || (!!phone && normPhone(r.managerPhone) === phone),
  );
  const duplicates = dup.slice(0, 5).map((r) => ({
    id: r.id,
    name: r.name,
    province: r.province,
    city: r.city,
    region: r.region,
    landline: r.landline,
    managerName: r.managerName,
    managerPhone: r.managerPhone,
    address: r.address,
    lat: r.lat,
    lng: r.lng,
    accuracy: r.accuracy,
    isPercent: r.isPercent,
    percentValue: r.percentValue,
    repName: r.repName,
    dateShamsi: r.dateShamsi,
    mine: r.userId === user.id,
  }));
}

```

```

    ));
    if (type !== "orders") return Response.json({ duplicates, count: dup.length });
    /* ----- */
    const orderRows = await dbRetrySafe(
        () => db.select().from(orders).orderBy(desc(orders.id)).limit(3000),
        [],
        "lookup:orders",
    );
    const history = orderRows.filter((o) => normalizeName(o.pharmacyName) === key);
    const products = await getProducts();
    const itemTotals: Record<string, { label: string; qty: number; bonus: number }> = {};
    for (const o of history) {
        for (const p of products) {
            const q = Number(o.items?.[p.key] ?? 0);
            const bn = Number(o.items?.[bonusKeyOf(p.key)] ?? 0);
            if (!q && !bn) continue;
            itemTotals[p.key] ??= { label: p.label, qty: 0, bonus: 0 };
            itemTotals[p.key].qty += q;
            itemTotals[p.key].bonus += bn;
        }
    }
    const last = history[0] ?? null;
    const base = duplicates[0] ?? null;
    // ( )
    const suggestion =
        last || base
        ? {
            source: last ? "order" : "pharmacy",
            pharmacyName: last?.pharmacyName ?? base?.name ?? rawName,
            managerName: last?.managerName || base?.managerName || "",
            managerPhone: last?.managerPhone || base?.managerPhone || "",
            address: last?.address || base?.address || "",
            lat: last?.lat ?? base?.lat ?? null,
            lng: last?.lng ?? base?.lng ?? null,
            accuracy: last?.accuracy ?? base?.accuracy ?? null,
            distributor: last?.distributor ?? "",
            visitor: last?.visitor ?? "",
        }
        : null;
    return Response.json({
        duplicates,
        count: dup.length,
        history: {
            orderCount: history.length,
            lastDate: last?.dateShamsi ?? "",
            lastRep: last?.repName ?? "",
            totalUnits: Object.values(itemTotals).reduce((a, x) => a + x.qty, 0),
            totalBonus: Object.values(itemTotals).reduce((a, x) => a + x.bonus, 0),
            items: Object.entries(itemTotals).map(([k, v]) => ({ key: k, ...v })),
            recent: history.slice(0, 5).map(o => ({
                id: o.id,
                dateShamsi: o.dateShamsi,
                repName: o.repName,
                units: products.reduce((a, p) => a + Number(o.items?.[p.key] ?? 0), 0),
            })),
        },
        suggestion,
    });
}

```

```

@@ src/app/api/sync/pull/route.ts typescript
import { collectChanges, verifySync } from "@lib/sync";
import { ensureSeed } from "@lib/bootstrap";
import { nodeName } from "@lib/sync-config";
export const dynamic = "force-dynamic";
/**
 *
 * POST /api/sync/pull { since, excludeOrigin } ( HMAC)
 */
export async function POST(req: Request) {
    await ensureSeed();
    const raw = await req.text();
    const ts = Number(req.headers.get("x-sync-timestamp") ?? 0);
    const sig = req.headers.get("x-sync-signature") ?? "";
    if (!verifySync(raw, ts, sig)) {
        return Response.json({ error: " " }, { status: 401 });
    }
    let body: { since?: string | null; excludeOrigin?: string } = {};
    try {
        body = JSON.parse(raw);
    } catch {
        /* ignore */
    }
    const since = body.since ? new Date(body.since) : null;

```

```

const changes = await collectChanges(Number.isNaN(since?.getTime()) ? null : since, body.excludeOrigin);
const count = Object.values(changes).reduce((a, r) => a + r.length, 0);
return Response.json(
  { node: nodeName(), cursor: new Date().toISOString(), count, changes },
  { headers: { "Cache-Control": "no-store" } },
);
}

```

```

@@ src/app/api/sync/push/route.ts typescript
import { applyChanges, verifySync, type ChangeSet } from "@lib/sync";
import { ensureSeed } from "@lib/bootstrap";
import { nodeName } from "@lib/sync-config";
export const dynamic = "force-dynamic";
/**
 *
 * POST /api/sync/push { node, changes } ( HMAC)
 */
export async function POST(req: Request) {
  await ensureSeed();
  const raw = await req.text();
  const ts = Number(req.headers.get("x-sync-timestamp") ?? 0);
  const sig = req.headers.get("x-sync-signature") ?? "";
  if (!verifySync(raw, ts, sig)) {
    return Response.json({ error: " " }, { status: 401 });
  }
  let body: { node?: string; changes?: ChangeSet } = {};
  try {
    body = JSON.parse(raw);
  } catch {
    return Response.json({ error: " " }, { status: 400 });
  }
  const from = String(body.node ?? req.headers.get("x-sync-node") ?? "peer");
  const res = await applyChanges(body.changes ?? {}, from);
  return Response.json(
    { node: nodeName(), applied: res.applied, skipped: res.skipped, tables: res.perTable },
    { headers: { "Cache-Control": "no-store" } },
  );
}

```

```

@@ src/app/api/sync/run/route.ts typescript
import { getSessionUser } from "@lib/auth";
import { ensureSeed } from "@lib/bootstrap";
import { syncAll } from "@lib/sync";
import { syncSecret } from "@lib/sync-config";
export const dynamic = "force-dynamic";
export const maxDuration = 60;
/**
 * / .
 * -
 * - Cron x-sync-key
 */
async function authorize(req: Request) {
  const key = req.headers.get("x-sync-key") ?? new URL(req.url).searchParams.get("key");
  if (key && syncSecret() && key === syncSecret()) return true;
  const user = await getSessionUser();
  return Boolean(user && (user.role === "admin" || user.permissions.includes("users")));
}
export async function POST(req: Request) {
  await ensureSeed();
  if (!(await authorize(req))) return Response.json({ error: " " }, { status: 401 });
  const reports = await syncAll();
  const ok = reports.every((r) => r.ok);
  return Response.json(
    {
      ok,
      count: reports.length,
      reports,
      summary:
        reports.length === 0
          ? " "
          : reports.map((r) => `${r.peer}: ${r.ok ? ` ` : ` `} ${r.pulled} / ${r.pushed}` : ` ${r.detail}`}).join(
            " | ",
          ),
    },
    { headers: { "Cache-Control": "no-store" } },
  );
}
export async function GET(req: Request) {
  return POST(req);
}

```

```

@@ src/app/api/sync/status/route.ts typescript
import { db, dbRetrySafe } from "@db";
import { syncLogs, syncState } from "@db/schema";
import { getSessionUser } from "@lib/auth";

```

```

import { ensureSeed } from "@lib/bootstrap";
import { ensurePeers, syncInfo } from "@lib/sync";
import { desc } from "drizzle-orm";
export const dynamic = "force-dynamic";
export async function GET() {
  await ensureSeed();
  const user = await getSessionUser();
  if (!user || (user.role !== "admin" && !user.permissions.includes("users"))) {
    return Response.json({ error: " " }, { status: 401 });
  }
  const peers = await ensurePeers();
  const logs = await dbRetrySafe(
    () => db.select().from(syncLogs).orderBy(desc(syncLogs.id)).limit(40),
    [],
    "sync:logs",
  );
  return Response.json({ info: syncInfo(), peers, logs }, { headers: { "Cache-Control": "no-store" } });
}
/** /
export async function PATCH(req: Request) {
  const user = await getSessionUser();
  if (!user || user.role !== "admin") return Response.json({ error: " " }, { status: 401 });
  const b = await req.json().catch(() => ({}));
  const id = Number(b.id);
  if (!Number.isFinite(id)) return Response.json({ error: " " }, { status: 400 });
  const patch: Record<string, unknown> = {};
  if (typeof b.enabled === "boolean") patch.enabled = b.enabled;
  if (typeof b.peerUrl === "string") patch.peerUrl = b.peerUrl.trim();
  if (b.resetCursor === true) {
    patch.pullCursor = null;
    patch.pushCursor = null;
  }
  if (Object.keys(patch).length === 0) return Response.json({ error: " " }, { status: 400 });
  const { eq } = await import("drizzle-orm");
  await db.update(syncState).set(patch).where(eq(syncState.id, id));
  return Response.json({ ok: true });
}
/**
export async function POST(req: Request) {
  const user = await getSessionUser();
  if (!user || user.role !== "admin") return Response.json({ error: " " }, { status: 401 });
  const b = await req.json().catch(() => ({}));
  const peer = String(b.peer ?? "").trim().slice(0, 80);
  const peerUrl = String(b.peerUrl ?? "").trim();
  if (!peer || !peerUrl.startsWith("http")) {
    return Response.json({ error: " " }, { status: 400 });
  }
  await db.insert(syncState).values({ peer, peerUrl }).onConflictDoNothing();
  return Response.json({ ok: true });
}

```

```

@@@ src/app/api/trips/points/route.ts typescript
import { db } from "@db";
import { liveLocations, tripPoints, trips } from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { asc, eq } from "drizzle-orm";
/**
function haversine(a: { lat: number; lng: number }, b: { lat: number; lng: number }) {
  const R = 6371000;
  const dLat = ((b.lat - a.lat) * Math.PI) / 180;
  const dLng = ((b.lng - a.lng) * Math.PI) / 180;
  const s =
    Math.sin(dLat / 2) ** 2 +
    Math.cos((a.lat * Math.PI) / 180) * Math.cos((b.lat * Math.PI) / 180) * Math.sin(dLng / 2) ** 2;
  const d = 2 * R * Math.asin(Math.sqrt(s));
  // GPS:
  return d < 5 || d > 5000 ? 0 : d;
}
export const dynamic = "force-dynamic";
export async function GET(req: Request) {
  const user = await getSessionUser();
  if (!user) return Response.json({ error: " " }, { status: 401 });

  const tripId = Number(new URL(req.url).searchParams.get("tripId"));
  if (!Number.isFinite(tripId)) return Response.json({ error: " " }, { status: 400 });
  const t = (await db.select().from(trips).where(eq(trips.id, tripId)).limit(1))[0];
  if (!t || (user.role !== "admin" && t.userId !== user.id)) {
    return Response.json({ error: " " }, { status: 404 });
  }
  const points = await db
    .select()
    .from(tripPoints)
    .where(eq(tripPoints.tripId, tripId))
    .orderBy(asc(tripPoints.recordedAt))
    .limit(5000);

```

```

    return Response.json({ trip: t, points });
}
export async function POST(req: Request) {
    const user = await getSessionUser();
    if (!user) return Response.json({ error: " " }, { status: 401 });
    const b = await req.json().catch(() => ({}));
    const tripId = Number(b.tripId);
    if (!Number.isFinite(tripId)) return Response.json({ error: " " }, { status: 400 });
    const t = (await db.select().from(trips).where(eq(trips.id, tripId)).limit(1))[0];
    if (!t || t.userId !== user.id) return Response.json({ error: " " }, { status: 404 });
    const raw = Array.isArray(b.points) ? b.points : [];
    const values = raw
        .map((p: Record<string, unknown>) => ({
            tripId,
            lat: Number(p.lat),
            lng: Number(p.lng),
            accuracy: Number(p.accuracy) || null,
            speed: Number(p.speed) || null,
            stopSeconds: Math.max(0, Math.round(Number(p.stopSeconds) || 0)),
            gpsOn: p.gpsOn !== false,
            kind: ["start", "move", "pause", "end"].includes(String(p.kind)) ? String(p.kind) : "move",
            note: String(p.note ?? "").slice(0, 200),
            recordedAt: p.recordedAt ? new Date(String(p.recordedAt)) : new Date(),
            synced: true,
        })))
        .filter((p: { lat: number; lng: number }) => Number.isFinite(p.lat) && Number.isFinite(p.lng))
        .slice(0, 500);
    if (values.length) await db.insert(tripPoints).values(values);
    //
    try {
        const all = await db
            .select({ lat: tripPoints.lat, lng: tripPoints.lng, stopSeconds: tripPoints.stopSeconds })
            .from(tripPoints)
            .where(eq(tripPoints.tripId, tripId))
            .orderBy(asc(tripPoints.recordedAt));
        let dist = 0;
        for (let i = 1; i < all.length; i++) dist += haversine(all[i - 1], all[i]);
        const stops = all.reduce((a, x) => a + (x.stopSeconds ?? 0), 0);
        await db.update(trips).set({ distanceM: Math.round(dist), stopSeconds: stops }).where(eq(trips.id, tripId));
        //
        const last = values[values.length - 1];
        if (last) {
            const lv = {
                userId: user.id,
                repName: user.fullName,
                lat: last.lat,
                lng: last.lng,
                accuracy: last.accuracy,
                speed: last.speed,
                gpsOn: last.gpsOn,
                online: true,
                tripId,
                recordedAt: last.recordedAt,
                updatedAt: new Date(),
            };
            await db.insert(liveLocations).values(lv).onConflictDoUpdate({ target: liveLocations.userId, set: lv });
        }
    } catch {
        /* */
    }
    return Response.json({ ok: true, inserted: values.length });
}

```

```

@@ src/app/api/records/[type]/send/route.ts typescript
import { db } from "@/db";

```

```

import { activityLogs, doctors, orders, pharmacies } from "@/db/schema";
import { getSessionUser } from "@/lib/auth";
import { dispatchOrder } from "@/lib/messaging";
import { notify } from "@/lib/notify";
import { eq, inArray, and } from "drizzle-orm";
export const dynamic = "force-dynamic";
type Ctx = { params: Promise<{ type: string }> };
export async function POST(req: Request, ctx: Ctx) {
    const user = await getSessionUser();
    if (!user) return Response.json({ error: " " }, { status: 401 });
    const { type } = await ctx.params;
    const body = await req.json().catch(() => ({}));
    const ids: number[] = Array.isArray(body.ids)
        ? body.ids.map((v: unknown) => Number(v)).filter((n: number) => Number.isFinite(n))
        : [];
    if (ids.length === 0) return Response.json({ error: " " }, { status: 400 });
    let sendStatusText = "";
    try {
        if (type === "pharmacies") {

```



```

    await db
      .update(pharmacies)
      .set({ sent: true })
      .where(and(inArray(pharmacies.id, ids), eq(pharmacies.userId, user.id)));
  } else if (type === "doctors") {
    await db
      .update(doctors)
      .set({ sent: true })
      .where(and(inArray(doctors.id, ids), eq(doctors.userId, user.id)));
  } else if (type === "orders") {
    const rows = await db
      .select()
      .from(orders)
      .where(and(inArray(orders.id, ids), eq(orders.userId, user.id)));
    let lastStatus = "";
    for (const row of rows) {
      const status = await dispatchOrder({
        id: row.id,
        repName: row.repName,
        dateShamsi: row.dateShamsi,
        pharmacyName: row.pharmacyName,
        managerName: row.managerName,
        managerPhone: row.managerPhone,
        address: row.address,
        lat: row.lat,
        lng: row.lng,
        items: row.items ?? {},
        distributor: row.distributor,
        visitor: row.visitor,
        notes: row.notes,
      });
      lastStatus = status;
      await db.update(orders).set({ sent: true, sendStatus: status }).where(eq(orders.id, row.id));
    }
    sendStatusText = lastStatus;
  } else {
    return Response.json({ error: " " }, { status: 400 });
  }
  await db.insert(activityLogs).values({
    userId: user.id,
    userName: user.fullName,
    action: " ",
    detail: `${type} - ${ids.length} `,
  });
  const typeLabel = type === "pharmacies" ? "" : type === "doctors" ? "" : "";
  await notify({
    toRole: "admin",
    fromName: user.fullName,
    kind: "record",
    title: ` ${ids.length} ${typeLabel} ${user.fullName}`,
    body: sendStatusText || ` ${typeLabel} `,
    link: `/admin/records/${type}`,
  });
  await notify({
    toRole: "supervisor",
    fromName: user.fullName,
    kind: "record",
    title: ` ${ids.length} ${typeLabel} ${user.fullName}`,
    link: `/admin/records/${type}`,
  });
  return Response.json({ ok: true, count: ids.length, status: sendStatusText });
} catch {
  return Response.json({ error: " " }, { status: 500 });
}
}

@@ src/app/api/tiles/[z]/[x]/[y]/route.ts typescript
import { fetchWithRetry } from "@lib/retry";
export const dynamic = "force-dynamic";
/**
 * OpenStreetMap. /Service Worker
 */
export async function GET(_req: Request, ctx: { params: Promise<{ z: string; x: string; y: string }> }) {
  const { z, x, y } = await ctx.params;
  if (!/^\\d+$/ .test(z) || !/^\\d+$/ .test(x) || !/^\\d+$/ .test(y)) return new Response("bad tile", { status: 400 });
  const zi = Number(z);
  if (zi < 0 || zi > 18) return new Response("bad zoom", { status: 400 });
  try {
    const sub = ["a", "b", "c"][(Number(x) + Number(y)) % 3];
    const res = await fetchWithRetry(`https://${sub}.tile.openstreetmap.org/${z}/${x}/${y}.png`, {
      headers: { "User-Agent": "SabtEtelaatKol/1.0 (offline map cache)" },
    }, { retries: 2, timeoutMs: 12000, label: "tile" });
    if (!res.ok) return new Response("tile unavailable", { status: res.status });
  }
}

```

```

const bytes = new Uint8Array(await res.arrayBuffer());
return new Response(bytes, {
  headers: {
    "Content-Type": "image/png",
    "Cache-Control": "public, max-age=2592000, stale-while-revalidate=86400",
    "Access-Control-Allow-Origin": "",
  },
});
} catch {
  // /
  return new Response(null, { status: 204, headers: { "Cache-Control": "no-store" } });
}
}

@@ src/app/api/map/tiles/[z]/[x]/[y]/route.ts typescript
import { fetchWithRetry } from "@lib/retry";
export const dynamic = "force-dynamic";
export const revalidate = 86400;
/**
 *      403
 *
 *      tile.openstreetmap.org
 *      Route      User-Agent
 *      .
 */
const SOURCES = [
  (z: number, x: number, y: number) => `https://tile.openstreetmap.org/${z}/${x}/${y}.png`,
  (z: number, x: number, y: number) => `https://a.tile.openstreetmap.fr/hot/${z}/${x}/${y}.png`,
  (z: number, x: number, y: number) => `https://basemaps.cartocdn.com/light_all/${z}/${x}/${y}.png`,
];
const TILE_HEADERS = {
  "User-Agent": "NdcHub-SabtEtelaat/1.0 (https://ndcohub.ir; map tile proxy)",
  Accept: "image/avif,image/webp,image/png,image/*/*/*;q=0.8",
  Referer: "https://ndcohub.ir/",
};
/** SVG — 403/ */
function fallbackTile(z: number, x: number, y: number) {
  const svg = `<svg xmlns="http://www.w3.org/2000/svg" width="256" height="256">
    <rect width="256" height="256" fill="#eef2f1"/>
    <path d="M0 64h256M0 128h256M0 192h256M64 0v256M128 0v256M192 0v256" stroke="#d9e3e0" stroke-width="1"/>
    <path d="M0 180Q80 100 150 155T256 75" fill="none" stroke="#c2d6d0" stroke-width="5"/>
    <text x="128" y="224" text-anchor="middle" fill="#94a3b8" font-size="10" font-family="sans-serif">${z}/${x}/${y}</te
xt>
</svg>`;
  return new Response(svg, {
    headers: {
      "Content-Type": "image/svg+xml",
      "Cache-Control": "public, max-age=300",
      "Access-Control-Allow-Origin": "",
      "X-Map-Fallback": "1",
    },
  });
}
export async function GET(
  _req: Request,
  ctx: { params: Promise<{ z: string; x: string; y: string }> },
) {
  const { z: rz, x: rx, y: ry } = await ctx.params;
  const z = Number(rz);
  const x = Number(rx);
  const y = Number(ry.replace(/\.png$/i, ""));
  if (
    !Number.isInteger(z) ||
    !Number.isInteger(x) ||
    !Number.isInteger(y) ||
    z < 1 ||
    z > 19 ||
    x < 0 ||
    y < 0 ||
    x >= 2 ** z ||
    y >= 2 ** z
  ) {
    return new Response("invalid tile", { status: 400 });
  }
  for (let i = 0; i < SOURCES.length; i++) {
    try {
      const res = await fetchWithRetry(
        SOURCES[i](z, x, y),
        { headers: TILE_HEADERS },
        { retries: 2, baseDelayMs: 250, maxDelayMs: 1200, timeoutMs: 9000, label: `tile-${i}` },
      );
      if (!res.ok) continue;
      const type = res.headers.get("content-type") ?? "";
      if (!type.startsWith("image/")) continue;
    }
  }
}

```

```

const bytes = new Uint8Array(await res.arrayBuffer());
if (bytes.length < 50) continue;
return new Response(bytes, {
  headers: {
    "Content-Type": type,
    "Content-Length": String(bytes.length),
    "Cache-Control": "public, max-age=604800, stale-while-revalidate=2592000",
    "Access-Control-Allow-Origin": "*",
    "X-Map-Source": String(i + 1),
  },
});
} catch {
  //
}
}
return fallbackTile(z, x, y);
}

```

```

@@ .env.ndcohub.example text
# =====
# — NdcoHub.ir ()
# .env
# =====
# --- ( Render) ---
DATABASE_URL=postgresql://USER:PASS@HOST:5432/DBNAME?sslmode=require
# --- ---
NODE_NAME=ndcohub
PUBLIC_BASE_URL=https://ndcohub.ir
# --- ---
#
SYNC_SECRET=-----
SYNC_PEERS=render|https://namayandeelemi-javad.onrender.com
SYNC_INTERVAL_MINUTES=5
# --- ---
NEXT_PUBLIC_ENDPOINTS=https://ndcohub.ir,https://namayandeelemi-javad.onrender.com
# --- ---
APP_SECRET=---
MOBILE_APP_SECRET=-----
# --- ---
ADMIN_PASSWORD=admin1234
ADMIN_PHONE=09120000000
BALE_BOT_TOKEN=
SMS_API_KEY=

```

```

@@ .gitlab-ci.yml yaml
# =====

# GitLab CI — GitHub NdcoHub
# =====
stages: [build, mirror, deploy, sync]
default:
  image: node:20
  cache:
    key: "$CI_COMMIT_REF_SLUG"
    paths: [.npm/, node_modules/]
variables:
  NEXT_PUBLIC_ENDPOINTS: "https://ndcohub.ir,https://namayandeelemi-javad.onrender.com"
  GIT_DEPTH: "0"
build:
  stage: build
  script:
    - npm ci --include=dev --cache .npm --prefer-offline
    - npx next typegen
    - npx tsc --noEmit
    - npm run build
  artifacts:
    expire_in: 1 day
    paths: [.next/, public/]
mirror-to-github:
  stage: mirror
  needs: [build]
  rules:
    - if: $CI_COMMIT_BRANCH == "main" || $CI_COMMIT_BRANCH == "master"
  script:
    - |
      if [ -z "$GITHUB_REPO_URL" ]; then echo "GITHUB_REPO_URL — "; exit 0; fi
      AUTH_URL=$(echo "$GITHUB_REPO_URL" | sed "s#https://#https://x-access-token:${GITHUB_TOKEN}@#")
      git remote add github "$AUTH_URL" || git remote set-url github "$AUTH_URL"
      git push github HEAD:$CI_COMMIT_BRANCH --force
      git push github --tags --force
deploy:
  stage: deploy
  needs: [build]
  rules:
    - if: $CI_COMMIT_BRANCH == "main" || $CI_COMMIT_BRANCH == "master"

```

```

script:
  - '[ -n "$NDCOHUB_DEPLOY_HOOK" ] && curl -fsSL -X POST "$NDCOHUB_DEPLOY_HOOK" && echo " NdcoHub" || echo "hook"'
  - '[ -n "$RENDER_DEPLOY_HOOK" ] && curl -fsSL -X POST "$RENDER_DEPLOY_HOOK" && echo " Render" || true'
sync-after-deploy:
  stage: sync
  needs: [deploy]
  rules:
    - if: $CI_COMMIT_BRANCH == "main" || $CI_COMMIT_BRANCH == "master"
  script:
    - |
      if [ -z "$SYNC_SECRET" ]; then echo "SYNC_SECRET "; exit 0; fi
      sleep 90
      curl -fsSL -X POST "https://ndcohub.ir/api/sync/run" -H "x-sync-key: $SYNC_SECRET" || true
      curl -fsSL -X POST "https://namayandeelmi-javad.onrender.com/api/sync/run" -H "x-sync-key: $SYNC_SECRET" || true

```

```

@@ drizzle.config.json json
{
  "dialect": "postgresql",
  "schema": "./src/db/schema.ts",
  "dbCredentials": {
    "url": "postgres://postgres:postgres@127.0.0.1:5432/app_db"
  }
}

```

```

@@ eslint.config.mjs javascript
import { defineConfig, globalIgnores } from "eslint/config";
import nextCoreWebVitals from "eslint-config-next/core-web-vitals";
export default defineConfig([
  // Keep the starter on the flat config export that actually runs under the pinned ESLint/Next toolchain.
  ...nextCoreWebVitals,
  globalIgnores([".next/**", "out/**", "build/**", "next-env.d.ts"]),
]);

```

```

@@ postcss.config.mjs javascript
const postcssConfig = {
  plugins: {
    "@tailwindcss/postcss": {},
  },
};

```

```

export default postcssConfig;

```

```

@@ render.yaml yaml
services:
  - type: web
    name: namayandeelmi
    runtime: node
    plan: free
    region: frankfurt
    branch: main
    # : Render npm start
    # "Could not find a production build in the '.next' directory" .
    buildCommand: npm install --include=dev && npm run build
    startCommand: npm run start
    healthCheckPath: /api/health
    autoDeploy: true
    envVars:
      - key: NODE_VERSION
        value: "20.18.0"
      - key: NODE_ENV
        value: production
      - key: NPM_CONFIG_PRODUCTION
        value: "false"
      # DATABASE_URL Neon ( Render)
      # : postgresql://user:pass@ep-xxx.eu-central-1.aws.neon.tech/neondb?sslmode=require
      - key: DATABASE_URL
        sync: false
      - key: APP_SECRET
        generateValue: true
      - key: ADMIN_PASSWORD
        sync: false
      - key: ADMIN_PHONE
        sync: false
      # ----
      - key: NODE_NAME
        value: render
      - key: SYNC_SECRET
        sync: false
      - key: SYNC_PEERS
        value: ndcohub|https://ndcohub.ir
      - key: SYNC_INTERVAL_MINUTES
        value: "5"
      - key: NEXT_PUBLIC_ENDPOINTS
        value: https://ndcohub.ir,https://namayandeelmi-javad.onrender.com

```

- key: PUBLIC_BASE_URL
value: https://namayandee.lmi-javad.onrender.com
- key: MOBILE_APP_SECRET
sync: false

@@ tsconfig.json json

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": [
      "dom",
      "dom.iterable",
      "esnext"
    ],
    "allowJs": false,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "react-jsx",
    "incremental": true,
    "baseUrl": ".",
    "paths": {
      "@/*": [
        "./src/*"
      ]
    },
    "plugins": [
      {
        "name": "next"
      }
    ]
  },
  "include": [
    "next-env.d.ts",
    "**/*.ts",

    "**/*.tsx",
    ".next/types/**/*.ts",
    ".next/dev/types/**/*.ts"
  ],
  "exclude": [
    "node_modules",
    "mobile"
  ]
}
```

@@ public/manifest.webmanifest text

```
{
  "id": "/?source=pwa",
  "name": " — ",
  "short_name": " ",
  "description": " ",
  "start_url": "/?source=pwa",
  "scope": "/",
  "display": "standalone",
  "display_override": [
    "window-controls-overlay",
    "standalone",
    "minimal-ui",
    "browser"
  ],
  "orientation": "any",
  "background_color": "#f1f5f9",
  "theme_color": "#0f766e",
  "dir": "rtl",
  "lang": "fa-IR",
  "categories": [
    "business",
    "productivity",
    "medical",
    "utilities"
  ],
  "prefer_related_applications": false,
  "icons": [
    {
      "src": "/icons/icon-96.png",
      "sizes": "96x96",
      "type": "image/png",
      "purpose": "any"
    }
  ]
}
```

```

    },
    {
      "src": "/icons/icon-128.png",
      "sizes": "128x128",
      "type": "image/png",
      "purpose": "any"
    },
    {
      "src": "/icons/icon-144.png",
      "sizes": "144x144",
      "type": "image/png",
      "purpose": "any"
    },
    {
      "src": "/icons/icon-152.png",
      "sizes": "152x152",
      "type": "image/png",
      "purpose": "any"
    },
    {
      "src": "/icons/icon-180.png",
      "sizes": "180x180",
      "type": "image/png",
      "purpose": "any"
    },
    {
      "src": "/icons/icon-192.png",
      "sizes": "192x192",
      "type": "image/png",
      "purpose": "any"
    },
    {
      "src": "/icons/icon-256.png",
      "sizes": "256x256",
      "type": "image/png",
      "purpose": "any"
    },
    {
      "src": "/icons/icon-384.png",
      "sizes": "384x384",
      "type": "image/png",
      "purpose": "any"
    },
    {
      "src": "/icons/icon-512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "any"
    },
    {
      "src": "/icons/icon-1024.png",
      "sizes": "1024x1024",
      "type": "image/png",
      "purpose": "any"
    },
    {
      "src": "/icons/maskable-192.png",
      "sizes": "192x192",
      "type": "image/png",
      "purpose": "maskable"
    },
    {
      "src": "/icons/maskable-512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "maskable"
    }
  ],
  "screenshots": [
    {
      "src": "/screenshots/desktop-1280x720.png",
      "sizes": "1280x720",
      "type": "image/png",
      "form_factor": "wide",
      "label": " "
    },
    {
      "src": "/screenshots/brand-1280x720.png",
      "sizes": "1280x720",
      "type": "image/png",
      "form_factor": "wide",
      "label": " "
    }
  ],
  {

```

```

        "src": "/screenshots/mobile-720x1280.png",
        "sizes": "720x1280",
        "type": "image/png",
        "form_factor": "narrow",
        "label": " "
    }
},
"shortcuts": [
    {
        "name": " ",
        "short_name": "",
        "url": "/panel/pharmacies",
        "icons": [
            {
                "src": "/icons/icon-192.png",
                "sizes": "192x192",
                "type": "image/png"
            }
        ]
    },
    {
        "name": " ",
        "short_name": "",
        "url": "/panel/doctors",
        "icons": [
            {
                "src": "/icons/icon-192.png",
                "sizes": "192x192",
                "type": "image/png"
            }
        ]
    },
    {
        "name": " ",
        "short_name": "",
        "url": "/panel/orders",
        "icons": [
            {
                "src": "/icons/icon-192.png",
                "sizes": "192x192",
                "type": "image/png"
            }
        ]
    },
    {
        "name": " ",
        "short_name": "",
        "url": "/panel/trip",
        "icons": [
            {
                "src": "/icons/icon-192.png",
                "sizes": "192x192",
                "type": "image/png"
            }
        ]
    }
],
"launch_handler": {
    "client_mode": "navigate-existing"
},
"edge_side_panel": {
    "preferred_width": 420
},
"handle_links": "preferred"
}

@@ public/style.css css
/* =====
   0000000000 00000 000000 0000000000 00000 000000000000 000000 0 00000000 (RTL)
   ===== */

@import url('https://fonts.googleapis.com/css2?family=Vazirmatn:wght@300;400;500;600;700;800&display=swap');

:root {
    --primary: #2563eb;
    --primary-hover: #1d4ed8;
    --primary-light: #eff6ff;
    --secondary: #64748b;
    --success: #10b981;
    --success-light: #ecfdf5;
    --warning: #f59e0b;
    --warning-light: #fffbeb;
    --danger: #ef4444;
    --danger-light: #fef2f2;

```

```

--info: #06b6d4;
--info-light: #ecfeff;
--bg-main: #f8fafc;
--card-bg: #ffffff;
--text-main: #1e293b;
--text-muted: #64748b;
--border-color: #e2e8f0;
--shadow-sm: 0 1px 2px 0 rgba(0, 0, 0, 0.05);
--shadow-md: 0 4px 6px -1px rgba(0, 0, 0, 0.1), 0 2px 4px -1px rgba(0, 0, 0, 0.06);
--shadow-lg: 0 10px 15px -3px rgba(0, 0, 0, 0.1), 0 4px 6px -2px rgba(0, 0, 0, 0.05);
--radius-md: 0.75rem;
--radius-lg: 1rem;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Vazirmatn', -apple-system, BlinkMacSystemFont, 'Segoe UI', Tahoma, Arial, sans-serif;
  background-color: var(--bg-main);
  color: var(--text-main);
  direction: rtl;
  text-align: right;
  line-height: 1.6;
  overflow-x: hidden;
}

/* Header & Navbar */
.app-header {
  background: linear-gradient(135deg, #0f766e 0%, #0d9488 100%);
  color: #ffffff;
  padding: 1rem 1.5rem;
  box-shadow: var(--shadow-md);
  position: sticky;
  top: 0;
  z-index: 1000;
}

.header-container {
  max-width: 1400px;
  margin: 0 auto;
  display: flex;
  align-items: center;
  justify-content: space-between;
  flex-wrap: wrap;
  gap: 1rem;
}

.logo-area {
  display: flex;
  align-items: center;
  gap: 0.75rem;
}

.logo-icon {
  font-size: 1.8rem;
  background: rgba(255, 255, 255, 0.2);
  width: 48px;
  height: 48px;
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: 12px;
  backdrop-filter: blur(4px);
}

.logo-text h1 {
  font-size: 1.25rem;
  font-weight: 700;
  margin-bottom: 2px;
}

.logo-text p {
  font-size: 0.75rem;
  opacity: 0.85;
}

.header-actions {
  display: flex;
  align-items: center;
  gap: 0.75rem;
}

```



```

.btn-header {
  background: rgba(255, 255, 255, 0.15);
  color: #fff;
  border: 1px solid rgba(255, 255, 255, 0.25);
  padding: 0.5rem 1rem;
  border-radius: var(--radius-md);
  font-family: inherit;
  font-size: 0.85rem;
  font-weight: 500;
  cursor: pointer;
  transition: all 0.2s ease;
  display: flex;
  align-items: center;
  gap: 0.4rem;
}

.btn-header:hover {
  background: rgba(255, 255, 255, 0.25);
  transform: translateY(-1px);
}

/* Navigation Tabs - Compact Wrapping Grid (Zero Left/Right Scroll) */
.app-nav {
  background: var(--card-bg);
  border-bottom: 2px solid var(--border-color);
  box-shadow: var(--shadow-sm);
  position: sticky;
  top: 76px;
  z-index: 999;
  overflow: hidden;
}

.nav-container {
  max-width: 1400px;
  margin: 0 auto;
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  justify-content: center;
  gap: 0.4rem;
  padding: 0.65rem 1rem;
}

.nav-item {
  padding: 0.45rem 0.85rem;
  border-radius: 8px;
  font-weight: 700;
  font-size: 0.84rem;
  color: var(--text-muted);
  background: #f8fafc;
  border: 1px solid var(--border-color);
  cursor: pointer;
  transition: all 0.18s ease;
  display: inline-flex;
  align-items: center;
  gap: 0.35rem;
  font-family: inherit;
  white-space: nowrap;
}

.nav-item:hover {
  background: #f0fdf4;
  border-color: #0d9488;
  color: #0d9488;
  transform: translateY(-1px);
}

.nav-item.active {
  background: #0d9488;
  color: #ffffff;
  border-color: #0f766e;
  box-shadow: 0 2px 6px rgba(13, 148, 136, 0.35);
}

.nav-badge {
  background: #e2e8f0;
  color: #334155;
  font-size: 0.72rem;
  padding: 0.1rem 0.45rem;
  border-radius: 999px;
  font-weight: 800;
}

.nav-item.active .nav-badge {

```

```

    background: #ffffff;
    color: #0f766e;
}

/* Main Layout & Cards */
.main-content {
    max-width: 1400px;
    margin: 1.5rem auto;
    padding: 0 1.5rem 3rem;
}

.tab-pane {
    display: none;
    animation: fadeIn 0.25s ease-in-out;
}

.tab-pane.active {
    display: block;
}

@keyframes fadeIn {
    from { opacity: 0; transform: translateY(6px); }
    to { opacity: 1; transform: translateY(0); }
}

.card {
    background: var(--card-bg);
    border-radius: var(--radius-lg);
    box-shadow: var(--shadow-md);
    border: 1px solid var(--border-color);
    padding: 1.5rem;
    margin-bottom: 1.5rem;
}

.card-header {
    display: flex;
    align-items: center;
    justify-content: space-between;
    margin-bottom: 1.25rem;
    padding-bottom: 0.75rem;
    border-bottom: 1px solid var(--border-color);
}

.card-title {
    font-size: 1.15rem;
    font-weight: 700;
    display: flex;
    align-items: center;
    gap: 0.5rem;
    color: #0f172a;
}

/* Top Sticky Alert Box for Existing Pharmacy in Orders (Requirement 3) */
#existingPharmacyTopAlert {
    background: linear-gradient(135deg, #ffffbeb 0%, #fef3c7 100%);
    border: 2px solid #f59e0b;
    border-radius: var(--radius-lg);
    padding: 1rem 1.25rem;
    margin-bottom: 1.5rem;
    box-shadow: 0 10px 15px -3px rgba(245, 158, 11, 0.2);
    display: flex;
    align-items: center;
    justify-content: space-between;
    flex-wrap: wrap;
    gap: 1rem;
    animation: slideDownAlert 0.35s ease-out;
}

@keyframes slideDownAlert {
    0% { opacity: 0; transform: translateY(-15px); }
    100% { opacity: 1; transform: translateY(0); }
}

.alert-info-content {
    display: flex;
    align-items: center;
    gap: 0.85rem;
    flex: 1;
    min-width: 280px;
}

.alert-info-icon {
    font-size: 2rem;
    background: #fde68a;
    width: 48px;

```

```

height: 48px;
display: flex;
align-items: center;
justify-content: center;
border-radius: 12px;
color: #d97706;
}

.alert-info-text h4 {
font-size: 1rem;
font-weight: 700;
color: #92400e;
margin-bottom: 4px;
}

.alert-info-text p {
font-size: 0.85rem;
color: #78350f;
margin: 0;
}

.btn-autofill-pharmacy {
background: linear-gradient(135deg, #10b981 0%, #059669 100%);
color: #ffffff;
border: none;
padding: 0.65rem 1.25rem;
border-radius: var(--radius-md);
font-family: inherit;
font-size: 0.9rem;
font-weight: 700;
cursor: pointer;
display: inline-flex;
align-items: center;
gap: 0.5rem;
box-shadow: 0 4px 6px -1px rgba(16, 185, 129, 0.3);
transition: all 0.2s ease;
}

.btn-autofill-pharmacy:hover {
transform: translateY(-2px);
box-shadow: 0 6px 12px -2px rgba(16, 185, 129, 0.4);
}

/* Forms & Inputs */
.form-grid {
display: grid;
grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
gap: 1.25rem;
}

.form-group {
display: flex;
flex-direction: column;
gap: 0.4rem;
}

.form-group.full-width {
grid-column: 1 / -1;
}

.form-label {
font-size: 0.88rem;
font-weight: 600;
color: var(--text-main);
display: flex;
align-items: center;
justify-content: space-between;
}

.form-input,
.form-select,
.form-textarea {
width: 100%;
padding: 0.65rem 0.9rem;
font-family: inherit;
font-size: 0.92rem;
color: var(--text-main);
background: #ffffff;
border: 1.5px solid var(--border-color);
border-radius: var(--radius-md);
transition: all 0.2s ease;
}

.form-input:focus,
.form-select:focus,

```

```

.form-textarea:focus {
  outline: none;
  border-color: var(--primary);
  box-shadow: 0 0 0 3px rgba(37, 99, 235, 0.15);
}

.form-textarea {
  min-height: 85px;
  resize: vertical;
}

/* Buttons */
.btn {
  padding: 0.65rem 1.25rem;
  font-family: inherit;
  font-size: 0.9rem;
  font-weight: 600;
  border-radius: var(--radius-md);
  border: none;
  cursor: pointer;
  transition: all 0.2s ease;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 0.5rem;
}

.btn-primary {
  background: var(--primary);
  color: #ffffff;
}
.btn-primary:hover { background: var(--primary-hover); }

.btn-success {
  background: var(--success);
  color: #ffffff;
}
.btn-success:hover { background: #059669; }

.btn-warning {
  background: var(--warning);
  color: #fff;
}
.btn-warning:hover { background: #d97706; }

.btn-danger {
  background: var(--danger);
  color: #ffffff;
}
.btn-danger:hover { background: #dc2626; }

.btn-outline {
  background: transparent;
  border: 1.5px solid var(--border-color);
  color: var(--text-main);
}
.btn-outline:hover {
  background: var(--bg-main);
  border-color: var(--text-muted);
}

.btn-sm {
  padding: 0.35rem 0.75rem;
  font-size: 0.8rem;
  border-radius: 6px;
}

.btn-add-option {
  font-size: 0.75rem;
  color: var(--primary);
  background: var(--primary-light);
  border: 1px dashed var(--primary);
  padding: 0.2rem 0.5rem;
  border-radius: 6px;
  cursor: pointer;
  font-weight: 600;
  transition: all 0.15s ease;
}

.btn-add-option:hover {
  background: var(--primary);
  color: #fff;
}

/* Map Containers */

```

```

.map-container {
  width: 100%;
  height: 380px;
  border-radius: var(--radius-lg);
  border: 2px solid var(--border-color);
  overflow: hidden;
  position: relative;
  z-index: 10;
  box-shadow: inset 0 2px 4px rgba(0, 0, 0, 0.05);
}

.map-container-large {
  height: 520px;
}

/* Custom Leaflet Map Tooltips & Permanent Labels (Requirement 5) */
.custom-map-tooltip {
  background: rgba(15, 23, 42, 0.92) !important;
  color: #ffffff !important;
  font-family: 'Vazirmatn', sans-serif !important;
  font-size: 0.82rem !important;
  font-weight: 700 !important;
  border: 1.5px solid rgba(255, 255, 255, 0.25) !important;
  border-radius: 8px !important;
  padding: 4px 10px !important;
  box-shadow: 0 4px 10px rgba(0, 0, 0, 0.25) !important;
  white-space: nowrap !important;
  direction: rtl !important;
}

.custom-map-tooltip:before {
  border-top-color: rgba(15, 23, 42, 0.92) !important;
}

/* Custom DivIcons for Pharmacies, Doctors, and Reps */
.marker-pharmacy-badge {
  background: #10b981;
  color: #ffffff;
  width: 38px;
  height: 38px;
  border-radius: 50%;
  border: 3px solid #ffffff;
  box-shadow: 0 3px 8px rgba(0, 0, 0, 0.35);
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 1.25rem;
  cursor: pointer;
  transition: transform 0.2s ease;
}

.marker-pharmacy-badge:hover {
  transform: scale(1.15);
}

.marker-doctor-badge {
  background: #2563eb;
  color: #ffffff;
  width: 38px;
  height: 38px;
  border-radius: 50%;
  border: 3px solid #ffffff;
  box-shadow: 0 3px 8px rgba(0, 0, 0, 0.35);
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 1.25rem;
  cursor: pointer;
  transition: transform 0.2s ease;
}

.marker-doctor-badge:hover {
  transform: scale(1.15);
}

.marker-rep-badge {
  background: #f59e0b;
  color: #ffffff;
  width: 38px;
  height: 38px;
  border-radius: 50%;
  border: 3px solid #ffffff;
  box-shadow: 0 3px 8px rgba(0, 0, 0, 0.35);
  display: flex;
  align-items: center;
}

```

```

    justify-content: center;
    font-size: 1.25rem;
    cursor: pointer;
    transition: transform 0.2s ease;
}

.marker-rep-badge:hover {
    transform: scale(1.15);
}

/* Tables */
.table-responsive {
    overflow-x: auto;
    border-radius: var(--radius-md);
    border: 1px solid var(--border-color);
}

.data-table {
    width: 100%;
    border-collapse: collapse;
    font-size: 0.9rem;
    text-align: right;
}

.data-table th,
.data-table td {
    padding: 0.85rem 1rem;
    border-bottom: 1px solid var(--border-color);
}

.data-table th {
    background: var(--bg-main);
    font-weight: 700;
    color: #334155;
    white-space: nowrap;
}

.data-table tr:hover {
    background: #f1f5f9;
}

/* Badges & Status */
.status-badge {
    display: inline-flex;
    align-items: center;
    gap: 0.35rem;
    padding: 0.25rem 0.65rem;
    border-radius: 999px;
    font-size: 0.78rem;
    font-weight: 700;
}

.status-online {
    background: var(--success-light);
    color: #047857;
}

.status-visiting {
    background: var(--warning-light);
    color: #b45309;
}

.status-offline {
    background: var(--danger-light);
    color: #b91c1c;
}

/* Dashboard Summary Cards */
.stats-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
    gap: 1.25rem;
    margin-bottom: 1.75rem;
}

.stat-card {
    background: var(--card-bg);
    border-radius: var(--radius-lg);
    padding: 1.25rem;
    box-shadow: var(--shadow-sm);
    border: 1px solid var(--border-color);
    display: flex;
    align-items: center;
    gap: 1rem;
}

```

```

.stat-icon {
  width: 56px;
  height: 56px;
  border-radius: 14px;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 1.75rem;
  flex-shrink: 0;
}

.stat-content h3 {
  font-size: 1.6rem;
  font-weight: 800;
  color: #0f172a;
  margin: 0;
}

.stat-content p {
  font-size: 0.85rem;
  color: var(--text-muted);
  margin: 0;
  font-weight: 600;
}

/* Backup & Restore Dropzone (Requirement 8) */
.dropzone-restore {
  border: 2px dashed #3b82f6;
  background: #fff6ff;
  border-radius: var(--radius-lg);
  padding: 2rem;
  text-align: center;
  cursor: pointer;
  transition: all 0.2s ease;
}

.dropzone-restore:hover {
  background: #dbeafe;
  border-color: #2563eb;
}

/* Modals */
.modal-overlay {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  background: rgba(15, 23, 42, 0.65);
  backdrop-filter: blur(4px);
  display: none;
  align-items: center;
  justify-content: center;
  z-index: 2000;
  padding: 1rem;
}

.modal-overlay.active {
  display: flex;
  animation: fadeIn 0.2s ease-out;
}

.modal-content {
  background: #ffffff;
  border-radius: var(--radius-lg);
  box-shadow: var(--shadow-lg);
  width: 100%;
  max-width: 600px;
  max-height: 90vh;
  overflow-y: auto;
  padding: 1.5rem;
}

/* Launchpad Grid on Dashboard (Zero Scroll Access to All 20 Modules) */
.launchpad-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(175px, 1fr));
  gap: 0.75rem;
}

.launchpad-card {
  background: #f8fafc;
  border: 1.5px solid var(--border-color);
  border-radius: 12px;

```

```

padding: 0.85rem;
display: flex;
align-items: center;
gap: 0.65rem;
cursor: pointer;
transition: all 0.2s ease;
font-family: inherit;
text-align: right;
width: 100%;
}

.launchpad-card:hover {
background: #f0fdf4;
border-color: #0d9488;
transform: translateY(-2px);
box-shadow: var(--shadow-md);
}

.launchpad-icon {
font-size: 1.6rem;
width: 42px;
height: 42px;
background: #ffffff;
border-radius: 10px;
display: flex;
align-items: center;
justify-content: center;
box-shadow: var(--shadow-sm);
flex-shrink: 0;
}

.launchpad-text h4 {
font-size: 0.9rem;
font-weight: 800;
color: #0f172a;
margin: 0;
}

/* =====
STYLES FOR SIDEBAR MENU DRAWER (Screenshot 1)
===== */

.btn-menu-hamburger {
background: rgba(255, 255, 255, 0.2);
color: #fff;
border: 1px solid rgba(255, 255, 255, 0.3);
width: 44px;
height: 44px;
border-radius: 10px;
font-size: 1.4rem;
display: flex;
align-items: center;
justify-content: center;
cursor: pointer;
transition: all 0.2s ease;
}

.btn-menu-hamburger:hover {
background: rgba(255, 255, 255, 0.3);
}

.side-menu-drawer {
position: fixed;
top: 0;
right: -320px;
width: 300px;
height: 100vh;
background: #0f766e;
color: #ffffff;
z-index: 3000;
box-shadow: -5px 0 25px rgba(0, 0, 0, 0.35);
transition: right 0.3s cubic-bezier(0.4, 0, 0.2, 1);
display: flex;
flex-direction: column;
overflow-y: auto;
}

.side-menu-drawer.active {
right: 0;
}

.side-menu-header {
padding: 1.25rem 1.5rem;
background: #115e59;
display: flex;
align-items: center;

```



```

    justify-content: space-between;
    border-bottom: 1px solid rgba(255, 255, 255, 0.15);
    position: sticky;
    top: 0;
    z-index: 10;
}

.btn-close-side {
    background: transparent;
    border: none;
    color: #fff;
    font-size: 1.5rem;
    cursor: pointer;
}

.side-menu-content {
    padding: 1rem 0;
    display: flex;
    flex-direction: column;
    gap: 0.25rem;
}

.side-menu-item {
    display: flex;
    align-items: center;
    justify-content: space-between;
    padding: 0.85rem 1.5rem;
    color: rgba(255, 255, 255, 0.9);
    background: transparent;
    border: none;
    font-family: inherit;
    font-size: 0.95rem;
    font-weight: 600;
    cursor: pointer;
    text-align: right;
    transition: all 0.2s ease;
    width: 100%;
}

.side-menu-item:hover,
.side-menu-item.active {
    background: rgba(255, 255, 255, 0.15);
    color: #ffffff;
    padding-right: 1.75rem;
}

.side-menu-overlay {
    position: fixed;
    top: 0;
    left: 0;
    right: 0;
    bottom: 0;
    background: rgba(0, 0, 0, 0.5);
    backdrop-filter: blur(3px);
    z-index: 2999;
    display: none;
}

.side-menu-overlay.active {
    display: block;
    animation: fadeIn 0.25s ease-out;
}

/* Header teal icons and badges matching Screenshot 1 */
.header-user-pill {
    display: flex;
    align-items: center;
    gap: 0.5rem;
    background: rgba(255, 255, 255, 0.15);
    padding: 0.4rem 0.85rem;
    border-radius: 999px;
    font-size: 0.85rem;
    font-weight: 600;
}

.badge-status-online {
    background: #10b981;
    color: #fff;
    font-size: 0.72rem;
    padding: 0.15rem 0.5rem;
    border-radius: 999px;
    font-weight: 700;
}

.btn-header-teal {

```

```

background: rgba(255, 255, 255, 0.22);
color: #fff;
border: 1px solid rgba(255, 255, 255, 0.4);
padding: 0.45rem 1rem;
border-radius: 999px;
font-family: inherit;
font-size: 0.85rem;
font-weight: 700;
cursor: pointer;
}

.btn-header-logout {
background: #0f766e;
color: #fff;
border: 1px solid rgba(255, 255, 255, 0.3);
padding: 0.45rem 0.85rem;
border-radius: 10px;
font-family: inherit;
font-size: 0.85rem;
font-weight: 700;
cursor: pointer;
}

.btn-header-icon {
background: transparent;
border: none;
color: #fff;
font-size: 1.3rem;
position: relative;
cursor: pointer;
}

.notif-count-badge {
position: absolute;
top: -4px;
right: -6px;
background: #ef4444;
color: #fff;
font-size: 0.65rem;
font-weight: 800;
width: 18px;
height: 18px;
border-radius: 50%;
display: flex;
align-items: center;
justify-content: center;
}

/* =====
STYLES FOR TOGGLES & PERMISSION BADGES (Screenshots 3, 4, 5, 6, 7)
===== */

.btn-toggle-option {
padding: 0.45rem 1.25rem;
border-radius: 8px;
border: 1px solid #d97706;
background: #ffffff;
color: #92400e;
font-family: inherit;
font-weight: 700;
font-size: 0.88rem;
cursor: pointer;
transition: all 0.2s ease;
}

.btn-toggle-option.active {
background: #1e3a8a;
color: #ffffff;
border-color: #1e3a8a;
box-shadow: 0 2px 5px rgba(30, 58, 138, 0.3);
}

.permission-group-title {
font-weight: 800;
color: #334155;
font-size: 0.95rem;
margin-bottom: 0.5rem;
border-bottom: 1px dashed var(--border-color);
padding-bottom: 0.35rem;
}

.permission-tag-chk {
display: inline-flex;
align-items: center;
gap: 0.4rem;
background: #f0fdf4;

```

```

border: 1px solid #bbf7d0;
color: #166534;
padding: 0.35rem 0.75rem;
border-radius: 999px;
font-size: 0.82rem;
font-weight: 700;
cursor: pointer;
transition: all 0.15s ease;
user-select: none;
}

.permission-tag-chk.unchecked {
background: #f8fafc;
border-color: #cbd5e1;
color: #64748b;
opacity: 0.75;
}

.permission-tag-chk:hover {
transform: translateY(-1px);
}

/* =====
   STYLES FOR BOTTOM OFFLINE DIAGNOSTIC BANNER (Screenshot 8)
   ===== */
.offline-diagnostic-banner {
position: fixed;
bottom: 0;
left: 0;
right: 0;
background: #f59e0b;
color: #ffffff;
padding: 1.25rem 1.5rem;
z-index: 5000;
box-shadow: 0 -8px 25px rgba(0, 0, 0, 0.3);
display: flex;
align-items: center;
justify-content: space-between;
flex-wrap: wrap;
gap: 1rem;
border-top: 4px solid #d97706;
animation: slideUpToast 0.35s ease-out;
}

@keyframes slideUpToast {
from { transform: translateY(100%); }
to { transform: translateY(0); }
}

.offline-banner-content {
display: flex;
align-items: center;
gap: 1rem;
flex: 1;
min-width: 280px;
}

.offline-icon {
font-size: 2.2rem;
background: rgba(255, 255, 255, 0.25);
width: 52px;
height: 52px;
border-radius: 14px;
display: flex;
align-items: center;
justify-content: center;
}

.offline-text h4 {
font-size: 1.1rem;
font-weight: 800;
margin-bottom: 4px;
}

.offline-text p {
font-size: 0.88rem;
opacity: 0.95;
margin: 0;
}

.offline-actions {
display: flex;
align-items: center;
gap: 0.65rem;
flex-wrap: wrap;
}

```

```

}

.btn-banner-retry {
  background: #ffffff;
  color: #d97706;
  border: none;
  padding: 0.5rem 1.1rem;
  border-radius: 8px;
  font-family: inherit;
  font-weight: 700;
  font-size: 0.88rem;
  cursor: pointer;
}

.btn-banner-diag {
  background: rgba(255, 255, 255, 0.25);
  color: #ffffff;
  border: 1px solid rgba(255, 255, 255, 0.5);
  padding: 0.5rem 1.1rem;
  border-radius: 8px;
  font-family: inherit;
  font-weight: 700;
  font-size: 0.88rem;
  cursor: pointer;
}

.btn-banner-close {
  background: transparent;
  color: #ffffff;
  border: none;
  font-size: 0.9rem;
  font-weight: 600;
  cursor: pointer;
  padding: 0.4rem 0.75rem;
}

/* =====
   STYLES FOR PERSIAN JALALI CALENDAR POPUP PICKER (Matching Screenshot)
   ===== */

.jalali-input-wrapper {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  position: relative;
  width: 100%;
}

.jalali-badge {
  background: #ffffff;
  border: 1.5px solid #0d9488;
  border-radius: 10px;
  padding: 0.25rem 0.5rem;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  min-width: 48px;
  cursor: pointer;
  box-shadow: var(--shadow-sm);
  transition: all 0.2s ease;
  user-select: none;
}

.jalali-badge:hover {
  background: #f0fdf4;
  transform: translateY(-1px);
}

.jalali-badge-header {
  background: #ef4444;
  color: #fff;
  font-size: 0.62rem;
  font-weight: 800;
  padding: 1px 6px;
  border-radius: 4px;
  margin-bottom: 2px;
  text-transform: uppercase;
}

.jalali-badge-day {
  font-size: 1.1rem;
  font-weight: 800;
  color: #0f172a;
  line-height: 1.1;
}

```

```

.jalali-helper-text {
  font-size: 0.78rem;
  color: #64748b;
  margin-top: 4px;
}

.jalali-popup-picker {
  position: absolute;
  top: 100%;
  right: 0;
  width: 310px;
  background: #ffffff;
  border: 2px solid #0d9488;
  border-radius: 16px;
  box-shadow: 0 12px 30px rgba(0, 0, 0, 0.25);
  padding: 1rem;
  z-index: 4000;
  display: none;
  direction: rtl;
}

.jalali-popup-picker.active {
  display: block;
  animation: fadeIn 0.2s ease-out;
}

.jalali-picker-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 0.5rem;
  margin-bottom: 0.85rem;
}

.jalali-nav-arrow {
  background: #f1f5f9;
  border: none;
  color: #334155;
  font-size: 1.1rem;
  font-weight: 800;
  width: 32px;
  height: 32px;
  border-radius: 8px;
  cursor: pointer;
  display: flex;
  align-items: center;
  justify-content: center;
}

.jalali-nav-arrow:hover {
  background: #e2e8f0;
}

.jalali-header-selects {
  display: flex;
  gap: 0.4rem;
  flex: 1;
  justify-content: center;
}

.jalali-select {
  padding: 0.3rem 0.6rem;
  border: 1px solid var(--border-color);
  border-radius: 8px;
  font-family: inherit;
  font-size: 0.88rem;
  font-weight: 700;
  color: #0f172a;
  background: #f8fafc;
}

.jalali-weekdays {
  display: grid;
  grid-template-columns: repeat(7, 1fr);
  text-align: center;
  font-size: 0.78rem;
  font-weight: 800;
  color: #64748b;
  margin-bottom: 0.5rem;
  border-bottom: 1px solid var(--border-color);
  padding-bottom: 0.35rem;
}

.jalali-days-grid {

```

```

display: grid;
grid-template-columns: repeat(7, 1fr);
gap: 0.3rem;
margin-bottom: 0.85rem;
}

.jalali-day-cell {
background: transparent;
border: none;
border-radius: 8px;
padding: 0.4rem 0;
text-align: center;
font-size: 0.9rem;
font-weight: 700;
color: #1e293b;
cursor: pointer;
transition: all 0.15s ease;
font-family: inherit;
}

.jalali-day-cell:hover {
background: #f0fdf4;
color: #0d9488;
}

.jalali-day-cell.active {
background: #0d9488;
color: #ffffff;
font-weight: 800;
box-shadow: 0 2px 6px rgba(13, 148, 136, 0.35);
}

.jalali-day-cell.empty {
cursor: default;
background: transparent;
}

.jalali-today-btn {
width: 100%;
background: #f8fafc;
border: 1.5px solid #cbd5e1;
border-radius: 10px;
padding: 0.55rem;
font-family: inherit;
font-size: 0.88rem;
font-weight: 800;
color: #0f172a;
cursor: pointer;
transition: all 0.2s ease;
}

.jalali-today-btn:hover {
background: #f0fdf4;
border-color: #0d9488;
color: #0d9488;
}

```

@@ scripts/build-sw.mjs javascript

#!/usr/bin/env node

/**

```

*      build.
*
*      React   HTML
*      .       HTML
*      ()      .
*      JS/CSS  build
*/

```

```

import { readdirSync, readFileSync, statSync, writeFileSync, existsSync } from "node:fs";
import path from "node:path";
const ROOT = process.cwd();
const NEXT_STATIC = path.join(ROOT, ".next", "static");
const SW_SRC = path.join(ROOT, "public", "sw-template.js");
const SW_OUT = path.join(ROOT, "public", "sw.js");
function walk(dir, base) {
  const out = [];
  if (!existsSync(dir)) return out;
  for (const name of readdirSync(dir)) {
    const full = path.join(dir, name);
    const st = statSync(full);
    if (st.isDirectory()) out.push(...walk(full, `${base}/${name}`));
    else out.push({ url: `${base}/${name}`, size: st.size });
  }
  return out;
}

```

```

const PAGES = [
  "/",
  "/login",
  "/offline",
  "/install",
  "/diagnostics",
  "/panel",
  "/panel/pharmacies",
  "/panel/doctors",
  "/panel/orders",
  "/panel/trip",
  "/panel/map",
  "/panel/home",
  "/panel/leaves",
  "/panel/notifications",
  "/panel/options",
  "/panel/reports",
  "/admin",
  "/admin/activity",
  "/admin/records/pharmacies",
  "/admin/records/doctors",
  "/admin/records/orders",
  "/admin/map",
  "/admin/live",
  "/admin/trips",
  "/admin/homes",
  "/admin/leaves",
  "/admin/notifications",
  "/admin/reports",
  "/admin/options",
  "/admin/columns",

  "/admin/users",
  "/admin/messengers",
  "/admin/backup",
];
const EXTRAS = [
  "/manifest.webmanifest",
  "/logo.svg",
  "/icons/icon-192.png",
  "/icons/icon-512.png",
  "/apple-touch-icon.png",
  "/data/iran-provinces.geojson",
];
function main() {
  if (!existsSync(SW_SRC)) {
    console.warn("  sw-template.js  sw.js  .");
    return;
  }
  const assets = walk(NEXT_STATIC, "_next/static")
    .filter((a) => /\.?(js|css|woff2?)$/i.test(a.url))
    // ( )
    .filter((a) => a.size < 700 * 1024)
    .map((a) => a.url)
    .sort();
  const total = walk(NEXT_STATIC, "_next/static").reduce((s, a) => s + a.size, 0);
  const build = `${Date.now().toString(36)}-${assets.length}`;
  // RSC
  const rscPages = PAGES.map((p) => `${p}${p.includes("?") ? "&" : "?"}_rsc=offline`);
  let sw = readFileSync(SW_SRC, "utf8");
  sw = sw
    .replace("__BUILD_ID__", build)
    .replace("__PAGES__", JSON.stringify(PAGES, null, 2))
    .replace("__RSC_PAGES__", JSON.stringify(rscPages, null, 2))
    .replace("__ASSETS__", JSON.stringify(assets, null, 2))
    .replace("__EXTRAS__", JSON.stringify(EXTRAS, null, 2));
  writeFileSync(SW_OUT, sw);
  console.log(
    `  : ${PAGES.length} + ${assets.length}    (${Math.round(total / 1024)} )
  `
  );
}
main();

@@ scripts/generate-assets.mjs javascript
#!/usr/bin/env node
/**
 *      PWA      SVG.
 *
 *      PNG      /
 * PWABuilder «Fix the links to your icons».
 *      build
 *      .
 */
import { existsSync, mkdirSync, writeFileSync } from "node:fs";

```

```

import path from "node:path";
const ROOT = process.cwd();
const PUB = path.join(ROOT, "public");
const ICONS = path.join(PUB, "icons");
const SHOTS = path.join(PUB, "screenshots");
const LOGO_SVG = `

```



```

        `<rect x="60" y="${590 + i * 32}" width="1160" height="26" rx="8" fill="#f8fafc"/>` +
        `<text x="1200" y="${608 + i * 32}" font-size="13" fill="#334155" text-anchor="end" font-family="${F}">${[" — ", "
— ", " — "][i]}</text>`,
    )
    .join("")))
</svg>`;
const NARROW = `<svg xmlns="http://www.w3.org/2000/svg" width="720" height="1280">
<rect width="720" height="1280" fill="#f1f5f9"/>
${header(720, " ")}
${card(24, 92, 672, 96)}
<text x="672" y="130" font-size="20" font-weight="bold" fill="#0f172a" text-anchor="end" font-family="${F}">
</text>
<text x="672" y="162" font-size="14" fill="#64748b" text-anchor="end" font-family="${F}"> // —
</text>
<rect x="24" y="208" width="672" height="92" rx="18" fill="#0f766e"/>
<text x="668" y="248" font-size="19" font-weight="bold" fill="ffffff" text-anchor="end" font-family="${F}">
</text>
<text x="668" y="276" font-size="13" fill="#c7f0ea" text-anchor="end" font-family="${F}"> </text>
<rect x="24" y="312" width="672" height="92" rx="18" fill="#0369a1"/>
<text x="668" y="352" font-size="19" font-weight="bold" fill="ffffff" text-anchor="end" font-family="${F}">
</text>

<text x="668" y="380" font-size="13" fill="#cfe9fb" text-anchor="end" font-family="${F}"> </text>
<rect x="24" y="416" width="672" height="92" rx="18" fill="#4f46e5"/>
<text x="668" y="456" font-size="19" font-weight="bold" fill="ffffff" text-anchor="end" font-family="${F}">
</text>
<text x="668" y="484" font-size="13" fill="#d8d6fb" text-anchor="end" font-family="${F}"> </text>
${["", "", "", "", "", ""]}
    .map((t, i) => {
        const x = 24 + (i % 3) * 228;
        const y = 528 + Math.floor(i / 3) * 116;
        return `${card(x, y, 208, 100)}<text x="${x + 104}" y="${y + 60}" font-size="16" font-weight="bold" fill="#334155" t
ext-anchor="middle" font-family="${F}">${t}</text>`;
    })
    .join(""));
${card(24, 772, 672, 300)}
<text x="668" y="806" font-size="16" font-weight="bold" fill="#0f172a" text-anchor="end" font-family="${F}">
</text>
${bars(56, 830, 610, 210, [30, 48, 42, 70, 62, 88, 96, 78, 110, 96, 124, 140], "#0ea5e9")}
${card(24, 1092, 672, 92)}
<text x="668" y="1128" font-size="15" font-weight="bold" fill="#0f766e" text-anchor="end" font-family="${F}">
</text>
<text x="668" y="1156" font-size="13" fill="#64748b" text-anchor="end" font-family="${F}"> + G
PS</text>
<rect x="0" y="1208" width="720" height="72" fill="#0f766e"/>
<text x="360" y="1252" font-size="15" font-weight="bold" fill="ffffff" text-anchor="middle" font-family="${F}">
— </text>
</svg>`;
const BRAND = `<svg xmlns="http://www.w3.org/2000/svg" width="1280" height="720">
<defs><linearGradient id="bg" x1="0" y1="0" x2="1" y2="1">
<stop offset="0%" stop-color="#0f766e"/><stop offset="100%" stop-color="#134e4a"/></linearGradient></defs>
<rect width="1280" height="720" fill="url(#bg)"/>
<text x="640" y="560" font-size="46" font-weight="bold" fill="ffffff" text-anchor="middle" font-family="${F}">
</text>
<text x="640" y="612" font-size="22" fill="#a7f3d0" text-anchor="middle" font-family="${F}"> —
</text>
</svg>`;
export const ICON_SIZES = [96, 128, 144, 152, 180, 192, 256, 384, 512, 1024];
const plate = (size) =>
    Buffer.from(
        `<svg xmlns="http://www.w3.org/2000/svg" width="${size}" height="${size}">
        <defs><linearGradient id="g" x1="0" y1="0" x2="1" y2="1">
        <stop offset="0%" stop-color="ffffff"/><stop offset="100%" stop-color="#eef2f6"/>
        </linearGradient></defs>
        <rect width="${size}" height="${size}" fill="url(#g)"/>
        </svg>`,
    );
/**
 */
export async function makeIcon(sharp, size) {
    return sharp(Buffer.from(LOGO_SVG), { density: 500 })
        .resize(size, size, { fit: "contain", background: { r: 0, g: 0, b: 0, alpha: 0 } })
        .png({ compressionLevel: 9 })
        .toBuffer();
}
/**
 */
export async function makeMaskable(sharp, size) {
    const inner = await sharp(Buffer.from(LOGO_SVG), { density: 500 })
        .resize(Math.round(size * 0.62), Math.round(size * 0.62))
        .png()
        .toBuffer();
    return sharp(plate(size)).composite([
        { input: inner, gravity: "center" }
    ]).png({ compressionLevel: 9 }).toBuffer();
}
export async function makeScreenshot(sharp, key) {
    if (key === "mobile-720x1280") return sharp(Buffer.from(NARROW)).png().toBuffer();
    if (key === "brand-1280x720") {

```

```

    const logo = await sharp(Buffer.from(LOGO_SVG), { density: 500 }).resize(300, 300).png().toBuffer();
    return sharp(Buffer.from(BRAND)).composite([ { input: logo, top: 140, left: 490 } ]).png().toBuffer();
  }
  return sharp(Buffer.from(WIDE)).png().toBuffer();
}
export { LOGO_SVG };
async function main() {
  let sharp;
  try {
    sharp = (await import("sharp")).default;
  } catch {
    console.warn(" sharp          .");
    return;
  }
}

mkdirSync(ICONS, { recursive: true });
mkdirSync(SHOTS, { recursive: true });
writeFileSync(path.join(PUB, "logo.svg"), LOGO_SVG);
writeFileSync(path.join(PUB, "icon.svg"), LOGO_SVG);
// SVG () PNG PNG
//
mkdirSync(path.join(PUB, "src-svg"), { recursive: true });
writeFileSync(path.join(PUB, "src-svg", "desktop-1280x720.svg"), WIDE);
writeFileSync(path.join(PUB, "src-svg", "mobile-720x1280.svg"), NARROW);
writeFileSync(path.join(PUB, "src-svg", "brand-1280x720.svg"), BRAND);
for (const s of ICON_SIZES) {
  writeFileSync(path.join(ICONS, `icon-${s}.png`), await makeIcon(sharp, s));
}
for (const s of [192, 512]) {
  writeFileSync(path.join(ICONS, `maskable-${s}.png`), await makeMaskable(sharp, s));
}
writeFileSync(path.join(PUB, "apple-touch-icon.png"), await makeMaskable(sharp, 180));
writeFileSync(path.join(PUB, "favicon-32.png"), await makeIcon(sharp, 32));
writeFileSync(path.join(PUB, "favicon-16.png"), await makeIcon(sharp, 16));
for (const k of ["desktop-1280x720", "mobile-720x1280", "brand-1280x720"]) {
  writeFileSync(path.join(SHOTS, `${k}.png`), await makeScreenshot(sharp, k));
}
console.log(
  ` PWA : ${ICON_SIZES.length} + maskable + (${existsSync(ICONS) ? "ok" : "err"})`,
);
}
if (import.meta.url === `file://${process.argv[1]}`) {
  main().catch((e) => {
    console.error(" : ", e?.message ?? e);
    process.exit(0); // build
  });
}

@@ scripts/simplify-geojson.mjs javascript
#!/usr/bin/env node
/** Ramer-Douglas-Peucker */
import { readFileSync, writeFileSync, statSync } from "node:fs";
const input = process.argv[2] || "public/data/iran-provinces.geojson";
const output = process.argv[3] || "public/data/iran-provinces-lite.geojson";
const tolerance = Number(process.argv[4] || 0.012);
const distanceToSegment = (p, a, b) => {
  const dx = b[0] - a[0];
  const dy = b[1] - a[1];
  if (!dx && !dy) return Math.hypot(p[0] - a[0], p[1] - a[1]);
  const t = Math.max(0, Math.min(1, ((p[0] - a[0]) * dx + (p[1] - a[1]) * dy) / (dx * dx + dy * dy)));
  return Math.hypot(p[0] - (a[0] + t * dx), p[1] - (a[1] + t * dy));
};
function simplify(points, eps) {
  if (points.length <= 4) return points;
  const wasClosed = points[0][0] === points.at(-1)[0] && points[0][1] === points.at(-1)[1];
  const line = wasClosed ? points.slice(0, -1) : points;
  //
  let split = 0;
  if (wasClosed) {
    let max = -1;
    for (let i = 1; i < line.length; i++) {
      const d = Math.hypot(line[i][0] - line[0][0], line[i][1] - line[0][1]);
      if (d > max) { max = d; split = i; }
    }
    const a = rdp(line.slice(0, split + 1), eps);
    const b = rdp([...line.slice(split), line[0]], eps);
    const result = [...a.slice(0, -1), ...b];
    return result.length >= 4 ? result : points;
  }
  return rdp(line, eps);
}
function rdp(points, eps) {
  if (points.length <= 2) return points;
  let max = 0;
  let idx = 0;

```

```

const first = points[0];
const last = points.at(-1);
for (let i = 1; i < points.length - 1; i++) {
  const d = distanceToSegment(points[i], first, last);
  if (d > max) { max = d; idx = i; }
}
if (max > eps) {
  const left = rdp(points.slice(0, idx + 1), eps);
  const right = rdp(points.slice(idx), eps);
  return [...left.slice(0, -1), ...right];
}
return [first, last];
}
const geo = JSON.parse(readFileSync(input, "utf8"));
for (const feature of geo.features) {
  const g = feature.geometry;
  if (g.type === "Polygon") g.coordinates = g.coordinates.map((ring) => simplify(ring, tolerance));
  if (g.type === "MultiPolygon") {
    g.coordinates = g.coordinates.map((poly) => poly.map((ring) => simplify(ring, tolerance)));
  }
  const raw = String(feature.properties?.province_name || feature.properties?.name || "");
  const name = raw.replace(/^\s+/, "");
  feature.properties.name_fa = name === " " ? " " : name;
}
writeFileSync(output, JSON.stringify(geo));
console.log(`${geo.features.length} : ${statSync(input).size} → ${statSync(output).size} bytes`);

```

@@ scripts/start.mjs javascript

#!/usr/bin/env node

/**

* Self-healing production starter.

*

* (Render Build Command) `npm start`

* `next`

* "Could not find a production build in the '.next' directory" .

* `next build`

*

*/

import { spawn } from "node:child_process";

import { existsSync } from "node:fs";

import path from "node:path";

import process from "node:process";

const root = process.cwd();

const buildIdFile = path.join(root, ".next", "BUILD_ID");

const port = process.env.PORT || "3000";

const host = process.env.HOST || "0.0.0.0";

const nextBin = path.join(root, "node_modules", "next", "dist", "bin", "next");

function run(args, label) {

return new Promise((resolve, reject) => {

const child = spawn(process.execPath, [nextBin, ...args], {

stdio: "inherit",

env: process.env,

});

child.on("exit", (code, signal) => {

if (signal) return reject(new Error(`\${label} terminated by \${signal}`));

if (code !== 0) return reject(new Error(`\${label} exited with code \${code}`));

resolve();

});

child.on("error", reject);

});

}

async function main() {

if (!existsSync(buildIdFile)) {

console.log(" production build not found – running `next build` first...");

await run(["build"], "next build");

console.log(" build finished, starting server...");

}

console.log(` starting Next.js on \${host}:\${port}`);

await run(["start", "-H", host, "-p", String(port)], "next start");

}

main().catch((err) => {

console.error(" start failed:", err.message);

process.exit(1);

});

@@ public/data/iran-provinces-lite.geojson text

```

{"type":"FeatureCollection","features":[{"type":"Feature","properties":{"province_name":" ","name_fa":" "},
  "geometry":{"type":"Polygon","coordinates":[[[58.0196353,36.5830045],[57.9382428,36.5841636],[57.7773976,
36.6302737],[57.4898883,36.7691637],[57.03292,36.8713972],[56.7212399,36.7988951],[56.6512373,36.7677781],[56.5320652,36
.6675034],[56.5017855,36.6644417],[56.4465028,36.6941673],[56.3595572,36.6827762],[56.3651105,36.7530568],[56.2645516,36
.8380601],[56.2214921,36.9008362],[56.0145378,36.8344501],[55.9842616,36.9000082],[55.9791784,36.9962453],[56.0145379,37
.0069571],[56.0400978,37.1002915],[55.9942616,37.1191819],[56.0628744,37.1608486],[56.0087049,37.2344567],[55.9514823,37
.2044565],[55.9087027,37.2055724],[55.908423,37.3323405],[56.013985,37.366409],[56.0012054,37.4425221],[56.0584315,37.51
91926],[56.0612082,37.5722467],[56.118711,37.6347476],[56.2156603,37.8594822],[56.2803861,37.8672577],[56.3103894,37.910

```

3136], [56.2889756, 38.0846583], [56.335248, 38.0772496], [56.352079, 38.1171643], [56.319101, 38.176137], [56.3327792, 38.1975537

], [56.442778, 38.26], [56.546667, 38.266111], [56.613889, 38.240556], [56.756389, 38.285], [56.8404712, 38.2298997], [56.9919985, 38.2143439], [57.0122329, 38.1871898], [57.025, 38.201944], [57.076111, 38.199444], [57.183675, 38.2841776], [57.216285, 38.2804549], [57.243889, 38.270833], [57.288333, 38.170556], [57.371111, 38.093056], [57.356944, 37.985833], [57.5121923, 37.919263], [57.5796241, 37.9380422], [57.7203676, 37.9218564], [57.7541404, 37.8974949], [57.7958352, 37.899553], [57.8254738, 37.8632465], [57.8915765, 37.8707624], [58.0278554, 37.8059025], [58.2054052, 37.7743691], [58.2287711, 37.7296962], [58.2270467, 37.6840365], [58.3112048, 37.662222], [58.3709134, 37.5846745], [58.3549337, 37.4083545], [58.3160405, 37.3558537], [58.3274269, 37.32585], [58.4229856, 37.2563997], [58.3896527, 37.2119583], [58.3243732, 37.1994596], [58.3049269, 37.1680631], [58.2429806, 37.1811205], [58.1590148, 37.0751305], [58.2440936, 37.046954], [58.2496468, 37.0200059], [58.2927063, 37.002787], [58.3010527, 36.9644639], [58.2588132, 36.9108352], [58.1343611, 36.8647246], [58.0988083, 36.8688929], [58.1107479, 36.7727824], [58.0879746, 36.722503], [58.1032478, 36.63139], [58.0824144, 36.5999959], [58.0196353, 36.5830045]]], {"type": "Feature", "properties": {"province_name": " ", "name_fa": " "}, "geometry": {"type": "Polygon", "coordinates": [[[60.533076, 34.0002495], [60.3344603, 33.9838216], [60.1830615, 33.8668734], [60.1441686, 33.856312], [60.0880562, 33.9277046], [59.9912849, 34.004841], [59.610536, 34.0154875], [59.5360996, 34.1040964], [59.347193, 34.1079869], [59.2594074, 34.1499297], [59.162462, 34.1190981], [59.0441236, 34.1238187], [58.9796708, 34.0946565], [58.8638325, 34.1143781], [58.7857734, 34.0818761], [58.7057745, 34.099937], [58.6363217, 34.0216001], [58.4652041, 34.1274391], [58.3707267, 34.2893055], [58.3201997, 34.3146603], [58.2640871, 34.3213297], [58.1890811, 34.2416029], [58.0710226, 34.2349424], [58.0351832, 34.2391031], [57.9160113, 34.32161], [57.7660067, 34.3471594], [57.6404487, 34.5205058], [57.5566901, 34.598], [57.7021148, 34.774395], [57.7865737, 34.822289], [57.7115619, 34.9199611], [57.5729435, 35.0001039], [57.552944, 35.0591252], [57.5151718, 35.0713477], [57.3176536, 35.0960721], [57.1526455, 35.0755155], [57.0006939, 34.9980118], [56.8798552, 34.9799613], [56.8337425, 34.954405], [56.820686, 34.9374544], [56.8515221, 34.9191232], [56.7901295, 34.8374511], [56.7398501, 34.7827327], [56.6723442, 34.7543938], [56.4384468, 34.682448], [56.2279736, 34.7104172], [56.4884468, 35.0285743], [56.6265119, 35.0649624], [56.6909577, 35.0624638], [56.8437427, 35.1977425], [56.9370816, 35.221075], [57.0002296, 35.2152486], [57.0520871, 35.3149691], [56.9912513, 35.441914], [57.0001414, 35.4874687], [57.0476409, 35.5355282], [57.0348643, 35.5641375], [56.7217953, 35.8599842], [56.7312388, 36.0899851], [56.800408, 36.2386072], [56.7331824, 36.313053], [56.6984597, 36.3883299], [56.8167982, 36.4619424], [56.8640245, 36.5208313], [56.8615248, 36.5680528], [56.7692958, 36.6158298], [56.7584628, 36.7038922], [56.6020678, 36.6691693], [56.5351185, 36.6783365], [56.6587373, 36.7727843], [56.7212399, 36.7988951], [57.0295866, 36.8716763], [57.4898883, 36.7691637], [57.5887801, 36.7341645], [57.7521249, 36.6405581], [57.9360229, 36.5847219], [58.0824144, 36.5999959], [58.1057475, 36.6455556], [58.0879746, 36.722503], [58.1107479, 36.7727824], [58.0988083, 36.8688929], [58.1343611, 36.8647246], [58.2588132, 36.9108352], [58.3010527, 36.9644639], [58.2927063, 37.002787], [58.2496468, 37.0200059], [58.2440936, 37.046954], [58.1590148, 37.0751305], [58.2429806, 37.1811205], [58.3049269, 37.1680631], [58.3243732, 37.1994596], [58.382706, 37.2077922], [58.4204854, 37.2439017], [58.3168736, 37.3561793], [58.3549337, 37.4083545], [58.3709134, 37.5846745], [58.3112048, 37.662222], [58.3596512, 37.6611066], [58.3884341, 37.6269846], [58.474167, 37.6405581], [58.55, 37.705833], [58.712778, 37.648611], [58.804444, 37.666944], [58.822778, 37.703611], [58.876389, 37.669167], [59.060278, 37.63], [59.2292497, 37.5120499], [59.3534794, 37.5284162], [59.392131, 37.4791374], [59.3814207, 37.4497464], [59.4113447, 37.4357739], [59.377877, 37.4075897], [59.3968977, 37.3173029], [59.4334955, 37.3054859], [59.4834259, 37.248484], [59.491833, 37.2300429], [59.4747378, 37.2147737], [59.5517307, 37.1775677], [59.5525408, 37.136204], [59.6433988, 37.1192367], [59.6534908, 37.1422099], [59.6789985, 37.1412196], [59.7474053, 37.1077803], [59.7718478, 37.1253495], [59.8995292, 37.0514459], [59.9814679, 37.0471137], [60.0430687, 37.0207635], [60.0890045, 36.9717244], [60.1042036, 36.9244661], [60.266905, 36.7572095], [60.3455423, 36.6293939], [60.6752205, 36.6222116], [61.1461942, 36.646493], [61.1860031, 36.5643208], [61.1822189, 36.525769], [61.1568736, 36.5074018], [61.1757576, 36.4641629], [61.166353, 36.4135861], [61.1425, 36.392778], [61.1604472, 36.3305686], [61.1491692, 36.3174795], [61.2193283, 36.1923075], [61.2251307, 36.1405553], [61.2106077, 36.1277446], [61.2309759, 36.1175896], [61.164722, 36.035278], [61.17, 35.995278], [61.1171678, 35.9621307], [61.1836284, 35.9430414], [61.245, 35.897222], [61.261944, 35.813333], [61.246389, 35.7175], [61.2166545, 35.675801], [61.2737591, 35.6094121], [61.2825792, 35.5379033], [61.22207, 35.442909], [61.227161, 35.421655], [61.187459, 35.399592], [61.200429, 35.373588], [61.17575, 35.343314], [61.190296, 35.292865], [61.157867, 35.275352], [61.108955, 35.282812], [61.091775, 35.263737], [61.09836, 35.2185], [61.117854, 35.209644], [61.08717, 35.164807], [61.13257, 35.135873], [61.141486, 35.104899], [61.126077, 35.095387], [61.145771, 35.080611], [61.117421, 35.073975], [61.125996, 35.058858], [61.1117, 35.060622], [61.099702, 35.024205], [61.121625, 35.003515], [61.062264, 34.917265], [61.073857, 34.908623], [61.055943, 34.904862], [61.080891, 34.876279], [61.075061, 34.844264], [61.002802, 34.7092165], [61.000798, 34.642976], [60.984492, 34.619758], [60.922214, 34.623018], [60.897272, 34.56726], [60.84865, 34.542077], [60.739083, 34.518086], [60.809274, 34.471653], [60.849631, 34.368448], [60.918373, 34.304756], [60.6720322, 34.3027538], [60.66949, 34.272504], [60.585931, 34.21266], [60.5374215, 34.150697], [60.51788, 34.0724107], [60.533076, 34.0002495]]]]], {"type": "Feature", "properties": {"province_name": " ", "name_fa": " "}, "geometry": {"type": "Polygon", "coordinates": [[[59.9397084, 30.7612473], [59.8774826, 30.7098487], [59.4635747, 30.5167946], [59.1938393, 30.8168007], [59.0785612, 30.9079166], [59.0585552, 31.008478], [58.780493, 31.1554155], [58.3899192, 31.4765441], [58.0782371, 31.6587712], [57.5334915, 31.8234911], [57.1026374, 31.8996052], [56.9395766, 31.9540503], [56.7734587, 31.9557208], [56.6048378, 31.922106], [56.5967812, 31.9996114], [56.6534506, 32.0873839], [56.5437256, 32.2315637], [56.3623284, 32.4057319], [56.221487, 32.5937882], [55.9720338, 32.7396312], [55.8764718, 32.7532385], [55.8075795, 32.7874056], [55.6861878, 32.931025], [55.6672981, 32.9987977], [55.5117337, 33.2515794], [55.386452, 33.3649236], [55.497844, 33.5290919], [55.4786782, 33.2518854], [55.9195327, 34.2457751], [56.2279736, 34.7104172], [56.4323336, 34.6818977], [56.6115081, 34.7324514], [56.7398501, 34.7827327], [56.8495757, 34.9149554], [56.8442988, 34.9352351], [56.8198526, 34.9391233], [56.8645754, 34.9744045], [57.0006939, 34.9980118], [57.1526455, 35.0755155], [57.3176536, 35.0960721], [57.5462775, 35.0638518], [57.5729435, 35.0001039], [57.7043352, 34.9260685], [57.7860073, 34.8238449], [57.7021148, 34.774395], [57.5566901, 34.598], [57.6404487, 34.5205058], [57.7660067, 34.3471594], [57.9160113, 34.32161], [58.0351832, 34.2391031], [58.0710226, 34.2349424], [58.1890811, 34.2416029], [58.2640871, 34.3213297], [58.3201997, 34.3146603], [58.3707267, 34.2893055], [58.4652041, 34.1274391], [58.6363217, 34.0216001], [58.7057745, 34.099937], [58.7857734, 34.0818761], [58.8638325, 34.1143781], [58.9796708, 34.0946565], [59.0441236, 34.1238187], [59.162462, 34.1190981], [59.2594074, 34.1499297], [59.347193, 34.1079869], [59.5360996, 34.1040964], [59.610536, 34.0154875], [59.9912849, 34.004841], [60.0880562, 33.9277046], [60.1441686, 33.856312], [60.1830615, 33.8668734], [60.3344603, 33.9838216], [60.533076, 34.0002495], [60.5822135, 33.8054437], [60.5468937, 33.7314095], [60.5514636, 33.6400296], [60.5812773, 33.5970233], [60.7034182, 33.54552], [60.9027952, 33.5399665], [60.9459035, 33.5134944], [60.9287078, 33.4972033], [60.8824864, 33.5052436], [60.8509231, 33.4820012], [60.8664836, 33.421487], [60.6340548, 33.2281412], [60.5833775, 33.1299045], [60.6294139, 33.0212334], [60.7095814, 32.6958555], [60.814771, 32.4812948], [60.882921, 32.197736], [60.8159668, 32.0265393], [60.832519, 31.979859], [60.8134835, 31.7087013], [60.8569712, 31.4632045], [60.8355847, 31.4140384], [60.7430788, 31.3290335], [60.6583536, 31.2859724], [60.677834, 31.2245871], [60.2780581, 31.1518107], [60.2486117, 31.1098626], [60.299171, 31.0226379], [60.2008322, 30.9401374], [59.9774885, 30.9198562], [59.9519285, 30.8823533], [59.9397084, 30.7612473]]]]], {"type": "Feature", "properties": {"province_name": " ", "name_fa": " "}, "geometry": {"type": "Polygon", "coordinates": [[[61.5497327, 24.8465103], [61.3546759, 24.8662768], [60.5765756, 25.0727683], [60.4193982, 25.0794386], [59.5677381, 25.1954933], [59.2187761, 25.1994033], [59.2104926, 25.5578014], [59.2357725, 25.6367001], [59.2113266, 25.6914167], [59.218833, 25.7405878], [59.0835491, 25.8508862], [59.072156, 25.8944817], [59.9977108, 25.937814], [59.0190967, 25.9733706], [59.0804892, 25.996152], [59.0827163, 26.0303127], [59.0218772, 26.1308698], [59.0288239, 26.166987], [58.9829846, 26.2127627], [59.9760453, 26.28671], [58.9432658, 26.3355961], [58.8796535, 26.353383], [58.8843786, 26.4236558], [58.8438142, 26.4831065], [58.9224331, 26.5150435], [59.0003551, 26.5003289], [59.0060451, 26.5281025], [59.0374382, 26.5475481], [58.9996588, 26.596439], [59.0177119, 26.6561592], [58.9904926, 26.75922], [58.9057608, 26.7872999], [58.8507615, 26.8311605], [58.8643748, 26.9086636], [58.9129876, 26.9999296], [58.8485422, 27.1017277], [58.8310364, 27.2536735], [58.9010426, 27.5764568], [58.8904901, 27.8981235], [58.9324362, 27.9731322], [58.9338231, 28.0298011], [59.0145546, 28.1070656], [59.1483843, 28.1273763], [59.1614392, 28.2507186], [59.1261603, 28.2984927], [59.1225478, 28.3632191], [59.0619893, 28.4162781], [59.2305063, 28.4764732], [59.536348, 28.4731397], [59.5652411, 28.5098105], [59.546075, 28.6236971], [59.5627415, 28.6903676], [59.5505155, 28.8159292], [59.4969032, 28.96093], [59.5055168, 28.9998199], [59.5707959, 29.0820391], [59.4944042

,29.2445422],[59.4635717,29.3742691],[59.4335736,29.8717785],[59.4494066,29.9934475],[59.4249612,30.2526259],[59.4635747,30.5167946],[59.8774826,30.7098487],[59.9422086,30.7656877],[59.9519285,30.8823533],[59.9774885,30.9198562],[60.1655526

,30.9315265],[60.2219452,30.9512486],[60.299171,31.0226379],[60.2486117,31.1098626],[60.2755585,31.1498609],[60.3877834,31.2245871],[60.6583536,31.2859724],[60.7430788,31.3290335],[60.8372511,31.4162593],[60.856057,31.4865192],[61.7066673,31.375896],[61.7783849,31.302694],[61.7716308,31.2062588],[61.8045461,31.1657792],[61.8390347,31.036451],[61.8238446,31.0008681],[61.8347019,30.9740985],[61.785528,30.9332587],[61.8016237,30.8325765],[60.874444,29.855556],[61.1625238,29.5333387],[61.308947,29.395202],[61.360123,29.378458],[61.372156,29.321058],[61.352909,29.275983],[61.430218,29.199095],[61.412681,29.175421],[61.420823,29.135652],[61.474371,29.123403],[61.471443,29.090872],[61.507259,29.061686],[61.478861,29.023639],[61.533361,29.010222],[61.6455716,28.8029274],[61.910018,28.564377],[62.043906,28.510134],[62.389486,28.432248],[62.488771,28.373405],[62.512278,28.322855],[62.597191,28.24842],[62.7929605,28.2789044],[62.7610215,28.0259231],[62.8289545,27.7644629],[62.8484229,27.4629391],[62.791328,27.338783],[62.824803,27.299191],[62.80882,27.264235],[62.775572,27.252896],[62.820731,27.218458],[62.873027,27.228859],[62.936974,27.20144],[63.040552,27.23729],[63.198716,27.259436],[63.224699,27.225621],[63.281777,27.213732],[63.3332704,27.1184003],[63.3071253,27.1296461],[63.2510944,27.0921506],[63.246334,26.849903],[63.185979,26.836726],[63.208945,26.77333],[63.191485,26.770481],[63.19692,26.647126],[63.108061,26.652392],[63.048857,26.627541],[62.989971,26.649943],[62.824409,26.637995],[62.7947,26.651003],[62.7535368,26.6450164],[62.738103,26.613237],[62.651571,26.610976],[62.604098,26.576273],[62.445165,26.566895],[62.4211762,26.5331578],[62.370225,26.539378],[62.303304,26.508287],[62.306943,26.462772],[62.284882,26.439747],[62.257767,26.446225],[62.259498,26.408339],[62.284695,26.380344],[62.250133,26.346581],[62.271944,26.328003],[62.228244,26.329538],[62.215011,26.267929],[62.125388,26.273986],[62.098288,26.285776],[62.085181,26.318861],[62.033364,26.334491],[62.021768,26.302815],[61.961143,26.260217],[61.896325,26.261399],[61.832838,26.059377],[61.828691,26.031244],[61.846411,26.018672],[61.818359,25.967194],[61.831706,25.911402],[61.85684,25.914788],[61.890722,25.883199],[61.84219,25.877674],[61.820408,25.849211],[61.851539,25.816431],[61.836235,25.798511],[61.840067,25.74484],[61.798234,25.717887],[61.736329,25.74172],[61.672325,25.647835],[61.6505818,25.3058125],[61.5755479,25.0453876],[61.5497327,24.8465103]]]},"type":"Feature","properties":{"province_name":"","name_fa":"","geometry":{"type":"Polygon","coordinates":[[[51.9649046,34.8652387],[51.7332272,35.0016298],[51.5993289,35.0177316],[51.5815524,35.0488522],[51.48877,35.1069061],[51.1451407,35.1805155],[51.0123591,35.1380151],[50.8692974,35.1596891],[50.8490093,35.1730286],[50.890687,35.2274604],[50.9615226,35.417188],[50.9920724,35.4371339],[50.8956036,35.4850921],[50.7676246,35.4869185],[50.6259494,35.5474747],[50.353419,35.5706023],[50.3722676,35.6189298],[50.3970214,35.6383536],[50.7015885,35.6741458],[50.7935226,35.7036144],[50.9976279,35.70036],[51.0298331,35.7422542],[51.0882389,35.7222773],[51.0941937,35.7577693],[51.1082599,35.7517637],[51.0951403,35.8122012],[51.2432017,35.9399772],[51.4130707,35.8931068],[51.4065431,35.977209],[51.4568225,36.00608238],[51.4592965,36.1400995],[51.760449,36.0066529],[51.8782311,35.8599847],[52.0726856,35.7633173],[52.2374138,35.7696997],[52.2921399,35.8155375],[52.3782555,35.8485917],[52.4357581,35.9122089],[52.5679864,35.943313],[52.7049383,35.9280361],[52.9368923,35.8338723],[53.0174514,35.8535946],[53.0471743,35.8199818],[53.1532932,35.8113644],[53.1527365,35.7919297],[52.999675,35.6813629],[52.8771663,35.6260885],[52.801054,35.5494151],[52.6977153,35.5519216],[52.555207,35.4119141],[52.3829791,35.3380212],[52.2576937,35.4405264],[52.0351831,35.4935832],[51.9857369,35.575526],[51.9296242,35.5585843],[51.9124013,35.4144115],[51.9439291,35.3613021],[51.8729552,35.2783007],[51.954901,35.1507968],[52.0346303,35.0905171],[52.0446302,35.0496826],[52.0015707,34.9185647],[51.9649046,34.8652387]]]},"type":"Feature","properties":{"province_name":"","name_fa":"","geometry":{"type":"Polygon","coordinates":[[[50.4676139,34.1904934],[50.4284443,34.211324],[50.4245543,34.2774425],[50.3601083,34.3488291],[50.3317721,34.4068837],[50.3351052,34.4421688],[50.1464877,34.4752237],[50.1570406,34.5091175],[50.0862046,34.5835621],[50.1081511,34.6238331],[50.1220405,34.7555112],[50.1689866,34.8216255],[50.2534357,34.8141193],[50.3412177,34.7599505],[50.4284433,34.7541222],[50.451223,34.8269037],[50.4892791,34.8602302],[50.705123,34.8682868],[50.6923434,34.9518966],[50.6498404,34.9996803],[50.6537302,35.0182898],[50.8490093,35.1730286],[50.9165201,35.1432915],[51.0123591,35.1380151],[51.1034746,35.178017],[51.1704203,35.1782967],[51.48877,35.1069061],[51.5815524,35.0488522],[51.5993289,35.0177316],[51.7332272,35.0016298],[51.9649046,34.8652387],[51.9143454,34.6969028],[51.8403823,34.5880611],[51.8341678,34.5059559],[51.3259864,34.441059],[51.1273654,34.4327213],[51.0115269,34.3721675],[51.0140271,34.2874043],[50.9948608,34.234664],[50.9081887,34.1518826],[50.8848555,34.1913332],[50.815125,34.2047764],[50.7962399,34.2360537],[50.727624,34.2152142],[50.6509551,34.2557736],[50.5956723,34.2168837],[50.4676139,34.1904934]]]},"type":"Feature","properties":{"province_name":"","name_fa":"","geometry":{"type":"Polygon","coordinates":[[[50.2948234,35.5610801],[50.2295442,35.541361],[50.1631517,35.6105336],[50.2437107,35.6477508],[50.250403,35.806938],[50.2970462,35.8224764],[50.3731618,35.8983085],[50.4259411,36.0016496],[50.5473363,36.0183135],[50.5776122,36.0824867],[50.6781745,36.1152612],[50.4320636,36.2452451],[50.409664,36.3022661],[50.4348301,36.3288809],[50.4959428,36.3422142],[50.7498434,36.3180525],[50.9506879,36.2622126],[51.2015319,36.1624881],[51.2679243,36.1841587],[51.315984,36.1544326],[51.4592965,36.1400995],[51.4568225,36.00608238],[51.4065431,35.977209],[51.4130707,35.8931068],[51.2432017,35.9399772],[51.0951403,35.8122012],[51.1082599,35.7517637],[51.0941937,35.7577693],[51.0882389,35.7222773],[51.0298331,35.7422542],[50.9976279,35.70036],[50.8353366,35.7086742],[50.7015885,35.6741458],[50.3970214,35.6383536],[50.353419,35.5706023],[50.2948234,35.5610801]]]},"type":"Feature","properties":{"province_name":"","name_fa":"","geometry":{"type":"Polygon","coordinates":[[[46.9310485,36.7552725],[46.8732689,36.7866601],[46.8988245,36.8947151],[46.8427122,36.9275011],[46.8307648,36.9675048],[46.8627115,36.999729],[46.7893754,37.0541661],[46.7063165,37.0722294],[46.6421472,37.0411149],[46.4607487,37.0202777],[46.4960282,37.0919479],[46.4476919,37.1013887],[46.4124124,37.0563881],[46.2862944,36.9716711],[46.1987926,36.9444408],[46.1774022,36.9552699],[46.175456,36.9955512],[46.0046132,37.1505605],[45.7523794,37.1822271],[45.7808896,37.228471],[45.5946336,37.4257989],[45.3563683,37.7386848],[45.292854,37.8604891],[45.2801507,37.942032],[45.3749078,38.0026729],[45.3484723,38.1245645],[45.30281,38.1583172],[45.1280592,38.1915139],[45.0820642,38.3325346],[45.2104025,38.4300311],[45.2698514,38.4417052],[45.339577,38.4864231],[45.2929078,38.5339291],[45.2334572,38.66504],[45.2756837,38.7319877],[45.2654031,38.8550443],[45.329295,38.8922705],[45.3926311,38.9714417],[45.4529047,38.9929965],[45.5415028,38.9648285],[45.580174,38.9789688],[45.6145737,38.9416943],[45.6924464,38.9514945],[45.736821,38.9275348],[45.797968,38.9249278],[45.8152735,38.9052549],[45.8493614,38.9095575],[45.9087076,38.8739169],[45.9445223,38.8965928],[46.0120046,38.8718894],[46.0654204,38.8874386],[46.1476006,38.8418111],[46.1841363,38.8414876],[46.3461404,38.9168198],[46.4210438,38.8830219],[46.4534998,38.8990073],[46.5225976,38.8889067],[46.5339832,38.867172],[46.6549773,38.9540911],[46.6969383,39.0190302],[46.7530928,39.025914],[46.7683432,39.0704193],[46.8342366,39.1144444],[46.8390194,39.1374974],[46.9255441,39.1647287],[46.9624954,39.1403575],[47.0191103,39.1670014],[47.0539374,39.1939118],[47.0580694,39.2398185],[47.0983303,39.2767338],[47.315114,39.3691936],[47.363399,39.4217256],[47.439111,39.3372671],[47.4307925,39.3186376],[47.5049349,39.2954356],[47.4810658,39.2725645],[47.518569,39.2197857],[47.5194023,39.1502891],[47.4974563,39.1283969],[47.5274594,39.0806154],[47.5052366,39.0353314],[47.5230132,39.0142262],[47.50218,39.0000537],[47.5630161,39.9775567],[47.5266267,38.9586645],[47.5363466,38.9500526],[47.5116235,38.9158902],[47.5980199,38.8189437],[47.4696776,38.7914396],[47.4313449,38.7564406],[47.4357886,38.7003296],[47.5257943,38.6697677],[47.564954,38.6422788],[47.5688509,38.6117084],[47.4985682,38.5811588],[47.4641225,38.4780964],[47.345784,38.3569797],[47.2907813,38.3319841],[47.2871719,38.302263],[47.3985677,38.2378125],[47.4688466,38.2494817],[47.4827366,38.1819793],[47.6171887,38.2183641],[47.7246942,38.2122557],[47.7874733,38.2514228],[47.8521997,38.1466985],[47.8622001,38.006699],[47.9097032,37.9908649],[47.9377595,37.946916],[47.887757,37.8978047],[47.8930368,37.8683574],[47.977486,37.877486],[47.999403,37.8949567],[48.1511045,37.8103041],[48.1352716,37.7522446],[48.0666557,37.7294704],[48.0255429,37.654462],[48.0216532,37.6122422],[48.0791556,37.5783566],[48.0530467,37.5333525],[48.0608231,37.5147348],[48.1552721,37.4783524],[48.1494391,37.416124],[48.2255533,37.3661248],[48.2441653,37.3269628],[48.2600016,37.329455],[48.3383375,37.2483443],[48.1502733,37.2211248],[48.0997139,37.1558395],[48.0411012,37.1494499],[47.9993331,37.112217],[47.9724852,37.1552812],[47.8719263,37.1747277],[47.7788675,37.1322247],[47.7660877,37.0763991],[47.663029,37.0994478],[47.59886,37.0375081],[47.5241339,37.0672274],[47.4924644,37.0000012],[47.3027333,36.8972278],[47.1982853,36.7994371],[47.1741153,36.791667],[47.1405

026,36.8322183],[47.1024463,36.8324973],[47.0218869,36.7908292],[47.0029977,36.7466648],[46.9310485,36.7552725]]]}},{"type":"Feature","properties":{"province_name":"","name_fa":""},"geometry":{"type":"Polygon","coordinates":[[[54.4816862,29.6409672],[54.2427864,29.5962233],[54.1030585,29.7928965],[54.0980585,29.9384533],[53.9055514,30.106788],[53.8644386,30.2026288],[53.9255511,30.3340175],[53.8986047,30.4456825],[53.9308275,30.5401367],[53.8794384,30.630138],[53.734987,30.6890253],[53.4585844,30.7373638],[53.3774721,30.8254161],[52.9693979,31.0145934],[52.9118953,31.1212617],[52.8335564,31.1790434],[52.8505028,31.2143124],[52.9457847,31.240988],[52.9782876,31.3226504],[52.9507846,31.3898795],[52.8010566,31.4948743],[52.9977306,31.4962647],[53.1355152,31.5423805],[53.2035743,31.5901582],[53.1402385,31.6804336],[53.0530131,31.7459903],[53.009677,31.8546066],[53.0424564,31.8989882],[53.058846,32.3321159],[53.051066,32.4146177],[53.0107865,32.5090657],[53.1866274,32.6176809],[53.4624737,32.6951803],[53.842492,32.6860155],[53.941384,32.7988012],[54.2241771,32.8988003],[54.283623,32.9601897],[54.3075126,33.0301923],[54.3483488,33.0529635],[54.5500228,33.0907497],[54.7447535,33.0657485],[54.790313,33.1179718],[54.8972616,33.1801987],[54.9917104,33.1257457],[55.2225541,33.324088],[55.386452,33.3649236],[55.5117337,33.2515794],[55.6672981,32.9987977],[55.6861878,32.931025],[55.8075795,32.7874056],[55.8764718,32.7532385],[55.9720338,32.7396312],[56.221487,32.5937882],[56.3623284,32.4057319],[56.5912282,32.1740559],[56.6545639,32.0812801],[56.5967812,31.9996114],[56.6065043,31.8682121],[56.5792811,31.7115449],[56.538725,31.7076627],[56.2556554,31.5534935],[56.1605022,31.5526083],[56.1211953,31.5906712],[56.0350885,31.5704307],[56.0128652,31.4648737],[55.9639761,31.4307118],[55.9267531,31.3673728],[55.7400756,31.2745965],[55.6009042,31.2437592],[55.399507,31.1518177],[54.9433733,31.0276458],[54.7116966,31.0659818],[54.5333558,31.0143137],[54.5594686,30.5826324],[54.4800197,30.4784661],[54.3366818,30.3851253],[54.547802,30.1781876],[54.541132,29.8917816],[54.5619684,29.6270564],[54.4816862,29.6409672]]]}},{"type":"Feature","properties":{"province_name":"","name_fa":""},"geometry":{"type":"Polygon","coordinates":[[[59.2187761,25.1994033],[59.0569858,25.2037298],[58.031375,25.3618275],[57.6780601,25.4491451],[57.1858621,25.6190402],[57.1125809,25.685279],[57.0835873,25.7454101],[57.0712282,25.8426445],[57.0020189,25.9007986],[56.9624421,25.9743274],[56.9514165,26.1048542],[56.8713617,26.2448854],[56.8160077,26.4318077],[56.8296427,26.5188522],[56.8591859,26.5687237],[56.8452011,26.6470807],[56.8084558,26.7407944],[56.6707157,26.9030693],[56.5868216,26.7419718],[56.5462498,26.7026737],[56.2417336,26.5902874],[56.1322549,26.5037413],[55.8933406,26.4122357],[55.4229986,26.0700522],[55.2766237,25.8867867],[55.26661847,25.7944904],[55.2077397,25.7132112],[55.1285553,25.6699815],[55.0008802,25.6543418],[54.8669759,25.7069237],[54.6504956,25.7147905],[54.5300633,25.6925179],[54.3932537,25.7216287],[54.2936202,25.8178637],[54.2696951,25.9814492],[54.223547,26.076023],[53.9642391,26.2946269],[53.849842,26.3222684],[53.1282214,26.6061774],[52.482398,26.989407],[52.7379214,27.2743065],[52.8165666,27.3237282],[52.8852276,27.2928411],[52.9457867,27.3292368],[53.0496784,27.2750621],[53.1307937,27.2720136],[53.3535802,27.1911741],[53.4096961,27.1139499],[53.4960887,27.1011746],[53.5419239,27.1578432],[53.6322058,27.0983966],[53.7933232,27.045617],[53.9333276,27.1236778],[54.0349994,27.230064],[54.015833,27.274235],[53.9963867,27.2731199],[53.9708304,27.3070144],[53.9686071,27.3500664],[54.1004482,27.408124],[54.3530677,27.4586846],[54.6289134,27.4678509],[55.1256022,27.430071],[55.2567201,27.4956291],[55.4370044,27.642573],[55.4714501,27.7186889],[55.5672887,27.7814665],[55.5800652,27.8086906],[55.5006199,27.9234117],[55.386725,27.998693],[55.4078383,28.4184168],[55.4511744,28.5523166],[55.3508892,28.6603707],[55.2539408,28.7217641],[55.247831,28.7470381],[55.5189535,28.6937067],[55.8156322,28.8195388],[56.0511988,28.880375],[56.0603652,28.7298187],[56.0364752,28.3762016],[56.0557361,28.2946235],[56.1042574,28.2348024],[56.255095,28.1611992],[56.4445487,28.1306398],[56.6520588,28.1431368],[56.7490074,28.2653651],[56.992627,28.241195],[57.0081829,27.9739722],[57.2040265,27.8947957],[57.416533,27.890628],[57.4251464,27.8100748],[57.4751451,27.7925702],[57.3256941,27.7064581],[57.2230512,27.5768406],[57.2290414,27.5600461],[57.2608172,27.5655853],[57.3015782,27.5980461],[57.3587125,27.574488],[57.3090081,27.4685003],[57.3251405,27.437571],[57.316254,27.3606232],[57.3868129,27.2661754],[57.5440368,27.1845091],[57.5982091,27.1011722],[57.748769,27.0001759],[57.7984922,26.9431154],[57.9918356,26.9156125],[58.1918457,26.7792228],[58.4354681,26.7322748],[58.6451978,26.5586596],[58.8438142,26.4831065],[58.8843736,26.4236558],[58.8796535,26.3533833],[58.9432658,26.3359611],[58.9760453,26.28671],[58.9829846,26.2172627],[59.0288239,26.166987],[59.0218772,26.1308698],[59.0827163,26.0303127],[59.0804892,25.996152],[59.0190967,25.9733706],[58.9977108,25.937814],[59.072156,25.8944817],[59.0835491,25.8508662],[59.218833,25.7405878],[59.2113266,25.6914167],[59.2357725,25.6367001],[59.2104926,25.5578014],[59.2187761,25.1994033]]]}},{"type":"Feature","properties":{"province_name":"","name_fa":""},"geometry":{"type":"Polygon","coordinates":[[[49.5801433,29.6674426],[49.4643006,29.7895783],[48.8544653,29.7477043],[48.826035,29.8117841],[48.7164401,29.8544406],[48.6150246,29.9323151],[48.4629249,29.9910509],[48.438333,30.0725],[48.3827499,30.1231946],[48.4159566,30.1705652],[48.408056,30.211389],[48.2727326,30.3360874],[48.2027331,30.3205769],[48.182222,30.399444],[48.1511605,30.4351201],[48.0244151,30.4773465],[48.0321895,30.9953741],[47.6873876,31.0029569],[47.682971,31.394609],[47.865855,31.783613],[47.852568,31.818223],[47.805337,31.842578],[47.80986,31.881373],[47.731711,31.939258],[47.705397,31.994321],[47.712418,32.018895],[47.690236,32.013243],[47.664111,32.048542],[47.8100625,32.116073],[47.9850235,32.2638935],[47.9619751,32.2943871],[47.9717102,32.3274573],[48.0356255,32.3553508],[48.0160638,32.5233567],[47.9805985,32.5363511],[47.9425773,32.5170105],[47.9558342,32.5357321],[47.9328002,32.5349499],[47.944046,32.561572],[47.9006195,32.5683061],[47.9328576,32.5824091],[47.9152184,32.6027823],[47.9883271,32.6114121],[47.9546661,32.6353339],[47.9691466,32.6582799],[48.0881156,32.6893842],[48.0639953,32.7087511],[48.0860481,32.7196815],[48.0599975,32.7403727],[48.0682412,32.7888861],[48.0490266,32.8057852],[48.1288223,32.8704693],[48.1186829,32.8990325],[48.1327477,32.9171759],[48.1796875,32.9009407],[48.2745972,32.9963338],[48.389679,32.9654538],[48.5056663,32.9616899],[48.5471992,32.9836302],[48.632473,32.9747246],[48.7093164,32.936459],[48.718581,32.8960203],[48.7686882,32.922986],[48.7648849,32.9683579],[48.8307382,32.9485216],[48.8538629,32.9108387],[48.8998756,32.8849407],[49.0061035,32.915383],[49.0138666,32.8994067],[49.0553438,32.9051092],[49.1370774,32.8698313],[49.1719742,32.8839216],[49.2179907,32.8521658],[49.3692588,32.8435754],[49.4283178,32.7793093],[49.5043142,32.7382883],[49.5157779,32.7248254],[49.5000772,32.7030932],[49.5868493,32.6092272],[49.9078903,32.320275],[49.9368208,32.1860741],[50.1182176,32.0164292],[50.2396202,31.9401716],[50.2903518,31.8704122],[50.2831,31.8500772],[50.4470368,31.6452259],[50.4575423,31.6141558],[50.4269791,31.5934585],[50.3923798,31.6013915],[50.3584136,31.5550185],[50.3751105,31.5412341],[50.2540909,31.4483382],[50.2479222,31.4182174],[50.2998317,31.3584786],[50.2467612,31.306489],[50.1527063,31.2518317],[50.099861,31.2575942],[49.95467,31.2114047],[49.9899942,31.1256725],[49.947515,31.0748626],[49.97143,31.0129259],[49.8862115,30.9642596],[49.9236754,30.9277065],[49.9511414,30.9293713],[49.9555053,30.877364],[50.0333405,30.8713077],[50.0697479,30.8299317],[50.2104225,30.9322635],[50.2434539,30.8325063],[50.2315138,30.7905283],[50.2541769,30.7555763],[50.3354417,30.7087156],[50.392841,30.7313077],[50.4237277,30.6651337],[50.5339013,30.5996434],[50.5495662,30.5630552],[50.5060119,30.5305816],[50.3864555,30.524084],[50.4526215,30.4322936],[50.5207594,30.3904245],[50.520584,30.1655169],[50.4165763,30.1821897],[50.2000071,30.2949624],[50.1390342,30.1759444],[49.5801433,29.6674426]]]}},{"type":"Feature","properties":{"province_name":"","name_fa":""},"geometry":{"type":"Polygon","coordinates":[[[58.8438142,26.4831065],[58.6451978,26.5586596],[58.4354681,26.7322748],[58.1918457,26.7792228],[57.9918356,26.9156125],[57.7984922,26.9431154],[57.748769,27.0001759],[57.5982091,27.1011722],[57.5440368,27.1845091],[57.3868129,27.2661754],[57.316254,27.3606232],[57.3251405,27.437571],[57.3090081,27.4685003],[57.3587125,27.574488],[57.3015782,27.5980461],[57.2608172,27.5655853],[57.2290414,27.5600461],[57.2230512,27.5768406],[57.3256941,27.7064581],[57.4751451,27.7925702],[57.4251464,27.8100748],[57.416533,27.890628],[57.2040265,27.8947957],[57.0081829,27.9739722],[56.992627,28.241195],[56.7490074,28.2653651],[56.6520588,28.1431368],[56.4445487,28.1306398],[56.255095,28.1611992],[56.1042574,28.2348024],[56.0557361,28.2946235],[56.0364752,28.3762016],[56.0548061,28.8793005],[55.8156322,28.8195388],[55.5189535,28.6937067],[55.247831,28.7470381],[55.2772774,29.033435],[55.2622777,29.0945466],[55.1653258,29.2523276],[54.983932,29.3948318],[54.8161445,29.5937251],[54.5614116,29.6264973],[54.541132,29.8917816],[54.5450221,30.1892939],[54.3366818,30.3809655],[54.4800197,30.4784661],[54.5594686,30.5826324],[54.5333558,31.0143137],[54.7116966,31.0659818],[54.9464299,31.0281963],[55.2106096,31.0912565],[55.494789,31.185976],[55.6009042,31.2437592],[55.7400756,31.2745965],[55.9384197,31.3768158],[55.9639761,31.4307118],[56.0128652,31.4648737],[56.0350885,31.5704307],[56.1211953,31.5906712],[56.1605022,31.5526083],[56.2556554,31.5534935],[56.538725,31.7076627],[56.5790044,31.7112741],[56.6048378,31.922106],[56.8851306,31.960163],[57.3393144,31.84

8772], [57.5334915, 31.8234911], [58.0671235, 31.6640438], [58.3899192, 31.4765441], [58.780493, 31.1554155], [59.0680015, 31.0000833], [59.0785612, 30.9079166], [59.1938393, 30.8168007], [59.4682947, 30.4995767], [59.4249612, 30.2526259], [59.4494066, 29.9934475], [59.4335736, 29.8717785], [59.4635717, 29.3742691], [59.4904042, 29.2445422], [59.5707959, 29.0820391], [59.5055168, 28.9998199], [59.4969032, 28.96093], [59.5505155, 28.8159292], [59.5627415, 28.6903676], [59.546075, 28.6236971], [59.5660743, 28.5184221], [59.536348, 28.4731397], [59.2305063, 28.4764732], [59.0619893, 28.4162781], [59.1225478, 28.3632191], [59.1261603, 28.2984927], [59.1614392, 28.2507186], [59.1483843, 28.1273763], [59.0145546, 28.1070656], [58.9338231, 28.0298011], [58.9324362, 27.9731322], [58.8904901, 27.8981235], [58.9010426, 27.5764568], [58.8310364, 27.2536735], [58.8485422, 27.1017277], [58.9129876, 26.9999296], [58.8643748, 26.9086636], [58.8507615, 26.8311605], [58.9057608, 26.7878299], [58.9904926, 26.75922], [59.0177119, 26.6561592], [58.9996588, 26.596439], [59.0374382, 26.5475481], [59.0060451, 26.5281025], [59.0003551, 26.5003229], [58.9224331, 26.5150435], [58.8438142, 26.4831065]]], [{"type": "Feature", "properties": {"province_name": " ", "name_fa": ""}, "geometry": {"type": "Polygon", "coordinates": [[[47.664111, 32.048542], [47.62147, 32.103803], [47.565708, 32.112076], [47.529201, 32.141752], [47.527892, 32.170546], [47.565278, 32.193232], [47.56696, 32.225089], [47.499849, 32.277875], [47.519494, 32.304516], [47.46314, 32.314316], [47.435256, 32.351867], [47.465348, 32.391316], [47.379567, 32.473212], [47.283461, 32.49637], [47.247204, 32.463729], [47.168401, 32.457281], [46.887238, 32.631954], [46.720615, 32.78596], [46.622615, 32.822218], [46.447421, 32.933275], [46.266369, 32.972447], [46.159405, 32.955131], [46.096511, 32.98168], [46.099162, 33.019813], [46.129911, 33.020975], [46.164625, 33.069428], [46.04037, 33.094539], [46.0531735, 33.1282554], [46.125454, 33.11708], [46.212346, 33.197072], [46.180584, 33.206344], [46.189827, 33.26397], [46.134, 33.292185], [46.080962, 33.351755], [46.077601, 33.389648], [46.007252, 33.50781], [45.939491, 33.481894], [45.863003, 33.493148], [45.961002, 33.56133], [45.905254, 33.606284], [45.907062, 33.638097], [45.814716, 33.628415], [45.753165, 33.592917], [45.77167, 33.636936], [45.69252, 33.69097], [45.7701754, 33.7040816], [45.8510102, 33.7949231], [45.8596235, 33.8432538], [45.9215692, 33.8379732], [45.9693482, 33.8588075], [45.8537902, 33.9265889], [45.8446235, 33.9490921], [45.8673995, 33.9890883], [45.9193491, 33.9688151], [45.9832378, 33.9724168], [46.0549102, 34.0377041], [46.2782541, 33.9507633], [46.2985308, 33.9954801], [46.62077, 33.8313134], [46.724106, 33.7421437], [46.7627183, 33.7713108], [46.8630042, 33.7332522], [46.9202271, 33.7685308], [46.9746759, 33.7418645], [46.9971753, 33.8115904], [47.0521778, 33.8332553], [47.0927342, 33.8243639], [47.1063476, 33.8404771], [47.1374601, 33.8146418], [47.2952454, 33.788536], [47.3866369, 33.8613118], [47.3180243, 33.9604875], [47.3913602, 33.9304835], [47.4213599, 33.9524271], [47.4855323, 33.9390951], [47.5069223, 33.90715], [47.5138657, 33.8282556], [47.4574732, 33.8052004], [47.4113605, 33.7371443], [47.3038552, 33.7238107], [47.2580222, 33.7460268], [47.2419094, 33.7257514], [47.1788504, 33.7227], [47.1557938, 33.7029751], [47.0680114, 33.709917], [47.0602357, 33.6646442], [47.0224562, 33.6282538], [47.038013, 33.5846409], [46.9921768, 33.5863019], [46.9424506, 33.6204718], [46.8846714, 33.6201917], [46.8457816, 33.6104778], [46.8643915, 33.5776981], [46.8393918, 33.5615833], [46.8638386, 33.5199195], [47.0235664, 33.4240774], [47.0102334, 33.3840821], [47.100236, 33.3493591], [47.1063496, 33.3324128], [47.151352, 33.3385251], [47.1799621, 33.3018604], [47.2019079, 33.3024113], [47.1955179, 33.2826838], [47.2252443, 33.27324], [47.2324649, 33.2479689], [47.3996957, 33.1976884], [47.5922039, 33.0451818], [47.8097147, 32.9718482], [47.9344431, 32.899041], [47.9952793, 32.8418461], [48.0100024, 32.7793454], [47.9394436, 32.7623972], [47.9102776, 32.7212864], [47.9693907, 32.6587288], [47.9546661, 32.6353339], [47.9892081, 32.6128352], [47.9152184, 32.6027823], [47.9328576, 32.5824091], [47.9005089, 32.5676159], [47.944046, 32.561572], [47.9328002, 32.5349499], [47.9558342, 32.5357321], [47.9425773, 32.5170105], [47.9805985, 32.5363511], [48.0156975, 32.5242384], [48.0354539, 32.3539695], [47.9760476, 32.335637], [47.9617768, 32.2954034], [47.9850235, 32.2638935], [47.8388139, 32.1374699], [47.664111, 32.048542]]]]], [{"type": "Feature", "properties": {"province_name": " ", "name_fa": ""}, "geometry": {"type": "Polygon", "coordinates": [[[49.7317497, 32.8082425], [49.6600773, 32.7793485], [49.5906318, 32.789073], [49.5775754, 32.7401792], [49.5155333, 32.7251812], [49.4283178, 32.7793093], [49.3671645, 32.8445515], [49.2179907, 32.8521658], [49.1719742, 32.8839216], [49.1370774, 32.8698313], [49.0553438, 32.9051092], [49.0138666, 32.8994067], [49.0061035, 32.915383], [48.8998756, 32.8849407], [48.8538629, 32.9108387], [48.8307382, 32.9485216], [48.7648849, 32.9683579], [48.7686882, 32.922986], [48.718581, 32.8960203], [48.7093164, 32.936459], [48.632473, 32.9747246], [48.5471992, 32.9836302], [48.5045663, 32.9616899], [48.389679, 32.9654538], [48.2796974, 32.9971281], [48.1796875, 32.9009407], [48.1280898, 32.9126331], [48.1288223, 32.8704693], [48.0488091, 32.8054172], [48.0682412, 32.7888861], [48.0599975, 32.7403727], [48.0860481, 32.7196815], [48.0639953, 32.7087511], [48.0897255, 32.6917658], [48.064911, 32.6745416], [48.0020447, 32.6745484], [47.9821206, 32.6534494], [47.9102776, 32.7212864], [47.9394436, 32.7623972], [47.996946, 32.7676794], [48.0125025, 32.7904605], [47.9941696, 32.8435165], [47.9344431, 32.8999041], [47.7533252, 32.9982401], [47.6905427, 33.0024024], [47.5922039, 33.0451818], [47.3996957, 33.1976884], [47.2324649, 33.2479689], [47.2252443, 33.27324], [47.1955179, 33.2826838], [47.2019079, 33.3024113], [47.1799621, 33.3018604], [47.151352, 33.3385251], [47.1063496, 33.3324128], [47.100236, 33.3493591], [47.0102334, 33.3840821], [47.0260664, 33.4201862], [47.0113434, 33.435471], [46.8616151, 33.5221313], [46.8396684, 33.5599222], [46.8643915, 33.5776981], [46.8457816, 33.6104778], [46.9410639, 33.6207516], [46.9921768, 33.5863019], [47.0374558, 33.584361], [47.0224562, 33.6282538], [47.0602357, 33.6646442], [47.0680114, 33.709917], [47.1557938, 33.7029751], [47.1788504, 33.7227], [47.2419094, 33.7257514], [47.2580222, 33.7460268], [47.3038552, 33.7238107], [47.4080276, 33.7352035], [47.4574732, 33.8052004], [47.5110891, 33.8249246], [47.5069223, 33.90715], [47.4799753, 33.942146], [47.4985851, 33.9757608], [47.4452492, 34.0310376], [47.4266392, 34.0843725], [47.4869185, 34.1132645], [47.488308, 34.1829876], [47.5649772, 34.1821484], [47.6944284, 34.2760476], [47.6791485, 34.3560528], [47.7002613, 34.3782714], [47.8280466, 34.3238259], [47.8483233, 34.2785479], [47.9002729, 34.2385487], [47.9386057, 34.2457689], [48.053891, 34.2127187], [48.1772298, 34.086044], [48.2405656, 34.0491006], [48.3044582, 34.0566015], [48.304735, 34.0213183], [48.3522379, 34.0068765], [48.4830761, 34.0102073], [48.5194655, 34.0507621], [48.5269684, 34.1071563], [48.5603017, 34.0665938], [48.5858581, 34.0635431], [48.6172508, 34.0960456], [48.6576542, 34.0760602], [48.7094765, 34.0918759], [48.7236428, 34.1232677], [48.756589, 34.072434], [48.8417048, 34.0363209], [48.8517047, 34.0027073], [48.9042073, 34.0063179], [48.9161503, 34.0285404], [48.948377, 34.0027074], [49.0111558, 34.0221501], [49.0775724, 34.01471], [49.0417125, 33.9763146], [49.0089329, 33.9818748], [48.9686532, 33.9596517], [48.9561538, 33.9224262], [48.9642103, 33.9049325], [49.0122663, 33.9060429], [48.9936567, 33.8904809], [49.0061566, 33.8671466], [48.9758771, 33.8396507], [48.9939334, 33.8299286], [48.9822672, 33.7982618], [49.0117136, 33.767705], [49.0211569, 33.696317], [49.0703263, 33.6888153], [49.075606, 33.7179829], [49.1417187, 33.7215939], [49.1950546, 33.6863149], [49.2156111, 33.6129748], [49.2842269, 33.6040916], [49.29645, 33.6502049], [49.3514492, 33.6529765], [49.3820087, 33.6946475], [49.3606191, 33.7382587], [49.410065, 33.7504818], [49.4311784, 33.736318], [49.4358981, 33.6996489], [49.5283801, 33.6879626], [49.6189621, 33.7424207], [49.633129, 33.6968687], [49.5972963, 33.6729819], [49.6034062, 33.6410334], [49.6361858, 33.625199], [49.6514656, 33.5863075], [49.7123016, 33.5685319], [49.7709142, 33.5176957], [49.7964708, 33.4465905], [49.9345325, 33.3849186], [50.018148, 33.3685323], [49.9987051, 33.3035274], [49.9375892, 33.2471437], [49.9470359, 33.2299169], [49.8895336, 33.2196333], [49.8714771, 33.191301], [49.8906435, 33.1446311], [49.8778636, 33.1296341], [49.9242565, 33.0896361], [49.9148102, 33.0657454], [49.8636975, 33.0690771], [49.837308, 33.0499084], [49.722026, 33.0843538], [49.6689667, 33.0501881], [49.6550771, 33.0074084], [49.7078563, 32.982407], [49.6373006, 32.9746328], [49.6823002, 32.8990767], [49.7486925, 32.8557365], [49.7317497, 32.8082425]]]]], [{"type": "Feature", "properties": {"province_name": " ", "name_fa": ""}, "geometry": {"type": "Polygon", "coordinates": [[[49.9478688, 33.3874196], [49.8067507, 33.4379776], [49.7623042, 33.5263085], [49.7123016, 33.5685319], [49.6514656, 33.5863075], [49.6361858, 33.625199], [49.5956295, 33.656597], [49.633129, 33.6968687], [49.6189621, 33.7424207], [49.5283801, 33.6879626], [49.4358981, 33.6996489], [49.4311784, 33.736318], [49.410065, 33.7504818], [49.3606191, 33.7382587], [49.3820087, 33.6946475], [49.3514492, 33.6529765], [49.29645, 33.6502049], [49.2842269, 33.6040916], [49.2156111, 33.6129748], [49.1950546, 33.6863149], [49.1417187, 33.7215939], [49.075606, 33.7179829], [49.0703263, 33.6888153], [49.0167135, 33.6990884], [49.0117136, 33.767705], [48.9822672, 33.7982618], [48.9939334, 33.8299286], [48.9758771, 33.8396507], [49.0061566, 33.8671466], [48.9936567, 33.8904809], [49.0122663, 33.9060429], [48.9594871, 33.9079835], [48.9619867, 33.9499301], [49.0089329, 33.9818748], [49.0417125, 33.9763146], [49.1753304, 34.115983], [49.1383843, 34.1193784], [49.093938, 34.1627197], [49.1233842, 34.1910418], [49.1217173, 34.2488219], [49.0208784, 34.3096594], [49.007265, 34.3843837], [48.9500427, 34.388553], [49.003655, 34.4238294], [49.0036549, 34.4657827], [49.0547675, 34.5052174], [49.0011545, 34.5974473], [49.060047, 34.6349478], [49.0811569, 34.5846704], [49.1347728, 34.6135541], [49.1031031, 34.7535606], [49.1492156, 34.7971742], [49.1681052, 34.8660652], [49.2117179, 34.8421758], [49.3036666, 34.9119035], [49.3897823, 34.9174518], [49.4142284, 35.0016269], [49.2889433, 35.0060648], [49.259777, 35.0421831], [49.3064462, 35.0849574], [49.4111719, 35.0341191], [49.4453414, 35.0871766], [49.4661739, 35.1791241], [49.426171

, 35.2221821], [49.3703351, 35.234962], [49.3747816, 35.2632994], [49.3506119, 35.2813548], [49.4070043, 35.3638547], [49.395338, 35.3863542], [49.4308941, 35.4188541], [49.5508994, 35.4035785], [49.6228485, 35.4488555], [49.6822942, 35.4471963], [49.6936838, 35.4810829], [49.6697942, 35.5183046], [49.6875739, 35.5341379], [49.7928561, 35.5133096], [49.8803583, 35.5469146], [49.9723072, 35.5060874], [50.0431466, 35.5055284], [50.1942646, 35.5244097], [50.353419, 35.5706023], [50.6259494, 35.5474747], [50.7676246, 35.4869185], [50.8956036, 35.4850921], [50.9920724, 35.4371339], [50.9615226, 35.417188], [50.890687, 35.2274604], [50.857074, 35.1816245], [50.654287, 35.0193994], [50.6498404, 34.9996803], [50.6923434, 34.9518966], [50.705123, 34.8682868], [50.4892791, 34.8602302], [50.451223, 34.8269037], [50.4340001, 34.7574514], [50.4092738, 34.7519027], [50.3412177, 34.7599505], [50.2514887, 34.8146787], [50.1673198, 34.8213458], [50.1220405, 34.7555112], [50.1081511, 34.6238331], [50.0862046, 34.5835621], [50.1570406, 34.5091175], [50.1481542, 34.4732836], [50.3351052, 34.4421688], [50.3317721, 34.4068837], [50.4245543, 34.2774425], [50.4420545, 34.2010531], [50.5170602, 34.1899339], [50.6456749, 34.2560534], [50.727624, 34.2152142], [50.7962399, 34.2360537], [50.815125, 34.2047764], [50.8848555, 34.1913332], [50.8973554, 34.1571626], [51.0526483, 34.0387274], [51.0200362, 33.9818351], [51.0487506, 33.9366011], [51.0320842, 33.8529864], [51.049584, 33.8202096], [51.0093045, 33.7729798], [50.8926328, 33.7071527], [50.7040117, 33.7271481], [50.5956733, 33.6621504], [50.5742836, 33.6307614], [50.4537253, 33.6346521], [50.3370535, 33.615757], [50.2731613, 33.5824265], [50.2328815, 33.6027028], [50.1917691, 33.5310397], [49.9837016, 33.4924176], [49.9720351, 33.4676986], [49.9903684, 33.4199214], [49.959812, 33.4157504], [49.9478688, 33.3874196]]], {"type": "Feature", "properties": {"province_name": "name_fa": " ", "geometry": {"type": "Polygon", "coordinates": [[[51.06952, 31.1878167], [51.0309765, 31.1818113], [50.9694423, 31.2265568], [50.963202, 31.2604226], [50.9245891, 31.2984821], [50.8923662, 31.3087554], [50.8570867, 31.2904206], [50.7892982, 31.3143162], [50.7198524, 31.3832062], [50.6395699, 31.42765], [50.5748441, 31.4087636], [50.5003954, 31.4526482], [50.476229, 31.4276498], [50.4009467, 31.4151461], [50.3306676, 31.4448749], [50.2932415, 31.4815706], [50.3756484, 31.5418299], [50.3584136, 31.5550185], [50.3923798, 31.6013915], [50.4269791, 31.5934585], [50.4575423, 31.6141558], [50.4464036, 31.646511], [50.2831, 31.8500772], [50.2903518, 31.8704122], [50.2396202, 31.9401716], [50.1182176, 32.0164292], [49.9368208, 32.1860741], [49.9078903, 32.320275], [49.5868493, 32.6092272], [49.5015438, 32.7165475], [49.5775754, 32.7401792], [49.5906318, 32.789073], [49.6600773, 32.7793485], [49.7298032, 32.8090731], [49.767306, 32.7640711], [49.9823168, 32.6346172], [50.0445391, 32.6274027], [50.153711, 32.6637915], [50.2559398, 32.5643506], [50.3220524, 32.5368379], [50.4912268, 32.5912856], [50.5176162, 32.631566], [50.4581705, 32.6607308], [50.5270627, 32.7179593], [50.5498848, 32.7138973], [50.548673, 32.6799061], [50.5622097, 32.6883087], [50.565016, 32.6723313], [50.6518463, 32.7105145], [50.8523551, 32.7121267], [50.887358, 32.6962892], [50.9170809, 32.664903], [50.9318041, 32.5985097], [50.99653, 32.5546268], [51.0079166, 32.5240621], [50.9684705, 32.4335058], [50.973194, 32.3882322], [51.1443119, 32.2876719], [51.2337608, 32.1743356], [51.3457126, 32.1396181], [51.3857121, 32.0915555], [51.3768256, 32.0557275], [51.4143253, 32.0323871], [51.4334282, 31.9721088], [51.4132154, 31.9523803], [51.4282151, 31.9268282], [51.4096022, 31.858497], [51.2968203, 31.8229413], [51.2854305, 31.8021027], [51.3690463, 31.7126555], [51.3054338, 31.6618246], [51.2965441, 31.5157101], [51.259598, 31.4607111], [51.2884877, 31.4451559], [51.3140441, 31.3934804], [51.2932112, 31.365702], [51.3371005, 31.2965324], [51.3298774, 31.1959862], [51.2237637, 31.2237608], [51.1804259, 31.2098696], [51.179041, 31.1748696], [51.1529267, 31.1498667], [51.06952, 31.1878167]]]]}, {"type": "Feature", "properties": {"province_name": " ", "name_fa": " "}, "geometry": {"type": "Polygon", "coordinates": [[[45.69252, 33.69097], [45.642104, 33.733966], [45.643764, 33.772], [45.603272, 33.803023], [45.511585, 33.945574], [45.431545, 33.945566], [45.407661, 33.977255], [45.466848, 34.009822], [45.473516, 34.075608], [45.51594, 34.14613], [45.554874, 34.195116], [45.581097, 34.23956], [45.583566, 34.302663], [45.538075, 34.352107], [45.496178, 34.336867], [45.439814, 34.458932], [45.521255, 34.480982], [45.51199, 34.551635], [45.5314626, 34.6018982], [45.594687, 34.5498888], [45.6834717, 34.5595769], [45.7432041, 34.5380347], [45.702625, 34.693772], [45.653259, 34.72311], [45.69876, 34.77002], [45.689538, 34.815161], [45.73689, 34.846417], [45.753623, 34.828685], [45.792074, 34.853026], [45.788431, 34.912296], [45.86504, 34.896328], [45.897312, 34.959992], [45.876525, 35.031513], [45.930795, 35.045347], [45.921987, 35.095489], [45.998198, 35.061935], [46.015202, 35.071155], [46.059426, 35.037953], [46.08028, 35.05592], [46.071919, 35.083131], [46.116562, 35.074065], [46.191291, 35.110888], [46.155059, 35.128999], [46.154758, 35.159001], [46.201247, 35.201229], [46.177792, 35.228359], [46.108403, 35.241129], [46.14894, 35.279894], [46.3471386, 35.1705126], [46.4465912, 35.0538401], [46.5010369, 35.0349514], [46.5562231, 34.9219012], [46.6399327, 34.8693974], [46.7435483, 34.8399508], [46.735492, 34.7874429], [46.7829948, 34.7393902], [46.894943, 34.7643916], [47.0132854, 34.7460585], [47.0885643, 34.7577246], [47.0671746, 34.7924511], [47.1252335, 34.8035574], [47.0721745, 34.8510576], [47.0810636, 34.8907797], [47.1732896, 34.9032836], [47.159679, 34.9610643], [47.1732895, 34.9724488], [47.2141256, 34.9371676], [47.2432948, 34.9532884], [47.2819082, 34.9277316], [47.3266309, 34.9385662], [47.3082975, 35.0063403], [47.3549665, 35.0180041], [47.3608, 35.052733], [47.399689, 35.0616181], [47.728592, 35.0621781], [47.8030414, 35.0149562], [47.8052646, 34.980515], [47.8399878, 34.938838], [47.9272133, 34.9299524], [47.9177704, 34.8955089], [47.863601, 34.8580062], [47.7971938, 34.8511636], [47.8652683, 34.7224487], [47.8555451, 34.6535598], [47.8730448, 34.6293872], [47.9547173, 34.6227191], [47.9994402, 34.5866082], [48.0344432, 34.5885573], [48.0497196, 34.6204997], [48.1013894, 34.5296699], [48.090556, 34.4935566], [48.1050029, 34.4674413], [48.0858332, 34.4182685], [48.0444438, 34.3896611], [47.989713, 34.4013069], [47.9936287, 34.364661], [47.9524951, 34.3454946], [47.8866593, 34.2552092], [47.8483233, 34.2785479], [47.8280466, 34.3238259], [47.6952616, 34.3754918], [47.6788719, 34.3510532], [47.6944284, 34.2760476], [47.5649772, 34.1821484], [47.488308, 34.1829876], [47.4869185, 34.1132645], [47.4319159, 34.0963137], [47.423583, 34.0646507], [47.4980321, 33.9771508], [47.4927519, 33.9485369], [47.4213599, 33.9524271], [47.3913602, 33.9304835], [47.3149679, 33.9571477], [47.3866369, 33.8613118], [47.343301, 33.8160326], [47.3116348, 33.8163123], [47.304412, 33.791867], [47.1374601, 33.8146418], [47.1063476, 33.8404771], [47.0927342, 33.8243639], [47.0521778, 33.8332553], [46.9971753, 33.8115904], [46.9746759, 33.7418645], [46.9202271, 33.7685308], [46.8596674, 33.733532], [46.7627183, 33.7713108], [46.7252193, 33.7418638], [46.62077, 33.8313134], [46.3557564, 33.9704863], [46.2985308, 33.9954801], [46.2782541, 33.9507633], [46.0549102, 34.0377041], [45.9832378, 33.9724168], [45.9193491, 33.9688151], [45.8673995, 33.9890883], [45.8446235, 33.9490921], [45.8537902, 33.9265889], [45.9693482, 33.8588075], [45.9215692, 33.8379732], [45.8596235, 33.8432538], [45.8510102, 33.7949231], [45.7701754, 33.7040816], [45.69252, 33.69097]]]]}, {"type": "Feature", "properties": {"province_name": " ", "name_fa": " ", "geometry": {"type": "Polygon", "coordinates": [[[48.4722371, 48.3791841, 48.0013163], [48.304735, 48.0213183], [48.3044582, 48.0566015], [48.2405656, 48.0491006], [48.1869531, 48.0796536], [48.053891, 48.2127187], [47.9386057, 48.2457689], [47.9002729, 48.2385487], [47.8866593, 48.2552092], [47.9524951, 48.3454946], [47.9936287, 48.364661], [47.989713, 48.4013069], [48.044438, 48.3896611], [48.0588332, 48.4182685], [48.1050029, 48.4674413], [48.090556, 48.4935566], [48.1013894, 48.5296699], [48.0466665, 48.6229991], [48.0344432, 48.5885573], [47.9994402, 48.5866082], [47.9547173, 48.6227191], [47.865545, 48.6332761], [47.8652683, 48.7224487], [47.7971938, 48.8511636], [47.8586012, 48.8557778], [47.9086037, 48.8877324], [47.9227703, 48.9518918], [47.9527696, 48.9705112], [47.999871, 48.9612717], [48.040832, 48.9860633], [48.074448, 48.9616167], [48.1438907, 48.9707822], [48.1758366, 48.9988461], [48.1922262, 48.0927395], [48.1525034, 48.1180134], [48.1813929, 48.1610646], [48.1506136, 48.2098726], [48.0958339, 48.2282944], [48.0897204, 48.2694025], [48.0266608, 48.3707958], [47.9447149, 48.389679], [47.8660987, 48.5582986], [47.8924882, 48.5855279], [47.9741575, 48.5741315], [48.0491627, 48.6391369], [48.2111141, 48.6874692], [48.2516703, 48.7202482], [48.2961198, 48.6805276], [48.3916789, 48.6430234], [48.4836278, 48.6452504], [48.4961278, 48.598864], [48.5427971, 48.5849702], [48.6639126, 48.5988642], [48.7164155, 48.5521872], [48.7530849, 48.5702471], [48.7766977, 48.6141376], [48.770031, 48.6785813], [48.7328049, 48.6916443], [48.7289148, 48.7119193], [48.7797506, 48.7335824], [48.8839229, 48.6994157], [48.1272706, 48.5688593], [49.1342137, 48.5224694], [49.1608804, 48.4933001], [49.428394, 48.4266363], [49.3506119, 48.2813548], [49.3747816, 48.2632994], [49.3703351, 48.234962], [49.426171, 48.2221821], [49.4661739, 48.1791241], [49.4628408, 48.1382925], [49.4111719, 48.0341191], [49.3239462, 48.0832977], [49.2608868, 48.0516185], [49.2828334, 48.0077337], [49.4142284, 48.0016269], [49.3897823, 48.914518], [49.3036666, 48.9119035], [49.2117179, 48.8421758], [49.1681052, 48.860652], [49.1492156, 48.7971742], [49.1031031, 48.7535606], [49.1347728, 48.6135541], [49.0811569, 48.5846704], [49.060047, 48.6349478], [49.0011545, 48.5974473], [49.0547675, 48.5052174], [49.0036549, 48.4657827], [49.003655, 48.4238294], [48.9500427, 48.388553], [49.007265, 48.3843837], [49.0208784, 48.3096594], [49.1217173, 48.2488219], [49.1233842, 48.1910418], [49.093938, 48.1627197], [49.1383843, 48.1193784], [49.1376636, 48.1163187], [49.1672741, 48.0935465], [49.0811585, 48.0152091], [49.0111558, 48.0221501], [48.948377, 48.0027074], [48.9161503, 48.0285404], [48.9042073, 48.0063179], [48.8611478, 48.0010465], [48.8417048, 48.0363209], [48.765589, 48.072434], [48.7308659, 48.1210474], [48.7133664, 48.1221575], [48.7094765, 48.0918759], [48.6576542, 48.0760602], [48.6172508, 48.0960456]]]]}]

, [48.5858581, 34.0635431], [48.5603017, 34.0665938], [48.5269684, 34.1071563], [48.5194655, 34.0507621], [48.4722371, 34.0063171]]], {"type": "Feature", "properties": {"province_name": " ", "name_fa": ""}, "geometry": {"type": "Polygon", "coordinates": [[[[49.5427042, 35.4027091], [49.391171, 35.4252475], [49.1608804, 35.4933001], [49.1342137, 35.5224694], [49.1339371, 35.5627462], [49.0475413, 35.6171944], [48.8839229, 35.6994157], [48.7797506, 35.7335824], [48.7363237, 35.7239575], [48.7247478, 35.7494213], [48.7844737, 35.8841459], [48.9033685, 35.9824777], [48.9342018, 35.9305373], [49.0553205, 35.8783147], [49.0758768, 35.8899781], [49.0717101, 35.9238678], [49.1517126, 35.9299788], [49.2089383, 35.9822079], [49.3019973, 35.9758182], [49.3619997, 36.0108135], [49.4128357, 36.07387], [49.4108922, 36.1366516], [49.4336719, 36.1930405], [49.3911688, 36.2680397], [49.4245017, 36.3272128], [49.3708889, 36.3385999], [49.3369993, 36.3894352], [49.2775532, 36.3883271], [49.1911575, 36.4286022], [49.109765, 36.4336014], [49.0605954, 36.4958249], [48.9719792, 36.5533242], [49.1083657, 36.7780687], [49.1891487, 36.7607539], [49.2806052, 36.8155578], [49.484227, 36.6080555], [49.615622, 36.606939], [49.7876613, 36.5593829], [49.9008466, 36.5825708], [49.9359834, 36.5724961], [49.938971, 36.6026379], [50.0581539, 36.6040616], [50.1714824, 36.6688887], [50.2320418, 36.6786142], [50.4462162, 36.6105516], [50.5920582, 36.5268767], [50.6798406, 36.4999976], [50.8387356, 36.4005536], [50.8565154, 36.3455479], [50.8479127, 36.2940161], [50.6534512, 36.33471], [50.4959428, 36.3422142], [50.4134978, 36.3090408], [50.4320636, 36.2452451], [50.6781745, 36.1152612], [50.5776122, 36.0824867], [50.5473363, 36.0183135], [50.4259411, 36.0016496], [50.3731618, 35.8983085], [50.2970462, 35.8224764], [50.250403, 35.806938], [50.2437107, 35.6477508], [50.1634282, 35.6141399], [50.2289876, 35.5405225], [50.0042535, 35.5035807], [49.8803583, 35.5469146], [49.7928561, 35.5133096], [49.6875739, 35.5341379], [49.6697942, 35.5183046], [49.6936838, 35.4810829], [49.6842407, 35.4488555], [49.6228485, 35.4488555], [49.5427042, 35.4027091]]]]], {"type": "Feature", "properties": {"province_name": " ", "name_fa": ""}, "geometry": {"type": "Polygon", "coordinates": [[[[49.878335, 36.5781833], [49.7876613, 36.5781833], [49.615622, 36.606939], [49.4750603, 36.6130538], [49.2375491, 36.8500039], [49.2144926, 36.9008328], [49.1703228, 36.9189003], [49.0567071, 37.0544501], [48.9308649, 37.1516745], [48.8953085, 37.2255624], [48.9047549, 37.3144585], [48.7152994, 37.4741889], [48.6800198, 37.5928009], [48.6252935, 37.6330795], [48.5958469, 37.6905837], [48.639463, 37.7216943], [48.6541827, 37.7789116], [48.5919597, 37.8030771], [48.5708464, 37.9019768], [48.6205722, 38.0133698], [48.6330717, 38.1130875], [48.6830743, 38.1522617], [48.6666845, 38.1786488], [48.6900177, 38.2222562], [48.6716842, 38.2897628], [48.6391812, 38.2908765], [48.5739016, 38.3617065], [48.5818264, 38.4603383], [48.6018782, 38.4173601], [48.6622715, 38.3929551], [48.7383031, 38.4111207], [48.7913236, 38.4517477], [49.2123986, 38.4083871], [49.0663083, 37.966461], [49.1204966, 37.7506298], [49.1957131, 37.6751317], [49.4286418, 37.5739205], [49.7869314, 37.5271119], [49.8572953, 37.5508405], [49.9567752, 37.5521229], [50.033611, 37.5008104], [50.2325693, 37.4655106], [50.292419, 37.4359754], [50.4388082, 37.1715669], [50.6550013, 37.0454031], [50.5201082, 36.8808404], [50.432606, 36.8566679], [50.4323293, 36.7300045], [50.3564903, 36.6394415], [50.2012056, 36.6780559], [50.0581539, 36.6040616], [49.938971, 36.6026379], [49.9359834, 36.5724961], [49.878335, 36.5781833]]]]], {"type": "Feature", "properties": {"province_name": " ", "name_fa": ""}, "geometry": {"type": "Polygon", "coordinates": [[[[48.7250284, 35.5548745], [48.6986389, 35.5530258], [48.6639126, 35.5988642], [48.5255742, 35.5883061], [48.495018, 35.5999729], [48.4836278, 35.6452504], [48.3916789, 35.6430234], [48.2961198, 35.6805276], [48.2491702, 35.7235583], [48.192501, 35.7291459], [48.1733344, 35.7883087], [48.0511056, 35.8074728], [47.9999924, 35.933032], [47.9097105, 35.8891465], [47.8677643, 35.9069207], [47.8622074, 35.9413596], [47.9822125, 35.9710839], [47.9030436, 36.0022039], [47.9549895, 36.049977], [47.994992, 36.1397035], [47.9469321, 36.2219334], [47.9563786, 36.2813709], [47.9413786, 36.3102638], [47.7822034, 36.3719379], [47.6596985, 36.3895406], [47.5283028, 36.4249885], [47.4032943, 36.4130417], [47.3552376, 36.4752668], [47.3294013, 36.6108249], [47.2913448, 36.6863798], [47.2130085, 36.7336091], [47.1741153, 36.791667], [47.3027333, 36.8972278], [47.4924644, 37.0000012], [47.5241339, 37.0672274], [47.59886, 37.0375081], [47.663029, 37.0994478], [47.7660877, 37.0763991], [47.7788675, 37.1322247], [47.8508135, 37.171956], [47.9483186, 37.1652881], [47.9833218, 37.1477849], [47.9993331, 37.112217], [48.0411012, 37.1494499], [48.0997139, 37.1558395], [48.1502733, 37.2211248], [48.3922305, 37.2516826], [48.4408434, 37.2116773], [48.4741762, 37.2102915], [48.5091796, 37.1383458], [48.5464026, 37.1352862], [48.5775155, 37.1030587], [48.6558547, 37.1291758], [48.7375235, 37.1855641], [48.7900263, 37.1391743], [48.8153061, 37.1541761], [48.8697522, 37.1464009], [48.9222549, 37.1766377], [48.9319782, 37.1502887], [49.0567071, 37.0544501], [49.1703228, 36.9189003], [49.2144926, 36.9008328], [49.2314356, 36.8575033], [49.2806052, 36.8155578], [49.1891487, 36.7607539], [49.1083657, 36.7780687], [49.0664862, 36.7191856], [49.061816, 36.6801731], [49.0186487, 36.6511105], [48.9755927, 36.5461009], [49.0605954, 36.4958249], [49.109765, 36.4336014], [49.1911575, 36.4286022], [49.2775532, 36.3883271], [49.3369993, 36.3894352], [49.3708889, 36.3385999], [49.4245017, 36.3272128], [49.3911688, 36.2680397], [49.3619997, 36.1930405], [49.3978359, 36.0463661], [49.3019973, 35.9758182], [49.2089383, 35.9822079], [49.1517126, 35.9299788], [49.0717101, 35.9238678], [49.0758768, 35.8899781], [49.0486539, 35.8780353], [48.9342018, 35.9305373], [48.9033685, 35.9824777], [48.7872534, 35.8872014], [48.7244712, 35.7513594], [48.7328049, 35.6916443], [48.770031, 35.6785813], [48.7750314, 35.6013619], [48.7250284, 35.5548745]]]]], {"type": "Feature", "properties": {"province_name": " ", "name_fa": ""}, "geometry": {"type": "Polygon", "coordinates": [[[[51.9685142, 35.1399642], [51.8723985, 35.2810784], [51.9439291, 35.3613021], [51.9124013, 35.4144115], [51.923511, 35.5458079], [51.9579572, 35.5780235], [51.9935134, 35.5685834], [52.0351831, 35.4935832], [52.2576937, 35.4405264], [52.3790892, 35.3374621], [52.5363172, 35.4016339], [52.6674356, 35.5358092], [52.7132718, 35.5552487], [52.801054, 35.5494151], [52.8991128, 35.6419201], [52.995785, 35.6788656], [53.1527365, 35.7919297], [53.1399566, 35.8183142], [53.0205078, 35.8355399], [53.0207845, 35.8841507], [53.0649541, 35.9149853], [53.2205156, 35.9269277], [53.2246855, 35.9680443], [53.3705239, 36.0602662], [53.6791502, 36.0983219], [53.8874946, 36.219994], [53.9538838, 36.305829], [53.8799913, 36.3672156], [53.9391606, 36.437497], [54.0124998, 36.4455501], [54.155285, 36.4997206], [54.508911, 36.5147261], [54.5708602, 36.5449978], [54.6494727, 36.6180556], [54.7008655, 36.6325007], [54.7780913, 36.6002781], [54.8653169, 36.6294477], [55.0100454, 36.767221], [55.4008978, 36.8169571], [55.5000701, 36.9011157], [55.5603495, 36.9252866], [55.6214658, 37.0991758], [55.705915, 37.1919577], [55.7164715, 37.2664082], [55.8223071, 37.322245], [55.908423, 37.3323405], [55.9087027, 37.2055724], [55.9514823, 37.2044565], [56.0087049, 37.2344567], [56.0628744, 37.1608486], [55.9942616, 37.1191819], [56.0400978, 37.1002915], [56.0145379, 37.0069571], [55.9791784, 36.9962453], [55.9842616, 36.9000882], [56.015651, 36.8336127], [56.2214921, 36.9008362], [56.2645516, 36.8380601], [56.3651105, 36.7530568], [56.3612239, 36.6816685], [56.4456028, 36.6941673], [56.4978956, 36.6647208], [56.5937346, 36.668611], [56.7584628, 36.7038922], [56.7692958, 36.6158298], [56.8420784, 36.5897217], [56.8654112, 36.5566687], [56.8640245, 36.5208313], [56.8167982, 36.4619424], [56.6984597, 36.3883299], [56.7331824, 36.313053], [56.800408, 36.2386072], [56.7312388, 36.0899851], [56.7217953, 35.8599842], [56.9154126, 35.6927568], [57.0451409, 35.5469161], [56.9909714, 35.4505255], [57.0501403, 35.3071863], [57.0002296, 35.2152486], [56.8556893, 35.2024685], [56.6909577, 35.0624638], [56.5367828, 35.0477427], [56.4537237, 34.9999609], [56.1639868, 34.6263434], [55.9161995, 34.2443852], [52.8863373, 34.2488261], [52.6732697, 34.3666098], [52.4999285, 34.4191138], [52.3943698, 34.4293925], [52.1551893, 34.4013356], [52.0146278, 34.4005022], [51.8335099, 34.5071703], [51.8403823, 34.5680611], [51.8885126, 34.635511], [51.9590713, 34.8580124], [52.0404635, 35.0152423], [52.0357403, 35.0888484], [51.9685142, 35.1399642]]]]], {"type": "Feature", "properties": {"province_name": " ", "name_fa": ""}, "geometry": {"type": "Polygon", "coordinates": [[[[51.7854549, 30.701524], [51.5084986, 30.874775], [51.3776605, 30.9965304], [51.3421007, 31.0626407], [51.3271009, 31.1212606], [51.3371005, 31.2965324], [51.2934877, 31.3640409], [51.3140441, 31.3798755], [51.3037675, 31.4209882], [51.259598, 31.4607111], [51.2965441, 31.5157101], [51.3054338, 31.6618246], [51.3690463, 31.7126555], [51.2854305, 31.8021027], [51.2968203, 31.8229413], [51.4121055, 31.8618287], [51.429325, 31.9998914], [51.3784923, 32.0526666], [51.3857121, 32.0915555], [51.3457126, 32.1396181], [51.2337608, 32.1743356], [51.1443119, 32.2876719], [50.973194, 32.3882322], [50.9684705, 32.4335058], [51.0079166, 32.5240621], [50.99653, 32.5546267], [50.9318041, 32.5985097], [50.9170809, 32.664903], [50.887358, 32.6962892], [50.8523551, 32.7121267], [50.6518463, 32.7105145], [50.565016, 32.6723313], [50.5622097, 32.6883087], [50.548673, 32.6799061], [50.5498848, 32.7138973], [50.5270627, 32.7179593], [50.4581705, 32.6607308], [50.5176162, 32.631566], [50.4912268, 32.5912856], [50.3153859, 32.5371178], [50.2559398, 32.5643506], [50.153711, 32.6637915], [50.0445391, 32.6274027], [49.9823168, 32.6346172], [49.767306, 32.7640711], [49.7317498, 32.806572], [49.7486925, 32.8557365], [49.6823002, 32.8990767], [49.6375772, 32.9774138], [49.7078563, 32.982407], [49.6536869, 33.0101894], [49.6689667, 33.0501881], [49.722026, 33.0843538], [49.837308, 33.0499084], [49.921479, 33.0721379], [49.9187001, 33.1004708], [49.8767539, 33.1321351], [49.8906435, 33.1446311], [49.8714771, 33.191301], [49.8895336, 33.2196333], [49.9470359, 33.2299169], [49.9375892, 33.2471437], [50.0000919, 33.3060283], [50.018148, 33.3685323], [49.9478688, 33.3874196], [49.959812, 33.4157504], [49.9903684, 33.4199214], [49.9720351, 33.4676986], [49.9837016, 33.4924176], [50.1917691, 33.5310397], [50.2353814, 33.6035334], [50.2731613, 33.582

4265], [50.3370535, 33.615757], [50.4537253, 33.6346521], [50.5631739, 33.6282607], [50.7006785, 33.7263176], [50.8870763, 33.7057624], [51.0093045, 33.7729798], [51.050974, 33.8274312], [51.0320842, 33.8529864], [51.0487506, 33.9366011], [51.0200362, 33.9818351], [51.0526483, 34.0387274], [50.908387, 34.1518826], [50.9948608, 34.2346461], [51.0140271, 34.287443], [51.0115269, 34.3721675], [51.1773681, 34.4443897], [51.3259864, 34.4410599], [51.8007816, 34.510984], [52.139633, 34.4027254], [52.3943698, 34.4293925], [52.5635444, 34.4049456], [52.7485521, 34.3332726], [52.8921704, 34.2485553], [55.4786782, 34.2518854], [55.497844, 33.5290919], [55.3858954, 33.3643638], [55.225541, 33.324088], [54.9917104, 33.1257457], [54.8972616, 33.1801987], [54.790313, 33.1179718], [54.7447535, 33.0657485], [54.5500228, 33.0907497], [54.3483488, 33.0529635], [54.3075126, 33.0301923], [54.283623, 32.9601897], [54.

2241771, 32.8988003], [53.941384, 32.7988012], [53.842492, 32.6860155], [53.4624737, 32.6951803], [53.1866274, 32.6176809], [53.0152331, 32.5154586], [53.058846, 32.3321159], [53.0424564, 31.9898882], [53.009677, 31.8546066], [53.0530131, 31.7459903], [53.1980177, 31.6134876], [53.1944078, 31.573212], [52.9938406, 31.4954342], [52.754944, 31.4962646], [52.3935389, 31.5482125], [52.2474209, 31.6615455], [52.1304691, 31.6565526], [52.0463004, 31.59571], [51.9365721, 31.4134866], [51.9679618, 31.1943167], [51.8976831, 30.9893098], [51.8354575, 30.8934789], [51.8585137, 30.8004145], [51.8326809, 30.7404135], [51.7854549, 30.7015241]]], [{"type": "Feature", "properties": {"province_name": " ", "name_fa": " "}, "geometry": {"type": "Polygon", "coordinates": [[[50.6031831, 29.9437282], [50.5737369, 29.9245595], [50.5342908, 29.9762298], [50.5392903, 30.0731804], [50.5770698, 30.1253977], [50.5138933, 30.1660606], [50.5207594, 30.3904245], [50.4526215, 30.4322939], [50.3864593, 30.5235268], [50.5060119, 30.5305816], [50.5495662, 30.5630552], [50.5339013, 30.5996434], [50.4237277, 30.6651337], [50.392841, 30.7313077], [50.3354417, 30.7087156], [50.2541769, 30.7555763], [50.2315138, 30.7905283], [50.2434539, 30.8325063], [50.2104225, 30.9322635], [50.0697479, 30.8299317], [50.0333405, 30.8713077], [49.9555053, 30.877364], [49.9511414, 30.9293713], [49.9236754, 30.9277065], [49.8865088, 30.9625502], [49.97143, 31.0129259], [49.9475175, 31.0748626], [49.9899942, 31.1256725], [49.95467, 31.2114047], [50.099861, 31.2575942], [50.1527063, 31.2518317], [50.2467612, 31.306489], [50.2998317, 31.3584786], [50.2479222, 31.4182174], [50.2540909, 31.4483382], [50.2932415, 31.4815706], [50.3795569, 31.4173669], [50.476229, 31.4276498], [50.5003954, 31.4526482], [50.5748441, 31.4087636], [50.6395699, 31.42765], [50.7198524, 31.3832062], [50.7892982, 31.3143162], [50.8570867, 31.2904206], [50.8923662, 31.3087554], [50.9245891, 31.2984821], [50.963202, 31.2604226], [50.9694423, 31.2265568], [51.0406997, 31.1787511], [51.0784776, 31.1887642], [51.1529267, 31.1408667], [51.179041, 31.1748696], [51.1804259, 31.2098696], [51.2237637, 31.2327608], [51.3298774, 31.1958862], [51.3298776, 31.1028075], [51.3776605, 30.9965304], [51.5084986, 30.8784775], [51.8340676, 30.6659746], [51.8882369, 30.5981899], [51.8335144, 30.519299], [51.8374012, 30.4537399], [51.7429526, 30.3715171], [51.6193375, 30.4437414], [51.5746115, 30.4223552], [51.3743279, 30.4759652], [51.3496013, 30.562633], [51.2112632, 30.6531948], [51.1773736, 30.6498557], [51.1365375, 30.5984689], [51.1429277, 30.5640225], [51.2195967, 30.4542988], [51.294879, 30.3845735], [51.2384865, 30.3362448], [51.2690428, 30.2795718], [51.179276, 30.2062363], [51.1423714, 30.2156827], [51.1065353, 30.1876232], [51.0662558, 30.2003987], [51.050423, 30.1806761], [50.9559742, 30.1559554], [50.9248614, 30.1151223], [50.9031953, 30.03484], [50.8465261, 29.9656759], [50.686795, 29.9959588], [50.6031831, 29.9437282]]], [{"type": "Feature", "properties": {"province_name": " ", "name_fa": " "}, "geometry": {"type": "Polygon", "coordinates": [[[46.8977236, 34.7643915], [46.7677187, 34.7432789], [46.7332716, 34.7941102], [46.7435483, 34.8399508], [46.6399327, 34.8693974], [46.5562231, 34.9219012], [46.4996471, 35.0363406], [46.4465912, 35.0538401], [46.3471386, 35.1705126], [46.2171325, 35.2377326], [46.050838, 35.387981], [45.985425, 35.496023], [45.992784, 35.519464], [46.017991, 35.520698], [45.972104, 35.581178], [46.029957, 35.576616], [46.013005, 35.627783], [45.992997, 35.634401], [46.009149, 35.678707], [46.042465, 35.706728], [46.165081, 35.688964], [46.19956, 35.722322], [46.277041, 35.728866], [46.305853, 35.761978], [46.333321, 35.763349], [46.340244, 35.819902], [46.294615, 35.835867], [46.212274, 35.794952], [46.183814, 35.812492], [46.167906, 35.799993], [46.137288, 35.843527], [46.078562, 35.85843], [46.024647, 35.834468], [45.964013, 35.843593], [45.954344, 35.822803], [45.896036, 35.837649], [45.874868, 35.811362], [45.80038, 35.827039], [45.763701, 35.799014], [45.665625, 35.926916], [45.5579743, 35.9951027], [45.5545969, 36.07414], [45.6746055, 36.107753], [45.6412687, 36.1388683], [45.6796053, 36.1724786], [45.6832145, 36.2247025], [45.7323843, 36.2388663], [45.8301661, 36.2191437], [45.9471146, 36.2391462], [45.9846175, 36.2810865], [45.9540609, 36.3933188], [45.9923967, 36.4249824], [46.1454585, 36.4633196], [46.1996276, 36.4544294], [46.2801875, 36.388041], [46.3221337, 36.3774833], [46.4168591, 36.3858163], [46.4596381, 36.4163721], [46.6743725, 36.3960954], [46.8029876, 36.4238767], [46.8971601, 36.383043], [46.9216069, 36.3227659], [47.0179989, 36.3191536], [47.080502, 36.2508172], [47.1605078, 36.2433122], [47.2985693, 36.2477635], [47.3463491, 36.2775677], [47.392185, 36.3344337], [47.3657952, 36.3869358], [47.3713514, 36.4380475], [47.4121843, 36.409988], [47.5216327, 36.4261035], [47.6596985, 36.3805406], [47.7822034, 36.3719379], [47.9413786, 36.3102638], [47.9563786, 36.2813709], [47.9469321, 36.2219334], [47.994992, 36.1397035], [47.9549895, 36.049977], [47.9030436, 36.0022039], [47.9821225, 35.9710839], [47.8622074, 35.9413596], [47.8608176, 35.9158076], [47.9097105, 35.8891465], [47.9999924, 35.933032], [48.0511056, 35.8074728], [48.1716676, 35.7891381], [48.192501, 35.7291459], [48.2516703, 35.7202482], [48.2111141, 35.6874692], [48.0491627, 35.6391369], [47.9741575, 35.5741315], [47.8924882, 35.5855279], [47.8658221, 35.5505264], [47.9447149, 35.389679], [48.0266608, 35.3707958], [48.0897204, 35.2694025], [48.0958339, 35.2282944], [48.139723, 35.2221796], [48.1811128, 35.1652316], [48.1525034, 35.1180134], [48.192262, 35.0927395], [48.1758366, 34.9988461], [48.1391673, 34.9685631], [48.0858314, 34.9616168], [48.043052, 34.9860633], [47.9999871, 34.9612717], [47.9527696, 34.9705112], [47.9223534, 34.9345351], [47.8836008, 34.926894], [47.8322113, 34.945225], [47.8030414, 35.0149562], [47.728592, 35.0621781], [47.399689, 35.0616181], [47.3608, 35.052733], [47.3549665, 35.0180041], [47.3082975, 35.0063403], [47.3266309, 34.9385662], [47.2819082, 34.9277316], [47.2432948, 34.9532884], [47.2141256, 34.9371676], [47.1732895, 34.9724488], [47.159679, 34.9610643], [47.1732896, 34.9032836], [47.0810636, 34.8907797], [47.0721745, 34.8510576], [47.1252335, 34.8035574], [47.0671746, 34.7924511], [47.0885643, 34.7577246], [47.0132854, 34.7460585], [46.8977236, 34.7643915]]], [{"type": "Feature", "properties": {"province_name": " ", "name_fa": " "}, "geometry": {"type": "Polygon", "coordinates": [[[45.5184017, 35.9957851], [45.4399233, 35.9989647], [45.3933796, 35.9669948], [45.3485103, 35.9785782], [45.341761, 36.008539], [45.381363, 36.08418], [45.327046, 36.139867], [45.316386, 36.183547], [45.330546, 36.214017], [45.265754, 36.253206], [45.304553, 36.273747], [45.260964, 36.301289], [45.283314, 36.336817], [45.272404, 36.393518], [45.236723, 36.434077], [45.149108, 36.404341], [45.081587, 36.42549], [45.053896, 36.506649], [45.001556, 36.538382], [45.037641, 36.560903], [45.028064, 36.605184], [45.0707771, 36.6302428], [45.057093, 36.702536], [45.012914, 36.753236], [44.972409, 36.749348], [44.940179, 36.785928], [44.844998, 36.778703], [44.838355, 36.815753], [44.903699, 36.86141], [44.912415, 36.917009], [44.882687, 36.954349], [44.905552, 37.009779], [44.854548, 37.04709], [44.807265, 37.044148], [44.777723, 37.075656], [44.738908, 37.092362], [44.749666, 37.112879], [44.7849294, 37.1450862], [44.7575679, 37.2283412], [44.814586, 37.27806], [44.799184, 37.310113], [44.732434, 37.342232], [44.7214731, 37.3756933], [44.6573683, 37.3859048], [44.630891, 37.431615], [44.5798293, 37.4476059], [44.614409, 37.602176], [44.56816, 37.644762], [44.6280768, 37.7021432], [44.6206225, 37.7270202], [44.5545718, 37.7806258], [44.4449506, 37.7709171], [44.4483769, 37.8065854], [44.4237529, 37.799449], [44.3944467, 37.8230608], [44.404986, 37.847003], [44.388189, 37.862511], [44.2673977, 37.8673051], [44.227458, 37.8882392], [44.251232, 37.9433704], [44.2407127, 37.9650564], [44.3013231, 38.0325345], [44.311362, 38.0728943], [44.3293645, 38.098353], [44.3645086, 38.1039085], [44.3511809, 38.1349942], [44.3926934, 38.1464431], [44.4117396, 38.2195101], [44.4022685, 38.2520003], [44.429251, 38.3036964], [44.5125314, 38.3412598], [44.4627198, 38.3594066], [44.4493712, 38.3854921], [44.3796451, 38.3585915], [44.299987, 38.3941676], [44.3198318, 38.4086056], [44.3019383, 38.4472285], [44.3254601, 38.502081], [44.3093356, 38.5302464], [44.3216199, 38.6240924], [44.2721185, 38.6462477], [44.2862414, 38.674969], [44.2540673, 38.718334], [44.2801473, 38.7329247], [44.2864982, 38.7831041], [44.3148195, 38.8087951], [44.3009879, 38.8408017], [44.245308, 38.8506821], [44.2454553, 38.8702175], [44.2113518, 38.8905571], [44.2188309, 38.9167755], [44.1897152, 38.9340273], [44.2002417, 38.9480087], [44.1802595, 38.9560676], [44.1914252, 38.9746819], [44.1717941, 38.9917907], [44.211677, 39.0209503], [44.1908665, 39.0745913], [44.22591, 39.0950499], [44.2135559, 39.1303045], [44.1663807, 39.1776245], [44.0972566, 39.2044917], [44.1032307, 39.2425733], [44.0843065, 39.2560217], [44.1080596, 39.2842966], [44.0812513, 39.2941275], [44.0732183, 39.3356242], [44.0318908, 39.3717259], [44.0710893, 39.4123425], [44.1301049, 39.4054092], [44.1301405, 39.3896441], [44.2331218, 39.4155361], [44.2952003, 39.3752581], [44.3770211, 39.4116375], [44.4219571, 39.4116039], [44.4328572, 39.4427251], [44.4084339, 39.4711492], [44.440367, 39.505069], [44.416204, 39.55644], [44.480148, 39.610424], [44.487663, 39.642549], [44.4671974, 39.6841019], [44.614695, 39.7824624], [44.6478404, 39.7273554], [44.7150829, 39.7142776], [44.8219144, 39.6112697], [44.8755201, 39.6152092], [44.8760247, 39.5930064], [44.8972058, 39.6052331], [44.8883296, 39.5817248], [44.9157487, 39.5659961], [44.9097716, 39.5250712], [44.9518498, 39.4921687], [44.9399284, 39.459659], [44.9

771798, 39.4487743], [44.9608079, 39.4421583], [44.9645597, 39.421841], [45.021813, 39.4044283], [45.0867037, 39.3512472], [45.0803208, 39.3335707], [45.1286003, 39.3012738], [45.1178194, 39.2868354], [45.1419814, 39.2844706], [45.1529322, 39.2163506], [45.1847708, 39.2205229], [45.2476296, 39.1856003], [45.316613, 39.1984975], [45.3275812, 39.1859033], [45.3042399, 39.166023], [45.3497813, 39.1705737], [45.3546876, 39.1263944], [45.4024111, 39.0916736], [45.4030843, 39.0727792], [45.4569034, 39.0511283], [45.4368899, 39.0117784], [45.4529047, 38.9929965], [45.3797074, 38.9625519], [45.329295, 38.8922705], [45.2651265, 38.8539317], [45.2756837, 38.7319877], [45.2334572, 38.66504], [45.2929078, 38.5339291], [45.339577, 38.4864231], [45.2698514, 38.4417052], [45.2104025, 38.4300311], [45.0820642, 38.3325346], [45.1280592, 38.1915139], [45.30281, 38.1583172], [45.3484723, 38.1245645], [45.3749078, 38.0226729], [45.2801507, 37.942032], [45.292854, 37.8604891], [45.3563683, 37.7386848], [45.5946336, 37.4257989], [45.7808896, 37.22

8471], [45.7509927, 37.1836129], [46.0046132, 37.1505605], [46.175456, 36.9955512], [46.1774022, 36.9552699], [46.1987926, 36.9444408], [46.2862944, 36.9716711], [46.4124124, 37.0563881], [46.4476919, 37.1013887], [46.4960282, 37.0919479], [46.4607487, 37.0202777], [46.6421472, 37.0411149], [46.7093696, 37.0724994], [46.8019014, 37.0492971], [46.8627115, 36.9997297], [46.8318781, 36.9761102], [46.8427122, 36.927501], [46.8988245, 36.8947151], [46.8779925, 36.7799973], [46.9979975, 36.7452781], [47.0368872, 36.8013815], [47.1405026, 36.8322183], [47.2130085, 36.7336091], [47.2913448, 36.6863798], [47.3294013, 36.6108249], [47.3713513, 36.4388763], [47.3657952, 36.3869358], [47.3880185, 36.3222085], [47.312736, 36.2513763], [47.1899545, 36.2416544], [47.080502, 36.2508172], [47.0179989, 36.3191536], [46.9216069, 36.3227659], [46.8971601, 36.383043], [46.7888209, 36.4269304], [46.7560413, 36.4055363], [46.7296517, 36.4155441], [46.6743725, 36.3960954], [46.4596381, 36.4163721], [46.4168591, 36.3858163], [46.3221337, 36.3774833], [46.2801875, 36.388041], [46.1996276, 36.4544294], [46.0976794, 36.4563747], [45.9923967, 36.4249824], [45.9540609, 36.3933188], [45.9846175, 36.2810865], [45.9551718, 36.2435882], [45.8301661, 36.2191437], [45.698771, 36.2352622], [45.6743245, 36.2138633], [45.6796053, 36.1724786], [45.6412687, 36.1388683], [45.6746055, 36.107753], [45.5545969, 36.07414], [45.5579743, 35.9951027], [45.5184017, 35.9957851]]], [{"type": "Feature", "properties": {"province_name": " ", "name_fa": ""}, "geometry": {"type": "Polygon", "coordinates": [[[53.9922201, 27.1889532], [53.9333276, 27.1236778], [53.7960999, 27.0458956], [53.6322058, 27.0983966], [53.5419239, 27.1578432], [53.496088, 27.1011746], [53.4096961, 27.1139499], [53.3535802, 27.1911741], [53.1307937, 27.2720136], [53.0496784, 27.2750621], [52.9438433, 27.3295156], [52.7546663, 27.5028527], [52.6238284, 27.5634067], [52.5632693, 27.6245148], [52.5016001, 27.7864703], [52.3879852, 27.9931387], [52.2652003, 28.0442452], [52.1707517, 28.1300882], [52.1254724, 28.1995309], [52.1229722, 28.2623041], [52.0540799, 28.4284232], [51.9604644, 28.8212057], [51.8674055, 28.9798252], [51.79407, 29.0445478], [51.8151794, 29.068163], [51.8093463, 29.0995487], [51.6740614, 29.2342696], [51.587116, 29.3914949], [51.5085004, 29.4528891], [51.4276615, 29.475107], [51.364606, 29.4634389], [51.3209933, 29.4976138], [51.2701572, 29.6081709], [51.2996034, 29.6784425], [51.2782135, 29.7006648], [51.1726518, 29.7142832], [50.9998675, 29.7828922], [50.8154136, 29.7753968], [50.7037418, 29.7915055], [50.6448493, 29.822064], [50.6751287, 29.8628971], [50.7137415, 29.873171], [50.6456824, 29.9354023], [50.6031831, 29.9437282], [50.6195994, 29.9601188], [50.686795, 29.9959588], [50.8465261, 29.9656759], [50.9031953, 30.03484], [50.9248614, 30.1151223], [50.9559742, 30.1559554], [51.050423, 30.1806761], [51.0662558, 30.2003987], [51.1065353, 30.1876232], [51.1423714, 30.2156827], [51.1779276, 30.2062363], [51.2690428, 30.2795718], [51.2384865, 30.3362448], [51.294879, 30.3845735], [51.2195967, 30.4542988], [51.1429277, 30.5640225], [51.1365375, 30.5984689], [51.1773736, 30.6498557], [51.2112632, 30.6531948], [51.3496013, 30.562633], [51.3665478, 30.4862435], [51.3993274, 30.4637379], [51.4471068, 30.4679069], [51.5746115, 30.4223552], [51.6193375, 30.4437414], [51.7429526, 30.3715171], [51.8262914, 30.4365226], [51.8429576, 30.4658674], [51.8335144, 30.519299], [51.8882369, 30.5981899], [51.7851782, 30.7006937], [51.8326809, 30.7404135], [51.8585137, 30.8004145], [51.8354575, 30.8934789], [51.8976831, 30.9893098], [51.9679618, 31.1943167], [51.9365721, 31.4134866], [52.0463004, 31.59571], [52.1304691, 31.6565526], [52.2474209, 31.6615455], [52.3935389, 31.5482125], [52.5596566, 31.5123797], [52.8010566, 31.4948743], [52.8485593, 31.4474688], [52.9346715, 31.4048746], [52.968841, 31.3618213], [52.9716178, 31.2770885], [52.9457847, 31.240988], [52.8688358, 31.2237639], [52.831613, 31.1865358], [52.9118953, 31.1212617], [52.9693979, 31.0145934], [53.3774721, 30.8254161], [53.4710875, 30.7315245], [53.7838764, 30.6751358], [53.854992, 30.6479168], [53.9108279, 30.5918013], [53.9308275, 30.5401367], [53.8986047, 30.4456825], [53.9255511, 30.3340175], [53.8641586, 30.1937419], [53.939976, 30.0615178], [54.0980585, 29.9384533], [54.1030585, 29.7928965], [54.2427864, 29.5962233], [54.4764096, 29.641243], [54.8161445, 29.5937251], [54.983932, 29.3948318], [55.1642158, 29.253716], [55.2622777, 29.0945466], [55.2806106, 28.9951025], [55.25481077, 28.842599], [55.2519975, 28.7239896], [55.3508892, 28.6603707], [55.4528411, 28.5439838], [55.4078383, 28.4184168], [55.3867267, 29.998693], [55.5006199, 27.9234117], [55.5772885, 27.7945249], [55.4806168, 27.7292443], [55.4103378, 27.6195123], [55.3314457, 27.5717377], [55.2567201, 27.4956291], [55.1392152, 27.4342357], [54.6289134, 27.4678509], [54.3530677, 27.4586846], [54.1577842, 27.4214541], [53.9686071, 27.3500664], [53.9708304, 27.3070144], [54.0349994, 27.2350637], [53.9922201, 27.1889532]]]]}, {"type": "Feature", "properties": {"province_name": " ", "name_fa": ""}, "geometry": {"type": "Polygon", "coordinates": [[[52.482398, 26.989407], [51.2988492, 27.699037], [50.1313982, 29.0842133], [49.5801433, 29.6674426], [50.1390342, 30.1759444], [50.2000071, 30.2949624], [50.4165763, 30.1821897], [50.5717901, 30.137902], [50.5748464, 30.1126229], [50.5392903, 30.0731804], [50.5342908, 29.9762298], [50.5737369, 29.9245595], [50.6031831, 29.9437282], [50.642349, 29.9373417], [50.7137415, 29.873171], [50.6751287, 29.8628971], [50.645727, 29.8287296], [50.6656856, 29.8056752], [50.8373599, 29.7734487], [50.9998675, 29.7828922], [51.1726518, 29.7142832], [51.2873801, 29.6948386], [51.2965467, 29.6584482], [51.2701572, 29.6081709], [51.3209933, 29.4976138], [51.364606, 29.4634389], [51.4421081, 29.4739979], [51.5515564, 29.425388], [51.6740614, 29.2342696], [51.8065698, 29.1037131], [51.8151794, 29.068163], [51.79407, 29.0445478], [51.8674055, 28.9798252], [51.9604644, 28.8212057], [52.0540799, 28.4284232], [52.1229722, 28.2623041], [52.1254724, 28.1995309], [52.1582518, 28.1456406], [52.2652003, 28.0442452], [52.3879852, 27.9931387], [52.5016001, 27.7864703], [52.5427127, 27.6631241], [52.6027152, 27.5806265], [52.7546663, 27.5028527], [52.9438433, 27.3297854], [52.8852276, 27.2928411], [52.8165666, 27.3237282], [52.739214, 27.2743065], [52.482398, 26.989407]]]]}, {"type": "Feature", "properties": {"province_name": " ", "name_fa": ""}, "geometry": {"type": "Polygon", "coordinates": [[[48.5872376, 37.1094519], [48.5091796, 37.1383458], [48.4780662, 37.2069532], [48.4208402, 37.2219532], [48.4169537, 37.2458431], [48.3350043, 37.2394544], [48.3352812, 37.2664028], [48.1494391, 37.416124], [48.1552721, 37.4783524], [48.0608231, 37.5147348], [48.0530467, 37.533525], [48.079156, 37.5783566], [48.0216532, 37.6122422], [48.0255429, 37.654462], [48.0666557, 37.7294704], [48.1352716, 37.7522446], [48.1511045, 37.8103041], [47.999403, 37.8949567], [47.977486, 37.8758633], [47.8930368, 37.8683574], [47.887757, 37.8978047], [47.9377595, 37.9466916], [47.9097032, 37.9908649], [47.8622001, 38.0066999], [47.8521997, 38.1466985], [47.7874733, 38.2514228], [47.7246942, 38.2122555], [47.6171887, 38.2183641], [47.4827366, 38.1819793], [47.4688466, 38.2494817], [47.3985677, 38.2378125], [47.2871719, 38.302263], [47.2907813, 38.3319841], [47.345784, 38.3569797], [47.4641225, 38.4780964], [47.4985682, 38.5811588], [47.5688509, 38.6117084], [47.564954, 38.6422788], [47.5257943, 38.6697677], [47.4357886, 38.7003296], [47.4313449, 38.7564406], [47.4696776, 38.7914396], [47.5980199, 38.8189437], [47.5116235, 38.9158942], [47.5363466, 38.9500526], [47.5266267, 38.9586465], [47.5630161, 38.9775567], [47.50218, 39.0000537], [47.5230132, 39.0142262], [47.5052366, 39.0353314], [47.5274594, 39.0806154], [47.4974563, 39.1283969], [47.5194023, 39.1508961], [47.518569, 39.2197857], [47.4810658, 39.2725645], [47.5049349, 39.2954356], [47.4307925, 39.3186376], [47.439111, 39.3372671], [47.363399, 39.4217256], [47.3775701, 39.4549047], [47.5629369, 39.5137751], [47.752969, 39.606023], [47.8073216, 39.6591904], [47.9174231, 39.6614998], [47.9821222, 39.7090003], [48.0320584, 39.6925275], [48.3686221, 39.3798361], [48.2653165, 39.3334445], [48.211001, 39.3291082], [48.1297734, 39.2713758], [48.1502833, 39.1982545], [48.2184151, 39.1627439], [48.2349786, 39.1296652], [48.3077928, 39.1085272], [48.3332227, 39.0211657], [48.2707532, 38.9673018], [48.0882345, 38.9469508], [48.0692648, 38.9109368], [48.01027, 38.907306], [48.0207028, 38.8378898], [48.1071082, 38.7885576], [48.1073103, 38.7698813], [48.2491558, 38.7260959], [48.252657, 38.6619076], [48.2949419, 38.6473191], [48.3102234, 38.6076539], [48.361489, 38.5948538], [48.4592932, 38.6076878], [48.464684, 38.5613214], [48.5055078, 38.5470001], [48.5133037, 38.518379], [48.579433, 38.4668173], [48.5739016, 38.3617065], [48.6391812, 38.2908765], [48.6716842, 38.2897628], [48.6900177, 38.222562], [48.6666845, 38.1786488], [48.6830743, 38.1522617], [48.6330717, 38.1130875], [48.6205722, 38.0133698], [48.5708464, 37.9019768], [48.5919597, 37.8030771], [48.6541827, 37.7789116], [48.639463, 37.7216943], [48.5958469, 37.6905837], [48.6252935, 37.6330795], [48.6800198, 37.5928009], [48.7152994, 37.4741889], [48.9105878, 37.3036261], [48.8953085, 37.2255624], [48.9222549, 37.1766737], [48.8697522, 37.1464009], [48.8153061, 37.1541761], [48.7900263, 37.1391743], [48.7375235, 37.1855641], [48.6558547, 37.1291758], [48.5872376, 37.1094519]]]]}, {"type": "Feature", "properties": {"province_name": " ", "name_fa": ""}, "geometry": {"type": "Polygon", "coordinates": [[[52.2165806, 35.7655351], [52.0726856, 35.7633173], [52.0062932, 35.7866467], [51.8782311, 35.8599847], [

51.760449,36.0066529],[51.4448757,36.1494318],[51.315984,36.1544326],[51.2679243,36.1841587],[51.2015319,36.1624881],[50.8479127,36.2940161],[50.8565154,36.3455479],[50.8345689,36.4047155],[50.6798406,36.4999976],[50.3598236,36.6416659],[50.4323293,36.7300045],[50.432606,36.8566679],[50.5201082,36.8808404],[50.6550013,37.0454031],[50.9556187,36.8932945],[51.2168547,36.8156359],[51.487796,36.7748336],[51.6236711,36.7197484],[51.9164495,36.6626774],[53.2905669,36.9308012],[53.8526691,36.9844424],[53.8647148,36.6794543],[53.9647204,36.675285],[54.1172253,36.5977745],[54.1255585,36.4911097],[54.0124998,36.4455501],[53.9391606,36.4374972],[53.8813813,36.3777735],[53.9538838,36.3099913],[53.9277742,36.2574933],[53.6791502,36.0983219],[53.3705239,36.0602662],[53.2246855,35.9680443],[53.2232956,35.9283157],[53.0649541,35.9149853],[52.9874517,35.83554],[52.9207825,35.8377573],[52.7049383,35.9280361],[52.5679864,35.943313],[52.4357581,35.9122089],[52.3782555,35.8485917],[52.2165806,35.7655351]]]}},{"type":"Feature","properties":{"province_name":"","name_fa":""

"},"geometry":{"type":"Polygon","coordinates":[[[54.3327929,36.510556],[54.1261152,36.4916591],[54.1280586,36.5702788],[54.1044488,36.6158313],[53.9647204,36.675285],[53.8647148,36.6794543],[53.8526691,36.9844424],[53.897152,37.0186755],[53.8623745,37.203783],[53.8777413,37.2469311],[53.8607569,37.32222],[53.9277235,37.3472619],[54.2446779,37.3199856],[54.2851929,37.351769],[54.3976597,37.3606995],[54.5867621,37.4600312],[54.6736676,37.437055],[54.7925151,37.5254889],[54.8152475,37.6062298],[54.7827548,37.6343399],[54.8402131,37.745694],[55.0233227,37.8429634],[55.0410295,37.8796158],[55.1276555,37.9491966],[55.1695862,37.9606129],[55.2015097,37.9473068],[55.2271298,37.9792886],[55.384722,38.041944],[55.442778,38.086111],[55.756111,38.125278],[55.9625,38.078333],[56.164444,38.095],[56.216111,38.072222],[56.2889756,38.0846583],[56.3103894,37.9103136],[56.2803861,37.8672577],[56.2156603,37.8594822],[56.118711,37.6347476],[56.0612082,37.5722467],[56.0584315,37.5191926],[56.0012054,37.4425221],[56.0164852,37.3702954],[55.8086256,37.3178633],[55.7200814,37.2711319],[55.705915,37.1919577],[55.6214658,37.0991758],[55.5534062,36.918346],[55.5000701,36.9011157],[55.4008978,36.8169571],[55.0014356,36.7641687],[54.9122631,36.6644427],[54.8478172,36.6222252],[54.7678114,36.6002781],[54.7008655,36.6325007],[54.6447528,36.6161104],[54.5175242,36.5172211],[54.3327929,36.510556]]]]}}}

@@ public/data/world-countries.geojson text

{"type":"FeatureCollection","attribution":"Natural Earth public domain","features":[{"type":"Feature","properties":{"name":"","bbox":[-180,-18.28799,180,-16.020882],"geometry":{"type":"MultiPolygon","coordinates":[[[180,-16.067133],[180,-16.555217],[179.364143,-16.801354],[178.725059,-17.012042],[178.596839,-16.63915],[179.096609,-16.433984],[179.413509,-16.379054],[180,-16.067133]]],[[178.12557,-17.50481],[178.3736,-17.33992],[178.71806,-17.62846],[178.55271,-18.15059],[177.93266,-18.28799],[177.38146,-18.16432],[177.28504,-17.72465],[177.67087,-17.38114],[178.12557,-17.50481]]],[[179.79332,-16.020882],[179.917369,-16.501783],[180,-16.555217],[180,-16.067133],[179.79332,-16.020882]]]},"type":"Feature","properties":{"name":"","bbox":[29.339998,-11.720938,40.31659,-0.95],"geometry":{"type":"Polygon","coordinates":[[[33.903711,-0.95],[34.07262,-1.05982],[37.69869,-3.09699],[37.7669,-3.67712],[39.20222,-4.67677],[38.74054,-5.90895],[38.79977,-6.47566],[39.44,-6.84],[39.47,-7.1],[39.19469,-7.7039],[39.25203,-8.00781],[39.18652,-8.48551],[39.53574,-9.11237],[39.9496,-10.0984],[40.316586,-10.317098],[40.31659,-10.3171],[39.521,-10.89688],[38.427557,-11.285202],[37.82764,-11.26879],[37.47129,-11.56876],[36.77515,-11.594537],[36.514082,-11.720938],[35.312398,-11.439146],[34.559989,-11.52002],[34.28,-10.16],[33.940838,-9.693674],[33.73972,-9.41715],[32.759375,-9.230599],[32.191865,-8.930359],[31.556348,-8.762049],[31.157751,-8.594579],[30.74001,-8.340006],[30.740015,-8.340007],[30.199997,-7.079981],[29.620032,-6.520015],[29.419993,-5.939999],[29.519987,-5.419979],[29.339998,-4.499983],[29.753512,-4.452389],[30.11632,-4.09012],[30.50554,-3.56858],[30.75224,-3.35931],[30.74301,-3.03431],[30.52766,-2.80762],[30.469674,-2.413855],[30.46967,-2.41383],[30.758309,-2.28725],[30.816135,-1.698914],[30.419105,-1.134659],[30.76986,-1.01455],[31.86617,-1.02736],[33.903711,-0.95]]]},"type":"Feature","properties":{"name":"","bbox":[-17.063423,20.999752,-8.665124,27.656426],"geometry":{"type":"Polygon","coordinates":[[[-8.66559,27.656426],[-8.665124,27.589479],[-8.6844,27.395744],[-8.687294,25.881056],[-11.969419,25.933353],[-11.937224,23.374594],[-12.874222,23.284832],[-13.118754,22.77122],[12.929102,21.327071],[-16.845194,21.333323],[-17.063423,20.999752],[-17.020428,21.42231],[-17.002962,21.420734],[-14.750955,21.5006],[14.630833,21.86094],[-14.221168,22.310163],[-13.89111,23.691009],[-12.500963,24.770116],[-12.030759,26.030866],[-11.71822,26.104092],[-11.392555,26.883424],[-10.551263,26.990808],[-10.189424,26.860945],[-9.735343,26.860945],[-9.413037,27.088476],[-8.794884,27.120696],[-8.817828,27.656426],[-8.66559,27.656426]]]},"type":"Feature","properties":{"name":"","bbox":[-140.99778,41.675105,-52.640899,83.23324],"geometry":{"type":"MultiPolygon","coordinates":[[[[-122.84,49],[122.97421,49.002538],[124.91024,49.98456],[125.62461,50.41656],[127.43561,50.83061],[127.99276,51.71583],[127.85032,52.32961],[129.12979,52.75538],[129.30523,53.56159],[130.51497,54.28757],[130.536109,54.802754],[130.53611,54.80278],[129.98,55.285],[130.00778,55.91583],[131.70781,56.55212],[132.73042,57.69289],[133.35556,58.41028],[134.27111,58.86111],[134.94559,59.27056],[135.47583,59.78778],[136.47972,59.46389],[137.4525,58.905],[138.34089,59.56211],[139.039,60],[140.013,60.27682],[140.99778,60.30639],[140.9925,66.00003],[140.986,69.712],[140.985988,69.711998],[139.12052,69.47102],[137.54636,68.99002],[136.50358,68.89804],[135.62576,69.31512],[134.41464,69.62743],[132.92925,69.50534],[131.43136,69.94451],[129.79471,70.19369],[129.10773,69.77927],[128.36156,70.01286],[128.13817,70.48384],[127.44712,70.37721],[125.75632,69.48058],[124.42483,70.1584],[124.28968,69.39969],[123.06108,69.56372],[122.6835,69.85553],[121.47226,69.79778],[119.94288,69.37786],[117.60268,69.01128],[116.22643,68.84151],[115.2469,68.90591],[113.89794,68.3989],[115.30489,67.90261],[113.49277,67.68815],[110.798,67.80612],[109.94619,67.98104],[108.8802,67.38144],[107.79239,67.8736],[108.81299,68.31164],[108.16721,68.65392],[106.95,68.7],[106.15,68.8],[105.34282,68.56122],[104.33791,68.018],[103.22115,68.09775],[101.45433,67.64689],[99.90195,67.80566],[98.4432,67.78165],[98.5586,68.40394],[97.66948,68.57864],[96.11991,68.23939],[96.12588,67.29338],[95.48943,68.0907],[94.685,68.06383],[94.23282,69.06903],[95.30408,69.68571],[96.47131,70.08976],[96.39115,71.19482],[95.2088,71.92053],[93.88997,71.76015],[92.87818,71.31869],[91.51964,70.19129],[92.40692,69.69997],[90.5471,69.49766],[90.55151,68.47499],[89.21515,69.25873],[88.01966,68.61508],[88.31749,67.87338],[87.35017,67.19872],[86.30607,67.92146],[85.57664,68.78456],[85.52197,69.88211],[84.10081,69.80539],[82.62258,69.65826],[81.28043,69.16202],[81.22022,68.66567],[81.96436,68.13253],[81.25928,67.59716],[81.38653,67.11078],[83.34456,66.41154],[84.73542,66.2573],[85.76943,66.55833],[86.0676,66.05625],[87.03143,65.21297],[87.32324,64.77563],[88.48296,64.09897],[89.91444,64.03273],[90.70398,63.61017],[90.77004,62.96021],[91.93342,62.83508],[93.15698,62.02469],[94.24153,60.89865],[94.62931,60.11021],[94.6846,58.94882],[93.21502,58.78212],[92.76462,57.84571],[92.29703,57.08709],[90.89769,57.28468],[89.03953,56.85172],[88.03978,56.47162],[87.32421,55.99914],[86.07121,55.72383],[85.01181,55.3026],[83.36055,55.24489],[82.27285,55.14832],[82.4362,54.28227],[82.12502,53.27703],[81.40075,52.15788],[79.91289,51.20842],[79.14301,51.53393],[78.60191,52.56208],[79.12421,54.14145],[79.82958,54.66772],[78.22874,55.13645],[77.0956,55.83741],[76.54137,56.53423],[76.62319,57.20263],[77.30226,58.05209],[78.51688,58.80458],[77.33676,59.85261],[77.77272,60.75788],[78.10687,62.31964],[77.41067,62.55053],[75.69621,62.2784],[74.6682,62.18111],[73.83988,62.4438],[72.90853,62.10507],[71.67708,61.52535],[71.37369,61.13717],[69.59042,61.06141],[69.62033,60.22125],[69.2879,58.95736],[68.37455,58.80106],[67.64976,58.21206],[66.20178,58.76731],[65.24517,59.87071],[64.58352,60.33558],[63.80475,59.4426],[62.50236,58.16708],[61.39655,56.96745],[61.79866,56.33945],[60.46853,55.77548],[59.56962,55.20407],[57.97508,54.94549],[57.3332,54.6265],[56.93689,53.78032],[56.15811,53.64749],[55.75632,53.27036],[55.68338,52.14664],[56.40916,51.7707],[57.12691,51.41972],[58.77482,51.0643],[60.03309,50.24277],[61.72366,50.08046],[63.86251,50.29099],[65.36331,50.2982],[66.39905,50.22897],[67.23631,49.51156],[68.51114,49.06836],[69.95362,47.74488],[71.10458,46.82171],[70.25522,46.98606],[68.65,48.3],[66.55243,49.1331],[65.05626,49.23278],[64.17099,48.74248],[65.11545,48.07085],[64.79854,46.99297],[64.47219,46.23849],[63.17329,45.73902],[61.52072,45.88377],[60.51815,47.00793],[60.4486,46.28264],[59.80287,45.9204],[61.03988,45.26525],[63.25471,44.67014],[64.24656,44.2653],[65.36406,43.54523],[66.1234,43.61867],[66.16173,44.46512],[64.42549,45.29204],[66.02605,45.25931],[67.13741,45.13753],[67.79134,45.70281],[67.79046,47.06636],[68.23444,47.35486],[68.905,47.185],[69.237216,47.447781],[69.9999,46.69307],[70.305,45.915],[70.66,45.46],[71.08482,45.30524],[71.405,45.255],[71.50506,45.0082],[73.34783,45.0073

8], [-74.867, 45.00048], [-75.31821, 44.81645], [-76.375, 44.09631], [-76.5, 44.018459], [-76.820034, 43.628784], [-77.737885, 43.629056], [-78.72028, 43.625089], [-79.171674, 43.466339], [-79.01, 43.27], [-78.92, 42.965], [-78.939362, 42.863611], [-80.247448, 42.3662], [-81.277747, 42.209026], [-82.439278, 41.675105], [-82.690089, 41.675105], [-83.02981, 41.832796], [-83.142, 41.975681], [-83.12, 42.08], [-82.9, 42.43], [-82.43, 42.98], [-82.137642, 43.571088], [-82.337763, 44.44], [-82.550925, 45.347517], [-83.592851, 45.816894], [-83.469551, 45.994686], [-83.616131, 46.116927], [-83.890765, 46.116927], [-84.091851, 46.275419], [-84.14212, 46.512222], [-84.3367, 46.40877], [-84.6049, 46.4396], [-84.543749, 46.538684], [-84.779238, 46.637102], [-84.87608, 46.900083], [-85.652363, 47.220219], [-86.461991, 47.553338], [-87.439793, 47.94], [-88.378114, 48.302918], [-89.272917, 48.019808], [-89.6, 48.01], [-90.83, 48.27], [-91.64, 48.14], [-92.61, 48.45], [-93.63087, 48.60926], [-94.32914, 48.67074], [-94.64, 48.84], [-94.81758, 49.38905], [-95.15609, 49.38425], [-95.15907, 49], [-97.22872, 49.0007], [-100.65, 49], [-104.04826, 48.99986], [-107.05, 49], [-110.05, 49], [-113.49], [-116.04818, 49], [-117.03121, 49], [-120, 49], [-122.84, 49]], [[[-83.99367, 62.4528], [-83.25048, 62.91409], [-81.87699, 62.90458], [-81.89825, 62.7108], [-83.06857, 62.15922], [-83.77462, 62.18231], [-83.99367, 62.4528]], [[[-79.775833, 72.802902], [-80.876099, 73.333183], [-80.833885, 73.693184], [-80.353058, 73.75972], [-78.064438, 73.651932], [-76.34, 73.102685], [-76.251404, 72.826385], [-77.314438, 72.855545], [-78.39167, 72.876656], [-79.486252, 72.742203], [-79.775833, 72.802902]], [[[-80.315395, 62.085565], [-79.92939, 62.3856], [-79.52002, 62.36371], [-79.26582, 62.158675], [-79.65752, 61.63308], [-80.09956, 61.7181], [-80.36215, 62.01649], [-80.315395, 62.085565]], [[[-93.612756, 74.979997], [-94.156909, 74.592347], [-95.608681, 74.666864], [-96.820932, 74.927623], [-96.288587, 75.377828], [-94.85082, 75.647218], [-93.977747, 75.29649], [-93.612756, 74.979997]], [[[-93.840003, 77.519997], [-94.295608, 77.491343], [-96.169654, 77.555111], [-96.436304, 77.834629], [-94.422577, 77.820005], [-93.720656, 77.634331], [-93.840003, 77.519997]], [[[-96.754399, 78.765813], [-95.559278, 78.418315], [-95.830295, 78.056941], [-97.309843, 77.850597], [-98.124289, 78.082857], [-98.552868, 78.458105], [-98.631984, 78.87193], [-97.337231, 78.831984], [-96.754399, 78.765813]], [[[-88.15035, 74.392307], [-89.764722, 74.515555], [-92.422441, 74.837758], [-92.768285, 75.38682], [-92.889906, 75.826655], [-93.893824, 76.319244], [-95.962457, 76.441381], [-97.121379, 76.751078], [-96.745123, 77.161389], [-94.684086, 77.097878], [-93.573921, 76.776296], [-91.605023, 76.778518], [-90.741846, 76.449597], [-90.969661, 76.074013], [-89.822238, 75.847774], [-89.187083, 75.610166], [-87.838276, 75.566189], [-86.379192, 75.482421], [-84.789625, 75.699204], [-82.753445, 75.784315], [-81.128531, 75.713983], [-80.057511, 75.336849], [-79.833933, 74.923127], [-80.455771, 74.657304], [-81.948843, 74.442459], [-83.228894, 74.564028], [-86.097452, 74.410032], [-88.15035, 74.392307]], [[[-111.264443, 78.152956], [-109.854452, 77.996325], [-110.186938, 77.697015], [-112.051191, 77.409229], [-113.534279, 77.732207], [-112.724587, 78.05105], [-111.264443, 78.152956]], [[[-110.963661, 78.804441], [-109.663146, 78.601973], [-110.881314, 78.40692], [-112.542091, 78.407902], [-112.525891, 78.550555], [-111.50001, 78.849994], [-110.963661, 78.804441]], [[[-55.600218, 51.317075], [-56.134036, 50.68701], [-56.795882, 49.812309], [-56.143105, 50.150117], [-55.471492, 49.935815], [-55.822401, 49.587129], [-54.935143, 49.313011], [-54.473775, 49.556691], [-53.476549, 49.249139], [-53.786014, 48.516781], [-53.086134, 48.687804], [-52.958648, 48.157164], [-52.648099, 47.535548], [-53.069158, 46.655499], [-53.521456, 46.618292], [-54.178936, 46.807066], [-53.961869, 47.625207], [-54.240482, 47.752279], [-55.400773, 46.884994], [-55.997481, 46.91972], [-55.291219, 47.389562], [-56.250799, 47.632545], [-57.325229, 47.572807], [-59.266015, 47.603348], [-59.419494, 47.899454], [-58.796586, 48.251525], [-59.231625, 48.523188], [-58.391805, 49.125581], [-57.35869, 50.718274], [-56.73865, 51.287438], [-55.870977, 51.632094], [-55.406974, 51.588273], [-55.600218, 51.317075]], [[[-83.882626, 65.109618], [-82.787577, 64.766693], [-81.642014, 64.455136], [-81.55344, 63.979609], [-80.817361, 64.057486], [-80.103451, 63.725981], [-80.99102, 63.411246], [-82.547178, 63.651722], [-83.108798, 64.101876], [-84.100417, 63.569712], [-85.523405, 63.052379], [-85.866769, 63.637253], [-87.221983, 63.541238], [-86.35276, 64.035833], [-86.224886, 64.822917], [-85.883848, 65.738778], [-85.161308, 65.657285], [-84.975764, 65.217518], [-84.464012, 65.371772], [-83.882626, 65.109618]], [[[-78.770639, 72.352173], [-77.824624, 72.749617], [-75.605845, 72.243678], [-74.28616, 71.767144], [-74.099141, 71.33084], [-72.242226, 71.556925], [-71.200015, 70.920013], [-68.786054, 70.525024], [-67.91497, 70.121948], [-66.969033, 69.186087], [-68.805123, 68.720198], [-66.449866, 68.067163], [-64.862314, 67.847539], [-63.424934, 66.928473], [-61.851981, 66.862121], [-62.163177, 66.160251], [-63.918444, 64.998669], [-65.14886, 65.426033], [-66.721219, 66.388041], [-68.015016, 66.262726], [-68.141287, 65.689789], [-67.089646, 65.108455], [-65.73208, 64.648406], [-65.320168, 64.382737], [-64.669406, 63.392927], [-65.013804, 62.674185], [-66.275045, 62.945099], [-68.783186, 63.74567], [-67.369681, 62.883966], [-66.328297, 62.280075], [-66.165568, 61.930897], [-68.877367, 62.330149], [-71.023437, 62.910708], [-72.235379, 63.397836], [-71.886278, 63.679989], [-73.378306, 64.193963], [-74.834419, 64.679076], [-74.818503, 64.389093], [-77.70998, 64.229542], [-78.555949, 64.572906], [-77.897281, 65.309192], [-76.018274, 65.326969], [-73.959795, 65.454765], [-74.293883, 65.811771], [-73.944912, 66.310578], [-72.651167, 67.284576], [-72.92606, 67.726926], [-73.311618, 68.069437], [-74.843307, 68.554627], [-76.869101, 68.894736], [-76.228649, 69.147769], [-77.28737, 69.76954], [-78.168634, 69.826488], [-78.957242, 70.16688], [-79.492455, 69.871808], [-81.305471, 69.743185], [-84.944706, 69.966634], [-87.060003, 70.260001], [-88.681713, 70.410741], [-89.51342, 70.762038], [-88.467721, 71.218186], [-89.888151, 71.225552], [-90.20516, 72.235074], [-89.436577, 73.129464], [-88.408242, 73.537889], [-85.826151, 73.803816], [-86.562179, 73.157447], [-85.774371, 72.534126], [-84.850112, 73.340278], [-82.31559, 73.750951], [-80.600088, 72.716544], [-80.748942, 72.061907], [-78.770639, 72.352173]], [[[-94.503658, 74.134907], [-92.420012, 74.100025], [-90.509793, 73.856732], [-92.003965, 72.966244], [-93.196296, 72.771992], [-94.269047, 72.024596], [-95.409856, 72.061881], [-96.033745, 72.940277], [-96.018268, 73.43743], [-95.495793, 73.862417], [-94.503658, 74.134907]], [[[-122.854924, 76.116543], [-122.854925, 76.116543], [-121.157535, 76.864508], [-119.103939, 77.51222], [-117.570131, 77.498319], [-116.198587, 77.645287], [-116.335813, 76.879692], [-117.106051, 76.530032], [-118.040412, 76.481172], [-119.899318, 76.053213], [-121.499995, 75.900019], [-122.854924, 76.116543]], [[[-132.710008, 54.040009], [-131.74999, 54.120004], [-132.04948, 52.984621], [-131.179043, 52.180433], [-131.57783, 52.182371], [-132.180428, 52.639707], [-132.544992, 53.100015], [-133.054611, 53.411469], [-133.239664, 53.85108], [-133.180004, 54.169975], [-132.710008, 54.040009]], [[[-105.492289, 79.301594], [-103.529282, 79.165349], [-100.825158, 78.800462], [-100.060192, 78.324754], [-99.670939, 77.907545], [-101.30394, 78.018985], [-102.949809, 78.343229], [-105.176133, 78.380332], [-104.210429, 78.67742], [-105.41958, 78.918336], [-105.492289, 79.301594]], [[[-123.510002, 48.510011], [-124.012891, 48.370846], [-125.655013, 48.825005], [-125.954994, 47.919996], [-126.850004, 49.53], [-127.029993, 49.814996], [-128.059336, 49.994959], [-128.444584, 50.539138], [-128.358414, 50.770648], [-127.308581, 50.552574], [-126.695001, 50.400903], [-125.755007, 50.295018], [-125.415002, 49.950001], [-124.920768, 49.475275], [-123.922509, 49.062484], [-123.510002, 48.510011]], [[[-121.53788, 74.44893], [-120.10978, 74.24135], [-117.55564, 74.18577], [-116.58442, 73.89607], [-115.51081, 73.47519], [-116.76794, 73.22922], [-119.22, 72.52], [-120.46, 71.82], [-120.46, 71.383602], [-123.09219, 70.90164], [-123.62, 71.34], [-125.928949, 71.868688], [-125.5, 72.292261], [-124.80729, 73.02256], [-123.94, 73.68], [-124.91775, 74.29275], [-121.53788, 74.44893]], [[[-107.81943, 75.84552], [-106.92893, 76.01282], [-105.881, 75.9694], [-105.70498, 75.47951], [-106.31347, 75.00527], [-109.7, 74.85], [-112.22307, 74.41696], [-113.74381, 74.39427], [-113.87135, 74.72029], [-111.79421, 75.1625], [-116.31221, 75.04343], [-117.7104, 75.2222], [-116.34602, 76.19903], [-115.40487, 76.47887], [-112.59056, 76.14134], [-110.81422, 75.54919], [-109.0671, 75.47321], [-110.49726, 76.42982], [-109.5811, 76.79417], [-108.54859, 76.67832], [-108.2114, 76.20168], [-107.81943, 75.84552]], [[[-106.52259, 73.07601], [-105.40246, 72.67259], [-104.77484, 71.6984], [-104.46476, 70.99297], [-102.78537, 70.49776], [-100.98078, 70.02432], [-101.08929, 69.58447], [-102.73116, 69.50402], [-102.09329, 69.11962], [-102.43024, 68.75282], [-104.24, 68.91], [-105.96, 69.18], [-107.12254, 69.11922], [-109.68, 68.78], [-111.534149, 68.630059], [-113.3132, 68.53554], [-113.85496, 69.00744], [-115.22, 69.28], [-116.10794, 69.16821], [-117.34, 69.96], [-116.67473, 70.06655], [-115.13112, 70.2373], [-113.72141, 70.19237], [-112.4161, 70.36638], [-114.35, 70.6], [-116.48684, 70.52045], [-117.9048, 70.54056], [-118.43238, 70.9092], [-116.11311, 71.30918], [-117.65568, 71.2952], [-119.40199, 71.55859], [-118.56267, 72.30785], [-117.86642, 72.70594], [-115.18909, 73.31459], [-114.16717, 73.12145], [-114.66634, 72.65277], [-112.44102, 72.9554], [-111.05039, 72.4504], [-109.92035, 72.96113], [-109.00654, 72.63335], [-108.18835, 71.65089], [-107.68599, 72.06548], [-108.39639, 73.08953], [-107.51645, 73.23598], [-106.52259, 73.07601]], [[[-100.43836, 72.70588], [-101.54, 73.36], [-100.35642, 73.84389], [-99.16387, 73.63339], [-97.38, 73.76], [-97.12, 73.47], [-98.05359, 72.99052], [-96.54, 72.56], [-96.72, 71.66], [-98.35966, 71.27285], [-99.32286, 71.35639], [-100.01482, 71.73827], [-102.5, 72.51], [-102.48, 72.83], [-100.43836, 72.70588]], [[[-106.6, 73.6], [-105.26, 73.64], [-104.5, 73.42], [-105.38, 72.76], [-106.94, 73.46], [-106.6, 73.6]], [[[-98.5, 76.72], [-97.735585, 76.25656], [-97.704415, 75.74344], [-98.16, 75], [-99.80874, 74.89744], [-100.88366, 75.05736], [-100.86292, 75.64075], [-102.50209, 75.5638], [-102.56552, 76.3366], [-101.48973, 76.30537], [-99.98349, 76.64634], [-98.57699, 76.58859], [-98.5, 76.72]], [[[-96.01644, 80.60233], [-95.32345, 80.90729], [-94.29843, 80.97

727], [-94.73542, 81.20646], [-92.40984, 81.25739], [-91.13289, 80.72345], [-89.45, 80.509322], [-87.81, 80.32], [-87.02, 79.66], [-85.81435, 79.3369], [-87.18756, 79.0393], [-89.03535, 78.28723], [-90.80436, 78.21533], [-92.87669, 78.34333], [-93.95116, 78.75099], [-93.93574, 79.11373], [-93.14524, 79.3801], [-94.974, 79.37248], [-96.07614, 79.70502], [-96.70972, 80.15777], [-96.01644, 80.60233]], [[[-91.58702, 81.89429], [-90.1, 82.085], [-88.93227, 82.11751], [-86.97024, 82.27961], [-85.5, 82.652273], [-84.260005, 82.6], [-83.18, 82.32], [-82.42, 82.86], [-81.1, 83.02], [-79.30664, 83.13056], [-76.25, 83.172059], [-75.71878, 83.06404], [-72.83153, 83.23324], [-70.665765, 83.169781], [-68.5, 83.106322], [-65.82735, 83.02801], [-63.68, 82.9], [-61.85, 82.6286], [-61.89388, 82.36165], [-64.334, 81.92775], [-66.75342, 81.72527], [-67.65755, 81.50141], [-65.48031, 81.50657], [-67.84, 80.9], [-69.4697, 80.61683], [-71.18, 79.8], [-73.2428, 79.63415], [-73.88, 79.430162], [-76.90773, 79.32309], [-75.52924, 79.19766], [-76.22046, 79.01907], [-75.39345, 78.52581], [-76.34354, 78.18296], [-77.88851, 77.89991], [-78.36269, 77.50859], [-79.75951, 77.20968], [-79.61965, 76.98336], [-77.91089, 77.022045], [-77.88911, 76.77955], [-80.56125, 76.17812], [-83.17439, 76.45403], [-86.11184, 76.29901], [-87.6, 76.42], [-89.49068, 76.47239], [-89.6161, 76.95213], [-87.76739, 77.17833], [-88.26, 77.9], [-87.65, 77.970222], [-84.97634, 77.53873], [-8

6.34, 78.18], [-87.96192, 78.37181], [-87.15198, 78.75867], [-85.37868, 78.9969], [-85.09495, 79.34543], [-86.50734, 79.73624], [-86.93179, 80.25145], [-84.19844, 80.20836], [-83.408696, 80.1], [-81.84823, 80.46442], [-84.1, 80.58], [-87.59895, 80.51627], [-89.36663, 80.85569], [-90.2, 81.26], [-91.36786, 81.5531], [-91.58702, 81.89429]], [[[-75.21597, 67.44425], [-75.86588, 67.14886], [-76.98687, 67.09873], [-77.23634, 67.58809], [-76.81166, 68.14856], [-75.89521, 68.28721], [-75.1145, 68.01036], [-75.10333, 67.58202], [-75.21597, 67.44425]], [[[-96.257401, 69.49003], [-95.647681, 69.10769], [-96.269521, 68.75704], [-97.617401, 69.06003], [-98.431801, 68.9507], [-99.797401, 69.40003], [-98.917401, 69.71003], [-98.218261, 70.14354], [-97.157401, 69.86003], [-96.557401, 69.68003], [-96.257401, 69.49003]], [[[-64.51912, 49.87304], [-64.17322, 49.95718], [-62.85829, 49.70641], [-61.835585, 49.28855], [-61.806305, 49.10506], [-62.29318, 49.08717], [-63.58926, 49.40069], [-64.51912, 49.87304]], [[[-64.01486, 47.03601], [-63.6645, 46.55001], [-62.9393, 46.41587], [-62.01208, 46.44314], [-62.50391, 46.03339], [-62.87433, 45.96818], [-64.1428, 46.39265], [-64.39261, 46.72747], [-64.01486, 47.03601]]]], {"type": "Feature", "properties": {"name": " ", "bbox": [-171.79111, 18.91619, -66.96466, 71.357764], "geometry": {"type": "MultiPolygon", "coordinates": [[[[[-122.84, 49], [-120, 49], [-117.03121, 49], [-116.04818, 49], [-113, 49], [-110.05, 49], [-107.05, 49], [-104.04826, 48.99986], [-100.65, 49], [-97.22872, 49.0007], [-95.15907, 49], [-95.15609, 49.38425], [-94.81758, 49.38905], [-94.64, 48.84], [-94.32914, 48.67074], [-93.63087, 48.60926], [-92.61, 48.45], [-91.64, 48.14], [-90.83, 48.27], [-89.6, 48.01], [-89.272917, 48.019808], [-88.378114, 48.302918], [-87.439793, 47.94], [-86.461991, 47.553338], [-85.652363, 47.220219], [-84.87608, 46.900083], [-84.779238, 46.637102], [-84.543749, 46.538684], [-84.6049, 46.4396], [-84.3367, 46.40877], [-84.14212, 46.512226], [-84.091851, 46.275419], [-83.890765, 46.116927], [-83.616131, 46.116927], [-83.469551, 45.994686], [-83.592851, 45.816894], [-82.550925, 45.347517], [-82.337763, 44.44], [-82.137642, 43.571088], [-82.43, 42.98], [-82.9, 42.43], [-83.12, 42.08], [-83.142, 41.975681], [-83.02981, 41.832796], [-82.690089, 41.675105], [-82.439278, 41.675105], [-81.277747, 42.209026], [-80.247448, 42.3662], [-78.939362, 42.863611], [-78.92, 42.965], [-79.01, 43.27], [-79.171674, 43.466339], [-78.72028, 43.625089], [-77.737885, 43.629056], [-76.820034, 43.628784], [-76.5, 44.018459], [-76.375, 44.09631], [-75.31821, 44.81645], [-74.867, 45.00048], [-73.34783, 45.00738], [-71.50506, 45.00822], [-71.405, 45.255], [-71.08482, 45.30524], [-70.66, 45.46], [-70.305, 45.915], [-69.99997, 46.69307], [-69.237216, 47.447781], [-68.905, 47.185], [-68.23444, 47.35486], [-67.79046, 47.06636], [-67.79134, 45.70281], [-67.13741, 45.13753], [-66.96466, 44.8097], [-68.03252, 44.3252], [-69.06, 43.98], [-70.11617, 43.68405], [-70.645476, 43.090238], [-70.81489, 42.8653], [-70.825, 42.335], [-70.495, 41.805], [-70.08, 41.78], [-70.185, 42.145], [-69.88497, 41.92283], [-69.96503, 41.63717], [-70.64, 41.475], [-71.12039, 41.49445], [-71.86, 41.32], [-72.295, 41.27], [-72.87643, 41.22065], [-73.71, 40.931102], [-72.24126, 41.11948], [-71.945, 40.93], [-73.345, 40.63], [-73.982, 40.628], [-73.952325, 40.75075], [-74.25671, 40.47351], [-73.96244, 40.42763], [-74.17838, 39.70926], [-74.90604, 38.93954], [-74.98041, 39.1964], [-75.20002, 39.24845], [-75.52805, 39.4985], [-75.32, 38.96], [-75.071835, 38.782032], [-75.05673, 38.40412], [-75.37747, 38.01551], [-75.94023, 37.21689], [-76.03127, 37.2566], [-75.72205, 37.93705], [-76.23287, 38.319215], [-76.35, 39.15], [-76.542725, 38.717615], [-76.32933, 38.08326], [-76.989998, 38.239992], [-76.30162, 37.917945], [-76.25874, 36.9664], [-75.9718, 36.89726], [-75.86804, 36.55125], [-75.72749, 35.55074], [-76.3631, 34.80854], [-77.397635, 34.51201], [-78.05496, 33.92547], [-78.55435, 33.86133], [-79.06067, 33.49395], [-79.20357, 33.15839], [-80.301325, 32.509355], [-80.86498, 32.0333], [-81.33629, 31.44049], [-81.49042, 30.72999], [-81.31371, 30.03552], [-80.98, 29.18], [-80.535585, 28.47213], [-80.53, 28.04], [-80.056539, 26.88], [-80.088015, 26.205765], [-80.13156, 25.816775], [-80.38103, 25.20616], [-80.68, 25.08], [-81.17213, 25.20126], [-81.33, 25.64], [-81.71, 25.87], [-82.24, 26.73], [-82.70515, 27.49504], [-82.85526, 27.88624], [-82.65, 28.55], [-82.93, 29.1], [-83.70959, 29.93656], [-84.1, 30.09], [-85.10882, 29.63615], [-85.28784, 29.68612], [-85.7731, 30.15261], [-86.4, 30.4], [-87.53036, 30.27433], [-88.41782, 30.3849], [-89.18049, 30.31598], [-89.593831, 30.159994], [-89.413735, 29.89419], [-89.43, 29.48864], [-89.21767, 29.29108], [-89.40823, 29.15961], [-89.77928, 29.30714], [-90.15463, 29.11743], [-90.880225, 29.148535], [-91.626785, 29.677], [-92.49906, 29.5523], [-93.22637, 29.78375], [-93.84842, 29.71363], [-94.69, 29.48], [-95.60026, 28.73863], [-96.59404, 28.30748], [-97.14, 27.83], [-97.37, 27.38], [-97.38, 26.69], [-97.33, 26.21], [-97.14, 25.87], [-97.53, 25.84], [-98.24, 26.06], [-99.02, 26.37], [-99.3, 26.84], [-99.52, 27.54], [-100.11, 28.11], [-100.45584, 28.69612], [-100.9576, 29.38071], [-101.6624, 29.7793], [-102.48, 29.76], [-103.11, 28.97], [-103.94, 29.27], [-104.45697, 29.57196], [-104.70575, 30.12173], [-105.03737, 30.64402], [-105.63159, 31.08383], [-106.1429, 31.39995], [-106.50759, 31.75452], [-108.24, 31.754854], [-108.24194, 31.34222], [-109.035, 31.34194], [-111.02361, 31.33472], [-113.30498, 32.03914], [-114.815, 32.52528], [-114.72139, 32.72083], [-115.99135, 32.61239], [-117.12776, 32.53534], [-117.295938, 32.04622], [-117.944, 33.621236], [-118.410602, 33.740909], [-118.519895, 34.027782], [-119.081, 34.078], [-119.438841, 34.348477], [-120.36778, 34.44711], [-120.62286, 34.60855], [-120.74433, 35.15686], [-121.71457, 36.16153], [-122.54747, 37.55176], [-122.51201, 37.78339], [-122.95319, 38.11371], [-123.7272, 38.95166], [-123.86517, 39.76699], [-124.39807, 40.3132], [-124.17886, 41.14202], [-124.2137, 41.99964], [-124.53284, 42.76599], [-124.14214, 43.70838], [-124.020535, 44.615895], [-123.89893, 45.52341], [-124.079635, 46.86475], [-124.39567, 47.72017], [-124.68721, 48.184433], [-124.566101, 48.379715], [-123.12, 48.04], [-122.58736, 47.096], [-122.34, 47.36], [-122.5, 48.18], [-122.84, 49]], [[[-155.40214, 20.07975], [-155.22452, 19.99302], [-155.06226, 19.8591], [-154.80741, 19.50871], [-154.83147, 19.45328], [-155.22217, 19.23972], [-155.54211, 19.08348], [-155.68817, 18.91619], [-155.93665, 19.05939], [-155.90806, 19.33888], [-156.07347, 19.70294], [-156.02368, 19.81422], [-155.85008, 19.97729], [-155.91907, 20.17395], [-155.86108, 20.26721], [-155.78505, 20.2487], [-155.40214, 20.07975]], [[[-155.99566, 20.76404], [-156.07926, 20.64397], [-156.41445, 20.57241], [-156.58673, 20.783], [-156.70167, 20.8643], [-156.71055, 20.92676], [-156.61258, 21.01249], [-156.25711, 20.91745], [-155.99566, 20.76404], [[[-156.75824, 21.17684], [-156.78933, 21.06873], [-157.32521, 21.09777], [-157.25027, 21.21958], [-156.75824, 21.17684]], [[[-158.0252, 21.71696], [-157.94161, 21.65272], [-157.65283, 21.32217], [-157.70703, 21.26442], [-157.7786, 21.27729], [-158.12667, 21.31244], [-158.2538, 21.53919], [-158.29265, 21.57912], [-158.0252, 21.71696]], [[[-159.36569, 22.21494], [-159.34512, 21.982], [-159.46372, 21.88299], [-159.80051, 22.06533], [-159.74877, 22.1382], [-159.5962, 22.23618], [-159.36569, 22.21494]], [[[-166.467792, 60.38417], [-165.67443, 60.293607], [-165.579164, 59.909987], [-166.19277, 59.754441], [-166.848337, 59.941406], [-167.455277, 60.213069], [-166.467792, 60.38417]], [[[-153.228729, 57.968968], [-152.564791, 57.901427], [-152.141147, 57.591059], [-153.006314, 57.115842], [-154.00509, 56.734677], [-154.516403, 56.992749], [-154.670993, 57.461196], [-153.76278, 57.816575], [-153.228729, 57.968968]], [[[-140.985988, 69.711998], [-140.986, 69.712], [-140.9925, 66.00003], [-140.99778, 60.30639], [-140.013, 60.27682], [-139.039, 60], [-138.34089, 59.56211], [-137.4525, 58.905], [-136.47972, 59.46389], [-135.47583, 59.78778], [-134.945, 59.27056], [-134.27111, 58.86111], [-133.35556, 58.41028], [-132.73042, 57.69289], [-131.70781, 56.55212], [-130.00778, 55.91583], [-129.98, 55.285], [-130.53611, 54.80278], [-130.53611, 54.80275], [-131.085818, 55.178906], [-131.967211, 55.497776], [-132.250011, 56.369996], [-133.539181, 57.178887], [-134.078063, 58.123068], [-135.038211, 58.187715], [-136.628062, 58.212209], [-137.800006, 58.499995], [-139.867787, 59.537762], [-140.825274, 59.727517], [-142.574444, 60.084447], [-143.958881, 59.99918], [-145.925557, 60.45861], [-147.114374, 60.884656], [-148.224306, 60.672989], [-148.018066, 59.978329], [-148.570823, 59.914173], [-149.727858, 59.705658], [-150.608243, 59.368211], [-151.716393, 59.155821], [-151.859433, 59.744984], [-151.409719, 60.725803], [-150.346941, 61.033588], [-150.621111, 61.284425], [-151.859839, 60.727198], [-152.57833, 60.061657], [-154.019172, 59.350279], [-153.287511, 58.864728], [-154.232492, 58.146374], [-155.307491, 57.727795], [-156.308335, 57.422774], [-156.556097, 56.979985], [-158.117217, 56.463608], [-158.433321, 55.994154], [-159.603327, 55.566686], [-160.28972, 55.643581], [-161.223048, 55.364735], [-162.237766, 55.024187], [-163.069447, 54.689737], [-164.785569, 54.404173], [-164.942226, 54.572225], [-163.84834, 55.039431], [-162.870001, 55.348043], [-161.804175, 55.894986], [-160.5

63605,56.008055],[-160.07056,56.418055],[-158.684443,57.016675],[-158.461097,57.216921],[-157.72277,57.570001],[-157.550274,58.328326],[-157.041675,58.918885],[-158.194731,58.615802],[-158.517218,58.787781],[-159.058606,58.424186],[-159.711667,58.93139],[-159.981289,58.572549],[-160.355271,59.071123],[-161.355003,58.670838],[-161.968894,58.671665],[-162.054987,59.266925],[-161.874171,59.633621],[-162.518059,59.989724],[-163.818341,59.798056],[-164.662218,60.267484],[-165.346388,60.507496],[-165.350832,61.073895],[-166.121379,61.500019],[-165.734452,62.074997],[-164.919179,62.633076],[-164.562508,63.146378],[-163.753332,63.219449],[-163.067224,63.059459],[-162.260555,63.541936],[-161.53445,63.455817],[-160.772507,63.766108],[-160.958335,64.222799],[-161.518068,64.402788],[-160.777778,64.788604],[-161.391926,64.777235],[-162.45305,64.559445],[-162.757786,64.338605],[-163.546394,64.55916],[-164.96083,64.446945],[-166.425288,64.686672],[-166.845004,65.088896],[-168.11056,65.669997],[-166.705271,66.088318],[-164.47471,66.57666],[-163.652512,66.57666],[-163.788602,66.07207],[-161.677774,66.11612],[-162.489715,66.735565],[-163.719717,67.116395],[-164.430991,67.616338],[-165.390287,68.042772],[-166.764441,68.358877],[-166.204707,68.883031],[-164.430811,68.915535],[-163.168614,69.371115],[-162.930566,69.858062],[-161.908897,70.33333],[-160.934797,70.44769],[-159.039176,70.891642],[-158.119723,70.824721],[-156.580825,71.35776

4],[-155.06779,71.147776],[-154.344165,70.696409],[-153.900006,70.889989],[-152.210006,70.829992],[-152.270002,70.600006],[-150.739992,70.430017],[-149.720003,70.53001],[-147.613362,70.214035],[-145.68999,70.12001],[-144.920011,69.989992],[-143.589446,70.152514],[-142.07251,69.851938],[-140.985988,69.711998],[-140.985988,69.711998]]],[[-171.731657,63.782515],[[-171.731657,63.782515]],{"type":"Feature","properties":{"name":"","bbox":[46.466446,40.662325,87.35997,55.38525],"geometry":{"type":"Polygon","coordinates":[[[87.35997,49.214981],[86.598776,48.549182],[85.768233,48.455751],[85.720484,47.452969],[85.16429,47.000956],[83.180484,47.330031],[82.458926,45.53965],[81.947071,45.317027],[79.966106,44.917517],[80.866206,43.180362],[80.18015,42.920006],[80.25999,42.349999],[79.643645,42.496683],[79.142177,42.856092],[77.658392,42.960686],[76.000354,42.988022],[75.636965,42.8779],[74.212866,43.298339],[73.645304,43.091272],[73.489758,42.500894],[71.844638,42.845395],[71.186281,42.704293],[70.962315,42.266154],[70.388965,42.081308],[69.070027,41.384244],[68.632483,40.666881],[68.259896,40.662325],[67.985856,41.135991],[66.714047,41.168444],[66.510649,41.987644],[66.023392,41.994646],[66.098012,42.99766],[64.900824,43.728081],[63.185787,43.650075],[62.0133,43.504477],[61.05832,44.405817],[60.239972,44.784037],[58.689989,45.500014],[58.503127,45.586804],[55.928917,44.995858],[55.968191,41.308642],[55.455251,41.259859],[54.755345,42.043971],[54.079418,42.324109],[52.944293,42.116034],[52.50246,41.783316],[52.446339,42.027151],[52.692112,42.443895],[52.501426,42.792298],[51.342427,43.132975],[50.891292,44.031034],[50.339129,44.284016],[50.305643,44.609836],[51.278503,44.514854],[51.316899,45.245998],[52.16739,45.408391],[53.040876,45.259047],[53.220866,46.234646],[53.042737,46.853006],[52.042023,46.804637],[51.191945,47.048705],[50.034083,46.60899],[49.10116,46.39933],[48.59325,46.56104],[48.694734,47.075628],[48.05725,47.74377],[47.31524,47.71585],[46.466446,48.394152],[47.043672,49.152039],[46.751596,49.356006],[47.54948,50.454698],[48.577841,49.87476],[48.702382,50.605128],[50.766648,51.692762],[52.328724,51.718652],[54.532878,51.02624],[55.71694,50.62171],[56.77798,51.04355],[58.36332,51.06364],[59.642282,50.545442],[59.932807,50.842194],[61.337424,50.79907],[61.588003,51.272659],[59.967534,51.96042],[60.927269,52.447548],[60.739993,52.719986],[61.699986,52.979996],[60.978066,53.664993],[61.4366,54.00625],[65.178534,54.354228],[65.66687,54.60125],[68.1691,54.970392],[69.068167,55.38525],[70.865267,55.169734],[71.180131,54.133285],[72.22415,54.376655],[73.508516,54.035617],[73.425679,53.40981],[74.38482,53.54685],[76.8911,54.490524],[76.525179,54.177003],[77.800916,53.404415],[80.03556,50.864751],[80.568447,51.388336],[81.945986,50.812196],[83.383004,51.069183],[83.935115,50.889246],[84.416377,50.3114],[85.11556,50.117303],[85.54127,49.692859],[86.829357,49.826675],[87.35997,49.214981]]]],{"type":"Feature","properties":{"name":"","bbox":[55.928917,37.144994,73.055417,45.586804],"geometry":{"type":"Polygon","coordinates":[[[55.968191,41.308642],[55.928917,44.995858],[58.503127,45.586804],[58.689989,45.500014],[60.239972,44.784037],[61.05832,44.405817],[62.0133,43.504477],[63.185787,43.650075],[64.900824,43.728081],[66.098012,42.99766],[66.023392,41.994646],[66.510649,41.987644],[66.714047,41.168444],[67.985856,41.135991],[68.259896,40.662325],[68.632483,40.666881],[69.070027,41.384244],[70.388965,42.081308],[70.962315,42.266154],[71.180131,42.167711],[70.420022,41.519998],[71.157859,41.143587],[71.870115,41.3929],[73.055417,40.866033],[71.774875,40.145844],[71.014198,40.244366],[70.601407,40.218527],[70.45816,40.496495],[70.666622,40.960213],[69.329495,40.727824],[69.011633,40.086158],[68.536416,39.533453],[67.701429,39.580478],[67.44222,39.140144],[68.176025,38.901553],[68.392033,38.157025],[67.83,37.144994],[67.075782,37.356144],[66.518607,37.362784],[66.54615,37.974685],[65.215999,38.402695],[64.170223,38.892407],[63.518015,39.363257],[62.37426,40.053886],[61.882714,41.084857],[61.547179,41.26637],[60.465953,41.220327],[60.083341,41.425146],[59.976422,42.223082],[58.629011,42.751551],[57.78653,42.170553],[56.932215,41.826026],[57.096391,41.32231],[55.968191,41.308642]]]],{"type":"Feature","properties":{"name":"","bbox":[141.00021,-10.652476,156.019965,-2.500002],"geometry":{"type":"MultiPolygon","coordinates":[[[141.00021,-2.600151],[142.735247,-3.289153],[144.583971,-3.861418],[145.27318,-4.373738],[145.829786,-4.876498],[145.981922,-5.465609],[147.648073,-6.083659],[147.891108,-6.614015],[146.970905,-6.721657],[147.191874,-7.388024],[148.084636,-8.044108],[148.734105,-9.104664],[149.306835,-9.071436],[149.266631,-9.514406],[150.038728,-9.684318],[149.738798,-9.872937],[150.801628,-10.293687],[150.690575,-10.582713],[150.028393,-10.652476],[149.78231,-10.393267],[148.923138,-10.280923],[147.913018,-10.130441],[147.135443,-9.492444],[146.567881,-8.942555],[146.048481,-8.067414],[144.744168,-7.630128],[143.897088,-7.91533],[143.286376,-8.245491],[143.413913,-8.983069],[142.628431,-9.326821],[142.068259,-9.159596],[141.033852,-9.117893],[141.017057,-5.859022],[141.00021,-2.600151]]],[[152.640017,-3.659983],[152.827292,-4.766427],[152.638673,-4.176127],[152.406026,-3.789743],[151.953237,-3.462062],[151.384279,-3.035422],[150.66205,-2.741486],[150.939965,-2.500002],[151.479984,-2.779985],[151.820015,-2.999972],[152.239989,-3.240009],[152.640017,-3.659983],[151.30139,-5.840728],[150.754447,-6.083763],[150.241197,-6.317754],[149.709963,-6.316513],[148.890065,-6.02604],[148.318937,-5.747142],[148.401826,-5.447756],[149.298412,-5.583742],[149.845562,-5.505503],[149.99625,-5.026101],[150.139756,-5.001348],[150.236908,-5.53222],[150.807467,-5.455842],[151.089672,-5.113693],[151.647881,-4.757074],[151.537862,-4.167807],[152.136792,-4.14879],[152.338743,-4.312966],[152.318693,-4.867661],[151.982796,-5.478063],[151.459107,-5.56028],[151.30139,-5.840728]]],[[154.759991,-5.339984],[155.062918,-5.566792],[155.547746,-6.200655],[156.019965,-6.540014],[155.880026,-6.819997],[155.599991,-6.919991],[155.166994,-6.535931],[154.729192,-5.900828],[154.514114,-5.139118],[154.652504,-5.042431],[154.759991,-5.339984]]]],{"type":"Feature","properties":{"name":"","bbox":[293026,-10.359987,141.033852,5.479821],"geometry":{"type":"MultiPolygon","coordinates":[[[141.00021,-2.600151],[141.017057,-5.859022],[141.033852,-9.117893],[140.143415,-8.297168],[139.127767,-8.096043],[138.881477,-8.380935],[137.614474,-8.411683],[138.039099,-7.597882],[138.668621,-7.320225],[138.407914,-6.232849],[137.92784,-5.393366],[135.98925,-4.546544],[135.164598,-4.462931],[133.66288,-3.538853],[133.367705,-4.024819],[132.983956,-4.112979],[132.756941,-3.746283],[132.753789,-3.311787],[131.989804,-2.820551],[133.066845,-2.460418],[133.780031,-2.479848],[133.696212,-2.214542],[132.232373,-2.212526],[131.836222,-1.617162],[130.94284,-1.432522],[130.519558,-0.93772],[131.867538,-0.695461],[132.380116,-0.369538],[133.985548,-0.78021],[134.143368,-1.151867],[134.422627,-2.769185],[135.457603,-3.367753],[136.293314,-2.307042],[137.440738,-1.703513],[138.329727,-1.702686],[139.184921,-2.051296],[139.926684,-2.409052],[141.00021,-2.600151]]],[[124.968682,-8.89279],[125.07002,-9.089987],[125.08852,-9.393173],[124.43595,-10.140001],[123.579982,-10.359987],[123.459989,-10.239995],[123.550009,-9.900016],[123.980009,-9.290027],[124.968682,-8.89279]]],[[134.210134,-6.895238],[134.112776,-6.142467],[134.290336,-5.783058],[134.499625,-5.445042],[134.727002,-5.737582],[134.724624,-6.214401],[134.210134,-6.895238]]],[[117.882035,4.137551],[117.313232,3.234428],[118.04833,2.28769],[117.875627,1.827641],[118.996747,0.902219],[117.811858,0.784242],[117.478339,0.102475],[117.521644,-0.803723],[116.560048,-1.487661],[116.533797,-2.483517],[116.148084,-4.012726],[116.000858,-3.657037],[114.864803,-4.106984],[114.468652,-3.495704],[113.755672,-3.43917],[113.256994,-3.118776],[112.068126,-3.478392],[111.703291,-2.994442],[111.04824,-3.049426],[110.223846,-2.934032],[110.070936,-1.592874],[109.571948,-1.314907],[109.091874,-0.459507],[108.952658,0.415375],[109.069136,1.341934],[109.66326,2.006467],[109.830227,1.338136],[110.514061,0.773131],[111.159138,0.976478],[111.797548,0.904441],[112.380252,1.410121],[112.859809,1

.49779],[113.80585,1.217549],[114.621355,1.430688],[115.134037,2.821482],[115.519078,3.169238],[115.865517,4.306559],[117.015214,4.306094],[117.882035,4.137551]]],[[129.370998,-2.802154],[130.471344,-3.093764],[130.834836,-3.858472],[129.990547,-3.446301],[129.155249,-3.362637],[128.590684,-3.428679],[127.898891,-3.393436],[128.135879,-2.843651],[129.370998,-2.802154]]],[[126.874923,-3.790983],[126.183802,-3.607376],[125.989034,-3.177273],[127.000651,-3.129318],[127.249215,-3.459065],[126.874923,-3.790983]]],[[127.932378,2.174596],[128.004156,1.628531],[128.594559,1.540811],[128.688249,1.132386],[128.635952,0.258486],[128.12017,0.356413],[127.968034,-0.252077],[128.379999,-0.780004],[128.100016,-0.899996],[127.696475,-0.266598],[127.39949,1.011722],[127.600512,1.810691],[127.932378,2.174596]]],[[122.927567,0.875192],[124.077522,0.917102],[125.065989,1.643259],[125.240501,1.419836],[124.437035,0.427881],[123.685505,0.235593],[122.723083,0.431137],[121.056725,0.381217],[120.183083,0.237247],[120.04087,-0.519658],[120.935905,-1.408906],[121.475821,-0.955962],[123.340565,-0.615673],[123.258399,-1.076213],[122.822715,-0.930951],[122.38853,-1.516858],[121.508274,-1.904483],[122.454572,-3.186058],[122.271896,-3.5295],[123.170963,-4.683693],[123.162333,-5.340604],[122.628515,-5.634591],[122.236394,-5.282933],[122.719569,-4.464172],[121.738234,-4.851331],[121.489463,-4.574553],[121.619171,-4.188478],[120.898182,-3.602105],[120.972389,-2.627643],[120.305453,-2.931604],[120.390047,-4.097579],[120.430717,-5.528241],[119.796543,-5.6734],[119.180974,-2.147104],[119.323394,-1.353147],[119.825999,0.154254],[120.035702,0.566477],[120.885779,1.309223],[121.66817,1.013944],[122.927567,0.875192]]],[[120.295014,-10.25865],[118.967808,-9.557969],[119.90031,-9.36134],[120.425756,-9.665921],[120.775502,-9.969675],[120.715609,-10.239581],[120.295014,-10.25865]]],[[121.341669,-8.53674],[122.007365,-8.46062],[122.903537,-8.094234],[122.756983,-8.649808],[121.254491,-8.933666],[119.924391,-8.810418],[119.920929,-8.444859],[120.715092,-8.236965],[121.341669,-8.53674]]],[[118.260616,-8.362383],[118.87846,-8.280683],[119.126507,-8.705825],[117.970402,-8.906639],[117.277731,-9.040895],[116.740141,-9.032937],[117.083737,-8.457158],[117.632024,-8.449303],[117.900018,-8.095681],[118.260616,-8.362383]]],[[108.486846,-6.421985],[108.623479,-6.777674],[110.539227,-6.877358],[110.759576,-6.465186],[112.614811,-6.946036],[112.978768,-7.594213],[114.478935,-7.776528],[115.705527,-8.370807],[114.564511,-8.751817],[113.464734,-8.348947],[112.559672,-8.376181],[111.522061,-8.302129],[110.58615,-8.122605],[109.427667,-7.740664],[108.693655,-7.6416],[108.277763,-7.766657],[106.454102,-7.3549],[106.280624,-6.9249],[105.365486,-6.851416],[106.051646,-5.895919],[107.265009,-5.954985],[108.072091,-6.345762],[108.486846,-6.421985]]],[[104.369991,-1.084843],[104.53949,-1.782372],[104.887893,-2.340425],[105.622111,-2.428844],[106.108593,-3.061777],[105.857446,-4.305525],[105.817655,-5.852356],[104.710384,-5.873285],[103.868213,-5.037315],[102.584261,-4.220259],[102.156173,-3.614146],[101.399113,-2.799777],[100.902503,-2.050262],[100.141981,-0.650348],[99.26374,0.183142],[98.970011,1.042882],[98.601351,1.823507],[97.699598,2.453184],[97.176942,3.308791],[96.424017,3.86886],[95.380876,4.970782],[95.293026,5.479821],[95.936863,5.439513],[97.484882,5.246321],[98.369169,4.26837],[99.142559,3.59035],[99.693998,3.174329],[100.641434,2.099381],[101.658012,2.083697],[102.498271,1.3987],[103.07684,0.561361],[103.838396,0.104542],[103.437645,-0.711946],[104.010789,-1.059212],[104.369991,-1.084843]]]],{"type":"Feature","properties":{"name":"","bbox":[-73.415436,-55.25,53.628349,-21.83231],"geometry":{"type":"MultiPolygon","coordinates":[[[-68.63401,-52.63637],[-68.25,-53.1],[-67.75,-53.85],[-66.45,-54.45],[-65.05,-54.7],[-65.5,-55.2],[-66.45,-55.25],[-66.95992,-54.89681],[-67.56244,-54.87001],[-68.63335,-54.8695],[-68.63401,-52.63637]]],[[-57.625133,-30.216295],[-57.874937,-31.016556],[-58.14244,-32.044504],[-58.132648,-33.040567],[-58.349611,-33.263189],[-58.427074,-33.909454],[-58.495442,-34.43149],[-57.22583,-35.288027],[-57.362359,-35.97739],[-56.737487,-36.413126],[-56.788285,-36.901572],[-57.749157,-38.183871],[-59.231857,-38.72022],[-61.237445,-38.928425],[-62.335957,-38.827707],[-62.125763,-39.424105],[-62.330531,-40.172586],[-62.145994,-40.676897],[-62.745803,-41.028761],[-63.770495,-41.166789],[-64.73209,-40.802677],[-65.118035,-41.064315],[-64.978561,-42.058001],[-64.303408,-42.359016],[-63.755948,-42.043687],[-63.458059,-42.563138],[-64.378804,-42.873558],[-65.181804,-43.495381],[-65.328823,-44.501366],[-65.565269,-45.036786],[-66.509966,-45.039628],[-67.293794,-45.551896],[-67.580546,-46.301773],[-66.597066,-47.033925],[-65.641027,-47.236135],[-65.985088,-48.133289],[-67.166179,-48.697337],[-67.816088,-49.869669],[-68.728745,-50.264218],[-69.138539,-50.73251],[-68.815561,-51.771104],[-68.149995,-52.349983],[-68.571545,-52.299444],[-69.498362,-52.142761],[-71.914804,-52.009022],[-72.329404,-51.425956],[-72.309974,-50.67701],[-72.975747,-50.74145],[-73.328051,-50.378785],[-73.415436,-49.318436],[-72.648247,-48.878618],[-72.331161,-48.244238],[-72.447355,-47.738533],[-71.917258,-46.884838],[-71.552009,-45.560733],[-71.659316,-44.973689],[-71.222779,-44.784243],[-71.329801,-44.407522],[-71.793623,-44.207172],[-71.464056,-43.787611],[-71.915424,-43.408565],[-72.148898,-42.254888],[-71.746804,-42.051386],[-71.915734,-40.832339],[-71.680761,-39.808164],[-71.41517,-38.916022],[-70.814664,-38.552995],[-71.118625,-37.576827],[-71.121881,-36.658124],[-70.364769,-36.005089],[-70.388049,-35.169688],[-69.817309,-34.193571],[-69.814777,-33.273886],[-70.074399,-33.091211],[-70.535069,-31.36501],[-69.919008,-30.336339],[-70.01355,-29.367923],[-69.65613,-28.459141],[-69.001235,-27.521214],[-68.295542,-26.89934],[-68.5948,-26.506909],[-68.386001,-26.185016],[-68.417653,-24.518555],[-67.328443,-24.025303],[-66.985234,-22.986349],[-67.106674,-22.735925],[-66.273339,-21.83231],[-64.964892,-22.075862],[-64.377021,-22.798091],[-63.986838,-21.993644],[-62.846468,-22.034985],[-62.685057,-22.249029],[-60.846565,-23.880713],[-60.028966,-24.032796],[-58.807128,-24.771459],[-57.777217,-25.16234],[-57.63366,-25.603657],[-58.618174,-27.123719],[-57.60976,-27.395899],[-56.486702,-27.548499],[-55.695846,-27.387837],[-54.788795,-26.621786],[-54.625291,-25.739255],[-54.13005,-25.547639],[-53.628349,-26.124865],[-53.648739,-26.923473],[-54.490725,-27.474757],[-55.162286,-27.881915],[-56.2909,-28.852761],[-57.625133,-30.216295]]]]}},{"type":"Feature","properties":{"name":"","bbox":[-75.644395,-55.61183,-66.95992,-17.580012],"geometry":{"type":"MultiPolygon","coordinates":[[[-68.63401,-52.63637],[-68.63335,-54.8695],[-67.56244,-54.87001],[-66.95992,-54.89681],[-67.29103,-55.30124],[-68.14863,-55.61183],[-68.639991,-55.580018],[-69.2321,-55.49906],[-69.95809,-55.19843],[-71.00568,-55.05383],[-72.2639,-54.49514],[-73.2852,-53.95752],[-74.66253,-52.83749],[-73.8381,-53.04743],[-72.43418,-53.7154],[71.10773,-54.07433],[-70.59178,-53.61583],[-70.26748,-52.93123],[-69.34565,-52.5183],[68.63401,-52.63637]]],[[-69.590424,-17.580012],[-69.100247,-18.260125],[-68.966818,-18.981683],[-68.442225,-19.405068],[-68.757167,-20.372658],[-68.219913,-21.494347],[-67.82818,-22.872919],[-67.106674,-22.735925],[-66.985234,-22.986349],[-67.328443,-24.025303],[-68.417653,-24.518555],[-68.386001,-26.185016],[-68.5948,-26.506909],[-68.295542,-26.89934],[-69.001235,-27.521214],[-69.65613,-28.459141],[-70.01355,-29.367923],[-69.919008,-30.336339],[-70.535069,-31.36501],[-70.074399,-33.091211],[-69.814777,-33.273886],[-69.817309,-34.193571],[-70.388049,-35.169688],[-70.364769,-36.005089],[-71.121881,-36.658124],[-71.118625,-37.576827],[-70.814664,-38.552995],[-71.41517,-38.916022],[-71.680761,-39.808164],[-71.915734,-40.832339],[-71.746804,-42.051386],[-72.148898,-42.254888],[-71.915424,-43.408565],[-71.464056,-44.207172],[-71.793623,-44.207172],[-71.329801,-44.407522],[-71.222779,-44.784243],[-71.659316,-44.973689],[-71.552009,-45.560733],[-71.917258,-46.884838],[-72.447355,-47.738533],[-72.331161,-48.244238],[-72.648247,-48.878618],[-73.415436,-49.318436],[-73.328051,-50.378785],[-72.975747,-50.74145],[-72.309974,-50.67701],[-72.329404,-51.425956],[-71.914804,-52.009022],[-69.498362,-52.142761],[-68.571545,-52.299444],[-69.461284,-52.291951],[-69.94278,-52.537931],[-70.845102,-52.899201],[-71.006332,-53.833252],[-71.429795,-53.856455],[-72.557943,-53.53141],[-73.702757,-52.835069],[-73.702757,-52.83507],[-74.946763,-52.262754],[-75.260026,-51.629355],[-74.976632,-51.043396],[-75.479754,-50.378372],[-75.608015,-48.673773],[-75.18277,-47.711919],[-74.126581,-46.939253],[-75.644395,-46.647643],[-74.692154,-45.763976],[-74.351709,-44.103044],[-73.240356,-44.454961],[-72.717804,-42.383356],[-73.3889,-42.117532],[-73.701336,-43.365776],[-74.331943,-43.224958],[-74.017957,-41.794813],[-73.677099,-39.942213],[-73.217593,-39.258689],[-73.505559,-38.282883],[-73.588061,-37.156285],[-73.166717,-37.12378],[72.553137,-35.50884],[71.861732,-33.909093],[-71.438485,-32.418899],[-71.668721,-30.920645],[-71.370083,-30.095682],[-71.489894,-28.861442],[-70.905124,-27.64038],[70.724954,-25.705924],[-70.403966,-23.628997],[-70.091246,-21.393319],[-70.16442,-19.756468],[-70.372572,-18.347975],[-69.858444,-18.092694],[-69.590424,-17.580012]]]]}},{"type":"Feature","properties":{"name":"","bbox":[12.182337,-13.257227,31.174149,5.256088],"geometry":{"type":"Polygon","coordinates":[[[29.339998,-4.499983],[29.519987,-5.419979],[29.419993,-5.939999],[29.620032,-6.520015],[30.199997,-7.079981],[30.740015,-8.340007],[30.74001,-8.340006],[30.346086,-8.238257],[29.002912,-8.407032],[28.734867,-8.526559],[28.449871,-9.164918],[28.673682,-9.605925],[28.49607,-10.789884],[28.372253,-11.793647],[28.642417,-11.971569],[29.341548,-12.360744],[29.616001,-12.178895],[29.699614

, -13.257227], [28.934286, -13.248958], [28.523562, -12.698604], [28.155109, -12.272481], [27.388799, -12.132747], [27.16442, -11.608748], [26.553088, -11.92444], [25.75231, -11.784965], [25.418118, -11.330936], [24.78317, -11.238694], [24.314516, -11.262826], [24.257155, -10.951993], [23.912215, -10.926826], [23.456791, -10.867863], [22.837345, -11.017622], [22.402798, -10.993075], [22.155268, -11.084801], [22.208753, -9.894796], [21.875182, -9.523708], [21.801801, -8.908707], [21.949131, -8.305901], [21.746456, -7.920085], [21.728111, -7.290872], [20.514748, -7.299606], [20.601823, -6.939318], [20.091622, -6.94309], [20.037723, -7.116361], [19.471502, -7.155429], [19.166613, -7.703184], [19.016752, -7.988246], [18.464176, -7.847014], [18.134222, -7.987678], [17.47297, -8.068551], [17.089996, -7.545689], [16.860191, -7.222298], [16.57318, -6.622645], [16.326528, -5.87747], [13.375597, -5.864241], [13.024869, -5.984389], [12.735171, -5.965682], [12.322432, -6.100092], [12.182337, -5.789931], [12.436688, -5.684304], [12.468004, -5.248362], [12.631612, -4.991271], [12.995517, -4.781103], [13.25824, -4.882957], [13.600235, -4.500138], [14.144956, -4.510009], [14.209035, -4.793092], [14.582604, -4.970239], [15.170992, -4.343507], [15.75354, -3.855165], [16.00629, -3.535133], [15.972803, -2.712392], [16.407092, -1.740927], [16.865307, -1.225816], [17.523716, -0.74383], [17.638645, -0.424832], [17.663553, -0.058084], [17.82654, 0.288923], [17.774192, 0.855659], [17.898835, 1.741832], [18.094276, 2.365722], [18.393792, 2.900443], [18.453065, 3.504386], [18.542982, 4.201785], [18.932312, 4.709506], [19.467784, 5.031528], [20.290679, 4.691678], [20.927591, 4.322786], [21.659123, 4.224342], [22.405124, 4.02916], [22.704124, 4.633051], [22.84148, 4.710126], [23.297214, 4.609693], [24.410531, 5.108784], [24.805029,

4.897247], [25.128833, 4.927245], [25.278798, 5.170408], [25.650455, 5.256088], [26.402761, 5.150875], [27.044065, 5.127853], [27.374226, 5.233944], [27.979977, 4.408413], [28.428994, 4.287155], [28.696678, 4.455077], [29.159078, 4.389267], [29.715995, 4.600805], [29.9535, 4.173699], [30.833852, 3.509172], [30.83386, 3.509166], [30.773347, 2.339883], [31.174149, 2.204465], [30.85267, 1.849396], [30.468508, 1.583805], [30.086154, 1.062313], [29.875779, 0.59738], [29.819503, -0.20531], [29.587838, -0.587406], [29.579466, -1.341313], [29.291887, -1.620056], [29.254835, -2.21511], [29.117479, -2.292211], [29.024926, -2.839258], [29.276384, -3.293907], [29.339998, -4.499983]]]], {"type": "Feature", "properties": {"name": ""}, "bbox": [40.98105, -1.68325, 51.13387, 12.02464], "geometry": {"type": "Polygon", "coordinates": [[[41.58513, -1.68325], [40.993, -0.85829], [40.98105, 2.78452], [41.855083, 3.918912], [42.12861, 4.23413], [42.76967, 4.25259], [43.66087, 4.95755], [44.9636, 5.00162], [47.78942, 8.003], [48.486736, 8.837626], [48.93813, 9.451749], [48.938233, 9.9735], [48.938491, 10.982327], [48.942005, 11.394266], [48.948205, 11.410617], [48.948205, 11.410617], [49.26776, 11.43033], [49.72862, 11.5789], [50.25878, 11.67957], [50.573202, 12.0219], [51.1112, 12.02464], [51.13387, 11.74815], [51.04153, 11.16651], [51.04531, 10.6409], [50.83418, 10.27972], [50.55239, 9.19874], [50.07092, 8.08173], [49.4527, 6.80466], [48.59455, 5.33911], [47.74079, 4.2194], [46.56476, 2.85529], [45.56399, 2.04576], [44.06815, 1.05283], [43.13597, 0.2922], [42.04157, -0.91916], [41.81095, -1.44647], [41.58513, -1.68325]]]]}, {"type": "Feature", "properties": {"name": ""}, "bbox": [33.893569, -4.67677, 41.855083, 5.506], "geometry": {"type": "Polygon", "coordinates": [[[39.20222, -4.67677], [37.7669, -3.67712], [37.69869, -3.09699], [37.07262, -1.05982], [33.903711, -0.95], [33.893569, 0.109814], [34.18, 0.515], [34.6721, 1.17694], [35.03599, 1.90584], [34.59607, 3.05374], [34.47913, 3.5556], [34.005, 4.249885], [34.620196, 4.847123], [35.298007, 5.506], [35.817448, 5.338232], [35.817448, 4.776966], [36.159079, 4.447864], [36.855093, 4.447864], [36.120915, 3.598605], [38.43697, 3.58851], [38.67114, 3.61607], [38.89251, 3.50074], [39.559384, 3.42206], [39.85494, 3.83879], [40.76848, 4.25702], [41.1718, 3.91909], [41.855083, 3.918912], [40.98105, 2.78452], [40.993, -0.85829], [41.58513, -1.68325], [40.88477, -2.08255], [40.63785, -2.49979], [40.26304, -2.57309], [40.12119, -3.27768], [39.80006, -3.68116], [39.60489, -4.34653], [39.20222, -4.67677]]]]}, {"type": "Feature", "properties": {"name": ""}, "bbox": [21.93681, 8.229188, 38.41009, 22], "geometry": {"type": "Polygon", "coordinates": [[[24.567369, 8.229188], [23.805813, 8.666319], [23.459013, 8.954286], [23.394779, 9.265068], [23.55725, 9.681218], [23.554304, 10.089255], [22.977544, 10.714463], [22.864165, 11.142395], [22.87622, 11.38461], [22.50869, 11.67936], [22.49762, 12.26024], [22.28801, 12.64605], [21.93681, 12.58818], [22.03759, 12.95546], [22.29658, 13.37232], [22.18329, 13.78648], [22.51202, 14.09318], [22.30351, 14.32682], [22.56795, 14.94429], [23.02459, 15.68072], [23.88689, 15.61084], [23.83766, 19.58047], [23.8520, 19.58047], [25.22, 19.022], [25.22, 19.022], [22.56795, 14.94429], [22.86623, 22], [37.18872, 21.01885], [36.96941, 20.83744], [37.1147, 19.80796], [37.48179, 18.61409], [37.86276, 18.36786], [38.41009, 17.998307], [37.904, 17.42754], [37.16747, 17.26314], [36.85253, 16.95655], [36.75389, 16.29186], [36.32322, 14.82249], [36.42951, 14.42211], [36.27022, 13.56333], [35.86363, 12.57828], [35.26049, 12.08286], [34.83163, 11.31896], [34.73115, 10.91017], [34.25745, 10.63009], [33.96162, 9.58358], [33.97498, 8.68456], [33.963393, 9.464285], [33.824963, 9.484061], [33.842131, 9.981915], [33.721959, 10.325262], [33.206938, 10.720112], [33.086766, 11.441141], [33.206938, 12.179338], [32.743419, 12.248008], [32.67475, 12.024832], [32.073892, 11.97333], [32.314235, 11.681484], [32.400072, 11.080626], [31.850716, 10.531271], [31.352862, 9.810241], [30.837841, 9.707237], [29.996639, 10.290927], [29.618957, 10.084919], [29.515953, 9.793074], [29.000932, 9.604232], [28.966597, 9.398224], [27.97089, 9.398224], [27.833551, 9.604232], [27.112521, 9.638567], [26.752006, 9.466893], [26.477328, 9.55273], [25.962307, 10.136421], [25.790633, 10.411099], [25.069604, 10.27376], [24.794926, 9.810241], [24.537415, 9.917538], [24.194068, 8.728696], [23.88698, 8.61973], [24.567369, 8.229188]]]]}, {"type": "Feature", "properties": {"name": ""}, "bbox": [13.540394, 7.421925, 23.88689, 23.40972], "geometry": {"type": "Polygon", "coordinates": [[[23.83766, 19.58047], [23.88689, 15.61084], [23.02459, 15.68072], [22.56795, 14.94429], [22.30351, 14.32682], [22.51202, 14.09318], [22.18329, 13.78648], [22.29658, 13.37232], [22.03759, 12.95546], [21.93681, 12.58818], [22.28801, 12.64605], [22.49762, 12.26024], [22.50869, 11.67936], [22.87622, 11.38461], [22.864165, 11.142395], [22.231129, 10.971889], [21.723822, 10.567056], [21.000868, 9.475985], [20.059685, 9.012706], [19.094008, 9.074847], [18.81201, 9.982915], [18.911022, 8.630895], [18.389555, 8.281304], [17.96493, 7.890914], [16.705988, 7.508328], [16.456185, 7.734774], [16.290562, 7.754307], [16.10623, 7.497088], [15.27946, 7.421925], [15.436092, 7.692812], [15.120866, 8.38215], [14.979996, 8.796104], [14.544467, 8.965861], [13.954218, 9.549495], [14.171466, 10.021378], [14.627201, 9.920919], [14.909354, 9.992129], [15.467873, 9.982337], [14.923565, 10.891325], [14.960152, 11.555574], [14.89336, 12.21905], [14.495787, 12.859396], [14.595781, 13.330427], [13.954477, 13.353449], [13.95669, 13.996691], [13.540394, 14.367134], [13.97217, 15.68437], [15.247731, 16.627306], [15.300441, 17.92795], [15.685741, 19.95718], [15.903247, 20.387619], [15.487148, 20.730415], [15.47106, 21.04845], [15.096888, 21.308519], [14.8513, 22.86295], [15.86085, 23.40972], [19.84926, 21.49509], [23.83766, 19.58047]]]]}, {"type": "Feature", "properties": {"name": ""}, "bbox": [-74.458034, 18.030993, -71.624873, 19.915684], "geometry": {"type": "Polygon", "coordinates": [[[71.712361, 19.714456], [71.624873, 19.169838], [71.701303, 18.785417], [71.945112, 18.6169], [71.677738, 18.31666], [71.708305, 18.044997], [71.732746, 18.214961], [72.84441, 18.145611], [73.454555, 18.217906], [73.922433, 18.030993], [74.458034, 18.34255], [74.369925, 18.664908], [73.449542, 18.526053], [72.694937, 18.445799], [72.334882, 18.668422], [72.79165, 19.101625], [72.784105, 19.483591], [73.415022, 19.639551], [73.189791, 19.915684], [72.579673, 19.871501], [71.712361, 19.714456]]]]}, {"type": "Feature", "properties": {"name": ""}, "bbox": [-71.945112, 17.598564, -68.317943, 19.884911], "geometry": {"type": "Polygon", "coordinates": [[[71.708305, 18.044997], [71.687738, 18.31666], [71.945112, 18.6169], [71.701303, 18.785417], [71.624873, 19.169838], [71.712361, 19.714456], [71.587304, 19.884911], [70.806706, 19.880286], [70.214365, 19.622885], [69.950815, 19.648], [69.76925, 19.293267], [69.222126, 19.313214], [69.254346, 19.015196], [68.809412, 18.979074], [68.317943, 18.612198], [68.689316, 18.205142], [69.16494, 18.422648], [69.623988, 18.380713], [69.952934, 18.428307], [70.133233, 18.245915], [70.517137, 18.184291], [70.669298, 18.426886], [70.99995, 18.283329], [71.40021, 17.598564], [71.657662, 17.757573], [71.708305, 18.044997]]]]}, {"type": "Feature", "properties": {"name": ""}, "bbox": [-180, 41.151416, 180, 81.2504], "geometry": {"type": "MultiPolygon", "coordinates": [[[178.7253, 71.0988], [180, 71.515714], [180, 70.832199], [178.903425, 70.78114], [178.7253, 71.0988]]], [[49.10116, 46.39933], [48.64541, 45.80629], [47.67591, 45.64149], [46.68201, 44.6092], [47.59094, 43.66016], [47.49252, 42.98658], [48.58437, 41.80888], [48.5843, 41.808869], [47.987283, 41.405819], [47.815666, 41.151416], [47.373315, 41.219732], [46.686071, 41.827137], [46.404951, 41.860675], [45.7764, 42.09244], [45.470279, 42.502781], [44.537623, 42.711993], [43.93121, 42.55496], [43.75599, 42.74083], [42.3944, 43.2203], [40.92219, 43.38215], [40.076965, 43.553104], [39.955099, 43.434998], [38.68, 44.28], [37.53912, 44.65721], [36.67546, 45.24469], [37.40317, 45.40451], [38.23295, 46.24087], [37.67372, 46.63657], [39.14767, 47.04475], [39.1212, 47.26336], [38.223538, 47.10219], [38.255112, 47.5464], [38.77057, 47.82562], [39.738278, 47.898937], [39.89562, 48.23241], [39.67465, 48.78382], [40.080789, 49.30743], [40.06904, 49.60105], [38.594988, 49.926462], [38.010631, 49.915662], [37.39346, 50.383953], [36.626168, 50.225591], [35.356116, 50.577197], [35.337791, 50.77394], [35.022183, 51.207572], [34.224816, 51.255993], [34.141978, 51.566413], [34.391731, 51.768882], [33.7527, 52.335075], [32.715761, 52.238465], [32.412058, 52.288699], [32.15944, 52.06125], [31.785992, 52.101678], [31.78597, 52.10168], [31.540018, 52.742052], [31.305201, 53.073996], [31.49764, 53.16743], [32.304519, 53.132726], [32.693643, 53.351421], [32.405599, 53.618045], [31.731273, 53.794029], [31.791424, 53.974639], [31.384472, 54.157056], [30.757534, 54.811771], [30.971836, 55

.081548], [30.873909, 55.550976], [29.896294, 55.789463], [29.371572, 55.670091], [29.229513, 55.918344], [28.176709, 56.16913], [27.855282, 56.759326], [27.770016, 57.244258], [27.288185, 57.474528], [27.716686, 57.791899], [27.42015, 58.72457], [28.131699, 59.300825], [27.98112, 59.47537], [27.981127, 59.475373], [29.1177, 60.02805], [28.070002, 60.503519], [28.07, 60.50352], [30.211107, 61.780028], [31.139991, 62.357693], [31.516092, 62.867687], [30.035872, 63.552814], [30.444685, 64.204453], [29.54443, 64.948672], [30.21765, 65.80598], [29.054589, 66.944286], [29.977426, 67.698297], [28.445944, 68.364613], [28.59193, 69.064777], [29.39955, 69.15692], [31.101042, 69.558101], [31.10108, 69.55811], [32.13272, 69.90595], [33.77547, 69.30142], [36.51396, 69.06342], [40.29234, 67.9324], [41.05987, 67.45713], [41.12595, 66.79158], [40.01583, 66.26618], [38.38295, 65.99953], [33.91871, 66.75961], [33.18444, 66.63253], [34.81477, 65.90015], [34.878574, 65.436213], [34.94391, 64.41437], [36.23129, 64.10945], [37.01273, 63.84983], [37.14197, 64.33471], [36.539579, 64.76446], [37.17604, 65.14322], [39.59345, 64.52079], [40.4356, 64.76446], [39.7626, 65.49682], [42.09309, 66.47623], [43.01604, 66.41858], [43.94975, 66.06908], [44.53226, 66.75634], [43.69839, 67.35245], [44.18795, 67.95051], [43.45282, 68.57079], [46.25, 68.25], [46.82134, 67.68997], [45.55517, 67.56652], [45.56202, 67.01005], [46.34915, 66.66767], [47.89416, 66.88455], [48.13876, 67.52238], [50.22766, 67.99867], [53.71743, 68.85738], [54.47171, 68.80815], [53.48582, 68.20131], [54.72628, 68.09702], [55.44268, 68.43866], [57.31702, 68.46628], [58.802, 68.88082], [59.94142, 68.27844], [61.07784, 68.94069], [60.03, 69.52], [60.55, 69.85], [63.504, 69.54739], [64.888115, 69.234835], [68.51216, 68.09233], [69.18068, 68.61563], [68.16444, 69.14436], [68.13522, 69.35649], [66.93008, 69.45461], [67.25976, 69.92873], [66.72492, 70.70889], [66.69466, 71.02897], [68.54006, 71.9345], [69.19636, 72.84336], [69.94, 73.04], [72.58754, 72.77629], [72.79603, 72.22006], [71.84811, 71.40898], [72.47011, 71.09019], [72.79188, 70.39114], [72.5647, 69.02085], [73.66787, 68.4079], [73.2387, 67.7404], [71.28, 66.32], [72.42301, 66.17267], [72.82077, 66.53267], [73.92099, 66.78946], [74.18651, 67.28429], [75.052, 67.76047], [74.46926, 68.32899], [74.93584, 68.98918], [73.84236, 69.07146], [73.60187, 69.62763], [74.3998, 70.63175], [73.1011, 71.44717], [74.89082, 72.12119], [74.65926, 72.83227], [75.15801, 72.85497], [75.68351, 72.30056], [75.28898, 71.33556], [76.35911, 71.15287], [75.90313, 71.87401], [77.57665, 72.26717], [79.65202, 72.32011], [81.5, 71.75], [80.61071, 72.58285], [80.51109, 73.6482], [82.25, 73.85], [84.65526, 73.80591], [86.8223, 73.93688], [86.00956, 74.45967], [87.1668, 75.11643], [88.31571, 75.14393], [90.26, 75.64], [92.90058, 75.77333], [93.23421, 76.0472], [95.86, 76.14], [96.67821, 75.91548], [98.92254, 76.44689], [100.75967, 76.43028], [101.03532, 76.86189], [101.99084, 77.28754], [104.3516, 77.69792], [106.06664, 77.37389], [104.705, 77.1274], [106.97013, 76.97419], [107.24, 76.48], [108.1538, 76.72335], [111.07726, 76.71], [113.33151, 76.22244], [114.13417, 75.84764], [113.88539, 75.32779], [112.77918, 75.03186], [110.1525, 74.47673], [109.4, 74.18], [110.64, 74.04], [112.11919, 73.78774], [113.01954, 73.97693], [113.52958, 73.33505], [113.96881, 73.59488], [115.56782, 73.75285], [118.77633, 73.58772], [119.02, 73.12], [123.20066, 72.97122], [123.25777, 73.73503], [125.38, 73.56], [126.97644, 73.56549], [128.59126, 73.03871], [129.05157, 72.39872], [128.46, 71.98], [129.71599, 71.19304], [131.28858, 70.78699], [132.2535, 71.8363], [133.85766, 71.38642], [135.56193, 71.65525], [137.49755, 71.34763], [138.23409, 71.62803], [139.86983, 71.48783], [139.14791, 72.41619], [140.46817, 72.84941], [149.5, 72.2], [150.35118, 71.60643], [152.9689, 70.84222], [157.00688, 71.03141], [158.99779, 70.86672], [159.83031, 70.45324], [159.70866, 69.72198], [160.94053, 69.43728], [162.27907, 69.64204], [164.05248, 69.66823], [165.94037, 69.47199], [167.83567, 69.58269], [169.57763, 68.6938], [170.81688, 69.01363], [170.0082, 69.65276], [170.45345, 70.09703], [173.64391, 69.81743], [175.72403, 69.87725], [178.6, 69.4], [180.68, 69.63636], [180.64, 69.97909], [179.99281, 64.97433], [178.7072, 64.53493], [177.41128, 64.60821], [178.313, 64.07593], [178.90825, 63.25197], [179.37034, 62.98262], [179.48636, 62.56894], [179.22825, 62.3041], [177.3643, 62.5219], [174.56929, 61.76915], [173.68013, 61.65261], [172.15, 60.95], [170.6985, 60.33618], [170.33085, 59.88177], [168.90046, 60.57355], [166.29498, 59.78855], [165.84, 60.16], [164.87674, 59.7316], [163.53929, 59.86871], [163.21711, 59.21101], [162.01733, 58.24328], [162.05297, 57.83912], [163.19191, 57.61503], [163.05794, 56.15924], [162.12958, 56.12219], [161.70146, 55.28568], [162.11749, 54.85514], [160.368, 77.54, 34433], [160.02173, 53.20257], [158.53094, 52.95868], [158.23118, 51.94269], [156.78979, 51.01105], [156.42, 51.7], [155.9918, 2, 53.15895], [155.43366, 55.38103], [155.91442, 56.76792], [156.75815, 57.3647], [156.81035, 57.83204], [158.36433, 58.05575], [160.15064, 59.31477], [161.87204, 60.343], [163.66969, 61.1409], [164.47355, 62.55061], [163.25842, 62.46627], [162.65791, 61.6425], [160.12148, 60.54423], [159.30232, 61.77396], [156.72068, 61.43442], [154.21806, 59.75818], [155.04375, 59.14495], [152.81185, 58.883, 85], [151.26573, 58.78089], [151.33815, 59.50396], [149.78371, 59.65573], [148.54481, 59.16448], [145.48722, 59.33637], [142.19782, 59.03998], [138.95848, 57.08805], [135.12619, 54.72959], [136.70171, 54.60355], [137.19342, 53.97732], [138.1647, 53.75501], [138.80463, 54.25455], [139.90151, 54.18968], [141.34531, 53.08957], [141.37923, 52.23877], [140.59742, 51.23967], [140.51308, 50.04553], [140.06193, 48.44671], [138.55472, 46.99965], [138.21971, 46.30795], [136.86232, 45.1435], [135.51535, 43.989], [134.86939, 43.39821], [133.53687, 42.81147], [132.90627, 42.79849], [132.27807, 43.28456], [130.93587, 42.55274], [130.780005, 42.22001], [130.780004, 42.220008], [130.78, 42.22], [130.779992, 42.22001], [130.64, 42.395], [130.64, 42.395024], [130.633866, 42.903015], [131.144688, 42.92999], [131.28855, 44.11152], [131.02519, 44.96796], [131.883454, 45.321162], [133.09712, 45.14409], [133.769644, 46.116927], [134.11235, 47.21248], [134.50081, 47.57845], [135.026311, 48.47823], [133.373596, 48.183442], [132.50669, 47.78896], [130.98726, 47.79013], [130.582293, 48.729687], [129.397818, 49.4406], [127.6574, 49.76027], [127.287456, 50.739797], [126.939157, 51.353894], [126.564399, 51.784255], [125.946349, 52.792799], [125.068211, 53.161045], [123.57147, 53.4588], [122.245748, 53.431726], [121.00308, 53.251401], [120.177089, 52.753886], [120.725789, 52.516226], [120.7382, 51.96411], [120.18208, 51.64355], [119.27939, 50.58292], [119.288461, 50.142883], [117.879244, 49.510983], [116.678801, 49.888531], [115.485695, 49.805177], [114.96211, 50.140247], [114.362456, 50.248303], [112.89774, 49.543565], [111.581231, 49.377968], [110.662011, 49.130128], [109.402449, 49.292961], [108.475167, 49.282548], [107.868176, 49.793705], [106.888804, 50.274296], [105.886591, 50.406019], [104.62158, 50.27532], [103.676545, 50.089966], [102.25589, 50.51056], [102.06521, 51.25991], [100.88948, 51.516856], [99.981732, 51.634006], [98.861491, 52.047366], [97.82574, 51.010995], [98.231762, 50.422401], [97.25976, 49.72605], [95.81402, 49.97746], [94.815949, 50.013433], [94.147566, 50.480537], [93.10421, 50.49529], [92.234712, 50.802171], [90.713667, 50.331812], [88.805567, 49.470521], [87.751264, 49.297198], [87.35997, 49.214981], [86.829357, 49.826675], [85.54127, 49.692859], [85.11556, 50.117303], [84.416377, 50.3114], [83.935115, 50.889246], [83.383004, 51.069183], [81.945986, 50.812196], [80.568447, 51.388336], [80.03556, 50.864751], [77.800916, 53.404415], [76.525179, 54.177003], [76.8911, 54.490524], [74.38482, 53.54685], [73.425679, 53.48981], [73.508516, 54.035617], [72.22415, 54.376655], [71.180131, 54.133285], [70.865267, 55.169734], [69.068167, 55.38525], [68.1691, 54.970392], [65.66687, 54.60125], [65.178534, 54.354228], [61.4366, 54.00625], [60.978066, 53.664993], [61.699986, 52.979996], [60.739993, 52.719986], [60.927269, 52.447548], [59.967534, 51.96042], [61.588003, 51.272659], [61.337424, 50.79907], [59.932807, 50.842194], [59.642282, 50.545442], [58.36332, 51.06364], [56.7779, 51.043355], [55.71694, 50.62171], [54.532878, 51.02624], [52.328724, 51.718652], [50.766648, 51.692762], [48.702382, 50.605128], [48.577841, 49.87476], [47.54948, 50.454698], [46.751596, 49.356006], [47.043672, 49.152039], [46.466446, 48.394152], [47.31524, 47.71585], [48.05725, 47.74377], [48.694734, 47.075628], [48.59325, 46.56104], [49.10116, 46.39933], [93.77766, 81.0246], [95.9408, 95.81.2504], [97.88385, 80.746975], [100.186655, 79.780135], [99.93976, 78.88094], [97.75794, 78.7562], [94.97259, 79.044745], [93.31288, 79.4265], [92.5454, 80.14379], [91.18107, 80.34146], [93.77766, 81.0246], [102.837815, 79.28129], [105.37243, 78.71334], [105.07547, 78.30689], [99.43814, 77.921], [101.2649, 79.23399], [102.08635, 79.34641], [102.837815, 79.28129], [138.831075, 76.13676], [141.471615, 76.09289], [145.086285, 75.562625], [144.3, 74.82], [140.61381, 74.84768], [138.95544, 74.61148], [136.97439, 75.26167], [137.51176, 75.94917], [138.831075, 76.13676], [148.22223, 75.345845], [150.73167, 75.08406], [149.575925, 74.68892], [147.977465, 74.778355], [146.11919, 75.17298], [146.358485, 75.49682], [148.22223, 75.345845], [139.86312, 73.36983], [140.81171, 73.76506], [142.06207, 73.85758], [143.48283, 73.47525], [143.60385, 73.21244], [142.08763, 73.20544], [140.038155, 73.31692], [139.86312, 73.36983], [44.846958, 80.58981], [46.799139, 80.771918], [48.318477, 80.78401], [48.522806, 80.514569], [49.09719, 80.753986], [50.039768, 80.918885], [51.522933, 80.699726], [51.136187, 80.54728], [49.793685, 80.415428], [48.894411, 80.339567], [48.754937, 80.175468], [47.586119, 80.010181], [46.508286, 80.247247], [47.072455, 80.559424], [44.846958, 80.58981], [32.71099, 54.327537], [20.892245, 34.312525], [19.66064, 54.426084], [19.888481, 54.86616], [21.268449, 55.190482], [22.315724, 55.061529], [22.757764, 54.856574], [22.651052, 54.582741], [22.731099, 54.327537], [53.50829, 73.749814], [55.902459, 74.627486], [55.631933, 75.081412], [57.868644, 75.60939], [61.170044, 76.251883], [64.498368, 76.439055], [66.210977, 76.809782], [68.15706, 76.939697], [68.852211, 76.544811], [68.180573, 76.233642], [64.637326, 75.737755], [61.583508, 75.260885], [58.477082, 74.309056], [56.986786, 73.333044], [55.419336, 72.371268], [55.622838, 71.540595], [57.535693, 70.720464], [56.944979, 70.632743], [53.677375, 70.762658], [53.412017, 71.206662], [51.601895, 71.474759], [51.455754, 72.014881], [52.478275, 72.229442], [52.444169, 72.774731], [54.427614, 73.627548], [53.50829, 73.749814], [142.914616, 53.704578], [143.260848, 52.74076], [143.235268, 51.75666], [143.

05, -30.545732], [28.291069, -30.226217], [28.8484, -30.070051], [29.018415, -29.743766], [29.325166, -29.257387], [28.978263, -28.955597]]], {"type": "Feature", "properties": {"name": ""}, "bbox": [26.999262, -30.645106, 29.325166, -28.647502], "geometry": {"type": "Polygon", "coordinates": [[[28.978263, -28.955597], [29.325166, -29.257387], [29.018415, -29.743766], [28.8484, -30.070051], [28.291069, -30.226217], [28.107205, -30.545732], [27.749397, -30.645106], [26.999262, -29.875954], [27.532511, -29.242711], [28.074338, -28.851469], [28.5417, -28.647502], [28.978263, -28.955597]]]]], {"type": "Feature", "properties": {"name": ""}, "bbox": [-117.12776, 14.538829, -86.811982, 32.72083], "geometry": {"type": "Polygon", "coordinates": [[[117.12776, 32.53534], [-115.99135, 32.61239], [-114.72139, 32.72083], [-114.815, 32.52528], [-113.30498, 32.03914], [-111.02361, 31.33472], [-109.035, 31.34194], [-108.24194, 31.34222], [-108.24, 31.754854], [-106.50759, 31.75452], [-106.1429, 31.39995], [-105.63159, 31.08383], [-105.03737, 30.64402], [-104.70575, 30.12173], [-104.45697, 29.57196], [-103.94, 29.27], [-103.11, 28.97], [-102.48, 29.76], [-101.6624, 29.7793], [-100.9576, 29.38071], [-100.45584, 28.69612], [-100.11, 28.11], [-99.52, 27.54], [-99.3, 26.84], [-99.02, 26.37], [-98.24, 26.06], [-97.53, 25.84], [-97.140008, 25.869997], [-97.528072, 24.992144], [-97.702946, 24.272343], [-97.776042, 22.93258], [-97.872367, 22.444212], [-97.699044, 21.898689], [-97.38896, 21.411019], [-97.189333, 20.635433], [-96.525576, 19.890931], [-96.292127, 19.320371], [-95.900885, 18.828024], [-94.839063, 18.562717], [-94.42573, 18.144371], [-93.548651, 18.423837], [-92.786114, 18.524839], [-92.037348, 18.704569], [-91.407903, 18.876083], [-90.77187, 19.28412], [-90.53359, 19.867418], [-90.451476, 20.707522], [-90.278618, 20.999855], [-89.601321, 21.261726], [-88.543866, 21.493675], [-87.658417, 21.458846], [-87.05189, 21.543543], [-86.811982, 21.331515], [-86.845908, 20.849865], [-87.383291, 20.255405], [-87.621054, 19.646553], [-87.43675, 19.472403], [-87.58656, 19.04013], [-87.837191, 18.259816], [-88.090664, 18.516648], [-88.300031, 18.499982], [-88.490123, 18.486831], [-88.848344, 17.883198], [-89.029857, 18.001511], [-89.150909, 17.955468], [-89.14308, 17.808319], [-90.067934, 17.819326], [-91.00152, 17.817595], [-91.002269, 17.254658], [-91.453921, 17.252177], [-91.08167, 16.918477], [-90.711822, 16.687483], [-90.600847, 16.470778], [-90.438867, 16.41011], [-90.464473, 16.069562], [-91.74796, 16.066565], [-92.229249, 15.251447], [-92.087216, 15.064585], [-92.20323, 14.830103], [-92.22775, 14.538829], [-93.359464, 15.61543], [-93.875169, 15.940164], [-94.691656, 16.200975], [-95.250227, 16.128318], [-96.053382, 15.752088], [-96.557434, 15.653515], [-97.263592, 15.917065], [-98.01303, 16.107312], [-98.947676, 16.566043], [-99.697397, 16.706164], [-100.829499, 17.171071], [-101.666089, 17.649026], [-101.918528, 17.91609], [-102.478132, 17.975751], [-103.50099, 18.292295], [-103.917527, 18.748572], [-104.99201, 19.316134], [-105.439308, 19.946767], [-105.731396, 20.434102], [-105.397773, 20.531719], [-105.500661, 20.816895], [-105.270752, 21.076285], [-105.265817, 21.422104], [-105.603161, 21.871146], [-105.693414, 22.26908], [-106.028716, 22.773752], [-106.90998, 23.767774], [-107.915449, 24.548915], [-108.401905, 25.172314], [-109.260199, 25.580609], [-109.444089, 25.824884], [-109.291644, 26.442934], [-109.801458, 26.676176], [-110.391732, 27.162115], [-110.641019, 27.859876], [-111.178919, 27.941241], [-111.759607, 28.467953], [-112.228235, 28.95409], [-112.271824, 29.266844], [-112.809594, 30.02114], [-113.163811, 30.786881], [-113.148669, 31.170966], [-113.871881, 31.567608], [-114.205737, 31.524045], [-114.776451, 31.799532], [-114.9367, 31.393485], [-114.771232, 30.913617], [-114.673899, 30.162681], [-114.330974, 29.750432], [-113.588875, 29.061611], [-113.424053, 28.26174], [-113.271969, 28.754783], [-113.140039, 28.411289], [-112.962298, 28.42519], [-112.761587, 27.780217], [-112.457911, 27.525814], [-112.244952, 27.17127], [-111.616489, 26.662817], [-111.284675, 25.73259], [-110.987819, 25.294606], [-110.710007, 24.826004], [-110.655049, 24.298595], [-110.172856, 24.265548], [-109.771847, 23.811183], [-109.409104, 23.364672], [-109.433392, 23.185588], [-109.854219, 22.818272], [-110.031392, 22.823078], [-110.295071, 23.430973], [-110.949501, 24.000964], [-111.670568, 24.484423], [-112.182036, 24.738413], [-112.148989, 25.470125], [-112.300711, 26.012004], [-112.77297, 26.32196], [-113.464671, 26.768186], [-113.59673, 26.63946], [-113.848937, 26.900064], [-114.465747, 27.14209], [-115.055142, 27.722727], [-114.982253, 27.7982], [-114.570366, 27.741485], [-114.199329, 28.115003], [-114.162018, 28.566112], [-114.931842, 29.279479], [-115.518654, 29.556362], [-115.887365, 30.180794], [-116.25835, 30.836464], [-116.721526, 31.635744], [-117.12776, 32.53534]]]]], {"type": "Feature", "properties": {"name": ""}, "bbox": [-58.427074, -34.952647, -53.209589, -30.109686], "geometry": {"type": "Polygon", "coordinates": [[[57.625133, -30.216295], [-56.976026, -30.109686], [-55.973245, -30.883076], [-55.60151, -30.853879], [-54.572452, -31.494511], [-53.787952, -32.047243], [-53.209589, -32.727666], [-53.650544, -33.202004], [-53.373662, -33.768378], [-53.806426, -34.396815], [-54.935866, -34.952647], [-55.67409, -34.752659], [-56.215297, -34.859836], [-57.139685, -34.430456], [-57.817861, -34.462547], [-58.427074, -33.909454], [-58.349611, -33.263189], [-58.132648, -33.040567], [-58.14244, -32.044504], [-57.874937, -31.016556], [-57.625133, -30.216295]]]]], {"type": "Feature", "properties": {"name": ""}, "bbox": [-73.987235, -33.768378, -34.729993, 5.244486], "geometry": {"type": "Polygon", "coordinates": [[[53.373662, -33.768378], [-53.650544, -33.202004], [-53.209589, -32.727666], [-53.787952, -32.047243], [-54.572452, -31.494511], [-55.60151, -30.853879], [-55.973245, -30.883076], [-56.976026, -30.109686], [-57.625133, -30.216295], [-56.2909, -28.852761], [-55.162286, -27.881915], [-54.490725, -27.474757], [-53.648735, -26.923473], [-53.628349, -26.124865], [-54.13005, -25.547639], [-54.625291, -25.739255], [-54.428946, -25.162185], [-54.293476, -24.5708], [-54.29296, -24.021014], [-54.652834, -23.839578], [-55.027902, -24.001274], [-55.400747, -23.956935], [-55.517639, -23.571998], [-55.610683, -22.655619], [-55.797958, -22.35693], [-56.473317, -22.0863], [-56.88151, -22.282154], [-57.937156, -22.090176], [-57.870674, -20.732688], [-58.166392, -20.176701], [-57.853802, -19.969995], [-57.949997, -19.400004], [-57.676009, -18.96184], [-57.498371, -18.174188], [-57.734558, -17.552468], [-58.280804, -17.27171], [-58.388058, -16.877109], [-58.24122, -16.299573], [-60.15839, -16.258284], [-60.542966, -15.09391], [-60.251149, -15.077219], [-60.264326, -14.645979], [-60.459198, -14.354007], [-60.503304, -13.775955], [-61.084121, -13.479384], [-61.713204, -13.489202], [-62.127081, -13.198781], [-62.80306, -13.000653], [-63.196499, -12.627033], [-64.316353, -12.461978], [-65.402281, -11.56627], [-65.321899, -10.895877], [-65.444837, -10.511451], [-65.338435, -9.761988], [-66.646908, -9.931331], [-67.173801, -10.306812], [-68.048192, -10.712059], [-68.271254, -11.014521], [-68.786158, -11.03638], [-69.529678, -10.951734], [-70.093752, -11.123972], [-70.548686, -11.009147], [-70.481894, -9.490118], [-71.302412, -10.079436], [-72.184891, -10.053598], [-72.563033, -9.520194], [-73.226713, -9.462213], [-73.015383, -9.032833], [-73.571059, -8.424447], [-73.987235, -7.52383], [-73.723401, -7.340999], [-73.724487, -6.918595], [-73.120027, -6.629931], [-73.219711, -6.089189], [-72.964507, -5.741251], [-72.891928, -5.274561], [-71.748406, -4.593983], [-70.928843, -4.401591], [-70.794769, -4.251265], [-69.893635, -4.298187], [-69.444102, -1.556287], [-69.420486, -1.122619], [-69.577065, -0.549992], [-70.02056, -0.185156], [-70.015566, -0.541414], [-69.452396, -0.706159], [-69.252434, -0.602651], [-69.218638, -0.985677], [-69.804597, -1.089081], [-69.816973, -1.714805], [-67.868565, -1.692455], [-67.53781, -2.037163], [-67.259998, -1.719999], [-67.065048, -1.130112], [-66.876326, -1.253361], [-66.325765, -0.724452], [-65.548267, -0.789254], [-65.354713, -1.095282], [-64.611012, -1.328731], [-64.199306, -1.492855], [-64.083085, -1.916369], [-63.368788, -2.2009], [-63.422867, -2.41068], [-64.269999, -2.497006], [-64.408828, -3.126788], [-64.368494, -3.79721], [-64.816064, -4.056445], [-64.628659, -4.148481], [-63.888343, -4.02053], [-63.093198, -3.770571], [-62.804533, -4.006965], [-62.08543, -4.162124], [-60.966893, -4.536468], [-60.601179, -4.918098], [-60.733574, -5.200277], [-60.213683, -5.244486], [-59.980959, -5.014061], [-60.111002, -4.574967], [-59.767406, -4.423503], [-59.53804, -3.958803], [-59.815413, -3.606499], [-59.974525, -2.755233], [-59.718546, -2.24963], [-59.646044, -1.786894], [-59.030862, -1.317698], [-58.540013, -1.268088], [-58.429477, -1.463942], [-58.11345, -1.507195], [-57.660971, -1.682585], [-57.335823, -1.948538], [-56.782704, -1.863711], [-56.539386, -1.899523], [-55.995698, -1.817667], [-55.9056, -2.021996], [-56.073342, -2.20795], [-55.973322, -2.510364], [-55.569755, -2.421506], [-55.097587, -2.523748], [-54.524754, -2.311849], [-54.088063, -2.105557], [-53.778521, -2.376703], [-53.554839, -2.334897], [-53.418465, -2.053389], [-52.939657, -2.124858], [-52.556425, -2.504705], [-52.249338, -3.241094], [-51.657797, -4.156232], [-51.317146, -4.203491], [-51.069771, -3.650398], [-50.508875, -1.901564], [-49.974076, -1.736483], [-49.947101, -1.04619], [-50.699251, -0.222984], [-50.388211, -0.078445], [-48.620567, -0.235489], [-48.584497, -1.237805], [-47.824956, -0.581618], [-46.566584, -0.941028], [-44.905703, -1.55174], [-44.417619, -2.13775], [-44.581589, -2.691308], [-43.418791, -2.38311], [-41.472657, -2.912018], [-39.978665, -2.873054], [-38.500383, -3.700652], [-37.223252, -4.820946], [-36.452937, -5.109404], [-35.597796, -5.149504], [-35.235389, -5.464937], [-34.896003, -6.738193], [-34.729993, -7.333221], [-35.128212, -8.996401], [-35.636967, -9.649282], [-37.046519, -11.040721], [-37.683612, -12.171195], [-38.423877, -13.034121], [-38.673887, -13.057652], [-38.953276, -13.79337], [-38.882298, -15.667054], [-39.161092, -17.208407], [-39.267339, -17.867746], [-39.583521, -18.262296], [-39.760823, -19.599113], [-40.774741, -20.904512], [-40.944756, -21.937317], [-41.754164, -22.370676], [-41.988284, -22.97007], [-43.074704, -22.967693], [-44.647812, -23.351959], [-45.352136, -23.796842], [-46.472093, -24.088969], [-47.648972, -24.885199], [-48.495458, -25.877025], [-48.641095, -26.623698], [-48.474736, -27.175912], [-48.66152, -28.186135], [-48.888457, -28.674115], [-49.587329, -29.224469], [-50.696874, -30.984465], [-51.576226, -31.777698], [-52.256081, -32.24537], [-52.7121, -33.196578], [-53.373662, -33.768378]]]]], {"type": "Feature", "properties": {"name": ""}, "bbox": [-69.590424, -22.87

2919, -57.498371, -9.761988], "geometry": {"type": "Polygon", "coordinates": [[[-69.529678, -10.951734], [-68.786158, -11.03638], [-68.271254, -11.014521], [-68.048192, -10.712059], [-67.173801, -10.306812], [-66.646908, -9.931331], [-65.338435, -9.761988], [-65.444837, -10.511451], [-65.321899, -10.895872], [-65.402281, -11.56627], [-64.316353, -12.461978], [-63.196499, -12.627033], [-62.80306, -13.000653], [-62.127081, -13.198781], [-61.713204, -13.489202], [-61.084121, -13.479384], [-60.503304, -13.775955], [-60.459198, -14.354007], [-60.264326, -14.645979], [-60.251149, -15.077219], [-60.542966, -15.09391], [-60.15839, -16.258284], [-58.24122, -16.299573], [-58.388058, -16.877109], [-58.280804, -17.27171], [-57.734558, -17.552468], [-57.498371, -18.174188], [-57.676009, -18.96184], [-57.949997, -19.400004], [-57.853802, -19.969995], [-58.166392, -20.176701], [-58.183471, -19.868399], [-59.115042, -19.356906], [-60.043565, -19.342747], [-61.786326, -19.633737], [-62.265961, -20.513735], [-62.291179, -21.051635], [-62.685057, -22.249029], [-62.846468, -22.034985], [-63.986838, -21.993644], [-64.377021, -22.798091], [-64.964892, -22.075862], [-66.273339, -21.83231], [-67.106674, -22.735925], [-67.82818, -22.872919], [-68.219913, -21.494347], [-68.757167, -20.372658], [-68.442225, -19.405068], [-68.966818, -18.981683], [-69.100247, -18.260125], [-69.590424, -17.580012], [-68.959635, -16.500698], [-69.389764, -15.660129], [-69.160347, -15.323974], [-69.339535, -14.953195], [-68.948887, -14.453639], [-68.929224, -13.602684], [-68.88008, -12.899729], [-68.66508, -12.5613], [-69.529678, -10.951734]]]], {"type": "Feature", "properties": {"name": ""}, "bbox": [-81.410943, -18.347975, -68.66508, -0.057205], "geometry": {"type": "Polygon", "coordinates": [[[-69.893635, -4.298187], [-70.794769, -4.251265], [-70.928843, -4.401591], [-71.748406, -4.593983], [-72.891928, -5.274561], [-72.964507, -5.741251], [-73.219711, -6.089189], [-73.120027, -6.629931], [-73.724487, -6.918595], [-73.723401, -7.340999], [-73.987235, -7.52383], [-73.571059, -8.424447], [-73.015383, -9.032833], [-73.226713, -9.462213], [-72.563033, -9.520194], [-72.184891, -10.053598], [-71.302412, -10.079436], [-70.481894, -9.490118], [-70.548686, -11.009147], [-70.093752, -11.123972], [-69.529678, -10.951734], [-68.66508, -12.5613], [-68.88008, -12.899729], [-68.929224, -13.602684], [-68.948887, -14.453639], [-69.339535, -14.953195], [-69.160347, -15.323974], [-69.389764, -15.660129], [-68.959635, -16.500698], [-69.590424, -17.580012], [-69.858444, -18.092694], [-70.372572, -18.347975], [-71.37525, -17.773799], [-71.462041, -17.363488], [-73.44453, -16.359363], [-75.237883, -15.265683], [-76.009205, -14.649286], [-76.423469, -13.823187], [-76.259242, -13.535051], [-77.106192, -12.22716], [-78.092153, -10.377712], [-79.036953, -8.386568], [-79.44592, -7.930833], [-79.760578, -7.194341], [-80.537482, -6.541668], [-81.249996, -6.136834], [-80.926347, -5.690557], [-81.410943, -4.736765], [-81.09967, -4.036394], [-80.302561, -3.404856], [-80.184015, -3.821162], [-80.469295, -4.059287], [-80.442242, -4.425724], [-80.028908, -4.346091], [-79.624979, -4.454198], [-79.205289, -4.959129], [-78.639897, -4.547784], [-78.450684, -3.873097], [-77.837905, -3.003021], [-76.635394, -2.608678], [-75.544996, -1.56161], [-75.233723, -0.911417], [-75.373223, -0.152032], [-75.106625, -0.057205], [-74.441601, -0.53082], [-74.122395, -1.002833], [-73.659504, -1.260491], [-73.070392, -2.308954], [-73.325787, -2.434218], [-71.774761, -2.16979], [-71.413646, -2.342802], [-70.813476, -2.256865], [-70.047709, -2.725156], [-70.692682, -3.742872], [-70.394044, -3.766591], [-69.893635, -4.298187]]]], {"type": "Feature", "properties": {"name": ""}, "bbox": [81.87, -66.876326, 12.437303], "geometry": {"type": "Polygon", "coordinates": [[[-66.876326, 1.253361], [-67.065048, 1.130112], [-67.259998, 1.719999], [-67.53781, 2.037163], [-67.868565, 1.692455], [-69.816973, 1.714805], [-69.804597, 1.089081], [-69.218638, 0.985677], [-69.252434, 0.602651], [-69.452396, 0.706159], [-70.015566, 0.541414], [-70.020656, -0.185156], [-69.577065, -0.549992], [-69.420486, -1.122619], [-69.444102, -1.556287], [-69.893635, -4.298187], [-70.394044, -3.766591], [-70.692682, -3.742872], [-70.047709, -2.725156], [-70.813476, -2.256865], [-71.413646, -2.342802], [-71.774761, -2.16979], [-72.325787, -2.434218], [-73.070392, -2.308954], [-73.659504, -1.260491], [-74.122395, -1.002833], [-74.441601, -0.53082], [-75.106625, -0.057205], [-75.373223, -0.152032], [-75.801466, 0.084801], [-76.292314, 0.416047], [-76.57638, 0.256936], [-77.424984, 0.395687], [-77.668613, 0.825893], [-77.855061, 0.809925], [-78.855259, 1.380924], [-78.990935, 1.69137], [-78.617831, 1.766404], [-78.662118, 2.267355], [-78.42761, 2.629556], [-77.931543, 2.696606], [-77.510431, 3.325017], [-77.12769, 3.849636], [-77.496272, 4.087606], [-77.307601, 4.667984], [-77.533221, 5.582812], [-77.731815, 5.845354], [-77.476661, 6.691116], [-77.881571, 7.223771], [-77.753414, 7.709984], [-77.431108, 7.638061], [-77.242566, 7.935278], [-77.474723, 8.524286], [-77.353361, 8.670505], [-76.836674, 8.638749], [-76.086384, 9.336821], [-75.6746, 9.443248], [-75.664704, 9.774003], [-75.480426, 10.61899], [-74.906895, 11.083045], [-74.276753, 11.102036], [-74.197223, 11.310473], [-73.414764, 11.227015], [-72.627835, 11.731972], [-72.238195, 11.95555], [-71.754409, 12.437303], [-71.399822, 12.376041], [-71.137461, 12.112982], [-71.331584, 11.776284], [-71.973922, 11.608672], [-72.227575, 11.108702], [-72.614658, 10.821975], [-72.905286, 10.450344], [-73.027604, 9.73677], [-73.304952, 9.152], [-72.78873, 9.085027], [-72.660495, 8.625288], [-72.439862, 8.405275], [-72.360901, 8.002638], [-72.479679, 7.632506], [-72.444487, 7.423785], [-72.198352, 7.340431], [-71.960176, 6.991615], [-70.674234, 7.087785], [-70.093313, 6.960376], [-69.38948, 6.099861], [-68.985319, 6.206805], [-68.265052, 6.153268], [-67.695087, 6.267318], [-67.34144, 6.095468], [-67.521532, 5.55687], [-67.744697, 5.221129], [-67.823012, 4.503937], [-67.621836, 3.839482], [-67.337564, 3.542342], [-67.303173, 3.318454], [-67.809938, 2.820655], [-67.447092, 2.600281], [-67.181294, 2.250638], [-66.876326, 1.253361]]]], {"type": "Feature", "properties": {"name": ""}, "bbox": [-82.965783, 7.220541, -77.242566, 9.61161], "geometry": {"type": "Polygon", "coordinates": [[[-77.353361, 8.670505], [-77.474723, 8.524286], [-77.242566, 7.935278], [-77.431108, 7.638061], [-77.753414, 7.70984], [-77.881571, 7.223771], [-78.214936, 7.512255], [-78.429161, 8.052041], [-78.182096, 8.319182], [-78.435465, 8.387705], [-78.622121, 8.718124], [-79.120307, 8.996092], [-79.557877, 8.932375], [-79.760578, 8.584515], [-80.164481, 8.33316], [-80.382659, 8.298409], [-80.480689, 8.090308], [-80.00369, 7.547524], [-80.276671, 7.419754], [-80.421158, 7.271572], [-80.886401, 7.220541], [-81.059543, 7.817921], [-81.189716, 7.647906], [-81.519515, 7.70661], [-81.721311, 8.108963], [-82.131441, 8.175393], [-82.390934, 8.292362], [-82.820081, 8.290864], [-82.850958, 8.073823], [-82.965783, 8.225028], [-82.913176, 8.423517], [-82.829771, 8.626295], [-82.868657, 8.807266], [-82.719183, 8.925709], [-82.927155, 9.07433], [-82.932891, 9.476812], [-82.546196, 9.566135], [-82.187123, 9.207449], [-82.207586, 8.995575], [-81.808567, 8.950617], [-81.714154, 9.031955], [-81.439287, 8.786234], [-80.947302, 8.858504], [-80.521901, 9.11072], [-79.9146, 9.12765], [-79.573303, 9.61161], [-79.021192, 9.552931], [-79.05845, 9.454565], [-78.500888, 9.420459], [-78.055928, 9.24773], [-77.729514, 8.946844], [-77.353361, 8.670505]]]], {"type": "Feature", "properties": {"name": ""}, "bbox": [-85.941725, 8.225028, -82.546196, 11.217119], "geometry": {"type": "Polygon", "coordinates": [[[-82.546196, 9.566135], [-82.932891, 9.476812], [-82.927155, 9.07433], [-82.719183, 8.925709], [-82.868657, 8.807266], [-82.829771, 8.626295], [-82.913176, 8.423517], [-82.965783, 8.225028], [-83.508437, 8.446927], [-83.711474, 8.656836], [-83.596313, 8.830443], [-83.632642, 9.051386], [-83.909886, 9.290803], [-84.303402, 9.487354], [-84.647644, 9.615537], [-84.713351, 9.908052], [-84.97566, 10.086723], [-84.911375, 9.795992], [-85.110923, 9.55704], [-85.339488, 9.834542], [-85.660787, 9.93347], [-85.797445, 10.134886], [-85.791709, 10.439337], [-85.659314, 10.754331], [-85.941725, 10.895278], [-85.71254, 11.088445], [-85.561852, 11.217119], [-84.903003, 10.952303], [-84.673069, 11.082657], [-84.355931, 10.999226], [-84.190179, 10.79345], [-83.895054, 10.726839], [-83.655612, 10.938764], [-83.40232, 10.395438], [-83.015677, 9.992982], [-82.546196, 9.566135]]]], {"type": "Feature", "properties": {"name": ""}, "bbox": [-87.668493, 10.726839, -83.147219, 15.016267], "geometry": {"type": "Polygon", "coordinates": [[[-83.655612, 10.938764], [-83.895054, 10.726839], [-84.190179, 10.79345], [-84.355931, 10.999226], [-84.673069, 11.082657], [-84.903003, 10.952303], [-85.561852, 11.217119], [-85.71254, 11.088445], [-86.058488, 11.403439], [-86.52585, 11.806877], [-86.745992, 12.143962], [-87.167516, 12.458258], [-87.668493, 12.90991], [-87.557467, 13.064552], [-87.392386, 12.914018], [-87.316654, 12.984686], [-87.005769, 13.025794], [-86.880557, 13.254204], [-86.733822, 13.263093], [-86.755087, 13.754845], [-86.520708, 13.778487], [-86.312142, 13.771356], [-86.096264, 14.038187], [-85.801295, 13.836055], [-85.698665, 13.960078], [-85.514413, 14.079012], [-85.165365, 14.35437], [-85.148751, 14.560197], [-85.052787, 14.551541], [-84.924501, 14.790493], [-84.820037, 14.819587], [-84.649582, 14.666805], [-84.449336, 14.621614], [-84.228342, 14.748764], [-83.975721, 14.749436], [-83.628585, 14.880074], [-83.489989, 15.016267], [-83.147219, 14.995829], [-83.233234, 14.899866], [-83.284162, 14.676624], [-83.182126, 14.310703], [-83.4125, 13.970078], [-83.519832, 13.567699], [-83.552207, 13.127054], [-83.498515, 12.869292], [-83.473323, 12.419087], [-83.626104, 12.32085], [-83.719613, 11.893124], [-83.650858, 11.629032], [-83.85547, 11.373311], [-83.808936, 11.103044], [-83.655612, 10.938764]]]], {"type": "Feature", "properties": {"name": ""}, "bbox": [-89.353326, 12.984686, -83.147219, 16.005406], "geometry": {"type": "Polygon", "coordinates": [[[-83.147219, 14.995829], [-83.489989, 15.016267], [-83.628585, 14.880074], [-83.975721, 14.749436], [-84.228342, 14.748764], [-84.449336, 14.621614], [-84.649582, 14.666805], [-84.820037, 14.819587], [-84.924501, 14.790493], [-85.052787, 14.551541], [-85.148751, 14.560197], [-85.165365, 14.35437], [-85.514413, 14.079012], [-85.698665, 13.960078], [-85.801295, 13.836055], [-86.096264, 14.038187], [-86.312142, 13.771356], [-86.520708, 13.778487], [-86.755087, 13.754845], [-86.733822, 13.263093], [-86.880557, 13.254204], [-87.005769, 13.025794], [-87.316654, 12.984686], [-87.489409, 13.297535], [-87.793111, 13.38448], [-87.723503, 13.78505]

, [-87.859515, 13.893312], [-88.065343, 13.964626], [-88.503998, 13.845486], [-88.541231, 13.980155], [-88.843073, 14.140507], [-89.058512, 14.340029], [-89.353326, 14.424133], [-89.145535, 14.678019], [-89.22522, 14.874286], [-89.154811, 15.066419], [-88.68068, 15.346247], [-88.225023, 15.727722], [-88.121153, 15.688655], [-87.901813, 15.864458], [-87.61568, 15.878799], [-87.522921, 15.797279], [-87.367762, 15.84694], [-86.903191, 15.756713], [-86.440946, 15.782835], [-86.119234, 15.893449], [-86.001954, 16.005406], [-85.683317, 15.953652], [-85.444004, 15.885749], [-85.182444, 15.909158], [-84.983722, 15.995923], [-84.52698, 15.857224], [-84.368256, 15.835158], [-84.063055, 15.648244], [-83.773977, 15.424072], [-83.410381, 15.270903], [-83.147219, 14.995829]]], {"type": "Feature", "properties": {"name": ""}, "bbox": [-90.095555, 13.149017, -87.723503, 14.424133], "geometry": {"type": "Polygon", "coordinates": [[[-89.353326, 14.424133], [-89.058512, 14.340029], [-88.843073, 14.140507], [-88.541231, 13.980155], [-88.503998, 13.845486], [-88.065343, 13.964626], [-87.859515, 13.893312], [-87.723503, 13.78505], [-87.793111, 13.38448], [-87.904112, 13.149017], [-88.483302, 13.163951], [-88.843228, 13.259734], [-89.256743, 13.458533], [-89.812394, 13.520622], [-90.095555, 13.735338], [-90.064678, 13.88197], [-89.721934, 14.134228], [-89.534219, 14.244816], [-89.587343, 14.362586], [-89.353326, 14.424133]]]]}, {"type": "Feature", "properties": {"name": ""}, "bbox": [-92.229249, 13.735338, -88.225023, 17.819326], "geometry": {"type": "Polygon", "coordinates": [[[-92.22775, 14.538829], [-92.20323, 14.830103], [-92.087216, 15.064585], [-92.229249, 15.251447], [-91.74796, 16.066565], [-90.464473, 16.069562], [-90.438867, 16.41011], [-90.600847, 16.470778], [-90.711822, 16.687483], [-91.08167, 16.918477], [-91.453921, 17.252177], [-91.002269, 17.254658], [-91.00152, 17.817595], [-90.067934, 17.819326], [-89.14308, 17.808319], [-89.150806, 17.015577], [-89.229122, 15.886938], [-88.930613, 15.887273], [-88.604586, 15.70638], [-88.518364, 15.855389], [-88.225023, 15.727722], [-88.68068, 15.346247], [-89.154811, 15.066419], [-89.22522, 14.874286], [-89.145535, 14.678019], [-89.353326, 14.424133], [-89.587343, 14.362586], [-89.534219, 14.244816], [-89.721934, 14.134228], [-90.064678, 13.88197], [-90.095555, 13.735338], [-90.608624, 13.909771], [-91.23241, 13.927832], [-91.689747, 14.126218], [-92.22775, 14.538829]]]]}, {"type": "Feature", "properties": {"name": ""}, "bbox": [-89.229122, 15.886938, -88.106813, 18.499982], "geometry": {"type": "Polygon", "coordinates": [[[-89.14308, 17.808319], [-89.150909, 17.955468], [-89.029857, 18.001511], [-88.848344, 17.883198], [-88.490123, 18.486831], [-88.300031, 18.499982], [-88.296336, 18.353273], [-88.106813, 18.348674], [-88.123479, 18.067675], [-88.285355, 17.644143], [-88.197867, 17.489475], [-88.302641, 17.131694], [-88.239518, 17.036066], [-88.355428, 16.530774], [-88.551825, 16.265467], [-88.732434, 16.236355], [-88.930613, 15.887273], [-89.229122, 15.886938], [-89.150806, 17.015577], [-89.14308, 17.808319]]]]}, {"type": "Feature", "properties": {"name": ""}, "bbox": [-73.304952, 0.724452, -59.758285, 12.162307], "geometry": {"type": "Polygon", "coordinates": [[[-60.733574, 5.200277], [-60.601179, 4.918098], [-60.966893, 4.536468], [-62.08543, 4.162124], [-62.804533, 4.006965], [-63.093198, 3.770571], [-63.888343, 4.02053], [-64.628659, 4.148481], [-64.816064, 4.056445], [-64.368494, 3.79721], [-64.408828, 3.126786], [-64.269999, 2.497006], [-63.422867, 2.411068], [-63.368788, 2.2009], [-64.083085, 1.916369], [-64.199306, 1.492855], [-64.611012, 1.328731], [-65.354713, 1.095282], [-65.548267, 0.789254], [-66.325765, 0.724452], [-67.676326, 1.253361], [-67.181294, 2.250638], [-67.447092, 2.600281], [-67.809938, 2.820655], [-67.303173, 3.318454], [-67.337564, 3.542342], [-67.621836, 3.839482], [-67.823012, 4.503937], [-67.744697, 5.221129], [-67.862153, 5.55687], [-67.34144, 6.095468], [-67.695087, 6.267318], [-68.265052, 6.153268], [-68.985319, 6.206805], [-69.38948, 6.099861], [-70.093313, 6.960376], [-70.674234, 7.087785], [-71.960176, 6.991615], [-72.198352, 7.340431], [-72.444487, 7.423785], [-72.479679, 7.632506], [-72.360901, 8.002638], [-72.439862, 8.405275], [-72.60495, 8.625288], [-72.78873, 9.085027], [-73.304952, 9.152], [-73.027604, 9.73677], [-72.905286, 10.453044], [-72.614658, 10.82197], [-72.227575, 11.108702], [-71.973922, 11.608672], [-71.331584, 11.776284], [-71.360006, 11.539994], [-71.94705, 11.423282], [-71.620868, 10.96946], [-71.633064, 10.446494], [-72.074174, 9.865651], [-71.695644, 9.072263], [-71.264559, 9.137195], [-71.039999, 9.859993], [-71.350084, 10.211935], [-71.400623, 10.968969], [-70.155299, 11.375488], [-70.293843, 11.846828], [-69.943245, 12.162307], [-69.5843, 11.459611], [-68.882999, 11.443385], [-68.233271, 10.885744], [-68.194127, 10.554653], [-67.296249, 10.545868], [-66.227864, 10.648627], [-65.655238, 10.200799], [-64.890452, 10.077215], [-64.329479, 10.389599], [-64.318007, 10.641418], [-63.079322, 10.701724], [-61.880946, 10.715625], [-62.730119, 10.420269], [-62.388512, 9.948204], [-61.588767, 9.873067], [-60.830597, 9.38134], [-60.671252, 8.580174], [-60.150096, 8.602757], [-59.758285, 8.367035], [-60.550588, 7.779603], [-60.637973, 7.415], [-60.29568, 7.043911], [-60.543999, 6.856584], [-61.159336, 6.696077], [-61.139415, 6.234297], [-61.410303, 5.959068], [-60.733574, 5.200277]]]]}, {"type": "Feature", "properties": {"name": ""}, "bbox": [-61.410303, 1.268088, -56.539386, 8.367035], "geometry": {"type": "Polygon", "coordinates": [[[-56.539386, 1.899523], [-56.782704, 1.863711], [-57.335823, 1.948538], [-57.660971, 1.682585], [-58.11345, 1.507195], [-58.429477, 1.463942], [-58.540013, 1.268088], [-59.038062, 1.317698], [-59.646044, 1.786894], [-59.718546, 2.24963], [-59.974525, 2.755233], [-59.815413, 3.606499], [-59.53804, 3.958803], [-59.767406, 4.423503], [-60.111002, 4.574967], [-59.980959, 5.014061], [-60.213683, 5.244486], [-60.733574, 5.200277], [-61.410303, 5.959068], [-61.139415, 6.234297], [-61.159336, 6.696077], [-60.543999, 6.856584], [-60.29568, 7.043911], [-60.637973, 7.415], [-60.550588, 7.779603], [-59.758285, 8.367035], [-59.101684, 7.999202], [-58.482962, 7.347691], [-58.454876, 6.832787], [-58.078103, 6.809094], [-57.542219, 6.321268], [-57.147436, 5.97315], [-57.307246, 5.073567], [-57.914289, 4.812626], [-57.86021, 4.576801], [-58.044694, 4.060864], [-57.601569, 3.334655], [-57.281433, 3.333492], [-57.150098, 2.768927], [-56.539386, 1.899523]]]]}, {"type": "Feature", "properties": {"name": ""}, "bbox": [-58.044694, 1.817667, -53.958045, 6.025291], "geometry": {"type": "Polygon", "coordinates": [[[-54.524754, 2.311849], [-55.07587, 2.523748], [-55.569755, 2.421506], [-55.973322, 2.510364], [-56.073342, 2.220795], [-55.9056, 2.021996], [-55.995698, 1.817667], [-56.539386, 1.899523], [-57.150098, 2.768927], [-57.281433, 3.333492], [-57.601569, 3.334655], [-58.044694, 4.060864], [-57.816021, 4.576801], [-57.914289, 4.812626], [-57.307246, 5.073567], [-57.147436, 5.97315], [-55.949318, 5.728788], [-55.84178, 5.953125], [-55.03325, 6.025291], [-53.958045, 5.756548], [-54.478633, 4.896756], [-54.399542, 4.212611], [-54.006931, 3.620038], [-54.181726, 3.18978], [-54.269705, 2.732392], [-54.524754, 2.311849]]]]}, {"type": "Feature", "properties": {"name": ""}, "bbox": [-54.524754, 2.053389, 51.148506], "geometry": {"type": "MultiPolygon", "coordinates": [[[[[-51.655797, 4.156232], [-52.249338, 3.241094], [-52.556425, 2.504705], [-52.939657, 2.124858], [-53.418465, 2.053389], [-53.554839, 2.334897], [-53.778521, 2.376703], [-54.088063, 2.105557], [-54.524754, 2.311849], [-54.269705, 2.732392], [-54.181726, 3.18978], [-54.006931, 3.620038], [-54.399542, 4.212611], [-54.478633, 4.896756], [-53.958045, 5.756548], [-53.618453, 5.646529], [-52.882141, 5.409851], [-51.823343, 4.565758], [-51.655797, 4.156232]]], [[6.18632, 49.463803], [6.65823, 49.201958], [8.099279, 49.017784], [7.593676, 48.333019], [7.466975, 47.620582], [7.192202, 47.449766], [6.736571, 47.541801], [6.768714, 47.287708], [6.037389, 46.725779], [6.022609, 46.27299], [6.500146, 46.429673], [6.843593, 45.991147], [6.802355, 45.70858], [7.096652, 45.33099], [6.749955, 45.028518], [7.007562, 44.254767], [7.549596, 44.127901], [7.435185, 43.693845], [6.529245, 43.128892], [4.556963, 43.399651], [3.100411, 43.075201], [2.985999, 42.473015], [1.826793, 42.343385], [0.701591, 42.795734], [0.338047, 42.579546], [-1.502771, 43.034014], [-1.901351, 43.422802], [-1.384225, 44.02261], [-1.193798, 46.014918], [-2.225724, 47.064363], [-2.963276, 47.570327], [-4.491555, 47.954954], [-4.59235, 48.68416], [-3.295814, 48.901692], [-1.616511, 48.644421], [-1.933494, 49.776342], [-0.989469, 49.347376], [1.338761, 50.127173], [1.639001, 50.946606], [2.513573, 51.148506], [2.658422, 50.796848], [3.123252, 50.780363], [3.588184, 50.378992], [4.286023, 49.907497], [4.799222, 49.985373], [5.674052, 49.529484], [5.897759, 49.442667], [6.18632, 49.463803]]], [[8.746009, 42.628122], [9.390001, 43.009985], [9.560016, 42.152492], [9.229752, 41.380007], [8.775723, 41.583612], [8.544213, 42.256517], [8.746009, 42.628122]]]]]]}, {"type": "Feature", "properties": {"name": ""}, "bbox": [-80.967765, -4.959129, -75.233723, 1.380924], "geometry": {"type": "Polygon", "coordinates": [[[-75.373223, -0.152032], [-75.233723, -0.911417], [-75.544996, -1.56161], [-76.635394, -2.608678], [-77.837905, -3.003021], [-78.450684, -3.873097], [-78.639897, -4.547784], [-79.205289, -4.959129], [-79.624979, -4.454198], [-80.028908, -4.346091], [-80.442242, -4.425724], [-80.469295, -4.059287], [-80.184015, -3.821162], [-80.302561, -3.404856], [-79.770293, -2.657512], [-79.986559, -2.220794], [-80.368784, -2.685159], [-80.967765, -2.246943], [-80.764806, -1.965048], [-80.933659, -1.057455], [-80.58337, -0.906663], [-80.399325, -0.283703], [-80.020898, 0.360334], [-80.09061, 0.768429], [-79.542762, 0.982938], [-78.855259, 1.380924], [-77.855061, 0.809925], [-77.668613, 0.825893], [-77.424984, 0.395687], [-76.57638, 0.256936], [-76.292314, 0.416047], [-75.801466, 0.084801], [-75.373223, -0.152032]]]]}, {"type": "Feature", "properties": {"name": ""}, "bbox": [-67.242428, 17.946553, -65.591004, 18.520601], "geometry": {"type": "Polygon", "coordinates": [[[-66.282434, 18.514762], [-65.771303, 18.426679], [-65.591004, 18.228035], [-65.847164, 17.975906], [-66.599934, 17.981823], [-67.184162, 17.946553], [-67.242428, 18.37446], [-67.100679, 18.520601], [-66.282434, 18.514762]]]]}, {"type": "Feature", "properties": {"name": ""}, "bbox": [-78.337719, 17.701116, -76.199659, 18.524218], "geometry": {"type": "Polygon", "coordinates": [[[-77.569601, 18.490525], [-76.896619, 18.400867], [-76.365359, 18.160701], [-76.199659, 17.886867], [-76.902561, 17.868238], [-77.206341, 17.701116], [-77.766023, 17.861597], [-78.337719, 17.701116]]]]}

337719,18.225968],[-78.217727,18.454533],[-77.797365,18.524218],[-77.569601,18.490525]]]]},{"type":"Feature","properties":{"name":"","bbox":[-84.974911,19.855481,-74.178025,23.188611],"geometry":{"type":"Polygon","coordinates":[[[-82.268151,23.188611],[-81.404457,23.117271],[-80.618769,23.105981],[-79.679524,22.765303],[-79.281486,22.399202],[-78.347434,22.512166],[-77.993296,22.277194],[-77.146422,21.657851],[-76.523825,21.20682],[-76.19462,21.220565],[-75.598222,21.016624],[-75.67106,20.735091],[-74.733896,20.693905],[-74.178025,20.284628],[-74.296648,20.050379],[-74.961595,19.923435],[-75.63468,19.873774],[-76.323656,19.952891],[-77.755481,19.855481],[-77.085108,20.413354],[-77.492655,20.673105],[-78.137292,20.739949],[-78.482827,21.028613],[-78.719867,21.598114],[-79.285,21.559175],[-80.217475,21.827324],[-80.517535,22.037079],[-81.820943,22.192057],[-82.169992,22.387109],[-81.795002,22.636965],[-82.775898,22.688151],[-83.494459,22.168518],[-83.9088,22.154565],[-84.052151,21.910575],[-84.54703,21.801228],[-84.974911,21.896028],[-84.447062,22.20495],[-84.230357,22.565755],[-83.77824,22.788118],[-83.267548,22.983042],[-82.510436,23.078747],[-82.268151,23.188611]]]}},{"type":"Feature","properties":{"name":"","bbox":[25.264226,-22.271612,32.849861,-15.507787],"geometry":{"type":"Polygon","coordinates":[[[31.191409,-22.25151],[30.659865,-22.151567],[30.322883,-22.271612],[29.839037,-22.102216],[29.432188,-22.091313],[28.794656,-21.639454],[28.02137,-21.485975],[27.727228,-20.851802],[27.724747,-20.499059],[27.296505,-20.39152],[26.164791,-19.293086],[25.850391,-18.714413],[25.649163,-18.536026],[25.264226,-17.73654],[26.381935,-17.846042],[26.706773,-17.961229],[27.044427,-17.938026],[27.598243,-17.290831],[28.467906,-16.4684],[28.825869,-16.389749],[28.947463,-16.043051],[29.516834,-15.644678],[30.274256,-15.507787],[30.338955,-15.880839],[31.173064,-15.860944],[31.636498,-16.07199],[31.852041,-16.319417],[32.328239,-16.392074],[32.847639,-16.713398],[32.849861,-17.979057],[32.654886,-18.67209],[32.611994,-19.419383],[32.772708,-19.715592],[32.659743,-20.30429],[32.508693,-20.395292],[32.244988,-21.116489],[31.191409,-22.25151]]]}},{"type":"Feature","properties":{"name":"","bbox":[19.895458,-26.828543,29.432188,-17.661816],"geometry":{"type":"Polygon","coordinates":[[[29.432188,-22.091313],[28.017236,-22.827754],[27.11941,-23.574323],[26

.786407,-24.240691],[26.485753,-24.616327],[25.941652,-24.696373],[25.765849,-25.174845],[25.664666,-25.486816],[25.025171,-25.71967],[24.211267,-25.670216],[23.73357,-25.390129],[23.312097,-25.26869],[22.824271,-25.500459],[22.579532,-25.979448],[22.105969,-26.280256],[21.605896,-26.726534],[20.889609,-26.828543],[20.66647,-26.477453],[20.758609,-25.868136],[20.165726,-24.917962],[19.895768,-24.76779],[19.895458,-21.849157],[20.881134,-21.814327],[20.910641,-18.252219],[21.65504,-18.219146],[23.196858,-17.869038],[23.579006,-18.281261],[24.217365,-17.889347],[24.520705,-17.887125],[25.084443,-17.661816],[25.264226,-17.73654],[25.649163,-18.536026],[25.850391,-18.714413],[26.164791,-19.293086],[27.296505,-20.39152],[27.724747,-20.499059],[27.727228,-20.851802],[28.02137,-21.485975],[28.794656,-21.639454],[29.432188,-22.091313]]]}},{"type":"Feature","properties":{"name":"","bbox":[11.734199,-29.045462,25.084443,-16.941343],"geometry":{"type":"Polygon","coordinates":[[[19.895768,-24.76779],[19.894734,-28.461105],[19.002127,-28.972443],[18.464899,-29.045462],[17.836152,-28.856378],[17.387497,-28.783514],[17.218929,-28.355943],[16.824017,-28.082162],[16.344977,-28.576705],[15.601818,-27.821247],[15.210472,-27.090956],[14.989711,-26.117372],[14.743214,-25.39292],[14.408144,-23.853014],[14.385717,-22.656653],[14.257714,-22.11208],[13.868642,-21.699037],[13.352498,-20.872834],[12.826845,-19.673166],[12.608564,-19.045349],[11.794919,-18.069129],[11.734199,-17.301889],[12.215461,-17.111668],[12.814081,-16.941343],[13.462362,-16.971212],[14.058501,-17.423381],[14.209707,-17.353101],[18.263309,-17.309951],[18.956187,-17.789095],[21.377176,-17.930636],[23.215048,-17.523116],[24.033862,-17.295843],[24.682349,-17.353411],[25.07695,-17.578823],[25.084443,-17.661816],[24.520705,-17.887125],[24.217365,-17.889347],[23.579006,-18.281261],[23.196858,-17.869038],[21.65504,-18.219146],[20.910641,-18.252219],[20.881134,-21.814327],[19.895458,-21.849157],[19.895768,-24.76779]]]}},{"type":"Feature","properties":{"name":"","bbox":[-17.625043,-12.33209,-11.467899,-16.598264],"geometry":{"type":"Polygon","coordinates":[[[-16.713729,13.594959],[-17.126107,14.373516],[-17.625043,14.729541],[-17.185173,14.919477],[-16.700706,15.621527],[-16.463098,16.135036],[-16.12069,16.455663],[-15.623666,16.369337],[-15.135737,16.587282],[-14.577348,16.598264],[-14.099521,16.304302],[-13.435738,16.039383],[-12.830658,15.303692],[-12.17075,14.616834],[-12.124887,13.994727],[-11.927716,13.422075],[-11.553398,13.141214],[-11.467899,12.754519],[-11.513943,12.442988],[-11.658301,12.386583],[-12.203565,12.465648],[-12.278599,12.35444],[-12.499051,12.33209],[-13.217818,12.575874],[-13.700476,12.586183],[-15.548477,12.62817],[-15.816574,12.515567],[-16.147717,12.547762],[-16.677452,12.384852],[-16.841525,13.151394],[-15.931296,13.130284],[-15.691001,13.270353],[-15.511813,13.27857],[-15.141163,13.509512],[-14.712197,13.298207],[-14.277702,13.280585],[-13.844963,13.505042],[-14.046992,13.794068],[-14.376714,13.62568],[-14.687031,13.630357],[-15.081735,13.876492],[-15.39877,13.860369],[-15.624596,13.623587],[-16.713729,13.594959]]]}},{"type":"Feature","properties":{"name":"","bbox":[-12.17075,10.096361,4.27021,24.974574],"geometry":{"type":"Polygon","coordinates":[[[-11.513943,12.442988],[-11.467899,12.754519],[-11.553398,13.141214],[-11.927716,13.422075],[-12.124887,13.994727],[-12.17075,14.616834],[-11.834208,14.799097],[-11.666078,15.388208],[-11.349095,15.411256],[-10.650791,15.132746],[-10.086846,15.330486],[-9.700255,15.264107],[-9.550238,15.486497],[-5.537744,15.50169],[-5.315277,16.201854],[-5.488523,16.325102],[-5.971129,20.640833],[-6.453787,24.956591],[-4.923337,24.974574],[-1.550955,22.792666],[1.823228,20.610809],[2.060991,20.142233],[2.683588,19.85623],[3.146661,19.693579],[3.158133,19.057364],[4.267419,19.155265],[4.27021,16.852227],[3.723422,16.184284],[3.638259,15.56812],[2.749993,15.409525],[1.385528,15.323561],[1.015783,14.968182],[0.374892,14.928908],[-0.266257,14.924309],[-0.515854,15.116158],[-1.066363,14.973815],[-2.01035,14.559008],[-2.191825,14.246418],[-2.967694,13.79815],[-3.103707,13.541267],[-3.522803,13.337662],[-4.006391,13.472485],[-4.280405,13.228444],[-4.427166,12.542646],[-5.220942,11.713859],[-5.197843,11.375146],[-5.470565,10.95127],[-5.40432,10.370737],[-5.816926,10.222555],[-6.050452,10.096361],[-6.205323,10.524061],[-6.493965,10.411303],[-6.666461,10.430811],[-6.850507,10.138994],[-7.622759,10.147236],[-7.89959,10.297282],[-8.029944,10.206535],[-8.335377,10.494812],[-8.282357,10.792597],[-8.407311,10.909257],[-8.620321,10.810891],[-8.581305,11.136246],[-8.376305,11.393646],[-8.786099,11.812561],[-8.905265,12.088358],[-9.127474,12.30806],[9.327616,12.334286],[-9.567912,12.194243],[-9.890993,12.060479],[-1.0165214,11.844084],[-10.593224,11.923975],[-10.87083,12.177887],[-11.036556,12.211245],[-11.297574,12.077971],[-11.456169,12.076834],[-11.513943,12.442988]]]}},{"type":"Feature","properties":{"name":"","bbox":[-17.063423,20.999752,-16.845194,21.333323],[-12.929102,21.327071,-13.118754,22.77122],[-12.874222,23.284832],[-11.937224,23.374594],[-11.969419,25.933353],[-8.687294,25.881056],[-8.6844,27.395744],[-4.923337,24.974574],[-6.453787,24.956591],[-5.971129,20.640833],[-5.488523,16.325102],[-5.315277,16.201854],[-5.537744,15.50169],[-9.550238,15.486497],[-9.700255,15.264107],[-10.086846,15.330486],[-10.650791,15.132746],[-11.349095,15.411256],[-11.666078,15.388208],[-11.834208,14.799097],[-12.17075,14.616834],[-12.830658,15.303692],[-13.435738,16.039383],[-14.099521,16.304302],[-14.577348,16.598264],[-15.135737,16.587282],[-15.623666,16.369337],[-16.12069,16.455663],[-16.463098,16.135036],[-16.549708,16.673892],[-16.270552,17.166963],[-16.146347,18.108482],[-16.256883,19.096716],[-16.377651,19.593817],[-16.277838,20.092521],[-16.536324,20.567866],[-17.063423,20.999752]]]}},{"type":"Feature","properties":{"name":"","bbox":[0.772336,6.142158,3.797112,12.235636],"geometry":{"type":"Polygon","coordinates":[[[2.691702,6.258817],[1.865241,6.142158],[1.618951,6.832038],[1.664478,9.12859],[1.463043,9.334624],[1.425061,9.825395],[1.077795,10.175607],[0.772336,10.470808],[0.899563,10.997339],[1.24347,11.110511],[1.447178,11.547719],[1.935986,11.64115],[2.154474,11.94015],[2.490164,12.233052],[2.848643,12.235636],[3.61118,11.660167],[3.572216,11.327939],[3.797112,10.734746],[3.60007,10.332186],[3.705438,10.06321],[3.229352,9.444153],[2.912308,9.137608],[2.723793,8.506845],[2.749063,7.870734],[2.691702,6.258817]]]}},{"type":"Feature","properties":{"name":"","bbox":[0.295646,11.660167,15.903247,18.734714],"geometry":{"type":"Polygon","coordinates":[[[14.8513,22.86295],[15.096888,21.308519],[15.47106,21.04845],[15.487148,20.730415],[15.903247,20.387619],[15.685741,19.95718],[15.300441,17.92795],[15.247731,16.627306],[13.97217,15.68437],[13.540394,14.367134],[13.956699,13.996691],[13.954477,13.353449],[14.595781,13.330427],[14.495787,12.859396],[14.213531,12.802035],[14.181336,12.483657],[13.995353,12.461565],[13.318702,13.556356],[13.083987,13.596147],[12.302071,13.037189],[11.527803,13.32898],[10.989593,13.387323],[10.701032,13.246918],[10.114814,13.277252],[9.524928,12.851102],[9.014933,12.826659],[7.804671,13.343527],[7.330747,13.098038],[6.820442,13.115091],[6.445426,13.492768],[5.443058,13.865924],[4.368344,13.747482],[4.107946,13.531216],[3.967283,12.956109],[3.680634,12.552903],[3.61118,11.660167],[2.848643,12.235636],[2.490164,12.233052],[2.154474,11.94015],[2.177108,12.625018],[1.024103,12.851826],[0.993046,13.33575],[0.429

928,13.988733],[0.295646,14.444235],[0.374892,14.928908],[1.015783,14.968182],[1.385528,15.323561],[2.749993,15.409525],[3.638259,15.56812],[3.723422,16.184284],[4.27021,16.852227],[4.267419,19.155265],[5.677566,19.601207],[8.572893,21.565661],[11.999506,23.471668],[13.581425,23.040506],[14.143871,22.491289],[14.8513,22.86295]]]]},{"type":"Feature","properties":{"name":"","bbox":[2.691702,4.240594,14.577178,13.865924],"geometry":{"type":"Polygon","coordinates":[[[2.691702,6.258817],[2.749063,7.870734],[2.723793,8.506845],[2.912308,9.137608],[3.220352,9.444153],[3.705438,10.06321],[3.60007,10.332186],[3.797112,10.734746],[3.572216,11.327939],[3.61118,11.660167],[3.680634,12.552903],[3.967283,12.956109],[4.107946,13.531216],[4.368344,13.747482],[5.443058,13.865924],[6.445426,13.492768],[6.820442,13.115091],[7.330747,13.098038],[7.804671,13.343527],[9.014933,12.826659],[9.524928,12.851102],[10.114814,13.277252],[10.701032,13.246918],[10.98959,13.387323],[11.527803,13.32898],[12.302071,13.037189],[13.083987,13.596147],[13.318702,13.556356],[13.995353,12.461565],[14.181336,12.483657],[14.577178,12.085361],[14.468192,11.904752],[14.415379,11.572369],[13.57295,10.798566],[13.30867,10.160362],[13.1676,9.640626],[12.955468,9.417772],[12.753672,8.717763],[12.218872,8.305824],[12.063946,7.799808],[11.839309,7.397042],[11.745774,6.981383],[11.058788,6.644427],[10.497375,7.055358],[10.118277,7.03877],[9.522706,6.453482],[9.233163,6.444491],[8.757533,5.479666],[8.500288,4.771983],[7.462108,4.412108],[7.082596,4.464689],[6.698072,4.240594],[5.898173,4.262453],[5.362805,4.887971],[5.033574,5.611802],[4.325607,6.270651],[3.57418,6.2583],[2.691702,6.258817]]]}},{"type":"Feature","properties":{"name":"","bbox":[8.488816,1.727673,16.012852,12.859396],"geometry":{"type":"Polygon","coordinates":[[[14.495787,12.859396],[14.89336,12.21905],[14.960152,11.555574],[14.923565,10.891325],[15.467873,9.982337],[14.909354,9.92129],[14.627201,9.920919],[14.171466,10.021378],[13.954218,9.549495],[14.544467,8.965861],[14.97996,8.796104],[15.120866,8.38215],[15.436092,7.692812],[15.27946,7.421925],[14.776545,6.408498],[14.53656,6.226959],[14.459407,5.451761],[14.558936,5.030598],[14.478372,4.732605],[14.950953,4.210389],[15.03622,3.851367],[15.405396,3.335301],[15.862732,3.013537],[15.907381,2.557389],[16.012852,2.26764],[15.940919,1.727673],[15.146342,1.964015],[14.337813,2.227875],[13.075822,2.267097],[12.951334,2.321616],[12.35938,2.192812],[11.751665,2.326758],[11.276449,2.261051],[9.64915

8,2.283866],[9.795196,3.073404],[9.404367,3.734527],[8.948116,3.904129],[8.744924,4.352215],[8.488816,4.495617],[8.500288,4.771983],[8.757533,5.479666],[9.233163,6.444491],[9.522706,6.453482],[10.118277,7.03877],[10.497375,7.055358],[11.058788,6.644427],[11.745774,6.981383],[11.839309,7.397042],[12.063946,7.799808],[12.218872,8.305824],[12.753672,8.717763],[12.955468,9.417772],[13.1676,9.640626],[13.308676,10.160362],[13.57295,10.798566],[14.415379,11.572369],[14.468192,11.904752],[14.577178,12.085361],[14.181336,12.483657],[14.213531,12.802035],[14.495787,12.859396]]]}},{"type":"Feature","properties":{"name":"","bbox":[-0.049785,5.928837,1.865241,11.018682],"geometry":{"type":"Polygon","coordinates":[[[0.899563,10.997339],[0.772336,10.470808],[1.077795,10.175607],[1.425061,9.825395],[1.463043,9.334624],[1.664478,9.12859],[1.618951,6.832038],[1.865241,6.142158],[1.060122,5.928837],[0.836931,6.279979],[0.570384,6.914359],[0.490957,7.411744],[0.712029,8.312465],[0.461192,8.677223],[0.365901,9.465004],[0.36758,10.191213],[-0.049785,10.706918],[0.023803,11.018682],[0.899563,10.997339]]]}},{"type":"Feature","properties":{"name":"","bbox":[-3.24437,4.710462,1.060122,11.098341],"geometry":{"type":"Polygon","coordinates":[[[0.023803,11.018682],[-0.049785,10.706918],[0.36758,10.191213],[0.365901,9.465004],[0.461192,8.677223],[0.712029,8.312465],[0.490957,7.411744],[0.570384,6.914359],[0.836931,6.279979],[1.060122,5.928837],[-0.507638,5.343473],[-1.063625,5.000548],[-1.964707,4.710462],[-2.856125,4.994476],[-2.810701,5.389051],[-3.24437,6.250472],[-2.983585,7.379705],[-2.56219,8.219628],[-2.827496,9.642461],[-2.963896,10.395335],[-2.940409,10.96269],[1.203358,11.009819],[-0.761576,10.93693],[-0.438702,11.098341],[0.023803,11.018682]]]}},{"type":"Feature","properties":{"name":"","bbox":[-8.60288,4.338288,-2.56219,10.524061],"geometry":{"type":"Polygon","coordinates":[[[-8.029944,10.206535],[-7.89959,10.297382],[-7.622759,10.147236],[-6.850507,10.138994],[-6.666461,10.430811],[-6.493965,10.411303],[-6.205223,10.524061],[-6.050452,10.096361],[-5.816926,10.222555],[-5.404342,10.370737],[-4.954653,10.152714],[-4.779884,9.821985],[-4.330247,9.610835],[-3.980449,9.862344],[-3.511899,9.900326],[-2.827496,9.642461],[-2.56219,8.219628],[-2.983585,7.379705],[-3.24437,6.250472],[-2.810701,5.389051],[-2.856125,4.994476],[-3.311084,4.984296],[-4.00882,5.179813],[-4.649917,5.168264],[-5.834496,4.993701],[-6.528769,4.705088],[-7.518941,4.338288],[-7.712159,4.364566],[-7.635368,5.188159],[-7.539715,5.313345],[-7.570153,5.707352],[-7.993693,6.12619],[8.311348,6.193033],[-8.60288,6.467564],[-8.385452,6.911801],[-8.485446,7.395208],[-8.439298,7.686043],[-8.280703,7.68718],[-8.221792,8.123329],[-8.299049,8.316444],[-8.203499,8.455453],[-7.8321,8.575704],[-8.079114,9.376224],[-8.309616,9.789532],[-8.229337,10.12902],[8.029944,10.206535]]]}},{"type":"Feature","properties":{"name":"","bbox":[-15.130311,7.309037,-7.8321,12.586183],"geometry":{"type":"Polygon","coordinates":[[[-13.700476,12.586183],[-13.217818,12.575874],[-12.499051,12.33209],[-12.278599,12.35444],[-12.203565,12.465648],[-11.658301,12.386583],[-11.513943,12.442988],[-11.456169,12.076834],[-11.297574,12.077971],[-11.036556,12.211245],[-10.87083,12.177887],[-10.593224,11.923975],[-10.165214,11.844084],[-9.890993,12.060479],[-9.567912,12.194243],[-9.327616,12.334286],[-9.127474,12.30806],[8.905265,12.088358],[-8.786099,11.812561],[-8.376305,11.393646],[-8.581305,11.136246],[-8.620321,10.810891],[-8.407311,10.909257],[-8.282357,10.792597],[-8.335377,10.494812],[-8.029944,10.206535],[-8.229337,10.12902],[-8.309616,9.789532],[-8.079114,9.376224],[-7.8321,8.575704],[-8.203499,8.455453],[-8.299049,8.316444],[-8.221792,8.123329],[-8.280703,7.68718],[-8.439298,7.686043],[-8.722124,7.711674],[-8.926065,7.309037],[-9.208786,7.313921],[-9.403348,7.526905],[-9.33728,7.928534],[-9.755342,8.541055],[-10.016567,8.428504],[-10.230094,8.406206],[-10.505477,8.348896],[-10.494315,8.715541],[-10.65477,8.977178],[-10.622395,9.26791],[-10.839152,9.688246],[-11.117481,10.045873],[-11.917277,10.046984],[-12.150338,9.858572],[-12.425929,9.835834],[-12.596719,9.620188],[-12.711958,9.342712],[-13.24655,8.903049],[-13.685154,9.494744],[-14.074045,9.886167],[-14.330076,10.01572],[-14.579699,10.21467],[-14.693232,10.656301],[-14.839554,10.876572],[-15.130311,10.040412],[-14.685687,11.527824],[-14.382192,11.509272],[-14.121406,11.677117],[-13.9008,11.678719],[-13.743161,11.811269],[-13.828272,12.142644],[-13.718744,12.247186],[-13.700476,12.586183]]]}},{"type":"Feature","properties":{"name":"","bbox":[-16.677452,11.040412,-13.700476,12.62817],"geometry":{"type":"Polygon","coordinates":[[[-16.677452,12.384852],[-16.147717,12.547762],[-15.816574,12.515567],[-15.548477,12.62817],[-13.700476,12.586183],[-13.718744,12.247186],[-13.828272,12.142644],[-13.743161,11.811269],[-13.9008,11.678719],[-14.121406,11.677117],[-14.382192,11.509272],[-14.685687,11.527824],[-15.130311,11.040412],[-15.66418,11.458474],[-16.085214,11.524594],[-16.314787,11.806515],[-16.308947,11.958702],[-16.613838,12.170911],[-16.677452,12.384852]]]}},{"type":"Feature","properties":{"name":"","bbox":[-11.438779,4.355755,-7.539715,8.541055],"geometry":{"type":"Polygon","coordinates":[[[-8.439298,7.686043],[-8.485446,7.395208],[-8.385452,6.911801],[-8.60288,6.467564],[-8.311348,6.193033],[-7.993693,6.12619],[-7.570153,5.707352],[-7.539715,5.313345],[-7.635368,5.188159],[-7.712159,4.364566],[-7.974107,4.355755],[-9.004794,4.832419],[-9.91342,5.593561],[-10.765384,6.140711],[-11.438779,6.785917],[-11.199802,7.105846],[-11.146704,7.396706],[-10.695595,7.939464],[-10.230094,8.406206],[-10.016567,8.428504],[-9.755342,8.541055],[-9.33728,7.928534],[-9.403348,7.526905],[-9.208786,7.313921],[-8.926065,7.309037],[-8.722124,7.711674],[-8.439298,7.686043]]]}},{"type":"Feature","properties":{"name":"","bbox":[-13.24655,6.785917,-10.230094,10.046984],"geometry":{"type":"Polygon","coordinates":[[[-13.24655,8.903049],[-12.711958,9.342712],[-12.596719,9.620188],[-12.425929,9.835834],[-12.150338,9.858572],[-11.917277,10.046984],[-11.117481,10.045873],[-10.839152,9.688246],[-10.622395,9.26791],[-10.65477,8.977178],[-10.494315,8.715541],[-10.505477,8.348896],[-10.230094,8.406206],[-10.016567,8.428504],[-11.146704,7.396706],[-11.199802,7.105846],[11.438779,6.785917],[-11.708195,6.860098],[-12.428099,7.262942],[-12.949049,7.798646],[-13.124025,8.163946],[-13.24655,8.903049]]]}},{"type":"Feature","properties":{"name":"","bbox":[-5.470565,9.610835,2.177108,15.116158],"geometry":{"type":"Polygon","coordinates":[[[-5.404342,10.370737],[-5.470565,10.95127],[-5.197843,11.375146],[-5.220942,11.713859],[-4.427166,12.542646],[-4.280405,13.228444],[-4.006391,13.472485],[-3.522803,13.337662],[-3.103707,13.541267],[-2.967694,13.79815],[-2.191825,14.246418],[-2.001035,14.559008],[-1.066363,14.973815],[-0.515854,15.116158],[-0.266257,14.924309],[0.374892,14.928908],[0.295646,14.444235],[0.429928,13.988733],[0.993046,13.33575],[1.024103,12.851826],[2.17108,12.625018],[2.154474,11.94015],[1.935986,11.64115],[1.447178,11.547719],[1.24347,11.110511],[0.899563,10.997339],[0.023803,11.018682],[-0.438702,11.098341],[-0.761576,10.93693],[-1.203358,11.009819],[-2.940409,10.96269],[-2.963896,10.395335],[-2.827496,9.642461],[-3.511899,9.900326],[-3.980449,9.862344],[-4.330247,9.610835],[-4.779884,9.821985],[-4.954653,10.152714],[-5.404342,10.370737]]]}},{"type":"Feature","properties":{"name":"","bbox":[14.459407

,2.26764,27.374226,11.142395],"geometry":{"type":"Polygon","coordinates":[[[27.374226,5.233944],[27.044065,5.127853],[26.402761,5.150875],[25.650455,5.256088],[25.278798,5.170408],[25.128833,4.927245],[24.805029,4.897247],[24.410531,5.108784],[23.297214,4.609693],[22.84148,4.710126],[22.704124,4.633051],[22.405124,4.02916],[21.659123,4.224342],[20.927591,4.322786],[20.290679,4.691678],[19.467784,5.031528],[18.932312,4.709506],[18.542982,4.201785],[18.453065,3.504386],[17.8099,3.560196],[17.133042,3.728197],[16.537058,3.198255],[16.012852,2.26764],[15.907381,2.557389],[15.862732,3.013537],[15.405396,3.335301],[15.03622,3.851367],[14.950953,4.210389],[14.478372,4.732605],[14.558936,5.030598],[14.459407,5.451761],[14.53656,6.226959],[14.776545,6.408498],[15.27946,7.421925],[16.106232,7.497088],[16.290562,7.754307],[16.456185,7.734774],[16.705988,7.508328],[17.96493,7.890914],[18.389555,8.281304],[18.911022,8.630895],[18.81201,8.982915],[19.094008,9.074847],[20.059685,9.012706],[21.000868,9.475985],[21.723822,10.567056],[22.231129,10.971889],[22.864165,11.142395],[22.977544,10.714463],[23.554304,10.089255],[23.55725,9.681218],[23.394779,9.265068],[23.459013,8.954286],[23.805813,8.666319],[24.567369,8.229188],[25.114932,7.825104],[25.124131,7.500085],[25.796648,6.979316],[26.213418,6.546603],[26.465909,5.946717],[27.213409,5.550953],[27.374226,5.233944]]]},"type":"Feature","properties":{"name":"","bbox":[11.093773,-5.037987,18.453065,3.728197],"geometry":{"type":"Polygon","coordinates":[[[18.453065,3.504386],[18.393792,2.900443],[18.094276,2.365722],[17.898835,1.741832],[17.774192,0.855659],[17.82654,0.288923],[17.663553,-0.058084],[17.638645,-0.424832],[17.523716,-0.74383],[16.865307,-1.225816],[16.407092,-1.740927],[15.972803,-2.712392],[16.00629,-3.535133],[15.75354,-3.855165],[15.170992,-4.343507],[14.582604,-4.970239],[14.209035,-4.793092],[14.144956,-4.510009],[13.600093,-4.500138],[13.25824,-4.882957],[12.995517,-4.781103],[12.62076,-4.438023],[12.318608,-4.60623],[11.914963,-5.037987],[11.093773,-3.978827],[11.855122,-3.426871],[11.478039,-2.765619],[11.820964,-2.514161],[12.495703,-2.391688],[12.575284,-1.948511],[13.109619,-2.42874],[13.992407,-2.470805],[14.29921,-1.998276],[14.425456,-1.333407],[14.316418,-0.552627],[13.843321,0.038758],[14.276266,1.19693],[14.026669,1.395677],[13.282631,1.314184],[13.003114,1.830896],[13.075822,2.267097],[14.337813,2.227875],[15.146342,1.964015],[15.940919,1.727673],[16.012852,2.26764],[16.537058,3.198255],[17.133042,3.728197],[17.8099,3.560196],[18.453065,3.504386]]]},"type":"Feature","properties":{"name":"","bbox":[8.797996,-3.97

8827,14.425456,2.326758],"geometry":{"type":"Polygon","coordinates":[[[11.276449,2.261051],[11.751665,2.326758],[12.35938,2.492812],[12.951334,2.321616],[13.075822,2.267097],[13.003114,1.830896],[13.282631,1.314184],[14.026669,1.395677],[14.276266,1.19693],[13.843321,0.038758],[14.316418,-0.552627],[14.425456,-1.333407],[14.29921,-1.998276],[13.992407,-2.470805],[13.109619,-2.42874],[12.575284,-1.948511],[12.495703,-2.391688],[11.820964,-2.514161],[11.478039,-2.765619],[11.855122,-3.426871],[11.093773,-3.978827],[10.066135,-2.969483],[9.405245,-2.144313],[8.797996,-1.111301],[8.830087,-0.779074],[9.04842,-0.459351],[9.291351,0.268666],[9.492889,1.01012],[9.830284,1.067894],[11.285079,1.057662],[11.276449,2.261051]]]},"type":"Feature","properties":{"name":"","bbox":[21.887843,-17.961229,33.485688,-8.238257],"geometry":{"type":"Polygon","coordinates":[[[30.74001,-8.340006],[31.157751,-8.594579],[31.556348,-8.762049],[32.191865,-8.930359],[32.759375,-9.230599],[33.231388,-9.676722],[33.485688,-10.525559],[33.31531,-10.79655],[33.114289,-11.607198],[33.306422,-12.435778],[32.991764,-12.783871],[32.688165,-13.712858],[33.214025,-13.97186],[30.179481,-14.796099],[30.274256,-15.507787],[29.516834,-15.644678],[28.947463,-16.043051],[28.825869,-16.389749],[28.467906,-16.4684],[27.598243,-17.290831],[27.044427,-17.938026],[26.706773,-17.961229],[26.381935,-17.846042],[25.264226,-17.73654],[25.084443,-17.661816],[25.07695,-17.578823],[24.682349,-17.353411],[24.033862,-17.295843],[23.215048,-17.523116],[22.562478,-16.898451],[21.887843,-16.08031],[21.933886,-12.898437],[24.016137,-12.911046],[23.930922,-12.565848],[24.079905,-12.191297],[23.904154,-11.722282],[24.017894,-11.237298],[23.912215,-10.926826],[24.257155,-10.951993],[24.314516,-11.262826],[24.78317,-11.238694],[25.418118,-11.330936],[25.75231,-11.784965],[26.553088,-11.92444],[27.16442,-11.608748],[27.388799,-12.132747],[28.155109,-12.272481],[28.523562,-12.698604],[28.934286,-13.248958],[29.699614,-13.257227],[29.616001,-12.178895],[29.341548,-12.360744],[28.642417,-11.971569],[28.372253,-11.793647],[28.49607,-10.789884],[28.673682,-9.605925],[28.449871,-9.164918],[28.734867,-8.526559],[29.002912,-8.407032],[30.346086,-8.238257],[30.74001,-8.340006]]]},"type":"Feature","properties":{"name":"","bbox":[32.688165,-16.8013,35.771905,-9.230599],"geometry":{"type":"Polygon","coordinates":[[[32.759375,-9.230599],[33.73972,-9.41715],[33.940838,-9.693674],[34.28,-10.16],[34.559989,-11.52002],[34.280006,-12.280025],[34.559989,-13.579998],[34.907151,-13.565425],[35.267956,-13.887834],[35.686845,-14.611046],[35.771905,-15.896859],[35.339063,-16.10744],[35.03381,-16.8013],[34.381292,-16.18356],[34.307291,-15.478641],[34.517666,-15.013709],[34.459633,-14.61301],[34.064825,-14.35995],[33.7897,-14.451831],[33.214025,-13.97186],[32.688165,-13.712858],[32.991764,-12.783871],[33.306422,-12.435778],[33.114289,-11.607198],[33.31531,-10.79655],[33.485688,-10.525559],[33.231388,-9.676722],[32.759375,-9.230599]]]},"type":"Feature","properties":{"name":"","bbox":[30.179481,-26.742192,40.775475,-10.317096],"geometry":{"type":"Polygon","coordinates":[[[34.559989,-11.52002],[35.312398,-11.439146],[36.514082,-11.720938],[36.775151,-11.594537],[37.47129,-11.56876],[37.82764,-11.26879],[38.427557,-11.285202],[39.521,-10.89688],[40.31659,-10.3171],[40.316586,-10.317098],[40.316589,-10.317096],[40.478387,-10.765441],[40.437253,-17.167111],[40.560811,-12.639177],[40.59962,-14.201975],[40.775475,-14.691764],[40.477251,-15.406294],[40.089264,-16.100774],[39.452559,-16.720891],[38.538351,-17.101023],[37.411133,-17.586368],[36.281279,-18.659688],[35.896497,-18.84226],[35.1984,-19.552811],[34.786383,-19.784012],[34.701893,-20.497043],[35.176127,-21.254361],[35.373428,-21.840837],[35.385848,-22.0474],[35.562546,-22.09],[35.533935,-23.070788],[35.371774,-23.535359],[35.60747,-23.706563],[35.458746,-24.12261],[35.040735,-24.478351],[34.215824,-24.816314],[33.01321,-25.357573],[32.574632,-25.727318],[32.660363,-26.148584],[32.915955,-26.215867],[32.83012,-26.742192],[32.071665,-26.73382],[31.985779,-26.29178],[31.837778,-25.843332],[31.752408,-25.484284],[31.930589,-24.369417],[31.670398,-23.658969],[31.191409,-22.25151],[32.244988,-21.116489],[32.508693,-20.395292],[32.659743,-20.30429],[32.772708,-19.715592],[32.611994,-19.419383],[32.654886,-18.67209],[32.849861,-17.979057],[32.847639,-16.713398],[32.328239,-16.392074],[31.852041,-16.319417],[31.636498,-16.07199],[31.173064,-15.860944],[30.338955,-15.880839],[30.274256,-15.507787],[30.179481,-14.796099],[33.214025,-13.97186],[33.7897,-14.451831],[34.064825,-14.35995],[34.459633,-14.61301],[34.517666,-15.013709],[34.307291,-15.478641],[34.381292,-16.18356],[35.03381,-16.8013],[35.339063,-16.10744],[35.771905,-15.896859],[35.686845,-14.611046],[35.267956,-13.887834],[34.907151,-13.565425],[34.559989,-13.579998],[34.280006,-12.280025],[34.559989,-11.52002]]]},"type":"Feature","properties":{"name":"","bbox":[30.676609,-27.285879,32.071665,-25.660191],"geometry":{"type":"Polygon","coordinates":[[[32.071665,-26.73382],[31.86806,-27.177927],[31.282773,-27.285879],[30.685962,-26.743845],[30.676609,-26.398078],[30.949667,-26.022649],[31.04408,-25.731452],[31.333158,-25.660191],[31.837778,-25.843332],[31.985779,-26.29178],[32.071665,-26.73382]]]},"type":"Feature","properties":{"name":"","bbox":[11.640096,-17.930636,24.079905,-4.438023],"geometry":{"type":"MultiPolygon","coordinates":[[[12.995517,-4.781103],[12.631612,-4.991271],[12.468004,-5.248362],[12.436688,-5.684304],[12.182337,-5.789931],[11.914963,-5.037987],[12.318608,-4.60623],[12.62076,-4.438023],[12.995517,-4.781103]]]}],[12.322432,-6.100092],[12.735171,-5.965682],[13.024869,-5.984389],[13.375597,-5.864241],[16.326528,-5.87747],[16.57318,-6.622645],[16.860191,-7.222298],[17.089996,-7.545689],[17.47297,-8.068551],[18.134222,-7.987678],[18.464176,-7.847014],[19.016752,-7.988246],[19.166613,-7.738184],[19.417502,-7.155429],[20.037723,-7.116361],[20.091622,-6.94309],[20.601823,-6.939318],[20.514748,-7.299606],[21.728111,-7.290872],[21.746456,-7.920085],[21.949131,-8.305901],[21.801801,-8.908707],[21.875182,-9.523708],[22.208753,-9.894796],[22.155268,-11.084801],[22.402798,-10.993075],[22.837345,-11.017622],[23.456791,-10.867863],[23.912215,-10.926826],[24.017894,-11.237298],[23.904154,-11.722282],[24.079905,-12.191297],[23.930922,-12.565848],[24.016137,-12.911046],[21.933886,-12.898437],[21.887843,-16.08031],[22.562478,-16.898451],[23.215048,-17.523116],[21.377176,-17.930636],[18.956187,-17.789095],[18.263309,-17.309951],[14.209707,-17.353101],[14.058501,-17.423381],[13.462362,-16.971212],[12.814081,-16.941343],[12.215461,-17.111668],[11.734199,-17.301889],[11.640096,-16.673142],[11.778537,-15.793816],[12.123581,-14.878316],[12.175619,-14.449144],[12.500095,-13.54777],[12.738479,-13.137906],[13.312914,-12.48363],[13.633721,-12.038645],[13.738728,-11.297863],[13.686379,-10.731076],[13.387328,-10.373578],[13.120988,-9.766897],[12.87537,-9.466934],[12.929061,-8.959091],[13.236433,-8.562629],[12.93304,-7.596539],[12.728298,-6.927122],[12.227347,-6.294448],[12.322432

, -6.100092]]]]}}, {"type": "Feature", "properties": {"name": ""}, "bbox": [29.024926, -4.499983, 30.75224, -2.348487], "geometry": {"type": "Polygon", "coordinates": [[[30.469674, -2.413855], [30.52766, -2.80762], [30.74301, -3.03431], [30.75224, -3.35931], [30.50554, -3.56858], [30.11632, -4.09012], [29.75312, -4.452389], [29.339998, -4.499983], [29.276384, -3.293907], [29.024926, -2.839258], [29.632176, -2.917858], [29.938359, -2.348487], [30.469674, -2.413855]]]]}}, {"type": "Feature", "properties": {"name": ""}, "bbox": [34.265433, 29.501326, 35.836397, 33.277426], "geometry": {"type": "Polygon", "coordinates": [[[35.719918, 32.709192], [35.545665, 32.393992], [35.18393, 32.532511], [34.974641, 31.866582], [35.225892, 31.754341], [34.970507, 31.616778], [34.927408, 31.353435], [35.397561, 31.489086], [35.420918, 31.100066], [34.922603, 29.501326], [34.823243, 29.761081], [34.26544, 31.21936], [34.265435, 31.219357], [34.265433, 31.219361], [34.556372, 31.548824], [34.488107, 31.605539], [34.752587, 32.072926], [34.955417, 32.827376], [35.098457, 33.080539], [35.126053, 33.0909], [35.460709, 33.08904], [35.552797, 33.264275], [35.821101, 33.277426], [35.836397, 32.868123], [35.700798, 32.716014], [35.719918, 32.709192]]]]}}, {"type": "Feature", "properties": {"name": ""}, "bbox": [35.126053, 33.08904, 36.61175, 34.644914], "geometry": {"type": "Polygon", "coordinates": [[[35.821101, 33.277426], [35.552797, 33.264275], [35.460709, 33.08904], [35.126053, 33.0909], [35.482207, 33.90545], [35.979592, 34.610058], [35.998403, 34.644914], [36.448194, 34.593935], [36.61175, 34.201789], [36.06646, 33.824912], [35.821101, 33.277426]]]]}}, {"type": "Feature", "properties": {"name": ""}, "bbox": [43.254187, -25.601434, 50.476537, -12.040557], "geometry": {"type": "Polygon", "coordinates": [[[49.543519, -12.469833], [49.808981, -12.895285], [50.056511, -13.555761], [50.217431, -14.758789], [50.476537, -15.226512], [50.377111, -15.706069], [50.200275, -16.000263], [49.860606, -15.414253], [49.672607, -15.710204], [49.863344, -16.451037], [49.774564, -16.875042], [49.498612, -17.106036], [49.435619, -17.953064], [49.041792, -19.118781], [48.548541, -20.496888], [47.930749, -22.391501], [47.547723, -23.781959], [47.095761, -24.94163], [46.282478, -25.178463], [45.409508, -25.601434], [44.833574, -25.346101], [44.03972, -24.988345], [43.763768, -24.460677], [43.697778, -23.574116], [43.345654, -22.776904], [43.254187, -22.057413], [43.433298, -21.336475], [43.893683, -21.163307], [43.89637, -20.830459], [44.374325, -20.072366], [44.464397, -19.435454], [44.232422, -18.961995], [44.042976, -18.331387], [43.963084, -17.409945], [44.312469, -16.850496], [44.446517, -16.216219], [44.944937, -16.179374], [45.502732, -15.974373], [45.872994, -15.793454], [46.312243, -15.780018], [46.882183, -15.210182], [47.70513, -14.594303], [48.005215, -14.091233], [47.869047, -13.663869], [48.293828, -13.784068], [48.84506, -13.089175], [48.863509, -12.487868], [49.194651, -12.040557], [49.543519, -12.469833]]]]}}, {"type": "Feature", "properties": {"name": ""}, "bbox": [34.927408, 31.353435, 35.397561, 31.489086], "geometry": {"type": "Polygon", "coordinates": [[[35.397561, 31.489086], [34.927408, 31.353435], [34.970507, 31.616778], [35.225892, 31.754341], [34.974641, 31.866582], [35.18393, 32.532511], [35.545665, 32.393992], [35.545252, 31.782505], [35.397561, 31.489086]]]]}}, {"type": "Feature", "properties": {"name": ""}, "bbox": [-16.841525, 13.130284, -13.844963, 13.876492], "geometry": {"type": "Polygon", "coordinates": [[[-16.713729, 13.594959], [-15.624596, 13.623587], [-15.39877, 13.860369], [-15.081735, 13.876492], [-14.687031, 13.630357], [-14.376714, 13.62568], [-14.046992, 13.794068], [-13.844963, 13.505042], [-14.277702, 13.280585], [-14.712197, 13.298207], [-15.141163, 13.509512], [-15.511813, 13.27857], [-15.691001, 13.270353], [-15.931296, 13.130284], [-16.841525, 13.151394], [-16.713729, 13.594959]]]]}}, {"type": "Feature", "properties": {"name": ""}, "bbox": [7.524482, 30.307556, 11.488787, 37.349994], "geometry": {"type": "Polygon", "coordinates": [[[9.48214, 30.307556], [9.055603, 32.102692], [8.439103, 32.506285], [8.430473, 32.748337], [7.612642, 33.344115], [7.524482, 34.097376], [8.140981, 34.655146], [8.376368, 35.479876], [8.217824, 36.433177], [8.420964, 36.946427], [9.509994, 37.349994], [10.210002, 37.230002], [10.18065, 36.724038], [11.028867, 37.092103], [11.100026, 36.899996], [10.600005, 36.41], [10.593287, 35.947444], [10.939519, 35.698984], [10.807847, 34.833507], [10.149593, 34.330773], [10.339659, 33.785742], [10.856836, 33.76874], [11.108501, 33.293343], [11.488787, 33.136996], [11.422523, 32.368903], [10.94479, 32.081815], [10.636901, 31.761421], [9.950225, 31.37607], [10.056575, 30.961831], [9.970017, 30.539325], [9.48214, 30.307556]]]]}}, {"type": "Feature", "properties": {"name": ""}, "bbox": [-8.6844, 19.057364, 11.999506, 37.118381], "geometry": {"type": "Polygon", "coordinates": [[[-8.6844, 27.395744], [-8.665124, 27.589479], [-8.66559, 27.656426], [-8.674116, 28.841289], [-7.059228, 29.579228], [-6.060632, 29.7317], [-5.242129, 30.000443], [-4.859646, 30.501188], [-3.690441, 30.896952], [-3.647498, 31.637294], [-3.06898, 31.724498], [-2.616605, 32.094344], [-1.307899, 32.262889], [-1.124551, 32.651522], [-1.388049, 32.864015], [-1.733455, 33.919713], [-1.792986, 34.527919], [-2.169914, 35.168396], [-1.208603, 35.714849], [-0.127454, 35.888662], [0.503877, 36.301273], [1.466919, 36.605647], [3.161699, 36.783905], [4.815758, 36.865037], [5.32012, 36.716519], [6.26182, 37.110655], [7.330385, 37.118381], [7.737078, 36.885708], [8.420964, 36.946427], [8.217824, 36.433177], [8.376368, 35.479876], [8.140981, 34.655146], [7.524482, 34.097376], [7.612642, 33.344115], [8.430473, 32.748337], [8.439103, 32.506285], [9.055603, 32.102692], [9.48214, 30.307556], [9.805634, 29.424638], [9.859998, 28.95999], [9.683885, 28.144174], [9.756128, 27.688259], [9.629056, 27.140953], [9.716286, 26.512206], [9.319411, 26.094325], [9.910693, 25.365455], [9.948261, 24.936954], [10.303847, 24.79313], [10.771364, 24.562532], [11.560669, 24.097909], [11.999506, 23.471668], [8.572893, 21.565661], [5.677566, 19.601207], [4.267419, 19.155265], [3.158133, 19.057364], [3.146661, 19.693579], [2.683588, 19.85623], [2.060994, 20.142233], [1.823228, 20.610809], [-1.550055, 22.792666], [-4.923337, 24.974574], [-8.6844, 27.395744]]]]}}, {"type": "Feature", "properties": {"name": ""}, "bbox": [34.922603, 29.501326, 35.719918, 32.709192], "geometry": {"type": "Polygon", "coordinates": [[[35.545665, 32.393992], [35.719918, 32.709192], [36.834062, 32.312938], [38.792341, 33.378686], [39.195468, 32.161009], [39.004886, 32.010217], [37.002166, 31.508413], [37.998849, 30.5085], [37.66812, 30.338665], [37.503582, 30.003776], [36.740528, 29.865283], [36.501214, 29.505254], [36.068941, 29.197495], [34.956037, 29.356555], [34.922603, 29.501326], [35.420918, 31.100066], [35.397561, 31.489086], [35.545252, 31.782505], [35.545665, 32.393992]]]]}}, {"type": "Feature", "properties": {"name": ""}, "bbox": [51.579519, 24.245497, 51.757441, 24.294073], "geometry": {"type": "Polygon", "coordinates": [[[51.579519, 24.245497], [51.757441, 24.294073], [51.794389, 24.019826], [52.577081, 24.177439], [53.404007, 24.151317], [54.008001, 24.121758], [54.693024, 24.797892], [55.439025, 25.439145], [56.070821, 26.05546], [56.261042, 25.714606], [56.396847, 24.924732], [55.886233, 24.920831], [55.804119, 24.269604], [55.981214, 24.130543], [55.528632, 23.933604], [55.525841, 23.524869], [55.234489, 23.110993], [55.208341, 22.70833], [55.006803, 22.496948], [52.000733, 23.001154], [51.617708, 24.014219], [51.579519, 24.245497]]]]}}, {"type": "Feature", "properties": {"name": ""}, "bbox": [50.743911, 24.556331, 51.006726, 11.4582], "geometry": {"type": "Polygon", "coordinates": [[[50.810108, 24.754743], [50.743911, 25.482424], [51.013352, 26.006992], [51.286462, 26.114582], [51.589079, 25.801113], [51.6067, 25.21567], [51.389608, 24.627386], [51.12415, 24.556331], [50.810108, 24.754743]]]]}}, {"type": "Feature", "properties": {"name": ""}, "bbox": [46.568713, 28.526063, 48.416094, 30.05907], "geometry": {"type": "Polygon", "coordinates": [[[47.974519, 29.975819], [48.183189, 29.534477], [48.093943, 29.306299], [48.416094, 28.526063], [47.708851, 28.526063], [47.459822, 29.002519], [46.568713, 29.099025], [47.302622, 30.05907], [47.974519, 29.975819]]]]}}, {"type": "Feature", "properties": {"name": ""}, "bbox": [38.792341, 29.099025, 48.567971, 37.385264], "geometry": {"type": "Polygon", "coordinates": [[[39.195468, 32.161009], [38.792341, 33.378686], [41.006159, 34.419372], [41.38965, 35.628317], [41.289707, 36.358815], [41.837064, 36.605854], [42.349591, 37.229873], [42.779126, 37.385264], [43.942259, 37.256228], [44.239452, 37.001514], [44.772677, 37.170437], [45.420618, 35.977546], [46.07634, 35.677383], [46.151788, 35.093259], [45.64846, 34.748138], [45.416691, 33.967798], [46.109362, 33.017287], [47.334661, 32.469155], [47.849204, 31.709176], [47.685286, 30.984853], [48.004698, 30.985137], [48.014568, 30.452457], [48.567971, 29.926778], [47.974519, 29.975819], [47.302622, 30.05907], [46.568713, 29.099025], [44.709499, 29.178891], [41.889981, 31.190009], [40.399994, 31.889992], [39.195468, 32.161009]]]]}}, {"type": "Feature", "properties": {"name": ""}, "bbox": [52.000001, 16.651051, 59.808006, 26.395934], "geometry": {"type": "MultiPolygon", "coordinates": [[[[55.208341, 22.70833], [55.234489, 23.110993], [55.525841, 23.524869], [55.528632, 23.933604], [55.981214, 24.130543], [55.804119, 24.269604], [55.886233, 24.920831], [56.396847, 24.924732], [56.84514, 24.241673], [57.403453, 23.878594], [58.136948, 23.747931], [58.729211, 23.565668], [59.180502, 22.992395], [59.450098, 22.660271], [59.808006, 22.533612], [59.806148, 22.310525], [59.442191, 21.714541], [59.282408, 21.433886], [58.861141, 21.114035], [58.487986, 20.428986], [58.034318, 20.481437], [57.826373, 20.243002], [57.665762, 19.736005], [57.7887, 19.06757], [57.694391, 18.94471], [57.234264, 18.947991], [56.609651, 18.574267], [56.512189, 18.087113], [56.283521, 17.876067], [55.661492, 17.884128], [55.269939, 17.632309], [55.2749, 17.228354], [54.791002, 16.950697], [54.239253, 17.044981], [53.570508, 16.707663], [53.108573, 16.651051], [52.782184, 17.349742], [52.000001, 19.000003], [54.999982, 19.999994], [55.666659, 22.000001], [55.208341, 22.70833]]]]}}, {"type": "Feature", "properties": {"name": ""}, "bbox": [166.629137, -16.59785, 167.844877, -14.626497], "geometry": {"type": "MultiPolygon", "coordinates": [[[[167.216801, -15.891846], [167.844877, -16.466333], [167.515181, -16.59785], [167.180008, -16.159995], [167.216801, -15.891846]]]]}}, [[166.793158, -15

.668811],[166.649859,-15.392704],[166.629137,-14.626497],[167.107712,-14.93392],[167.270028,-15.740021],[167.001207,-15.614602],[166.793158,-15.668811]]]]},{ "type": "Feature", "properties": { "name": "", "bbox": [102.348099, 10.486544, 107.614548, 14.570584], "geometry": { "type": "Polygon", "coordinates": [[[102.584932, 12.186595], [102.348099, 13.394247], [102.988422, 14.225721], [104.281418, 14.416743], [105.218777, 14.273212], [106.043946, 13.881091], [106.496373, 14.570584], [107.382727, 14.202441], [107.614548, 13.535531], [107.491403, 12.337206], [105.810524, 11.567615], [106.24967, 10.961812], [105.199915, 10.88931], [104.334335, 10.486544], [103.49728, 10.632555], [103.09069, 11.153661], [102.584932, 12.186595]]]]}, {"type": "Feature", "properties": { "name": "", "bbox": [97.375896, 5.691384, 105.589039, 20.41785], "geometry": { "type": "Polygon", "coordinates": [[[105.218777, 14.273212], [104.281418, 14.416743], [102.988422, 14.225721], [102.348099, 13.394247], [102.584932, 12.186595], [101.68715, 12.64574], [100.83181, 12.627085], [100.978467, 13.412722], [100.097797, 13.406856], [100.018733, 12.307001], [99.478921, 10.846367], [99.153772, 9.963061], [99.223299, 9.239255], [99.873832, 9.207862], [100.279647, 8.295153], [100.459274, 7.429573], [101.017328, 6.856869], [101.623079, 6.740622], [102.141187, 6.221636], [101.814282, 5.810808], [101.154219, 5.691384], [101.075516, 6.204867], [100.259596, 6.642825], [100.085757, 6.464489], [99.690691, 6.848213], [99.519642, 7.343454], [98.988253, 7.907993], [98.50378, 8.382305], [98.339662, 7.794512], [98.150009, 8.350007], [98.25915, 8.973923], [98.553551, 9.93296], [99.038121, 10.960546], [99.587286, 11.892763], [99.196354, 12.804748], [99.212012, 13.269294], [99.097755, 13.827503], [98.430819, 14.622028], [98.192074, 15.123703], [98.537376, 15.308497], [98.903348, 16.177824], [98.493761, 16.837836], [97.859123, 17.567946], [97.375896, 18.445438], [97.797783, 18.62708], [98.253724, 19.708203], [98.959676, 19.752981], [99.543309, 20.186598], [100.115988, 20.41785], [100.548881, 20.109238], [100.606294, 19.508344], [101.282015, 19.462585], [101.035931, 18.408928], [101.059548, 17.512497], [102.113592, 18.109102], [102.413005, 17.932782], [102.998706, 17.961695], [103.200192, 18.309632], [103.956477, 18.240954], [104.716947, 17.428859], [104.779321, 16.441865], [105.589039, 15.570316], [105.544338, 14.723934], [105.218777, 14.273212]]]]}, {"type": "Feature", "properties": { "name": "", "bbox": [100.115988, 13.881091, 107.564525, 22.464753], "geometry": { "type": "Polygon", "coordinates": [[[107.382727, 14.202441], [106.496373, 14.570584], [106.043946, 13.881091], [105.218777, 14.273212], [105.544338, 14.723934], [105.589039, 15.570316], [104.779321, 16.441865], [104.716947, 17.428859], [103.956477, 18.240954], [103.200192, 18.309632], [102.998706, 17.961695], [102.413005, 17.932782], [102.113592, 18.109102], [101.059548, 17.512497], [101.035931, 18.408928], [101.282015, 19.462585], [100.606294, 19.508344], [100.548881, 20.109238], [100.115988, 20.41785], [100.329101, 20.786122], [101.180005, 21.436573

], [101.270026, 21.201652], [101.80312, 21.174367], [101.652018, 22.318199], [102.170436, 22.464753], [102.754896, 21.675137], [103.203861, 20.766562], [104.435, 20.758733], [104.822574, 19.886642], [104.183388, 19.624668], [103.896532, 19.265181], [105.094598, 18.666975], [105.925762, 17.485315], [106.556008, 16.604284], [107.312706, 15.908538], [107.564525, 15.202173], [107.382727, 14.202441]]]]}, {"type": "Feature", "properties": { "name": "", "bbox": [92.303234, 9.93296, 101.180005, 28.335945], "geometry": { "type": "Polygon", "coordinates": [[[100.115988, 20.41785], [99.543309, 20.186598], [98.959676, 19.752981], [98.253724, 19.708203], [97.797783, 18.62708], [97.375896, 18.445438], [97.859123, 17.567946], [98.493761, 16.837836], [98.903348, 16.177824], [98.537376, 15.308497], [98.192074, 15.123703], [98.430819, 14.622028], [99.097755, 13.827503], [99.212012, 13.269294], [99.196354, 12.804748], [99.587286, 11.892763], [99.038121, 10.960546], [98.553551, 9.93296], [98.457174, 10.675266], [98.764546, 11.441292], [98.428339, 12.032987], [98.509574, 13.122378], [98.103604, 13.64046], [97.777732, 14.837286], [97.597072, 16.100568], [97.16454, 16.928734], [96.505769, 16.427241], [95.369352, 15.71439], [94.808405, 15.803454], [94.188804, 16.037936], [94.533486, 17.27724], [94.324817, 18.213514], [93.540988, 19.366493], [93.663255, 19.726962], [93.078278, 19.855145], [92.368554, 20.670883], [92.303234, 21.475485], [92.652257, 21.324048], [92.672721, 22.041239], [93.166128, 22.27846], [93.060294, 22.703111], [93.286327, 23.043658], [93.325188, 24.078566], [94.106742, 23.850741], [94.552658, 24.675238], [94.603249, 25.162495], [95.155153, 26.001307], [95.124768, 26.573572], [96.419366, 27.264589], [97.133999, 27.083774], [97.051989, 27.690559], [97.402561, 27.882536], [97.327114, 28.261583], [97.91198, 28.335945], [98.246231, 27.747221], [98.68269, 27.508812], [98.712094, 26.743536], [98.671838, 25.918703], [97.724609, 25.083637], [97.60472, 23.897405], [98.660262, 24.063286], [98.898749, 23.142722], [99.531992, 22.949039], [99.240899, 22.118314], [99.98348, 21.742937], [100.416538, 21.558839], [101.150033, 21.849984], [101.180005, 21.436573], [100.329101, 20.786122], [100.115988, 20.41785]]]]}, {"type": "Feature", "properties": { "name": "", "bbox": [102.170436, 8.59976, 109.33527, 23.352063], "geometry": { "type": "Polygon", "coordinates": [[[104.334335, 10.486544], [105.199915, 10.88931], [106.24967, 10.961812], [105.810524, 11.567615], [107.491403, 12.337206], [107.614548, 13.535531], [107.382727, 14.202441], [107.564525, 15.202173], [107.312706, 15.908538], [106.556008, 16.604284], [105.925762, 17.485315], [105.094598, 18.666975], [103.896532, 19.265181], [104.183388, 19.624668], [104.822574, 19.886642], [104.435, 20.758733], [103.203861, 20.766562], [102.754896, 21.675137], [102.170436, 22.464753], [102.706992, 22.708795], [103.504515, 22.703757], [104.476858, 22.81915], [105.329209, 23.352063], [105.811247, 22.976892], [106.725403, 22.794268], [106.567273, 22.218205], [107.04342, 21.811899], [108.05018, 21.55238], [106.715068, 20.696851], [105.881682, 19.75205], [105.662006, 19.058165], [106.426817, 18.004121], [107.361954, 16.697457], [108.269495, 16.079742], [108.877107, 15.276691], [109.33527, 13.426028], [109.200136, 11.666859], [108.36613, 11.008321], [107.220929, 10.364484], [106.405113, 9.53084], [105.158264, 8.59976], [104.795185, 9.241038], [105.076202, 9.918491], [104.334335, 10.486544]]]]}, {"type": "Feature", "properties": { "name": "", "bbox": [124.265625, 37.669071, 130.780007, 42.985387], "geometry": { "type": "MultiPolygon", "coordinates": [[[130.780004, 42.220008], [130.780005, 42.22001], [130.780007, 42.220007], [130.780004, 42.220008]]], [[[130.64, 42.395024], [130.64, 42.395], [130.779992, 42.22001], [130.400031, 42.280004], [129.965949, 41.941368], [129.667362, 41.601104], [129.705189, 40.882828], [129.188115, 40.661808], [129.0104, 40.485436], [128.633368, 40.189847], [127.967414, 40.025413], [127.533436, 39.75685], [127.50212, 39.323931], [127.385434, 39.213472], [127.783343, 39.050898], [128.349716, 38.612243], [128.205746, 38.370397], [127.780035, 38.304536], [127.073309, 38.256115], [126.68372, 37.804773], [126.237339, 37.840378], [126.174759, 37.749686], [125.689104, 37.94001], [125.568439, 37.752089], [125.27533, 37.669071], [125.240087, 37.857224], [124.981033, 37.948821], [124.712161, 38.108346], [124.985994, 38.548474], [125.221949, 38.665857], [125.132859, 38.848559], [125.38659, 39.387958], [125.321116, 39.551385], [124.737482, 39.660344], [124.265625, 39.928493], [125.079942, 40.569824], [126.182045, 41.107336], [126.869083, 41.816569], [127.343783, 41.503152], [128.208433, 41.466772], [128.052215, 41.994285], [129.596669, 42.424982], [129.994267, 42.985387], [130.64, 42.395024]]]]}}, {"type": "Feature", "properties": { "name": "", "bbox": [126.174759, 37.749686], [126.237339, 37.840378], [126.68372, 37.804773], [127.073309, 38.256115], [127.780035, 38.304536], [128.205746, 38.370397], [128.349716, 38.612243], [129.21292, 37.432392], [129.46045, 36.784189], [129.468304, 35.632141], [129.091377, 35.082484], [128.18585, 34.890377], [127.386519, 34.475674], [126.485748, 34.390046], [126.37392, 34.93456], [126.559231, 35.684541], [126.117398, 36.725485], [126.860143, 36.893924], [126.174759, 37.749686]]]]}, {"type": "Feature", "properties": { "name": "", "bbox": [87.751264, 49.297198], [88.805567, 49.470521], [90.713667, 50.331812], [92.234712, 50.802171], [93.10421, 50.49529], [94.147566, 50.480537], [94.815949, 50.013433], [95.81402, 49.97746], [97.25976, 49.72605], [98.231762, 50.422401], [97.82574, 51.010995], [98.861491, 52.047366], [99.981732, 51.634006], [100.88948, 51.516856], [102.06521, 51.25991], [102.25589, 50.51056], [103.676545, 50.089966], [104.62158, 50.27532], [105.886591, 50.406019], [106.888804, 50.274296], [107.868176, 49.793705], [108.475167, 49.282548], [109.402449, 49.292961], [110.662011, 49.130128], [111.581231, 49.377968], [112.89774, 49.543565], [114.362456, 50.248303], [114.96211, 50.140247], [115.485695, 49.805177], [116.678801, 49.888531], [116.191802, 49.134598], [115.485282, 48.135383], [115.742837, 47.726545], [116.308953, 47.85341], [117.295507, 47.697709], [118.064143, 48.06673], [118.866574, 47.74706], [119.772824, 47.048059], [119.66327, 46.69268], [118.874326, 46.805412], [117.421701, 46.672733], [116.717868, 46.388202], [115.985096, 45.727235], [114.460332, 45.339817], [113.463907, 44.808893], [112.436062, 45.011646], [111.873306, 45.102079], [111.348377, 44.457442], [111.66737, 44.073176], [111.829588, 43.743118], [111.129682, 43.406834], [110.412103, 42.871234], [109.243596, 42.519446], [107.744773, 42.481516], [106.129316, 42.134328], [104.964994, 41.59741], [104.522282, 41.908347], [103.312278, 41.907468], [101.83304, 42.514873], [100.845866, 42.663804], [99.515817, 42.524691], [97.451757, 42.74889], [96.349396, 42.725635], [95.762455, 43.319449], [95.306875, 44.241331], [94.688929, 44.352332], [93.480734, 44.975472], [92.133891, 45.115076], [90.94554, 45.286073], [90.585768, 45.719716], [90.970809, 46.888146], [90.280826, 47.693549], [88.854298, 48.069082], [88.013832, 48.599463], [87.751264, 49.297198]]]]}, {"type": "Feature", "properties": { "name": "", "bbox": [68.176645, 7.965535, 97.402561, 35.49401], "geometry": { "type": "Polygon", "coordinates": [[[97.327114, 28.261583], [97.402561, 27.882536], [97.051989, 27.699059], [97.133999, 27.083774], [96.419366, 27.264589], [95.124768, 26.573572], [95.155153, 26.001307], [94.603249, 25.162495], [94.552658, 24.675238], [94.106742, 23.850741], [93.3

25188, 24.078556], [93.286327, 23.043658], [93.060294, 22.703111], [93.166128, 22.27846], [92.672721, 22.041239], [92.146035, 23.627499], [91.869928, 23.624346], [91.706475, 22.985264], [91.158963, 23.503527], [91.46773, 24.072639], [91.915093, 24.130414], [92.376202, 24.976693], [91.799596, 25.147432], [90.872211, 25.132601], [89.920693, 25.26975], [89.832481, 25.965082], [89.355094, 26.014407], [88.563049, 26.446526], [88.209789, 25.768066], [88.931554, 25.238692], [88.306373, 24.866079], [88.084422, 24.501657], [88.69994, 24.233715], [88.52977, 23.631142], [88.876312, 22.879146], [89.031961, 22.055708], [88.888766, 21.690588], [88.208497, 21.703172], [86.975704, 21.495562], [87.033169, 20.743308], [86.499351, 20.151638], [85.060266, 19.478579], [83.941006, 18.30201], [83.189217, 17.671221], [82.192792, 17.016636], [82.191242, 16.556664], [81.692719, 16.310219], [80.791999, 15.951972], [80.324896, 15.899185], [80.025069, 15.136415], [80.233274, 13.835771], [80.286294, 13.006261], [79.862547, 12.056215], [79.857999, 10.357275], [79.340512, 10.308854], [78.885345, 9.546136], [79.18972, 9.216544], [78.277941, 8.933047], [77.941165, 8.252959], [77.539898, 7.965535], [76.592979, 8.899276], [76.130061, 10.29963], [75.746467, 11.308251], [75.396101, 11.781245], [74.864816, 12.741936], [74.616717, 13.992583], [74.443859, 14.617222], [73.534199, 15.990652], [73.119909, 17.92857], [72.820909, 19.208234], [72.824475, 20.419503], [72.630533, 21.356009], [71.175273, 20.757441], [70.470459, 20.877331], [69.16413, 22.089298], [69.644928, 22.450775], [69.349597, 22.84318], [68.176645, 23.691965], [68.842599, 24.359134], [71.04324, 24.356524], [70.844699, 25.215102], [70.282873, 25.722229], [70.168927, 26.491872], [69.514393, 26.940966], [70.616496, 27.989196], [71.777666, 27.91318], [72.823752, 28.961592], [73.450638, 29.976413], [74.42138, 30.979815], [74.405929, 31.692639], [75.258642, 32.271105], [74.451559, 32.7649], [74.104294, 33.441473], [73.749948, 34.317699], [74.240203, 34.748887], [75.757061, 34.504923], [76.871722, 34.653544], [77.837451, 35.49401], [78.912269, 34.321936], [78.811086, 33.506198], [79.208892, 32.994395], [79.176129, 32.48378], [78.458446, 32.618164], [78.738894, 31.515906], [79.721367, 30.882715], [81.111256, 30.183481], [80.476721, 29.729865], [80.088425, 28.79447], [81.057203, 28.416095], [81.999987, 27.925479], [83.304249, 27.364506], [84.675018, 27.234901], [85.251779, 26.726198], [86.024393, 26.630985], [87.227472, 26.397898], [88.060238, 26.414615], [88.174804, 26.810405], [88.043133, 27.445819], [88.120441, 27.876542], [88.730326, 28.086865], [88.814248, 27.299316], [88.835643, 27.098966], [89.744528, 26.719403], [90.373275, 26.875724], [91.217513, 26.808648], [92.033484, 26.83831], [92.103712, 27.452614], [91.696657, 27.771742], [92.503119, 27.896876], [93.413348, 28.640629], [94.56599, 29.277438], [95.404802, 29.031717], [96.117679, 29.452802], [96.586591, 28.83098], [96.248833, 28.411031], [97.327114, 28.261583]]], {"type": "Feature", "properties": {"name": ""}, "bbox": [88.084422, 20.670883, 92.672721, 26.446526], "geometry": {"type": "Polygon", "coordinates": [[[92.672721, 22.041239], [92.652257, 21.324048], [92.303234, 21.475485], [92.368554, 20.670883], [92.082886, 21.192195], [92.

.025215, 21.70157], [91.834891, 22.182936], [91.417087, 22.765019], [90.496006, 22.805017], [90.586957, 22.392794], [90.272971, 21.836368], [89.847467, 22.039146], [89.70205, 21.857116], [89.418863, 21.966179], [89.031961, 22.055708], [88.876312, 22.879146], [88.52977, 23.631142], [88.69994, 24.233715], [88.084422, 24.501657], [88.306373, 24.866079], [88.931554, 25.238692], [88.209789, 25.768066], [88.563049, 26.446526], [89.355094, 26.014407], [89.832481, 25.965082], [89.920693, 25.26975], [90.872211, 25.132601], [91.799596, 25.147432], [92.376202, 24.976693], [91.915093, 24.130414], [91.46773, 24.072639], [91.158963, 23.503527], [91.706475, 22.985264], [91.869928, 23.624346], [92.146035, 23.627499], [92.672721, 22.041239]]]], {"type": "Feature", "properties": {"name": ""}, "bbox": [88.814248, 26.719403, 92.103712, 28.261583], "geometry": {"type": "Polygon", "coordinates": [[[91.696657, 27.771742], [92.103712, 27.452614], [92.033484, 26.83831], [91.217513, 26.808648], [90.373275, 26.875724], [89.744528, 26.719403], [88.835643, 27.098966], [88.814248, 27.299316], [89.47581, 28.042759], [90.015829, 28.296439], [90.730514, 28.064954], [91.258854, 28.040614], [91.696657, 27.771742]]]], {"type": "Feature", "properties": {"name": ""}, "bbox": [80.088425, 26.397898, 88.120441, 30.422717], "geometry": {"type": "Polygon", "coordinates": [[[88.120441, 27.876542], [88.043133, 27.445819], [88.174804, 26.810405], [88.060238, 26.414615], [87.227472, 26.397898], [86.024393, 26.630985], [85.251779, 26.726198], [84.675018, 27.234901], [83.304249, 27.364506], [81.999987, 27.925479], [81.057203, 28.416095], [80.088425, 28.79447], [80.476721, 29.729865], [81.111256, 30.183481], [81.525804, 30.422717], [82.327513, 30.115268], [83.337115, 29.463732], [83.898993, 29.320226], [84.23458, 28.839894], [85.011638, 28.642774], [85.82332, 28.203576], [86.954517, 27.974262], [88.120441, 27.876542]]]], {"type": "Feature", "properties": {"name": ""}, "bbox": [60.874248, 23.691965, 77.837451, 37.133031], "geometry": {"type": "Polygon", "coordinates": [[[77.837451, 35.49401], [76.871722, 34.653544], [75.757061, 34.504923], [74.240203, 34.748887], [73.749948, 34.317699], [74.104294, 33.441473], [74.451559, 32.7649], [75.258642, 32.271105], [74.405929, 31.692639], [74.42138, 30.979815], [73.450638, 29.976413], [72.823752, 28.961592], [71.777666, 27.91318], [70.616496, 27.989196], [69.514393, 26.940966], [70.168927, 26.491872], [70.282873, 25.722229], [70.844699, 25.215102], [71.04324, 24.356524], [68.842599, 24.359134], [68.176645, 23.691965], [67.443667, 23.944844], [67.145442, 24.663611], [66.372828, 25.425141], [64.530408, 25.237039], [62.905701, 25.218409], [61.497363, 25.078237], [61.874187, 26.239975], [63.316632, 26.756532], [63.23898, 27.210477], [62.755426, 27.378923], [62.72783, 28.259645], [61.771868, 28.699334], [61.369309, 29.303276], [60.874248, 29.829239], [62.549857, 29.318572], [63.550261, 29.468331], [64.148002, 29.340819], [64.350419, 29.560031], [65.046862, 29.472181], [66.346473, 29.887943], [66.381458, 30.738899], [66.938891, 31.304911], [67.683394, 31.303154], [67.792689, 31.58293], [68.556932, 31.71331], [68.926677, 31.620189], [69.317764, 31.901412], [69.262522, 32.501944], [69.687147, 33.105499], [70.323594, 33.358533], [69.930543, 34.02012], [70.881803, 33.988856], [71.156773, 34.348911], [71.115019, 34.733126], [71.613076, 35.153203], [71.498768, 35.650563], [71.262348, 36.074388], [71.846292, 36.509942], [72.920025, 36.720007], [74.067552, 36.836176], [74.575893, 37.020841], [75.158028, 37.133031], [75.896897, 36.666806], [76.192848, 35.898403], [77.837451, 35.49401]]]], {"type": "Feature", "properties": {"name": ""}, "bbox": [60.52843, 29.318572, 75.158028, 38.486282], "geometry": {"type": "Polygon", "coordinates": [[[66.518607, 37.362784], [67.075782, 37.356144], [67.83, 37.144994], [68.135562, 37.023115], [68.859446, 37.344336], [69.196273, 37.151144], [69.518785, 37.608997], [70.116578, 37.588223], [70.270574, 37.735165], [70.376304, 38.138396], [70.806821, 38.486282], [71.348131, 38.258905], [71.239404, 37.953265], [71.541918, 37.905774], [71.448693, 37.065645], [71.844638, 36.738171], [72.193041, 36.948288], [72.63689, 37.047558], [73.260056, 37.495257], [73.948696, 37.421566], [74.980002, 37.41999], [75.158028, 37.133031], [74.575893, 37.020841], [74.067552, 36.836176], [72.920025, 36.720007], [71.846292, 36.509942], [71.262348, 36.074388], [71.498768, 35.650563], [71.613076, 35.153203], [71.115019, 34.733126], [71.156773, 34.348911], [70.881803, 33.988856], [69.930543, 34.02012], [70.323594, 33.358533], [69.868714, 33.105499], [69.262522, 32.501944], [69.317764, 31.901412], [68.926677, 31.620189], [68.556932, 31.71331], [67.792689, 31.58293], [67.683394, 31.303154], [66.938891, 31.304911], [66.381458, 30.738899], [66.346473, 29.887943], [65.046862, 29.472181], [64.350419, 29.560031], [64.148002, 29.340819], [63.550261, 29.468331], [62.549857, 29.318572], [60.874248, 29.829239], [61.781222, 30.73585], [61.699314, 31.379506], [60.914945, 31.548075], [60.863655, 32.18292], [60.536078, 32.981269], [60.9637, 33.528832], [60.52843, 33.676446], [60.803193, 34.404102], [61.210817, 35.650072], [62.230651, 35.270664], [62.984662, 35.404041], [63.193538, 35.857166], [63.982896, 36.007957], [64.546479, 36.312073], [64.746105, 37.111818], [65.588948, 37.305217], [65.745631, 37.661164], [66.217385, 37.39379], [66.518607, 37.362784]]]], {"type": "Feature", "properties": {"name": ""}, "bbox": [67.44222, 36.738171, 74.980002, 40.960213], "geometry": {"type": "Polygon", "coordinates": [[[67.83, 37.144994], [68.392033, 38.157025], [68.176025, 38.901553], [67.44222, 39.140144], [67.701429, 39.580478], [68.536416, 39.533453], [69.011633, 40.086158], [69.329495, 40.727824], [70.666622, 40.960213], [70.45816, 40.496495], [70.601407, 40.218527], [71.014198, 40.244366], [70.648019, 39.935754], [69.55961, 40.103211], [69.464887, 39.526683], [70.549162, 39.604198], [71.784694, 39.279463], [73.675379, 39.431237], [73.928852, 38.505815], [74.257514, 38.606507], [74.864816, 38.378846], [74.829986, 37.990007], [74.980002, 37.41999], [73.948696, 37.421566], [73.260056, 37.495257], [72.63689, 37.047558], [72.193041, 36.948288], [71.844638, 36.738171], [71.448693, 36.065645], [71.541918, 37.905774], [71.239404, 37.953265], [71.348131, 38.258905], [70.806821, 38.486282], [70.376304, 38.138396], [70.270574, 37.735165], [70.116578, 37.588223], [69.518785, 37.608997], [69.196273, 37.151144], [68.859446, 37.344336], [68.135562, 37.023115], [67.83, 37.144994]]]], {"type": "Feature", "properties": {"name": ""}, "bbox": [69.464887, 39.279463, 80.25999, 43.298339], "geometry": {"type": "Polygon", "coordinates": [[[70.962315, 42.266154], [71.186281, 42.704293], [71.844638, 42.845395], [73.489758, 42.500894], [73.645304, 43.091272], [74.212866, 43.298339], [75.636965, 42.8779], [76.000354, 42.988022], [77.658392, 42.960686], [79.142177, 42.856092], [79.643645, 42.496683], [80.25999, 42.349999], [80.11943, 42.123941], [78.543661, 41.582243], [78.187197, 41.185316], [76.904484, 41.066486], [76.526368, 40.427946], [75.467828, 40.562072], [74.776862, 40.366425], [73.822244, 39.93973], [73.960013, 39.660008], [73.675379, 39.431237], [71.784694, 39.279463], [70.549162, 39.604198], [69.464887, 39.526683], [69.55961, 40.103211], [70.648019, 39.935754], [71.014198, 40.244366], [71.784875, 40.145844], [73.055417, 42.988022], [71.870115, 41.3929], [71.157859, 41.143587], [70.420022, 41.519998], [71.259248, 42.167711], [70.962315, 42.266154]]]], {"type": "Feature", "properties": {"name": ""}, "bbox": [52.50246, 35.270664, 66.54615, 42.751551], "geometry": {"type": "Polygon", "coordinates": [[

[illegible]

23],[26.294602,40.936261],[26.604196,41.562115],[26.117042,41.826905]]]]},{ "type": "Feature", "properties": {"name": "bbox": [19.304486,39.624998,21.02004,42.688247], "geometry": {"type": "Polygon", "coordinates": [[[21.02004,40.842727],[20.99999,40.580004],[20.674997,40.435],[20.615,40.110007],[20.150016,39.624998],[19.98,39.694993],[19.960002,39.915006],[19.406082,40.250773],[19.319059,40.72723],[19.40355,41.409566],[19.540027,41.719986],[19.371769,41.877548],[19.371768,41.877551],[19.304486,42.195745],[19.738051,42.688247],[19.801613,42.500093],[20.0707,42.58863],[20.283755,42.32026],[20.52295,42.21787],[20.590247,41.855409],[20.590247,41.855404],[20.463175,41.515089],[20.605182,41.086226],[21.020004,40.842727]]]}}, {"type": "Feature", "properties": {"name": "", "bbox": [13.656976,42.479991,19.390476,46.503751], "geometry": {"type": "Polygon", "coordinates": [[[16.564808,46.503751],[16.882515,46.380632],[17.630066,45.951769],[18.456062,45.759481],[18.829825,45.908872],[19.072769,45.521511],[19.390476,45.236516],[19.005485,44.860234],[18.553214,45.08159],[17.861783,45.067774],[17.002146,45.233777],[16.534939,45.211608],[16.318157,45.004127],[15.959367,45.233777],[15.750026,44.818712],[16.23966,44.351143],[16.456443,44.04124],[16.916156,43.667722],[17.297373,43.446341],[17.674922,43.028563],[18.56,42.65],[18.450017,42.479992],[18.450016,42.479991],[17.50997,42.849995],[16.930006,43.209998],[16.015385,43.507215],[15.174454,44.243191],[15.37625,44.317915],[14.920309,44.738484],[14.901602,45.07606],[14.258748,45.233777],[13.952255,44.802124],[13.656976,45.136935],[13.679403,45.484149],[13.71506,45.500324],[14.411968,45.466166],[14.595109,45.634941],[14.935244,45.471695],[15.327675,45.452316],[15.323954,45.731783],[15.67153,45.834154],[15.768733,46.238108],[16.564808,46.503751]]]}}, {"type": "Feature", "properties": {"name": "", "bbox": [6.022609,45.776948,10.442701,47.830828], "geometry": {"type": "Polygon", "coordinates": [[[9.594226,47.525058],[9.632932,47.347601],[9.47997,47.10281],[9.932448,46.920728],[10.442701,46.893546],[10.363378,46.483571],[9.922837,46.314899],[9.182882,46.440215],[8.966306,46.036932],[8.489952,46.005151],[8.31663,46.163642],[7.755992,45.82449],[7.273851,45.776948],[6.843593,45.991147],[6.5001,46.429673],[6.022609,46.27299],[6.037389,46.725779],[6.768714,47.287708],[6.736571,47.541801],[7.192202,47.449766],[7.466759,47.620582],[8.317301,47.61358],[8.522612,47.830828],[9.594226,47.525058]]]}}, {"type": "Feature", "properties": {"name": "", "bbox": [5.674052,49.442667,6.242751,50.128052], "geometry": {"type": "Polygon", "coordinates": [[[6.043073,50.128052],[6.242751,49.902226],[6.186322,49.463803],[5.897759,49.442667],[5.674052,49.529484],[5.782417,50.090328],[6.043073,50.128052]]]}}, {"type": "Feature", "properties": {"name": "", "bbox": [2.513573,49.529484,6.156658,51.475024], "geometry": {"type": "Polygon", "coordinates": [[[6.156658,50.803721],[6.043073,50.128052],[5.782417,50.090328],[5.674052,49.529484],[4.799222,49.985373],[4.286023,49.907497],[3.588184,50.378992],[3.123252,50.780363],[2.658422,50.796848],[2.513573,51.148506],[3.314971,51.345781],[3.315011,51.345777],[3.314971,51.345755],[4.047071,51.267259],[4.973991,51.475024],[5.606976,51.037298],[6.156658,50.803721]]]}}, {"type": "Feature", "properties": {"name": "", "bbox": [3.314971,50.803721,7.092053,53.510403], "geometry": {"type": "Polygon", "coordinates": [[[6.90514,53.482162],[7.092053,53.144043],[6.84287,52.22844],[6.589397,51.852029],[5.988658,51.85161],[6.156658,50.803721],[5.606976,51.037298],[4.973991,51.475024],[4.047071,51.267259],[3.314971,51.345755],[3.315011,51.345777],[3.830289,51.620545],[4.705997,53.091798],[6.074183,53.510403],[6.90514,53.482162]]]}}, {"type": "Feature", "properties": {"name": "", "bbox": [-9.526571,36.838269,-6.389088,42.280469], "geometry": {"type": "Polygon", "coordinates": [[[-9.034818,41.880571],[-8.671946,42.134689],[-8.263857,42.280469],[-8.013175,41.790886],[-7.422513,41.792075],[-7.251309,41.918346],[-6.668606,41.883387],[-6.389088,41.381815],[-6.851127,41.11083],[6.86402,40.330872],[-7.026413,40.184524],[-7.066592,39.711892],[-7.498632,39.629571],[-7.098037,39.030073],[-7.374092,38.373059],[-7.029281,38.375764],[-7.166508,37.803894],[-7.537105,37.428904],[-7.453726,37.097788],[-7.855613,36.838269],[-8.382816,36.97888],[-8.898857,36.868809],[-8.746101,37.651346],[-8.839998,38.266243],[-9.287464,38.358466],[-9.526571,38.737429],[-9.446989,39.392066],[-9.048305,39.755093],[-8.977353,40.159306],[-8.768684,40.760634],[-8.790853,41.184334],[-8.990789,41.543459],[-9.034818,41.880571]]]}}, {"type": "Feature", "properties": {"name": "", "bbox": [-9.392884,35.94685,3.039484,43.748338], "geometry": {"type": "Polygon", "coordinates": [[[-7.453726,37.097788],[-7.537105,37.428904],[-7.166508,37.803894],[-7.029281,38.075764],[-7.374092,38.373059],[-7.098037,39.030073],[-7.498632,39.629571],[-7.066592,39.711892],[-7.026413,40.184524],[-6.86402,40.330872],[-6.851127,41.11083],[-6.389088,41.381815],[-6.668606,41.883387],[-7.251309,41.918346],[-7.422513,41.792075],[-8.013175,41.790886],[-8.263857,42.280469],[-8.671946,42.134689],[-9.034818,41.880571],[-8.984433,42.592775],[-9.392884,43.026625],[-7.97819,43.748338],[-6.754492,43.567909],[-5.411886,43.57424],[4.347843,43.403449],[-3.517532,43.455901],[-1.901351,43.422802],[-1.502771,43.034014],[0.338047,42.579546],[0.701591,42.795734],[1.826793,42.343385],[2.985999,42.473015],[3.039484,41.89212],[2.091842,41.226089],[0.810525,41.014732],[0.721331,40.678318],[0.106692,40.123934],[-0.27871,39.309978],[0.111291,38.738514],[-0.467124,38.292366],[-0.683389,37.642354],[-1.438382,37.443064],[-2.146453,36.674144],[-3.415781,36.6589],[-4.368901,36.677839],[-4.995219,36.324708],[-5.37716,35.94685],[-5.866432,36.029817],[-6.236694,36.367677],[-6.520191,36.942913],[-7.453726,37.097788]]]}}, {"type": "Feature", "properties": {"name": "", "bbox": [-9.97086,51.669301,-6.032985,55.131622], "geometry": {"type": "Polygon", "coordinates": [[[-6.197885,53.867565],[-6.032985,53.153164],[-6.788857,52.260118],[-8.561617,51.669301],[-9.977086,51.820455],[-9.166283,52.864629],[-9.688525,53.881363],[-8.327987,54.664519],[-7.512168,55.131622],[-7.366031,54.595841],[-7.572168,54.059956],[-6.95373,54.073702],[-6.197885,53.867565]]]}}, {"type": "Feature", "properties": {"name": "", "bbox": [164.029606,-22.399976,167.120011,-20.105646], "geometry": {"type": "Polygon", "coordinates": [[[-165.77999,-21.080005],[166.599991,-21.700019],[167.120011,-22.159991],[166.740035,-22.399976],[166.189732,-22.129708],[165.474375,-21.679607],[164.829815,-21.14982],[164.162011,-20.444747],[164.029606,-20.105646],[164.459967,-20.120012],[165.020036,-20.459991],[165.460009,-20.800022],[165.77999,-21.080005]]]}}, {"type": "Feature", "properties": {"name": "", "bbox": [156.491358,-6.599338], "geometry": {"type": "MultiPolygon", "coordinates": [[[[162.119025,-10.482719],[162.398646,-10.826367],[161.700032,-10.820011],[161.319797,-10.204751],[161.917383,-10.446701],[162.119025,-10.482719]]],[[161.679982,-9.599982],[161.529397,-9.784312],[160.788253,-8.917543],[160.579997,-8.320009],[160.920028,-8.320009],[161.280006,-9.120011],[161.679982,-9.599982]]],[[160.852229,-9.872937],[160.462588,-9.89521],[159.849447,-9.794027],[159.640003,-9.63998],[159.702945,-9.24295],[160.362956,-9.40003],[160.688518,-9.610162],[160.852229,-9.872937]]],[[159.640003,-8.020027],[159.875027,-8.33732],[159.917402,-8.53829],[159.133677,-8.114181],[158.586114,-7.754824],[158.21115,-7.421872],[158.359978,-7.320018],[158.820001,-7.560003],[159.640003,-8.020027]]],[[157.14,-7.021638],[157.538426,-7.34782],[157.33942,-7.404767],[156.90203,-7.176874],[156.491358,-6.765943],[156.542828,-6.599338],[157.14,-7.021638]]]}}, {"type": "Feature", "properties": {"name": "", "bbox": [509144,-46.641235,178.517094,-34.450662], "geometry": {"type": "MultiPolygon", "coordinates": [[[[176.885824,-40.065978],[176.508017,-40.604808],[176.01244,-41.289624],[175.239567,-41.688308],[175.067898,-41.425895],[174.650973,-41.281821],[175.22763,-40.459236],[174.900157,-39.908933],[173.824047,-39.508854],[173.852262,-39.146602],[174.574802,-38.797683],[174.743474,-38.027808],[174.697017,-37.381129],[174.292028,-36.711092],[174.319004,-36.534824],[173.840997,-36.121981],[173.054171,-35.237125],[172.636005,-34.529107],[173.007042,-34.450662],[173.551298,-35.006183],[174.32939,-35.265496],[174.612009,-36.156397],[175.336616,-37.209098],[175.357596,-36.526194],[175.808887,-36.798942],[175.95849,-37.555382],[176.763195,-37.881253],[177.438813,-37.961248],[178.010354,-37.579825],[178.517094,-37.695373],[178.274731,-38.582813],[177.97046,-39.166343],[177.206993,-39.145776],[176.939981,-39.449736],[177.032946,-39.879943],[176.885824,-40.065978]]],[[169.667815,-43.555326],[170.52492,-43.031688],[171.12509,-42.512754],[171.569714,-41.767424],[171.948709,-41.514417],[172.097227,-40.956104],[172.79858,-40.493962],[173.020375,-40.919025],[173.247234,-41.331999],[173.958405,-40.926701],[174.247587,-41.349155],[174.248517,-41.770008],[173.876447,-42.233184],[173.22274,-42.970038],[172.711246,-43.372288],[173.080113,-43.853344],[172.308584,-43.865694],[171.452925,-44.242519],[171.185138,-44.897104],[170.616697,-45.908929],[169.831422,-46.355775],[169.332331,-46.641235],[168.411354,-46.619945],[167.763745,-46.290197],[166.676886,-46.219917],[166.509144,-45.852705],[167.046424,-45.110941],[168.303763,-44.123973],[168.949409,-43.935819],[169.667815,-43.555326]]]]]}}, {"type": "Feature", "properties": {"name": "", "bbox": [113.338953,-43.634597,153.569469,-10.668186], "geometry": {"type": "MultiPolygon", "coordinates": [[[[147.689259,-40.808258],[148.289068,-40.875438],[148.359865,-42.062445],[148.017301,-42.407024],[147.914052,-43.211522],[147.564564,-42.937689],[146.870343,-43.634597],[146.663327,-43.580854],[146.048378,-43.549745],[145.43193,-42.693776],[145.29509,-42.03361],[144.718071,-41.162552],[144.743755,-40.703975],[145.397978,-40.792549],[146.364121,-41.137695],[146.908584,-41.000546],[147.689259,-40.808258]]],[[126.148714,-32.215966],[125.088623,-32.215966]]]]]}}, {"type": "Feature", "properties": {"name": "", "bbox": [113.338953,-43.634597,153.569469,-10.668186], "geometry": {"type": "MultiPolygon", "coordinates": [[[[147.689259,-40.808258],[148.289068,-40.875438],[148.359865,-42.062445],[148.017301,-42.407024],[147.914052,-43.211522],[147.564564,-42.937689],[146.870343,-43.634597],[146.663327,-43.580854],[146.048378,-43.549745],[145.43193,-42.693776],[145.29509,-42.03361],[144.718071,-41.162552],[144.743755,-40.703975],[145.397978,-40.792549],[146.364121,-41.137695],[146.908584,-41.000546],[147.689259,-40.808258]]],[[126.148714,-32.215966],[125.088623,-32.215966]]]]]}}, {"type": "Feature", "properties": {"name": "", "bbox": [113.338953,-43.634597,153.569469,-10.668186], "geometry": {"type": "MultiPolygon", "coordinates": [[[[147.689259,-40.808258],[148.289068,-40.875438],[148.359865,-42.062445],[148.017301,-42.407024],[147.914052,-43.211522],[147.564564,-42.937689],[146.870343,-43.634597],[146.663327,-43.580854],[146.048378,-43.549745],[145.43193,-42.693776],[145.29509,-42.03361],[144.718071,-41.162552],[144.743755,-40.703975],[145.397978,-40.792549],[146.364121,-41.137695],[146.908584,-41.000546],[147.689259,-40.808258]]],[[126.148714,-32.215966],[125.088623,-32.215966]]]]]}}, {"type": "Feature", "properties": {"name": "", "bbox": [113.338953,-43.634597,153.569469,-10.668186], "geometry": {"type": "MultiPolygon", "coordinates": [[[[147.689259,-40.808258],[148.289068,-40.875438],[148.359865,-42.062445],[148.017301,-42.407024],[147.914052,-43.211522],[147.564564,-42.937689],[146.870343,-43.634597],[146.663327,-43.580854],[146.048378,-43.549745],[145.43193,-42.693776],[145.29509,-42.03361],[144.718071,-41.162552],[144.743755,-40.703975],[145.397978,-40.792549],[146.364121,-41.137695],[146.908584,-41.000546],[147.689259,-40.808258]]],[[126.148714,-32.215966],[125.088623,-32.215966]]]]]}}, {"type": "Feature", "properties": {"name": "", "bbox": [113.338953,-43.634597,153.569469,-10.668186], "geometry": {"type": "MultiPolygon", "coordinates": [[[[147.689259,-40.808258],[148.289068,-40.875438],[148.359865,-42.062445],[148.017301,-42.407024],[147.914052,-43.211522],[147.564564,-42.937689],[146.870343,-43.634597],[146.663327,-43.580854],[146.048378,-43.549745],[145.43193,-42.693776],[145.29509,-42.03361],[144.718071,-41.162552],[144.743755,-40.703975],[145.397978,-40.792549],[146.364121,-41.137695],[146.908584,-41.000546],[147.689259,-40.808258]]],[[126.148714,-32.215966],[125.088623,-32.215966]]]]]}}, {"type": "Feature", "properties": {"name": "", "bbox": [113.338953,-43.634597,153.569469,-10.668186], "geometry": {"type": "MultiPolygon", "coordinates": [[[[147.689259,-40.808258],[148.289068,-40.875438],[148.359865,-42.062445],[148.017301,-42.407024],[147.914052,-43.211522],[147.564564,-42.937689],[146.870343,-43.634597],[146.663327,-43.580854],[146.048378,-43.549745],[145.43193,-42.693776],[145.29509,-42.03361],[144.718071,-41.162552],[144.743755,-40.703975],[145.397978,-40.792549],[146.364121,-41.137695],[146.908584,-41.000546],[147.689259,-40.808258]]],[[126.148714,-32.215966],[125.088623,-32.215966]]]]]}}, {"type": "Feature", "properties": {"name": "", "bbox": [113.338953,-43.634597,153.569469,-10.668186], "geometry": {"type": "MultiPolygon", "coordinates": [[[[147.689259,-40.808258],[148.289068,-40.875438],[148.359865,-42.062445],[148.017301,-42.407024],[147.914052,-43.211522],[147.564564,-42.937689],[146.870343,-43.634597],[146.663327,-43.580854],[146.048378,-43.549745],[145.43193,-42.693776],[145.29509,-42.03361],[144.718071,-41.162552],[144.743755,-40.703975],[145.397978,-40.792549],[146.364121,-41.137695],[146.908584,-41.000546],[147.689259,-40.808258]]],[[126.148714,-32.215966],[125.088623,-32.215966]]]]]}}, {"type": "Feature", "properties": {"name": "", "bbox": [113.338953,-43.634597,153.569469,-10.668186], "geometry": {"type": "MultiPolygon", "coordinates": [[[[147.689259,-40.808258],[148.289068,-40.875438],[148.359865,-42.062445],[148.017301,-42.407024],[147.914052,-43.211522],[147.564564,-42.937689],[146.870343,-43.634597],[146.663327,-43.580854],[146.048378,-43.549745],[145.43193,-42.693776],[145.29509,-42.03361],[144.718071,-41.162552],[144.743755,-40.703975],[145.397978,-40.792549],[146.364121,-41.137695],[146.908584,-41.000546],[147.689259,-40.808258]]],[[126.148714,-32.215966],[125.088623,-32.215966]]]]]}}, {"type": "Feature", "properties": {"name": "", "bbox": [113.338953,-43.634597,153.569469,-10.668186], "geometry": {"type": "MultiPolygon", "coordinates": [[[[147.689259,-40.808258],[148.289068,-40.875438],[148.359865,-42.062445],[148.017301,-42.407024],[147.914052,-43.211522],[147.564564,-42.937689],[146.870343,-43.634597],[146.663327,-43.580854],[146.048378,-43.549745],[145.43193,-42.693776],[145.29509,-42.03361],[144.718071,-41.162552],[144.743755,-40.703975],[145.397978,-40.792549],[146.364121,-41.137695],[146.908584,-41.000546],[147.689259,-40.808258]]],[[126.148714,-32.215966],[125.088623,-32.215966]]]]]}}, {"type": "Feature", "properties": {"name": "", "bbox": [113.338953,-43.634597,153.569469,-10.668186], "geometry": {"type": "MultiPolygon", "coordinates": [[[[147.689259,-40.808258],[148.289068,-40.875438],[148.359865,-42.062445],[148.017301,-42.407024],[147.914052,-43.211522],[147.564564,-42.937689],[146.870343,-43.634597],[146.663327,-43.580854],[146.048378,-43.549745],[145.43193,-42.693776],[145.29509,-42.03361],[144.718071,-41.162552],[144.743755,-40.703975],[145.397978,-40.792549],[146.364121,-41.137695],[146.908584,-41.000546],[147.689259,-40.808258]]],[[126.148714,-32.215966],[125.088623,-32.215966]]]]]}}, {"type":

728751], [124.221648, -32.959487], [124.028947, -33.483847], [123.659667, -33.890179], [122.811036, -33.914467], [122.183064, -34.003402], [121.299191, -33.821036], [120.580268, -33.930177], [119.893695, -33.976065], [119.298899, -34.509366], [119.007341, -34.464149], [118.505718, -34.746819], [118.024972, -35.064733], [117.295507, -35.025459], [116.625109, -35.025097], [115.564347, -34.386428], [115.026809, -34.196517], [115.048616, -33.623425], [115.545123, -33.487258], [115.714674, -33.259572], [115.679379, -32.900369], [115.801645, -32.205062], [115.689611, -31.612437], [115.160909, -30.601594], [114.997043, -30.030725], [115.040038, -29.461095], [114.641974, -28.810231], [114.616498, -28.516399], [114.173579, -28.118077], [114.048884, -27.334765], [113.477498, -26.543134], [113.338953, -26.116545], [113.778358, -26.549025], [113.440962, -25.621278], [113.936901, -25.911235], [114.232852, -26.298446], [114.216161, -25.786281], [113.721255, -24.998939], [113.625344, -24.683971], [113.393523, -24.384764], [113.502044, -23.80635], [113.706993, -23.560215], [113.843418, -23.059987], [113.736552, -22.475475], [114.149756, -21.755881], [114.225307, -22.517488], [114.647762, -21.82952], [115.460167, -21.495173], [115.947373, -21.068688], [116.711615, -20.701682], [117.166316, -20.623599], [117.441545, -20.746899], [118.229559, -20.374208], [118.836085, -20.263311], [118.987807, -20.044203], [119.252494, -19.952942], [119.805225, -19.976506], [120.85622, -19.683708], [121.399856, -19.239756], [121.655138, -18.705318], [122.241665, -18.197649], [122.286624, -17.798603], [122.312772, -17.254967], [123.012574, -16.4052], [123.433789, -17.268558], [123.859345, -17.069035], [123.503242, -16.596506], [123.817073, -16.111316], [124.258287, -16.327944], [124.379726, -15.56706], [124.926153, -15.0751], [125.167275, -14.680396], [125.670087, -14.51007], [125.685796, -14.230656], [126.125149, -14.347341], [126.142823, -14.095987], [126.582589, -13.952791], [127.065867, -13.817968], [127.804633, -14.276906], [128.35969, -14.86917], [128.985543, -14.875991], [129.621473, -14.969784], [129.4096, -14.42067], [129.888641, -13.618703], [130.339466, -13.357376], [130.183506, -13.10752], [130.617795, -12.536392], [131.223495, -12.183649], [131.735091, -12.302453], [132.575298, -12.114041], [132.557212, -11.603012], [131.824698, -11.273782], [132.357224, -11.128519], [133.019561, -11.376411], [133.550846, -11.786515], [134.393068, -12.042365], [134.678632, -11.941183], [135.298491, -12.248606], [135.882693, -11.962267], [136.265381, -12.049342], [136.492475, -11.857209], [136.95162, -12.351959], [136.685125, -12.887223], [136.305407, -13.29123], [135.961758, -13.324509], [136.077617, -13.724278], [135.783836, -14.223989], [135.428664, -14.715432], [135.500184, -14.997741], [136.295175, -15.550265], [137.06536, -15.870762], [137.580471, -16.215082], [138.303217, -16.807604], [138.585164, -16.806622], [139.108543, -17.062679], [139.260575, -17.371601], [140.215245, -17.710805], [140.875463, -17.369069], [141.07111, -16.832047], [141.274095, -16.38887], [141.398222, -15.840532], [141.702183, -15.044921], [141.56338, -14.561333], [141.63552, -14.270395], [141.519869, -13.698078], [141.65092, -12.944688], [141.842691, -12.741548], [141.68699, -12.407614], [141.928629, -11.877466], [142.118488, -11.328042], [142.143706, -11.042737], [142.51526, -10.668186], [142.79731, -11.157355], [142.866763, -11.784707], [143.115947, -11.90563], [143.158632, -12.325656], [143.522124, -12.834358], [143.597158, -13.400422], [143.561811, -13.763656], [143.922099, -14.548311], [144.563714, -14.171176], [144.894908, -14.594458], [145.374724, -14.984976], [145.271991, -15.428205], [145.48526, -16.285672], [145.637033, -16.784918], [145.888904, -16.9069

26], [146.160309, -17.761655], [146.063674, -18.280073], [146.387478, -18.958274], [147.471082, -19.480723], [148.177602, -19.955939], [148.848414, -20.39121], [148.717465, -20.633469], [149.28942, -21.260511], [149.678337, -22.342512], [150.077382, -22.122784], [150.482939, -22.556142], [150.727265, -22.402405], [150.899554, -22.062237], [151.609175, -24.076256], [152.07354, -24.457887], [152.855197, -25.267501], [153.136162, -26.071173], [153.161949, -26.641319], [153.092909, -27.2603], [153.569469, -28.110067], [153.512108, -28.995077], [153.339095, -29.458202], [153.069241, -30.35024], [153.089602, -30.923642], [152.891578, -31.640446], [152.450002, -32.550003], [151.709117, -33.041342], [151.343972, -33.816023], [151.010555, -34.31036], [150.714139, -35.17346], [150.32822, -35.671879], [150.075212, -36.420206], [149.946124, -37.109052], [149.997284, -37.425261], [149.423882, -37.772681], [148.304622, -37.809061], [147.381733, -38.219217], [146.922123, -38.606532], [146.317922, -39.035757], [145.489652, -38.593768], [144.876976, -38.417448], [145.032212, -37.896188], [144.485682, -38.085324], [143.609974, -38.809465], [142.754427, -38.538268], [142.17833, -38.380034], [141.606582, -38.308514], [140.638579, -38.019333], [139.992158, -37.402936], [139.806588, -36.643603], [139.574148, -36.138362], [139.082808, -35.732754], [138.120748, -35.612296], [138.449462, -35.127261], [138.207564, -34.384723], [137.71917, -35.076825], [136.829406, -35.260535], [137.352371, -34.707339], [137.503886, -34.130268], [137.890116, -33.640479], [137.810328, -32.900007], [136.996837, -33.752771], [136.372069, -34.094766], [135.989043, -34.890118], [135.208213, -34.47867], [135.239218, -33.947953], [134.613417, -33.222778], [134.085904, -32.848072], [134.273903, -32.617234], [132.990777, -32.011224], [132.288081, -31.982647], [131.326331, -31.495803], [129.535794, -31.590423], [128.240938, -31.948489], [127.102867, -32.282267], [126.148714, -32.215966]]], [{"type": "Feature", "properties": {"name": ""}, "bbox": [79.695167, 5.96837, 81.787959, 9.824078], "geometry": {"type": "Polygon", "coordinates": [[[81.787959, 7.523055], [81.637322, 6.481775], [81.21802, 6.197141], [80.348357, 5.96837], [79.872469, 6.763463], [79.695167, 8.200843], [80.147801, 9.824078], [80.838818, 9.268427], [81.304319, 8.564206], [81.787959, 7.523055]]]}], [{"type": "Feature", "properties": {"name": ""}, "bbox": [73.675379, 18.197701, 135.026311, 53.45868], "geometry": {"type": "MultiPolygon", "coordinates": [[[109.47521, 18.197701], [108.655208, 18.507682], [108.626217, 19.367888], [109.119056, 19.821039], [110.211599, 20.101254], [110.786551, 20.077534], [111.010051, 19.69593], [110.570647, 19.255879], [110.339188, 18.678395], [109.47521, 18.197701]]], [[[80.25999, 42.349999], [80.18015, 42.920068], [80.866206, 43.180362], [79.966106, 44.917517], [81.947071, 45.317027], [82.458926, 45.53965], [83.180484, 47.330031], [85.16429, 47.000956], [85.720484, 47.47867], [85.768233, 48.455751], [86.598776, 48.549182], [87.35997, 49.214981], [87.751264, 49.297198], [88.013832, 48.599463], [88.854298, 48.069082], [90.280826, 47.693549], [90.970809, 46.888164], [90.585768, 45.719716], [90.94554, 45.286073], [92.133891, 45.115076], [93.480734, 44.975472], [94.688929, 44.352332], [95.306875, 44.241331], [95.762455, 43.319449], [96.349396, 42.725633], [97.451757, 42.74889], [99.515817, 42.524691], [100.845866, 42.663804], [101.83304, 42.514873], [103.312278, 41.907468], [104.522282, 41.908347], [104.964994, 41.59741], [106.129316, 42.134328], [107.744773, 42.481516], [109.243596, 42.519446], [110.412103, 42.871234], [111.129682, 43.406834], [111.829588, 43.743118], [111.667737, 44.073176], [111.348377, 44.457442], [111.873306, 45.102079], [112.436062, 45.011646], [113.463907, 44.808893], [114.460332, 45.339817], [115.985096, 45.727235], [116.717868, 46.388202], [117.421701, 46.672733], [118.874326, 46.805412], [119.66327, 46.69268], [119.772824, 47.048059], [118.866574, 47.74706], [118.064143, 48.06673], [117.295507, 49.697709], [116.308953, 47.85341], [115.742837, 47.726545], [115.485282, 48.135383], [116.191802, 49.134598], [116.678801, 49.888531], [117.879244, 49.510983], [119.288461, 50.142883], [119.27939, 50.58292], [120.18208, 51.64355], [120.7382, 51.96411], [120.725789, 52.516226], [120.177089, 52.753886], [121.003085, 53.251401], [122.245748, 53.431726], [123.57147, 53.4588], [125.068211, 53.161045], [125.946349, 52.792799], [126.564399, 51.784255], [126.939157, 51.353894], [127.287456, 50.739797], [127.6574, 49.76027], [129.397818, 49.4406], [130.582293, 48.729687], [130.98726, 47.79013], [132.50669, 47.78896], [133.373596, 48.183442], [135.026311, 48.47823], [134.50081, 47.57845], [134.11235, 47.21248], [133.769644, 46.116927], [133.09712, 45.14409], [131.883454, 45.321162], [131.02519, 44.96796], [131.288555, 44.11152], [131.144688, 42.92999], [130.633866, 42.903015], [130.64, 42.395024], [129.994267, 42.985387], [129.596669, 42.424982], [128.052215, 41.994285], [128.208433, 41.466772], [127.343783, 41.503152], [126.869083, 41.816569], [126.182045, 41.107336], [125.079942, 40.569824], [124.265625, 39.928493], [122.86757, 39.637788], [122.131388, 39.170452], [121.054554, 38.897471], [121.585995, 39.360854], [121.376757, 39.750261], [122.168595, 40.422443], [121.640359, 40.94639], [120.768629, 40.593388], [119.639602, 39.898056], [119.023464, 39.252333], [118.042749, 39.204274], [117.532702, 38.737636], [118.059699, 38.061476], [118.87815, 37.897325], [118.911636, 37.448464], [119.702802, 37.156389], [120.823457, 37.870428], [121.711259, 37.481123], [122.357937, 37.454484], [122.519995, 36.930614], [121.104164, 36.651329], [120.637009, 36.11144], [119.664562, 35.609791], [119.151208, 34.909859], [120.227525, 34.360332], [120.620369, 33.376723], [121.229014, 32.460319], [121.908146, 31.692174], [121.891919, 30.949352], [121.264257, 30.676267], [121.503519, 30.142915], [122.092114, 29.83252], [121.938428, 29.018022], [121.684439, 28.225513], [121.125661, 28.135673], [120.395473, 27.053207], [119.585497, 25.740781], [118.656871, 24.547391], [117.281606, 23.624501], [115.890735, 22.782873], [114.763827, 22.668074], [114.152547, 22.22376], [113.80678, 22.54834], [113.241078, 22.051367], [111.843592, 21.550494], [110.785466, 21.397144], [110.444039, 20.341033], [109.889861, 20.282457], [109.627655, 21.008227], [109.864488, 21.395051], [108.522813, 21.715212], [108.05018, 21.55238], [107.04342, 21.811899], [106.567273, 22.218205], [106.725403, 22.794268], [105.811247, 22.976892], [105.329209, 23.352063], [104.476858, 22.81915], [103.504515, 22.703757], [102.706992, 22.708795], [102.170436, 22.464753], [101.652018, 22.318199], [101.80312, 21.174367], [101.270026, 21.201652], [101.180005, 21.436573], [101.150033, 21.849984], [100.416538, 21.558839], [99.983489, 21.742937], [99.240899, 22.118314], [99.531992, 22.949039], [98.898749, 23.142722], [98.660262, 24.063286], [97.60472, 23.897405], [97.724609, 25.083637], [98.671838, 25.918703], [98.712094, 26.743536], [98.68269, 27.508812], [98.246231, 27.747221], [97.911988, 28.335945], [97.327114, 28.261583], [96.248833, 28.411031], [9

6.586591,28.83098],[96.117679,29.452802],[95.404802,29.031717],[94.56599,29.277438],[93.413348,28.640629],[92.503119,27.896876],[91.696657,27.771742],[91.258854,28.040614],[90.730514,28.064954],[90.015829,28.296439],[89.47581,28.042759],[88.814248,27.299316],[88.730326,28.086865],[88.120441,27.876542],[86.954517,27.974262],[85.82332,28.203576],[85.011638,28.642774],[84.23458,28.839894],[83.898993,29.320226],[83.337115,29.463732],[82.327513,30.115268],[81.525804,30.422717],[81.111256,30.183481],[79.721367,30.882715],[78.738894,31.515906],[78.458446,32.618164],[79.176129,32.48378],[79.208892,32.994395],[78.811086,33.506198],[78.912269,34.321936],[77.837451,35.49401],[76.192848,35.898403],[75.896897,36.666806],[75.158028,37.133031],[74.980002,37.41999],[74.829986,37.990007],[74.864816,38.378846],[74.257514,38.606507],[73.928852,38.505815],[73.675379,39.431237],[73.960013,39.660008],[73.822244,39.893973],[74.776862,40.366425],[75.467828,40.562072],[76.526368,40.427946],[76.904484,41.066486],[78.187197,41.185316],[78.543661,41.582243],[80.11943,42.123941],[80.25999,42.349999]]]]],{"type":"Feature","properties":{"name":"","","bbox":[120.106189,21.970571,121.951244,25.295459]}},"geometry":{"type":"Polygon","coordinates":[[[121.777818,24.394274],[121.175632,22.790857],[120.74708,21.970571],[120.220083,22.814861],[120.106189,23.556263],[120.69468,24.538451],[121.495044,25.295459],[121.951244,24.997596],[121.777818,24.394274]]]]},{"type":"Feature","properties":{"name":"","","bbox":[6.749955,36.619987,18.480247,47.115393]}},"geometry":{"type":"MultiPolygon","coordinates":[[[10.442701,46.893546],[11.048556,46.751359],[11.164828,46.941579],[12.153088,47.115393],[12.376485,46.767559],[13.806475,46.509306],[13.69811,46.016778],[13.93763,45.591016],[13.141606,45.736692],[12.328581,45.381778],[12.383875,44.885374],[12.261453,44.600482],[12.589237,44.091366],[13.526906,43.587727],[14.029821,42.761008],[15.14257,41.95514],[15.926191,41.961315],[16.169897,41.740295],[15.889346,41.541082],[16.785002,41.179606],[17.519169,40.877143],[18.376687,40.355625],[18.480247,40.168866],[18.293385,39.810774],[17.73838,40.277671],[16.869596,40.442235],[16.448743,39.795401],[17.17149,39.4247],[17.052841,38.902871],[16.635088,38.843572],[16.100961,37.985899],[15.684087,37.908849],[15.687963,38.214593],[15.891981,38.750942],[16.109332,38.964547],[15.718814,39.544072],[15.413613,40.048357],[14.998496,40.172949],[14.703268,40.60455],[14.060672,40.786348],[13.627985,41.188287],[12.888082,41.25309],[12.106683,41.704535],[11.191906,42.355425],[10.511948,42.931463],[10.200029,43.920007],[9.702488,44.036279],[8.888946,44.366336],[8.428561,44.231228],[7.850767,43.767148],[7.435185,43.693845],[7.549596,44.127901],[7.007562,44.254767],[6.749955,45.028518],[7.096652,45.333099],[6.802355,45.70858],[6.843593,45.991147],[7.273851,45.776948],[7.755992,45.82449],[8.31663,46.163642],[8.489952,46.005151],[8.966306,46.036937],[9.182882,46.440215],[9.922837,46.314899],[10.363378,46.483571],[10.442701,46.893546]]]]],[[14.761249,38.143874],[14.709991,40.899984],[2.010012,41.209991],[9.809975,40.500009],[9.669519,39.177376],[9.214818,39.240473],[8.806936,38.906618],[8.428302,39.171847],[8.388253,40.378311],[8.159998,40.950007],[8.709991,40.899984]]]]],{"type":"Feature","properties":{"name":"","","bbox":[8.089977,54.800015,12.690006,57.730017]}},"geometry":{"type":"MultiPolygon","coordinates":[[[9.921906,54.983104],[9.282049,54.830865],[8.526229,54.962744],[8.120311,55.517723],[8.089977,56.540012],[8.256582,56.809969],[8.543438,57.110003],[9.424469,57.172066],[9.775559,57.447941],[10.580006,57.730017],[10.546106,57.215733],[10.25,56.890016],[10.369993,56.609982],[10.912182,56.458621],[10.667804,56.081383],[10.369993,56.190007],[9.649985,55.469999],[9.921906,54.983104]]]]],[[12.370904,56.111407],[12.690006,55.609991],[12.089991,54.800015],[10.43543,55.364864],[10.903914,55.779955],[12.370904,56.111407]]]]],{"type":"Feature","properties":{"name":"","","bbox":[-7.572168,49.96,1.681531,58.635]}},"geometry":{"type":"MultiPolygon","coordinates":[[[[-6.197885,53.867565],[-6.95373,54.073702],[-7.572168,54.059956],[-7.366031,54.595841],[-7.572168,55.131622],[-6.733847,55.172861],[-5.661949,54.546003],[-6.197885,53.867565]]]]],[[-3.093831,53.404547],[-3.09208,53.404441],[-2.945009,53.985],[-3.614701,54.600937],[-3.630005,54.615013],[-4.844169,54.790971],[-5.082527,55.061601],[-4.719112,55.508473],[-5.047981,55.783986],[-5.586398,55.311146],[-5.644999,56.275015],[-6.149981,56.78501],[-5.786825,57.818848],[-5.009999,58.630013],[-4.211495,58.550845],[-3.005005,58.635],[-4.073828,57.553025],[-3.055002,57.690019],[-1.959281,57.6848],[-2.219988,56.870017],[-3.119003,55.973793],[-2.085609,55.909998],[-2.005676,55.804903],[-1.114991,54.624986],[-0.430485,54.464376],[0.184981,53.325014],[0.469977,52.929999],[1.681531,52.73952],[1.559988,52.099998],[1.050562,51.806761],[1.449865,51.289428],[0.550334,50.765739],[-0.787517,50.774989],[-2.489998,50.500019],[-2.956274,50.69688],[3.617448,50.228356],[-4.542508,50.341837],[-5.245023,49.96],[-5.776567,50.159678],[-4.30999,51.210001],[-3.414851,51.426009],[-3.422719,51.426848],[-4.984367,51.593466],[-5.267296,51.9914],[-4.222347,52.301356],[-4.770013,52.840005],[-4.579999,53.495004],[-3.093831,53.404547]]]]]]],{"type":"Feature","properties":{"name":"","","bbox":[-24.326184,63.496383,-13.609732,66.526792]}},"geometry":{"type":"Polygon","coordinates":[[[-14.508695,66.455892],[-14.739637,65.808748],[-13.609732,66.526792],[-14.909834,64.364082],[-17.794438,63.678749],[-18.656246,63.496383],[-19.972755,63.643635],[-22.762972,63.960179],[-21.778484,64.402116],[-23.955044,64.89113],[22.184403,65.084968],[22.227423,65.378594],[24.326184,65.611189],[23.650515,66.262519],[22.134922,66.410469],[20.576284,65.732112],[-19.056842,66.276601],[-17.798624,65.993853],[-16.167819,66.526792],[-14.508695,66.455892]]]]],{"type":"Feature","properties":{"name":"","","bbox":[44.79399,38.270378,50.392821,41.860675]}},"geometry":{"type":"MultiPolygon","coordinates":[[[46.404951,41.860675],[46.686071,41.827137],[47.373315,41.219732],[47.815666,41.151416],[47.987283,41.405819],[48.584353,41.808869],[49.110264,41.282287],[49.618915,40.572924],[50.08483,40.526157],[50.392821,40.256561],[49.569202,40.176101],[49.395259,39.399482],[49.223228,39.049219],[48.856532,38.815486],[48.883249,38.320245],[48.634375,38.270378],[48.010744,38.794015],[48.355529,39.288765],[48.060095,39.582235],[47.685079,39.508364],[46.50572,38.770605],[46.483499,39.464155],[46.034534,39.628021],[45.610012,39.899994],[45.891907,40.218476],[45.359175,40.561504],[45.560351,40.81229],[45.179496,40.985354],[44.97248,41.248129],[45.217426,41.411452],[45.962601,41.123873],[46.501637,41.064445],[46.637908,41.181673],[46.145432,41.722802],[46.404951,41.860675]]]]],[[46.143623,38.741201],[45.457722,38.874139],[44.952688,39.335765],[44.79399,39.713003],[45.001987,39.740004],[45.298145,39.471751],[45.739978,39.473999],[45.735379,39.319719],[46.143623,38.741201]]]]],{"type":"Feature","properties":{"name":"","","bbox":[39.955009,43.434998],[40.076965,43.553104],[40.92219,43.38215],[42.3944,43.2203],[43.75599,42.74083],[43.93121,42.55496],[44.537623,42.711993],[45.470279,42.502781],[45.7764,42.09244],[46.404951,41.860675],[46.145432,41.722802],[46.637908,41.181673],[46.501637,41.064445],[45.962601,41.123873],[45.217426,41.411452],[44.97248,41.248129],[43.582746,41.02143],[42.619549,41.583173],[0.1554084,41.535656],[41.703171,41.962943],[41.45347,42.645123],[40.875469,43.013628],[40.321394,43.128634],[39.955009,43.434998]]]]],{"type":"Feature","properties":{"name":"","","bbox":[117.174275,8.3675],[117.664477,9.066889],[118.386914,9.6845],[118.987342,10.376292],[119.511496,11.369668],[119.689677,10.554291],[119.029458,10.093653],[118.504581,9.316383]]]]],[[122.336957,18.224883],[122.174279,17.810268],[122.515654,17.093505],[122.252311,16.262444],[121.662786,15.931018],[121.50507,15.124814],[121.728829,14.328376],[122.258925,14.218202],[122.701276,14.336541],[123.950295,13.782131],[123.855107,13.237771],[124.181289,12.997527],[124.077419,12.536677],[123.298035,13.027526],[122.928652,13.55292],[122.617135,13.185836],[122.03465,13.784482],[121.126385,13.636687],[120.628637,13.857656],[120.679384,14.271016],[120.991819,14.525393],[120.693336,14.756671],[120.564145,14.396279],[120.070429,14.970869],[119.920929,15.406347],[119.883773,16.363704],[120.286488,16.034629],[120.390047,17.599081],[120.715867,18.505227],[121.321308,18.504065],[121.937601,18.218552],[122.246006,18.47895],[122.336957,18.224883]]]]],[[122.03837,11.415841],[121.883548,11.891755],[122.48382

1,11.582187],[123.120217,11.58366],[123.100838,11.165934],[122.637714,10.741308],[122.00261,10.441017],[121.967367,10.905691],[122.03837,11.415841]]],[[125.502552,12.162695],[125.783465,11.046122],[125.011884,11.311455],[125.032761,10.975816],[125.277449,10.358722],[124.801819,10.134679],[124.760168,10.837995],[124.459101,10.88993],[124.302522,11.495371],[124.891013,11.415583],[124.87799,11.79419],[124.266762,12.557761],[125.227116,12.535721],[125.502552,12.162695]]]]},{\"type\":\"Feature\",\"properties\":{\"name\":\"\"},\"bbox\":[100.085757,0.773131,119.181904,6.928053],\"geometry\":{\"type\":\"MultiPolygon\",\"coordinates\":[[[100.085757,6.464489],[100.259596,6.642825],[101.075516,6.204867],[101.154219,5.691384],[101.814282,5.810808],[102.141187,6.221636],[102.371147,6.128205],[102.961705,5.524495],[103.381215,4.855001],[103.438575,4.181606],[103.332122,3.726698],[103.429429,3.382869],[103.502448,2.791019],[103.854674,2.515454],[104.247932,1.631141],[104.228811,1.293048],[103.519707,1.226334],[102.573615,1.967115],[101.390638,2.760814],[101.27354,3.270292],[100.695435,3.93914],[100.557408,4.76728],[100.196706,5.312493],[100.30626,6.040562],[100.085757,6.464489]]],[[117.882035,4.137551],[117.015214,4.306094],[115.865517,4.306559],[115.519078,3.169238],[115.134037,2.821482],[114.621355,1.430688],[113.80585,1.217549],[112.859809,1.49779],[112.380252,1.410121],[111.797548,0.904441],[111.159138,0.976478],[110.514061,0.773131],[109.830227,1.338136],[109.66326,2.006467],[110.396135,1.663775],[111.168853,1.850637],[111.370081,2.697303],[111.796928,2.885897],[112.995615,3.102395],[113.712935,3.893509],[114.204017,4.525874],[114.659596,4.007637],[114.869557,4.348314],[115.347461,4.316636],[115.4057,4.955228],[115.45071,5.44773],[116.220741,6.143191],[116.725103,6.924771],[117.129626,6.928053],[117.643393,6.422166],[117.689075,5.98749],[118.347691,5.708696],[119.181904,5.407836],[119.110694,5.016128],[118.439727,4.966519],[118.618321,4.478202],[117.882035,4.137551]]]]},{\"type\":\"Feature\",\"properties\":{\"name\":\"\"},\"bbox\":[114.204017,4.007637,115.45071,5.44773],\"geometry\":{\"type\":\"Polygon\",\"coordinates\":[[[115.45071,5.44773],[115.4057,4.955228],[115.347461,4.316636],[114.869557,4.348314],[114.659596,4.007637],[114.204017,4.525874],[114.599961,4.900011],[115.45071,5.44773]]]]},{\"type\":\"Feature\",\"properties\":{\"name\":\"\"},\"bbox\":[13.69811,45.452316,16.564808,46.852386],\"geometry\":{\"type\":\"Polygon\",\"coordinates\":[[[13.806475,46.509306],[14.632472,46.431817],[15.137092,46.658703],[16.011664,46.683611],[16.202298,46.852386],[16.370505,46.841327],[16.564808,46.503751],[15.768733,46.238108],[15.67153,45.834154],[15.323954,45.731783],[15.327675,45.452316],[14.935244,45.471695],[14.595109,45.634941],[14.411968,45.466166],[13.71506,45.500324],[13.93763,45.591016],[13.69811,46.016778],[13.806475,46.509306]]]]},{\"type\":\"Feature\",\"properties\":{\"name\":\"\"},\"bbox\":[20.645593,59.846373,31.516092,70.164193],\"geometry\":{\"type\":\"Polygon\",\"coordinates\":[[[28.59193,69.064777],[28.444685,64.204453],[30.035872,63.552814],[31.516092,62.867687],[31.139991,62.357693],[30.211107,61.780028],[28.07,60.50352],[28.070002,60.503519],[28.069998,60.503517],[26.255173,60.423961],[24.496624,60.057316],[22.869695,59.846373],[22.290764,60.391921],[21.322244,60.72017],[21.544866,61.705329],[21.059211,62.607393],[21.536029,63.189735],[22.442744,63.81781],[24.730512,64.902344],[25.398068,65.111427],[25.294043,65.534346],[23.903379,66.006927],[23.56588,66.396051],[23.539473,67.936009],[21.978535,68.616846],[20.645593,69.106247],[21.244936,69.370443],[22.356238,68.841741],[23.66205,68.891247],[24.735679,68.649557],[25.689213,69.092114],[26.179622,69.825299],[27.732292,70.164193],[29.015573,69.766491],[28.59193,69.064777]]]]},{\"type\":\"Feature\",\"properties\":{\"name\":\"\"},\"bbox\":[16.879983,47.758429,22.558138,49.571574],\"geometry\":{\"type\":\"Polygon\",\"coordinates\":[[[22.558138,49.085738],[22.280842,48.825392],[22.085608,48.422264],[21.872236,48.319971],[20.801294,48.623854],[20.473562,48.56285],[20.239054,48.327567],[19.769471,48.202691],[19.661364,48.266615],[19.174365,48.111379],[18.777025,48.081768],[18.696513,47.880954],[17.857133,47.758429],[17.488473,47.867466],[16.979667,48.123497],[16.879983,48.470013],[16.960288,48.596982],[17.101985,48.816969],[17.545007,48.800019],[17.886485,48.903475],[17.93512,48.996493],[18.104973,49.043983],[18.170498,49.271515],[18.399994,49.315001],[18.554971,49.495015],[18.853144,49.49623],[18.909575,49.435846],[19.320713,49.571574],[19.825023,49.217125],[20.415839,49.431453],[20.887955,49.328772],[21.607808,49.470107],[22.558138,49.085738]]]]},{\"type\":\"Feature\",\"properties\":{\"name\":\"\"},\"bbox\":[12.240111,48.555305,18.853144,51.117268],\"geometry\":{\"type\":\"Polygon\",\"coordinates\":[[[15.016996,51.106674],[15.490972,50.78473],[16.238627,50.697733],[16.176253,50.422607],[16.719476,50.215747],[16.868769,50.473974],[17.554567,50.362146],[17.649445,50.049038],[18.392914,49.988629],[18.853144,49.49623],[18.554971,49.495015],[18.399994,49.315001],[18.170498,49.271515],[18.104973,49.043983],[17.913512,48.996493],[17.886485,90.3475],[17.545007,48.800019],[17.101985,48.816969],[16.960288,48.596982],[16.499283,48.785808],[16.029647,48.738998],[15.253416,49.039074],[14.901447,48.964402],[14.338898,48.555305],[13.595946,48.877172],[13.031329,49.307068],[12.521024,49.547415],[12.415191,49.969121],[12.240111,50.266338],[12.966837,50.484076],[13.338132,50.733234],[14.056228,50.926918],[14.307013,51.117268],[14.570718,51.002339],[15.016996,51.106674]]]]},{\"type\":\"Feature\",\"properties\":{\"name\":\"\"},\"bbox\":[36.32322,12.455416,43.081226,17.998307],\"geometry\":{\"type\":\"Polygon\",\"coordinates\":[[[36.42951,14.42211],[36.32322,14.82249],[36.75389,16.29186],[36.85253,16.95655],[37.16747,17.26314],[37.904,17.42754],[38.41009,17.998307],[38.990623,16.840626],[39.26611,15.922723],[39.814294,15.435647],[41.179275,14.49108],[41.734952,13.921037],[42.276831,13.343992],[42.589576,13.000421],[43.081226,12.699639],[42.779642,12.455416],[42.35156,12.54223],[42.00975,12.86582],[41.59856,13.45209],[41.15523,13.77333],[40.8966,14.11864],[40.02625,14.51959],[39.34061,14.53155],[39.0994,14.74064],[38.51295,14.50547],[37.90607,14.95943],[37.59377,14.2131],[36.42951,14.42211]]]]},{\"type\":\"Feature\",\"properties\":{\"name\":\"\"},\"bbox\":[129.408463,31.029579,145.543137,45.551483],\"geometry\":{\"type\":\"MultiPolygon\",\"coordinates\":[[[141.884601,39.180865],[140.959489,38.174001],[140.976388,37.142074],[140.59977,36.343983],[140.774074,35.842877],[140.253279,35.138114],[138.975528,34.6676],[137.217599,34.606286],[135.792983,33.464805],[135.120983,33.849071],[135.079435,34.596545],[133.340316,34.375938],[132.156771,33.904933],[130.986145,33.885761],[132.000036,33.149992],[131.33279,31.450355],[130.686318,31.029579],[130.20242,31.418238],[130.447676,32.319475],[129.814692,32.61031],[129.408463,33.296056],[130.353935,33.604151],[130.878451,34.232743],[131.884229,34.749714],[132.617673,35.433393],[134.608301,35.731618],[135.677538,35.527134],[136.723831,37.304984],[137.390612,36.827391],[138.857602,37.827485],[139.426405,38.215962],[140.05479,39.438807],[139.883379,40.563312],[140.305783,41.195005],[141.368973,41.37856],[141.914263,39.991616],[141.884601,39.180865]]],[[144.613427,43.960883],[145.320825,44.384733],[145.543137,43.262088],[144.059662,42.988358],[143.18385,41.995215],[141.611491,42.678791],[141.067286,41.584594],[139.955106,41.569556],[139.817544,42.563759],[140.312087,43.333273],[141.380549,43.388825],[141.671952,44.772125],[141.967645,45.551483],[143.14287,45.510358],[143.910162,44.1741],[144.613427,43.960883]]],[[132.371176,33.463642],[132.924373,34.060299],[133.492968,33.944621],[133.904106,34.364931],[134.638428,34.149234],[134.766379,33.806335],[134.203416,33.201178],[133.79295,33.521985],[133.280268,33.28957],[133.014858,32.704567],[132.363115,32.989382],[132.371176,33.463642]]]]},{\"type\":\"Feature\",\"properties\":{\"name\":\"\"},\"bbox\":[-62.685057,-27.548499,-54.29296,-19.342747],\"geometry\":{\"type\":\"Polygon\",\"coordinates\":[[[-58.166392,-20.176701],[-57.870674,-20.732688],[-57.937156,-22.090176],[-56.88151,-22.282154],[-56.473317,-22.0863],[-55.797958,-22.35693],[-55.610683,-22.655619],[-55.517639,-23.571998],[-55.400747,-23.956935],[-55.027902,-24.001274],[-54.652834,-23.839578],[-54.29296,-24.021014],[-54.293476,-24.5708],[-54.428946,-25.162185],[-54.625291,-25.739255],[-54.788795,-26.621786],[-55.695846,-27.387837],[-56.486702,-27.548499],[-57.60976,-27.395899],[-58.618174,-27.123719],[-57.63366,-25.603657],[-57.777217,-25.16234],[-58.807128,-24.771459],[-60.028966,-24.032796],[-60.846565,-23.880713],[-62.685057,-22.249029],[-62.291179,-21.051635],[-62.265961,-20.513735],[-61.786326,-19.633737],[-60.043565,-19.342747],[-59.115042,-19.356906],[-58.183471,-19.868399],[-58.166392,-20.176701]]]]},{\"type\":\"Feature\",\"properties\":{\"name\":\"\"},\"bbox\":[12.58595,53.108573,19.000003],\"geometry\":{\"type\":\"Polygon\",\"coordinates\":[[[52.00001,19.000003],[52.782184,17.349742],[53.108573,16.651051],[52.385206,16.382411],[52.191729,15.938433],[52.168165,15.59742],[51.172515,15.17525],[49.574576,14.708767],[48.679231,14.003202],[48.238947,13.94809],[47.938914,14.007233],[47.354454,13.59222],[46.710706,13.396999],[45.877593,13.347764],[45.62505,13.290946],[45.406459,13.026905],[45.144356,12.953938],[44.989533,12.699587],[44.494576,12.721653],[44.175113,12.58595],[43.482959,12.6368],[43.222871,13.22095],[43.251448,13.767584],[43.087944,14.06263],[42.892245,14.802249],[42.604873,15.213335],[42.805015,15.261963],[42.702438,15.718886],[42.823671,15.911742],[42.779332,16.347891],[43.218375,16.66689],[43.115798,17.08844],[43.380794,17.579987],[43.791519,17.319977],[44.062613,17.410359],[45.216651,17.433329],[45.399999,17.333335],[46.366659,17.233315],[46.749994,17.283338],[47.000005,16.949999],[47.466695,17.116682],[48.183344,18.166669],[49.116672,18.616668],[52.00001,19.000003]]]]},{\"type\":\"Feature\",\"properties\":{\"name\":\"\"}}

"", "bbox": [34.632336, 16.347891, 55.666659, 32.161009], "geometry": {"type": "Polygon", "coordinates": [[[34.956037, 29.356555], [36.068941, 29.197495], [36.501214, 29.505254], [36.740528, 29.865283], [37.503582, 30.003776], [37.66812, 30.338665], [37.998849, 30.5085], [37.002166, 31.508413], [39.004886, 32.010217], [39.195468, 32.161009], [40.399994, 31.889992], [41.889981, 31.190009], [44.709499, 29.178891], [46.568713, 29.099025], [47.459822, 29.002519], [47.708851, 28.526063], [48.416094, 28.552004], [48.807595, 27.689628], [49.299554, 27.461218], [49.470914, 27.109999], [50.152422, 26.689663], [50.212935, 26.277027], [50.113303, 25.943972], [50.239859, 25.60805], [50.527387, 25.327808], [50.660557, 24.999896], [50.810108, 24.754743], [51.112415, 24.556331], [51.389608, 24.627386], [51.579519, 24.245497], [51.617708, 24.014219], [52.000733, 23.001154], [55.006803, 22.496948], [55.208341, 22.70833], [55.666659, 22.000001], [54.999982, 19.999994], [52.00001, 19.000003], [49.116672, 18.616668], [48.183344, 18.166669], [47.466695, 17.116682], [47.000005, 16.949999], [46.749994, 17.283338], [46.366659, 17.233315], [45.399999, 17.333335], [45.216651, 17.433329], [44.062613, 17.410359], [43.791519, 17.319977], [43.380794, 17.759987], [43.115798, 17.08844], [43.218375, 16.66689], [42.779332, 16.347891], [42.649573, 16.774635], [42.347989, 17.075806], [42.270888, 17.474722], [41.754382, 17.833046], [41.221391, 18.6716], [40.939341, 19.486485], [40.247652, 20.174635], [39.801685, 20.338862], [39.139399, 21.291905], [39.023696, 21.986875], [39.066329, 22.579656], [38.492772, 23.688451], [38.02386, 24.078686], [37.483635, 24.285495], [37.154818, 24.858483], [37.209491, 25.084542], [36.931627, 25.602959], [36.639604, 25.826228], [36.249137, 26.570136], [35.640182, 27.37652], [35.130187, 28.063352], [34.632336, 28.058546], [34.787779, 28.607427], [34.83222, 28.957483], [34.956037, 29.356555]]], "type": "Feature", "properties": {"name": "", "bbox": [-180, -90, 180, -63.27066], "geometry": {"type": "MultiPolygon", "coordinates": [[[[-48.660616, -78.047019], [-48.151396, -78.04707], [-46.662857, -77.831476], [-45.154758, -78.04707], [-43.920828, -78.478103], [-43.48995, -79.08556], [-43.372438, -79.516645], [-43.333267, -80.026123], [-44.880537, -80.339644], [-46.506174, -80.594357], [-48.386421, -80.829485], [-50.482107, -81.025442], [-52.851988, -80.966685], [-54.164259, -80.633528], [-53.987991, -80.222028], [-51.853134, -79.94773], [-50.991326, -79.614623], [-50.364595, -79.183487], [-49.914131, -78.811209], [-49.306959, -78.458569], [-48.660616, -78.047018], [-48.660616, -78.047019]]], [[[-66.290031, -80.255773], [-64.037688, -80.294944], [-61.883246, -80.39287], [-61.138976, -79.981371], [-60.610119, -79.628679], [-59.572095, -80.040179], [-59.865849, -80.549657], [-60.159656, -81.000327], [-62.255393, -80.863178], [-64.488125, -80.921934], [-65.741666, -80.588827], [-65.741666, -80.549657], [-66.290031, -80.255773]]], [[[-73.915819, -71.269344], [-73.230331, -71.15178], [-72.074717, -71.190951], [-71.780962, -70.681473], [-71.72218, -70.309196], [-71.741791, -69.505782], [-71.173815, -69.035475], [-70.253252, -68.87874], [-69.724447, -69.251017], [-69.489422, -69.623346], [-69.058518, -70.074016], [-68.725541, -70.505153], [-68.451346, -70.955823], [-68.333834, -71.406493], [-68.510128, -71.798407], [-68.784297, -72.170736], [-69.959471, -72.307885], [-71.075889, -72.503842], [-72.388134, -72.484257], [-71.8985, -72.092343], [-73.073622, -72.229492], [-74.19004, -72.366693], [-74.953895, -72.072757], [-75.012625, -71.661258], [-73.915819, -71.269345], [-73.915819, -71.269344], [-72.330725, -71.894164], [-102.330725, -71.894164], [-101.703967, -71.717792], [-100.430919, -71.854993], [-98.98155, -71.93334], [-97.884743, -72.070535], [-96.787937, -71.952971], [-96.20035, -72.521205], [-96.983765, -72.442864], [-98.198083, -72.482035], [-99.432013, -72.442864], [-100.783455, -72.50162], [-101.801868, -72.305663], [-102.330725, -71.894164]]], [[[-122.621735, -73.657778], [-122.621735, -73.657777], [-122.406245, -73.324619], [-121.211511, -73.50099], [-119.918851, -73.657725], [-118.724143, -73.481353], [-119.292119, -73.834097], [-120.232217, -74.08881], [-121.62283, -74.010468], [-122.621735, -73.657778]]], [[[-127.28313, -73.461769], [-127.28313, -73.461768], [-126.558472, -73.246226], [-125.559566, -73.481353], [-124.031882, -73.873268], [-124.619469, -73.834097], [-125.912181, -73.736118], [-127.28313, -73.461769]]], [[[-163.712896, -78.595667], [-163.105801, -78.223338], [-161.245113, -78.380176], [-160.246208, -78.693645], [-159.482405, -79.046338], [-159.208184, -79.497059], [-161.127601, -79.634209], [-162.439847, -79.281465], [-163.027408, -78.928774], [-163.066604, -78.869966], [-163.712896, -78.595667]]], [[[180, -90], [180, -90], [-180, -90], [-180, -84.71338], [-179.942499, -84.721443], [-179.058677, -84.139412], [-177.256772, -84.452933], [-177.140807, -84.417941], [-176.084673, -84.099259], [-175.947235, -84.110449], [-175.829882, -84.117914], [-174.382503, -84.534323], [-173.116559, -84.117914], [-172.889106, -84.061019], [-169.951223, -83.884647], [-168.999899, -84.117914], [-168.530199, -84.23739], [-167.022099, -84.570497], [-164.182144, -84.82521], [-161.929775, -85.138731], [-158.07138, -85.37391], [-155.192253, -85.09956], [-150.942099, -85.295517], [-148.533073, -85.609038], [-145.888918, -85.315102], [-143.107718, -85.040752], [-142.892279, -84.570497], [-146.829068, -84.531274], [-150.060732, -84.296146], [-150.902928, -83.904232], [-153.586201, -83.88869], [-153.409907, -83.23802], [-153.037759, -82.82652], [-152.665637, -82.454192], [-152.861517, -82.042692], [-154.526299, -81.768394], [-155.29018, -81.41565], [-156.83745, -81.102129], [-154.408787, -81.160937], [-152.097662, -81.004151], [-150.648293, -81.337309], [-148.865998, -81.043373], [-147.22075, -80.671045], [-146.417749, -80.337938], [-146.770286, -79.926439], [-148.062947, -79.652089], [-149.531901, -79.358205], [-151.558416, -79.299397], [-153.390322, -79.162248], [-155.32393, -79.064269], [-155.975668, -78.69194], [-157.2568302, -78.378419], [-158.051768, -78.025676], [-158.365134, -76.889207], [-157.875474, -76.987238], [-156.974573, -77.300759], [-155.329376, -77.202728], [-153.742832, -77.065579], [-152.920247, -77.496664], [-151.33378, -77.398737], [-150.00195, -77.183143], [-148.748486, -76.908845], [-147.612483, -76.575738], [-146.104409, -76.47776], [-146.143528, -76.105431], [-146.496091, -75.733154], [-146.20231, -75.380411], [-144.909624, -75.204039], [-144.322037, -75.537197], [-142.794353, -75.34124], [-141.638764, -75.086475], [-140.209007, -75.06689], [-138.85759, -74.968911], [-137.5062, -74.733783], [-136.428901, -74.518241], [-135.214583, -74.302699], [-134.431194, -74.361455], [-133.745654, -74.439848], [-132.257168, -74.302699], [-130.925311, -74.479019], [-129.554284, -74.459433], [-128.242038, -74.322284], [-126.890622, -74.420263], [-125.402082, -74.518241], [-124.011496, -74.479019], [-122.562152, -74.498604], [-121.073613, -74.518241], [-119.70256, -74.479019], [-118.684145, -74.185083], [-117.469801, -74.028348], [-116.216312, -74.243891], [-115.021552, -74.067519], [-113.944331, -73.714828], [-113.297988, -74.028348], [-112.945452, -74.38104], [-112.299083, -74.714198], [-111.261059, -74.420263], [-110.066325, -74.79254], [-108.714909, -74.910103], [-107.559346, -75.184454], [-106.149148, -75.125698], [-104.876074, -74.949326], [-103.367949, -74.988497], [-102.016507, -75.125698], [-100.645531, -75.302018], [-100.1167, -74.870933], [-100.763043, -74.537826], [-101.252703, -74.185083], [-102.545337, -74.106742], [-103.113313, -73.734413], [-103.328752, -73.362084], [-103.681289, -72.61753], [-102.917485, -72.754679], [-101.60524, -72.813436], [-100.312528, -72.754679], [-99.13738, -72.911414], [-98.118889, -73.20535], [-97.688037, -73.558041], [-96.336595, -73.616849], [-95.043961, -73.4797], [-93.672907, -73.283743], [-92.439003, -73.166179], [-91.420564, -73.401307], [-90.088733, -73.322914], [-89.226951, -72.558722], [-88.423951, -73.009393], [-87.268337, -73.185764], [-86.014822, -73.087786], [-85.192236, -73.74797], [-83.879991, -73.518871], [-82.665646, -73.636434], [-81.470913, -73.851977], [-80.687447, -73.4797], [-80.295791, -73.126956], [-79.296886, -73.518871], [-77.925858, -73.420892], [-76.907367, -73.636434], [-76.221879, -73.969541], [-74.890049, -73.871614], [-73.852024, -73.65602], [-72.833533, -73.401307], [-71.619215, -73.264157], [-70.209042, -73.146542], [-68.935916, -73.009393], [-67.956622, -72.79385], [-67.369061, -72.480329], [-67.134036, -72.049244], [-67.251548, -71.637745], [-67.56494, -71.245831], [-67.917477, -70.853917], [-68.230843, -70.462055], [-68.485452, -70.109311], [-68.544209, -69.717397], [-68.446282, -69.325535], [-67.976233, -68.953206], [-67.5845, -68.541707], [-67.427843, -68.149844], [-67.62367, -67.718759], [-67.741183, -67.326845], [-67.251548, -66.876175], [-66.703184, -66.58224], [-66.056815, -66.209963], [-65.371327, -65.89639], [-64.568276, -65.602506], [-64.176542, -65.171423], [-63.628152, -64.897073], [-63.001394, -64.642308], [-62.041686, -64.583552], [-61.414928, -64.270031], [-60.709855, -64.074074], [-59.887269, -63.95651], [-59.162585, -63.701745], [-58.594557, -63.388224], [-57.811143, -63.27066], [-57.223582, -63.525425], [-57.59573, -63.858532], [-58.614143, -64.152467], [-59.045073, -64.36801], [-59.789342, -64.211223], [-60.611928, -64.309202], [-61.297416, -64.54433], [-62.0221, -64.799094], [-62.51176, -65.09303], [-62.648858, -65.484942], [-62.590128, -65.857219], [-62.120079, -66.190326], [-62.805567, -66.425505], [-63.74569, -66.503847], [-64.294106, -66.837004], [-64.881693, -67.150474], [-65.508425, -67.58161], [-65.665082, -67.953887], [-65.312545, -68.365335], [-64.783715, -68.678908], [-63.961103, -68.913984], [-63.1973, -69.227556], [-62.785955, -69.619414], [-62.570516, -69.991747], [-62.276736, -70.383661], [-61.806661, -70.716768], [-61.512906, -71.089045], [-61.375809, -72.010074], [-61.081977, -72.382351], [-61.003661, -72.774265], [-60.690269, -73.166179], [-60.827367, -73.695242], [-61.375809, -74.106742], [-61.96337, -74.439848], [-63.295201, -74.576997], [-63.74569, -74.92974], [-64.352836, -75.262847], [-65.860987, -75.635124], [-67.192818, -75.79191], [-68.446282, -76.007452], [-69.797724, -76.222995], [-70.600724, -76.634494], [-72.206776, -76.736655], [-73.969536, -76.634494], [-75.555977, -76.712887], [-77.24037, -76.712887], [-76.926979, -77.104802], [-75.399294, -77.28107], [-74.282876, -77.55542], [-73.656119, -77.908112], [-74.772536, -78.221633], [-76.4961, -78.123654], [-77.925858, -78.378419], [-77.984666, -78.789918], [-78.023785, -79.181833], [-76.848637, -79.514939], [-76.633224, -79.887216], [-75.360097, -80.259545], [-73.244852, -80.416331], [-71.442946, -80.69063],

[-70.013163, -81.004151], [-68.191646, -81.317672], [-65.704279, -81.474458], [-63.25603, -81.748757], [-61.552026, -82.042692], [-59.691416, -82.37585], [-58.712121, -82.846106], [-58.222487, -83.218434], [-57.008117, -82.865691], [-55.362894, -82.571755], [-53.619771, -82.258235], [-51.543644, -82.003521], [-49.76135, -81.729171], [-47.273931, -81.709586], [-44.825708, -81.846735], [-42.808363, -82.081915], [-42.16202, -81.65083], [-40.771433, -81.356894], [-38.244818, -81.337309], [-36.26667, -81.121715], [-34.386397, -80.906172], [-32.310296, -80.769023], [-30.097098, -80.592651], [-28.549802, -80.337938], [-29.254901, -79.985195], [-29.685805, -79.632503], [-29.685805, -79.260226], [-31.624808, -79.299397], [-33.681324, -79.456132], [-35.639912, -79.456132], [-35.914107, -79.083855], [-35.77701, -78.339248], [-35.326546, -78.123654], [-33.896763, -77.888526], [-32.212369, -77.65345], [-30.998051, -77.359515], [-29.783732, -77.065579], [-28.882779, -76.673665], [-27.511752, -76.497345], [-26.160336, -76.360144], [-25.474822, -76.281803], [-23.927552, -76.24258], [-22.458598, -76.105431], [-21.224694, -75.909474], [-20.010375, -75.674346], [-18.913543, -75.439218], [-17.522982, -75.125698], [-16.641589, -74.79254], [-15.701491, -74.498604], [-15.40771, -74.106742], [-16.46532, -73.871614], [-16.112784, -73.460114], [-15.446855, -73.146542], [-14.408805, -72.950585], [-13.311973, -72.715457], [-12.293508, -72.401936], [-11.510067, -72.010074], [-11.020433, -71.539767], [-10.295774, -71.265416], [-9.101015, -71.324224], [-8.611381, -71.65733], [-7.416622, -71.696501], [-7.377451, -71.324224], [-6.868232, -70.93231], [-5.790985, -71.030289], [-5.536375, -71.402617], [-4.341667, -71.461373], [-3.048981, -71.285053], [-1.795492, -71.167438], [-0.659489, -71.226246], [-0.228637, -71.637745], [0.868195, -71.304639], [1.886686, -71.128267], [3.022638, -70.991118], [4.139055, -70.853917], [5.157546, -70.618789], [6.273912, -70.62055], [7.13572, -70.246512], [7.742866, -69.893769], [8.48711, -70.148534], [9.52135, -70.011333], [10.249845, -70.48164], [11.0817821, -70.834332], [11.953824, -70.638375], [12.404287, -70.246512], [13.422778, -69.972162], [14.734998, -70.030918], [15.126757, -70.403247], [15.949342, -70.030918], [17.026589, -69.913354], [18.201711, -69.874183], [19.259373, -69.893769], [20.375739, -70.011333], [21.452985, -70.07014], [21.923034, -70.403247], [22.569403, -70.697182], [23.666184, -70.520811], [24.841357, -70.48164], [25.977309, -71.48164], [27.093726, -70.462055], [28.09258, -70.324854], [29.150242, -70.20729], [30.031583, -69.93294], [30.971733, -69.75662], [31.990172, -69.658641], [32.754053, -69.384291], [33.302443, -68.835642], [33.870419, -68.502588], [34.908495, -68.659271], [35.300202, -69.012014], [36.16201, -69.247142], [37.200035, -69.168748], [37.905108, -69.52144], [38.649404, -69.776205], [39.667894, -69.541077], [40.020431, -69.109941], [40.921358, -68.933621], [41.959434, -68.600514], [42.938702, -68.463313], [44.113876, -68.267408], [44.897291, -68.051866], [45.719928, -67.816738], [46.503343, -67.601196], [47.44344, -67.718759], [48.344419, -67.366068], [48.990736, -67.091718], [49.930885, -67.111303], [50.753471, -66.876175], [50.949325, -66.523484], [51.791547, -66.249133], [52.614133, -66.053176], [53.613038, -65.89639], [54.53555, -65.818049], [55.414943, -65.876805], [56.355041, -65.974783], [57.158093, -66.249133], [57.255968, -66.680218], [58.137361, -67.013324], [58.744508, -67.287675], [59.939318, -67.405239], [60.605221, -67.679589], [61.427806, -67.953887], [62.387489, -68.012695], [63.19049, -67.816738], [64.052349, -67.405239], [64.992447, -67.620729], [65.971715, -67.738345], [66.911864, -67.855909], [67.891133, -67.934302], [68.890038, -67.934302], [69.712624, -68.972791], [69.673453, -69.227556], [69.555941, -69.678226], [68.596258, -69.93294], [67.81274, -70.305268], [67.949889, -70.697182], [69.066307, -70.677545], [68.929157, -71.069459], [68.419989, -71.441788], [67.949889, -71.853287], [68.71377, -72.166808], [69.869307, -72.264787], [71.024895, -72.088415], [71.573285, -71.696501], [71.906288, -71.324224], [72.454627, -71.010703], [73.08141, -70.716768], [73.33602, -70.364024], [73.864877, -69.874183], [74.491557, -69.776205], [75.62756, -69.737034], [76.626465, -69.619419], [77.644904, -69.462684], [78.134539, -69.07077], [78.428371, -68.698441], [79.113859, -68.326216], [80.093127, -68.071503], [80.93535, -67.875546], [81.483792, -67.542388], [82.051767, -67.366068], [82.776426, -67.209282], [83.775331, -67.30726], [84.676206, -67.209282], [85.655527, -67.091718], [86.752359, -67.150474], [87.477017, -66.876175], [87.986289, -66.209911], [88.358411, -66.484261], [88.828408, -66.954568], [89.67063, -67.150474], [90.630365, -67.228867], [91.5901, -67.111303], [92.608539, -67.189696], [93.548637, -67.209282], [94.17542, -67.111303], [95.017591, -67.170111], [95.781472, -67.385653], [96.682399, -67.248504], [97.759646, -67.248504], [98.68021, -67.111303], [99.718182, -67.248504], [100.384188, -66.915346], [100.893356, -66.58224], [101.578896, -66.30789], [102.832411, -65.563284], [103.478676, -65.700485], [104.242557, -65.974783], [104.90846, -66.327527], [106.181561, -66.934931], [107.160881, -66.954568], [108.081393, -66.954568], [109.15864, -66.837004], [110.235835, -66.699804], [111.058472, -66.425505], [111.74396, -66.13157], [112.860378, -66.092347], [113.604673, -65.876805], [114.388088, -66.072762], [114.897308, -66.386283], [115.602381, -66.699804], [116.699161, -66.660633], [117.384701, -66.915346], [118.57946, -67.170111], [119.832924, -67.268089], [120.871, -67.189696], [121.654415, -66.876175], [122.320369, -66.562654], [123.221296, -66.484261], [124.122274, -66.621462], [125.100247, -66.719389], [126.100396, -66.562654], [127.001427, -66.562654], [127.882768, -66.660633], [128.80328, -66.758611], [129.704259, -66.58224], [130.781454, -66.425505], [131.799945, -66.386283], [132.935896, -66.386283], [133.85646, -66.289304], [134.757387, -66.209963], [135.031582, -65.72007], [135.070753, -65.308571], [135.697485, -65.58269], [135.873805, -66.038391], [136.206705, -66.44509], [136.618049, -66.778197], [137.460271, -66.954568], [138.596223, -66.895761], [139.908442, -66.876175], [140.809421, -66.817367], [142.121692, -66.817367], [143.061842, -66.977782], [144.374061, -66.837004], [145.490427, -66.915346], [146.195552, -67.228867], [145.999699, -67.601196], [146.646067, -67.895131], [147.723263, -68.130259], [148.839629, -68.385024], [150.132314, -68.561292], [151.483705, -68.71813], [152.502247, -68.874813], [153.638199, -68.894502], [154.284567, -68.561292], [155.165857, -68.835642], [155.92979, -69.149215], [156.811132, -69.384291], [158.025528, -69.482269], [159.181013, -69.599833], [159.670699, -69.991747], [160.80665, -70.226875], [161.570479, -70.579618], [162.686897, -70.736353], [163.842434, -70.716768], [164.919681, -70.775524], [166.11444, -70.755938], [167.309095, -70.834332], [168.425616, -70.971481], [169.463589, -71.20666], [170.501665, -71.402617], [171.20679, -71.696501], [171.089227, -72.088415], [170.560422, -72.441159], [170.109958, -72.891829], [169.75737, -73.24452], [169.287321, -73.65602], [167.975101, -73.812806], [167.387489, -74.165498], [166.094803, -74.38104], [165.644391, -74.772954], [164.958851, -75.145283], [164.234193, -75.458804], [163.82197, -75.870303], [163.568239, -76.24258], [163.47026, -76.693302], [163.489897, -77.065579], [164.057873, -77.457442], [164.273363, -77.82977], [164.743464, -78.182514], [166.604126, -78.319611], [166.995781, -78.750748], [165.193876, -78.907483], [163.666217, -79.123025], [161.766385, -79.162248], [160.924162, -79.730482], [160.747894, -80.200737], [160.316964, -80.573066], [159.788211, -80.945395], [161.120016, -81.278501], [161.629287, -81.690001], [162.490992, -82.062278], [163.705336, -82.395435], [165.095949, -82.708956], [166.604126, -83.022477], [168.895665, -83.335998], [169.404782, -83.825891], [172.283934, -84.041433], [172.477049, -84.117914], [173.224083, -84.41371], [175.985672, -84.158997], [178.277212, -84.472518], [180, -84.71338]]], [{"type": "Feature", "properties": {"name": ""}, "bbox": [32.73178, 35.000345, 34.576474, 35.671596], "geometry": {"type": "Polygon", "coordinates": [[[32.73178, 35.140026], [32.802474, 35.145504], [32.946961, 35.386703], [33.667227, 35.373216], [34.576474, 35.671596], [33.900804, 35.245756], [33.973617, 35.058506], [33.86644, 35.093595], [33.675392, 35.017863], [33.525685, 35.038688], [33.475817, 35.000345], [33.455922, 35.101424], [33.383833, 35.162712], [33.190977, 35.173125], [32.919572, 35.087833], [32.73178, 35.140026]]]]}, {"type": "Feature", "properties": {"name": ""}, "bbox": [32.256667, 34.571869, 34.004881, 35.173125], "geometry": {"type": "Polygon", "coordinates": [[[32.73178, 35.140026], [32.919572, 35.087833], [33.190977, 35.173125], [33.383833, 35.162712], [33.455922, 35.101424], [33.475817, 35.000345], [33.525685, 35.038688], [33.675392, 35.017863], [33.86644, 35.093595], [33.973617, 35.058506], [34.004881, 34.978098], [32.979827, 34.571869], [32.490296, 34.701655], [32.256667, 35.103232], [32.73178, 35.140026]]]]}, {"type": "Feature", "properties": {"name": ""}, "bbox": [17.020428, 21.420734, -1.124551, 35.759988], "geometry": {"type": "Polygon", "coordinates": [[[17.020428, 35.168396], [-1.792986, 34.527919], [-1.733455, 33.919713], [-1.388049, 32.864015], [-1.124551, 32.651522], [-1.307899, 32.262889], [-2.616605, 32.094346], [-3.06898, 31.724498], [-3.647498, 31.637294], [-3.690441, 30.896952], [-4.859646, 30.501188], [-5.242129, 30.000443], [-6.060632, 29.7317], [-7.059228, 29.579228], [-8.674116, 28.841289], [-8.66559, 27.656426], [-8.817828, 27.656426], [-8.794884, 27.120696], [-9.413037, 27.088476], [-9.735343, 26.860945], [-10.189424, 26.860945], [-10.551263, 26.990808], [-11.392555, 26.883424], [-11.71822, 26.104092], [-12.030759, 26.030866], [-12.500963, 24.770116], [-13.89111, 23.691009], [-14.221168, 22.310163], [-14.630833, 21.86094], [-14.750955, 21.5006], [-17.002962, 21.420734], [-17.020428, 21.42231], [-16.973248, 21.885745], [-16.589137, 22.158234], [-16.261922, 22.67934], [-16.326414, 23.017768], [-15.982611, 23.723358], [-15.426004, 24.359134], [-15.089332, 24.520261], [-14.824645, 25.103533], [-14.800926, 25.632625], [-14.43994, 26.254418], [-13.773805, 26.618892], [-13.139942, 27.640148], [-13.121613, 27.654148], [-12.618837, 28.038186], [-11.688919, 28.148644], [-10.900957, 28.832142], [-10.399592, 29.098586], [-9.564811, 29.933574], [-9.814718, 31.177736], [-9.434793, 32.038096], [-9.300693, 32.564679], [-8.657476, 33.240245], [-7.654178, 33.697065], [-6.912544, 34.110476], [-6.244342, 35.145865], [-5.929994, 35.759988], [-5.193863, 35.755182], [-4.591006, 35.330712], [-3.640057, 35.399855], [-2.604306, 35.179093], [-2.169914, 35.168396]]]]}, {"type": "Feature", "properties": {"name": ""}, "b


```
], [29.515953, 9.793074], [29.618957, 10.084919], [29.996639, 10.290927], [30.837841, 9.707237], [31.352862, 9.810241], [31.850716, 10.531271], [32.400072, 11.080626], [32.314235, 11.681484], [32.073892, 11.97333], [32.67475, 12.024832], [32.743419, 12.248008], [33.206938, 12.179338], [33.086766, 11.441141], [33.206938, 10.720112], [33.721959, 10.325262], [33.842131, 9.981915], [33.824963, 9.484061], [33.963393, 9.464285], [33.97498, 8.68456], [33.8255, 8.37916], [33.2948, 8.35458], [32.95418, 7.78497], [33.56829, 7.71334], [34.0751, 7.22595], [34.25032, 6.82607], [34.70702, 6.59422], [35.298007, 5.506], [34.620196, 4.847123], [34.005, 4.249885], [33.39, 3.79], [32.68642, 3.79232], [31.88145, 3.55827], [31.24556, 3.7819], [30.833852, 3.509172]]]]]]}
```

```
@@ src/app/globals.css css
@import "tailwindcss";
:root {
  color-scheme: light;
}
html,
body {
  max-width: 100vw;
  overflow-x: hidden;
}
body {
  font-family: Vazirmatn, "Segoe UI", Tahoma, system-ui, sans-serif;
  -webkit-tap-highlight-color: transparent;
  background:
    radial-gradient(circle at 100% 0%, rgba(20, 184, 166, 0.09), transparent 28rem),
    radial-gradient(circle at 0% 35%, rgba(14, 165, 233, 0.06), transparent 24rem),
    #f3f6f8;
}
@font-face {
  font-family: Vazirmatn;
  src: local("Vazirmatn"), local("Vazir"), local("IRANSans"), local("Tahoma");
  font-display: swap;
}

input,
select,
textarea,
button {
  font-family: inherit;
}
/* thin scrollbars for tables */
.scroll-x {
  overflow-x: auto;
  -webkit-overflow-scrolling: touch;
}
.scroll-x::-webkit-scrollbar {
  height: 6px;
}
.scroll-x::-webkit-scrollbar-thumb {
  background: #94a3b8;
  border-radius: 99px;
}
.leaflet-container {
  font-family: inherit;
  z-index: 0;
  background: #e8f0ed;
}
/*
.leaflet-tooltip.sek-province-label {
  border: 0;
  border-radius: 7px;
  background: rgba(255, 255, 255, 0.86);
  box-shadow: 0 1px 4px rgba(15, 23, 42, 0.18);
  color: #334155;
  direction: rtl;
  font-family: inherit;
  font-size: 9px;
  font-weight: 800;
  line-height: 1;
  padding: 4px 6px;
  pointer-events: none;
  white-space: nowrap;
}
.leaflet-tooltip.sek-province-label::before {
  display: none;
}
.leaflet-tooltip.sek-province-label.is-active {
  background: #0f766e;
  color: white;
  font-size: 10px;
}
.leaflet-tooltip.sek-point-tooltip {
  border: 0;
  border-radius: 9px;
  direction: rtl;
  font-family: inherit;
  font-size: 11px;
  font-weight: 800;
```

```
padding: 6px 9px;
}
.sek-map-marker {
  background: transparent !important;
  border: 0 !important;
}
@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(6px);
  }
  to {
    opacity: 1;
    transform: none;
  }
}
.fade-in {
  animation: fadeIn 0.18s ease-out;
}
```